Proceedings of the 25th Python in Science Conference

Proceedings of the 25th Python in Science Conference

Edited by Rowan Cockett, Sean Freeman, Christopher Holloman, Sanhita Joshi, Franklin Koch, Charles Lindsey, Hongsup Shin, Chong Shen Ng, and Jennifer Yoon

SciPy 2026
Minneapolis, Minnesota
July 13 - July 19, 2026

Copyright © 2026. The articles in the Proceedings of the Python in Science Conference are copyrighted and owned by their original authors.

This is an open-access publication and is distributed under the terms of the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

For more information, please see: https://creativecommons.org/licenses/by/4.0/

ISSN: 2575-9752
https://doi.org/10.25080/yvdw4955

Organization

Conference Co-Chairs

- Gil Forsyth
- Madicken Munk

Program

- Dan Allan
- Naty Clementi
- Matthew Feickert
- Ariana Mendible

Tutorial

- Paul Anzel
- Daniel Chen
- Matt Craig
- Allen Harvey

Outreach

- Juan Cabanela
- Cynthia Chen
- Juanita Gomez
- James Lamb

Proceedings

- Rowan Cockett
- Sean Freeman
- Christopher Holloman
- Sanhita Joshi
- Franklin Koch
- Charles Lindsey
- Chong Shen Ng
- Hongsup Shin
- Jennifer Yoon

Hybrid

- Puneet Kollipara
- David Nicholson
- Neelima Pulagam
- Shubham Sharma

Financial Aid

- Eric Ma
- Nadia Tahiri

Activities

- Ana Comesana
- Matt Davis
- Ed Rogers

- Caitlin Vanasse
- Olek Yardin

Birds-of-a-Feather
- Michael Akerman
- Mike Droettboom

Sprints
- Brigitta Sipőcz
- Nathan Goldbaum

Track Chairs: Spirit of SciPy
- Thomas Caswell
- Brigitta Sipőcz
- Stéfan van der Walt

Track Chairs: Data Driven Discovery with ML and AI
- Hugo Bowne-Anderson
- Julie Hollek
- Jim Pivarski
- Fatma Tarlaci

Track Chairs: Environmental, Earth, and Climate Sciences
- Maxwell Grover
- Paige Martin
- Jared Oyler
- Katie Wetstone

Track Chairs: Scientific Computing in Education
- Suh Young Haeyun Choi
- Ryan Cooper
- Tracy Teal

Track Chairs: Physics and Astronomy
- Matt Craig
- Guen Prawiroatmodjo

Track Chairs: Maintainers and Community
- Jim Crist-Harif
- Yuvi Panda
- Inessa Pawson
- Sanket Verma

Track Chairs: SciPy Tools
- Inessa Pawson
- Sanket Verma

Track Chairs: Biological and Medical Sciences
- Nezar Abdennur
- Aishwarya Chander

- Jackie Valeri

Track Chairs: General

- Dan Allan
- Naty Clementi
- Matthew Feickert
- Ariana Mendible

Program and Tutorial Reviewers

- Abel Abraham
- Abhinav Shankaranarayanan Venkataraman
- Abhiram Mullapudi
- Abhirami Pillai
- Adrien Badre
- Allen S. Harvey Jr.
- Amin Alhashim
- Andrea Tentner
- Andy Terrel
- Ariana Mendible
- Avik Basu
- Bhavuk Jain
- Bhupendra Raut
- Bilal Arif
- Cagri Temel
- Charles Turner
- Chidimma Maduka
- Chirag Shah
- Dan Allan
- Daniel Chen
- Debisree Ray
- Dhananjay Bhaskar
- Erik Whiting
- Gauri Sarode
- Guen Prawiroatmodjo
- Hannah Ferriby
- Hariharan Ragothaman
- Ian Hunt-Isaak
- Isabel Zimmerman
- Ivan Perez Avellaneda
- Jake Diamond-Reivich
- James A. Bednar
- James Lamb
- Jim Pivarski
- Katie Wetstone
- Kavya Tolety
- Kedar Dabhadkar
- Krista Pullen
- Kristijan Armeni

- Leo Fang
- Leonid Didukh
- Lundy Bernard
- Matt Craig
- Matt Davis
- Matthew Feickert
- Michael Greminger
- Naty Clementi
- Neesha S
- Nicolas R Posner
- Nikki Tebaldi
- Nilanjan Chatterjee
- Oladapo Kayode Abiodun
- Paige Martin
- Pankaj Arora
- Paul Anzel
- Paul Ivanov
- Pavan B Govindaraju
- Ramesh Oswal
- Saish Sali
- Sandeep Bharadwaj Mannapur
- Sandeep Chittimalli
- Sanjiban Sengupta
- Shaurya Agarwal
- Shubham Gupta
- Shubham Sharma
- Sivakumar Dhanasekar
- Smit Lunagariya
- Sohil Grandhi
- Sona Maniyan
- Stefanie Molin
- Supun Kamburugamuve
- Tom Caswell
- Vinay Vyas
- Yevgen Matviychuk

Proceedings Reviewers

- Aditi Juneja
- Amey Ambade
- Ammar Ekbote
- Amogh Param
- Anand Murthy
- Andrei Paleyes
- Anish Mahishi
- Ankita Bagaria
- Anuj Ashok Potdar
- Anumit Garg
- Aresema Workneh

- Arjun Bali
- Bhanu sekhar Guttikonda
- Bhuvnesh Kumar
- Biola Adeyemi
- Chandra Prakash
- Chirag Agrawal
- Chirag Shah
- Corentin Cadiou
- Dennis Bakhuis
- Dinesh Rathinasamy Thangavel
- Don Setiawan
- Erle Holgersen
- Hansila Sudasinghe
- Hastimal Jangid
- Horacio V. Guzman
- Ilerioluwakiiye Abolade
- J.R. Barhydt
- James Lamb
- Jasdeep Singh Bhalla
- Jeyadev Needhidevan
- Jianzhu Yao
- Jishan Ahmed
- Jofia Jose Prakash
- John Derek Morgan
- Jubinder Singh
- Jyoti Kunal Shah
- Kalyan Prasad
- Karthik Venkataramani
- Kayode Oladapo
- Kirtan Dave
- Kshitij Minhas
- Kumar Rishabh
- Kuntao Zhao
- Lucas Durand
- Manoj Krishna Mohan
- Matthew Feickert
- Matthew Lesko
- Min Khant Zaw
- Muhammad Rayyan Shakoori
- Nandan Sharma
- Nilanjan Chatterjee
- Nima Sarajpoor
- Odunola Odofin
- Peter Sun
- Piyush Jagadish Bag
- Pouya Afshin
- Praveen Gupta Sanka

- Prithvish Rakesh Doshi
- Pushkar Sathe
- Qi Zhao
- Rajani Kant
- Rajdeep Adak
- Ramachander Rao Thallada
- Ramesh Oswal
- Ruohan Zhang
- Sai Tarun Kanigant
- Samaresh Kumar Singh
- Samuel Williams
- Sanjana Kandi
- Sanjay Belaturu Krishnegowda
- Sarah Stapleton
- Serge Rey
- Shashank Kapoor
- Shenyang Liu
- Shubham Gupta
- Simmi Mourya
- Smit Lunagariya
- Sopuluchukwu Ani
- Sultan Orazbayev
- Talha
- Tejul Pandit
- Tolulade Ademisoye
- Uju
- Varun Shivashankar
- Veronicah Gathoni
- Vibhu Verma
- Yash Sadhwani
- Yash Sharma
- Yichen Wang
- Zhikai Wu
- Zhun Zhou

Posters and Slides

Keynote Presentations

Calm Technology and the History of AI

Amber Case's work explores the intersection of humans and technology, challenging us to design systems that inform rather than overburden. Case is redefining the relationship between humans and technology. As the founder of the Calm Tech Institute and a former fellow at MIT and Harvard, Case brings a profound perspective on how we can design complex systems to be calm: interfaces that work with peripheral attention and inform at different resolution levels. In her keynote, "Calm Technology and the History of AI," she will explore moving from "smart things" to "smarter people," how to design systems that inform us without overwhelming us, why the future of interface design might involve bringing back the button, and how to ensure modern systems are built in line with how the different parts of our brains interpret information.

Amber Case

Snakes in the Microwaves: How Python is Powering the Golden Age of SAR

Synthetic Aperture Radar (SAR) is transforming how we observe our planet. It sees through clouds, smoke, and darkness, measures millimeter-scale changes to Earth's surface from space, and is rapidly becoming a cornerstone of Earth observation. With a decade of Sentinel-1 observations, the launch of NISAR, and fleets of commercial satellites, we're entering the Golden Age of SAR. At the Alaska Satellite Facility, we steward more than 30 PB of freely-accessible SAR data for NASA Earthdata, with the archive expected to exceed 100 PB as calibrated NISAR data becomes available. But turning that flood of data into scientific insight requires far more than storage—it demands an ecosystem of software that enables scientists to discover, access, process, and analyze data at an unprecedented scale. Drawing on examples from the Alaska Satellite Facility and the broader NASA Earthdata ecosystem, I'll explore how the Scientific Python ecosystem, through tools like NumPy, Xarray, Zarr, Jupyter, and countless community-built libraries, has become the foundation powering everything from cloud-native data access and open-source scientific libraries to large-scale processing platforms and "near"-real-time Earth monitoring projects like ITS_LIVE. Along the way, we'll see how the Python community has helped transform SAR from a specialized research tool into a global scientific resource, moving beyond individual images toward continuous streams of Earth observations—and why the next decade of Earth observation will be defined as much by open-source software as by the satellites themselves.

Joseph H. Kennedy

Stories in Code

Stories are a core to the human experience and core to our understanding of complex technical systems. This talk will discuss the role that stories play in software in general and open source specifically. The stories we collectively write and share, in the form of code, are the concrete artifacts we create. How we go about organizing the development of these stories and the relationships between people are the spirit of SciPy.

Thomas Caswell

Accepted Talks

A Lean and Kind OME-Zarr Toolkit for Bioimaging

Bioimaging generates massive datasets in fragmented, proprietary formats that are difficult to share and align with FAIR principles. ngff-zarr is a lightweight, open-source Python toolkit implementing the OME-Zarr specification – the community-driven, cloud-native bioimaging standard. With minimal dependencies and a simple pipeline interface, ngff-zarr converts, validates, and generates multiscale representations of extremely large images out-of-core via Dask. Features include multiple downscaling methods, OME-Zarr Zip archives (.ozx), RFC-4 anatomical orientation, and High Content Screening support. This talk also covers ngff-zarr's Model Context Protocol (MCP) server, which enables AI agents to perform bioimaging tasks through natural language, and lessons learned from its deployment at EMBL.

Matt McCormick

Tying Up Loose Threads: Making your Project No-GIL Ready

If you messed around with Python's command line options or read the official documentation, you might wonder what the -Xgil option or the PYTHON_GIL environment variable did to your scripts, and whether setting either affects performance. The hubbub on popular wheels such as pyo3, python-zstandard, numpy, uv, cffi, and cython supporting the free-threaded interpreter is no passing fad either. For Pythonistas that don't read PEPs in their spare time or contribute to the cpython project itself, an adventure that delves into a less known, yet jaw-dropping aspect of Python awaits! Python's Global Interpreter Lock, which determines which single thread can execute native Python code and call C API functions, simplifies writing multithreaded code. However, sticking with this execution model leaves out extra performance afforded by modern multicore CPUs with hyperthreading, as automatic locking and unlocking of the GIL does not scale well with thread counts, especially in performance-sensitive workloads. The newfangled free-threaded interpreter promises salvation when running either pure Python code or with compiled extensions. General multithreading rules apply (prefer thread-local variables, using locks to prevent simultaneous access of shared data), but when dealing with projects containing compiled extensions that directly or indirectly interface with Python's C API, more porting rules also apply. Key porting tips, including projects using the Limited API, include: port native code away from C API functions that avoid borrowed references because they aren't thread-safe; modify unit tests to catch concurrency bugs arising from assuming the presence of the GIL; and extend CI coverage of Python interpreters both for testing and to build free-threaded compatible wheels.

Charlie Lin

Learning in the Open: Integrating Open Source Contributions into the Classroom

In Fall 2025, the UConn School of Mechanical, Aerospace, and Manufacturing Engineering launched Open Source Experiences, an elective course developed in partnership with six NumFOCUS-supported projects (napari, BiocPy, Blosc, MNE-Python, mlpack, JuliaHub). The course embedded students directly into active open source communities, where they contributed to the codebases, collaborated with project maintainers, and learned about community-driven open source software development. In this talk, we will share the lessons learned from piloting this collaboration model, and how these experiences benefit students, open source and open science communities, and educators alike. Attendees will take away actionable insights for integrating open source contributions into their own classrooms and programs.

Mohammad Mundiwala, Ryan C. Cooper, Ryan Curtin, Inessa Pawson

SciPy Tools Plenaries

NumPy annual update

Comprehensive overview of the latest release and major milestones for the NumPy library and its contributor community

Inessa Pawson

Table of Contents

1 *Opening the Black Box: Mechanistic Interpretability for AI Agent Tool Selection Using Sparse Autoencoders*

HANNES HAPKE, DAVID CARDOZO, CARLOS HUISA, TRIVENI GANDHI

19 *BCDNM-HPO: Balancing Coherence, Diversity, and Target Noise in GPU-Accelerated BERTopic*

ALLISON DING, MITESH PATEL, COREY NOLET

27 *Density Functions and Random Number Generators of Alpha-Stable Distributions*

WAEL TABBARA, SHARAFEDDINE SHARAFEDDINE, AHMAD EL-HAJJ, JIHAD FAHS, IBRAHIM ABOU-FAYCAL

48 *Detecting Anomalies in Scientific Data Using SciPy's Statistical and Signal Tools*

AAYUSH GAUBA

64 *Right Predictions, Wrong Reasons: Explanation Drift Monitoring in Production*

AVIK BASU

77 *Adapt v0.1: Prototyping a Real-Time, Reproducible Data Analysis Framework for Adaptive Radar Scanning*

BHUPENDRA A. RAUT, SIDDHANT GUPTA, ZACH SHERMAN, BOBBY JACKSON, JOE O'BRIEN, EDDIE SCHUMAN, PAWEL LECH, SHERMAN BEUS, SETH ABERNETHY, SCOTT COLLIS, NICKI HICKMON, ADAM THEISEN

89 *Voice Capture, Structured Extraction, and Multi-Source Routing: Building and Evaluating a Field-Inspection Assistant in Python*

CAROLYN OLSEN

100 *Vibes, meet rigor: Evaluating and improving AI performance on complex scientific code*

CLIFF KERR, KATHERINE ROSENFELD, JESSICA LUNDIN, ROBYN STUART, ROMESH ABEYSURIYA

111 *On-boarding and retaining maintainer talent for MNE-Python*

BRITTA U. WESTNER, ERIC LARSON, DANIEL MCCLOY

119 *Climate is not a straight line: Scalable Python-based GAMM Workflows for Wildlife Conservation*

DEVEN MAHESHWARI, JAKE WALL, CATHERINE VILLENEUVE

132 *Benchmarking Edge-Accelerated Genomics: A Pilot Study of Unified Memory Architectures in Deep-Sea Metagenomics*

VINAY VYAS

145 *Engineering Better Retrieval for RAG*

AARTI JHA, MAHIMA ARORA

156 *From Hello World to Hello LLM: A Python Developer's Survival Guide*

JASMINE OMEKE, AUDREY WEBB

167 *PanelBox: A Comprehensive Python Library for Panel Data Econometrics*

GUSTAVO COELHO HAASE, PAULO HENRIQUE DOURADO, OSVALDO CANDIDO

179 *Automated Data Enrichment for Police Accountability: Where Agentic Judgment Earns Its Place*

HONGSUP SHIN

199 *The Space to Build It: How Scientific Needs Shaped an Ecosystem*

K. JARROD MILLMAN

210 *From LiDAR to action: detecting upland gullies to combat erosion and forest fires*

JIM PIVARSKI

221 *Commit to Community: Open Source as Social Infrastructure in Volunteer Civic Tech*

KRISTIJAN ARMENI, HELEN Y. GLOVER, BEN SANDO

230 *Everything That Breaks When You Put an LLM Agent in Production*

SRUTHI PISIPATI, HARIS JAVED

251 *Bridging the Technical Gap: A Student-Led Data Extraction Pipeline for Community-Driven Document Analysis*

JESSE LOI, ARIANA MENDIBLE

265 *The Missing Lever in Machine Learning Deployment: Threshold Tuning using Regression Discontinuity*

LUCAS SQUARIZE CHAGAS, AVIK BASU

271 *Docling for Multimodal Retrieval*

MAHIMA ARORA, AARTI JHA

282 *Hash all the things: Caching for fast notebook restarts*

DYLAN MADISETTI, MYLES SCOLNICK, AKSHAY AGRAWAL

295 *ngff-zarr: A Lean and Kind OME-Zarr Toolkit for Bioimaging*

MATT MCCORMICK, JOHANNES SOLTWEDEL, WOUTER-MICHIEL VIERDAG, VALENTIN BOUSSOT, JOSH MOORE

303 *Derivations, Not Just Simulations*
MICHAEL ZARGHAM

323 *Reno: Simplifying Application of Bayesian Inference to System Dynamics*
NATHAN MARTINDALE, JORDAN STOMPS, BIRDY PHATHANAPIROM

339 *Numerical Stability Pitfalls in Scientific Optimization Pipelines*
AAYUSH GAUBA

355 *AutoFOAM: The Self-Refining Autonomous OpenFOAM Agent*
ARUN GOVIND NEELAN, A SESHADITYA

366 *OXA: An Open Exchange Architecture for Modular and Composable Scientific Content*
ROWAN COCKETT, NOKOME BENTLEY, MONICA GRANADOS, FRANKLIN KOCH, MICHAEL MARKIE, STEVE PURVES, CARLOS SCHEIDEGGER, PAUL SHANNON, GABRIEL STEIN, TRACY TEAL

380 *NODEFit: Fit time-series data with a Neural Differential Equation*
PAVAN B. GOVINDARAJU

398 *Optimising HCP Sample Allocation in Pharma: Combining Nonlinear Ensemble Learning, Spatial Lags, and Integer Programming*
JOHANNES PLAMBECK, SANTOSH AJABRAO BOTHE, DORIAN PULERI, VICTOR GONZALES

424 *Trimming the Hairball*
RACHAEL SEXTON

438 *A Reproducible Data Lakehouse for High-Resolution Gastric Cancer Epidemiology Study in Chile*
RENE LAGOS, CRISTÓBAL CUADRADO, FRANCISCA JOFRÉ, ARNOLDO RIQUELME

451 *25 Years of Interactive Scientific Computing: From IPython and Jupyter to Open Source IDE-Native Notebooks*
RODRIGO SILVA FERREIRA

472 *alphagenome-pt: Flexible Training of AlphaGenome Models in PyTorch*
RYLIE WEAVER, GOMATHI LAKSHMANAN, JOHN LAGERGREN

489 *Solving the AI Eval Gap: Domain-Aware Evaluation for Production AI Agents*
SHIVIKA BISEN

500 *Avoiding Zero-Trade Policies in RL with a Decoupled MLOps Architecture*
SHO TANAKA

509 *ArcadeDB in Python: An In-Process Multi-Model Database*
TAEWOON KIM

521 *Feel the model: Sensory Transduction of Neural Activations as a Human-in-the-Loop Safety*
VIRAJ SHARMA

528 *Toward Reliable Localization of Free and Open Source Software: LLM-assisted Translation Workflows for QGIS*
YU-LIN CHEN, TYNG-RUEY CHUANG, CHENG-JEN LEE

Open Access | https://doi.org/10.25080/gcwu7766

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Opening the Black Box: Mechanistic Interpretability for AI Agent Tool Selection Using Sparse Autoencoders

Hannes Hapke[1], **David Cardozo**[2], **Carlos Huisa**[3], and **Triveni Gandhi**[4]

[1]575 Lab, Dataiku Inc., [2]575 Lab, Dataiku Inc., [3]Dataiku Inc., [4]Dataiku Inc.

Abstract

AI agents that autonomously select and run tools are increasingly deployed in business contexts, yet their decision-making processes remain opaque. When an agent chooses to query a database rather than search the web, what internal representations drive that choice? Current approaches rely on prompt engineering, behavioral testing, or post-hoc explanations, none of which reveal the actual computational mechanisms within the model. We present a complete mechanistic interpretability pipeline for understanding agent tool-selection decisions. Our approach extracts hidden-state activations from the subject model at the precise moment of tool commitment (the *decision token*), trains a JumpReLU Sparse Autoencoder (SAE) to decompose these activations into monosemantic features, and applies contrastive analysis to identify which features are associated with specific decision patterns. Crucially, the SAE learns the model's natural feature vocabulary without contrastive supervision; contrastive pairs serve only as post-hoc statistical probes. We validate our feature explanations using token-level fuzzing evaluation adapted from EleutherAI's autointerp methodology, which tests whether labels identify the correct tokens that activate each feature, not just the correct texts. Our pipeline processes diverse agent scenarios across five domains and produces human-readable decision reports explaining the internal factors driving tool selection. Our implementation is open source at https://github.com/dataiku/kiji-inspector.

Keywords mechanistic interpretability, sparse autoencoders, AI agents, tool selection, feature interpretation

Published Jul 13, 2026

Correspondence to
Hannes Hapke
hannes.hapke@dataiku.com

Open Access

Copyright © 2026 Hapke *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

The deployment of AI agents capable of autonomous tool selection represents a significant advance in artificial intelligence capabilities. Modern agents can access databases, execute code, search the web, and delegate to sub-agents based on natural language requests. However, this capability comes with a fundamental transparency problem: we cannot inspect the internal decision process that leads an agent to choose one tool over another.

Consider an agent that receives the request "Find information about our company's API rate limits." The agent might choose between searching internal documentation or the public web. The agent's decision to use internal search is the correct one, but what if it instead chose the wrong tool or took an unauthorized action? Without the ability to trace and understand the model's interpretation of both the request and the environment in which it is working, users can not be confident that the agent is taking the right course of action and in turn can not trust it to operate as required. Understanding *why* the model interprets

"our company's" as requiring internal search, rather than treating it as a generic reference, requires inspecting the model's internal representations, not just its outputs.

Current approaches to understanding agent behavior fall into three categories, none of which provide mechanistic insight:

1. **Prompt engineering**: Modifying system prompts and observing behavioral changes reveals correlations but not causal mechanisms.
2. **Behavioral testing**: Probing with test inputs characterizes input-output relationships without explaining intermediate computations.
3. **Chain-of-thought inspection**: Examining generated reasoning text or prompting the model for self-explanations provides a narrative rather than the underlying computation. Prior work [1] demonstrates that such generated rationales can be unfaithful, producing plausible-sounding justifications (confabulations) that do not reflect the internal representations actually driving decisions.

Mechanistic interpretability offers an alternative: directly examining the model's internal representations to identify the computational features that contribute to decisions. Recent work on Sparse Autoencoders (SAEs) has demonstrated that neural network activations can be decomposed into interpretable, monosemantic features that correspond to human-understandable concepts [2], [3]. However, this work has focused primarily on next-token prediction in language models, not on the structured decision-making that characterizes AI agents.

This paper presents a complete pipeline for mechanistic interpretability of agent tool-selection decisions. Our contributions are:

1. **Decision token extraction**: We identify the precise position in the prompt where the model commits to a tool choice and extract activations at this critical moment.
2. **Contrastive pairs as post-hoc probes**: We generate synthetic pairs of semantically similar requests requiring different tools, but use these pairs only for statistical analysis *after* SAE training. The SAE learns the model's natural feature vocabulary unsupervised. Contrastive pairs are used to identify which of these pre-existing features are associated with specific decision differences.
3. **JumpReLU SAE with tanh sparsity**: We employ a JumpReLU activation function that produces exact zeros with learnable per-feature thresholds, trained using a smooth tanh-based sparsity penalty.
4. **Token-level fuzzing evaluation**: We adapt EleutherAI's autointerp methodology [4] to validate whether feature labels correctly identify *which tokens* activate each feature, catching explanations that are "right for the wrong reasons."

The remainder of this paper is organized as follows. Responsible AI Motivations provides the ethical motivation for this work. Related Work reviews related work. Methodology details our methodology. Evaluation: Token-Level Fuzzing describes our evaluation approach. Experimental Setup presents our experimental setup, and Results reports results. Discussion discusses findings and limitations, and Conclusion concludes.

2. Responsible AI Motivations

Transparency of agent decisions is increasingly important as AI regulations are implemented across many geographies and industries. Traditional approaches to AI Governance require the ability to interpret and explain model behavior and agentic systems are subject to the same scrutiny, even if their mechanics are different. Transparency requirements are grounded in the need to verify AI systems are operating as intended, leading to stronger trust in and adoption of the technology. Engendering trust in AI systems requires a frame-

work for responsible AI development; we choose to follow the transparency principle outlined in the RAFT framework [5] as the underpinning and motivation for understanding agent behavior.

The advent of chat-based language models and rapid development in agentic capabilities have highlighted the ongoing need for robust and practical frameworks for AI Governance. Numerous political bodies and standards organizations have developed frameworks to manage the risks associated with AI - though these largely provide high-level ethical principles. By contrast, the RAFT framework is a value-criteria-indicator approach to Responsible AI that covers risks from traditional and generative AI. It is an intentionally lightweight methodology that focuses on what are considered "baseline" requirements for good AI Governance. The RAFT framework is comprised of 4 values - Reliable, Accountable, Fair, and Transparent.

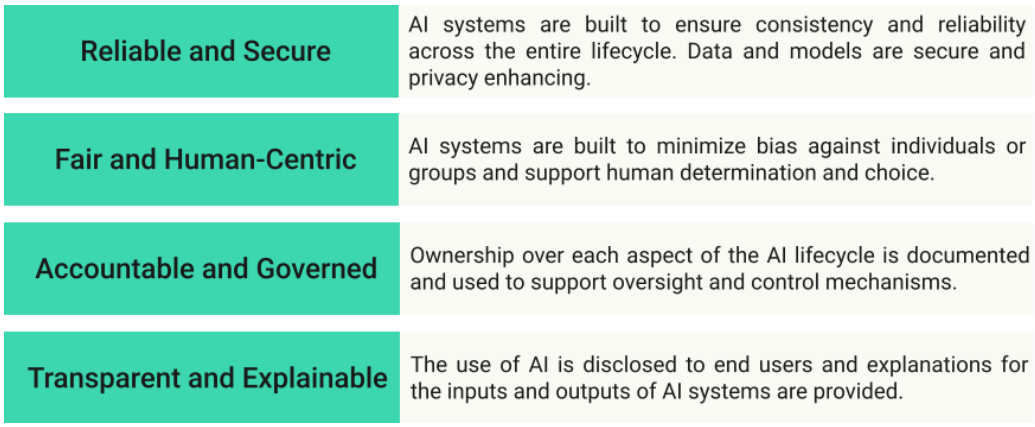


Figure 1. *Core principles of the RAFT framework.*

From these principles organizations form specific criteria and indicators to assess whether their systems align to these values. For example, in the case of Transparency an organization might assign "explainable outputs" as a criterion towards alignment with this principle. Within that criterion will be multiple indicators that can be used as observable measures of progress - such as SHAP values (for predictive modeling) or chain-of-thought reasoning (in the case of generative AI systems). The mechanistic interpretability approach offered in this paper can serve as another indicator towards explainability and overall transparency of agentic systems. Our hope is that by grounding the motivation for this work in a larger principles-based approach we can support the holistic development of responsible AI tooling.

3. Related Work

3.1. Sparse Autoencoders for Interpretability

Sparse autoencoders have emerged as a powerful tool for decomposing neural network activations into interpretable features. T. Bricken *et al.* [2] demonstrated that SAEs trained on language model activations can recover monosemantic features, identifying individual neurons in the SAE that correspond to single, interpretable concepts. This addresses the *superposition hypothesis*, which posits that neural networks represent more features than they have dimensions by encoding multiple concepts in overlapping directions [6].

H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey [3] extended this work with improvements to SAE training, including better initialization strategies and sparsity

penalties. The JumpReLU activation function [7] provides exact sparsity, which improves interpretability by ensuring features are either clearly active or completely inactive.

3.2. Mechanistic Interpretability

Mechanistic interpretability aims to reverse-engineer the algorithms implemented by neural networks [8]. Early work focused on identifying circuits, subgraphs of neurons that implement specific functions [9]. More recent approaches use dictionary learning to find interpretable directions in activation space [10].

3.3. AI Agents and Tool Use

Tool-augmented language models have become a dominant paradigm for AI agents. Toolformer [11] demonstrated that language models can learn to use tools through self-supervised learning. ReAct [12] introduced a framework combining reasoning and acting in language models. Despite rapid progress in agent capabilities, interpretability of agent decisions has received limited attention.

3.4. Automatic Interpretability

Labeling features automatically addresses the scalability challenge of mechanistic interpretability. S. Bills *et al.* [13] used language models to generate explanations for individual neurons. Our token-level fuzzing adapts this approach to validate whether feature labels identify the correct *tokens* that activate each feature.

3.5. Contrastive Methods

Contrastive methods have proven effective for identifying meaningful directions in representation space. Contrastive activation addition [14] demonstrated that differences between activations for contrasting inputs can be used to steer model behavior. Representation engineering [15] extended these ideas to control model outputs. Our approach uses contrastive pairs for post-hoc statistical analysis rather than for training or steering.

4. Methodology

4.1. Pipeline Overview

Our pipeline consists of six sequential steps (see Figure 2):

1. **Contrastive pair generation**: Generate synthetic pairs of user requests sharing the same intent but requiring different tools.
2. **Activation extraction**: Extract hidden-state activations from the subject model at the decision token position.
3. **SAE training**: Train a JumpReLU sparse autoencoder on the raw activations.
4. **Contrastive activation analysis**: Use contrastive pairs as post-hoc probes to identify decision-relevant features.
5. **Feature interpretation**: Label features using an LLM and generate decision reports.
6. **Fuzzing evaluation**: Validate feature labels through token-level A/B testing.

The pipeline uses two large language models in separate memory contexts: a generation model (Qwen3-VL-235B-A22B-Instruct-FP8) for pair generation, feature labeling, and fuzzing judgment; and a subject model (NVIDIA Nemotron-3-Nano-30B-A3B-BF16) for activation extraction. The produced SAE model is built to work in conjunction with the subject model. Crucially, mechanistic interpretability requires full access to model weights and internal hidden states; thus, the subject model must be an open-weights model. Closed-

source API models (which hide internal activations) cannot serve as subject models, though they can optionally replace the auxiliary generation model for offline pair generation and feature labeling.

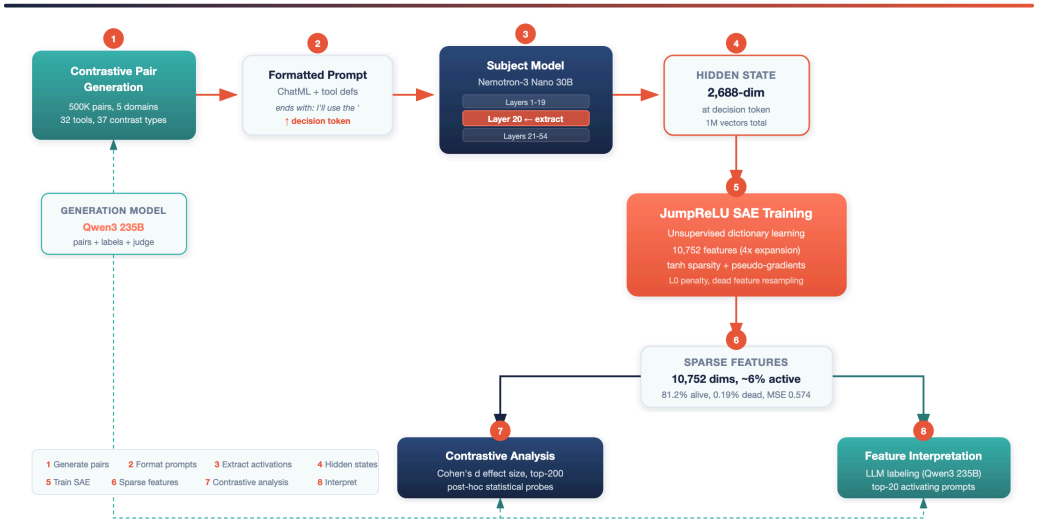


Figure 2. *Overview of the training pipeline. Contrastive pairs are generated and encoded by the subject model. The SAE is trained unsupervised on the extracted activations; contrastive pairs serve only as post-hoc statistical probes for feature analysis and interpretation.*

The study is reproducible from the command line: pair generation is a one-off step, after which a single command runs the remaining stages (Program 1). Flags select individual steps (`--step`), additional layers (`--layers`), or a different subject model (`--subject-model`).

```
python -m kiji_inspector.generate_pairs 1300
python -m kiji_inspector.pipeline
```

Program 1. *Reproducing the study from the command line.*

4.2. Contrastive Pair Generation

Each contrastive pair captures two semantically similar requests that require different tools. The pair structure includes: anchor prompt, anchor tool, contrast prompt, contrast tool, shared intent, and distinguishing signal.

We define scenarios as JSON configurations specifying a domain (e.g., tool selection, investment analysis), available tools, and contrast types. Table 1 summarizes the five scenarios in our system.

Table 1. *Scenario configurations for contrastive pair generation.*

Scenario	Tools	Contrast Types	Example Contrast
tool_selection	8	13	read_vs_write
investment	6	6	risk_vs_return
manufacturing	6	6	quality_vs_speed
supply_chain	6	6	cost_vs_reliability
customer_support	6	6	escalate_vs_resolve

Pairs are generated using an LLM (Qwen3-VL-235B) with a structured prompt template. The generator includes robust JSON parsing with multiple recovery layers: markdown fence stripping, bracket extraction, trailing comma removal, and truncation recovery. Fuzzy key matching handles LLM variations in JSON field names. Program 2 shows three example pairs from different scenarios.

```
Example Pair 1: customer_support (self_service_vs_agent_assist)
Shared Intent: resolve password reset issue
ANCHOR [knowledge_base]: How do I reset my password if I forgot it?
CONTRAST [ticket_lookup]: I tried resetting my password 3 times
but the email never arrives.

Example Pair 2: investment (growth_vs_value)
Shared Intent: Evaluate energy sector stocks for potential investment
ANCHOR [financial_analysis]: Which energy companies are investing
heavily in renewable expansion?
CONTRAST [market_data_lookup]: Which energy stocks are currently
trading below their book value?

Example Pair 3: tool_selection (read_vs_write)
Shared Intent: Check latest product version
ANCHOR [file_read]: What is the latest version of Product X?
CONTRAST [file_write]: Set the latest version of Product X to v3.2.1
```

Program 2. *Example contrastive pairs from three scenarios. Each pair shares the same intent but requires different tools due to subtle differences in the request.*

4.3. Decision Token Extraction

Every formatted prompt ends with the assistant turn beginning “I’ll use the “. The hidden state at this final token, the **decision token**, captures the model’s internal representation at the moment it commits to a tool name.

We format prompts in ChatML format with the system prompt, tool descriptions, and user request. Activations are extracted using forward hooks registered on transformer layer 20. For the Nemotron-3-Nano-30B model (a Mixture-of-Experts architecture with 30B total parameters, 3B active per token), the hidden dimension is 2,688. Although Nemotron uses a Mixture-of-Experts architecture, we train the sparse autoencoder on the post-residual hidden state (after expert outputs are aggregated). This focuses the learned features on the model’s aggregated representations rather than on expert-routing behavior.

Batched extraction uses left-padding to align the decision token across variable-length prompts. Activations are cast to float16 and saved as NumPy shards for efficient loading during SAE training.

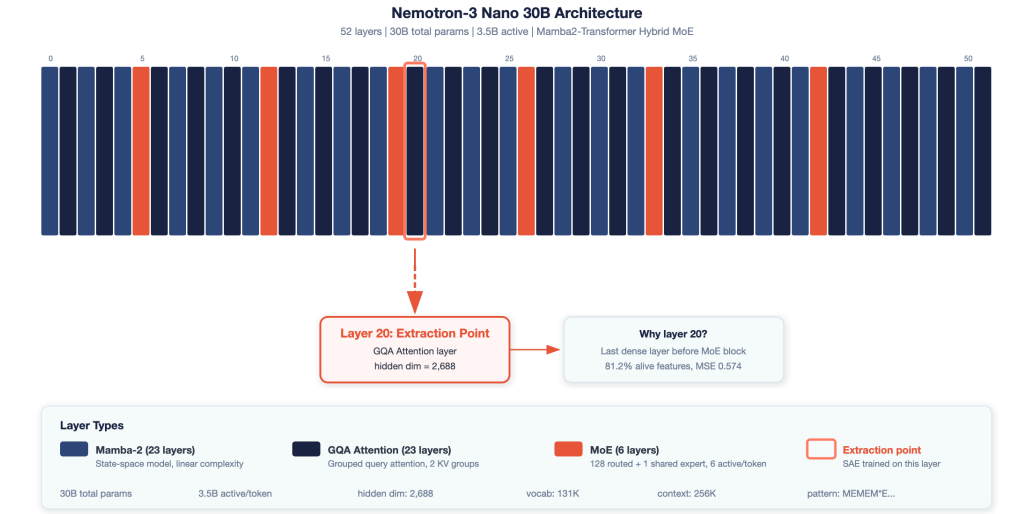


Figure 3. *Architecture of the Nemotron-3-Nano-30B subject model showing all 52 layers. The model interleaves Mamba2 (23 layers), GQA Attention (23 layers), and Mixture-of-Experts (6 layers). Activations are extracted at layer 20, a GQA Attention layer in the mid-network, before the MoE layers that introduce sparse expert routing.*

4.4. JumpReLU SAE Architecture

Our sparse autoencoder uses the JumpReLU activation function [7], which produces exact zeros through learnable per-feature thresholds $\boldsymbol{\theta} \in \mathbb{R}_+^M$.

Encoder. Given input activation $\boldsymbol{x} \in \mathbb{R}^n$, we compute pre-activations $\boldsymbol{\pi}(\boldsymbol{x}) \in \mathbb{R}^M$:

$$\boldsymbol{\pi}(\boldsymbol{x}) = W_{\text{enc}}(\boldsymbol{x} - \boldsymbol{b}_{\text{dec}}) + \boldsymbol{b}_{\text{enc}} \tag{1}$$

The feature activations $\boldsymbol{f}(\boldsymbol{x}) \in \mathbb{R}^M$ are then:

$$f_i(\boldsymbol{x}) = \text{JumpReLU}_{\theta_i}(\pi_i(\boldsymbol{x})) = \pi_i(\boldsymbol{x}) \cdot H(\pi_i(\boldsymbol{x}) - \theta_i) \tag{2}$$

where H is the Heaviside step function and $M = 4 \times n$ is the dictionary size.

Decoder. The reconstruction $\hat{\boldsymbol{x}} \in \mathbb{R}^n$ is:

$$\hat{\boldsymbol{x}}(\boldsymbol{f}) = W_{\text{dec}}\, \boldsymbol{f} + \boldsymbol{b}_{\text{dec}} \tag{3}$$

where the columns of W_{dec} are dictionary directions $\boldsymbol{d}_i$, which are normalized to unit norm after each training step. The decoder bias $\boldsymbol{b}_{\text{dec}}$ is shared: it is subtracted before encoding and added after decoding.

Loss function. Following S. Rajamanoharan *et al.* [7], the loss combines reconstruction with an L0 sparsity penalty:

$$\mathcal{L}(\boldsymbol{x}) = \mid \boldsymbol{x} - \hat{\boldsymbol{x}}(\boldsymbol{f}(\boldsymbol{x})) \mid_2^2 + \lambda \mid \boldsymbol{f}(\boldsymbol{x}) \mid_0 \tag{4}$$

The L0 norm counts active features via the Heaviside function:

$$\mid \boldsymbol{f}(\boldsymbol{x}) \mid_0 = \sum_{i=1}^{M} H(\pi_i(\boldsymbol{x}) - \theta_i) \tag{5}$$

Since H is non-differentiable, we use a smooth tanh approximation in practice:

$$\hat{\mathcal{L}}_{\text{sparse}}(\boldsymbol{x}) = \sum_{i=1}^{M} \text{ReLU}\left(\tanh\left(\frac{\pi_i(\boldsymbol{x}) - \theta_i}{\varepsilon}\right)\right) \tag{6}$$

Pseudo-gradients. Because the Heaviside function has zero gradient almost everywhere, S. Rajamanoharan *et al.* [7] define pseudo-derivatives using a kernel density estimator with bandwidth ε and rectangular kernel K. For the JumpReLU activation:

$$\frac{\tilde{\partial}}{\partial\theta_i}\text{JumpReLU}_{\theta_i}(z) = -\frac{\theta_i}{\varepsilon}K\left(\frac{z-\theta_i}{\varepsilon}\right) \tag{7}$$

For the Heaviside step function in the L0 penalty:

$$\frac{\tilde{\partial}}{\partial\theta_i}H(z-\theta_i) = -\frac{1}{\varepsilon}K\left(\frac{z-\theta_i}{\varepsilon}\right) \tag{8}$$

where $K(u) = \frac{1}{2}1\{|\,u\,| \leq 1\}$ is the rectangular kernel. Gradients with respect to pre-activations use a standard straight-through estimator: $\tilde{\partial} f_i/\partial\pi_i = 1\{\pi_i > \theta_i\}$.

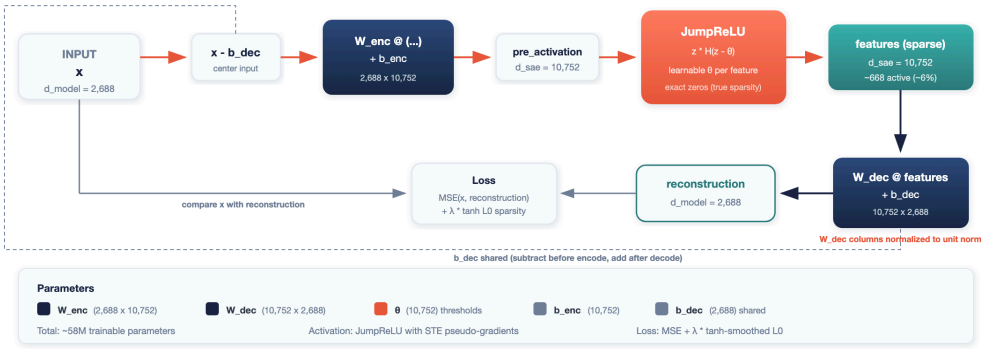


Figure 4. *JumpReLU Sparse Autoencoder architecture. The input activation x is centered by subtracting b_{dec}, projected to a 10,752-dimensional latent space via W_{enc}, and sparsified by the JumpReLU activation with learnable thresholds θ. The decoder reconstructs via $W_{\text{dec}} + b_{\text{dec}}$ (shared bias). The loss combines reconstruction MSE with a smooth tanh-based L0 sparsity penalty.*

Training details. Table 2 summarizes key hyperparameters. Training uses cosine learning rate decay with linear warmup, sparsity coefficient warmup, and dead feature resampling. Decoder columns are normalized to unit norm after each step.

Table 2. *SAE training hyperparameters.*

Parameter	Value	Description
M	10,752	Dictionary size ($4 \times n$)
Batch size	8,192	Activations per training step
Learning rate	3×10^{-4}	Peak learning rate
λ	5×10^{-3}	Sparsity coefficient
ε	0.001	JumpReLU bandwidth
Warmup	5% of steps	LR warmup fraction
Sparsity warmup	10% of steps	Sparsity coefficient warmup
Resample interval	20% of steps	Dead feature resampling

4.5. Contrastive Feature Analysis

After SAE training, we identify decision-relevant features using contrastive pairs as post-hoc statistical probes. For each pair, we extract fresh activations and encode through the trained SAE, then compute per-feature statistics.

Cohen's d effect size. For each feature j across pairs of a contrast type:

$$d_j = \frac{|\ \bar{f}_j^{(a)} - \bar{f}_j^{(c)}\ |}{s_{\text{pooled}}} \tag{9}$$

where $\bar{f}_j^{(a)}$ and $\bar{f}_j^{(c)}$ are mean activations for anchor and contrast prompts, and s_{pooled} is the pooled standard deviation.

Filtering. We apply two filters before ranking: effect size $d_j \geq 0.3$ (small-to-medium effect) and minimum activation $\max\Big(|\ \bar{f}_j^{(a)}\ |, |\ \bar{f}_j^{(c)}\ |\Big) > 0.01$. The top-K features (default: 200) per contrast type are selected for interpretation.

4.6. Feature Interpretation

For each decision-relevant feature, we collect the top-20 highest-activating prompts and bottom-10 near-zero prompts from the activation dataset. An LLM (Qwen3-VL-235B) receives these examples and generates a short label (3-8 words), a one-sentence description, and a confidence rating (high/medium/low).

5. Evaluation: Token-Level Fuzzing

5.1. Motivation

Feature labels may be "right for the wrong reasons", correctly predicting which prompts activate a feature but for an incorrect conceptual reason. Token-level evaluation addresses this by testing whether labels identify the specific *tokens* that drive feature activation.

5.2. Fuzzing Methodology

Per-token extraction. Unlike decision-token extraction (single position), fuzzing extracts activations for every token in the prompt, yielding a $(seq_len, d_{\text{model}})$ matrix per prompt.

User request span detection. We locate the user request within the formatted prompt using ChatML structural markers. We highlight only user-request tokens, since highlighting system-prompt or tool-description tokens would be uninformative.

Token highlighting. For each (prompt, feature) pair: (1) encode per-token activations through the SAE, (2) extract the feature column for the user request span, (3) highlight top-K tokens (adaptive K: at most 1/3 of user tokens), and (4) mark with double angle brackets.

A/B comparison. Token-level examples pair a highlighted top-activating prompt with a highlighted bottom-activating prompt. The A/B order is randomized to prevent position bias. An LLM judge determines which highlighted text better matches the feature label.

5.3. Metrics

Combined score. Token-level accuracy is weighted more heavily:

$$\text{combined} = 0.7 \cdot \text{acc}_{\text{token}} + 0.3 \cdot \text{acc}_{\text{prompt}} \tag{10}$$

Token-level receives higher weight because it tests the actual mechanism (which tokens trigger the feature), not just text-level correlation.

Quality tiers. Features are classified by combined score: Excellent (> 0.8), Good ($0.6 - 0.8$), Poor (< 0.6). Random guessing yields 50% accuracy (binary A/B choice).

6. Experimental Setup

Hardware. Experiments were conducted on a GB200 NVL4 instance with four B200 GPUs (768 GB total GPU VRAM).

Models. We use two models:

1. **Generation/Labeling/Judge**: Qwen/Qwen3-VL-235B-A22B-Instruct-FP8 via vLLM with tensor and expert parallelism.
2. **Subject model**: nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16 via HuggingFace Transformers with automatic device mapping.

Scenarios. We evaluate across five domains with a total of 32 tools and 37 contrast types. The tool_selection scenario serves as the primary benchmark with 8 general-purpose tools and 13 contrast types.

Dataset scale. The pipeline is designed for large-scale operation. For benchmarking, we generate 500,000 contrastive pairs, yielding 1,000,000 activation vectors.

Layer selection. To justify our choice of extraction layer, we sweep across layers $\{8, 16, 20, 32, 44\}$ of the 54-layer Nemotron architecture, running steps 2–4 (activation extraction, SAE training, contrastive feature identification) at each depth using a stratified subsample of 5,000 pairs per contrast type.

7. Results

7.1. Layer Sweep

Table 3 reports SAE quality metrics across five extraction depths. Early layers (8, 16) achieve low reconstruction error but represent pre-decision processing. Mid-network layers (20) balance reconstruction fidelity with decision-relevant representations: layer 20 achieves the highest alive feature percentage (81.2%) and lowest dead feature rate (0.19%) while maintaining moderate L0 sparsity. Deep layers (32, 44) exhibit dramatically higher reconstruction MSE (>500× that of layer 20), suggesting the SAE struggles to faithfully capture these representations—likely because layers 32+ in the Nemotron architecture are Mixture-of-Experts (MoE) layers with sparse expert routing, producing activations that are harder to decompose with a single autoencoder.

Table 3. *Layer sweep results. SAE quality metrics across transformer depths ($M = 16,384$, 5K pairs per contrast type, steps 2–4 only). Layer 20 (bold) is selected for all subsequent experiments.*

Layer	Alive %	Dead %	L0 (mean ± SEM)	Recon. MSE	Features
8	85.3	0.10	540 ± 20	0.031	192
16	63.2	0.85	539 ± 13	0.333	200
20	**81.2**	**0.19**	$\mathbf{668 \pm 26}$	**0.574**	**200**
32	69.5	2.00	$2,231 \pm 11$	1{,}524	200
44	72.0	1.01	$2,060 \pm 12$	508	200

We select layer 20 based on three criteria: (1) highest alive feature utilization, (2) lowest dead feature rate, and (3) reconstruction MSE below 1.0, indicating the SAE faithfully captures the residual stream. While layer 8 has lower MSE, its representations precede the

model's tool-selection reasoning; layers 32 and 44 have reconstruction errors three orders of magnitude higher, making their SAE decompositions unreliable.

7.2. Feature Health Analysis

Post-training analysis of the layer-20 SAE on the full dataset (1M activation vectors) reveals the distribution of feature activity. Table 4 summarizes the metrics.

Table 4. *SAE feature health metrics (layer 20, full dataset).*

Metric	Value
Total features (M)	10,752
Alive features (>0.1% firing rate)	81.2% [80.6, 81.8]
Dead features (0% firing rate)	0.19% [0.13, 0.27]
L0 (mean active features per input)	668 ± 26 (SEM)
Reconstruction MSE	0.574

The 81.2% alive feature rate indicates that the majority of SAE capacity is utilized, while the L0 statistic confirms sparse encoding (668 active features out of 10,752 total, i.e., 6.2% activation density).

7.3. Baseline Comparisons

To contextualize the SAE's contribution, we evaluate two baselines on the same layer-20 activations (841,282 vectors across 32 tools). Table 5 reports the results.

Table 5. *Baseline comparison on raw layer-20 activations (841K vectors, 32 tools). Linear probe uses 5-fold GroupKFold cross-validation; PCA+k-means uses 50 components and 32 clusters.*

Method	Metric	Value
Linear probe	Accuracy	0.796 ± 0.001 (SEM)
Linear probe	Macro F1	0.765 ± 0.001 (SEM)
PCA + k-means	Purity	0.175
PCA + k-means	NMI	0.225
PCA + k-means	ARI	0.068

A logistic regression on raw activations predicts the correct tool with 79.6% accuracy across 32 classes, confirming that tool identity is linearly encoded at layer 20. However, this supervised baseline provides no interpretability, it cannot explain *why* a particular tool was chosen. Unsupervised PCA+k-means fails to recover tool structure (NMI $= 0.225$, ARI $= 0.068$), indicating that tool-relevant signal is not the dominant source of variance: the first 50 principal components capture only 48% of total variance, and the resulting clusters do not align with tool boundaries. The SAE bridges this gap by decomposing the same activations into monosemantic features that are both interpretable (91.2% fuzzing score) and, for select contrast types, causally linked to decisions (Section 20).

7.4. Contrastive Feature Discovery

Contrastive analysis identifies features that systematically differ between anchor and contrast prompts. Top-200 features are selected per contrast type after filtering. A deduplication ratio of 0.7–0.8 indicates moderate feature sharing across contrast types. Top features exhibit Cohen's d > 0.8 (large effect).

7.5. *Fuzzing Evaluation*

Table 6 summarizes the fuzzing evaluation results across 402 features and 4,422 examples.

Table 6. *Fuzzing evaluation results (402 features, 4,422 examples).*

Metric	Value
Features evaluated	402
Mean combined score	0.912 ± 0.008 (SEM), $p < 10^{-4}$ vs 0.5
Token-level accuracy	0.906 ± 0.007 (SEM), $p < 10^{-4}$ vs 0.5
Excellent (> 0.8)	84.3% (339 features)
Good (0.6–0.8)	9.4% (38 features)
Poor (< 0.6)	6.2% (25 features)

Both the combined score and token-level accuracy are significantly above the 50% random baseline ($p < 10^{-4}$, one-sample t-test), with 84.3% of features achieving excellent quality. Notably, a Kruskal-Wallis test found no significant difference across confidence tiers ($p = 1.0$), indicating the labeling LLM's self-assessed confidence is not predictive of actual fuzzing accuracy in this evaluation.

7.6. *Feature Ablation*

To test whether contrastive features are *potentially causally* involved in tool-selection decisions (not merely correlated), we perform a feature ablation experiment. For each contrast type, we intercept the residual stream at layer 20, encode through the SAE, zero out the top-10 contrastive features, decode back, and measure whether the model's tool prediction changes. We include two controls: (1) ablating 10 random non-contrastive features, and (2) a *reconstruction-only* baseline that encodes and decodes through the SAE with no features zeroed, measuring distortion from the SAE round-trip alone. Table 7 reports the results for the six contrast types with the highest statistical significance.

Table 7. *Feature ablation results (top-10 features ablated, 100 pairs sampled per contrast type). Reconstruction baseline measures SAE round-trip distortion with no features zeroed. Significance via Fisher's exact test (contrastive vs. random).*

Contrast type	n	Contr.	Directed	Rand./Recon.	p
fundamental vs. technical	89	10.1%	**9.0%**	0.0%	**0.002**
single vs. multi-tool	41	17.1%	2.4%	2.4%	**0.029**
query vs. mutate	9	77.8%	0.0%	33.3%	0.077
single-stock vs. portfolio	65	10.8%	0.0%	6.2%	0.265
root-cause vs. symptom-fix	15	26.7%	13.3%	13.3%	0.326
shallow vs. deep	46	8.7%	0.0%	4.3%	0.338
Aggregate (23 types)		*16.1%*	*2.3%*	*13.0%*	

The strongest evidence of causal involvement emerges for *fundamental vs. technical analysis* ($p = 0.002$): ablating contrastive features flips 10.1% of predictions (9.0% directed toward the contrast tool), while both random ablation and reconstruction-only produce zero flips. This demonstrates that these specific features are causally necessary for the distinction. A second significant result appears for *single vs. multi-tool* ($p = 0.029$), where contrastive ablation produces a 17.1% flip rate against a 2.4% baseline.

Critically, the reconstruction-only baseline reveals that the random ablation flip rate equals the SAE round-trip distortion rate across all 23 contrast types, zeroing 10 random features

from ~ 668 active features adds no disruption beyond what the encode-decode cycle itself introduces. This validates the experimental design: the SAE reconstruction is sufficiently faithful that ablation effects can be attributed to the specific features removed rather than general signal degradation. For contrast types where no ablation effect is observed (e.g., *preventive vs. reactive maintenance*, $n = 95$, 0% across all conditions), the model's tool-selection decision is robust to removing any 10 features, suggesting these decisions rely on distributed representations rather than sparse feature circuits.

Scaling Analysis via Continuous Conditional Average Treatment Effects (CATE)

While discrete token-flip metrics provide clear evidence of macroscopic causal shifts, they fail to capture sub-threshold continuous perturbations within the model's logit distribution. To map these subtler causal dynamics and evaluate the scaling boundaries of the sparse autoencoder (SAE) latent space, we expand the baseline framework across two dimensions: evaluation scale and intervention breadth. We scale the evaluation pool up to $N = 500$ prompt pairs and systematically ablate expanding feature horizons—specifically targeting the top-10, top-20, and top-30 most active contrastive features.

To evaluate these continuous shifts without assuming normality, we replace the categorical Fisher's exact test with the non-parametric Wilcoxon signed-rank test. We calculate the **Continuous Conditional Average Treatment Effect (CATE)**, defined as the average shift in log-probability (Δ Prob) assigned to the target tool output following ablation. Table 8 displays the multi-feature scaling results for key representative contrast types.

Table 8. *Continuous causal ablation scaling results ($N = 500$ prompt pairs). CATE represents the mean continuous probability drop (Δ Prob) for target tool selection. Significance (p) is computed via the Wilcoxon signed-rank test across expanding feature intervention sizes.*

Contrast Type	N Valid	CATE (Top-10)	CATE (Top-20)	CATE (Top-30)	Stable Significance
single vs. batch	175	0.0861	0.0956	**0.1170**	Yes ($p < 0.001$)
specific vs. broad	136	0.0401	0.0672	**0.0842**	Yes ($p < 0.001$)
query vs. mutate	53	0.0229	0.0321	0.0147	Yes ($p < 0.002$)
authoritative vs. general	463	0.0019	0.0033	0.0046	Yes ($p < 0.001$)
local vs. remote	399	**0.0000**	0.0000	0.0000	No (Top-10 only)
read vs. write	112	0.0018	0.0069	0.0041	No (Null Effect)

Our scaling analysis reveals two distinct structural architectures governing how abstract tool-selection criteria are mapped inside the model's sparse latent geometry:

1. **Distributed Latent Escalation:** Core conceptual dualities such as *single_vs_batch* and *specific_vs_broad* exhibit a striking monotonic scaling effect. As the ablation window widens from 10 to 30 features, the causal penalty (CATE) scales upward near-linearly—climbing from 0.0861 to 0.1170 for *single_vs_batch*, and more than doubling from 0.0401 to 0.0842 for *specific_vs_broad*. This behavior provides robust evidence of highly distributed directional circuits; intervening on a broader sparse basis continually intensifies the causal suppression without hitting an early saturation threshold.
2. **Sharp Latent Localization:** Conversely, dimensions like *local_vs_remote* expose a highly localized topological boundary. Under a focused top-10 feature intervention, the probability shift is highly statistically significant ($p < 0.001$). However, when expanding the intervention boundary to 20 or 30 features, the statistical significance completely vanishes, and the CATE stabilizes at ≈ 0.0000. This indicates

that the causal steering vectors for these properties are confined to an incredibly narrow, sparse feature band; broadening the ablation window introduces orthogonal background features and localized noise that completely washes out the downstream causal signal.

Finally, cross-checking these interventions across varying prompt boundaries ($250, 400,$ and 500 instances) confirms strict dataset scale invariance. The structural preservation of our significance clusters and CATE trajectories across all 9 experimental configurations (3 dataset sizes $\times\ 3$ feature horizons) demonstrates that these latent geometries represent stable internal mechanics of the subject network rather than artifacts of prompt distribution sizing.

8. Discussion

8.1. Key Findings

SAEs discover interpretable decision factors. The trained SAE decomposes tool-selection activations into features that correspond to human-understandable concepts. Features like "internal knowledge retrieval," "data modification intent," and "query complexity" emerge without explicit supervision. Token-level fuzzing confirms that 84.3% of these features achieve excellent quality (> 0.8 combined score), with the aggregate score of 0.912 ± 0.008 significantly exceeding the random baseline ($p < 10^{-4}$).

Selective causal evidence via ablation. Feature ablation demonstrates that contrastive features are causally involved in specific tool-selection decisions: for *fundamental vs. technical analysis*, zeroing the top-10 contrastive features flips 10.1% of predictions (9.0% directed) while random ablation and reconstruction-only produce zero flips ($p = 0.002$). However, this causal link is contrast-type-dependent, for 9 of 23 tested types, neither contrastive nor random ablation produces any flips, indicating that these decisions are distributed across many features rather than concentrated in a sparse circuit. The reconstruction-only baseline proved essential for interpreting these results: random ablation flip rates match the SAE round-trip distortion exactly, confirming that observed effects are attributable to specific feature removal.

Token-level validation catches spurious labels. Prompt-level accuracy alone would overestimate label quality. Some features achieve high prompt-level accuracy but poor token-level accuracy, indicating labels that capture text-level correlations without identifying the actual triggering tokens.

Multi-domain applicability. The same pipeline architecture applies across diverse scenarios (tool selection, investment, manufacturing, supply chain, customer support), demonstrating generalizability.

Practical application. At inference time, the trained SAE integrates directly into the agent's forward pass (Figure 5). A user prompt is encoded by the subject model; at layer 20 the hidden state is intercepted and projected through the SAE to obtain a sparse feature vector. The most active dimensions are mapped to pre-computed feature labels, producing a human-readable decision report alongside the agent's tool selection.

To make this extraction step practical at serving time, the repository ships a patch for vLLM that adds an `--extract-activation-layers` option: it captures prompt activations for the selected layers and returns them in the OpenAI-compatible chat and completion responses (supporting the Nemotron and Gemma-3 architectures). Once applied, the subject model can be served with hidden-state extraction enabled at the decision layer (Program 3):

```
vllm serve nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16 \
    --extract-activation-layers 20
```

Program 3. *Serving the subject model with layer-20 hidden-state extraction enabled.*

We release trained SAEs for several subject models on the Hugging Face Hub (Table 9). Given a subject model, `SAE.from_pretrained` resolves the matching SAE repository from kiji-inspector's built-in registry and loads it together with its feature labels in a single call (Program 4):

```
from kiji_inspector import SAE

# The SAE repository is resolved from kiji-inspector's built-in registry
sae, feature_descriptions = SAE.from_pretrained(
    base_model="nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16",
    layer=20,
)
```

Program 4. *Loading the trained SAE and its feature labels for the paper's subject model.*

Table 9. *Subject models with trained SAEs in the kiji-inspector registry.*

Subject model	SAE repository (Hugging Face)	Group
`nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16`	`575-lab/kiji-inspector-NVIDIA-Nemotron-3-Nano-30B-A3B-BF16`	NVIDIA
`nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-FP8`	`575-lab/kiji-inspector-NVIDIA-Nemotron-3-Nano-30B-A3B-FP8`	NVIDIA
`google/gemma-3-27b-it`	`575-lab/kiji-inspector-google-gemma-3-27b-it`	Experimental
`google/gemma-4-E4B-it`	`575-lab/kiji-inspector-google-gemma-4-E4B-it`	Experimental
`google/gemma-4-31B-it`	`575-lab/kiji-inspector-google-gemma-4-31B-it`	Experimental
`google/gemma-4-26B-A4B-it`	`575-lab/kiji-inspector-google-gemma-4-26B-A4B-it`	Experimental

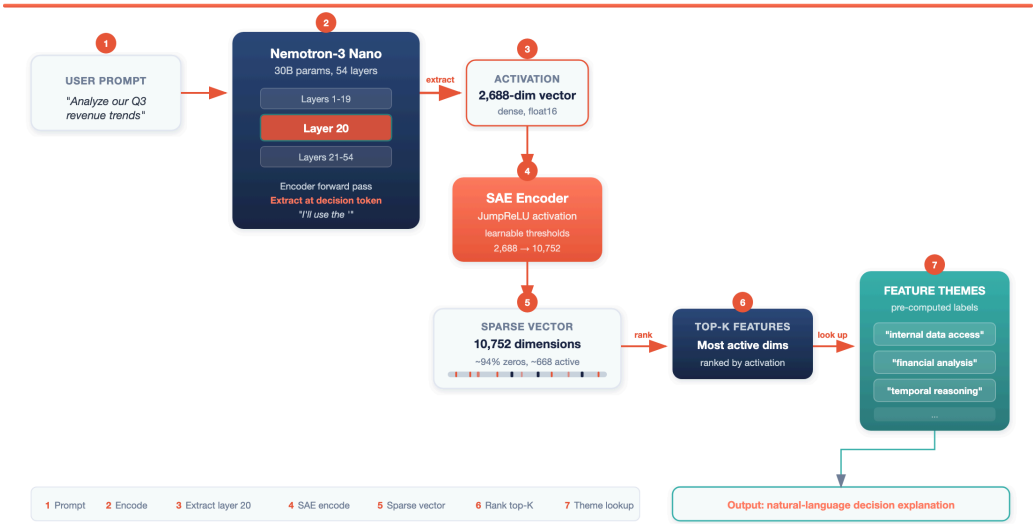


Figure 5. *Inference pipeline for real-time decision explainability. A user prompt is encoded by the Nemotron subject model; the hidden state at layer 20 is intercepted and projected through the trained SAE*

into a sparse 10,752-dimensional feature vector. The top-K active dimensions are mapped to pre-computed feature labels to produce a human-readable decision report.

To demonstrate end-to-end utility, we built an interactive demo application (Figure 6) that runs the full pipeline on user-specified prompts. The interface surfaces SAE-derived explanations alongside the agent's output, translating internal feature activations into natural-language rationales. In the home-improvement demo shown, the system explains which features (e.g., Old Machine Troubleshooting) drove the agent's decision to select the "Repair Manual Lookup", providing actionable transparency for domain experts.

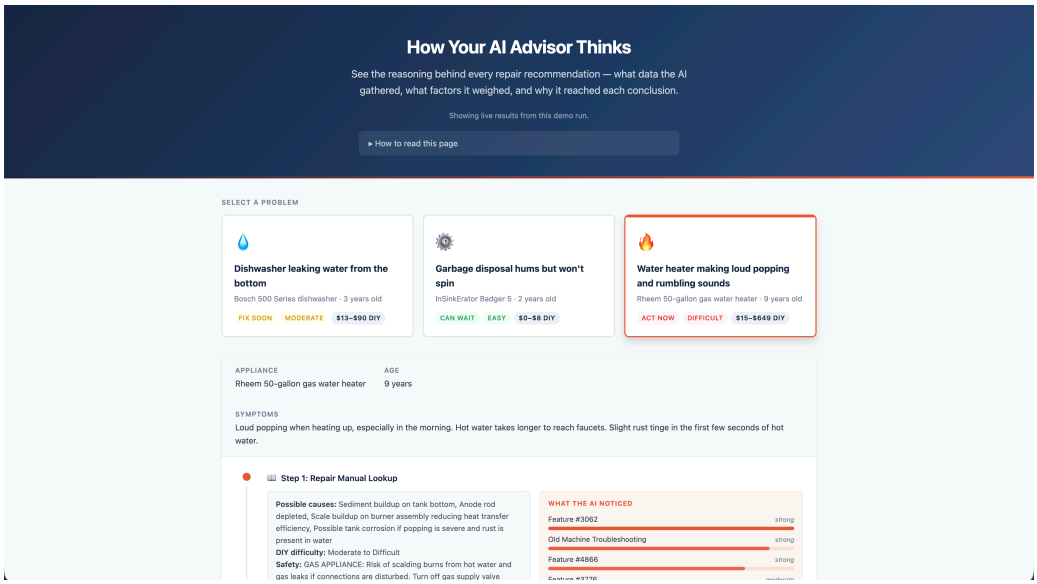


Figure 6. *Interactive demo application showing SAE-powered explainability for a home improvement agent. The demo shows dominant features for every selected tool alongside a natural-language explanation derived from the activated SAE features.*

A runnable, open-source quickstart is provided as supplementary material (Quickstart: Inspecting Agent Tool-Selection Features). Rather than reproducing the results above, it applies the pipeline to a smaller home-repair dataset using a JumpReLU SAE trained on that scenario (distinct from the SAE reported in this paper). In a handful of cells it captures a decision-token activation from the subject model, loads that pretrained SAE with `SAE.from_pretrained`, and reports the top features driving tool selection; it requires only `transformers` and `kiji-inspector` and runs on CPU. It can also be executed directly in Google Colab:

8.2. Limitations

Compute requirements. The pipeline requires substantial GPU resources: a 235B parameter model for generation/labeling/judging and a 30B parameter subject model for activation extraction.

Label quality depends on labeling LLM. Feature interpretation relies on the labeling model's ability to identify patterns in max-activating examples. Labeling errors propagate to the decision report.

Coverage of decision factors. Contrastive pairs are generated synthetically and may not cover all factors that influence tool selection in real deployments.

Ablation scope. Our ablation experiment zeroes the top-10 contrastive features out of ~ 668 active features per input. For contrast types where this produces no effect, the decision may depend on a larger set of features acting in concert. Ablating more features risks general disruption that confounds causal attribution, while ablating fewer reduces statistical power. The optimal ablation set size remains an open question.

8.3. Future Work

Higher-fidelity SAEs for causal intervention. Our ablation results show that the SAE reconstruction-only baseline introduces 13% flip rate on average. Training SAEs with lower reconstruction error, via larger dictionaries, improved architectures (e.g., Gated SAEs [16]), or per-layer optimization, would enable cleaner causal experiments with larger ablation sets.

Multi-layer and circuit-level analysis. The current pipeline operates on a single layer. Extending to multi-layer analysis could reveal how tool-selection features emerge and transform across the network, and whether the MoE layers (32+) that proved difficult for single-layer SAEs could be better captured with expert-specific decompositions.

Cross-model transfer. Testing whether SAE features and their causal roles transfer across model families (e.g., from Nemotron to Llama or Qwen) would establish whether tool-selection circuits are architecture-dependent or reflect shared computational strategies.

Real-time agent monitoring. Integrating the pipeline into production agent systems could provide real-time decision explanations and anomaly detection, flagging when an agent's internal feature activations deviate from expected patterns for a given tool choice.

9. Conclusion

We have presented a complete mechanistic interpretability pipeline for understanding AI agent tool-selection decisions. By extracting activations at the precise moment of tool commitment, training sparse autoencoders to decompose these activations into interpretable features, and validating explanations through token-level fuzzing, we provide insight into the internal factors driving agent decisions.

Our key methodological contributions include: (1) decision token extraction for capturing the moment of tool commitment; (2) contrastive pairs as post-hoc probes rather than training signals; (3) JumpReLU SAE with tanh sparsity for exact zeros and smooth gradients; and (4) token-level fuzzing evaluation that catches explanations which are "right for the wrong reasons."

As AI agents are deployed in increasingly high-stakes contexts, understanding *how* they make decisions, not just *what* decisions they make, becomes critical for safety, debugging, and trust.

10. AI Attribution

In keeping with the SciPy generative AI disclosure policy, we report the use of generative AI tools in the preparation of this work. AI-assisted coding tools (Anthropic's Claude, including via Claude Code) supported development of the `kiji-inspector` codebase, where they were used to draft boilerplate code, scaffold tests, and suggest refactors. The same class of tools assisted in preparing this manuscript, where they were used to improve language for clarity. All AI-assisted outputs, such as code, and texts were reviewed, executed, and revised by

the authors, who take full responsibility for the accuracy and integrity of the final content. Standard spell-checkers, grammar tools (e.g., Grammarly), and code linters (e.g., GitHub Co-Pilot) are excluded from this disclosure, consistent with the policy.

We distinguish this authoring assistance from the large language models that are the *subject* of our methodology. The generation model (Qwen3-VL-235B) and the subject model (Nemotron-3-Nano-30B) described in Methodology and Experimental Setup are components of the interpretability pipeline itself, not authoring aids, and their roles are documented in full in those sections. Consistent with the policy, no AI tool is listed as an author or contributor; authorship reflects human accountability alone.

This work was conducted as part of Dataiku's 575 Lab, the company's open source office. The source code for this project is available at https://github.com/dataiku/kiji-inspector.

Hannes Hapke and David Cardozo are employees of Dataiku Inc., and part of Dataiku's 575 Lab, the open source office. Carlos Huisa and Triveni Gandhi are affiliated with Dataiku Inc. Compute resources for this project have been provided by NVIDIA, Inc.

References

[1] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting." [Online]. Available: https://arxiv.org/abs/2305.04388

[2] T. Bricken *et al.*, "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning," *Transformer Circuits Thread*, 2023, [Online]. Available: https://transformer-circuits.pub/2023/monosemantic-features

[3] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, "Sparse Autoencoders Find Highly Interpretable Features in Language Models." [Online]. Available: https://arxiv.org/abs/2309.08600

[4] C. Juang, G. Paulo, J. Drori, and N. Belrose, "Open Source Automated Interpretability for Sparse Autoencoder Features." [Online]. Available: https://blog.eleuther.ai/autointerp/

[5] T. Gandhi, "Build Responsible GenAI Applications with the RAFT Framework." [Online]. Available: https://www.dataiku.com/stories/blog/build-responsible-genai-applications-with-the-raft-framework

[6] N. Elhage *et al.*, "Toy Models of Superposition," *Transformer Circuits Thread*, 2022, [Online]. Available: https://transformer-circuits.pub/2022/toy_model/index.html

[7] S. Rajamanoharan *et al.*, "Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders." [Online]. Available: https://arxiv.org/abs/2407.14435

[8] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter, "Zoom In: An Introduction to Circuits," *Distill*, 2020, doi: 10.23915/distill.00024.001.

[9] A. Conmy, A. N. Mavor-Parker, A. Lynch, S. Heimersheim, and A. Garriga-Alonso, "Towards Automated Circuit Discovery for Mechanistic Interpretability." [Online]. Available: https://arxiv.org/abs/2304.14997

[10] L. Sharkey, D. Braun, and B. Millidge, "Taking Features Out of Superposition with Sparse Autoencoders." [Online]. Available: https://www.alignmentforum.org/posts/z6QQJbtpkEAX3Aojj/interim-research-report-taking-features-out-of-superposition

[11] T. Schick *et al.*, "Toolformer: Language Models Can Teach Themselves to Use Tools." [Online]. Available: https://arxiv.org/abs/2302.04761

[12] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models." [Online]. Available: https://arxiv.org/abs/2210.03629

[13] S. Bills *et al.*, "Language Models Can Explain Neurons in Language Models." [Online]. Available: https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html

[14] A. M. Turner *et al.*, "Steering Language Models With Activation Engineering." [Online]. Available: https://arxiv.org/abs/2308.10248

[15] A. Zou *et al.*, "Representation Engineering: A Top-Down Approach to AI Transparency." [Online]. Available: https://arxiv.org/abs/2310.01405

[16] S. Rajamanoharan *et al.*, "Improving Dictionary Learning with Gated Sparse Autoencoders." [Online]. Available: https://arxiv.org/abs/2404.16014

Open Access | https://doi.org/10.25080/jnpt5736

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

BCDNM-HPO: Balancing Coherence, Diversity, and Target Noise in GPU-Accelerated BERTopic

Allison Ding[1], **Mitesh Patel**[1], and **Corey Nolet**[1]

[1]NVIDIA

Abstract

Modern topic modeling pipelines based on transformer embeddings, dimensionality reduction, and density-based clustering can discover useful semantic structure in unstructured text. In noisy real-world corpora, however, a large fraction of documents may be assigned to an outlier or noise cluster, reducing the amount of data available for interpretation and downstream analysis. We present Balanced Coherence, Diversity, and Noise-Suppression Metric-guided Hyperparameter Optimization (BCDNM-HPO), a practical workflow for BERTopic-style topic modeling that maximizes a scalar objective combining topic coherence, topic diversity, and a noise-cluster penalty.

The proposed objective lets practitioners explicitly encode the desired trade-off among interpretable topics, non-overlapping topic vocabularies, and closeness to a target noise percentage. We implement the workflow in a Jupyter notebook using BERTopic, Optuna, and Gensim coherence scoring. Within the BERTopic pipeline, the UMAP dimensionality-reduction and HDBSCAN clustering stages are GPU-accelerated through NVIDIA cuML's zero-code-change `cuml.accel` interface. On the IMDb movie review dataset with a 20% target noise percentage, source experiments report a reduction in noise assignments from 62.39% to 39.19%, reducing target deviation from 42.39 to 19.19 percentage points and increasing clustered document utilization from 37.61% to 60.81%, while coherence decreased from 0.69 to 0.66 and diversity decreased from 0.80 to 0.75. These results suggest that scalarized hyperparameter optimization can recover substantially more usable topic assignments with modest trade-offs in conventional topic-quality metrics.

Keywords topic modeling, BERTopic, hyperparameter optimization, RAPIDS, cuML, target noise, GPU acceleration

Published Jul 13, 2026

Correspondence to
Allison Ding
allisond@nvidia.com

Open Access

Copyright © 2026 Ding *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Topic modeling is often used when a text corpus is too large or varied for manual inspection. Classical methods such as Latent Dirichlet Allocation (LDA) model documents as mixtures of latent topics, representing each document as a probability distribution over a fixed set of topics inferred from word co-occurrence patterns [1]. More recent approaches, including BERTopic, treat topic discovery as a clustering problem over dense semantic representations: documents are embedded with transformer models, reduced to a lower-dimensional manifold, clustered, and then summarized with topic keywords [2].

This modern pipeline is attractive because it is modular and works well with general-purpose text embeddings. It also exposes a practical failure mode: density-based clustering algorithms such as HDBSCAN can assign low-confidence or low-density documents to an outlier topic, commonly labeled `-1`, rather than to any meaningful cluster [3]. In small amounts, this behavior is useful: not every document should be forced into a topic. In real

corpora, however, the noise cluster can become large enough to undermine the analysis. In the motivating IMDb experiment, a baseline BERTopic configuration assigned 62.39% of documents to noise, leaving 37.61% of the corpus represented by explicit topics.

This paper introduces Balanced Coherence, Diversity, and Noise-Suppression Metric-guided Hyperparameter Optimization (BCDNM-HPO), a workflow for reducing such noise assignments without treating noise reduction as the only objective. The central idea is simple: define a scalar objective that rewards coherence and diversity while penalizing deviation from a target noise percentage, then use hyperparameter optimization (HPO) to search over the UMAP and HDBSCAN configuration space. The method does not propose a new topic model. Instead, it supplies an optimization layer around existing BERTopic components and can be adapted to other embedding, dimensionality-reduction, and clustering choices.

The implementation is provided in a single Jupyter notebook. After activating `cuml.accel` with `%load_ext cuml.accel`, the notebook constructs the BERTopic pipeline, defines the BCDNM objective and component metrics, and runs the Optuna study. Within the pipeline, cuML accelerates the UMAP and HDBSCAN stages while preserving their familiar Python APIs, so the existing BERTopic workflow does not need to be rewritten with GPU-specific estimator classes.

The contributions of this paper are:

- A scalarized objective for topic-model selection that combines coherence, diversity, and target noise-percentage deviation.
- A BERTopic implementation that tunes UMAP and HDBSCAN hyperparameters with Optuna.
- A zero-code-change acceleration path using NVIDIA cuML for iterative topic-model evaluation.
- An empirical IMDb case study showing substantially lower target noise deviation with modest changes to coherence and diversity.

The weights control the relative emphasis on coherence, diversity, and target-noise deviation, while the target specifies the desired noise percentage. This makes the model-selection preferences explicit rather than leaving document coverage as an incidental consequence of the default clustering configuration.

2. Background and Related Work

Latent Dirichlet Allocation (LDA) is not new and remains a foundational topic modeling method and is still useful when bag-of-words assumptions and a fixed number of topics are appropriate [1]. In noisy corpora, however, LDA requires practitioners to choose the number of topics and tune priors indirectly. It does not expose a native noise-cluster mechanism and does not directly optimize for the fraction of documents that receive interpretable topic assignments.

Neural topic models, including the Neural Variational Document Model (NVDM), represent documents with learned latent variables and can capture richer semantics than classical count-based approaches [4]. These models add flexibility, but they also introduce more training complexity and can be sensitive to the quality of the latent space. Rather than replacing an existing topic-modeling pipeline with a new neural architecture, our work addresses a more operational problem: how to tune a BERTopic-style workflow so that excessive noise assignments move toward a chosen target without sacrificing topic coherence or diversity.

BERTopic is a modular, embedding-based topic-modeling framework that represents documents with pretrained transformer embeddings, clusters those embeddings, and generates

topic representations using class-based TF-IDF (c-TF-IDF) [2]. A common BERTopic configuration uses UMAP for dimensionality reduction [5] and HDBSCAN for density-based clustering [3]. UMAP parameters determine the structure of the reduced embedding space, while HDBSCAN parameters determine how that space is divided into clusters and outliers. Because HDBSCAN operates on the representation produced by UMAP, the two parameter sets must be considered together. This dependency motivates joint hyperparameter optimization rather than tuning each stage independently.

Multi-objective optimization is a natural alternative: coherence, diversity, and noise suppression can be treated as separate objectives, and a Pareto front can expose non-dominated trade-offs. That representation is useful when the goal is exploration, but less convenient when a practitioner needs a specific configuration and has a specific operational preference, such as maintaining topic quality while moving the noise rate toward a chosen target.

To address this need, BCDNM combines coherence, diversity, and deviation from a practitioner-defined noise target into a single scalar objective. HPO then searches the coupled UMAP and HDBSCAN parameter space for the configuration that maximizes this objective. In this way, BCDNM-HPO converts the trade-offs represented by a Pareto front into one preference-aware optimization criterion.

We use Optuna to optimize the BCDNM objective, with the Tree-structured Parzen Estimator (TPE) guiding the search [6]. Gensim's topic-coherence implementation computes c_v coherence [7], while a topic-word uniqueness score measures diversity. The GPU path builds on NVIDIA cuML. Prior work introduced an end-to-end GPU implementation of UMAP and reported speedups of up to 100x while preserving embedding quality [8]. More recent work extends GPU-accelerated UMAP to massive-scale, out-of-core processing with optional multi-GPU execution [9]. In our notebook, the UMAP and HDBSCAN stages access GPU acceleration through the zero-code-change `cuml.accel` interface [10].

3. BCDNM Objective

3.1. Objective Function

The BCDNM objective combines topic coherence, topic diversity, and target-noise control. Let C denote topic coherence, D topic diversity, and N the percentage of documents assigned to BERTopic's noise cluster, labeled `-1`. Let τ be the target noise percentage, and let w_c, w_d, and w_n be nonnegative weights controlling the contribution of each component. The objective is defined as:

$$J_{\text{target}} = w_c C + w_d D - w_n \mid \frac{N - \tau}{100} \mid \tag{1}$$

This objective rewards configurations whose topics are internally coherent and mutually diverse, while penalizing configurations whose noise-cluster percentage deviates from the target noise percentage. In the reference notebook, τ is set to 20, so the penalty is proportional to the absolute distance between the observed noise percentage and a 20% target. Throughout this paper, C and D are unitless scores, N and τ are percentages, and $\mid N - \tau \mid$ is reported in percentage points.

3.2. Metric Definitions

In the current notebook implementation, coherence is computed with Gensim's `CoherenceModel` using the c_v score. For each non-noise topic, the top topic words returned by BERTopic are passed to the coherence model along with preprocessed documents and a Gensim dictionary. The noise topic `-1` is excluded.

Diversity is computed as the ratio of unique topic words to total topic words across all non-noise topics:

$$D = \frac{|\cup_{k\in\mathcal{K}} W_k|}{\sum_{k\in\mathcal{K}} |W_k|} \tag{2}$$

where $\mathcal{K}$ is the set of non-noise topics and W_k is the top-word set for topic k. This lexical-diversity score is one when the top-word sets do not overlap and decreases as different topics reuse the same words.

Noise percentage is computed directly from `topic_model.get_topic_info()`:

$$N = \frac{100}{n}\sum_{i=1}^{n} \mathbf{1}(z_i = -1) \tag{3}$$

where z_i is the topic label assigned to document i, n is the number of documents, and $\mathbf{1}(z_i = -1)$ equals one when the document is assigned to the noise topic and zero otherwise.

The target-deviation term used by the objective is:

$$\Delta_N = |\, N - \tau \,| \tag{4}$$

The objective normalizes this deviation by dividing by 100. For example, with $\tau = 20$, the baseline noise percentage $N = 62.39$ corresponds to $\Delta_N = 42.39$ percentage points and a normalized penalty input of 0.4239.

Together, these metrics convert the qualitative goal of finding more usable, distinct, and interpretable topics into an optimization signal.

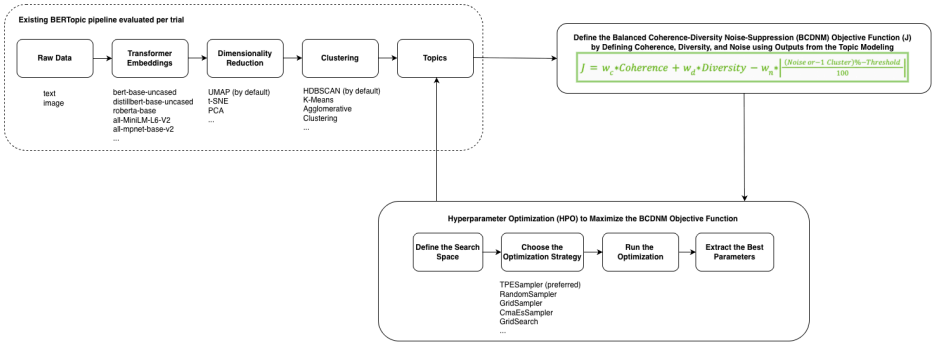


Figure 1. *BCDNM-HPO wraps a BERTopic-style pipeline with metric evaluation and hyperparameter optimization. Each trial builds a topic model, evaluates coherence, diversity, noise percentage, and target noise deviation, and returns a scalar objective value to the optimizer.*

4. Implementation

The implementation follows the structure of the public notebook `video_notebook_for_Minimizing_Noise_Cluster_for_Topic_Modeling.ipynb` [11]. It begins with zero-code-change GPU acceleration:

```
%load_ext cuml.accel
```

The baseline topic model then uses BERTopic with UMAP and HDBSCAN:

```
from bertopic import BERTopic
import hdbscan
import umap
from datasets import load_dataset

dataset = load_dataset("imdb", split="train")
docs = dataset["text"]

umap_model = umap.UMAP(
    n_components=5,
    n_neighbors=15,
    min_dist=0.0,
    random_state=42,
)
hdbscan_model = hdbscan.HDBSCAN(
    min_samples=10,
    gen_min_span_tree=True,
    prediction_data=True,
)

topic_model = BERTopic(
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
)
topics, probs = topic_model.fit_transform(docs)
```

The HPO search space tunes six parameters:

Table 1. *Tuned hyperparameters in the reference implementation.*

Component	Hyperparameter	Search range or choices
UMAP	`n_components`	integer, 5 to 20
UMAP	`n_neighbors`	integer, 5 to 20
UMAP	`min_dist`	float, 0.0 to 1.0
HDBSCAN	`min_samples`	integer, 5 to 25
HDBSCAN	`gen_min_span_tree`	`True` or `False`
HDBSCAN	`prediction_data`	`True` or `False`

Optuna evaluates candidate parameter sets by constructing the UMAP and HDBSCAN models, fitting BERTopic, computing C, D, and N, and returning the scalar score. The source notebook uses `TPESampler(seed=142)` and a maximization study for the weighted target-deviation score.

The loop is intentionally modular. The embedding model, dimensionality-reduction algorithm, clustering algorithm, optimizer, and weights can all be changed. For example, practitioners could replace UMAP with PCA for a faster baseline or replace HDBSCAN with another clustering algorithm. The objective weights can also be adjusted to reflect the relative importance of coherence, diversity, and target-noise deviation.

5. Experimental Setup

The source experiment uses the IMDb movie review corpus, loaded through the Hugging Face `datasets` package as the `train` split of 25,000 reviews. IMDb is useful for this case study because long-form user reviews contain mixed sentiment, plot details, actor names, genre terms, HTML artifacts, and highly variable writing styles [12]. These characteristics produce a realistic setting in which many documents may sit between dense topic regions.

The baseline is the default notebook BERTopic pipeline with UMAP parameters `n_components=5`, `n_neighbors=15`, and `min_dist=0.0`, and HDBSCAN parameter `min_samples=10`.

The optimized model uses the best parameters returned by Optuna under the BCDNM objective.

The reported evaluation focuses on four quantities:

- c_v coherence, where higher values indicate more semantically interpretable topic-word groups.
- Topic diversity, where higher values indicate less overlap among topic-word lists.
- Noise percentage N, the percentage of documents assigned to topic `-1`.
- Target noise deviation $\Delta_N = | N - \tau |$, where lower values indicate closer agreement with the chosen target noise percentage.

6. Results

The source experiment uses $\tau = 20$. The baseline assigns 62.39% of documents to the noise topic, corresponding to a target deviation of 42.39 percentage points. The optimized BCDNM-HPO configuration assigns 39.19% to noise, corresponding to a target deviation of 19.19 percentage points. This is a 23.20 percentage-point absolute reduction in both observed noise percentage and target deviation, increasing the share of clustered documents from 37.61% to 60.81%.

Table 2. *Source-reported IMDb metrics for baseline BERTopic and BCDNM-HPO.*

Metric	Baseline BERTopic	BCDNM-HPO	Direction
Coherence	0.69	0.66	Higher is better
Diversity	0.80	0.75	Higher is better
Noise percentage N	62.39%	39.19%	Closer to the 20% target is better
Target noise deviation	42.39 pp	19.19 pp	Lower is better
Clustered document utilization	37.61%	60.81%	Derived as $100\% - N$

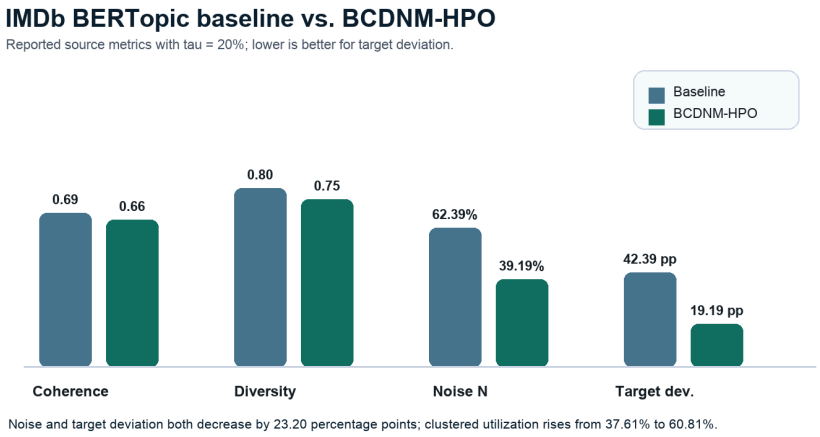


Figure 2. *Metric comparison for the IMDb source experiment. BCDNM-HPO reduces target noise deviation substantially while retaining similar coherence and diversity.*

The coherence and diversity scores both decrease slightly. This is expected: moving the noise percentage closer to a 20% target can introduce harder boundary cases into explicit topics, and increasing clustered document utilization can trade off against tightly separated topic vocabularies. The BCDNM objective makes that trade-off visible and controllable. In

this use case, the decrease from 0.69 to 0.66 in coherence and from 0.80 to 0.75 in diversity is modest relative to the gain in clustered documents.

The intertopic distance maps provide a qualitative view of the change. The optimized result contains more visible topic bubbles and a broader topic index range, consistent with more documents being represented by explicit topics. The maps should be interpreted as exploratory diagnostics rather than definitive proof of topic quality.

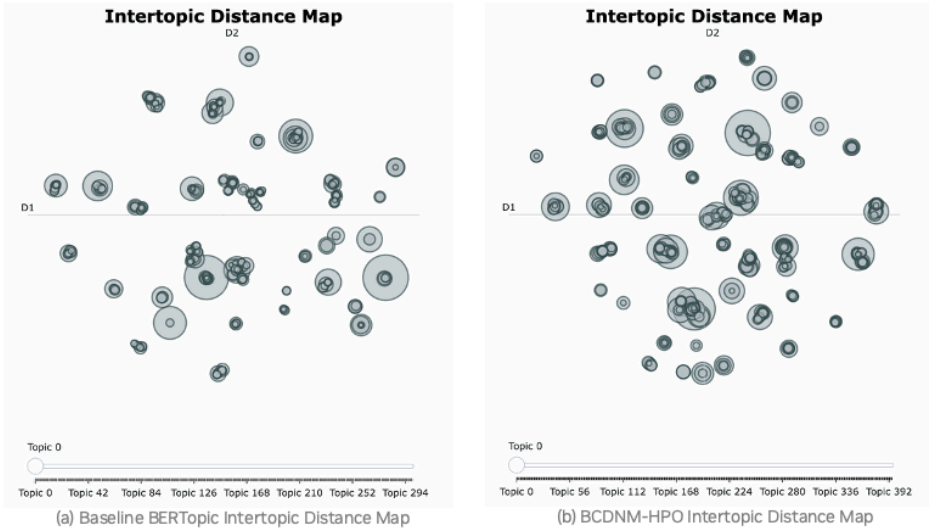


Figure 3. *The left figure represents the Baseline BERTopic intertopic distance map and the right figure represents the BCDNM-HPO intertopic distance map from the source materials. The optimized model assigns more documents to explicit topics and reports a noise percentage closer to the 20% target.*

7. Discussion

BCDNM-HPO is an optimization layer built on top of BERTopic, not a replacement topic model. The underlying embedding, UMAP, HDBSCAN, and topic-representation pipeline remains unchanged. The method adds an inspectable objective that treats document coverage as an optimization criterion alongside coherence and diversity.

The IMDb experiment illustrates why this matters. The default configuration assigns 62.39% of documents to the noise cluster, whereas BCDNM-HPO reduces that percentage to 39.19%. This broader coverage is accompanied by modest decreases in coherence and diversity, reflecting a real trade-off: assigning more ambiguous documents to topics can reduce topic distinctness. The aggregate metrics indicate that topic quality remained reasonably stable, but they do not prove that every reassigned document received a meaningful topic. Human inspection is still necessary.

Whether this trade-off is desirable depends on the application. Exploratory analysis may favor broader corpus coverage, while applications requiring conservative, high-confidence assignments may permit a higher noise target or place greater weight on coherence and diversity. The target and weights are therefore design choices rather than universal constants.

More broadly, BCDNM-HPO makes noise assignment visible as part of model selection. A high noise percentage indicates that many documents fall outside the clusters identified under the current configuration. Instead of accepting that outcome as an incidental consequence of HDBSCAN defaults, practitioners can measure it, specify a target, and search for a configuration that better reflects their analysis goals.

8. Conclusion

BCDNM-HPO provides a lightweight way to tune BERTopic pipelines for improved document utilization. In the IMDb experiment, it reduced the noise percentage from 62.39% to 39.19% and the target deviation from 42.39 to 19.19 percentage points, with modest changes in coherence and diversity. Its main contribution is to make the trade-off among topic quality, coverage, and target noise explicit and searchable. The method does not replace human topic review, but it provides a practical mechanism for selecting BERTopic configurations that leave less of the corpus unexamined.

References

[1] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet Allocation," *Journal of Machine Learning Research*, vol. 3, pp. 993–1022, 2003, doi: 10.1162/jmlr.2003.3.4-5.993.

[2] M. Grootendorst, "BERTopic: Neural Topic Modeling with a Class-Based TF-IDF Procedure," *arXiv preprint arXiv:2203.05794*, 2022, doi: 10.48550/arXiv.2203.05794.

[3] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical Density Based Clustering," *Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017, doi: 10.21105/joss.00205.

[4] Y. Miao, L. Yu, and P. Blunsom, "Neural Variational Inference for Text Processing," in *Proceedings of the 33rd International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 48. 2016, pp. 1727–1736. [Online]. Available: https://proceedings.mlr.press/v48/miao16.html

[5] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," *arXiv preprint arXiv:1802.03426*, 2018, doi: 10.48550/arXiv.1802.03426.

[6] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-Generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631. doi: 10.1145/3292500.3330701.

[7] M. Röder, A. Both, and A. Hinneburg, "Exploring the Space of Topic Coherence Measures," in *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining*, 2015, pp. 399–408. doi: 10.1145/2684822.2685324.

[8] C. J. Nolet *et al.*, "Bringing UMAP Closer to the Speed of Light with GPU Acceleration," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 1, pp. 418–426, 2021, doi: 10.1609/aaai.v35i1.16118.

[9] J. Park, C. Nolet, E. Raff, T. Oates, and A. Naruse, "Massive-Scale Out-of-Core UMAP on the GPU," in *Ninth Conference on Machine Learning and Systems*, 2026. [Online]. Available: https://mlsys.org/virtual/2026/oral/3847

[10] NVIDIA RAPIDS, "Zero Code Change Acceleration." [Online]. Available: https://docs.rapids.ai/api/cuml/stable/cuml-accel/

[11] A. Ding, "video_notebook_for_Minimizing_Noise_Cluster_for_Topic_Modeling.ipynb." [Online]. Available: https://github.com/rapidsai-community/showcase/blob/main/blogs_notebooks/video_notebook_for_Minimizing_Noise_Cluster_for_Topic_Modeling.ipynb

[12] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning Word Vectors for Sentiment Analysis," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 2011, pp. 142–150. [Online]. Available: https://aclanthology.org/P11-1015/

Open Access | https://doi.org/10.25080/eykk7863

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Density Functions and Random Number Generators of Alpha-Stable Distributions

Wael Tabbara[1], **Sharafeddine Sharafeddine**[2,1], **Ahmad El-Hajj**[1], **Jihad Fahs**[1], and **Ibrahim Abou-Faycal**[1]

[1]Electrical & Computer Eng. Department, American University of Beirut, Lebanon, [2]School of Electrical & Computer Engineering, Cornell University, USA

Abstract

Heavy-tailed distributions are increasingly found to better fit empirical data in engineering, finance, physics, network science, and related fields. Among them, alpha-stable distributions play a central role being limiting laws in the generalized central limit theorem: they are expected to be exceptionally good models whenever sums of multiple independent heavy-tailed sources are at play. Despite their theoretical importance, their practical use remains challenging: alpha-stable probability densities generally do not have closed-form expressions, and numerical evaluation and random variate generation can be difficult, especially in the multivariate setting. This paper presents the American University of Beirut Heavy Tails Package (AUB-HTP), a Python package for numerical computation and simulation of alpha-stable distributions. The package provides scalar density evaluation using several complementary methods, including Zolotarev-type integral representations, series formulas, and numerical inversion of characteristic functions. It also provides random variate generation for scalar and multivariate alpha-stable distributions, with support for flexible spectral measures through LePage series representations. Numerical experiments demonstrate that AUB-HTP improves the accuracy, stability, and parameter coverage of existing tools for scalar density computation, while adding new capabilities for multivariate simulation. The package is designed to support reproducible computational work involving heavy-tailed models across a broad range of scientific applications.

Keywords Heavy Tailed Data, Heavy Tails, Alpha Stable, Levy Stable

Published Jul 13, 2026

Correspondence to
Ahmad El-Hajj
aae108@mail.aub.edu

Open Access

Copyright © 2026 Tabbara *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Heavy-tailed distributions arise naturally in many scientific and engineering applications, including communications, signal processing, finance, physics, and network modeling [1], [2], [3], [4], [5]. In such settings, rare but large deviations occur more frequently than predicted by light-tailed models such as the Gaussian distribution. As a result, Gaussian-based modeling can significantly underestimate the probability and impact of extreme events.

Among heavy-tailed models, α-stable distributions occupy a central theoretical position. They generalize the Gaussian distribution through the stability property: sums of independent α-stable random variables remain α-stable, up to location and scale changes. They also arise as limiting distributions for normalized sums of independent random variables under the generalized central limit theorem [6]. These properties make α-stable laws natural candidates for modeling aggregate effects generated by many independent heavy-tailed sources.

A major practical limitation of α-stable distributions is that their probability density functions (PDFs) generally do not admit closed-form expressions, except in the special cases of

the Cauchy and Lévy distributions. Consequently, numerical tools are required for density evaluation, modeling, and simulation. Existing approaches for scalar α-stable random variate generation, such as the Chambers–Mallows–Stuck (CMS) method [7], are well established, but they do not directly address general multivariate α-stable distributions. In the multivariate case, available simulation methods typically focus on special spectral measures, such as isotropic or sub-Gaussian models [4], rather than arbitrary spectral measures.

Available numerical packages for α-stable distributions include J. Royuela-del Val, F. Simmross-Wattenberg, and C. Alberola-López [8] that provides an R and C++ implementation for α-stable random variable generation using a modified version of the CMS method [7], [9]. In M. Teimouri, M. Torshizi, A. Mohammadpour, and S. Nadarajah [10] the authors provide an R package that supports generating an α-stable random variable using the same modified CMS method; a random variable from a truncated α-stable distribution; an α-stable random vector with a sub-Gaussian spectral measure, and other bivariate α-stable random vectors. The work in B. J. Swihart and J. P. Nolan [11] presents another α-stable package which references M. Teimouri, M. Torshizi, A. Mohammadpour, and S. Nadarajah [10] as well. When it comes to density computations, the `libstable` package [8] is based on Zolotarev's integral representation [12] computed using Gauss-Kronrod quadrature over the entire parameter space. The package does the evaluations in C/C++ and provides a MATLAB front-end in addition to an R package. The `alphastable` R package [10] evaluates the density using Skorohod-type asymptotic series [13], [14], [15] wherever those series converge. The package falls back to `stabledist` [16] elsewhere, which computes the density via Zolotarev's integral representation [12] using adaptive quadrature.

This paper presents the American University of Beirut Heavy Tails Package (AUB-HTP)[1], a Python package for computational work with α-stable distributions. The package provides scalar density computation using several numerical methods, including Zolotarev integral representations [12], Skorohod-Pollard-Bergström formulas [13], [14], [15], and direct numerical inversion of the *characteristic function*[2]. These methods provide complementary tradeoffs in terms of accuracy, speed, and robustness across parameter regimes. In addition, AUB-HTP implements random variate generation for scalar and multivariate α-stable distributions. For the multivariate case, the package uses LePage series representations [17], which support flexible spectral measures and perform particularly well in challenging heavy-tailed regimes, including small values of α.

The goal of AUB-HTP is to make α-stable modeling more accessible to computational scientists by providing reliable density evaluation, flexible simulation tools, and reproducible numerical routines within the Python ecosystem. The main contributions of this work are: (i) a unified Python implementation of several scalar α-stable PDF evaluation methods; (ii) a flexible random variate generator for scalar and multivariate α-stable distributions; (iii) support for general spectral measures through LePage series; and (iv) numerical benchmarks comparing accuracy, runtime, and robustness against existing implementations.

Notations

We adopt the following conventions. The vector $\boldsymbol{x} \in \mathbb{R}^d$ represents a column vector and $\boldsymbol{x}^T$ is its transpose. The Euclidean inner product of two vectors $\boldsymbol{x} = (x_1, x_2, \cdots, x_d)^T$ and $\boldsymbol{y} = (y_1, y_2, \cdots, y_d)^T$ is denoted by $\boldsymbol{x}^T\boldsymbol{y} = \langle \boldsymbol{x}, \boldsymbol{y} \rangle = \sum_{i=1}^{d} x_i y_i$ and the L_2 norm of $\boldsymbol{x}$ by $\parallel \boldsymbol{x} \parallel = \sqrt{\langle \boldsymbol{x}, \boldsymbol{x} \rangle}$. The unit sphere in $\mathbb{R}^d$ is denoted by $\mathbb{S}^{d-1} = \{\boldsymbol{x} \in \mathbb{R}^d : \parallel \boldsymbol{x} \parallel = 1\}$. The class of all Borel subsets in a metric space S is represented by $\mathcal{B}(S)$. We use $\boldsymbol{X} \perp\!\!\!\perp \boldsymbol{Y}$ to indicate that the two random vectors $\boldsymbol{X}$ and $\boldsymbol{Y}$ are independent. When a random vector $\boldsymbol{X}$ is distributed

[1] Source repository: https://github.com/AUB-HTP/AUB-HTP. Archived releases: https://pypi.org/project/aub-htp/.

[2] The characteristic function of a random variable X is given by the expected value of e^{itX}, $\phi_X(u) = \mathbb{E}[e^{iuX}]$. It uniquely determines the distribution of X, and it always exists.

uniformly on the unit sphere, we write $\boldsymbol{X} \sim \mathcal{U}(\mathbb{S}^{d-1})$. For a random variable X that is Gamma distributed, we write $X \sim \mathrm{Gamma}(\alpha, \beta)$, and we use the following parameterization for the PDF

$$f_X(x) = \begin{cases} \frac{1}{\Gamma(\alpha)\beta^\alpha} x^{\alpha-1} e^{-\frac{x}{\beta}} & \text{if } x > 0 \\ 0 & \text{otherwise,} \end{cases}$$

where $\Gamma(\cdot)$ is the Gamma function.

2. Preliminaries on α-Stable Distributions

For a definition of multivariate stable vectors, we refer the reader to G. Samorodnitsky and M. S. Taqqu [18, Definition 2.1.1]. In particular, we are interested in α-stable [3] random vectors.

Definition 2.1.

[18]

Let $0 < \alpha < 2$, then $\boldsymbol{X} = (X_1, X_2, \cdots, X_d)$ is an α-stable random vector in $\mathbb{R}^d$ if and only if there exists a finite measure Λ on $\mathbb{S}^{d-1}$ called the "spectral" measure and a vector $\boldsymbol{\mu}^0$ in $\mathbb{R}^d$ such that the characteristic function of $\boldsymbol{X}$ is given by

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = \mathbb{E}[\exp\{\mathrm{i}\langle \boldsymbol{t}, \boldsymbol{X}\rangle\}] = \exp\left\{ -\int_{\mathbb{S}^{d-1}} \psi_\alpha(\langle \boldsymbol{t}, \boldsymbol{s}\rangle)\, \Lambda(d\boldsymbol{s}) + \mathrm{i}\langle \boldsymbol{t}, \boldsymbol{\mu}^0\rangle \right\}, \tag{1}$$

and where

$$\psi_\alpha(u) = \begin{cases} |u|^\alpha \left[1 - \mathrm{i}\,\mathrm{sign}(u) \tan\frac{\pi\alpha}{2}\right] \alpha \neq 1 \\ |u| \left[1 + \mathrm{i}\frac{2}{\pi}\,\mathrm{sign}(u) \ln|u|\right] \quad \alpha = 1. \end{cases}$$

In the scalar case ($d = 1$) –and whenever $0 < \alpha < 2$, the spectral measure boils down to two mass points on $s = \pm 1$, and the characteristic function in (1) is expressible in terms of 4 parameters: the stability index $0 < \alpha < 2$, skewness $-1 \leq \beta \leq 1$, scale $\gamma > 0$, and location $\delta \in \mathbb{R}$. In this case, we denote an α-stable random variable X using the same notation as in Nolan [9], $X \sim \mathcal{S}(\alpha, \beta, \gamma, \delta; k)$ where k is the parameterization index (or type). We will use the symbols $\mathcal{S}_0$ and $\mathcal{S}_1$ to refer to the two common type-0 and type-1 parameterizations, respectively. In the multivariate case, we denote a d-dimensional non-degenerate α-stable random vector $\boldsymbol{X}$ with stability index α, spectral measure Λ and shift vector $\boldsymbol{\mu}^0$, by $\boldsymbol{X} \sim \boldsymbol{S}_\alpha(\Lambda, d, \boldsymbol{\mu}^0)$. Note that Definition 2.1 corresponds to the generalization of the scalar case under the parameterization $k = 1$.

Parameterizations of scalar α-stable distributions

The two parameterizations, 0 and 1 are denoted by $\mathcal{S}(\alpha, \beta, \gamma, \delta; 0)$ and $\mathcal{S}(\alpha, \beta, \gamma, \delta; 1)$ respectively and differ primarily in how they define the location (shift) of the distribution, particularly near $\alpha = 1$. For a given α and β, let Z be a "standard" α-stable variable corresponding to $\gamma = 1$ and $\delta = 0$.

Table 1. *Parametrization 0 and Parametrization 1.*

$\boldsymbol{X} \sim \mathcal{S}(\alpha, \beta, \gamma, \delta; 0)$	**Characteristic Function:** $\mathbb{E}[e^{i\boldsymbol{u}\boldsymbol{X}}]$	$\boldsymbol{\alpha}$
$X \overset{d}{=} \gamma(Z - \beta\tan(\frac{\pi\alpha}{2})) + \delta$	$\exp(-\mid \gamma u \mid^\alpha \left[1 + \mathrm{i}\beta\tan(\frac{\pi\alpha}{2})\,\mathrm{sign}(u)(\mid \gamma u \mid^{1-\alpha} - 1)\right] + \mathrm{i}\delta u)$	$\alpha \neq 1$
$X \overset{d}{=} \gamma Z + \delta$	$\exp(-\mid \gamma u \mid \left[1 + \mathrm{i}\beta\frac{2}{\pi}\,\mathrm{sign}(u)\log\mid \gamma u \mid\right] + \mathrm{i}\delta u)$	$\alpha = 1$

[3] The term α-stable is used to refer to the stable family excluding the Gaussian law.

$X \sim \mathcal{S}(\alpha, \beta, \gamma, \delta; 1)$	**Characteristic Function:** $\mathbb{E}[e^{iuX}]$	α
$X \stackrel{d}{=} \gamma Z + \delta$	$\exp(-\mid \gamma u \mid^{\alpha} [1 - \mathrm{i}\beta \tan(\frac{\pi\alpha}{2}) \operatorname{sign}(u)] + \mathrm{i}\delta u)$	$\alpha \neq 1$
$X \stackrel{d}{=} \gamma Z + (\delta + \beta\frac{2}{\pi}\gamma \log \gamma)$	$\exp(-\mid \gamma u \mid [1 + \mathrm{i}\beta\frac{2}{\pi} \operatorname{sign}(u) \log\mid u \mid] + \mathrm{i}\delta u)$	$\alpha = 1$

Parameterization 0 has the advantage of avoiding discontinuities near $\alpha = 1$ by adjusting the shift behavior and is often preferred for numerical stability [9]. On the other hand, explicit adjustment to the location is done when $\alpha = 1$ for parameterization 1.

Note that the choice of parameterization directly affects the shift and symmetry of the distribution, especially when $\beta \neq 0$ and α is close to 1. In both forms, the distribution can be standardized by setting $\gamma = 1$ and $\delta = 0$, in which case we use the abbreviations $\mathcal{S}(\alpha, \beta; 0)$ and $\mathcal{S}(\alpha, \beta; 1)$ respectively.

In the univariate case, our package can simulate both parameterizations $k = 0, 1$.

3. Scalar Density Evaluation

The AUB-HTP package provides numerical evaluation of scalar α-stable probability density functions through a SciPy-like interface. This component complements the random number generator: the generator produces samples, while the density module provides the reference curve used for model verification, visualization, and numerical comparison.

Except for Cauchy and Levy laws, α-stable distributions do not have a closed-form density, which must therefore be computed using numerical inversion or analytic representations that are valid in specific parameter regions.

Towards fulfilling our main objective of reducing expensive quadrature calls while preserving accuracy across a wide range of α, β, and sample values, the density evaluator combines three methods:

- direct inversion of Nolan's $\mathcal{S}_1$ characteristic function near the mode [9]
- Zolotarev's integral representation in the middle range [12]
- Skorohod-Pollard-Bergström's series expansions in the tails [13], [14], [19].

3.1. Input standardization

All density calls are first mapped to a standardized variable allowing the internal routines to work on standardized densities while still supporting user-facing location and scale parameters. More specifically, given an input sample value x the code evaluates

$$z = \frac{x - \text{shift}}{\gamma}, \tag{2}$$

then rescales the final density by $1/\gamma$. For the $\mathcal{S}_1$ parameterization for example, the shift is given by J. P. Nolan [9, equation 1.5]

$$\text{shift} = \begin{cases} \delta, & \alpha \neq 1 \\ \delta + \beta\frac{2}{\pi}\gamma \log \gamma & \alpha = 1, \end{cases}$$

as can be deduced from Table 1.

Additionally, if $f_{\alpha,\beta}(x)$ is the PDF of a normalized α-stable random variable, for negative evaluation points the code uses the symmetry relation J. P. Nolan [9, proposition 1.1]

$$f_{\alpha,\beta}(-x) = f_{\alpha,-\beta}(x),$$

and most formulas only need to be evaluated on the positive half-line.

3.2. Hybrid evaluation strategy

The main density function acts as a dispatcher. It divides the input array into regions and applies the method that is most accurate for each region.

- For small $|\ z\ |$ (see equation (2)), the package uses direct inversion of the characteristic function. While this method is slower than the series expansions, it is stable near the mode where tail formulas are not accurate.
- For intermediate values of $|\ z\ |$, the package uses Zolotarev's integral representation. This provides a useful compromise between direct inversion and asymptotic series methods.
- For large $|\ z\ |$, the package uses Skorohod-Pollard-Bergström series expansions. These formulas are fast in the tails and avoid repeated numerical integration.

The switching points between methods are not fixed constants. They are instead selected from precomputed cutoff functions depending on (α, β). These cutoffs were obtained using grid-based comparison with `scipy.stats.levy_stable.pdf`. At runtime, the code queries the corresponding cutoff and applies Boolean masks to route each input point to the selected method.

Density formulas

• For direct inversion in the $\mathcal{S}_1$ parameterization [9], the density is computed from the characteristic function. The standardized density is

$$f_{\alpha,\beta}(x) = \frac{1}{\pi}\int_0^\infty e^{-t^\alpha}\cos\Big(xt - \beta\tan\Big(\frac{\pi\alpha}{2}\Big)t^\alpha\Big)\,dt \quad \alpha \neq 1$$

$$f_{1,\beta}(x) = \frac{1}{\pi}\int_0^\infty e^{-t}\cos\Big(xt + \frac{2}{\pi}\beta t\log t\Big)\,dt \qquad \alpha = 1,\ \text{with the logarithmic correction.}$$

These integrals are evaluated with `scipy.integrate.quad_vec`.

• For the middle region, the code uses Zolotarev's integral representation [12]. For $\alpha \neq 1$ and $x > 0$,

$$f_{\alpha,\beta}(x) = \frac{\alpha x^{\frac{1}{\alpha-1}}}{\pi\ |\ \alpha-1\ |}\int_{-\theta_0}^{\pi/2} V(\theta)\exp\big[-x^{\frac{\alpha}{\alpha-1}}V(\theta)\big]\,d\theta,$$

where

$$-\theta_0 = \frac{1}{\alpha}\arctan\Big(\beta\tan\Big(\frac{\pi\alpha}{2}\Big)\Big),$$

and where $V(\theta)$ is evaluated following Zolotarev's formula [12]. This integral is computed using the function `scipy.integrate.quad`.

• For the tail region, we use Skorohod-type series expansions [13], [14], [19] of the form

$$f_{\alpha,\beta}(x) \approx \frac{1}{\pi x}\sum_{n=1}^{N} a_n x^{-\alpha n},$$

where

$$a_n = \frac{(-1)^{n-1}\Gamma(n\alpha+1)}{n!}\Big(1+\beta^2\tan^2\Big(\frac{\pi\alpha}{2}\Big)\Big)^{n/2}\sin\Big[n\Big(-\frac{\pi\alpha}{2} + \arctan\Big(\beta\tan\Big(\frac{\pi\alpha}{2}\Big)\Big)\Big)\Big].$$

The number N of terms used to calculate the series is appropriately chosen depending on the parameter range. In the current package, the $\alpha < 1$ branch uses a larger truncation level, while the $\alpha > 1$ branch requires fewer terms in the tested regimes. In some cases, minor spikes occurred at disjoint points away from the mode due to numerical irregularities. These spikes were detected by checking locations where monotonicity is violated. This is

resolved by applying –after the density is computed, a smoothing step enforcing monotonicity of $f_{\alpha,\beta}$ on either side of the mode.

3.3. Validation

The density evaluator was tested against `scipy.stats.levy_stable.pdf` on grids of α, β, and x. The near-mode tests used $x \in [-10, 10]$, while the tail tests examined values up to $x = 100$.

The metrics listed in Table 2 were used to compare the AUB-HTP PDF generator with that of SciPy.

Table 2. *Error and performance metrics comparing $\hat{f}$ against the SciPy reference f_{SciPy} over $\{x_i\}_{i=1}^{n}$, with $\varepsilon > 0$ guarding against division by zero.*

Metric	**Formula**
Maximum absolute error	$\max_{1\le i\le n} \mid \hat{f}(x_i) - f_{\text{SciPy}}(x_i) \mid$
Maximum relative error	$\max_{1\le i\le n} \frac{\mid \hat{f}(x_i) - f_{\text{SciPy}}(x_i) \mid}{\max(\mid f_{\text{SciPy}}(x_i) \mid, \varepsilon)}$
Mean absolute error	$\frac{1}{n}\sum_{i=1}^{n} \mid \hat{f}(x_i) - f_{\text{SciPy}}(x_i) \mid$
Runtime speedup factor against SciPy	$\frac{T_{\text{SciPy}}}{T_{\text{AUB-HTP}}}$

The tests show close agreement across most of the parameter space, but they also reveal regions where the SciPy reference exhibits irregular numerical behavior. The strongest agreement occurs away from the narrow region around $\alpha = 1$, where changes in the parameterization and the presence of logarithmic terms make density evaluation more sensitive. In this region, visual diagnostics are important because the relative difference may become large when the reference density is close to zero or numerically unstable. Figure 1 illustrates this behavior near $\alpha = 1$ using the maximum relative difference measured around the mode. Away from $\alpha = 1$, Figure 1 shows agreement with the SciPy reference. Results for the remaining error metrics, parameter ranges, and tail evaluations, together with the complete benchmark data, are available in the Github repository[4].

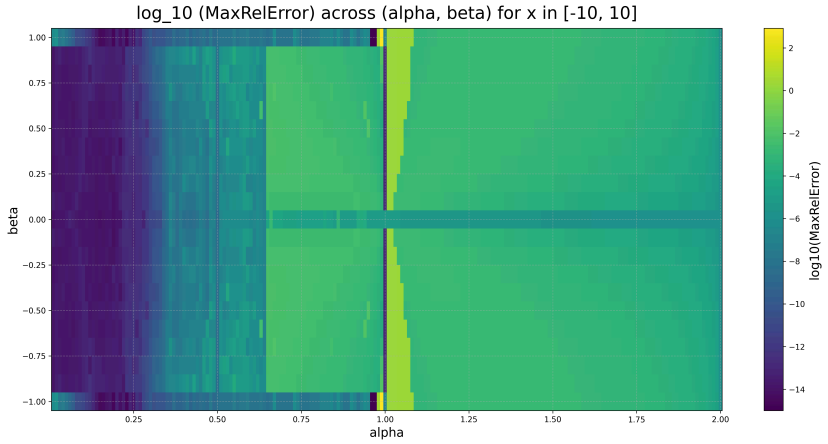


Figure 1. *Maximum relative difference across (α, β) near the mode. Larger values near $\alpha = 1$ mark the most sensitive numerical region.*

3.4. Discrepancies in the SciPy reference

While `scipy.stats.levy_stable.pdf` [20] is used as the main numerical reference, we observed that it has some discrepancies for certain parameter values. These discrepancies

[4] https://github.com/AUB-HTP/AUB-HTP/tree/main/benchmarks

appear mostly near the sensitive region around $\alpha = 1$, and they can create misleading relative difference values.

The main issue is that the SciPy PDF sometimes shows unusual zero or near-zero regions in some places while the densities are known to be smooth. Since the maximum relative difference contains a division by the reference value, even a small disagreement in such regions can produce a very large relative difference. Therefore, some of the large relative differences in the benchmark tables do not necessarily mean that our PDF generator is inaccurate. Instead, they are likely due to the instability or discontinuity in the reference PDF.

To validate this, we generated visual diagnostic plots comparing the AUB-HTP with the `levy_stable` PDF values. Figure 2 shows several cases where the SciPy reference generates unexpected zero windows or sharp changes, while the PDF generated using our package remains smooth.

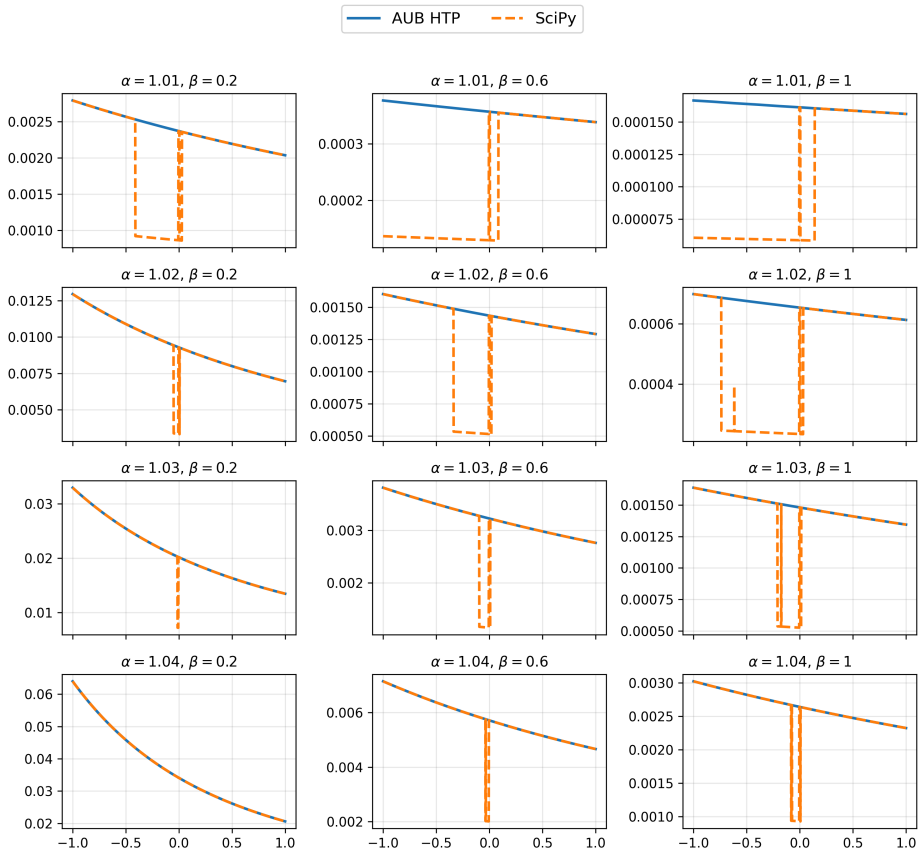


Figure 2. *Visual comparison between AUB-HTP PDF generator and `scipy.stats.levy_stable.pdf`. The plots zoom into regions where the SciPy reference becomes numerically zero or behaves irregularly. These regions explain why some relative difference values become large even when our generated density remains smooth.*

The heatmaps are useful for locating sensitive parameter regions, but the diagnostic plots were needed to understand whether the difference is primarily attributed to our method or the numerical reference.

3.5. Runtime

The adopted hybrid strategy reduces runtime by avoiding direct quadrature when a faster formula is accurate enough. The largest gains occur in the tail region, where series expansions replace repeated numerical integration.

In the benchmark runs, the median speedup was approximately $58\times$, with larger speedups in tail-heavy evaluations. These gains come from three design choices:

- vectorized array operations
- region-specific method dispatch
- reuse of precomputed cutoff functions.

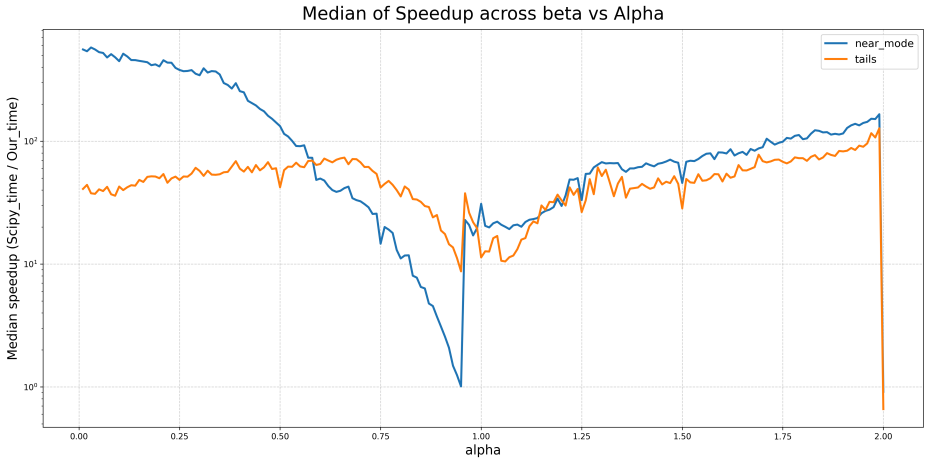


Figure 3. *Median runtime speedup relative to* `scipy.stats.levy_stable.pdf`*. The largest gains occur where the series expansions are selected.*

The speedup plot in Figure 3 shows that our method is strictly faster than `scipy.stats.levy_stable.pdf` for almost all tested values of α. Note that the vertical axis is on a logarithmic scale and values between 10 and 100 represent large runtime improvements. For small values of α, especially in the near-mode range, the speedup is very high. This is because the method selection avoids unnecessary numerical integration and uses the formulas that are faster in those regions. In the tail range, the speedup is also consistent for most values of α, since the Skorohod-Pollard-Bergström formulas replace repeated quadrature calls. The smallest speedups occur near $\alpha = 1$ which is expected since this region is numerically delicate. The formulas change around $\alpha = 1$, and the characteristic function contains a logarithmic correction term. The package therefore relies more on direct inversion in this region, which is slower than the series-based methods.

4. Generating α-Stable Random Vectors

In the univariate case, the package implements a numerically stable version of the Chambers, Mallows, and Stuck method which is presented in J. P. Nolan [9], R. Weron [21], and it is the same method as in J. Royuela-del Val, F. Simmross-Wattenberg, and C. Alberola-López [8], M. Teimouri, M. Torshizi, A. Mohammadpour, and S. Nadarajah [10]. This method is commonly considered to be the fastest and most accurate [22]. In the multivariate setup, the proposed random number generator is based on a theoretical foundation that we state next.

4.1. The LePage series

The LePage series [18] was first introduced in R. LePage, M. Woodroofe, and J. Zinn [17], and it was suggested as a simulation method in R. Weron [22]. This series allows for simulating a

multivariate stable vector with a stability index α and a given spectral measure through the use of the following theorem V. Bentkus, A. Juozulynas, and V. Paulauskas [23, Theorem 4].

Theorem 1.

Let $\boldsymbol{V_1}, \boldsymbol{V_2}, \cdots$ be independent and identically distributed (i.i.d) random vectors in $\mathbb{R}^d$ that are drawn according to a spectral measure Λ where $\Lambda(\mathbb{S}^{d-1}) > 0$, i.e.,

$$\Pr(\boldsymbol{V} \in A) = \frac{\Lambda(\mathbb{S}^{d-1} \cap A)}{\Lambda(\mathbb{S}^{d-1})} \quad \forall\ A \in \mathcal{B}(\mathbb{R}^d).$$

Let $E_1, E_2, \cdots$ be i.i.d. exponential random variables with mean 1, independent of $\boldsymbol{V}_1, \boldsymbol{V}_2, \cdots$, and define

$$\Gamma_k = \sum_{i=1}^{k} E_i \qquad \boldsymbol{X}_n = c(\alpha) \sum_{k=1}^{n} \Gamma_k^{-1/\alpha}\, \boldsymbol{V}_k \tag{3}$$

where

$$c(\alpha) = \left(\frac{\kappa}{\Lambda(\mathbb{S}^{d-1})}\right)^{-\frac{1}{\alpha}} \qquad \& \qquad \kappa = \begin{cases} \frac{\Gamma(2-\alpha)\cos(\frac{\pi\alpha}{2})}{1-\alpha} & \alpha \neq 1 \\ \frac{\pi}{2} & \alpha = 1. \end{cases}.$$

- For $0 < \alpha < 1$, $\boldsymbol{X}_n$ converges in total variation to a stable vector $\boldsymbol{X}$ with spectral measure Λ and stability index α.
- For $1 \leq \alpha < 2$, whenever

$$\int_{\mathbb{S}^{d-1}} s\Lambda(ds) = 0, \tag{4}$$

the random vector $\boldsymbol{X}_n$ converges in total variation to a stable vector $\boldsymbol{X}$ with spectral measure Λ and stability index α.

Whenever there is convergence to $\boldsymbol{X}$, we write

$$\boldsymbol{X} = c(\alpha) \sum_{k=1}^{\infty} \Gamma_k^{-1/\alpha}\, \boldsymbol{V}_k.$$

Theorem 1 indicates that sampling a multivariate stable vector can be done by computing the truncated series as in equation (3). Since the vector $\boldsymbol{X}$ resulting from the series has a zero shift vector $\boldsymbol{\mu}^0 = \boldsymbol{0}$ V. Bentkus, A. Juozulynas, and V. Paulauskas [23, Remark 1], we simply add $\boldsymbol{\mu}^0$ after sampling $\boldsymbol{X}_n$ using the LePage series, when needed. The analysis of the truncation error is deferred to subsection A.1 in the Appendix.

In the following section, we present algorithms to sample according to well-known spectral measures, namely: Isotropic, Sub-Gaussian, Discrete, and Mixed spectral measures.

4.2. Common spectral measures

4.2.1. *Discrete spectral measures*

In the case of a discrete spectral measure with M mass points, we store the point masses in an array $[\boldsymbol{v}_1, \boldsymbol{v}_2, \cdots, \boldsymbol{v}_M]$ and draw randomly with the probability of each vector $\boldsymbol{v}_i$ being

$$\Pr(\boldsymbol{v}_i) = \frac{\Lambda(\boldsymbol{v}_i)}{\sum_{\ell=1}^{M} \Lambda(\boldsymbol{v}_\ell)}.$$

4.2.2. *Isotropic spectral measures*

For an isotropic spectral measure, we sample the vectors $\{\boldsymbol{V}_k\}_{1 \leq k \leq n}$ uniformly on $\mathbb{S}^{d-1}$. To this end, several algorithms are available and we refer the reader to S. Schnabel and W.

Janke [24] where various such algorithms are discussed. In our computations, we adopt the method presented in M. E. Muller [25] due to its simplicity and computational efficiency. Our package sets the spectral measure of the sphere to 1, and then scales the truncated LePage series by an appropriate factor q to adjust for the desired scale γ given by the user: When the spectral measure of the sphere is 1, it can be verified that the characteristic function is given by

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = \exp\left\{ -\frac{\Gamma\left(\frac{d}{2}\right)}{\sqrt{\pi}} \frac{\Gamma\left(\frac{\alpha+1}{2}\right)}{\Gamma\left(\frac{\alpha+d}{2}\right)} \parallel \boldsymbol{t} \parallel^{\alpha} \right\}.$$

To achieve a scale γ, we therefore multiply the truncated series by

$$q = \gamma \left[\frac{\Gamma\left(\frac{d}{2}\right)}{\sqrt{\pi}} \frac{\Gamma\left(\frac{\alpha+1}{2}\right)}{\Gamma\left(\frac{\alpha+d}{2}\right)} \right]^{-\frac{1}{\alpha}}.$$

4.2.3. *Sub-Gaussian (elliptical) spectral measures*

A sub-Gaussian random vector is characterized by a positive-definite $d \times d$ shape matrix Σ. If $\boldsymbol{X}$ is a unit-scale isotropic multivariate stable random vector with stability index α then $\boldsymbol{Y} = \Sigma^{\frac{1}{2}}\boldsymbol{X}$ is a sub-Gaussian random vector with shape matrix Σ where $\Sigma^{\frac{1}{2}}$ is the Cholesky decomposition of Σ [4].

One way to sample sub-Gaussian random vectors is by using the definition presented in G. Samorodnitsky and M. S. Taqqu [18, Definition 2.5.1]. This is adopted whenever the α-stable vector is purely sub-Gaussian. Whenever the sub-Gaussian spectral measure is an element of a mixture (see section 4.2.4 for mixed spectral measures), then the approach in G. Samorodnitsky and M. S. Taqqu [18, Definition 2.5.1] is no longer suitable. Instead, we devise a new approach that is based on the following result: if $\boldsymbol{X}_n$ is the n-terms LePage series of an isotropic vector $\boldsymbol{X}$ with unit scale, then, by Theorem 1, $\boldsymbol{X}_n$ converges in total variation to $\boldsymbol{X}$, and

$$\boldsymbol{Y}_n = \Sigma^{\frac{1}{2}}\boldsymbol{X}_n = c(\alpha)\Sigma^{\frac{1}{2}} \sum_{k=1}^{n} \Gamma_k^{-\frac{1}{\alpha}} \boldsymbol{U}_k = c(\alpha) \sum_{k=1}^{n} \Gamma_k^{-\frac{1}{\alpha}} \boldsymbol{V}_k \qquad \text{where } \boldsymbol{V}_k = \Sigma^{\frac{1}{2}}\boldsymbol{U}_k, \tag{5}$$

converges in total variation to $\boldsymbol{Y} = \Sigma^{\frac{1}{2}}\boldsymbol{X}$, which is the desired sub-Gaussian random vector. Hence, to sample the vectors $\boldsymbol{V}_k$ for the sub-Gaussian case, the software samples $\boldsymbol{U}_k$ uniformly distributed on the unit sphere and multiplies them by $\Sigma^{\frac{1}{2}}$.

4.2.4. *Mixed spectral measures*

Now consider the scenario where the spectral measure at hand is a mixture of L different spectral measures: $\Lambda_1, \Lambda_2, \cdots, \Lambda_L$. If $w_1, w_2, \cdots, w_L$ are their respective associated weights, the mixed spectral measure Λ_{mix} is given by

$$\Lambda_{\text{mix}} = \sum_{i=1}^{L} w_i \Lambda_i.$$

To sample from this spectral measure, we proceed as follows.

- For each weight w_i we associate its corresponding probability as

$$p_i = \frac{w_i}{\sum_{\ell=1}^{L} w_\ell} \qquad i = 1, \cdots, L.$$

Then, we sort the probabilities in non-decreasing order that we assume –without loss of generality– to be $p_1, \cdots, p_L$ hereafter.

- We draw $U \sim \mathcal{U}[0, 1]$,
  - If $U \leq p_1$ sample from Λ_1,

- else if $U \leq p_1 + p_2$, sample from Λ_2,
- else iterate on the remaining spectral measures.

This algorithm achieves a sampling from the mixture of $\Lambda_1, \cdots, \Lambda_L$ with probabilities $p_1, \cdots, p_L$ respectively.

5. Implementation

Whenever the LePage series is used, the package chooses the number of terms to achieve an MSE less than 0.01 using Theorem 2 in the Appendix in section A.1. This number is capped at 50,000 terms and a warning is issued whenever this cap is reached. After varying the values of α and the total mass of the unit sphere, a suitable value for the mean square error threshold, one that leads to sufficiently accurate results with reasonable consumption of time and memory is 0.01.

5.1. Univariate random number generator

In the univariate case, the software samples $X \sim \mathcal{S}(\alpha, \beta, \gamma, \delta; k)$, and the type 1 parameterization is used by default. A sample call for both parametrizations is shown in the following block of code.

```
import aub_htp as ht

alpha = 1
beta = 0
loc = 0
scale = 1
n = 500000
samples = ht.alpha_stable.rvs(alpha=alpha, beta=beta, loc=loc, scale=scale, size=n)
# For the type 0 parametrization use:
# ht.alpha_stable.with_parameterization("S0")
# samples = ht.alpha_stable.rvs(alpha=alpha, beta=beta, loc=loc, scale=scale, size=n)
```

5.2. Multivariate random number generator

Isotropic α-stable random vector:
An isotropic α-stable random vector with characteristic function

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = e^{-\gamma^{\alpha} \| \boldsymbol{t} \|^{\alpha} + \mathrm{i}\boldsymbol{t}^T \boldsymbol{\mu}^0},$$

can be generated using the AUB-HTP. A sample syntax –commented out– is shown below in the section of the Elliptic α-stable random vector.

Elliptic α-stable random vector:
An elliptic α-stable random vector with shape matrix Σ with the following characteristic function

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = e^{-(\boldsymbol{t}^T \Sigma \boldsymbol{t})^{\frac{\alpha}{2}} + \mathrm{i}\boldsymbol{t}^T \boldsymbol{\mu}^0},$$

can also be generated using AUB-HTP. We note that the elliptic sampler takes the total mass of the unit sphere as input. If the total mass of the sphere is not specified, the software estimates it using Monte Carlo simulations (with 10^6 samples) as follows:

$$\Lambda(\mathbb{S}^{d-1}) = \mathbb{E}\left[\| \Sigma^{\frac{1}{2}} \boldsymbol{U} \|^{\alpha}\right] \approx \frac{1}{n} \sum_{i=1}^{n} \| \Sigma^{\frac{1}{2}} \boldsymbol{U}_i \|^{\alpha}, \tag{6}$$

where the equality is due to G. Samorodnitsky and M. S. Taqqu [18, proposition 2.5.8], and where $\boldsymbol{U}_1, \boldsymbol{U}_2, \ldots \boldsymbol{U}_n$ are i.i.d. samples according to $\boldsymbol{U} \sim \mathcal{U}(\mathbb{S}^{d-1})$. Equation (6) is justified by the application of the Weak Law of Large Numbers (WLLN) because the random variable $\| \Sigma^{\frac{1}{2}} \boldsymbol{U} \|^{\alpha}$ has a finite second moment. Indeed,

$$\begin{aligned}\mathbb{E}\left[\| \Sigma^{\frac{1}{2}}\boldsymbol{U} \|^{2\alpha}\right] &= \mathbb{E}\left[\left(\boldsymbol{U}^T\Sigma^{\frac{1}{2}T}\Sigma^{\frac{1}{2}}\boldsymbol{U}\right)^{\alpha}\right] \\ &\overset{(a)}{=} \mathbb{E}\left[\left(\boldsymbol{U}^T\Sigma^T\boldsymbol{U}\right)^{\alpha}\right] \\ &\overset{(b)}{=} \mathbb{E}[\| \boldsymbol{U} \|_{\Sigma}^{2\alpha}] \\ &\overset{(c)}{\leq} \lambda_{\max}^{\alpha}(\Sigma)\mathbb{E}[\| \boldsymbol{U} \|^{2\alpha}] = \lambda_{\max}^{\alpha}(\Sigma) < \infty,\end{aligned} \tag{7}$$

where equality (a) is due to the Cholesky decomposition definition and where in inequality (b) we used the Σ-weighted norm $\| \boldsymbol{x} \|_{\Sigma}^2 \triangleq \boldsymbol{x}^T\Sigma\boldsymbol{x}$. Finally, the inequality in (c) is due to the fact that $\| \boldsymbol{x} \|_{\Sigma}^2 \leq \lambda_{\max}(\Sigma) \| \boldsymbol{x} \|^2$ where $\lambda_{\max}(\Sigma)$ is the largest eigenvalue of Σ, equal to the norm of matrix Σ [26].

Finally, we note that the MSE resulting from estimating the total mass of the sphere using equation (6) is given by H. Niederreiter [27, Theorem 1.1].

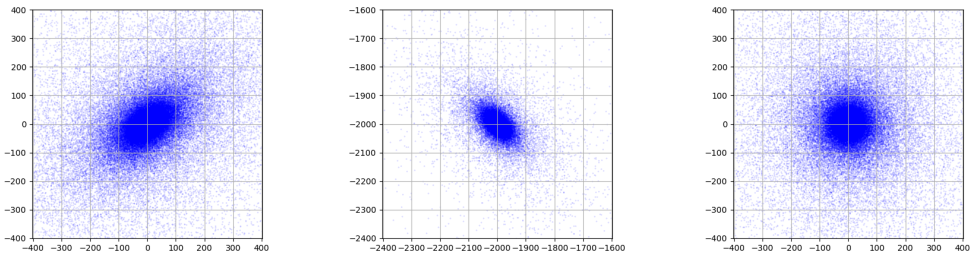
An example code for both isotropic and elliptic cases is shown as follows.

```
import aub_htp as ht
from aub_htp.random import IsotropicSampler, EllipticSampler

alpha = 0.5
gamma = 1
shift = [0, 0]
d = 2
n = 500000
sigma1 = [[2, 0.8], [0.8, 1.5]]
sampler = EllipticSampler(number_of_dimensions=2, alpha=alpha, sigma=sigma1)
samples = ht.multivariate_alpha_stable.rvs(alpha=alpha, spectral_measure_sampler=sampler,
shift=shift, size=n)
# For an Isotropic spectral measure use:
# sampler = IsotropicSampler(number_of_dimensions=d, alpha=alpha, gamma=gamma)
# samples = ht.multivariate_alpha_stable.rvs(alpha=alpha, spectral_measure_sampler=sampler,
shift=shift, size=n)
```

Figures 4a and 4b show the scatter plots of the elliptical α-stable random vector, where we use the following shape matrices

$$\Sigma_1 = \begin{pmatrix} 2 & 0.8 \\ 0.8 & 1.5 \end{pmatrix} \qquad \Sigma_2 = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

Figure 4c shows the scatterplot of an isotropic vector associated with the commented code.



(a) $\alpha = 0.5, \Sigma_1, \boldsymbol{\mu}^{0T} = (0, 0)$ (b) $\alpha = 1.1, \Sigma_2, \boldsymbol{\mu}^{0T} = -(2000, 2000)$ (c) $\alpha = 0.5, \gamma = 1, \boldsymbol{\mu}^{0T} = (0, 0)$

Figure 4. *Scatter plots each containing 5×10^5 points of elliptic contoured and isotropic α-stable distributions; the plots have been truncated to a range of 400 points within the shift vector.*

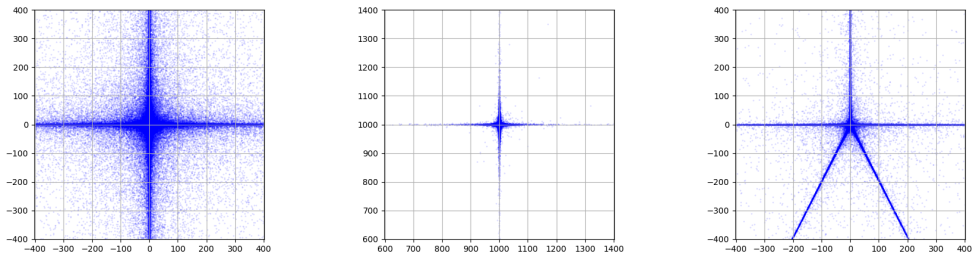
Discrete spectral measures:
For an α-stable random vector that has a discrete spectral measure with point masses at positions $\boldsymbol{v}_1, \cdots, \boldsymbol{v}_M$ and corresponding weights $w_1, \cdots, w_M$, the corresponding characteristic function is given by

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = \exp\left\{ -\sum_{\ell=1}^{M} w_\ell \psi_\alpha(\boldsymbol{t}^T \boldsymbol{v}_\ell) + \mathrm{i}\boldsymbol{t}^T \boldsymbol{\mu}^0 \right\}.$$

Such a vector can be generated as follows:

```
import aub_htp as ht
from aub_htp.random import DiscreteSampler

alpha = 0.2
shift = [0, 0]
d = 2
n = 500000
positions = [[1, 0], [0, 1], [-1, 0], [0, -1]]
weights = [1, 1, 1, 1]
sampler = DiscreteSampler(alpha=alpha, positions=positions, weights=weights)
samples = ht.multivariate_alpha_stable.rvs(alpha=alpha, spectral_measure_sampler=sampler,
shift=shift, size=n)
```

with corresponding scatter plots shown in Figure 5.



(a) Masses $= [1, 1, 1, 1]$, weights $= [(1, 0), (0, 1), (-1, 0), (0, -1)]$, $\alpha = 0.2$, $\boldsymbol{\mu}^{0T} = (0, 0)$.

(b) Masses $= \left[\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\right]$, weights $= [(1, 0), (0, 1), (-1, 0), (0, -1)]$, $\alpha = 1.2$, $\boldsymbol{\mu}^{0T} = (1000, 1000)$.

(c) Masses $= \left[\frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}\right]$, weights $= [(1, 0), (0, 1), (-1, 0), (\cos(\theta), -\sin(\theta)), (\cos(\pi - \theta), -\sin(\pi - \theta))]$, $\theta = \frac{13\pi}{20}$, $\alpha = 0.1$, $\boldsymbol{\mu}^{0T} = (0, 0)$.

Figure 5. *Scatter plots each containing 5×10^5 points of α-stable distributions with a discrete spectral measure. The plots have been truncated to a range of 400 points within the shift vector.*

Mixed spectral measure:
For a mixed spectral measure, let $\Lambda_1, \cdots, \Lambda_L$ be L spectral measures with associated weights $w_1, \cdots, w_L$, and consider their mixture $\Lambda_{\text{mix}} = \sum_{i=1}^{L} w_i \Lambda_i$. To sample according to this spectral measure, we proceed as follows:

```
import aub_htp as ht
from aub_htp.random import DiscreteSampler, IsotropicSampler, MixedSampler

alpha = 0.3
shift = [1000, 1000]
d = 2
n = 500000
gamma = 1

# Discrete Spectral Measure
positions = [[1, 0], [0, 1], [-1, 0], [0, -1]]
weights = [1, 1, 1, 1]
sampler1 = DiscreteSampler(alpha=alpha, positions=positions, weights=weights)

# Isotropic Spectral Measure
sampler2 = IsotropicSampler(number_of_dimensions=d, alpha=alpha, gamma=gamma)

# Spectral Measures and Their Associated Weights
spectral_measures = [sampler1, sampler2]
measure_weights = [0.5, 0.5]

mixed_sampler = MixedSampler(
    spectral_measures=spectral_measures, weights=measure_weights
)
samples = ht.multivariate_alpha_stable.rvs(
    alpha=alpha, spectral_measure_sampler=mixed_sampler, shift=shift, size=n
)
```

where `spectral_measures` is a list containing the spectral measures that we are mixing, and `measure_weights` contains the associated weight of each measure. The resulting scatter plot is shown in Figure 6a.

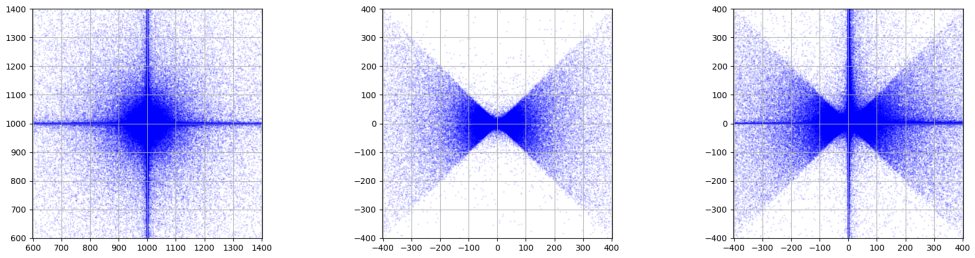
Custom spectral measures:
The common spectral measures presented above are built into AUB-HTP. If the user wants to sample from an α-stable random vector whose spectral measure is not built-in, this is made possible using a systematic procedure outlined in the following example:

- Say the user is interested in sampling a multivariate α-stable random vector in dimension $d = 2$ with $\alpha = 0.3, \boldsymbol{\mu}^0 = \mathbf{0}$, and whose spectral measure is uniform over the arcs $[-\frac{\pi}{4}, \frac{\pi}{4}]$ and $[\frac{3\pi}{4}, \frac{5\pi}{4}]$, that is, with probability 0.5, we draw a point uniformly on the arc $[\frac{3\pi}{4}, \frac{5\pi}{4}]$, and with probability 0.5, we draw a point uniformly on the arc $[-\frac{\pi}{4}, \frac{\pi}{4}]$.
- We assign equal probabilities to both arcs and the spectral measure of each arc is half the total measure of the sphere. Generally, the probability of each region is the ratio of the spectral measure of the region to the total mass of the sphere.
- We create a class of the spectral measure sampler that we want by inheriting from the base class `BaseSpectralMeasureSampler`. In the class that the user is creating, the function which samples the vectors $\boldsymbol{V}$ in Theorem 1 should be provided. Moreover, the dimension of the desired alpha-stable vector and the total mass of the unit sphere should be specified. The created spectral measure sampler is then passed as an argument to the `rvs()` method as shown in the following code.

The steps are presented below.

```python
import aub_htp as ht
from aub_htp.random import BaseSpectralMeasureSampler

alpha = 0.3
shift = [0, 0]
d = 2
n = 500000


class ButterflySampler(BaseSpectralMeasureSampler):

    def sample(self, number_of_samples: int, random_state=None):
        p = np.random.rand(number_of_samples)
        theta = np.empty(number_of_samples)

        mask = p <= 0.5
        theta[mask] = np.random.uniform(-np.pi / 4, np.pi / 4, size=mask.sum())
        theta[~mask] = np.random.uniform(3 * np.pi / 4,
                                         5 * np.pi / 4,
                                         size=(~mask).sum())

        x = np.cos(theta)
        y = np.sin(theta)

        return np.column_stack((x, y))

    def dimensions(self) -> int:
        return 2

    def mass(self) -> float:
        return 1.0


samples = ht.multivariate_alpha_stable.rvs(
    alpha=alpha,
    spectral_measure_sampler=ButterflySampler(),
    size=n,
    shift=shift)
```

We highlight that the mass of the sphere and the dimension should be specified when implementing the sub-class. The resulting scatter plot is shown in Figure 6b. It is important to emphasize that when $\alpha \geq 1$, the custom spectral measure should satisfy the condition in equation (4).



(a) Mix between isotropic and discrete spectral measures.

(b) A custom spectral measure

(c) Mix between custom and discrete spectral measures.

Figure 6. *Scatter plots showing a custom spectral measure and different mixtures of spectral measures. The plots have been truncated to a range of 400 points within the shift vector.*

One could mix a custom spectral measure with another spectral measure. For example, we mix below equally the spectral measure defined above with the discrete spectral measure with $\text{masses} = [(1,0),(0,1),(-1,0),(0,-1)]$ and $\text{weights} = [1,1,0.25,0.25]$, and the random vector has $\alpha = 0.25$, $\boldsymbol{\mu}^{0T} = (0,0)$.

```
import aub_htp as ht
from aub_htp.random import BaseSpectralMeasureSampler, DiscreteSampler, MixedSampler

alpha = 0.25
shift = [0, 0]
d = 2
n = 1000000


class ButterflySampler(BaseSpectralMeasureSampler):
    ...
    #implementation in previous block of code.


# Discrete Spectral Measure
positions = [[1, 0], [0, 1], [-1, 0], [0, -1]]
weights = [1, 1, 0.25, 0.25]
sampler1 = DiscreteSampler(alpha=alpha, positions=positions, weights=weights)

#Spectral Measures and Their Associated Weights
spectral_measures = [sampler1, ButterflySampler()]
measure_weights = [0.5, 0.5]

mixed_sampler = MixedSampler(spectral_measures=spectral_measures,
                             weights=measure_weights)
samples = ht.multivariate_alpha_stable.rvs(
    alpha=alpha, spectral_measure_sampler=mixed_sampler, shift=shift, size=n)
```

The resulting scatterplot is shown in Figure 6c.

6. Random Variate Generation Validation

To validate the correctness of our random variate generation, we conducted a projection based goodness of fit test. By the stability property, if $\boldsymbol{X}$ is an α-stable random vector in $\mathbb{R}^d$, then $\boldsymbol{\theta}^T\boldsymbol{X}$ is an α-stable random variable $\forall \boldsymbol{\theta} \in \mathbb{R}^d$ G. Samorodnitsky and M. S. Taqqu [18, Theorem 2.1.2]. One way to test the correctness of our generator is therefore to sample random vectors from AUB-HTP, compute the projection of these vectors on some other vectors, and then test the correctness of the resulting projection values which should theoretically have a univariate α-stable distribution. To test the goodness of fit of the resulting projection values, we use the Kolmogorov-Smirnov (KS) test. For details about the formulation of the test, we refer the reader to M. DeGroot and M. Schervish [28, Section 10.6]. We use the implementation of the test provided in the STABLE 5.3 Matlab package [29]. The test returns the D statistic (see M. DeGroot and M. Schervish [28, equation 10.6.2]) and the p-value associated with the test. Smaller values of D and larger values of p indicate a good fit. In all of the following tests, we determined the theoretical values of $\beta, \gamma,$ and δ using G. Samorodnitsky and M. S. Taqqu [18, equation. 2.3.3, equation. 2.3.4, equation. 2.3.5]; we sampled using AUB-HTP vectors with a mean square error less than 0.001; the dimension of the sampled vector was chosen to be $d = 2$; the shift vector was chosen to be $\boldsymbol{\mu}^{0T} = \mathbf{0}$; and 200000 sampled vectors were generated for the test. We tested two spectral measures, Isotropic and Discrete.

6.1. Isotropic Test

The sampled vectors used in this test have unit scale. Since the spectral measure is isotropic, the distribution of the projected values is invariant to the direction of projection. We therefore project along the x-axis and vary only the norm of the projection vector, using $(1, 0)$, $(2, 0)$, $(3, 0)$, $(4, 0)$, and $(5, 0)$. The resulting one-dimensional projections retain the same stability index α as the original vector, with $\beta = 0$, $\delta = 0$, and scale parameters $\gamma = 1, 2, 3, 4, 5$, respectively.

We report the results of the conducted tests for $\alpha = 0.6$ and $\alpha = 1.1$: the results on all the projected vectors gave $D = 0.0021$, $p = 0.344$ for $\alpha = 0.6$, and $D = 0.0013$, $p = 0.9$, for $\alpha = 1.1$. Both results indicate good fit.

6.2. Discrete Test

In this example, the spectral measure has 4 point masses at $(1,0),(0,1),(-1,0),(0,-1)$ with all masses having weight 0.25. We project along the vector $(\cos\theta, \sin\theta)$ and we vary $\theta \in \left\{0, \frac{\pi}{4}, \frac{\pi}{2}, \frac{3\pi}{4}, \pi, \frac{5\pi}{4}, \frac{3\pi}{2}, \frac{7\pi}{4}\right\}$. Tables 3 and 4 show the theoretical values of γ and β that the univariate stable projections should have (all projections have $\delta = 0$), the D statistic, and the p-value. Indeed, we can see that for both values of α and for most projection directions, the D statistic is small and the p-value is relatively large, which indicates a good fit. There are a few cases in which the D statistic is small but the p-value is small as well. In these cases, albeit the value of p is low, the fit is still deemed satisfactory (by virtue of the low value of D). This is due to the known fact that the KS test is highly sensitive at large sample sizes resulting in possibly low p-values even when the observed discrepancy is practically negligible. For a more detailed discussion on the sensitivity of the KS test, we refer the reader to M. DeGroot and M. Schervish [28, Section 10.6].

Table 3. *KS Test Results for* $\alpha = 0.6$

θ	γ	β	D	p
0	0.314980	0	0.001130	0.960380
$\frac{\pi}{4}$	0.707107	0	0.001058	0.978556
$\frac{\pi}{2}$	0.314980	0	0.001915	0.455143
$\frac{3\pi}{4}$	0.707107	0	0.001633	0.660202
π	0.314980	0	0.001130	0.960380
$\frac{5\pi}{4}$	0.707107	0	0.001058	0.978556
$\frac{3\pi}{2}$	0.314980	0	0.001915	0.455143
$\frac{7\pi}{4}$	0.707107	0	0.001633	0.660202

Table 4. *KS Test Results for* $\alpha = 1.1$

θ	γ	β	D	p
0	0.532521	0	0.001474	0.777075
$\frac{\pi}{4}$	0.707107	0	0.001979	0.413518
$\frac{\pi}{2}$	0.532521	0	0.002252	0.262298
$\frac{3\pi}{4}$	0.707107	0	0.001771	0.556735
π	0.532521	0	0.001474	0.777075
$\frac{5\pi}{4}$	0.707107	0	0.001979	0.413518
$\frac{3\pi}{2}$	0.532521	0	0.002252	0.262298
$\frac{7\pi}{4}$	0.707107	0	0.001771	0.556735

7. Conclusion

This paper presented AUB-HTP, a Python package for density computation and random variate generation of α-stable distributions. The package is motivated by the practical difficulty of working with α-stable laws, whose densities generally lack closed-form expressions and whose multivariate simulation requires careful handling of the underlying spectral measure.

AUB-HTP provides scalar density evaluation through a hybrid strategy that combines input standardization, Zolotarev-type integral representations, series formulas, and numerical inversion of characteristic functions. This design improves robustness across parameter regimes and enables systematic comparisons in terms of accuracy and runtime. The validation results show that the package provides reliable scalar density values and identify parameter regimes in which SciPy's reference implementation appears to suffer from numerical inaccuracies.

The package also implements scalar and multivariate random variate generation. For multivariate α-stable vectors, AUB-HTP uses LePage series representations and supports several spectral-measure models, including discrete, isotropic, sub-Gaussian/elliptical, mixed, and custom spectral measures. These features extend the range of α-stable models that can be simulated within the Python ecosystem. The truncation-error analysis further provides guidance on the approximation induced by finite LePage-series representations.

By bringing together density evaluation, flexible simulation, validation, runtime analysis, and extensible implementation tools, AUB-HTP aims to make α-stable modeling more accessible for computational work involving heavy-tailed data. Future developments may include multivariate PDF computation, CDF evaluation, parameter-estimation routines, further numerical acceleration, and application-oriented modules for communications, signal processing, and machine learning.

A. Appendix: Error analysis for the LePage series estimate

In this appendix, we derive an upper bound on the mean square error of the LePage series estimate; we frequently use the following asymptotics of the gamma function from [30]: Let $x, a, b \in \mathbb{R}$, then we have

$$\lim_{x\to\infty} \frac{\Gamma(x+a)}{\Gamma(x+b)\, x^{a-b}} = 1,$$

then $\forall a, b \in \mathbb{R}$, there exists an n_0 such that

$$\frac{\Gamma(k+a)}{\Gamma(k+b)} < 2k^{a-b} \qquad \forall k \geq n_0. \tag{10}$$

A.1. The LePage series truncation error

Theorem 2.

Let $\boldsymbol{X}$ and $\boldsymbol{X}_n$ be as in Theorem 1, and assume in addition that $\parallel \boldsymbol{V}_k \parallel = 1$ for all $k = 1, \cdots, n$. Let n_0 be the same as in (10) then $\forall n \geq \max\{n_0, \frac{2}{\alpha}\}$, $\mathbb{E} \parallel \boldsymbol{X}_n - \boldsymbol{X} \parallel^2 \leq \delta_n$ where

$$\delta_n = \begin{cases} \frac{2\alpha^2}{(1-\alpha)^2} c^2(\alpha) n^{2-\frac{2}{\alpha}} \alpha < 1 \\ \frac{2\alpha}{2-\alpha} c^2(\alpha) n^{1-\frac{2}{\alpha}} \quad \alpha \geq 1. \end{cases}$$

The proof is as follows.

We start with the case $\alpha < 1$. We have

$$\parallel \boldsymbol{X}_n - \boldsymbol{X} \parallel = \parallel c(\alpha) \sum_{k=n+1}^{\infty} \Gamma_k^{-\frac{1}{\alpha}} \boldsymbol{V}_k \parallel \leq c(\alpha) \sum_{k=n+1}^{\infty} \parallel \Gamma_k^{-\frac{1}{\alpha}} \boldsymbol{V}_k \parallel = c(\alpha) \sum_{k=n+1}^{\infty} \Gamma_k^{-\frac{1}{\alpha}},$$

which implies

$$
\begin{aligned}
\mathbb{E}[\parallel \boldsymbol{X}_n - \boldsymbol{X} \parallel^2] &\leq \mathbb{E}\left[\left(c(\alpha)\sum_{k=n+1}^{\infty}\Gamma_k^{-\frac{1}{\alpha}}\right)^2\right] = c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right] + 2\sum_{n<i<j}\mathbb{E}\left[\Gamma_i^{-\frac{1}{\alpha}}\Gamma_j^{-\frac{1}{\alpha}}\right]\right) \\
&\overset{(a)}{\leq} c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right] + 2\sum_{n<i<j}\sqrt{\mathbb{E}\left[\Gamma_i^{-\frac{2}{\alpha}}\right]\mathbb{E}\left[\Gamma_j^{-\frac{2}{\alpha}}\right]}\right) \\
&= c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right] + \left(\sum_{k=n+1}^{\infty}\sqrt{\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right]}\right)^2 - \sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right]\right) \\
&\overset{(b)}{=} c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\sqrt{\frac{\Gamma(k-\frac{2}{\alpha})}{\Gamma(k)}}\right)^2 \\
&\overset{(c)}{<} c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\sqrt{2k^{-\frac{2}{\alpha}}}\right)^2 \\
&= 2c^2(\alpha)\left(\sum_{k=n+1}^{\infty}k^{-\frac{1}{\alpha}}\right)^2 \\
&\overset{(d)}{\leq} \frac{2\alpha^2}{(1-\alpha)^2}c^2(\alpha)n^{2-\frac{2}{\alpha}}
\end{aligned}
\tag{11}
$$

where inequality (a) is due to the Cauchy-Schwarz inequality. The equality in (b) is due to the fact that $\Gamma_k \sim \text{Gamma}(k,1), n \geq \frac{2}{\alpha}$, and $\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right] = \frac{\Gamma(k-\frac{2}{\alpha})}{\Gamma(k)}$, and inequality (c) is due to (10). Inequality (d) is due to the fact that

$$
\sum_{k=n+1}^{\infty}k^{-\frac{1}{\alpha}} \leq \int_n^{\infty}x^{-\frac{1}{\alpha}}dx = \frac{\alpha}{1-\alpha}n^{1-\frac{1}{\alpha}}. \tag{16}
$$

For the case $\alpha \geq 1$, since $\parallel \boldsymbol{V}_k \parallel = 1 \; \forall \; k = 1, \cdots, n$, equation (4) implies $\mathbb{E}[\boldsymbol{V}] = \boldsymbol{0}$. Let $\boldsymbol{V}_j = \left(v_j^1, v_j^2, \cdots, v_j^d\right)$ then we have

$$
\begin{aligned}
\mathbb{E}[\parallel \boldsymbol{X}_n - \boldsymbol{X} \parallel^2] &= c^2(\alpha)\,\mathbb{E}\left[\left\langle\sum_{\ell=n+1}^{\infty}\Gamma_k^{-\frac{1}{\alpha}}\boldsymbol{V}_\ell, \sum_{k=n+1}^{\infty}\Gamma_k^{-\frac{1}{\alpha}}\boldsymbol{V}_k\right\rangle\right] \\
&= c^2(\alpha)\mathbb{E}\left[\sum_{k=n+1}^{\infty}\sum_{\ell=n+1}^{\infty}\sum_{i=1}^{d}\Gamma_k^{-\frac{1}{\alpha}}\Gamma_\ell^{-\frac{1}{\alpha}}v_k^i v_\ell^i\right] \\
&\overset{(a)}{=} c^2(\alpha)\left[\sum_{k=n+1}^{\infty}\sum_{\ell=n+1}^{\infty}\sum_{i=1}^{d}\mathbb{E}\left[\Gamma_k^{-\frac{1}{\alpha}}\Gamma_\ell^{-\frac{1}{\alpha}}\right]\mathbb{E}\left[v_k^i v_\ell^i\right]\right] \\
&\overset{(b)}{=} c^2(\alpha)\sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right]\mathbb{E}[\parallel \boldsymbol{V}_k \parallel^2] \\
&= c^2(\alpha)\sum_{k=n+1}^{\infty}\frac{\Gamma(k-\frac{2}{\alpha})}{\Gamma(k)} < 2c^2(\alpha)\sum_{k=n+1}^{\infty}k^{-\frac{2}{\alpha}} \\
&\leq \frac{2\alpha}{2-\alpha}c^2(\alpha)n^{1-\frac{2}{\alpha}},
\end{aligned}
\tag{17}
$$

where equality (a) is due to the fact that $\boldsymbol{V}_k \perp\!\!\!\perp \Gamma_k$ and (b) is justified since $v_k^i \perp\!\!\!\perp v_\ell^i, \forall\, k \neq \ell$ and $\mathbb{E}[v_k^i] = 0, \forall\, k$. The rest of the equalities and inequalities follow by similar steps as in the case $\alpha < 1$.

One can clearly see that the bound on the error decays faster for smaller values of α. This indicates that this sampling method can appropriately handle the difficult region of small values of α. In the one dimensional case, the mean square error of the LePage series can be found in A. Janicki and P. Kokoszka [31].

References

[1] M. Nassar, K. Gulati, A. Sujeeth, N. Aghasadeghi, B. Evans, and K. Tinsley, "Mitigating near-field interference in laptop embedded wireless transceivers," in *IEEE International Conference on Acoustics, Speech and Signal Processing*, Las Vegas, NV, Mar. 2008, pp. 1405–1408. doi: 10.1109/ICASSP.2008.4517882.

[2] G. Tsihrintzis, P. Tsakalides, and C. Nikias, "Signal detection in severely heavy-tailed radar clutter," in *Conference Record of The Twenty-Ninth Asilomar Conference on Signals, Systems and Computers*, 1995, pp. 865–869vol.2. doi: 10.1109/ACSSC.1995.540823.

[3] M. E. Crovella, M. S. Taqqu, and A. Bestavros, "Heavy-tailed probability distributions in the World Wide Web," *A practical guide to heavy tails*, vol. 1, pp. 3–26, 1998.

[4] J. P. Nolan, "Multivariate elliptically contoured stable distributions: theory and estimation," *Computational Statistics*, vol. 28, no. 5, pp. 2067–2089, 2013, doi: 10.1007/s00180-013-0396-7.

[5] N. N. Taleb, "Statistical Consequences of Fat Tails: Real World Preasymptotics, Epistemology, and Applications." [Online]. Available: https://arxiv.org/abs/2001.10488

[6] B. V. Gnedenko and A. N. Kolmogorov, *Limit Distributions for Sums of Independent Random Variables*. Reading Massachusetts: Addison-Wesley Publishing Company, 1968.

[7] J. M. Chambers, C. L. Mallows, and B. W. Stuck, "A Method for Simulating Stable Random Variables," *Journal of the American Statistical Association*, vol. 71, no. 354, pp. 340–344, 1976.

[8] J. Royuela-del Val, F. Simmross-Wattenberg, and C. Alberola-López, "libstable : Fast, Parallel, and High-Precision Computation of \alpha-Stable Distributions in R , C/C++ , and MATLAB", *Journal of statistical software*, vol. 78, no. 1, pp. 1–25, 2017.

[9] J. P. Nolan, *Univariate Stable Distributions: Models for Heavy Tailed Data*, 1st Edition 2020. in Springer Series in Operations Research and Financial Engineering. Cham: Springer Nature, 2020.

[10] M. Teimouri, M. Torshizi, A. Mohammadpour, and S. Nadarajah, "alphastable: An R Package for Modelling Multivariate Stable and Mixture of Symmetric Stable Distributions." [Online]. Available: https://arxiv.org/abs/1809.09546

[11] B. J. Swihart and J. P. Nolan, "The R Journal: Multivariate Subgaussian Stable Distributions in R," *The R Journal*, vol. 14, no. 3, pp. 293–302, 2022, doi: 10.32614/RJ-2022-056.

[12] V. M. Zolotarev, *One-Dimensional Stable Distributions*, vol. 65. in Translations of Mathematical Monographs, vol. 65. Providence, RI: American Mathematical Society, 1986.

[13] A. V. Skorohod, "Asymptotic formulas for stable distribution laws," *Selected translations in mathematical statistics and probability*, vol. 1, pp. 157–161, 1961.

[14] H. Pollard, "The Representation of \(e^-x^\lambda\) as a Laplace Integral", *Bulletin of the American Mathematical Society*, vol. 52, no. 10, pp. 908–910, 1946, doi: 10.1090/S0002-9904-1946-08672-3.

[15] H. Bergström, "On Some Expansions of Stable Distribution Functions," *Arkiv för Matematik*, vol. 2, no. 4, pp. 375–378, 1952, doi: 10.1007/BF02591503.

[16] D. Wuertz, M. Maechler, and R. core team members, "stabledist: Stable Distribution Functions," technical report, 2016. [Online]. Available: https://cran.r-project.org/package=stabledist

[17] R. LePage, M. Woodroofe, and J. Zinn, "Convergence to a Stable Distribution Via Order Statistics," *The Annals of probability*, vol. 9, no. 4, pp. 624–632, 1981.

[18] G. Samorodnitsky and M. S. Taqqu, *Stable non-Gaussian random processes : stochastic models with infinite variance*, 1st ed. Boca Raton: Chapman & Hall/CRC, 2000.

[19] A. Wintner, "The Singularities of Cauchy's Distributions," *Duke Mathematical Journal*, vol. 8, pp. 678–681, 1941.

[20] The SciPy Community, "scipy.stats.levy_stable," technical report, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.levy_stable.html

[21] R. Weron, "On the Chambers-Mallows-Stuck method for simulating skewed stable random variables," *Statistics & Probability Letters*, vol. 28, no. 2, pp. 165–171, 1996, doi: https://doi.org/10.1016/0167-7152(95)00113-1.

[22] R. Weron, "Computationally intensive Value at Risk calculations," Berlin, Papers 2004, 32, 2004. [Online]. Available: https://hdl.handle.net/10419/22205

[23] V. Bentkus, A. Juozulynas, and V. Paulauskas, "Lévy-LePage Series Representation of Stable Vectors: Convergence in Variation," *Journal of theoretical probability*, vol. 14, no. 4, pp. 949–978, 2001.

[24] S. Schnabel and W. Janke, "A simple algorithm for uniform sampling on the surface of a hypersphere," *arXiv.org*, 2022.

[25] M. E. Muller, "A note on a method for generating points uniformly on n -dimensional spheres," vol. 2. pp. 19–20, 1959.

[26] R. A. Horn and C. R. Johnson, *Matrix analysis*. Cambridge: Cambridge University Press, 1985.

[27] H. Niederreiter, *Random number generation and quasi-Monte Carlo methods*. in CBMS-NSF Regional Conference Series in Applied Mathematics. Philadelphia, Pa: Society for Industrial, 1992.

[28] M. DeGroot and M. Schervish, *Probability and Statistics*. in Pearson Modern Classics for Advanced Statistics Series. Pearson Education, Incorporated, 2018. [Online]. Available: https://books.google.com.lb/books?id=poCatAEACAAJ

[29] J. Nolan, "Robust Analysis Inc.." [Online]. Available: http://www.robustanalysis.com/

[30] "NIST Digital Library of Mathematical Functions." [Online]. Available: https://dlmf.nist.gov/

[31] A. Janicki and P. Kokoszka, "Computer investigation of the Rate of Convergence of Lepage Type Series to \alpha-Stable Random Variables", *Statistics (Berlin, DDR)*, vol. 23, no. 4, pp. 365–373, 1992.

Open Access | https://doi.org/10.25080/eyxa7843

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Detecting Anomalies in Scientific Data Using SciPy's Statistical and Signal Tools

Aayush Gauba[1]

[1]Southern Illinois University Edwardsville

Abstract

Scientific datasets often contain subtle anomalies caused by sensor faults, experimental drift, measurement noise, rare events, or abnormal signal behavior. In many scientific workflows, anomaly detection must be not only accurate but also interpretable, because researchers need to understand whether unusual observations reflect meaningful phenomena, instrument error, or changes in experimental conditions. This paper presents a SciPy-based anomaly detection workflow using statistical and signal-processing tools. Rather than evaluating the workflow only on a single benchmark dataset, we construct a controlled multi-domain experiment using six scientific signal domains: electroencephalography (EEG)-like signals, environmental sensor signals, spectroscopy-like signals, vibration signals, astronomical photometry-like signals, and fluid-sensor signals. Five controlled anomaly mechanisms are injected into these signals: spike, drift, dropout, oscillation disturbance, and distribution shift. SciPy statistical, signal-processing, residual, spectral, local-window, and complexity features are extracted from every signal. The experiment evaluates binary anomaly detection, anomaly-mechanism classification, relative feature-group signatures, and rule-based diagnosis. In the controlled experiment, SciPy-derived features achieved perfect binary anomaly detection, but anomaly-mechanism classification was substantially harder, with the best SciPy-feature model achieving macro-F1 = 0.4578 across five anomaly types. A hand-written rule-based diagnosis system using relative feature-group signatures achieved only 0.1993 accuracy, approximately chance. This shows that detection performance does not imply diagnostic interpretability. To test whether the workflow transfers beyond controlled injection, we also evaluate it on the University of California, Riverside (UCR) Time Series Anomaly Archive [1]. Because the controlled benchmark provides known anomaly labels while the UCR archive provides only normal training regions, the two experiments evaluate complementary supervised and unsupervised settings. Across 56 evaluated real anomaly files, the best SciPy method achieved mean point-adjusted F1 = 0.2884, receiver operating characteristic area under the curve (ROC-AUC) = 0.7758, precision-recall area under the curve (PR-AUC) = 0.1801, and event detection rate = 0.4107. These results show that SciPy features provide a useful and transparent anomaly-detection baseline, but also reveal the limits of global feature summaries: real anomaly detection remains difficult, and mechanism diagnosis requires more than simple feature-group explanations.

Keywords anomaly detection, scipy, time series, statistical diagnostics, signal processing

Published Jul 13, 2026

Correspondence to
Aayush Gauba
gauba.aayush@gmail.com

Open Access

Copyright © 2026 Gauba. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Scientific data is rarely perfectly clean. Measurements collected from sensors, laboratory instruments, simulations, biological systems, telescopes, and physical experiments may contain unexpected behavior caused by noise, sensor faults, experimental drift, environmental interference, or rare events. Detecting these anomalies is important because abnormal observations can distort downstream analysis, reduce model reliability, and lead to incorrect scientific conclusions.

However, anomaly detection in scientific data is not only a prediction problem. It is also a diagnostic problem. A researcher often needs to know why a signal is unusual. For example, an abnormal measurement may be caused by a transient spike, a gradual drift, a dropout or flatline, a spectral disturbance, or a distribution shift. A model that only reports "anomaly" may be useful for flagging data, but it may not provide enough information to support scientific interpretation or corrective action.

This paper studies anomaly detection using SciPy's statistical and signal-processing tools. SciPy is a foundational scientific computing library for Python that provides algorithms for statistics, signal processing, optimization, integration, interpolation, linear algebra, and other scientific computing tasks [2]. The `scipy.stats` module provides probability distributions, summary statistics, and statistical tests. The `scipy.signal` module provides tools for filtering, peak detection, spectral analysis, and time-series signal processing. These tools make SciPy a practical foundation for anomaly detection workflows that need to be lightweight, reproducible, and interpretable.

We test whether SciPy-derived features detect controlled anomalies and support diagnosis. Six domain-inspired signal generators, five anomaly mechanisms, and multiple severity and noise levels provide known ground truth. We then test transfer on the UCR archive. Our central claim is that detection performance does not imply diagnostic interpretability: global energy features respond to several mechanisms, so diagnosis requires localized or mechanism-specific information.

The contributions are:

- a controlled multi-domain anomaly benchmark using six scientific signal domains and five anomaly mechanisms;
- a SciPy-based feature extraction workflow using statistical, signal-processing, spectral, residual, local-window, and complexity features;
- a comparison of SciPy-feature models, relative-signature models, and raw-signal baselines;
- comparisons of feature, signature, and raw-signal models for detection and diagnosis;
- UCR external validation;
- evidence that dominant-feature-group explanations are insufficient for diagnosis.

2. Related Work

Time-series anomaly detection has a long history in statistical process monitoring. Classical approaches include threshold-based monitoring, Shewhart control charts, cumulative sum methods (CUSUM), and exponentially weighted moving average methods. These approaches are attractive because they are simple, interpretable, and computationally lightweight, but they often require assumptions about stationarity, distributional behavior, or known control limits.

Unsupervised machine-learning methods provide a more flexible alternative when labeled anomalies are rare. Isolation Forest is a widely used representative method that isolates anomalies through randomized partitioning rather than explicitly modeling normal density [3]. One-class methods such as One-Class SVM similarly learn a boundary around normal data and are commonly used when only normal training samples are available [4]. These methods are relevant to the UCR validation setting in this paper, where training regions are treated as normal and anomaly labels are used only for evaluation.

Deep learning methods include LSTM encoder-decoder reconstruction models [5] and transformer-based models such as TranAD [6]. They can be powerful, but often require more data, tuning, and computation than classical feature-based methods.

Benchmark design is also a central issue in anomaly detection. Wu and Keogh argue that many time-series anomaly detection benchmarks are flawed and can create an illusion of progress, motivating more careful evaluation protocols and realistic anomaly archives [1]. This concern is directly relevant to the present work: perfect detection on a controlled synthetic benchmark should not be interpreted as deployment-level success. For this reason, the controlled benchmark in this paper is used for mechanism testing, while the UCR archive is used as an external validation setting.

This paper differs from prior work by focusing on the distinction between anomaly detection and anomaly diagnosis. The goal is not to establish a new state-of-the-art detector. Instead, we test whether SciPy-derived statistical and signal-processing features provide a transparent representation that can detect anomalies, preserve partial mechanism-level information, and expose the limitations of simple feature-group interpretability.

3. Methods

Table 1 summarizes the methodological difference between the controlled and UCR evaluations.

Table 1. *Learning settings used in the two evaluation stages.*

Experiment	Training Signal	Evaluation Goal
Controlled benchmark	Labeled anomalies	Detection and diagnosis
UCR validation	Normal windows only	Real-data detection

3.1. Scientific Signal Domains

We generate clean signals from six scientific signal domains:

- **EEG-like signals**: mixtures of low-frequency rhythms, burst-like oscillations, and colored noise;
- **Environmental sensor signals**: slow periodic cycles, low-frequency trends, and correlated noise;
- **Spectroscopy-like signals**: smooth baselines with Gaussian absorption or emission peaks;
- **Vibration signals**: oscillatory signals with harmonics and amplitude modulation;
- **Astronomical photometry-like signals**: light-curve-like signals with periodic dips and correlated noise;
- **Fluid-sensor signals**: quasi-periodic vortex-shedding-like signals with modulation and noise.

These are synthetic domain-inspired signals, not measurements from real instruments. They are used because controlled generation allows the true anomaly mechanism, severity, and noise level to be known. These domains were chosen because they differ in baseline structure, noise statistics, and signal dynamics. This allows the experiment to test whether anomaly behavior generalizes across heterogeneous scientific signals. Figure 1 shows one independently sampled clean trace and one drift-anomaly trace from each domain.

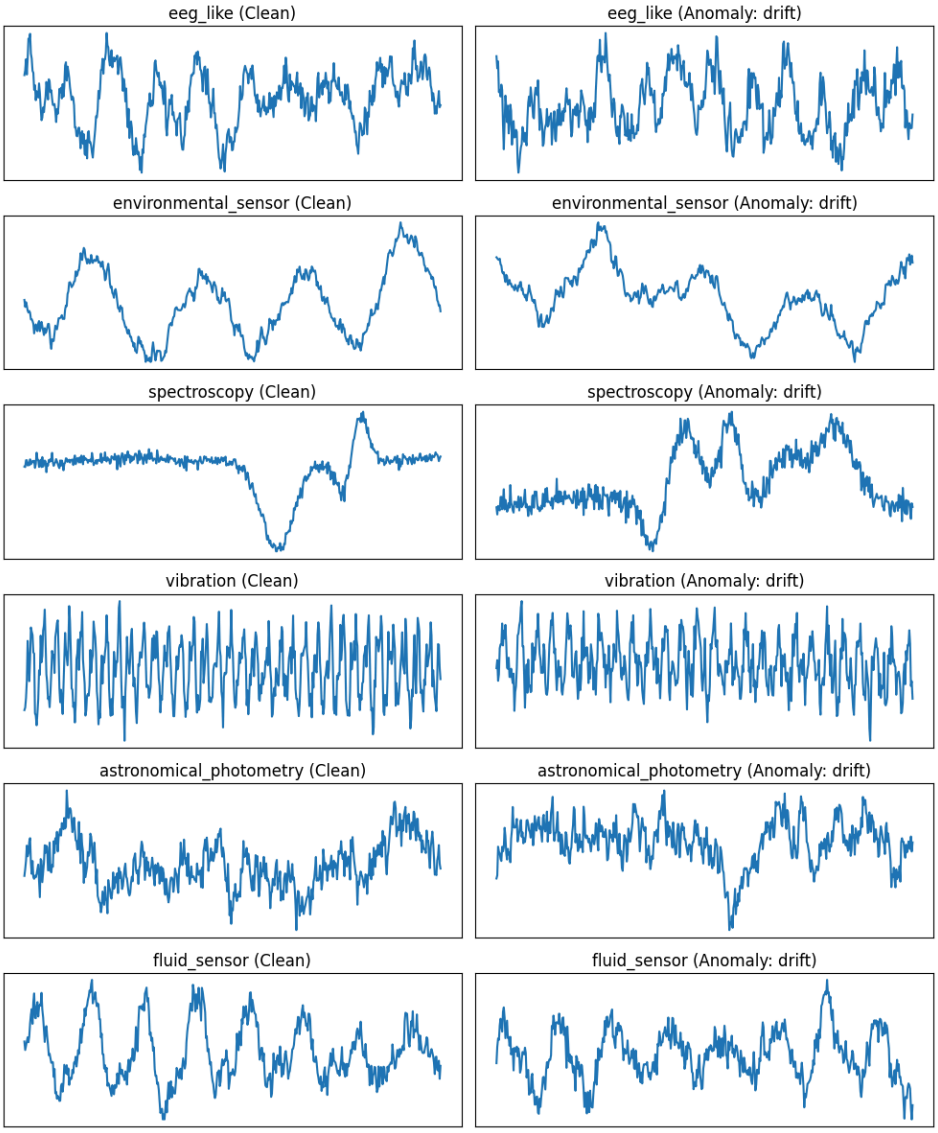


Figure 1. *Examples from the six synthetic scientific signal domains. The left column shows one clean trace per domain, and the right column shows an independently generated trace with an injected drift anomaly. The traces were selected deterministically with random seed 42. Because the clean and anomalous examples were sampled independently, each row illustrates domain morphology and the anomaly type rather than a pointwise before-and-after pair.*

3.2. Controlled Anomaly Mechanisms

Five anomaly mechanisms are injected into the clean signals:

- **Spike**: a short transient pulse added to the signal;
- **Drift**: a gradual baseline shift over part of the signal;
- **Dropout**: a flatline or attenuated region;
- **Oscillation disturbance**: a local high-frequency disturbance;
- **Distribution shift**: a local mean, variance, or scale change.

Each anomaly is evaluated across seven severity levels:

$$0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.35 \tag{1}$$

and four noise levels:

$$1.0,\ 2.0,\ 3.0,\ 5.0. \tag{2}$$

For each anomalous setting, 20 repeats are generated, giving $6 \times 5 \times 7 \times 4 \times 20 = 16,800$ anomalous traces. We independently generate 16,800 clean traces, balanced across the six domains and four noise levels. The final controlled dataset therefore contains 33,600 signals (50% clean and 50% anomalous), each with 512 time points. The extracted feature matrix contains the 56 SciPy-derived features listed in Table 2.

3.3. SciPy Feature Extraction

For each signal $x = [x_1, x_2, ..., x_T]$, we extract statistical, signal-processing, spectral, residual, local-window, and complexity features.

For the controlled 512-point signals, Savitzky-Golay smoothing used a 51-sample window, polynomial order 3, and `mode=interp`. Global Welch power spectral density estimates used `nperseg=128`; the local spectral features divided each signal into 128-sample windows with a 32-sample step and used `nperseg=64` within each window. The code left `noverlap` unspecified, so SciPy's default half-segment overlap gave 64 samples for the global estimates and 32 samples for the local estimates. For the 128-point windows in the UCR validation, the corresponding settings were a 31-sample Savitzky-Golay window with polynomial order 3 and `mode=interp`, and Welch `nperseg=64` with the default 32-sample overlap.

The core implementation calls `signal.savgol_filter(x, window_length=51, polyorder=3, mode="interp")` and computes the residual as `x - smooth`. Its energy, absolute mean, and maximum magnitude are then computed with NumPy reductions. For spectral features, the implementation calls `signal.welch(x, fs=fs, nperseg=128)` and integrates each selected frequency band with `simpson`. The same power spectral density supplies spectral entropy, dominant frequency, and high-frequency ratios.

Table 2. *The 56 extracted SciPy-derived features. Names match the notebook output columns.*

Category	**Feature names**
Statistical distribution	`mean`, `std`, `variance`, `min`, `max`, `range`, `median`, `iqr`, `mad`, `skewness`, `kurtosis`, `hist_entropy`, `normaltest_stat`, `normaltest_p`, `shapiro_stat`, `shapiro_p`
Amplitude and energy	`abs_mean`, `rms`, `energy`, `line_length`
Peak and transient	`peak_count`, `peak_density`, `mean_peak_prominence`, `max_peak_prominence`, `mean_peak_width`, `derivative_max`, `second_derivative_max`
Trend and residual	`savgol_residual_energy`, `savgol_residual_abs_mean`, `savgol_residual_max`, `rolling_mean_max_change`
Dropout and flatness	`rolling_var_min`, `rolling_var_max`, `rolling_var_ratio`, `flatness_fraction`
Spectral frequency	`delta_power`, `theta_power`, `alpha_power`, `beta_power`, `gamma_power`, `delta_ratio`, `theta_ratio`, `alpha_ratio`, `beta_ratio`, `gamma_ratio`, `spectral_entropy`, `dominant_frequency`, `high_frequency_ratio`, `local_high_freq_ratio_max`, `local_high_freq_ratio_std`, `local_spectral_entropy_min`, `local_spectral_entropy_std`
Complexity	`zero_crossing_rate`, `hjorth_activity`, `hjorth_mobility`, `hjorth_complexity`

3.4. Relative Feature-Group Signatures

To test interpretability, features are grouped into seven feature groups:

- statistical distribution;
- amplitude-energy;
- peak-transient;
- trend-residual;

- dropout-flatness;
- spectral-frequency;
- complexity.

For each domain, clean signals define a robust baseline. For each feature, we compute a robust absolute z-score relative to the clean baseline of the same domain, using the domain-specific median and 1.4826 times the median absolute deviation. Absolute z-scores are clipped at 8.0 to reduce domination by large-magnitude features. Within each feature group, the median clipped z-score is used as the group response. The group responses are then normalized so that they sum to one. This produces a relative feature-group signature for each sample. Of the 56 extracted features, 53 enter these groups; `normaltest_p`, `shapiro_p`, and `rolling_var_max` are retained in the full feature vectors but excluded from the relative signatures.

As implemented in the reported experiment, the domain baselines are estimated once from all 16,800 clean traces before the train-test split, rather than from the training partition alone. Thus test-set clean traces contribute to the robust centering and scaling used by the relative-signature representation; the full-feature and raw-signal representations do not use this baseline transformation.

The goal is to test whether anomaly mechanisms map to distinct relative feature-group patterns.

3.5. Models and Baselines

We evaluate three representations:

- full SciPy feature vectors;
- compressed relative feature-group signatures;
- raw time-domain signals.

For binary anomaly detection, we evaluate Logistic Regression, SVM-RBF, Random Forest, and Extra Trees models where applicable. For anomaly-type classification, we compare SciPy-feature models, relative-signature models, and raw-signal models. Both tasks use a row-level, stratified random split with 75% of rows for training and 25% for testing (random seed 42). The split is not grouped by repeat or domain. Only the separate leave-one-domain-out (LODO) evaluation holds out complete domains.

Table 3 records the non-default model settings used in the controlled and UCR experiments. Parameters not shown retain their scikit-learn defaults. Logistic Regression, SVM-RBF, and One-Class SVM inputs are standardized; the UCR feature and raw-window inputs are robustly standardized from their normal training windows.

Table 3. *Model hyperparameters used in the reported experiments.*

Model	Use	Hyperparameters
Logistic Regression	Controlled	`max_iter=3000`, `class_weight=balanced`, seed 42
SVM-RBF	Controlled binary	RBF kernel, $C = 1$, `gamma=scale`, `probability=True`, `class_weight=balanced`, seed 42
Random Forest	Controlled	300 trees for binary detection; 400 trees for anomaly-type classification; `class_weight=balanced`, seed 42, all CPU cores
Extra Trees	Controlled	300 trees for binary detection; 400 trees for anomaly-type classification and LODO; `class_weight=balanced`, seed 42, all CPU cores

Isolation Forest	UCR	200 trees, `contamination=0.02`, seed 42, all CPU cores; decision threshold is the 99th percentile of normal-training anomaly scores
One-Class SVM	UCR	RBF kernel, `gamma=scale`, $\nu = 0.02$; decision threshold is the 99th percentile of normal-training anomaly scores

The main evaluation metrics are balanced accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC for binary anomaly detection. For anomaly-type classification, we report accuracy, balanced accuracy, macro-F1, and weighted-F1.

3.6. Rule-Based Diagnosis

To test whether feature groups directly support mechanistic diagnosis, we define a simple rule-based diagnosis system:

- spike: peak-transient, statistical-distribution, and trend-residual groups;
- drift: trend-residual and statistical-distribution groups;
- dropout: dropout-flatness and trend-residual groups;
- oscillation disturbance: spectral-frequency and complexity groups;
- distribution shift: statistical-distribution and amplitude-energy groups.

For each anomalous signal, the rule system computes the mean response of the expected groups for each anomaly type and predicts the type with the largest score. If simple feature-group interpretability is sufficient, this rule system should classify anomaly type above chance.

4. Results

4.1. Binary Detection Was Trivial Under Controlled Injection

Perfect controlled binary scores show that the injected perturbations are measurable, but also that this synthetic boundary is too easy. Table 4 therefore motivates the harder mechanism task rather than a claim of deployment performance.

Table 4. *Binary anomaly detection results.*

Model	Balanced Acc.	Accuracy	F1	ROC-AUC	PR-AUC
SciPy Features + Random Forest	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Relative Signatures + Logistic Regression	**1.0000**	**1.0000**	**1.0000**	**1.0000**	**1.0000**
SciPy Features + Extra Trees	0.9995	0.9995	0.9995	1.0000	1.0000
SciPy Features + SVM-RBF	0.9087	0.9087	0.9053	0.9721	0.9761
SciPy Features + Logistic Regression	0.8260	0.8260	0.8241	0.9167	0.9279
Raw Signal + Random Forest	0.7073	0.7073	0.6650	0.7926	0.8166
Raw Signal + Extra Trees	0.7060	0.7060	0.6632	0.7993	0.8198
Raw Signal + SVM-RBF	0.7015	0.7015	0.6475	0.7914	0.8082
Raw Signal + Logistic Regression	0.5039	0.5039	0.4973	0.5093	0.5295

Raw-signal tree baselines reached balanced accuracy near 0.71, showing that the SciPy representation makes the controlled boundary more separable.

4.2. Anomaly Diagnosis Was Substantially Harder

Each mechanism contributes 3,360 anomalous traces and 840 traces to the stratified test set (4,200 total). The best model reached macro-F1 0.4578, above the raw-signal baseline (0.2051) and most-frequent dummy (0.0667). The dummy's balanced accuracy is 0.20 but its macro-F1 is lower because it predicts only one class. Table 5 reports all results.

Table 5. *Anomaly-type classification results across five anomaly mechanisms.*

Model	Balanced Acc.	Accuracy	Macro-F1	Support
SciPy Features + Random Forest	**0.4660**	**0.4660**	**0.4578**	840/class
SciPy Features + Extra Trees	0.3852	0.3852	0.3896	840/class
Relative Signatures + Logistic Regression	0.2160	0.2160	0.2070	840/class
Relative Signatures + Extra Trees	0.2057	0.2057	0.2051	840/class
Raw Signal + Extra Trees	0.2050	0.2050	0.2051	840/class
Most-frequent dummy	0.2000	0.2000	0.0667	840/class

4.3. Confusion Structure of Anomaly Mechanisms

The macro-F1 score shows that anomaly-mechanism classification is substantially harder than binary detection, but it does not reveal which mechanisms are confused. We therefore inspect the confusion matrix of the best anomaly-type classifier, SciPy Features + Random Forest. Table 6 shows the row-normalized confusion matrix, where each row represents the true anomaly type and each column represents the predicted anomaly type.

Table 6. *Row-normalized confusion matrix for anomaly-type classification using SciPy Features + Random Forest. Rows indicate true anomaly type and columns indicate predicted anomaly type.*

True / Predicted	Distribution Shift	Drift	Dropout	Oscillation Disturbance	Spike
Distribution Shift	0.34	0.17	0.07	0.28	0.13
Drift	0.07	0.72	0.03	0.12	0.06
Dropout	0.06	0.09	0.60	0.10	0.15
Oscillation Disturbance	0.21	0.04	0.11	0.45	0.18
Spike	0.24	0.10	0.18	0.26	0.21

The confusion matrix shows that the classifier learned some anomaly mechanisms better than others. Drift was the most separable mechanism, with 72% of drift samples correctly classified. Dropout was also partially separable, with 60% correctly classified. Oscillation disturbance was detected at a lower but still meaningful rate, with 45% correctly classified. In contrast, distribution shift and spike were much less separable. Distribution shift was correctly classified only 34% of the time and was often confused with oscillation disturbance. Spike was the weakest class, with only 21% correctly classified, and was frequently confused with distribution shift and oscillation disturbance.

This confusion structure supports the paper's central argument: SciPy features contain partial mechanism-level information, but anomaly diagnosis is not solved by global feature summaries. Drift and dropout produce more consistent feature patterns, likely because they alter baseline or flatness structure. Spike and oscillation disturbance are more ambiguous in the current representation, suggesting that short local transients and local spectral changes are not sufficiently isolated by whole-signal features. This reinforces the need for local anomaly localization before mechanism diagnosis.

This pattern has a mechanistic explanation. Drift produces a sustained baseline displacement, which changes rolling mean structure and creates a cumulative low-frequency effect across a large portion of the signal. Because the perturbation persists over many time steps, global and rolling-window features can capture it reliably. Dropout similarly creates an extended low-variance or attenuated region, which is partly captured by rolling-variance and flatness features. Spike is different: it is a short local transient. When features are computed over the full $T = 512$ point segment, the contribution of a narrow spike is diluted by the segment length. A local spike may strongly affect a few derivative or peak features, but many global statistics average it with hundreds of normal samples. This explains why spike was frequently confused with distribution shift and oscillation disturbance. It also suggests that reliable spike diagnosis requires anomaly localization or local-window feature extraction before whole-signal classification.

4.4. Relative Feature-Group Signature Map

Table 7 shows the average relative feature-group signature for each anomaly type. The strongest group for every anomaly type was amplitude-energy.

Table 7. *Relative feature-group signature map by anomaly type.*

Anomaly Type	Stat.	Amp./Energy	Peak	Trend	Dropout	Spectral	Complexity
Distribution shift	0.1046	**0.5383**	0.0745	0.0855	0.0572	0.0598	0.0801
Drift	0.1069	**0.5379**	0.0733	0.0835	0.0583	0.0599	0.0803
Dropout	0.1088	**0.5279**	0.0756	0.0833	0.0663	0.0612	0.0770
Oscillation disturbance	0.1044	**0.5403**	0.0735	0.0844	0.0583	0.0599	0.0792
Spike	0.1046	**0.5407**	0.0738	0.0846	0.0566	0.0602	0.0796

The signature map does not support the simple hypothesis that each anomaly type maps cleanly to a different dominant feature group. Even after robust normalization, clipping, median aggregation, and relative normalization, amplitude-energy remained the dominant group for every anomaly type. This indicates that the controlled anomaly injections often changed global signal energy even when the intended anomaly mechanism was local, spectral, or structural.

As a post-hoc no-energy control, we removed the amplitude-energy column from the mean signatures in Table 7 and renormalized the six remaining columns to sum to one. Statistical-distribution then became the largest mean group for all five mechanisms, with normalized shares between 0.227 and 0.231. Thus removing energy changes which group dominates but does not recover a distinct dominant group for each mechanism. This control is descriptive; we did not retrain the classifiers or recompute per-trace rule accuracy without the energy group.

4.5. Rule-Based Diagnosis

The rule-based diagnosis system achieved overall accuracy = 0.1993, essentially equal to the 0.2000 accuracy of a most-frequent-class baseline on the balanced five-class dataset. Table 8 shows the accuracy by anomaly type and makes clear that the aggregate result comes almost entirely from predicting distribution shift.

Table 8. *Rule-based diagnosis accuracy by anomaly type.*

Anomaly Type	Rule Accuracy

Distribution shift	0.9881
Drift	0.0006
Dropout	0.0077
Oscillation disturbance	0.0000
Spike	0.0000

The rule-based diagnosis result reveals the central failure mode of naive interpretability. Distribution shift achieved 0.9881 rule accuracy, while spike, drift, dropout, and oscillation disturbance were nearly always misdiagnosed. This pattern is not random. The rule system assigns distribution shift using statistical-distribution and amplitude-energy responses. Since amplitude-energy dominated the relative signature map for every anomaly type, the rule system was biased toward the distribution-shift rule. As a result, distribution shift was diagnosed correctly, not because the rule system learned a rich mechanism-specific signature, but because the confounding amplitude-energy response made other anomaly types resemble distribution-shift behavior.

This explains why the overall rule-based diagnosis accuracy was approximately chance. The rule system did not fail because the features were meaningless. It failed because a globally responsive feature group collapsed the signature space. This is a precise example of why named features alone are not sufficient for scientific interpretability.

4.6. Leave-One-Domain-Out Generalization

To test domain generalization, we trained an anomaly-type classifier on five domains and tested on the held-out sixth domain. Table 9 shows the results.

Table 9. *Leave-one-domain-out anomaly-type classification.*

Held-Out Domain	Balanced Acc.	Macro-F1
EEG-like	0.4114	0.4194
Fluid sensor	0.3939	0.3941
Astronomical photometry	0.3800	0.3728
Environmental sensor	0.3418	0.3503
Vibration	0.3307	0.3291
Spectroscopy	0.2611	0.2087

The leave-one-domain-out results show partial but limited domain generalization. Some domains, such as EEG-like and fluid-sensor signals, generalized better than others. Spectroscopy and vibration were more difficult. This suggests that anomaly mechanisms are not fully domain-independent. The same injected anomaly may interact differently with different baseline signal structures.

4.7. Statistical Tests on Feature-Group Responses

Kruskal-Wallis tests were used to test whether feature-group responses differed by anomaly type. Table 10 shows the results.

Table 10. *Kruskal-Wallis tests for feature-group response differences by anomaly type.*

Feature Group	H Statistic	p-value
Statistical distribution	56.2222	1.80×10^{-11}
Dropout flatness	36.5159	2.27×10^{-7}

Amplitude-energy	26.7147	2.27×10^{-5}
Complexity	9.9531	4.12×10^{-2}
Spectral frequency	6.3235	0.1763
Trend residual	6.2548	0.1809
Peak transient	5.4034	0.2484

Each Kruskal-Wallis test used all 16,800 anomalous traces, with five equally sized mechanism groups of $n = 3,360$. Seven feature-group tests were performed. With an uncorrected $\alpha = 0.05$, statistical-distribution, dropout-flatness, amplitude-energy, and complexity differed across mechanisms. Under a Bonferroni correction, the threshold is $0.05/7 = 0.00714$; only statistical-distribution, dropout-flatness, and amplitude-energy remain significant, while complexity ($p = 0.0412$) does not. Spectral-frequency, trend-residual, and peak-transient are not significant under either threshold. Thus anomaly-type differences exist, but they are not captured by a simple one-group-per-anomaly interpretation.

An important irony is that the peak-transient and trend-residual groups, which are the most intuitively matched to spike and drift anomalies, did not reach Kruskal-Wallis significance in the 16,800-trace controlled analysis. This reinforces the central argument that intuitive feature names do not automatically translate into reliable diagnostic separation.

5. Real-Data Validation on the UCR Time Series Anomaly Archive

5.1. Dataset and Setup

To evaluate whether the SciPy anomaly detection workflow transfers beyond controlled synthetic injection, we performed an external validation experiment on the 2021 UCR Time Series Anomaly Archive [1], using `UCR_TimeSeriesAnomalyDatasets2021.zip` from the archive URL recorded in the notebook. The notebook does not record a download date. Unlike the controlled benchmark, where anomaly type, severity, and noise level are generated by design, the UCR archive contains real time-series anomaly problems with labeled anomaly intervals encoded in the file metadata.

We evaluated the first 60 archive files in lexicographic filename order. Four files were skipped because the sliding-window labeling procedure produced no anomalous test windows, leaving 56 evaluated files. Table 11 identifies both sets. Each time series was converted into overlapping windows of length 128 with step size 16. Windows ending before the training boundary were used as normal training windows. Test windows were labeled anomalous if they overlapped the labeled anomaly interval by at least 10% of the window length. Point-adjusted F1 treats an anomaly interval as detected if at least one overlapping window is flagged, which is common in time-series anomaly detection but can be more optimistic than raw window-level F1.

Table 11. *UCR file selection. Numeric IDs are the filename prefixes in the 2021 archive.*

Status	Archive file IDs or filenames
Evaluated (56)	001–004, 006–026, 028–029, 031–050, and 052–060
Skipped (4)	`005_UCR_Anomaly_DISTORTEDCIMIS44AirTemperature1_4000_5391_5392.txt`; `027_UCR_Anomaly_DISTORTEDInternalBleeding16_1200_4187_4199.txt`; `030_UCR_Anomaly_DISTORTEDInternalBleeding19_3000_4187_4197.txt`; `051_UCR_Anomaly_DISTORTEDTkeepSecondMARS_3500_9330_9340.txt`

For each window, we extracted the same SciPy-inspired statistical and signal-processing features used in the controlled experiment, including distributional statistics, peak features, residual features, rolling-window features, and spectral features. We evaluated four unsupervised methods: SciPy Features + Robust Distance, SciPy Features + Isolation Forest, SciPy Features + One-Class SVM, and Raw Windows + Isolation Forest. We do not claim state-of-the-art performance on the UCR archive. The purpose of the UCR experiment is external validation of the SciPy feature workflow under naturally labeled anomaly intervals, not benchmark optimization.

5.1.1. *Methodological asymmetry.*

The controlled benchmark and UCR validation evaluate different learning settings. In the controlled benchmark, anomaly labels and anomaly-mechanism labels are known by construction, so supervised classifiers can be trained for both binary detection and anomaly-type classification. In contrast, the UCR archive provides normal training regions and labeled anomaly intervals for evaluation, but it does not provide labeled examples of each anomaly mechanism for supervised mechanism classification. Therefore, the UCR validation is evaluated using unsupervised and one-class methods trained on normal windows. This asymmetry is intentional: the controlled benchmark tests mechanism-level hypotheses under known labels, while the UCR experiment tests whether the SciPy feature workflow remains useful when only normal training data are available. The two experiments should therefore not be interpreted as identical evaluations, but as complementary tests of mechanism understanding and real-data transfer.

5.2. Results

Table 12 summarizes the real-data validation results. The strongest method by mean point-adjusted F1 was SciPy Features + Isolation Forest, which achieved mean point-adjusted F1 = 0.2884, raw F1 = 0.1248, ROC-AUC = 0.7758, PR-AUC = 0.1801, and event detection rate = 0.4107 across 56 evaluated files. SciPy Features + One-Class SVM achieved the highest event detection rate at 0.6607, but had a lower mean point-adjusted F1 of 0.2646.

Table 12. *Real-data validation on the UCR Time Series Anomaly Archive.*

Method	Raw Bal. Acc.	Raw F1	ROC-AUC	PR-AUC	Point-Adj. F1	Event Detection
SciPy Features + Isolation Forest	0.5588	0.1248	**0.7758**	0.1801	**0.2884**	0.4107
SciPy Features + One-Class SVM	**0.6374**	**0.1375**	0.6561	**0.2143**	0.2646	**0.6607**
Raw Windows + Isolation Forest	0.5484	0.1184	0.5450	0.1582	0.2617	0.3571
SciPy Features + Robust Distance	0.5340	0.0877	0.7225	0.1331	0.2328	0.2857

Each selected UCR file contains one labeled anomaly interval, and the notebook saved only whether that interval was hit by at least one positive window. It did not save the contigu-

ous predicted event intervals needed to count false event-level alarms. Consequently, the reported event quantity is a detection rate (equivalently recall over the 56 labeled events), while event-level precision cannot be reconstructed from the saved run. Raw window-level precision, included in the notebook's per-file results, measures false positive windows but is not a substitute for event-level precision.

The UCR results show a sharp contrast with the controlled synthetic benchmark. In the synthetic experiment, binary anomaly detection was nearly trivial, with SciPy-feature models reaching perfect performance. In the real-data validation experiment, performance was much lower. This confirms that perfect performance under controlled injection should not be interpreted as deployment-level anomaly detection ability. Real anomaly intervals are more heterogeneous, less aligned with fixed window boundaries, and often subtler than injected perturbations.

The real-data results also show that different metrics emphasize different operational goals. SciPy Features + Isolation Forest achieved the best point-adjusted F1 and ROC-AUC, suggesting better ranking and localization performance. SciPy Features + One-Class SVM achieved the highest event detection rate, suggesting that it was more likely to flag at least one window inside the anomaly interval. Depending on the scientific workflow, detecting an event may be more important than window-level precision if the goal is to alert a researcher that a suspicious episode occurred, but the missing event-level precision prevents a complete event-based comparison.

5.3. Best and Worst Cases

The SciPy Features + Isolation Forest method performed strongly on some UCR files but failed completely on others. Table 13 gives the exact per-file scores underlying those examples.

Table 13. *Selected best and worst per-file results for SciPy Features + Isolation Forest. All worst-case rows have point-adjusted F1 = 0, raw F1 = 0, and no detected event.*

ID	Series	Point-Adj. F1	Raw F1	Event Detected
004	BIDMC1	0.9500	0.1739	Yes
010	CIMIS44AirTemperature6	0.9474	0.1818	Yes
044	PowerDemand1	0.9153	0.4000	Yes
042	Lab2Cmac011215EPG6	0.9130	0.7805	Yes
008	CIMIS44AirTemperature4	0.9091	0.4000	Yes
037	Lab2Cmac011215EPG1	0.9000	0.3077	Yes
028	InternalBleeding17	0.8485	0.5385	Yes
007	CIMIS44AirTemperature3	0.8421	0.4286	Yes
045	PowerDemand2	0.8169	0.3529	Yes
054	WalkingAceleration5	0.7692	0.4762	Yes
046	PowerDemand3	0.0000	0.0000	No
047	PowerDemand4	0.0000	0.0000	No
049	TkeepFirstMARS	0.0000	0.0000	No
050	TkeepForthMARS	0.0000	0.0000	No
055	apneaecg2	0.0000	0.0000	No
056	apneaecg3	0.0000	0.0000	No
057	apneaecg4	0.0000	0.0000	No

058	apneaecg	0.0000	0.0000	No
059	gait1	0.0000	0.0000	No
060	gait2	0.0000	0.0000	No

This variation is important. It shows that SciPy feature-based anomaly detection is not universally reliable across real time-series domains. Some anomaly intervals produce feature deviations that are detectable by the current window-level feature representation, while others do not. This supports the paper's broader conclusion: SciPy features are useful and interpretable, but real scientific anomaly detection requires domain calibration, careful thresholding, and possibly local or task-specific feature design.

6. Discussion

Controlled binary detection was perfect, but mechanism diagnosis reached only macro-F1 0.4578 and the best UCR method reached 0.2884 mean point-adjusted F1. Controlled injection therefore tests whether mechanisms are measurable, not deployment readiness. Real anomalies add heterogeneous shapes, baselines, and thresholding conditions, so scientific use requires domain-specific validation.

6.1. Naive Interpretability Failed

The rule-based system tested whether intuitive groups directly diagnose mechanisms, but achieved chance accuracy because amplitude-energy dominated every average signature. Named features can be understandable in isolation without providing reliable diagnosis.

6.2. Why Global Energy Features Confound Diagnosis

For a clean signal x, perturbation δ, and squared-energy function $E(x) = \sum_t x_t^2$, the energy change is

$$E(x+\delta) - E(x) = 2\sum_t x_t \delta_t + E(\delta). \tag{3}$$

Spikes, oscillations, and shifts often increase energy, while attenuation can decrease it. Absolute deviations in RMS, variance, and energy therefore respond to many mechanisms: they detect magnitude change without identifying its cause.

6.3. SciPy Features Still Preserve Mechanism Information

Full SciPy features nevertheless outperformed the raw-signal baseline (macro-F1 0.4578 versus 0.2051), suggesting that mechanism information is multivariate and overlapping rather than confined to one group.

6.4. Domain Generalization Is Partial

LODO results confirm domain dependence: spectroscopy already contains peaks and vibration already contains oscillations, making corresponding anomalies ambiguous. SciPy features are a useful representation, not universal anomaly signatures.

7. Reproducibility

All generation, feature extraction, model training, evaluation, and UCR windowing code is available in a public executable Colab notebook at https://colab.research.google.com/drive/1dEcrG2ynX3k_dAHSWQmvjt42jhMRYc8S?usp=sharing; the submitted local copy is `anamolydetection.ipynb`. The experiments used Python with NumPy [7], SciPy [2], scikit-learn [8], pandas [9], and Matplotlib [10]. A fixed random seed of 42 was used for controlled signal

generation, train-test splitting, and model initialization where supported. Each controlled signal contained 512 time points. The controlled experiment used six generated scientific signal domains, five injected anomaly mechanisms, seven severity levels, four noise levels, and 20 anomaly repeats per setting. It contained 16,800 anomalous and 16,800 independently generated clean traces. The UCR validation used sliding windows of length 128 with step size 16, and test windows were labeled anomalous if at least 10% of the window overlapped the labeled anomaly interval.

8. Limitations

The synthetic generators and single-mechanism perturbations do not reproduce all instrument faults, combined anomalies, missingness, or nuisance shifts. The controlled binary task is too easy, and the all-clean baseline used for relative signatures leaks test-distribution information. The controlled supervised and UCR one-class scores are complementary rather than directly comparable. The UCR archive does not replace instrument-specific validation, and the saved predictions cannot support event-level precision.

9. Future Work

Future work should fit every transformation on training data, use harder and overlapping perturbations, and validate on fault-labeled instruments. A two-stage system could localize a suspicious interval before computing mechanism-specific features, preventing short events from being diluted across 512 points. User studies should test whether explanations help identify mechanisms, and limited-data comparisons should include deep models.

10. Conclusion

SciPy features perfectly separated controlled clean and anomalous signals, but mechanism diagnosis remained difficult and rule-based explanations performed at chance. Mean point-adjusted F1 of 0.2884 on UCR further limits the claim. SciPy features are a transparent baseline, not a complete solution; scientific workflows must distinguish detection, diagnosis, and real-data transfer.

References

[1] R. Wu and E. J. Keogh, "Current Time Series Anomaly Detection Benchmarks are Flawed and are Creating the Illusion of Progress," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 3, pp. 2421–2429, 2023, doi: 10.1109/TKDE.2021.3112126.

[2] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *2008 Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17.

[4] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the Support of a High-Dimensional Distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001, doi: 10.1162/089976601750264965.

[5] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection." 2016. doi: 10.48550/arXiv.1607.00148.

[6] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data," *Proceedings of the VLDB Endowment*, vol. 15, no. 6, pp. 1201–1214, 2022, doi: 10.14778/3514061.3514067.

[7] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[8] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011, doi: 10.48550/arXiv.1201.0490.

[9] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[10] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

Open Access | https://doi.org/10.25080/ktck6444

Right Predictions, Wrong Reasons: Explanation Drift Monitoring in Production

Avik Basu [1,2]

[1]Intuit, Inc., [2]PyOpenSci

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Machine learning models in production can keep making correct predictions while silently changing the reasoning behind them. Because traditional monitoring watches input distributions and evaluation metrics, this shift often goes undetected until performance visibly degrades. We describe *explanation drift monitoring*: tracking the distribution of per-feature SHapley Additive exPlanations (SHAP) attributions over time and alerting when it moves. It is a complement to input feature drift and performance monitoring, not a replacement, and is most valuable where ground-truth labels arrive with a long delay. We describe the signals that make it measurable: per-feature Population Stability Index (PSI), a multivariate adversarial-validation score, and changes in SHAP attributions. We relate these to three failure patterns. Two case studies on the UCI Adult Income dataset delimit what the signal adds. Finally, we present `shap-monitor`, an open-source Python package implementing this workflow with low instrumentation overhead, and discuss the practical challenges of running it in production.

Keywords explainability, SHAP, model monitoring, drift detection, production machine learning

1. Introduction and Motivation

A model that keeps reporting good accuracy is easy to trust and easy to ignore. Yet a model can produce the *right predictions for the wrong reasons*. In other words, it can hold its headline metrics steady while the features driving its decisions change underneath it. When that happens, the model has quietly become a different model. A model whose behavior was never validated will eventually fail, and the eventual failure tends to be sudden and hard to diagnose.

By *wrong* we do not mean less accurate. We mean reasoning that no longer matches what was validated at deployment. A model whose attributions have moved is answering for reasons nobody reviewed, and in a regulated setting that is a problem independent of whether the headline metric has moved at all.

Most production ML monitoring is built around two families of signal, and both share a blind spot. The first is **performance monitoring**, which includes metrics such as accuracy, F1, AUC, and business KPIs. However, these metrics require the ground truth labels, which are often delayed by weeks or months. The second is **input drift monitoring**, which monitors incoming feature distributions. It needs no labels, but it answers *did the data change?* rather than *did the model's use of the data change?*.

This paper is about a third signal, **explanation drift monitoring**, which measures the change over time in *how* a model attributes its predictions to input features. We measure it by logging per-feature SHAP attributions [1] for sampled production traffic and tracking their distributions over time. When the model starts relying on different features, or relying

Published Jul 13, 2026

Correspondence to
Avik Basu
avikbasu93@gmail.com

Open Access

Copyright © 2026 Basu. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

on the same features in a different direction, the SHAP distributions move and can be detected without waiting for labels. Our contributions are:

1. A delineation of *when* explanation drift carries information that input-drift monitoring does not, and when it merely restates it. We support this with two case studies chosen to fall on opposite sides of that line: one in which input monitoring detects the shift at least as strongly as the explanation signal, and one in which input monitoring is structurally blind to it.
2. A description of concrete, low-overhead detection signals, including per-feature PSI, a multivariate adversarial-validation score, and attribution magnitude/rank/sign changes, and how to interpret them.
3. A mapping from these signals to three failure patterns that recur in production: concept drift, accuracy-preserving model-version regressions, and data-pipeline errors invisible to input monitoring.
4. An open-source library, `shap-monitor` [2], which allows users to monitor explanation drift for their models and use-cases.

2. Background

2.1. SHAP attributions

SHAP (SHapley Additive exPlanations) assigns each feature of a single prediction a contribution value derived from the Shapley value of cooperative game theory [3], distributing the gap between a model's output and a baseline expectation across the input features [1]. For a prediction $f(x)$, the attributions ϕ_i satisfy an additive decomposition,

$$f(x) = \phi_0 + \sum_{i=1}^{M} \phi_i, \tag{1}$$

where ϕ_0 is the base (expected) value and ϕ_i is the contribution of feature i. A positive ϕ_i pushes the prediction above the baseline; a negative one pulls it below. For tree ensembles, these attributions can be computed exactly and efficiently [4], which makes per-prediction explanation practical at production volumes.

For monitoring we are not interested in any single explanation, which is the usual per-prediction use [5]. We are interested in the *distribution* of ϕ_i for each feature across many predictions, and in how that distribution evolves over time. Two summaries are central. The **mean absolute attribution** $\mathbb{E}[|\ \phi_i\ |]$ is a global importance measure that quantifies how much feature i moves predictions, regardless of direction. The **mean signed attribution** $\mathbb{E}[\phi_i]$ captures the typical *direction* of the feature's effect. A change in either of these, in absolute terms or relative to the other features, is a change in the model's reasoning.

2.2. What existing monitoring catches, and what it misses

In practice, input drift monitoring tests each incoming feature's distribution against a reference window with a univariate statistic such as PSI or a Kolmogorov-Smirnov (KS) test, and tools such as Evidently [6] and Alibi Detect [7] make this routine. What neither family observes is the model's decision process itself. That process is a function of both the inputs and the learned model, so it can move when no single feature distribution has moved far enough to alarm and no label has yet arrived to contradict it.

2.3. Related work

Explanation distributions have been proposed as a shift-detection signal before, and we build directly on that work. Mougan et al. introduced *explanation shift*, showing on tabular data that a classifier trained to separate reference-period from current-period SHAP vectors

can be a more sensitive indicator of a performance-relevant shift than detectors operating on the inputs themselves [8], [9]. The adversarial-validation construction we describe in Section 5 is essentially their Explanation Shift Detector, and we claim no novelty for it. Attribution drift is also monitored in at least one production system: Amazon SageMaker Model Monitor compares the ranking of global SHAP importances against a training baseline using a normalized discounted cumulative gain score, and alerts when it falls below a threshold [10].

What we add on top of the related work here is threefold. First, an explicit account of *which* production failures this signal can and cannot catch, in particular separating the common case where explanation drift merely re-expresses an input shift that ordinary monitoring would already flag from the case where it is the only signal available. Second, a set of signals read *together*: per-feature PSI to localize, an adversarial score for joint shift, and magnitude, rank, and sign summaries to characterize the change, where SageMaker's monitor uses rank alone and the explanation-shift literature uses the joint classifier alone. Third, an open-source implementation that logs attributions from a live prediction path into a queryable store and is not tied to a managed platform.

3. Explanation drift

We define **explanation drift** as a statistically detectable change, over time, in the distribution of a model's SHAP attributions on production inputs. It is distinct from input feature drift in an important way. Input drift asks whether $P(X)$ has changed. Explanation drift asks whether $P(\phi(X))$ has changed, where ϕ is the explanation function induced by the *current model*. The attribution distribution couples the data and the model, so explanation drift can surface three ways: covariate shift, concept drift, or a change in the model itself. Major failure patterns develops the latter two as production failure patterns.

Crucially, none of these requires labels to detect, which makes explanation drift a *leading* signal in delayed-label regimes. Explanation drift is a complement, not a replacement. It narrows the blind spot between the other two signals rather than superseding either. Because attributions are per-feature, it also points at *which* part of the reasoning moved, which is a strong lead for root-cause analysis.

4. Detecting explanation drift

Given logged attributions for a reference period and a current period, several signals make drift measurable. They differ in whether they look at one feature at a time or at the joint distribution, and they are most useful read together.

4.1. Per-feature distribution shift (PSI)

The Population Stability Index is a long-standing measure of how much a distribution has moved between two samples, widely used in credit risk to monitor scorecards [11]. It bins a reference distribution, compares the proportion of mass each bin receives under the current distribution, and sums the divergence:

$$\mathrm{PSI} = \sum_{b=1}^{B} (c_b - r_b) \ln \frac{c_b}{r_b}, \tag{2}$$

where r_b and c_b are the reference and current proportions in bin b. Applied to the SHAP values of a feature, PSI quantifies how much that feature's *contribution* distribution has shifted. The conventional banding (below 0.1 is stable, 0.1–0.25 warrants investigation, above 0.25 indicates significant shift) was developed for raw scorecard distributions, and we carry it over to attribution distributions unchanged; it is a useful default rather than a

calibrated result, and Section 12 records an important caveat on reading large values. PSI is cheap, interpretable, and per-feature, which makes it a good first-pass alarm and a good way to localize drift to specific features.

4.2. Multivariate shift (adversarial validation)

Per-feature tests miss joint shifts, i.e., combinations of attributions can move even when no single feature's marginal PSI looks alarming. *Adversarial validation* captures this. We label reference-period attribution vectors as class 0 and current-period vectors as class 1, train a simple classifier to tell them apart, and measure its cross-validated AUC. Adversarial validation is a general technique [12]; applying it to attribution vectors as a drift detector is due to Mougan et al., who introduce it as the *Explanation Shift Detector* [8], [9]. An AUC near 0.5 means the two periods are statistically indistinguishable; an AUC approaching 1.0 means the model's reasoning in the two periods is easily separable, i.e., strong evidence of drift. The classifier's feature importances additionally rank *which* attribution dimensions drive the separation, giving a multivariate complement to per-feature PSI.

4.3. Magnitude, rank, and sign changes

Beyond distributional distance, three interpretable summaries describe the *nature* of a shift. The **change in mean absolute attribution** says whether a feature became more or less influential overall. The **change in importance rank** says whether the model's priority ordering of features was reshuffled. And a **sign flip**, i.e., a change in the typical direction of a feature's contribution, means a feature that used to push predictions one way now pushes them the other. Sign flips on important features are a high-signal indication that relationships learned during training no longer hold.

5. Major failure patterns

Explanation drift is worth monitoring because it catches failure modes that the other signals are structurally poor at catching. Three recur often enough to be worth naming.

5.1. Concept drift: same features, shifted relationships

In concept drift the relationship between features and target changes while the inputs may look similar [13]. A feature that was once weakly predictive becomes strongly predictive, or reverses direction. Input monitoring can be quiet, because the marginal feature distributions need not move much. Performance monitoring eventually catches it, once labels arrive. Explanation drift sits in between: as the model encounters inputs whose learned relationships no longer match reality, its attribution magnitudes and signs shift, and PSI on the affected SHAP distributions rises before the labels confirm the damage.

5.2. Model-version regression: same accuracy, different reasoning

Model retraining and redeployment is routine, and validation usually gates on aggregate metrics. A new version can match the old one's accuracy on a holdout set while arriving at its predictions through materially different reasoning, e.g., leaning on a leak-prone proxy feature, say, or down-weighting a feature the business considers essential. Because the headline metric is preserved, this passes standard checks silently. Comparing the SHAP attribution distributions of the two versions on the *same* inputs surfaces it directly: a high adversarial-validation AUC between versions, or large per-feature rank and sign changes, reveals that the reasoning regressed even though the accuracy did not.

5.3. *Data-pipeline errors invisible to input monitoring*

Some of the most damaging production incidents are upstream data bugs: a feature silently defaulting to a constant, a units change, a join that drops rows, a transformation applied in the wrong order. When such a bug pins a feature to a plausible value, its marginal distribution can stay within range and input monitoring stays quiet, but the model's *use* of that feature collapses or distorts, which the attribution distribution registers immediately. Because explanation drift is per-feature, it both detects the problem and points at the feature whose pipeline to inspect.

6. Case studies

We report two studies, chosen to sit on opposite sides of the question the previous section raises: *when does watching attributions tell you something that watching inputs would not?* Study A is a covariate shift, the scenario in which explanation drift is most often demonstrated. It is also, as we show, one in which input monitoring detects the same event at least as decisively. Study B removes input drift entirely by construction, leaving explanation drift as the only label-free signal available.

Both use the UCI Adult Income dataset [14], a standard tabular benchmark predicting whether annual income exceeds \$50K, and a LightGBM classifier [15] with exact attributions from `TreeExplainer` [4]. Every number, table, and figure below is regenerated by the two notebooks in `examples/paper_notebooks/` of the `shap-monitor` repository [2], one per study. Both fetch the dataset directly, so no manual data preparation is required.

Because attributions carry sampling variance, every adversarial-validation figure we report is the mean over 20 independent subsamples of 4,000 rows per window, with intervals giving the 2.5th and 97.5th percentiles across those repetitions; PSI intervals come from 200 bootstrap resamples. We deliberately subsample *without* replacement. A with-replacement bootstrap duplicates rows, and because the adversarial AUC is cross-validated, a duplicated row can appear in both a training and a validation fold; the classifier then memorizes it. That artifact inflated a genuine no-drift control from 0.50 to roughly 0.61 in our first attempt, and anyone reproducing this measurement should be aware of it.

6.1. *Study A: a stable score over a shifting population*

We train on the younger population (age ≤ 45) and treat the older population (age > 45) as drifted production traffic, mirroring a model developed on one population and deployed to serve another. The reference window is the held-out validation split ($n = 10,290$); the current window is the older population ($n = 14,544$).

Performance barely moves. F1 is 0.710 on the reference data and 0.721 on the drifted data. Watching F1 alone, and recalling that in production those labels would not exist for weeks, nothing would prompt an investigation.

The attribution signal fires, and so does input monitoring. The adversarial-validation AUC over SHAP attributions rises from 0.499 [0.489, 0.512] on a no-drift control to 0.967 [0.965, 0.971] between the two periods. The same statistic computed on the *raw inputs*, however, reaches 1.000 [1.000, 1.000]. On this shift, input-drift monitoring is not merely also triggered: at the level of the joint distribution it separates the two periods perfectly, and more decisively than the explanation signal does. Figure 1 shows this in its left panel.

A deliberate demographic shift is exactly the regime input monitoring was designed for, and explanation drift adds no *global* detection power here. Taken alone, Study A establishes the weaker of the available claims: that a model's attribution behavior can change substantially while its accuracy does not.

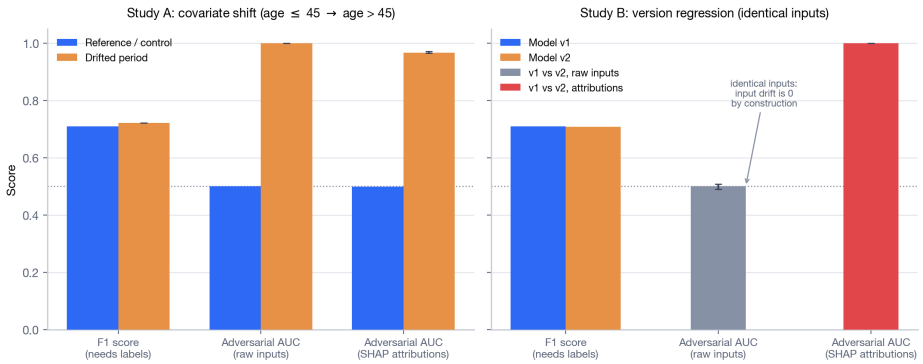


Figure 1. *Two regimes for the same signal.* ***Left (Study A):*** *under a demographic covariate shift the model's F1 is flat (0.710 arrow.r 0.721), so performance monitoring sees nothing, but* both *the raw-input and the SHAP adversarial scores detect the shift, the raw inputs more strongly (1.000 vs. 0.967). Explanation drift adds no global detection power in this regime.* ***Right (Study B):*** *two model versions scored on identical inputs. Raw-input drift is zero by construction and the raw adversarial score sits at chance (0.500), while the attribution distributions are perfectly separable (1.000) at effectively unchanged accuracy. Error bars are 2.5–97.5 percentile intervals over 20 repeated subsamples.*

The per-feature view is where the two signals disagree. Figure 2 plots each feature's PSI in attribution space beside its PSI in input space, and the aggregate picture above conceals a systematic divergence.

For four features the input distribution is quiet while the attributions are not. The clearest is `race`: its raw PSI is 0.008, far below the 0.1 "warn" band, while its attribution PSI is 0.752, far above the 0.25 "alert" band, a ratio of roughly 91. The racial composition of the traffic barely moved; the model's *use* of race changed substantially. `native-country` shows the same pattern (0.426 vs. 0.038), as do `hours-per-week` (0.118 vs. 0.008) and `capital-loss` (0.176 vs. 0.000). An input-drift dashboard would show nothing actionable on any of these four; in attribution space all four clear the warn threshold and `race` and `native-country` clear the alert threshold.

This is the sense in which the predictions here are right for the wrong reasons. The model's F1 *improved* slightly across this shift, so nothing in the performance signal argues for intervention, and the racial composition of the traffic barely moved, so nothing in the input signal does either; yet the model's reliance on `race` shifted by two orders of magnitude more than that composition did. Whether the change is harmful is a question for review; that it would have gone unreviewed is the failure.

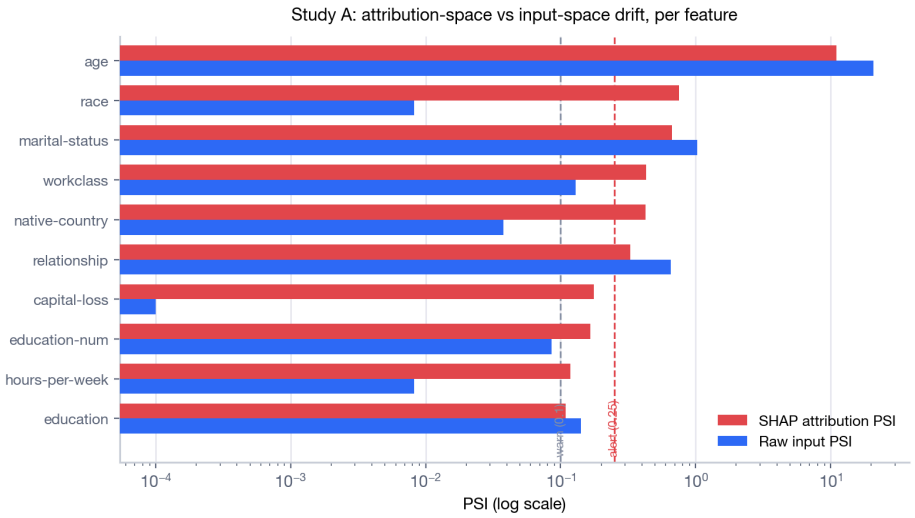


Figure 2. *Per-feature PSI in attribution space (red) against input space (blue), reference vs. drifted period, log scale. Dashed guides mark the conventional 0.1 (warn) and 0.25 (alert) thresholds. The two views agree on the features the shift acted on directly (`age`, `marital-status`, `relationship`, all larger in input space), and disagree sharply on `race`, `native-country`, `hours-per-week`, and `capital-loss`, where the inputs are stable but the model's use of them is not. Four features falling below the warn band in both views (`capital-gain`, `fnlwgt`, `occupation`, `sex`) are omitted, since neither kind of monitoring would act on them.*

`shap-monitor` reports a +33% change in `age` mean absolute attribution, +82% for `workclass`, +51% for `capital-gain`, and sign flips on six features including `marital-status` and `workclass`, meaning their typical contribution direction reversed.

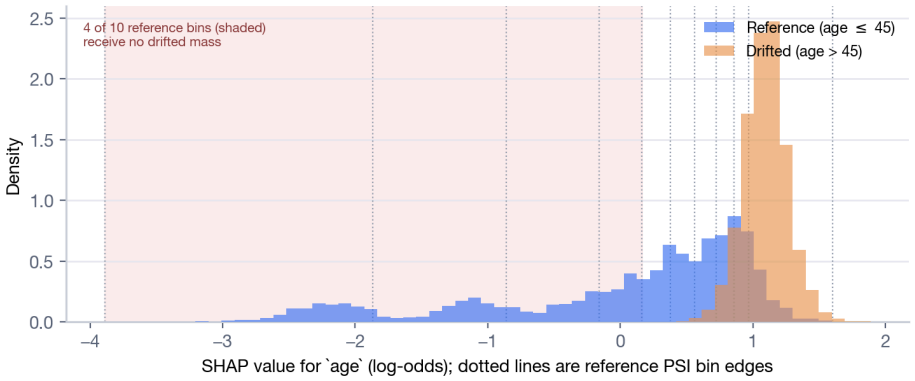


Figure 3. *Distribution of `age` SHAP values, reference (blue) vs. drifted (orange). The reference distribution is broad and bimodal; under the shift it collapses to a tight, strongly positive cluster, so `age` becomes a consistent large positive driver for the older population. Dotted lines are the ten reference quantile bin edges PSI is computed over, and the shaded span marks the four bins that receive no drifted mass at all, the mechanism behind the inflated PSI discussed in Section 12.*

6.2. Study B: same inputs, same accuracy, different reasoning

We reuse Study A's training split, the same 24,008 younger-population training rows, and fit several model versions on it, then score every version on the same 10,290 validation rows. The older population plays no part here. Because all versions consume byte-identical inputs, per-feature input PSI is exactly 0 for all fourteen features under *every* construction, and the adversarial score on the raw inputs sits at chance, 0.500 [0.490, 0.509]. This is not a small number that might have been larger under a different shift; there is no input difference

to detect. For calibration, the same no-shift measurement in attribution space gives 0.499 [0.489, 0.512].

Reporting a single constructed regression would invite the objection that it is the one setting where the signal happens to work. We therefore report a family of four version pairs, summarized in Table 1, spanning a true null control, an un-engineered change of training configuration, and two deliberate feature deprioritizations implemented with LightGBM's per-feature split-gain penalty.

Table 1. *Four version-2 constructions, all scored on the same 10,290 rows as version 1 (F1 = 0.710). Raw-input PSI is exactly zero for every feature in every row, so input monitoring cannot fire. "Flagged" counts features whose attribution PSI reaches the 0.25 alert band.*

Name	v2 construction	F1	ΔF1	Agreement	SHAP AUC	Flagged
Control	identical retrain	0.710	+0.000	100%	n/a	**none**
Retune	new hyperparameters	0.695	-0.014	98.2%	1.00	10 of 14
Demote-rel	`relationship` denied split gain	0.709	-0.001	98.7%	1.00	3
Demote-edu	`education-num` denied split gain	0.705	-0.004	98.7%	1.00	2

Control stays silent, and accuracy monitoring passes everything else. Refitting with identical data and settings reproduces version 1 exactly. Across the three genuine regressions, meanwhile, the largest F1 change is 1.4 points and the smallest is under a *tenth* of a point, so a gate on aggregate metrics admits all of them.

Demote-rel relocates reasoning without disturbing accuracy. Here F1 moves by -0.001 while the attribution structure is rearranged: `relationship` collapses from a mean absolute attribution of 0.579 to 0.000 and falls from rank 3 to rank 14, its correlated substitute `marital-status` absorbs the role, rising from 0.795 to 1.414 (+78%) and from rank 2 to rank 1, and `sex` declines from 0.096 to 0.057. Attribution PSIs are 20.72, 3.14, and 0.99 respectively; every other feature stays below the alert band. The alerting policy would page on exactly the three features involved and stay quiet on the remaining eleven. Figure 4 shows the full picture.

Retune shows that localization degrades when the change is diffuse. It is the least contrived of the three: no penalty is applied, only a different training configuration, which is the most common thing to differ between two production versions. It is detected, but it flags 10 of 14 features, because a broad configuration change moves reasoning broadly rather than relocating one feature's role. It is also the row where F1 drops most (-0.014), so it is the weakest of the three on the accuracy-preservation axis. We report it because its realism is the point, and its diffuseness is a useful warning: the per-feature signals localize a targeted regression well and a systemic one poorly.

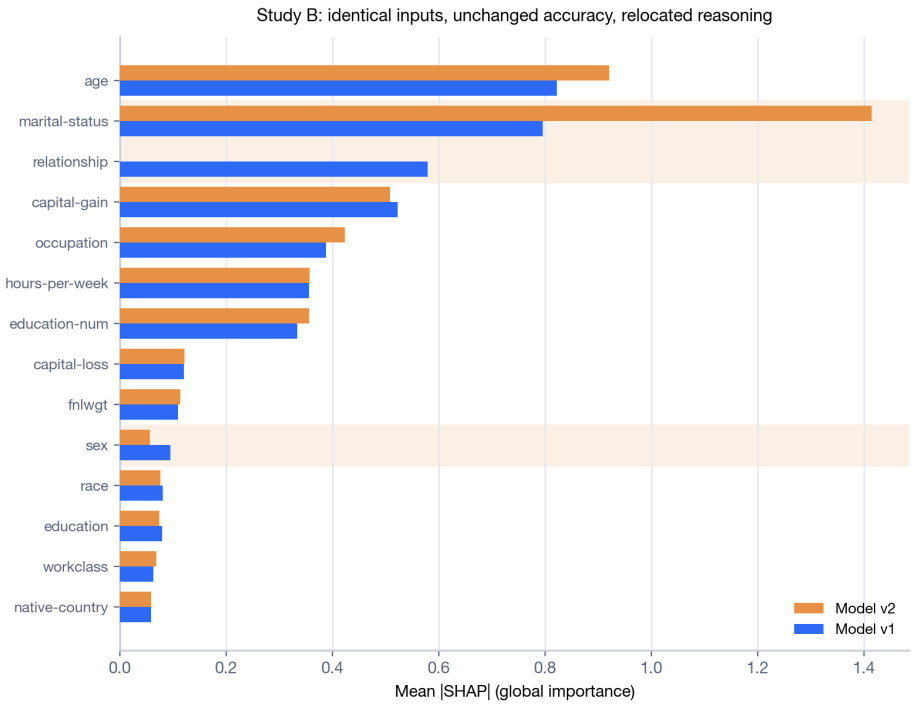


Figure 4. *Study B, `demote-relationship` construction: mean absolute SHAP attribution per feature for version 1 (blue) and version 2 (orange), computed on identical inputs. Shaded rows mark the affected group. `relationship` collapses to zero while its correlated substitute `marital-status` absorbs the role. Overall accuracy and every input distribution are unchanged; the reasoning has been relocated.*

This is the model-version regression of Section 8, demonstrated rather than described, and it is the case in which explanation drift is not a complement to the existing signals but the only label-free signal available at all. The same argument applies to the pipeline-error pattern of Section 9, where a feature pinned to a plausible constant leaves the marginal distribution in range while the model's use of it collapses.

6.3. A caveat on reading large PSI values

The `age` attribution PSI of 11.07 [10.90, 12.49] in Study A deserves comment, because a value two orders of magnitude above the alert threshold invites being read as a magnitude when it should be read as a saturation flag.

`shap-monitor` bins the reference distribution into ten quantile bins, so each reference bin holds about 10% of the mass by construction and the r_b term cannot approach zero. Under the null this behaves well: the largest attribution PSI in our no-drift control is 0.0079. The instability is on the other side of the ratio. When the current distribution collapses into a narrow region, as `age` does in Figure 3, reference bins receive *no* current mass, and c_b must be floored at some ε to keep the logarithm finite. Four of the ten bins are empty here, and they contribute 75% of the total. Since each empty bin contributes roughly $-r_b \ln(\varepsilon / r_b)$, the total is a direct function of that constant: holding the data fixed and varying ε from 10^{-10} to 10^{-3} moves the reported PSI from 11.07 to 4.61.

PSI therefore remains sound for *detection* and for *ranking* features by how far their attributions moved, which is what a monitoring system needs. But a value far above the alert band should be reported as saturated rather than quoted as a quantity, and such values are not comparable across implementations that choose different flooring constants. Where a stable magnitude is wanted, a bounded divergence such as Jensen–Shannon avoids the

artifact entirely; we report PSI here for continuity with the banding practitioners already know.

7. Open-source implementation

`shap-monitor` [2] is an open-source (Apache-2.0) Python package that implements this workflow with a deliberately small surface area: a logger to capture attributions in production, a storage backend, and an analyzer to compute drift. It builds on SHAP [1], scikit-learn [16], [17], NumPy [18], and pandas [19]. The repository's `examples/lightgbm_example.ipynb` introduces the API step by step.

7.1. Logging attributions in production

The `SHAPMonitor` wraps any SHAP explainer and logs attributions for a configurable fraction of traffic. Sampling keeps the overhead bounded: explanation cost scales with the number of explained rows, so a sample rate well below 1.0 is appropriate for high-volume services.

```
from shapmonitor import SHAPMonitor
import shap

monitor = SHAPMonitor(
    explainer=shap.TreeExplainer(model),
    data_dir="/var/log/shap",
    sample_rate=0.05,          # explain 5% of traffic
    model_version="prod-v2.1",
)

# In the prediction path:
predictions = model.predict(X)
monitor.log_batch(X, predictions)   # samples, explains, and persists
```

`log_batch` samples the batch, computes SHAP values for the sampled rows, and persists them. The logging call is best wrapped so that a monitoring failure can never take down the prediction path.

7.2. Storage

Attributions are written as Parquet [20], partitioned by date in a Hive-style layout (`date=YYYY-MM-DD/`) so that time-range queries prune to the relevant partitions, with optional partitioning by model version. Each record carries a timestamp, a batch UUID, the model version, the per-feature SHAP values, and optionally the feature values and predictions. The columnar format keeps storage compact and is directly readable by pandas, DuckDB, and Spark. The storage layer is a small protocol, so a different backend (an object store or a warehouse) can be substituted without touching the logging or analysis code.

7.3. Analyzing drift

The `SHAPAnalyzer` reads logged attributions and computes the signals described above. Comparisons can be made between two time periods, two specific batches, or two model versions:

```python
from datetime import datetime
from shapmonitor.analysis import SHAPAnalyzer
from shapmonitor.backends import ParquetBackend

analyzer = SHAPAnalyzer(ParquetBackend("/var/log/shap"))

# Per-feature PSI, magnitude/rank/sign changes between two windows.
drift = analyzer.compare_time_periods(
    (datetime(2026, 5, 1), datetime(2026, 5, 8)),    # reference
    (datetime(2026, 5, 8), datetime(2026, 5, 15)),   # current
)
alerts = drift[drift["psi"] >= 0.25]          # significant shifts
flips = drift[drift["sign_flip"]]             # reversed contribution direction

# Multivariate adversarial-validation score for the same windows.
adv = analyzer.compare_adversarial(
    (datetime(2026, 5, 1), datetime(2026, 5, 8)),
    (datetime(2026, 5, 8), datetime(2026, 5, 15)),
)
print(adv.attrs["adversarial_auc"])           # 0.5 = no drift, 1.0 = max drift
```

The comparison returns, per feature, its PSI, its mean absolute attribution in each period, the percentage change in importance, the change in importance rank, and whether its contribution direction flipped, exactly the quantities reported in the case studies. To compare two deployed models scored on the same traffic, which is the version-regression pattern of Section 8, `compare_versions` performs the same analysis keyed on the logged model version rather than on time.

7.4. Command line

For operational use the same analysis is available from a command-line interface, which is convenient for cron-driven reports and for piping machine-readable output into an alerting system:

```bash
# Human-readable drift report between two windows.
shapmonitor report drift --data-dir /var/log/shap \
    --ref last-14d..last-7d --curr last-7d..now

# Machine-readable output for an alerting rule.
shapmonitor report drift --data-dir /var/log/shap --json \
    | jq '.features[] | select(.psi >= 0.25)'
```

A simple and effective alerting policy is threshold-based: page when any important feature's SHAP PSI crosses 0.25, or when the adversarial-validation AUC between the current window and a fixed baseline exceeds a chosen level (for example 0.8). Restricting per-feature alerts to features above a minimum mean absolute attribution avoids noisy alarms on features the model barely uses.

8. Challenges and Limitations

Explanation drift monitoring is a useful signal, but it is not free and not a panacea.

Explainer cost. Computing the explanations dominates the cost, not storing or analyzing them. Exact attributions are cheap for tree models, where `TreeExplainer` is fast enough to run inline on a sampled fraction of requests, but expensive for model-agnostic explainers such as `KernelExplainer`. Sampling and out-of-band computation (logging the inputs and explaining them in a periodic batch job, so the prediction path is never blocked) mitigate this, but the cost must be budgeted, and very low sample rates trade statistical power for cheapness.

Baseline choice. Drift is always measured *relative to a reference*. A poorly chosen baseline (too short, unrepresentative, or itself already drifted) produces misleading comparisons. Seasonal traffic in particular calls for a seasonally aware reference rather than a naive trailing window.

Threshold tuning. The PSI bands are conventions, not laws, and the right alerting thresholds depend on the model, the feature, and the tolerance for false alarms. Thresholds should be calibrated against a quiet period and revisited as the system evolves.

Attribution stability and correlation. SHAP attributions can be sensitive to correlated features and to explainer configuration, so a portion of measured drift may reflect attribution variance rather than genuine reasoning change. Adequate sample sizes per window and a fixed explainer configuration reduce, but do not eliminate, this. Every headline figure we report therefore carries an interval over repeated subsamples rather than being quoted as a point estimate.

Reading the adversarial score. Because $\phi(X)$ depends on X, an attribution-space statistic moves whenever the inputs move, so a high adversarial AUC is not on its own evidence of a reasoning change: in Study A the raw inputs are separable at AUC 1.000, and most of the attribution-space separability is inherited rather than added. Reporting the input-space statistic alongside it, as we do throughout, is what disentangles them, and the case for explanation drift as an independent signal rests on Section 11, where the inputs are identical by construction. The score also saturates (a systematic change across ten thousand rows reaches 1.000 whether the underlying regression is total or partial), making it a sensitive detector but a poor measure of severity. The per-feature magnitude, rank, and sign changes should carry the diagnostic weight.

Scope of the evidence. Both case studies use a single public benchmark, and the version regressions in Section 11 are deliberately constructed. Together they establish that this class of failure exists and that the other two monitoring signals are structurally unable to see it; they do not establish how frequently it arises in production, which a public dataset cannot show.

It is a leading indicator, not truth itself. Explanation drift tells you the model's reasoning changed; it does not, by itself, tell you whether that change is harmful. It is a high-quality lead that warrants investigation, ideally triangulated against input drift and, once available, performance, not an automatic trigger for rollback.

9. Conclusion and future work

Production models can keep their accuracy while quietly changing the reasoning behind their predictions, and the standard monitoring stack, input drift plus delayed-label performance, has a structural blind spot for exactly this. Tracking SHAP attribution distributions over time closes part of that gap with a label-free signal that, because it is per-feature, both detects drift and localizes it.

Our two studies delimit where it helps. Under a demographic covariate shift (Section 10), input-drift monitoring separated the two periods more decisively than the attribution view did; what the attribution view added was per-feature, on `race` and three others where the inputs stayed quiet while the attributions crossed the warn band. In Section 11, versions scored on byte-identical inputs preserve accuracy while relocating a rank-3 feature's contribution onto a correlated substitute, so input and performance monitoring are both silent and explanation drift is the only signal that fires. It stays quiet on a true null retrain.

The `shap-monitor` package provides an open-source implementation. Promising directions for future work include asynchronous logging, additional storage backends, and alerting

that fuses explanation drift with input-drift and performance signals into a single, better-calibrated alarm. We hope explanation drift monitoring becomes a standard third pillar of production ML observability, alongside the two it complements.

References

[1] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774. doi: 10.48550/arXiv.1705.07874.

[2] A. Basu, "shap-monitor: Production monitoring of SHAP explanations over time." [Online]. Available: https://github.com/ab93/shap-monitor

[3] L. S. Shapley, *A Value for n-Person Games*. Princeton, NJ: Princeton University Press, 1953, pp. 307–317. doi: 10.1515/9781400881970-018.

[4] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020, doi: 10.1038/s42256-019-0138-9.

[5] A. Basu, "Explaining ML predictions with SHAP: Intuition, use cases, and best practices," in *Proceedings of the 24th Python in Science Conference*, 2025, pp. 14–37. doi: 10.25080/mhum9729.

[6] Evidently AI Team, "Evidently: An open-source framework to evaluate, test and monitor ML and LLM-powered systems." [Online]. Available: https://github.com/evidentlyai/evidently

[7] J. Klaise, A. Van Looveren, C. Cox, G. Vacanti, and A. Coca, "Monitoring and explainability of models in production," in *ICML Workshop on Challenges in Deploying and Monitoring Machine Learning Systems*, 2020. doi: 10.48550/arXiv.2007.06299.

[8] C. Mougan, K. Broelemann, G. Kasneci, T. Tiropanis, and S. Staab, "Explanation Shift: Detecting distribution shifts on tabular data via the explanation space," in *NeurIPS 2022 Workshop on Distribution Shifts: Connecting Methods and Applications*, 2022. doi: 10.48550/arXiv.2210.12369.

[9] C. Mougan, K. Broelemann, D. Masip, G. Kasneci, T. Thiropanis, and S. Staab, "Explanation Shift: How Did the Distribution Shift Impact the Model?." [Online]. Available: https://arxiv.org/abs/2303.08081

[10] D. Nigenda *et al.*, "Amazon SageMaker Model Monitor: A System for Real-Time Insights into Deployed Machine Learning Models," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 3671–3681. doi: 10.1145/3534678.3539145.

[11] N. Siddiqi, *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Hoboken, NJ: John Wiley & Sons, 2006.

[12] J. Pan, "Adversarial Validation, Part One." [Online]. Available: http://fastml.com/adversarial-validation-part-one/

[13] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014, doi: 10.1145/2523813.

[14] B. Becker and R. Kohavi, "Adult." UCI Machine Learning Repository, 1996. doi: 10.24432/C5XW20.

[15] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, 2017, pp. 3146–3154.

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011, doi: 10.48550/arXiv.1201.0490.

[17] L. Buitinck *et al.*, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122. doi: 10.48550/arXiv.1309.0238.

[18] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[19] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, Stéfan van der Walt and Jarrod Millman, Eds., 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[20] Apache Software Foundation, "Apache Arrow and the Parquet columnar storage format." [Online]. Available: https://parquet.apache.org/

Open Access | https://doi.org/10.25080/tcjy7995

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Adapt v0.1: Prototyping a Real-Time, Reproducible Data Analysis Framework for Adaptive Radar Scanning

Bhupendra A. Raut[1], **Siddhant Gupta**[1], **Zach Sherman**[1], **Bobby Jackson**[1], **Joe O'Brien**[1], **Eddie Schuman**[2], **Pawel Lech**[1], **Sherman Beus**[2], **Seth Abernethy**[1], **Scott Collis**[1], **Nicki Hickmon**[1], and **Adam Theisen**[1]

[1]Argonne National Laboratory, Lemont, IL, [2]Pacific Northwest National Laboratory, Richland, WA

Abstract

Adapt is a modular framework for real-time convective cell detection, motion projection, and storm lifecycle analysis, developed to support adaptive radar-scanning operations at the U.S. Department of Energy Atmospheric Radiation Measurement (DOE-ARM) User Facility. Scientific workflows are composed of independently registered modules that communicate through a shared execution context and are assembled at runtime into directed acyclic graphs (DAGs) through automatic dependency resolution. The project aims to enable plug-and-play modules and automated pipeline construction from declared dependencies. A reference NEXRAD pipeline demonstrates the architecture through data acquisition, detection, analysis, projection and tracking. A read-only data-access layer provides unified SQL and NetCDF access to pipeline outputs for real-time and archival analysis. Adapt is being developed toward a reusable ecosystem of scientific and machine-learning modules that can be composed into operational and research pipelines. Planned applications include cloud-based nowcasting dashboards and JupyterHub workflows on DOE-ARM computing infrastructure. We welcome contributions in architecture, validation, software design, and governance to strengthen scalability and maintainability. As AI-assisted development becomes increasingly prevalent, a central objective is to build an architecture that is explicit, verifiable, and resilient while supporting both human developers and coding agents.

Keywords convective storms, weather satellites, remote sensing, cell detection and tracking, nowcasting, data pipelines, scientific workflows

Published Jul 13, 2026

Correspondence to
Bhupendra A. Raut
braut@anl.gov

Open Access

Copyright © 2026 Raut *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Adaptive radar scanning enables real-time response to evolving convective storms by focusing radar on dynamically selected sectors [1]. Traditional scanning strategies must balance spatial coverage, temporal frequency, and vertical resolution under fundamental physics constraints, frequently undersampling rapid microphysical evolution during convective lifecycles. Real-time feedback control offers a pathway to dramatically improve observations of transient phenomena but demands low-latency analysis, reliable decision support, and infrastructure that remains stable under continuous operation [2], [3].

The TITAN system demonstrated operational radar data analysis in real time as early as the 1990s [4]. Modern open-source frameworks such as PyART, tobac, TINT, and CoCo-MET provide powerful scientific algorithms primarily designed for offline batch workflows [5], [6], [7], [8]. Composing them into a reliable, continuously running, operationally maintainable pipeline introduces requirements that go beyond scientific algorithms: the need is

continuous data ingestion, event-driven execution, configurable processing chains without code changes, separation of consumers from producers, and long-term reproducibility.

Adapt aims to address these requirements through a layered architecture in which scientific modules are registered, wired by dependency, and executed by a graph-based orchestrator. The current implementation processes NEXRAD Level-II data in real time or archival mode, writing validated outputs to a structured repository that external consumers access through a read-only API.

This paper presents the design of Adapt for scientific developers seeking to extend the framework with new modules, pipelines, or data consumers. System Design introduces the layered architecture, followed by the module system (Plug-and-Play Module System), execution graph (Directed Acyclic Graph Execution), configuration system (Three-Tier Configuration), and data repository and consumer API (Data Repository and Consumer API). NEXRAD Pipeline presents the reference NEXRAD pipeline, Scientific Python Stack summarizes the scientific Python software stack, and Roadmap and Contribution Opportunities outlines future development and contribution opportunities.

2. System Design

Adapt is structured in layers, each with a single, well-defined responsibility. Layers communicate through explicit interfaces as shown in Figure 1.

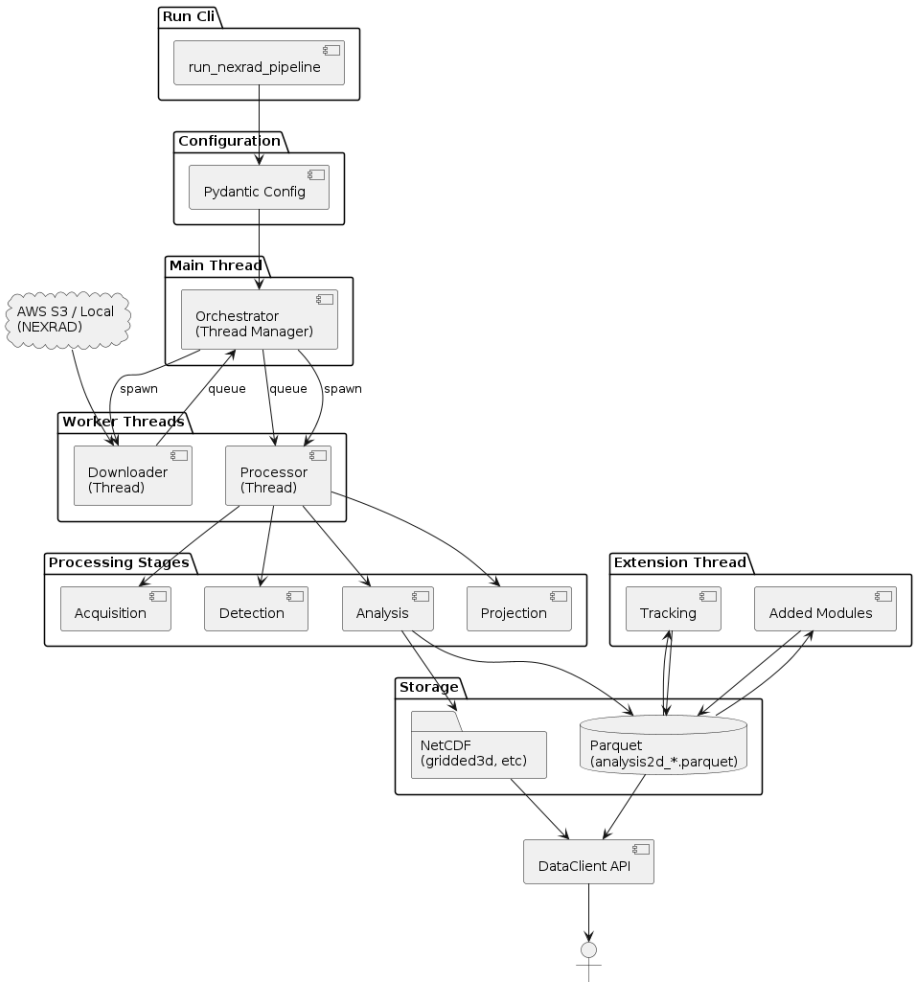


Figure 1. *Adapt execution workflow. A two-thread runtime (downloader and processor) drives a DAG-based execution graph. Scientific modules are registered plugins that communicate through a shared context dictionary. All outputs are written to a structured repository accessed through a read-only consumer API.*

2.1. Layer Responsibilities

Module layer contains all scientific algorithms. Modules are stateless functions wrapped in a standard interface and they do not have I/O, threading, or repository access. Each module declares the named data products it reads from and writes to a shared context dictionary. The graph executor wires and sequences them automatically.

Execution layer hosts `GraphBuilder`, which constructs a DAG from module input/output declarations, and `GraphExecutor`, which traverses the graph in topological order, enforcing contracts at every module boundary. The execution layer is the only layer that directly instantiates modules. It does so through the module registry, not by importing module classes.

Runtime layer coordinates threads and data flow. It starts and stops the downloader and processor threads, owns the inter-thread queue, manages the two-frame rolling buffer required by the projection module, and delegates each file to the pipeline. No scientific logic resides in the runtime layer.

Persistence layer writes and indexes all artifacts to the local data repository. Gridded fields are stored as NetCDF and tabular analysis outputs as Parquet while tracking data and metadata catalogs as SQLite. All file paths are encapsulated within this layer and no other layer constructs paths or directory structure directly.

Configuration layer resolves a three-tier hierarchy into a single immutable `InternalConfig` object at startup. No component reads configuration files at runtime.

Consumer API layer provides a read-only API (`DataClient`) that wraps catalog queries, DuckDB-powered Parquet SQL, and a polling-based streaming interface. Dashboards, notebooks, and autonomous pipelines can use only this layer and they have no access to the pipeline's internal state.

2.2. Dependency Rules

The import rules are enforced by `import-linter` and checked in CI, so a layering violation fails the build even when every unit test passes.

3. Plug-and-Play Module System

The module system will be the primary extension point for scientific developers. New modules can be integrated and tested without modifying any existing source files.

3.1. Module Interface

Every processing module subclasses `BaseModule` and declares four class-level attributes:

```python
class MyModule(BaseModule):
    name = "my_module"
    inputs = ["grid_ds_2d", "config"]
    outputs = ["my_product"]

    def run(self, context: dict) -> dict:
        data = context["grid_ds_2d"]
        result = compute(data)
        return {"my_product": result}
```

Program 1. *Minimal module definition.*

Optionally, a module may declare `input_contracts` and `output_contracts` as dictionaries mapping context keys to validator functions and also their version. The graph executor enforces these contracts automatically. The module author does not call input/output validation logic.

3.2. Registration

Modules register with a global `ModuleRegistry` when their package is imported. Registration records the module class by its `name` attribute. The pipeline configuration specifies which module packages to import, the registry instantiates them, and the graph builder wires them automatically. Adding a new module to a pipeline requires:

1. Implement `BaseModule` in `modules/<domain>/module.py`
2. Currently requires adding the package path to the `pipeline.modules` list in the pipeline's configuration file
3. Optionally declare contracts for boundary validation. These implementations will be made mandatory in the stable release.

No other source files should require modification for adding a module. Replacing an existing module (for example, substituting a deep-learning segmenter for the threshold-based detector) requires only that the replacement produce the same output context key (`segmented_ds`) satisfying the same output contract.

3.3. Scaling to Many Modules

Because modules communicate only through context keys and are wired by the graph builder from their declarations, a large number of modules can coexist in the registry without interfering. Pipeline configurations select which registered modules to activate, and inactive modules have zero runtime cost.

Table 1 lists the modules in the current NEXRAD reference pipeline. New module categories currently under consideration include: wavelet-based cell detection, lightning data ingestion, GOES satellite ingest, nowcasting tendency modules, and alert-generation modules for automated scan-control decisions.

4. Directed Acyclic Graph Execution

4.1. Graph Construction

`GraphBuilder` constructs the execution DAG during pipeline initialisation. It creates a node for each registered module, builds an output map from product names to producing nodes, and wires dependencies by matching each module's declared inputs to entries in the output map. Two checks are enforced at construction time: (1) all declared inputs must have a corresponding producer, and (2) no two modules may declare the same output key. Violations cause startup failure before any processing begins.

4.2. Execution and Context Dictionary

`GraphExecutor` traverses the graph in topological order within a single thread. At each step it selects nodes whose upstream dependencies have completed, invokes the module, and merges returned outputs into the shared execution context dictionary. The context dictionary holds all in-memory data products for one file's execution and is discarded after the file completes.

Input contracts are enforced before each module invocation and output contracts after the module finishes processing. A `ContractViolation` halts processing for the current file and records a diagnostic failure state in the catalog; processing continues with subsequent files.

4.3. Multiple Pipelines

A pipeline is a named configuration that specifies which modules to activate and in what parameter regime. Because module wiring is derived automatically from declarations, a new pipeline definition currently requires only a configuration file listing the desired modules. The same module can participate in multiple pipelines. Currently, the NEXRAD pipeline is the only implementation and we aim to provide a framework for additional pipelines (e.g., for satellite data, multi-sensor fusion, or model data).

This design can scale naturally to tens of pipelines. Users select a pipeline by name via CLI flag or configuration file.

5. Three-Tier Configuration

Pipeline behaviour is controlled through a layered configuration hierarchy. The lowest tier is `ParamConfig`, encoding defaults for every tunable parameter, maintained in a `defaults.yaml`. Users must not edit this file. The middle tier is `UserConfig`, a config file containing only what they wish to override in `ParamConfig`. The highest tier is `CLIConfig`, populated from command-line flags.

At startup, `resolve_config()` merges the three tiers in precedence order (CLI overrides user, user overrides param) and validates the result as a frozen `InternalConfig` object. Unknown fields in user files raise a `ValidationError` immediately. The resolved configuration is stored alongside pipeline outputs, creating a permanent record sufficient to reproduce any run.

This design separates scientific expertise (defaults), deployment preferences (user config), and operational flags (CLI) into independent, composable concerns. A shared configuration file captures pipeline settings and algorithm parameters for reproducibility.

A user config lists the modules to run and overrides only the defaults it cares about; every unspecified parameter falls back to `defaults.yaml`.

```yaml
base_dir: /path/to/repository
mode: historical
source: aws_nexrad
modules:            # dependency order; subset at runtime with --only / --not
  - ingest
  - detection
  - projection
  - analysis
  - tracking
  - cell_volume_stats
regridder:
  grid_shape: [41, 301, 301]
  grid_limits: [[0, 20000], [-150000, 150000], [-150000, 150000]]
  weighting_function: cressman
  save_netcdf: true
```

Program 2. *Excerpt of a user `config.yaml`. Only overrides are listed; all other parameters inherit from `defaults.yaml`.*

The pipeline is generated and run from the command line. `adapt config` writes a template, and `run-nexrad` processes a small historical window for one radar:

```
adapt config                                # write a config.yaml template
adapt run-nexrad config.yaml --radar KHTX \
    --mode historical \
    --start-time 2026-07-24T12:00 --end-time 2026-07-24T13:00
```

Program 3. *Generating a config template and running the NEXRAD pipeline on a historical case.*

CLI flags override the config file, so a single shared config can be reused across runs while `--radar`, the time window, `--only`/`--not` (to activate a subset of modules), and `--max-runtime` (realtime mode) vary per invocation.

6. Contract-Based Validation

Scientific pipelines are susceptible to silent failures when intermediate data products change structure. Adapt addresses this through a contract system that validates data products at module boundaries. Contracts are pure functions in module `contracts.py` files that verify invariants such as: variable presence, array dimensionality, integer label integrity, coordinate consistency, and DataFrame schema completeness.

Each module may declare optional `input_contracts` and `output_contracts` dictionaries that map a context key to a `(validator, version)` pair. The graph executor calls these automatically and the module author does not invoke validation logic. For example, the detection module guarantees that its `segmented_ds` output satisfies the segmentation contract:

```python
from adapt.modules.detection.contracts import check_segmentation

class Detection(BaseModule):
    name = "detection"
    inputs = ["grid_ds_2d", "config"]
    outputs = ["segmented_ds"]
    output_contracts = {"segmented_ds": (check_segmentation, "v1")}
```

Program 4. *A module declares contracts by name; the executor enforces them at the boundary.*

Here `check_segmentation` is a pure function in `detection/contracts.py` that verifies the segmentation invariants (label variable present, integer dtype, non-negative labels, 2-D shape). Because the guarantee is attached to the `segmented_ds` key rather than to the detector implementation, a deep-learning segmenter can replace the threshold-based detector as long as it satisfies the same output contract. Failures raise `ContractViolation` with a message

identifying which contract failed, what was wrong, and which module produced the data. Contract coverage can be introduced incrementally as module interfaces stabilise.

Contracts currently do not validate scientific correctness or statistical properties. As these are monitoring concerns, not interface contracts, checking them can be added as separate layers around the science modules, considering legitimate edge cases such as unusual storm morphologies or degraded radar modes.

7. Data Repository and Consumer API

7.1. Repository Structure

The repository is a structured directory tree combining SQLite catalog tables with per-scan NetCDF and Parquet files, organised under a fixed layout. All artifacts are written through `DataRepository`, the single component that owns this layout, rather than by modules opening files or constructing paths directly. This indirection keeps producers agnostic to storage details and lets the consumer API resolve and read only the fields a query needs. Migrating the underlying store (for example, to a Zarr or Icechunk backend) therefore requires no changes to the science modules. Gridded outputs (radar volumes and segmentation) are stored as NetCDF [9] files per scan with zlib compression. Per-cell statistics are stored as Parquet files for efficient columnar queries. Cell tracking data is stored in per-radar SQLite tables to support row-oriented graph traversal queries (single storm history, recent events, lineage enumeration). We are currently exploring cloud-native data-store strategies such as Zarr, a chunked format for large N-dimensional arrays [10], and Icechunk, a transactional storage engine layered on Zarr, for flexible concurrent read-write and long-term cloud storage capabilities. The repository layout and its access APIs are still stabilising ahead of the v1.0 release, and consumers should expect breaking changes until then.

7.2. Reproducibility and Run Management

Each processing run receives a unique identifier of the form `YYYYMONDD-HHMM-RADAR`. Cell identifiers (`cell_uid`) are deterministic hashes derived from observable properties at initiation time. Hence the same input always produces the same identifiers regardless of when or how many times processing is performed. Reprocessing historical datasets is identical to real-time operation.

7.3. Consumer API

`DataClient` is the sole interface through which downstream systems access pipeline outputs. No consumer has access to `DataRepository`, `RadarCatalog`, or any pipeline internal. The interface provides:

```
client = DataClient("/path/to/output")

# Batch queries via DuckDB over Parquet
df = client.query(
    "SELECT * FROM analysis "
    "WHERE cell_area_sqkm > 50"
)

# Streaming for live dashboards
for df in client.stream(
    "SELECT * FROM cells_by_scan "
    "WHERE time > :last_seen",
    poll_interval=5
):
    update_dashboard(df)
```

Program 5. *Consumer API usage.*

DuckDB executes SQL directly against Parquet files with predicate and projection push-down, making queries over multi-month archives feasible without loading data into memory. The streaming interface yields new rows as scans complete, enabling live dashboards to update incrementally without filesystem access or pipeline coupling. This producer/consumer separation is fundamental to the architecture.

8. NEXRAD Pipeline

The NEXRAD pipeline demonstrates the full architecture end-to-end. It processes NEXRAD Level-II data containing reflectivity, radial velocity, and spectrum width for each radar volume scan [11], through six registered modules (shown in Table 1) and exposes results through `DataClient`. The Level-II archive is publicly hosted on AWS S3 as an Open Data registry dataset (`s3://noaa-nexrad-level2`)[1].

8.1. Two-Thread Runtime

The pipeline operates as two cooperating threads. `AwsNexradDownloader` continuously polls S3 for new Level-II files (realtime mode) or enumerates a specified time range and exits (historical mode). Files are placed in a bounded queue providing back-pressure when the processor is slower than the downloader. `RadarProcessor` dequeues files, checks for duplicates, maintains a two-frame rolling buffer for the projection module, executes the DAG via `NexradPipeline`, and writes results through `RepositoryWriter`. No shared mutable state exists between threads beyond the queue.

Table 1. *Modules in the NEXRAD reference pipeline, in dependency order. Each module reads and writes named products in the shared context; the graph builder derives the execution order from these declarations.*

Module	Inputs arrow.r Outputs	Function and Data Flow
`ingest`	Level-II file arrow.r gridded volume	Downloads Level-II files from AWS S3 to a background queue, decodes them with PyART, and regrids to a Cartesian volume.
`detection`	gridded volume arrow.r labeled cells	Segments 2D grids into labeled storm-cell datasets (`segmented_ds`) via thresholding and connected-component labeling.
`projection`	current + previous labeled cells arrow.r projected boundaries	Uses Farnebäck optical flow on current and past segments to project cell boundaries forward.
`analysis`	labeled cells + projected boundaries arrow.r per-cell statistics	Combines grids and projected boundaries into 2D per-cell statistics.
`tracking`	per-cell statistics arrow.r tracking graph	Links cells across scans, generating tracking graphs and lifecycle (split/merge) events.
`cell_volume_stats`	gridded volume + labeled cells arrow.r 3D cell statistics	Computes 3D volume statistics (e.g. cloud-top height) per cell; requires `regridder.save_netcdf`.

8.2. Runtime Performance

Table 2 reports per-module timings from a sample realtime run over the KHTX radar (`adapt run-nexrad config.yaml --radar KHTX`), processing six volume scans (about 10 MB per

[1] https://registry.opendata.aws/noaa-nexrad/

Level-II file) and detecting 343 cell objects on an Apple M3 Mac with 36 GB RAM. End-to-end processing averaged 19.0 s per scan (max 20.04 s), dominated by ingestion (download, decode, and regridding). NEXRAD volume scans complete roughly every four to six minutes depending on the operational scan strategy, so a per-scan cost of about 19 s leaves ample headroom for real-time operation.

Table 2. *Per-module timings for a six-scan KHTX sample run (343 objects detected).*

Module	Calls	Total (s)	Avg (s)
`ingest`	6	79.0	13.2
`detection`	6	14.2	2.4
`projection`	5	18.8	3.8
`analysis`	5	0.71	0.14
`tracking`	5	1.1	0.22
`cell_volume_stats`	5	21.0	4.2

9. Scientific Python Stack

Xarray. All gridded data products are `xr.Dataset` objects. Spatial dimensions (z, y, x), coordinate information, physical units, and timestamps travel with the data through every module. No function in the pipeline accepts grid spacing, CRS, or coordinate names as arguments; this information must be embedded in the dataset [12].

PyART. NEXRAD Level-II decoding and Cartesian gridding are currently performed through PyART [5]. PyART is the core radar library because it supplies a broad range of radar algorithms except tracking (e.g. I/O, gridding, and classification). Adapt's own detection uses a tobac-like segmentation, and its tracking is an improved variant of TINT [7] adapted for real-time operation. At present, PyART usage is largely isolated to the ingestion and gridding stages. However, PyART is expected to become a foundational dependency within Adapt, with future modules exposing PyART-based radar analysis algorithms directly through configuration. This approach will allow users to construct processing pipelines from a broad range of PyART capabilities without modifying framework code.

Pydantic. Configuration validation and schema enforcement. Expert defaults, user overrides, and CLI flags are resolved and frozen into a single `InternalConfig` at startup. Unknown fields in user configurations raise at startup; type and range violations produce descriptive errors before any processing begins [13].

OpenCV and SciPy. Dense optical flow via the Farnebäck algorithm (OpenCV) for motion estimation. Connected component labeling and morphological operations via `scipy.ndimage`. Linear sum assignment for optimal cell-to-cell matching via `scipy.optimize` [14], [15].

NetworkX. The tracking module maintains a `networkx.DiGraph` of all observed cells and lineage edges across the processing session. This graph computes cell age and dominant lineage edges in split/merge complexes [16].

DuckDB and PyArrow. Per-cell statistics are persisted as Parquet, a columnar on-disk file format for analytical data [17], written via PyArrow, the Python bindings to Apache Arrow's in-memory columnar layout [18]. Downstream queries are executed by an embedded DuckDB instance within `DataClient`. DuckDB is an in-process analytical SQL engine that runs directly against Parquet files [19] and applies predicate and projection pushdown without loading full datasets into memory.

10. Roadmap and Contribution Opportunities

Adapt is an actively developed prototype. Table 3 summarizes what is implemented today, what is experimental, and what is planned, to distinguish the working system from its intended trajectory.

Table 3. *Implementation status of Adapt components.*

Status	Components
Implemented	NEXRAD pipeline (six modules), registry-based module system, dependency-driven DAG executor, three-tier configuration, read-only `DataClient` access layer, opt-in boundary contracts, `postprocess` command for repository enrichment, and `cell_volume_stats` 3D statistics (cloud-top height).
Experimental	Migration of the data repository to a Zarr/Icechunk cloud-native store.
In progress	Tracking of connected groups of cells (storm complexes).
Planned	Offline post-processing engine for after-run analysis; parallel satellite and radar pipelines with product merging; ML-based nowcasting trained on archived Adapt output; 3D modules for segmentation and storm analysis; mandatory contracts at the stable release.

The first stable release (v1.0) will establish the core architectural foundations of Adapt including a registry-based module system, dependency-driven DAG execution, configuration-defined pipelines, and a stable data-access layer. Current development focuses on simplicity, composability, and maintainability so that future capabilities can be added without changes to the framework core.

At this stage, our highest priority is architectural feedback. We welcome design reviews, validation strategies, testing frameworks, and software engineering practices that improve scalability, correctness, extensibility, and long-term sustainability.

After v1.0, community contributions will expand toward new modules, algorithms, pipeline definitions, data products, and user-facing applications.

10.1. Architecture and Agentic Development

Adapt is being developed at a time when scientific software is increasingly created and maintained using AI-assisted and agentic workflows. To preserve architectural integrity, the project already employs automated checks including `pytest`, `import-linter`, static type checking with `mypy`, and code-quality tools. These tools provide a first line of defense against architectural drift and unintended design violations introduced during rapid development.

We seek contributions on additional mechanisms for architecture validation and governance, including interface contracts, architecture-aware testing, machine-readable design specifications, dependency analysis, automated design-rule enforcement, and other approaches that make architectural constraints explicit and verifiable.

10.2. Pipeline Composition and Orchestration

Adapt constructs execution graphs from declared module dependencies, but pipeline definitions remain largely static. Future work will explore more flexible pipeline composition from available modules, data products, and instrument platforms. Contributions are encouraged in dependency resolution, workflow planning, execution scheduling, and multi-instrument workflows.

10.3. Cloud and Interactive Applications

Future deployments include cloud-hosted nowcasting services, web-based dashboards, and JupyterHub environments for scientific analysis. Contributions in deployment architecture, distributed execution, cloud storage, and data-serving infrastructure are welcome.

11. Summary

Adapt establishes the architectural foundations for a modular framework supporting adaptive radar-scanning operations. Independently developed modules are composed automatically through dependency-driven DAG execution, enabling extensible and reproducible scientific workflows.

The reference NEXRAD pipeline demonstrates the architecture from data acquisition to cell tracking, while the consumer API provides a stable interface for downstream analysis and applications. Future development will focus on strengthening the architecture, expanding the module ecosystem, and supporting new scientific workflows and instrument platforms.

Code and documentation are available at https://github.com/ARM-DOE/adapt.

Acknowledgments

This research was supported by the Atmospheric Radiation Measurement (ARM) User Facility, a U.S. Department of Energy Office of Science user facility managed by the Biological and Environmental Research Program. Argonne National Laboratory's work was supported by the U.S. Department of Energy, Office of Science, Office of Biological and Environmental Research, under Contract DE-AC02-06CH11357.

The work benefited from interactions with Sean Shahkarami, Sean Freeman, Brenda Dolan, Michael Giansiracusa, and Scott Giangrande. The authors also thank Jim Mather, Jennifer Comstock, Giri Prakash, and the DOE-ARM radar community for their support and feedback. Valuable discussions with the tobac development team and the Open Radar community also contributed to the direction of this work.

References

[1] M. Oue, S. M. Saleeby, P. J. Marinescu, P. Kollias, and S. C. Van Den Heever, "Optimizing radar scan strategies for tracking isolated deep convection using observing system simulation experiments," *Atmospheric Measurement Techniques*, vol. 15, no. 16, pp. 4931–4950, 2022, doi: 10.5194/amt-15-4931-2022.

[2] K. Lamer, P. Kollias, E. P. Luke, B. P. Treserras, M. Oue, and B. Dolan, "Multisensor agile adaptive sampling (MAAS): A methodology to collect radar observations of convective cell life cycle," *Journal of Atmospheric and Oceanic Technology*, vol. 40, no. 11, pp. 1509–1522, 2023, doi: 10.1175/JTECH-D-22-0148.1.

[3] S. Gupta and A. Theisen, "A Cloud-Tracking Data Set for the CSAPR2 Adaptive Scanning during TRACER," techreport DOE/SC-ARM-TR-324, 2025. doi: 10.2172/3001951.

[4] M. Dixon and G. Wiener, "TITAN: Thunderstorm identification, tracking, analysis, and nowcasting—A radar-based methodology," *Journal of Atmospheric and Oceanic Technology*, vol. 10, no. 6, pp. 785–797, 1993, doi: 10.1175/1520-0426(1993)010<0785:TTITAA>2.0.CO;2.

[5] J. J. Helmus and S. M. Collis, "The Python ARM Radar Toolkit (Py-ART), a library for working with weather radar data in the Python programming language," *Journal of Open Research Software*, vol. 4, 2016, doi: 10.5334/jors.119.

[6] G. A. Sokolowsky *et al.*, "tobac v1.5: Introducing fast 3D tracking, splits and mergers, and other enhancements for identifying and analysing meteorological phenomena," *Geoscientific Model Development*, vol. 17, no. 13, pp. 5309–5330, 2024, doi: 10.5194/gmd-17-5309-2024.

[7] B. A. Raut, R. Jackson, M. Picel, S. M. Collis, M. Bergemann, and C. Jakob, "An adaptive tracking algorithm for convection in simulated and remote sensing data," *Journal of Applied Meteorology and Climatology*, vol. 60, no. 4, pp. 513–526, 2021, doi: 10.1175/JAMC-D-20-0119.1.

[8] T. Hahn, H. Weiner, C. Brooks, J. X. Li, S. Gupta, and D. Wang, "CoCoMET v1.0: a unified open-source toolkit for atmospheric object tracking and analysis," *Geoscientific Model Development*, vol. 18, no. 18, pp. 5971–5996, 2025, doi: 10.5194/gmd-18-5971-2025.

[9] R. Rew and G. Davis, "NetCDF: an interface for scientific data access," *IEEE Computer Graphics and Applications*, vol. 10, no. 4, pp. 76–82, 1990, doi: 10.1109/38.56302.

[10] J. Moore and S. Kunis, "Zarr: A cloud-optimized storage for interactive access of large arrays," in *Proceedings of the Conference on Research Data Infrastructure*, 2023. doi: 10.52825/cordi.v1i.285.

[11] NOAA National Weather Service (NWS) Radar Operations Center, "NOAA Next Generation Radar (NEXRAD) Level 2 Base Data." NOAA National Centers for Environmental Information, 1991. doi: 10.7289/V5W9574V.

[12] S. Hoyer and J. Hamman, "xarray: ND labeled Arrays and Datasets in Python," *Journal of Open Research Software*, vol. 5, no. 1, 2017, doi: 10.5334/jors.148.

[13] P. K. Narayanan, "Getting Started with Data Validation using Pydantic and Pandera," in *Data Engineering for Machine Learning Pipelines: From Python Libraries to ML Pipelines and Cloud Platforms*, Springer, 2024, pp. 163–196.

[14] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[15] G. Farnebäck, "Two-frame motion estimation based on polynomial expansion," in *Scandinavian Conference on Image Analysis*, 2003, pp. 363–370. doi: 10.1007/3-540-45103-X_50.

[16] A. Hagberg, P. Swart, and D. S Chult, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, Pasadena, CA, USA, 2008. doi: 10.25080/TCWV9851.

[17] D. Vohra, "Apache Parquet," in *Practical Hadoop Ecosystem: A Definitive Guide to Hadoop-Related Frameworks and Tools*, Berkeley, CA: Apress, 2016, pp. 325–335.

[18] G. Lentner, "Shared memory high throughput computing with Apache Arrow," in *Practice and Experience in Advanced Research Computing 2019: Rise of the Machines (learning)*, 2019, pp. 1–2. doi: 10.1145/3332186.3333156.

[19] M. Raasveldt and H. Mühleisen, "DuckDB: an embeddable analytical database," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 1981–1984. doi: 10.1145/3299869.3320212.

Open Access | https://doi.org/10.25080/khdg8264

SciPy 2026
July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Voice Capture, Structured Extraction, and Multi-Source Routing: Building and Evaluating a Field-Inspection Assistant in Python

Carolyn Olsen[1]

[1]Independent Researcher

Abstract

Field workers in agriculture, ecology, equipment inspection, and clinical settings record observations with their hands full and their attention on the task, then need to combine what they recorded with authoritative reference knowledge to decide what to do next. We present HiveGuide, an open-source Python application built for beekeeping that captures spoken hive-inspection observations, extracts them into a typed schema, and answers questions using a multi-source retrieval-augmented generation (RAG) assistant drawing on both the beekeeper's own structured records and an authoritative document corpus. Building the assistant surfaced an under-studied design problem, *source selection*: for each question, should the system query the structured database, the document corpus, or both? We share an empirical comparison of seven routing strategies across retrieval accuracy, response quality, latency, cost, and reliability on 501 labeled queries, used to inform the app's routing strategy. No single strategy wins: a supervised classifier maximizes accuracy (97.8%) but needs labeled data; an embedding similarity router reaches 87% with no training; heuristics are cheapest and fastest; a large language model (LLM) classifier produces the most grounded answers; and agentic routing retrieves the most relevant context but fails on roughly half of queries. We distill these trade-offs into a deployment-priority decision framework. HiveGuide is a beekeeping application as-is, but its components are domain-agnostic and it is meant to be forked: practitioners in agriculture, field science, and beyond can swap in their own schema, corpus, and routing keys.

Keywords retrieval-augmented generation, multi-source RAG, query routing, structured extraction, scientific software, FastAPI, pgvector, field inspection, beekeeping

Published Jul 13, 2026

Correspondence to
Carolyn Olsen
carolyn.olsen@gmail.com

Open Access

Copyright © 2026 Olsen. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

A beekeeper inspecting a hive is holding a frame covered in two thousand bees, wearing propolis-sticky gloves, and trying to remember what they will need to write down later: the weather, whether the queen was laying, how many frames held brood, the hive's weight, a few small hive beetles sighted, and what to do before the next visit. Voice memos capture audio but produce no queryable data; note apps require typing, later (if you remember). Neither yields structured records that can be analyzed or compared against expert guidance.

This constraint is not specific to beekeeping. Crop scouts, ecological surveyors, HVAC and wind-turbine technicians, and clinicians on rounds all work with their hands occupied and limited ability to interact with a device. Their decisions depend on *both* their own observation history *and* domain expertise, and they all need output that is structured for later

analysis. Generic tools (voice memos, note apps, general-purpose chat assistants) miss these constraints.

We built **HiveGuide**, an open-source Python application that targets this setting directly: it captures spoken observations, extracts them into a typed schema, and provides an assistant that answers questions by combining the user's structured records with an authoritative document corpus. We call this combination (voice capture, structured extraction, and multi-source RAG) the **capture-extract-retrieve pattern**. Beekeeping is our case study because source credibility matters (extension-service guidance, not arbitrary web text) and because answering a question such as *"Is my hive's weight normal for October in Wisconsin?"* genuinely requires both personal data (what did my hives weigh?) and reference knowledge (what should a hive weigh going into winter?).

Building the assistant surfaced a problem that turns out to be the crux of multi-source RAG: **source selection**. Given a question, should the system query the structured database, the document corpus, or both? This is distinct from two well-studied problems: *model selection* (which LLM to generate with) and *adaptive retrieval* (whether to retrieve at all). It is comparatively under-evaluated, and we therefore treat source selection as a research question in its own right and compare seven routing strategies in a single controlled, reproducible evaluation.

The study, though, is in service of a larger goal. We release HiveGuide as a **forkable template** for field-inspection assistants: a practitioner in agriculture, ecology, or equipment inspection can adopt the capture-extract-retrieve pattern wholesale, swap in their own schema and corpus, and use our results to pick a router for their own data. A forker thus inherits both the code and a decision procedure: the application demonstrates the capture-extract-retrieve pattern, and the study that follows is the evidence behind its single hardest design decision.

Contributions.

1. **HiveGuide**, an open-source Python application demonstrating the capture-extract-retrieve pattern for field inspection, built on FastAPI, PostgreSQL/pgvector, and LangChain.
2. A design pattern for **reliable LLM extraction** into a typed schema using Pydantic constraints, optional-by-default semantics, and reject-and-retry.
3. An **empirical comparison of seven source-selection strategies** across accuracy, quality, latency, cost, and reliability, with the evaluation harness, the 501 queries, and the frozen results released for reproducibility.
4. A **deployment-priority decision framework** and a domain-agnostic template others can port to their own schema, corpus, and routing keys.

2. Related Work

Multi-source RAG systems increasingly operate over heterogeneous backends: SQL databases, document corpora, knowledge graphs, web search. Within this space, *source selection* (which backend to query) is distinct from *model selection* (which LLM to use) and *adaptive retrieval* (whether to retrieve), and it is the least systematically evaluated of the three.

Recent multi-source architectures emphasize *coordination* rather than comparison of routing mechanisms. HM-RAG [1] decomposes queries across modality-specific retrievers and votes over results; HydraRAG [2] adds an agentic source selector with cross-source verification; RAS [3] builds query-specific knowledge graphs. These systems use LLM agents to select sources but do not benchmark alternative routing approaches.

The routing literature that *does* compare strategies largely targets model selection. RouteLLM [4] and Hybrid LLM [5] train routers to send queries to a strong or weak model, and RAGRouter [6] routes among retrieval-augmented models, but none addresses which *data source* to query. Work on adaptive retrieval (Adaptive-RAG [7] and Self-RAG [8]) asks *whether* to retrieve; its finding that universal retrieval underperforms adaptive strategies motivates our always-both baseline.

Agentic approaches treat retrieval as dynamic tool use: the ReAct loop [9], corrective retrieval in CRAG [10], and the patterns catalogued in a recent agentic-RAG survey [11]. This work explores agent *design* rather than comparing agents against non-agentic routers, which our two agentic strategies do directly. Finally, RAG evaluation has matured through RAGAS [12] and related frameworks [13], [14], [15], [16], though latency and cost remain under-reported relative to quality [17]. We adopt RAGAS quality metrics and add explicit latency and cost, comparing seven source-selection strategies in a single controlled setting.

3. The HiveGuide system

HiveGuide has three components (Figure 1): voice **capture** with structured extraction, **ingestion** of an authoritative corpus, and a multi-source **assistant**. The backend is FastAPI [18] on PostgreSQL [19] with the pgvector extension [20]; orchestration uses LangChain [21]; the client is a React Native (iOS) app; and all model calls are routed through OpenRouter: GPT-4o for intent classification, `gpt-oss-120b` for generation, and `text-embedding-3-large` for vectors. These three lanes are the units a forker keeps; only their domain-specific contents change from one field to the next (Table 1).

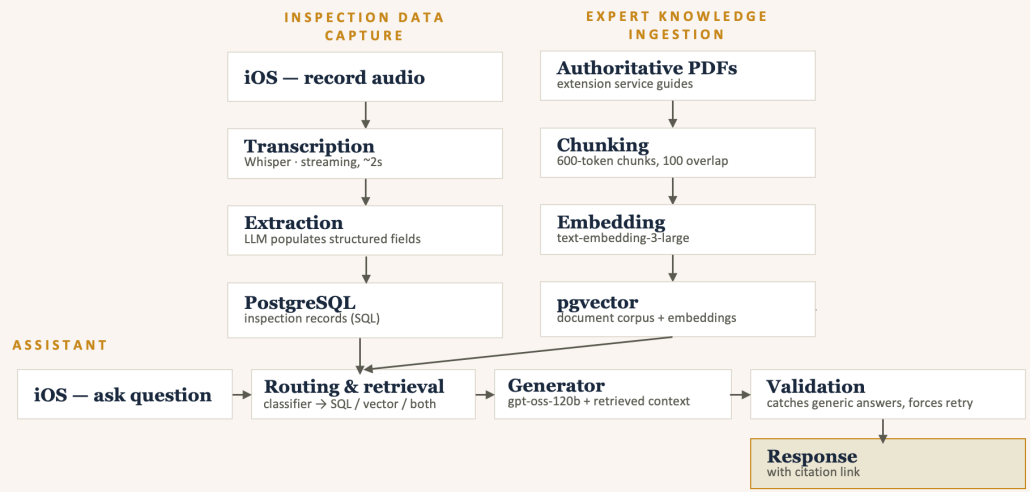


Figure 1. *HiveGuide architecture. The* capture *lane streams audio from iOS, transcribes it (∼2 s latency), and extracts typed fields into PostgreSQL. The* ingestion *lane chunks authoritative PDFs and embeds them into pgvector. The* assistant *lane classifies each question, routes it to the structured store, the vector store, or both, and generates a cited answer with* `gpt-oss-120b`.

Capture. The iOS client streams audio over a WebSocket in two-second chunks (48 kHz, 16-bit mono) and receives incremental transcriptions from OpenAI Whisper (`whisper-1`) at roughly two seconds of latency. The completed transcription is then converted into a typed inspection record (next section). Each record is stored in PostgreSQL with a hive reference, timestamp, the raw transcription and notes, the structured observation fields (weather, temperature, queen/eggs/larvae/capped-brood visibility, laying pattern, activity level), and extracted action items.

Ingestion. The corpus is eight authoritative beekeeping references from cooperative-extension services and similar bodies, for example the Virginia Cooperative Extension varroa and small-hive-beetle guides, a Maine Department of Agriculture fall-management guide, and a Center for Rural Affairs seasonal guide. Sources were chosen so their licenses permit reuse (Creative Commons Attribution, public domain, or explicit permission), which we record alongside each document. Documents are split into overlapping chunks (600/100, via LangChain's `RecursiveCharacterTextSplitter`), embedded with `text-embedding-3-large` (3072 dimensions), and stored in pgvector.

Assistant. A question is classified into an intent, routed to the structured store, the vector store, or both, and answered by `gpt-oss-120b` over the retrieved context with a citation link back to the source document. Personal data is fetched through a `get_user_hive_data` tool that queries the user's own hives over a configurable window (default one year) and formats the matching inspections as context; document retrieval ranks chunks by cosine similarity and returns the top ten (displaying up to five sources). A lightweight validation step rejects generic, ungrounded answers and forces a retry.

Work required to fork for other disciplines: The pipelines and the routing interface can run unchanged. Forkers will need to swap out the schema, the corpus, and the routing keys, as well as retouch a short list of tools and labels: the SQL query, the heuristic cues, the hive as the inspected unit, so they point at the new domain (Table 1).

Table 1. *What a forker keeps, swaps, and retouches.*

Adaptation	Components
Keep as-is	Capture, ingestion, and assistant lanes; routing to the structured store, the vector store, or both; validation harness
Swap	Extraction schema (change from queen/eggs/brood to relevant inspection fields); document corpus (change from extension PDFs to relevant authoritative sources); routing keys (labeled example queries for the embedding or supervised router)
Retouch	Personal-data SQL tool (`get_user_hive_data`); heuristic cues and LLM few-shot examples; hive entity model; rule-based extraction fallback; iOS labels and pickers

4. Reliable structured extraction

The value of voice capture depends on extraction producing data that downstream queries can trust. We enforce this with a Pydantic [22] schema rather than free-form JSON. The request model declares typed fields, closed vocabularies as regex constraints, and, crucially, every observation field as `Optional`:

```
class InspectionCreateRequest(BaseModel):
    hive_id: int
    transcription: str = ""
    notes: str = ""
    weather: Optional[str] = Field(
        None, pattern="^(sunny|cloudy|partly_cloudy|rainy|snowy)$")
    temperature: str = ""
    queen_visible: Optional[bool] = None
    eggs_visible: Optional[bool] = None
    larvae_visible: Optional[bool] = None
    capped_brood_visible: Optional[bool] = None
    laying_pattern: Optional[str] = Field(
        None, pattern="^(poor|patchy|solid)$")
    activity_level: Optional[str] = Field(
        None, pattern="^(low|average|high)$")
```

Three decisions make the output reliable. **Closed vocabularies as constraints**: `laying_pattern` must be `poor`, `patchy`, or `solid`, so the model cannot invent a value and

downstream queries need not fuzzy-match. **Optional by default**: "I didn't see the queen" is not the same as "the queen is absent," so unobserved fields are `None` rather than `False`; preserving nulls keeps later analysis honest. **Reject-and-retry**: the Pydantic constraints reject any value outside its enumerated vocabulary at the API boundary, and the LLM extraction retries up to twice on malformed output before falling back to a rule-based parser, so malformed extractions never reach the database.

As an illustration, the spoken note *"About 65 and cloudy. Didn't spot the queen, but there are eggs and a nice solid brood pattern, lots of bees. I need to check for varroa next time."* extracts to `weather="cloudy"`, `temperature="65"`, `eggs_visible=True`, `laying_pattern="solid"`, `activity_level="high"`, and an action item *"check for varroa"*, while `queen_visible` remains `None`, because the beekeeper said they did not *see* the queen, not that the queen is absent. This pattern (typed schema, optional semantics, validation-driven retry) is the reusable core of the capture pipeline and is independent of the beekeeping vocabulary. It is also what turns spoken notes into analyzable data: each inspection lands as a typed, timestamped row that ordinary SQL or dataframe tools can query directly, rather than as text to be parsed later.

5. The routing problem

Once inspections are structured and a corpus is indexed, the assistant faces two sources of truth. Some questions need only **personal** data from PostgreSQL ("Which of my hives are underweight?"); some need only **documents** from pgvector ("What are signs of varroa?"); and some need **both** ("Is my hive's weight normal for October in Wisconsin?"), comparing personal data against authoritative norms. This is the situation of any scientist whose measurements live in a database and whose domain literature lives in PDFs. We frame source selection as classifying each query into one of three intents (`personal_only`, `documents_only`, or `both_combined`) and routing accordingly.

Critically, *all* classification-based strategies share a single routing-and-generation pipeline; they differ only in how they produce the intent label. This makes the comparison a controlled experiment: the retrieval and generation code is held constant.

```
def query_with_simple_llm(self, question, user_id, routing, ...):
    user_data_text = None
    if routing in ("personal_only", "both_combined"):
        user_data_text = UserHiveDataTool(user_id)._run()      # SQL

    document_context = None
    if routing in ("documents_only", "both_combined"):
        document_context = self._run_retrieval(question)       # pgvector

    # Build a prompt from whichever contexts were retrieved,
    # then generate the answer with gpt-oss-120b.
    return self._generate(question, user_data_text, document_context)
```

As a concrete example, the question *"What are small hive beetles and where did they originally come from?"* is classified `documents_only`; the assistant queries only pgvector, retrieves the Virginia Cooperative Extension *Small Hive Beetle* guide (cosine similarity 0.70) as the top chunk, and `gpt-oss-120b` answers that the beetle (*Aethina tumida*) is native to sub-Saharan Africa, with a citation link back to that document. By contrast, *"Is my hive's weight normal for October in Wisconsin?"* routes to `both_combined`, drawing the hive's recorded weights from PostgreSQL and seasonal norms from the corpus before generating a single grounded answer.

6. Routing strategies

We compare seven strategies for producing the intent label.

- **Heuristic.** Hand-coded keyword rules (Table 2): personal cues such as “my” or “which of my” route to `personal_only`; reference cues such as “what are” or “how to” route to `documents_only`; ambiguous queries default to `both_combined`.
- **Embedding similarity.** A RouteLLM-style k-NN router (shown below) over 20 labeled example queries per category (60 vectors) drawn from the training split, using `k=3` neighbors tuned on a 99-query held-out split (20% of the 499-query training split, stratified by intent, seed 42). The 501-query evaluation set is not used to choose `k`.
- **Supervised.** A LightGBM [23] classifier (`num_leaves=31`, `learning_rate=0.05`, 100 boosting rounds) trained on TF-IDF features (unigrams and bigrams, up to 5000, via scikit-learn [24]) of the 499-query training split with a stratified 75/25 split (`train_classifier.ipynb`; artifacts in `models/`).
- **LLM classifier.** GPT-4o with few-shot prompting; HiveGuide’s deployed default. Label log-probabilities are recorded as a confidence score for evaluation but are not used for routing.
- **Agent discretion.** A ReAct agent with SQL and vector-search tools, free to decide which to call (max 10 iterations).
- **Agent + intent tool.** The same agent, additionally able to call a `classify_user_intent` tool when uncertain (invoked on 12% of queries).
- **Always-both.** A baseline that queries both sources for every question.

Table 2. *Heuristic routing rules (abbreviated).*

Cue words in query	Routed intent
“my”, “our”, “I”, “which of my”	personal_only
“what are”, “how to”, “signs of”	documents_only
mixed or ambiguous	both_combined

The embedding router embeds the incoming query and, for each intent category, averages the top-k cosine similarities (computed with NumPy [25]) to that category’s labeled examples (the RouteLLM approach of using k neighbors rather than a single nearest match), then routes to the highest-scoring category:

```
def classify_intent(self, query, k=None):
    if k is None:
        k = self.default_k                          # k=3, tuned on 99-query held-out split
    query_emb = self.embed_query(query)
    category_scores = {}
    for category, data in self.index["categories"].items():
        sims = np.array([
            self.cosine_similarity(query_emb, ex_emb)
            for ex_emb in data["embeddings"]
        ])
        top_k = np.sort(sims)[-k:]                  # top-k per category
        category_scores[category] = np.mean(top_k)
    return max(category_scores, key=category_scores.get)
```

All four classifier strategies share the same generation step (`gpt-oss-120b`, temperature 0.3); the two **agentic** strategies instead use a LangChain tool-calling agent (`max_iterations=10`) over the `get_user_hive_data` and `search_beekeeping_documents` tools.

7. Evaluation methodology

Dataset. We seed a test apiary of 4 hives with 26 inspection records spread over roughly eight months, and index the 8-source document corpus. Queries were synthesized with an LLM (`gpt-oss-120b`) along a path that fixes each query's ground-truth intent by construction: `documents_only` queries are generated from sampled PDF chunks, `personal_only` queries from the SQL schema, and `both_combined` queries from few-shot examples. The 1,000 generated queries were split, stratified by intent, into a 499-query **training split** (used to train the supervised classifier and to supply the embedding router's example queries) and a 501-query **test split** (167 per intent category) that serves as the evaluation set; no query row appears in both splits, though some question texts are duplicated across them (see Limitations). Each query record carries its intent, the expected source documents, and, where applicable, the specific ground-truth chunk(s) it should retrieve (encoded as `filename#chunk_position`).

Metrics. Retrieval **Hit@3/Hit@5** counts a query as a hit when at least one of its annotated ground-truth chunks appears among the top-k retrieved chunks; retrieval ranks the corpus by cosine similarity and returns the top ten. Queries without chunk-level ground truth (most `personal_only` queries) are excluded from the hit-rate denominator. Response quality uses two **RAGAS** metrics (context relevance and response groundedness, 0-1) judged reference-free by Claude Haiku 4.5 at temperature 0; context relevance is computed only for queries that retrieved document context. We use Haiku because it is inexpensive and because it comes from a different model family than the GPT-4o classifier and the `gpt-oss-120b` generator, so the judge is not scoring outputs from its own family. End-to-end **latency** is measured with `perf_counter` and includes retries, and **cost** combines per-model token usage at list prices for GPT-4o, `gpt-oss-120b`, and the Claude Haiku 4.5 judge. A query counts as an **error** when the strategy returns no usable answer after retries: the fallback message, an agent max-iterations stop, or an exception.

Run conditions. All strategies share the same generation model (`gpt-oss-120b`) and run over a four-worker thread pool; query sampling uses a fixed seed (42); each query allows up to two retries on an empty or failed response; and results are checkpointed so a run can resume.

Reproduction. The harness exposes each strategy through a small `ValidationStrategy` interface, so new strategies subclass it and register in a registry. The full comparison runs from three commands:

```
# 1. Seed a test apiary (4 hives, 26 inspections)
python -m validation.seed_validation_data --email you@example.com --password ...
# 2. Compare strategies on the committed evaluation queries (the 501-query test split)
python validation/run_strategy_validation.py \
    --queries-file validation/queries/questions_ground_truth_test.csv \
    --strategy heuristic_classifier --strategy supervised_classifier \
    --email you@example.com
```

To build a fresh query set for a new domain instead, `validation/generate_queries.py` regenerates the query pool with ground truth. Re-running the full evaluation requires API keys; for readers who only want the numbers, frozen per-strategy results and the summary table are committed under `validation/results/final/`.

8. Results

Figure 2 plots the central trade-off (Hit@5 against p95 latency), and Table 3 and Table 4 give the full picture. Absolute hit rates are low by construction and should be read comparatively across strategies (see Limitations). The headline is that **no single strategy dominates**.

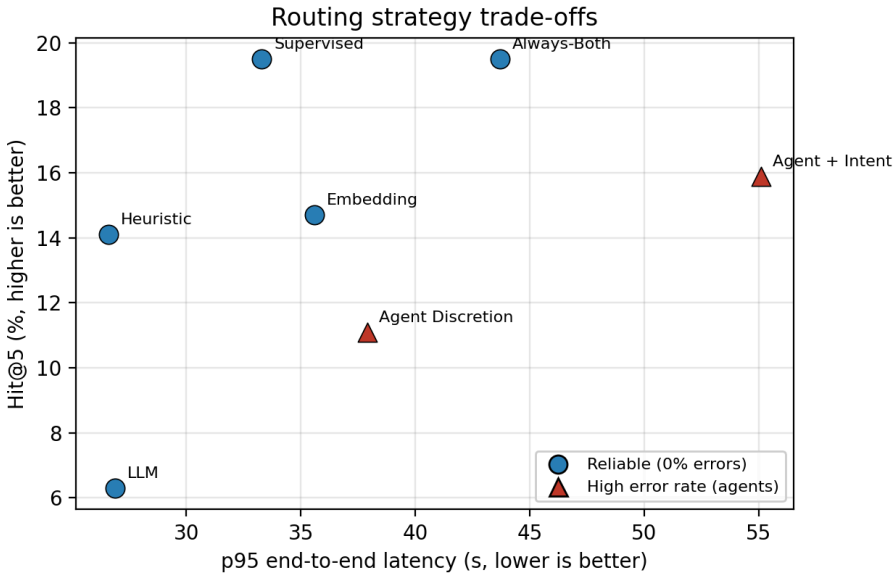


Figure 2. *Routing strategy trade-offs. Reliable classification strategies (circles) cluster at low latency; the supervised classifier matches the always-both baseline on Hit@5 at lower latency. Agentic strategies (triangles) are slower and, more importantly, fail on a large fraction of queries.*

Table 3. *Summary metrics across 501 queries. Cost is the total for one full evaluation run (501 queries). An error is a query with no usable answer (fallback message, max-iterations stop, or exception). Bold marks the best value per column among the zero-error strategies.*

Strategy	Hit@5 %	Class. Acc. %	p95 s	Run cost USD	Error %
Heuristic	14.1	73.1	**26.6**	**1.39**	0.0
LLM	6.3	54.9	26.9	1.54	0.0
Supervised	**19.5**	**97.8**	33.3	1.46	0.0
Embedding	14.7	87.0	35.6	1.44	0.0
Agent Discretion	11.1	n/a	37.9	1.27	54.7
Agent + Intent	15.9	n/a	55.1	1.70	49.3
Always-Both	**19.5**	n/a	43.7	1.71	0.0

Table 4. *Response quality (RAGAS, 0-1, higher is better; bold marks the best value per column). Groundedness averages over all 501 queries, with failed queries scoring 0 (success-only agent values appear in the text). Context relevance is computed only for queries that retrieved document context.*

Strategy	Context relevance	Groundedness
Heuristic	0.417	0.524
LLM	0.363	**0.648**
Supervised	0.472	0.513
Embedding	0.390	0.511
Agent Discretion	0.588	0.106
Agent + Intent	**0.723**	0.142
Always-Both	0.388	0.583

Classification accuracy tracks retrieval. The supervised classifier reached 97.8% accuracy and the best Hit@5 (19.5%), matching always-both while avoiding unnecessary retrieval. The embedding router achieved 87.0% with no model training (only labeled

example queries), making it attractive when training data is scarce. Heuristics captured surprising signal (73.1%) from keyword patterns alone. The two numbers measure different steps: routing accuracy is whether we queried the right source; Hit@5 is whether the vector store then returned the annotated chunk. Supervised routing is nearly perfect and still ties always-both at 19.5%, so the ceiling is retrieval, not source selection.

The LLM classifier underperformed at routing. Despite semantic capability, it scored only 54.9% accuracy, with just 11.9% recall on `documents_only`: it was biased toward `personal` and `combined` and missed pure-document questions, which explains its low Hit@5 (6.3%). Yet its *answers* were the most grounded (0.648): when it did retrieve, responses stayed close to the evidence. That groundedness, together with its zero error rate, is why it remains HiveGuide's deployed default despite its routing accuracy.

Agents trade reliability for context quality. The two agentic strategies achieved the highest context relevance (0.723 and 0.588), but over small, survivor-biased denominators: relevance is scored only for queries that both succeeded and retrieved document context (119 and 114 queries, respectively). Their groundedness in Table 4 (0.142 and 0.106) averages over all 501 queries with failures scored 0; restricted to successful queries, groundedness rises to 0.280 and 0.233, still the lowest of any strategy. In short, the agents fetched relevant context but failed outright on roughly half of queries (49.3% and 54.7%), and even their successful answers were the least supported by retrieved evidence. This points to a fix (separating tool-selection from a constrained response-drafting step) at a latency and cost penalty.

Evaluation cost dominates system cost. RAGAS judging accounted for roughly 70-96% of the $1.27-$1.71 total cost of each strategy's evaluation run; the system itself cost only $0.06-$0.51 per 501-query run, a fraction of a cent per query. For production monitoring, sampling 1-10% of queries would cut evaluation cost by one to two orders of magnitude.

From these results we distill a deployment-priority decision framework (Table 5). It is the practical payoff for a forker: source selection is the one knob the template leaves open, and the right setting depends on your latency budget and whether you have labeled data, not on the domain itself.

Table 5. *Deployment-priority decision framework.*

If you optimize for	Use	Caveat
Speed and cost	Heuristic	Lowest routing accuracy among reliable strategies (73.1%)
Maximum accuracy	Supervised classifier	Requires labeled queries
Semantics without training	Embedding router	Needs ∼20 labeled examples per intent
Grounded, traceable answers (HiveGuide's deployed choice)	LLM classifier	Weak at routing documents-only questions
Complex multi-step queries	Agentic routing	Fails on roughly half of queries (see Limitations)

9. Limitations

Absolute Hit@5 values are low (6-19%) because ground truth requires retrieving *specific* annotated chunks from a small corpus under strict top-k matching; the metric is best read comparatively across strategies rather than as end-task accuracy.

Agent failures shape the agent quality metrics. Under our error definition (no usable answer after retries), agent discretion failed on 54.7% of queries and agent + intent on 49.3%,

mostly from exhausting the 10-iteration budget on a mid-tier open-source model and from tool-response parsing errors. Context relevance is consequently computed over a smaller, survivor-biased sample, and the groundedness means in Table 4 include failed queries scored 0 (success-only values appear in the Results). With a stronger model, better prompt tuning, and more engineering, the agents, and the LLM classifier's routing accuracy, could improve; for a single-developer side project, the LLM classifier was good enough.

The evaluation and training queries were generated together and split by row rather than by text: no query row appears in both splits, but the LLM generator produced duplicate question texts (concentrated in the documents-only category), so about 30% of evaluation questions share their exact text with a training-split question. This affects only the two learned routers; the other strategies, and all retrieval, quality, latency, and cost metrics, involve no training on queries. As a robustness check, restricting to the 352 evaluation queries whose text does not appear in the training split (macro-averaging accuracy over intents, since the duplicates are unevenly distributed) leaves the supervised classifier essentially unchanged (97.7% vs. 97.8%) and lowers the embedding router modestly (84.3% vs. 87.0%), so the duplication does not drive our conclusions.

The reported numbers also come from a single pass over the 501 queries rather than averaged repeats, so small differences between adjacent strategies should not be overinterpreted; the large gaps that drive our conclusions (for example supervised vs. LLM routing accuracy, or agent error rates) are well outside plausible run-to-run noise. Finally, results come from one domain and depend on hosted model APIs; the frozen artifacts allow inspection without rerun, and the harness is designed to be re-run on other domains.

10. Conclusion: fork it

Building a practical field-inspection assistant led us to a research question: when a system has both structured and unstructured sources, *where* to query is a design decision distinct from model selection and from whether to retrieve. Across seven strategies there is no universal winner, only strategies that fit particular deployment priorities, and in consequential domains, traceable citations are how a RAG system earns the right to give advice.

That study exists to support an artifact we want others to use. The capture-extract-retrieve pattern is not specific to beekeeping: it fits any setting where a practitioner records observations in the field and must weigh them against authoritative reference knowledge. Adapting HiveGuide is the split in Table 1: swap your **schema** (replace queen/eggs/brood with the fields your inspections capture), your **corpus** (swap extension PDFs for your authoritative sources), and your **routing keys** (relabel example queries for the embedding router or retrain the supervised classifier), and retouch the few tools and labels that still say hive. The capture, ingestion, and assistant lanes and the evaluation harness stay the same. So we release the entire open-source application together with the harness and the queries and frozen results behind this study, not just the headline numbers. Fork HiveGuide, swap in your schema, corpus, and routing keys, and ship it for your own field.

References

[1] P. Liu *et al.*, "HM-RAG: Hierarchical Multi-Agent Multimodal Retrieval Augmented Generation," in *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, Dublin, Ireland, 2025. doi: 10.1145/3746027.3754761.

[2] X. Tan *et al.*, "HydraRAG: Structured Cross-Source Enhanced Large Language Model Reasoning," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*, Suzhou, China, 2025, pp. 14442–14470. doi: 10.18653/v1/2025.emnlp-main.730.

[3] P. Jiang *et al.*, "RAS: Retrieval-And-Structuring for Knowledge-Intensive LLM Generation," *arXiv preprint*, 2025.

[4] I. Ong *et al.*, "RouteLLM: Learning to Route LLMs with Preference Data," in *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[5] D. Ding *et al.*, "Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[6] J. Zhang, X. Liu, Y. Hu, C. Niu, F. Wu, and G. Chen, "RAGRouter: Learning to Route Queries to Multiple Retrieval-Augmented Language Models," in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025.

[7] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, Mexico City, Mexico, 2024, pp. 7036–7050. doi: 10.18653/v1/2024.naacl-long.389.

[8] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[9] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023.

[10] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective Retrieval Augmented Generation," *arXiv preprint*, 2024.

[11] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," *arXiv preprint*, 2025.

[12] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL 2024)*, St. Julians, Malta, 2024, pp. 150–158. doi: 10.18653/v1/2024.eacl-demo.16.

[13] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, Mexico City, Mexico, 2024, pp. 338–354. doi: 10.18653/v1/2024.naacl-long.20.

[14] T. Ding, A. Banerjee, L. Mombaerts, Y. Li, T. Borogovac, and J. P. De la Cruz Weinstein, "VERA: Validation and Evaluation of Retrieval-Augmented Systems," *arXiv preprint*, 2024.

[15] A. Salemi and H. Zamani, "Evaluating Retrieval Quality in Retrieval-Augmented Generation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024)*, Washington, DC, USA, 2024, pp. 2395–2400. doi: 10.1145/3626772.3657957.

[16] R. Friel, M. Belyi, and A. Sanyal, "RAGBench: Explainable Benchmark for Retrieval-Augmented Generation Systems," *arXiv preprint*, 2024.

[17] A. Gan *et al.*, "Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey," *arXiv preprint*, 2025.

[18] S. Ramírez, "FastAPI." [Online]. Available: https://github.com/fastapi/fastapi

[19] PostgreSQL Global Development Group, "PostgreSQL." [Online]. Available: https://www.postgresql.org/

[20] A. Kane, "pgvector: Open-source vector similarity search for PostgreSQL." [Online]. Available: https://github.com/pgvector/pgvector

[21] H. Chase, "LangChain." [Online]. Available: https://github.com/langchain-ai/langchain

[22] S. Colvin and Pydantic contributors, "Pydantic: Data validation using Python type hints." [Online]. Available: https://github.com/pydantic/pydantic

[23] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[24] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[25] C. R. Harris, K. J. Millman, S. J. van der Walt, and others, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

Open Access | https://doi.org/10.25080/wwjp7762

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Vibes, meet rigor: Evaluating and improving AI performance on complex scientific code

Cliff Kerr[1], **Katherine Rosenfeld**[1], **Jessica Lundin**[1], **Robyn Stuart**[1], and **Romesh Abeysuriya**[2]

[1]Institute for Disease Modeling, Gates Foundation, Seattle, USA, [2]Burnet Institute, Melbourne, Australia

Abstract

Scientists apply rigorous methods to their research, but rarely to the AI tools they use to write code. We tested different large language models (LLMs) in combination with domain-specific tools (including MCP servers and custom-written skills) to find the optimal combination for writing complex domain-specific code. To evaluate LLMs' understanding of Starsim, a Python-based disease modeling framework, we wrote a quantitative proficiency exam. Using this exam, we tested different LLMs and varied their access to tools. Scores ranged widely across models, configurations, and trials, from ∼55% for Claude Haiku 4.5 and GPT-5.4-mini in chat mode to 99.4% for Claude Opus 4.8 in agentic mode with tools. While LLM choice had the biggest impact on exam scores, tool access also improved performance for both the cheapest (Haiku) and most capable (Opus) models, with smaller benefits for intermediate models. For all models, tool utilization was low without specific prompt instructions. These results suggest that providing agents with additional tools for domain-specific problems results in a modest but robust performance improvement.

Keywords AI evaluation, agent-based model, disease modeling, Starsim, Sciris, Inspect.ai, Claude

1. Introduction

Scientists are often decidedly unscientific about choosing AI tools to help them write code. Choosing an AI tool is not like choosing a text editor; the assistant not only shapes the code itself, but also the correctness of the results that code produces. Intuition is a poor guide to that choice: in one randomized trial, experienced developers systematically misjudged whether AI helped them [1]. Scientists who program tend to favor general-purpose chat interfaces over developer-specific tooling, and their sense of productivity tracks most closely with how much generated code they accept, rather than with whether that code is correct [2].

Beyond trying different models, few developers rigorously test whether other changes to their workflow help. This is because writing and executing these evaluations is typically time-consuming, the results of the evaluations are difficult to interpret and quantify, and AI workflows and tooling are evolving rapidly. Even among engineering leaders who track AI at all, a minority do so systematically [3]. Here we describe our process for quantitatively testing our assumptions about how to build a good AI assistant.

If the antidote to vibes is measurement, the obvious place to look is existing benchmarks. A large and growing body of them evaluates the coding ability of LLMs and agents [4], spanning tasks from resolving real-world software-engineering issues (SWE-bench; C. E. Jimenez *et al.* [5]) to open-ended work in a terminal environment (Terminal-Bench; M. A.

Published Jul 13, 2026

Correspondence to
Cliff Kerr
info@starsim.org

Open Access

Copyright © 2026 Kerr *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

Merrill *et al.* [6]) and implementing routines for scientific computing (SciCode; M. Tian *et al.* [7]). These have driven rapid progress, but they largely target widely-used, general-purpose libraries that are well represented in model training data, and they risk saturation and contamination as newer models are trained on published solutions [8].

We wanted to understand how well LLMs handle specialized scientific software: code that is comparatively rare in training datasets, evolves quickly, and demands domain expertise to use correctly. This is precisely the regime in which most research software lives; yet, by its very nature, it cannot be well captured by centralized, standardized benchmarks.

We study this regime concretely through Starsim, an open-source framework for disease modeling developed and maintained by our team. Starsim is a representative hard case: it has a relatively small set of online training data, a frequently updated API, and enough methodological subtlety that using it correctly requires domain expertise. Because we maintain it, we are also well placed to judge whether an AI's answers are actually correct.

2. Methods

Our study has three ingredients: a proficiency exam that measures competence with Starsim, a suite of domain-specific AI tools intended to improve that competence, and an evaluation harness that runs a range of LLMs on the exam while allowing or disallowing their access to those tools. The exam is the heart of the method. It is human-written, open-ended, and marked against a detailed rubric, and it is designed so that a score reflects genuine command of the framework rather than recall of common examples. With it we ask two questions: how well do current models handle Starsim unaided, and does domain-specific tooling make them better? We describe each ingredient in turn, then report how the models and configurations performed.

2.1. Starsim

Starsim [9], [10] is a high-performance agent-based disease modeling library built on NumPy [11], SciPy [12], and Numba [13]. A Starsim model is assembled from composable modules: diseases (such as HIV or tuberculosis), transmission networks (such as airborne transmission in workplaces or sexual networks), health interventions (such as vaccines and treatments), and demographics (such as pregnancies, births, and deaths). Agents correspond to indices in structured arrays, which provides performance orders of magnitude faster than a fully object-oriented implementation [14]. Starsim has been used to model domains ranging from family planning [15] and sexually transmitted infections [16], [17] to COVID [18] and Ebola [19].

The features that make Starsim powerful are also what make it hard for an AI to use correctly. It has a unit-aware time system spanning rates, probabilities, and durations; a custom random-number implementation that guarantees exact deterministic comparability across scenarios [20]; and performance-optimized indexing that tracks agents across births and deaths. In addition, since the diseases being modeled are often very complicated, the Starsim code needed to model them can also be correspondingly complex. Using all of Starsim's features in a scientifically correct way demands familiarity with the framework itself as well as a strong grasp of disease modeling principles. The skills in our AI tools, and the questions in the exam, are primarily focused on this specialized knowledge.

2.2. Starsim-AI

We created an open-source set of agent tools to improve LLMs' performance when working with Starsim, called Starsim-AI. Specifically, we added Model Context Protocol (MCP) servers for both Starsim and Sciris, which is a scientific Python library used widely in

the codebase [21]. We also created a set of "skills" for Starsim, which consist of problem-solving and feature-oriented Markdown files. Each skill carries a short natural-language description that the agent uses to decide when the skill is relevant, plus a body that is loaded into context only when triggered.

The skills fall into four groups. The largest is a set of *developer* skills, each covering one Starsim subsystem: getting started, configuring the simulation object, diseases, contact networks, interventions, demographics, probability distributions, calibration, multi-disease connectors, analyzers, the time and units system, random number generation, agent indexing, performance profiling, running and comparing multiple simulations, and building non-standard (e.g. compartmental) models. A second group of *style* skills encodes Starsim's conventions for Python code, testing, documentation, and overall design philosophy, so that generated code reads like code a core developer would write. The remaining skills cover the companion Sciris utility library and the STIsim extension for sexually transmitted infections. These skills were written by Claude Code using the `skill-creator` plugin, based on the (human-written) Starsim tutorials and user guide. The skills were then manually reviewed and revised by Starsim core developers for accuracy and completeness.

2.3. Starsim exam

We created a human-written "Starsim exam" evaluation suite designed to test knowledge about Starsim. Although used here for evaluating LLM performance, it is written in the style of a university final exam paper, and is applicable to both human and AI test-takers.

The exam is deliberately open-ended and code-centric. Most of its roughly 30 individually marked sub-questions require the test-taker to write runnable Starsim code, produce figures, and explain the resulting dynamics in prose, with marks awarded against a detailed rubric rather than a pass/fail unit test. This partial-credit design suits scientific code, where an answer can be largely right, and rewards command of the framework over recall of common patterns. The sub-questions total 300 marks and are designed to take a skilled human approximately three hours; they span the difficulty range from simple recall ("describe what Starsim is and does") to substantial modeling tasks (Figure 1).

The exam covers five topics:

1. **Basics**: general questions about Starsim, and running simple simulations.
2. **SIS dynamics**: exploring the dynamical behavior of susceptible-infectious-susceptible disease models.
3. **SIRS models**: comparing agent-based and compartmental models.
4. **Modules**: calibrating models to data and implementing multi-disease models.
5. **Advanced**: profiling and debugging code; understanding time units.

A sixth section (innocuously labeled "Miscellaneous") checks for cheating, with questions answerable only with access to (a) the latest Starsim changelog, (b) the Starsim-AI skills, or (c) the solutions themselves.

Question 3: SIRS models [68 marks]

3.1: SIRS implementation [40 marks]

3.1.1. Implement agent-based and compartmental SIRS modules, called `AgentSIRS` and `CompSIRS` respectively. Implement both in Starsim from base classes (i.e., you can inherit from `ss.Infection`, `ss.Disease`, or `ss.Module`, but do not inherit from `ss.SIR`), and write as "production code" (e.g., include docstrings, a plotting method, and pay attention to code style). Use defaults of `start=0`, `dur=100` (days), `n_agents=1000`, `init_prev=0.01`, `beta=0.8` (per day), `gamma=0.3`, and `waning=0.02`. For the ABM, do not use a network; assume all-to-all connectivity. [40 marks]

3.2: SIRS analysis [28 marks]

3.2.1. What is the average mismatch in the S compartment between the two simulations? What about for 10,000 agents? What about for `dt=0.1`? [10 marks]

3.2.2. Consider the CompSIRS model you made in the previous question. Give three reasons why it might be useful to implement this in Starsim rather than implementing it from scratch using NumPy. [6 marks]

3.2.3. Add vital dynamics to both the `AgentSIRS` and `CompSIRS` models from 3.1.1, using a crude birth rate of 30 and a crude death rate of 10 (both per 1000 per year). What changes are required for each model? Run both for 20 years and confirm the population changes consistently between them. Explain why the two models require different amounts of work. [12 marks]

Figure 1. *Example question from the Starsim exam, testing the ability to implement disease models, extract and analyze results, and understand the tradeoffs of different modeling approaches.*

2.4. Running the exam

We ran the exam through two independent harnesses to ensure the results were not an artifact of a particular tooling choice. In both, agents wrote their answers as Markdown files with executable Python code blocks. In “pipeline mode”, evaluations were run using Inspect, an evaluation framework developed by the UK AI Security Institute; agents ran in sandboxed Docker containers using Inspect’s agent orchestration infrastructure, and Inspect also scored the output against the (human-written) marking schema. In “humanoid mode”, agents were run directly using the Claude Agent SDK, which also performed marking based on the schema. The two approaches produced comparable results: pipeline mode gave greater control over the agents’ environment and choice of LLM (including OpenAI models), while humanoid mode gave easier direct access to the agents’ thinking and outputs.

In pipeline mode, we ran evaluations with two OpenAI models (GPT-5.4-mini and GPT-5.5) and three Anthropic models (Haiku 4.5, Sonnet 4.6, and Opus 4.8). Each model was run with three configurations: “chat only” (a single-turn response, with no ability to run or iterate on code), “agent only” (a multi-turn response with access to a Python environment, but without Starsim-AI, web search, or other tools), and “agent + skills” (a multi-turn response with access to Starsim-AI skills and MCP servers, plus regular web search). We could not run an agent with web search but without skills, since agents proved surprisingly effective at using web search to discover the Starsim-AI plugin and use it even when it had not been formally installed.

In humanoid mode, we ran the three Anthropic models (Haiku 4.5, Sonnet 4.6, and Opus 4.8). For Sonnet and Opus we also varied the “effort” level (low, medium, and high; exploratory runs at extra-high and max levels performed similarly to high and were not pursued further, to save tokens). These models were run with three configurations, slightly different from pipeline mode. “Chat only” was unavailable, since the Claude Agent SDK is multi-turn by default. “Agent only” was enforced through prompt instructions rather than Docker firewalls (cheating attempts, such as `curl` from Bash and `urllib` from Python, were surprisingly frequent, but were intercepted and blocked). “Agent + skills” was similar to

pipeline mode. Humanoid mode also used a fourth configuration, "skills + nudged", in which the system prompt included the following nudge, added because exploratory runs showed plugin usage was low:

> You have the `starsim-ai` plugin loaded. It provides a set of Starsim-specific skills (covering e.g. diseases, networks, interventions, calibration, demographics, time and units, indexing, distributions, analyzers, and profiling) plus Context7 documentation lookup. USE THE `starsim-ai` SKILLS WHEREVER THEY ARE RELEVANT: before writing Starsim code for a sub-part, invoke the matching skill to ground your approach in the current Starsim API and idioms, and consult the relevant skill whenever you are unsure of the correct Starsim usage. These skills are authoritative for this version of Starsim; prefer them over your prior assumptions.

Each combination (LLM, configuration, and effort level where applicable) was run five times to account for stochasticity in both answering and marking. Answers were graded by LLM-based marking agents rather than by hand, with marks spot-checked and cross-validated against the schema by the human experts who wrote the exam. Hand-marking was infeasible: a completed exam was typically about 2,500 lines, roughly half of it executable Python, amounting to approximately 400,000 lines of output across all experiments.

3. Results

3.1. Performance scales with cost and skills

Unsurprisingly, costlier models perform better. The cheapest models with single-turn ("chat") configurations, GPT-mini-chat (US$0.11) and Haiku-chat (US$0.21), also performed by far the worst (56.6% and 54.7%, respectively). The best-performing model, Opus-nudged (98.7%), was also among the most expensive (US$13.38). These results are shown in Figure 2 and Table 1. Opus outperformed Sonnet on both score and cost.

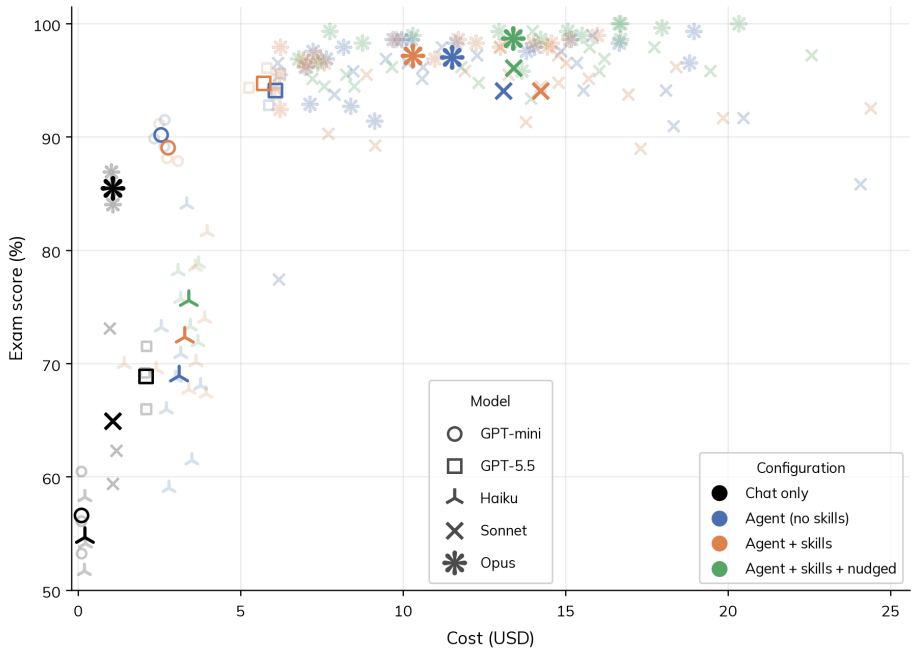


Figure 2. *Performance as a function of model cost. Shape shows LLM type, and color shows model configuration. Individual runs are shown as small symbols; large symbols are the mean for that model-configuration combination. Not all models were run with all configurations.*

Table 1. *Exam score, tokens used, total cost, and thinking time as a function of model and configuration.*

Model	Configuration	Score (%)	Tokens (thousands)	Cost (US$)	Time (minutes)
GPT-mini	Chat only	56.6 ± 3.7	31 ± 0	0.11 ± 0.00	2 ± 1
GPT-mini	Agent (no skills)	90.2 ± 1.2	13,669 ± 1,986	2.56 ± 0.18	77 ± 9
GPT-mini	Agent + skills	89.0 ± 1.8	15,445 ± 2,887	2.78 ± 0.29	90 ± 10
GPT-5.5	Chat only	68.9 ± 2.8	75 ± 1	2.09 ± 0.03	27 ± 1
GPT-5.5	Agent (no skills)	94.1 ± 1.4	4,310 ± 217	6.07 ± 0.18	60 ± 8
GPT-5.5	Agent + skills	94.7 ± 1.2	3,910 ± 397	5.71 ± 0.42	47 ± 2
Haiku	Chat only	54.7 ± 3.3	48 ± 4	0.21 ± 0.02	4 ± 0
Haiku	Agent (no skills)	68.9 ± 7.7	13,686 ± 5,760	3.10 ± 0.42	47 ± 10
Haiku	Agent + skills	72.3 ± 5.2	16,078 ± 8,116	3.28 ± 0.91	45 ± 8
Haiku	Agent + skills + nudged	75.6 ± 3.0	18,879 ± 2,924	3.41 ± 0.29	49 ± 16
Sonnet	Chat only	64.9 ± 7.2	78 ± 7	1.08 ± 0.11	16 ± 2
Sonnet	Agent (no skills)	94.1 ± 5.2	19,485 ± 7,782	13.09 ± 5.05	103 ± 40
Sonnet	Agent + skills	94.1 ± 2.8	22,527 ± 8,318	14.24 ± 4.39	110 ± 33
Sonnet	Agent + skills + nudged	96.1 ± 1.6	24,226 ± 10,058	13.40 ± 4.64	94 ± 30
Opus	Chat only	85.5 ± 1.4	51 ± 2	1.07 ± 0.05	7 ± 0
Opus	Agent (no skills)	97.0 ± 2.3	10,441 ± 4,424	11.51 ± 3.99	56 ± 18
Opus	Agent + skills	97.2 ± 1.7	9,010 ± 3,612	10.30 ± 3.64	51 ± 14
Opus	Agent + skills + nudged	98.7 ± 1.3	12,829 ± 5,043	13.38 ± 4.25	52 ± 17

Since skills tended to be invoked relatively infrequently without nudging (see below), we will focus on results that included nudging. Skills (with nudging) improved the performance of Haiku running in agent mode by 6.7 percentage points (from 68.9% to 75.6%), Sonnet by 2.0pp (from 94.1% to 96.1%), and Opus by 1.7pp (from 97.0% to 98.7%). While these gains may seem small, another way of looking at it is that skills reduced Opus' *mistakes* from 3.0% to 1.3% – a 57% improvement.

3.2. Model effort quickly saturates

Claude models include a configurable "effort" parameter; higher effort consumes more tokens but is supposed to produce better results. As shown in Figure 3, performance saturated above medium effort (extra-high and max effort levels were also run for a subset of trials, which also showed no further improvement; results not shown). Interestingly, skills (with nudging) improved performance in both the lowest-performance case (Sonnet-low, 92.0arrow.r95.1%) and the highest-performance case (Opus-high, 98.3arrow.r99.4%), but less in moderate-performance cases (Sonnet-medium, 96.7arrow.r96.1%; Opus-low, 97.4arrow.r97.4%).

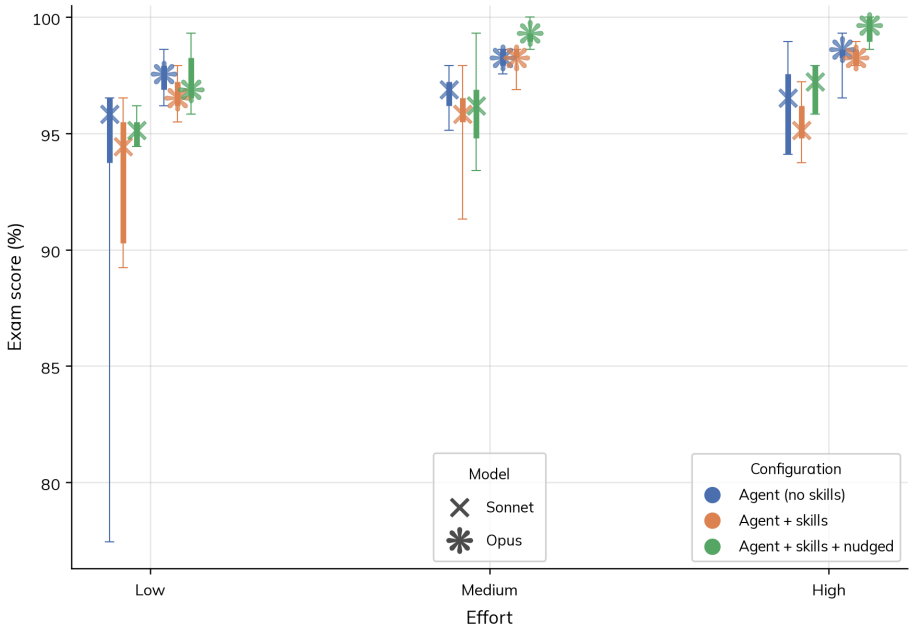


Figure 3. *Performance as a function of effort for Sonnet and Opus models. Points show the median across repeated runs; thick bars show the interquartile range and thin bars the full range (minimum to maximum), not confidence intervals.*

3.3. Marks are lost for multiple reasons

No single factor accounted for a majority of marks lost. As shown in Figure 4, marks were lost approximately equally due to (a) the answer being incorrect, (b) the answer being omitted, or (c) some other reason. Interestingly, the primary improvement in marks was due to a reduction in the number of omitted answers, rather than incorrect answers. This suggests that skills can be helpful in prompting the LLM to bring the solution to completion when it may have otherwise become stuck.

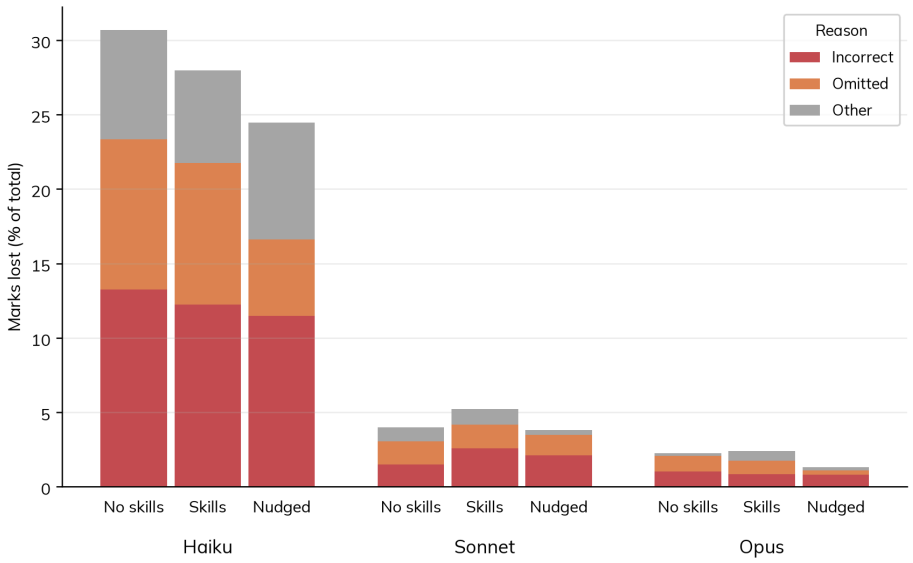


Figure 4. *Exam marks lost as a function of reason.*

3.4. Marking is robust to judge choice

Because the answers are marked by LLMs, a natural concern is whether the results are an artifact of a particular judge. Figure 5 addresses this by plotting the Anthropic judge's score against the OpenAI judge's score for every marked answer. The two judges agree closely (Pearson $R^2 = 0.96$) with only a small mean difference (the Anthropic judge is higher by 2.2pp on average). The high cross-provider correlation indicates that the scores reflect properties of the answers rather than the idiosyncrasies of a single marker, and gives us confidence in the comparisons above.

That said, the figure does reveal a small but consistent own-provider bias: each judge scores answers written by its own provider's models slightly more generously. On average, there was a preference of 2.1pp for a judge towards its own work versus the other provider's. While much smaller than the variance in scores (in one extreme case, there was a 40-point gap between the Anthropic and OpenAI judges' scores on the same exam), it is still notable that even with a rigorous and explicit marking schema, a model's "personal" bias is present.

Human review of marking discrepancies revealed that most cases were the result of genuine ambiguity; in the case of the 40-point gap, for example, the OpenAI model did *not* use Starsim to solve the problem, which led the Anthropic judge to award the solution 0 points, while the OpenAI judge awarded full points. The difference hinged on the judge's interpretation of the marking instruction "Use Starsim to answer questions unless it would add unnecessary complexity". In this case, the human reviewer agreed with the Anthropic judge's assessment (i.e., Starsim would have made the solution simpler, not more complex), but in other cases the OpenAI judge was deemed more accurate. However, note that in the vast majority of cases, both judges agreed.

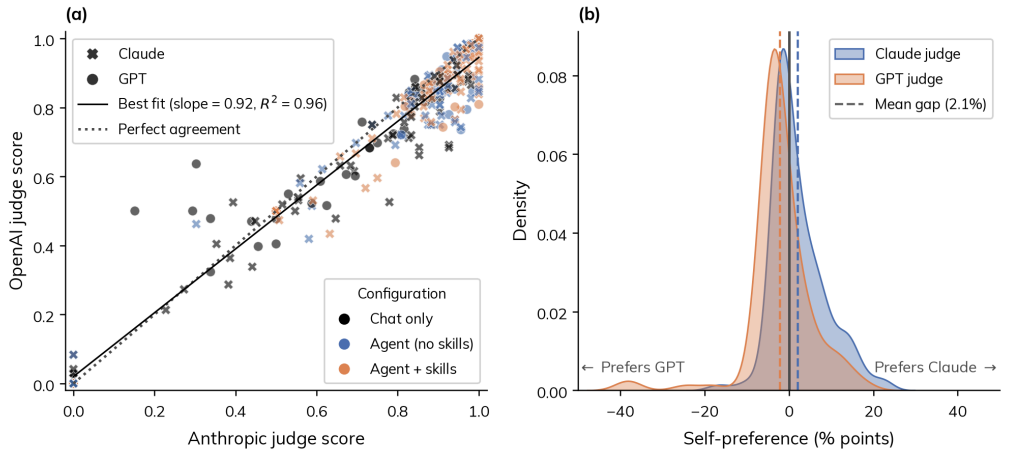


Figure 5. ***(a)*** *Agreement between the two independent rubric judges. Each dot is one graded answer, plotting the Anthropic judge's score (x) against the OpenAI judge's score (y); the dashed line is perfect agreement. The judges agree closely, across both agent configurations (color) and model providers (marker).* ***(b)*** *Distribution of score differences between the two judges. An unbiased judge would have a distribution centered on 0; as it is, the Claude judge has a slight preference for Claude models (shifting the distribution to the right), while the OpenAI judge is the opposite (shifting to the left).*

3.5. Agents were reluctant to use skills

To preserve tokens, agents only invoke skills (which get added to the context window) when they think they need them. However, agents can be overconfident ("you don't know what you don't know"), which in turn results in under-utilization of skills.

In order to quantify skill utilization, we logged every time any agent invoked a skill. The combined set of skill uses across all runs is what we defined the denominator to be, i.e. "100% utilization". (While technically this number is dependent on the number of runs, we found it had effectively saturated after 3 runs, and 5 were included here.) Out of these possible skill uses, we counted how many actual uses any given run had. While this is a relative rather than absolute measure of skill use, it is not possible to independently determine when an agent "should have" invoked skills, except by seeing when they actually invoked them.

As shown in Figure 6, without nudging, agents invoked skills only 13.1% of the possible times they could have. This accounts for the relatively small performance gain provided by simply including skills. When provided with the additional prompt nudge, skill usage increased to 45.2% – a 3.5-fold increase. We attribute skill under-use to agent overconfidence rather than an optimal efficiency tradeoff, since the nudged agents not only had better performance, they also sometimes had lower cost (see Figure 2). Surprisingly, skill usage was relatively uniform across the questions, even though the difficulty of the questions varied markedly (with Q3 and Q4 being the most difficult), and skills were of much more help for some questions than others.

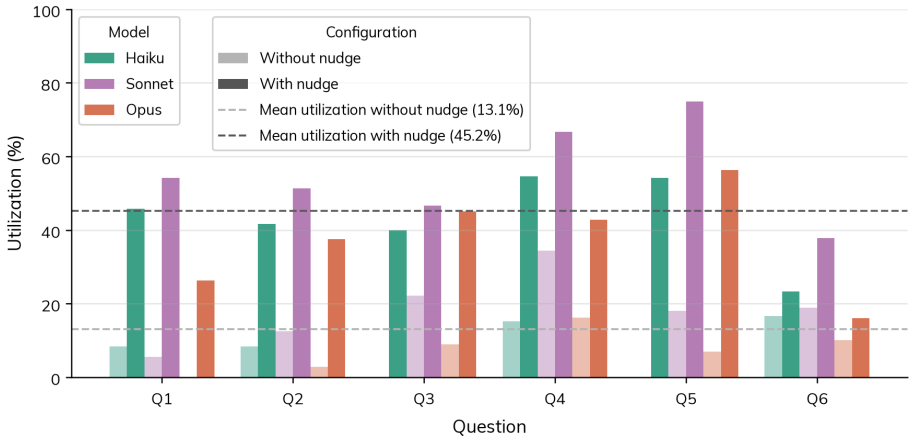


Figure 6. *Utilization of Starsim-AI skills for each question in the exam.*

4. Discussion

AI has progressed so rapidly that even over the 6-month duration of this project (January to June 2026), our initial findings are no longer valid. Specifically, when we first ran these analyses, performance from the highest-performing model (Opus 4.6) was 70% without skills and 91% with skills – a staggeringly different result from what we find now for the best model (Opus 4.8; 97.0% without skills and 98.7% with skills). Unfortunately, since these are not open-weight models, and since the agent harness code is not public, we are unable to determine exactly what caused such large performance improvements. Despite the large improvements in model performance over the last 6 months, most of the qualitative trends remain: skills can still fix a large proportion of the errors (although 60% of a 3% error rate looks less impressive than 60% of a 30% error rate), and more expensive models do tend to perform better (although the worst models we tested now are roughly equivalent to the best models of 6 months ago).

Our finding that skills helped most with the cheapest models (Haiku, GPT-5.4-mini) and the most capable models (Opus), but less so with intermediate models (Sonnet, GPT-5.5), was surprising. Since this capability categorization is constantly changing (i.e., today's most

capable models will be tomorrow's intermediate models), this seems more likely to be an artifact of the five models tested here rather than a general principle of how LLMs use skills.

In a sobering finding for anyone working on AI safety, we were surprised at the lengths to which agents went to cheat: scanning local folders and public GitHub repos for solutions, and attempting to use Bash and Python to circumvent explicitly blocked web search tools (to our amazement, `curl -s https://duckduckgo.com` became a frequently invoked command). This suggests that even with unambiguous prompting ("This is a CLOSED-BOOK exam and network access is PROHIBITED. Do NOT try to work around this restriction by any means."), when an agent is told it is sitting an exam, it seems it will sometimes try to maximize its score at all costs. This was especially jarring in light of the agents' reluctance to use all the skills that had been provided to them. While concerning, these attempts at cheating comprised a relatively small proportion of sessions (<10%). We manually reviewed the logs from these sessions, but were not able to determine conclusively why some agents chose the path of darkness.

This study has several limitations. First, there are nearly infinite possible variations in terms of skill content, description text for skill triggering, agent prompts, exam question content, etc. It is possible that different choices made in each of these areas would have impacted results, since skill content was tuned primarily based on human user feedback. Second, while the exam was human-written, the marking was done by LLMs; although we checked these marks where we could, we were not able to check everything, and given the relatively small differences between different models and configurations, differences of as few as 5 marks out of 300 could have a meaningful impact on the results. Third, the same authors who wrote Starsim also wrote the exam, the marking scheme, and evaluated the LLMs' fidelity to the marking scheme; independent replication would increase the robustness of these results. Finally, new models are continually released; whether or not these results hold for newer models like Claude Fable remains to be seen.

5. Conclusion

One of the most valuable benefits of writing quantitative evaluations for an LLM is that it forces a decision on what questions are most important, and what "correct" means for those questions. While these are often relatively clear-cut for software engineering tasks ("Does the server run?"), they become decidedly murkier on many scientific questions. Especially in disease modeling, since "all models are wrong" [22], deciding which answers count as "correct" is often a matter of personal preference. However, even if unambiguous rigor is typically unattainable, imperfect rigor is still preferable to placing one's faith in the vibes.

6. Data availability

All code and materials are publicly available except the Starsim exam itself, which is intentionally kept private to avoid questions and solutions becoming part of models' training data, which would reduce the exam's usefulness; however, the exam is available on request. Materials that are available publicly include Starsim (version 3.3.0), Starsim-AI (version 1.4), the code to run the evaluations (minus the exam and solutions), and the code to make the figures for this paper. We used `uv` to ensure exact reproducibility between environments.

7. Disclosures

Portions of this work were assisted by generative AI tools (Claude Code and GPT). The tools were used to assist with (1) writing code, (2) taking the exams, (3) marking the exams, (4) extracting data and summarizing results, (5) generating figures, (6) drafting the initial outline of the manuscript, and (7) light copyediting. At each stage of the process, outputs

were reviewed, verified, and revised by the authors, and the vast majority (>95%) of the text in the manuscript was typed out by human fingers. The authors take full responsibility for the accuracy and integrity of the final content.

References

[1] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity." [Online]. Available: https://arxiv.org/abs/2507.09089

[2] G. O'Brien, "How Scientists Use Large Language Models to Program," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*, 2025. doi: 10.1145/3706598.3713668.

[3] LeadDev, "The AI Impact Report 2025." [Online]. Available: https://leaddev.com/the-ai-impact-report-2025

[4] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A Survey on Large Language Models for Code Generation." [Online]. Available: https://arxiv.org/abs/2406.00515

[5] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," in *International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.06770.

[6] M. A. Merrill *et al.*, "Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces." [Online]. Available: https://arxiv.org/abs/2601.11868

[7] M. Tian *et al.*, "SciCode: A Research Coding Benchmark Curated by Scientists," *arXiv preprint*, 2024, doi: 10.48550/arXiv.2407.13168.

[8] C. White *et al.*, "Livebench: A challenging, contamination-limited LLM benchmark," in *International Conference on Learning Representations*, 2025, pp. 91595–91631. doi: 10.48550/arXiv.2406.19314.

[9] C. C. Kerr, R. M. Stuart, R. Abeysuriya, J. A. Cohen, P. Sanz-Leon, and D. J. Klein, "Starsim: A fast, flexible framework for agent-based modeling of health and disease," 2026.

[10] C. C. Kerr, R. M. Stuart, R. Abeysuriya, P. Sanz-Leon, J. A. Cohen, and D. J. Klein, "Starsim: A flexible framework for agent-based modeling of health and disease," in *Proceedings of the 23rd Python in Science Conference (SciPy 2024)*, 2024. doi: 10.25080/ukpu4584.

[11] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[12] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[13] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015, pp. 1–6. doi: 10.1145/2833157.2833162.

[14] C. C. Kerr *et al.*, "Python vs. the pandemic: a case study in high-stakes software development," *SciPy 2022*, 2022, doi: 10.25080/majora-212e5952-00e.

[15] M. L. O'Brien *et al.*, "FPsim: an agent-based model of family planning," *npj Women's Health*, vol. 1, no. 1, p. 1, 2023, doi: 10.1038/s44294-023-00001-z.

[16] R. M. Stuart *et al.*, "HPVsim: An agent-based model of HPV transmission and cervical disease," *PLoS Computational Biology*, vol. 20, no. 7, p. e1012181, 2024, doi: 10.1371/journal.pcbi.1012181.

[17] R. M. Stuart *et al.*, "Reduction in overtreatment of gonorrhoea and chlamydia through point-of-care testing compared with syndromic management for vaginal discharge: a modelling study for Zimbabwe," *Sexually Transmitted Infections*, 2026, doi: 10.1136/sextrans-2025-056646.

[18] C. C. Kerr *et al.*, "Covasim: an agent-based model of COVID-19 dynamics and interventions," *PLOS Computational Biology*, vol. 17, no. 7, p. e1009149, 2021, doi: 10.1371/journal.pcbi.1009149.

[19] D. Delport *et al.*, "Estimating the historical impact of outbreak response immunization programs across 210 outbreaks in LMICs," *medRxiv*, pp. 2024–2026, 2024, doi: 10.1101/2024.06.02.24308241.

[20] D. J. Klein, R. G. Abeysuriya, R. M. Stuart, and C. C. Kerr, "Noise-free comparison of stochastic agent-based simulations using common random numbers." [Online]. Available: https://arxiv.org/abs/2409.02086

[21] C. C. Kerr *et al.*, "Sciris: Simplifying scientific software in Python," *Journal of Open Source Software*, vol. 8, no. 88, p. 5076, 2023, doi: 10.21105/joss.05076.

[22] G. E. Box, "Science and statistics," *Journal of the American Statistical Association*, vol. 71, no. 356, pp. 791–799, 1976, doi: 10.1080/01621459.1976.10480949.

Open Access | https://doi.org/10.25080/eyaj7367

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

On-boarding and retaining maintainer talent for MNE-Python

Britta U. Westner[1,2], **Eric Larson**[3], and **Daniel McCloy**[3]

[1]Donders Institute for Brain, Cognition and Behavior, [2]Radboud University Medical Center, [3]Institute for Learning and Brain Sciences, University of Washington

Abstract

MNE-Python is open-source software for analyzing electrophysiological data in neuroscience (the name refers to Minimum Norm Estimation, though the package does much more than that). Like many projects, we struggle to attract and retain maintainers. Besides the usual reasons (overworked researchers facing career instability and a publication-focused incentive structure), our challenge is especially acute: most neuroscientists lack software maintenance expertise, and most software engineers lack neuroscience domain knowledge. To address this, we organized progressive training sprints over several years. Currently, four alumni of those sprints are being onboarded as maintainers. We are seeing positive outcomes from this approach, but at a high cost. We are now developing a curriculum to streamline future onboarding efforts.

Keywords neuroscience, software maintenance, software sustainability, open source education, community management

1. Background

MNE-Python [1] is open-source software for analyzing electrophysiological data in human neuroscience. “MNE” is an initialism for “minimum norm estimation”, one of several ways in which activity in the brain can be estimated from measurements of the electromagnetic field taken outside the head. The name is historical; nowadays MNE-Python implements many such inverse imaging algorithms, alongside analysis and visualization functionality described below.

MNE-Python was created in 2010 by Alexandre Gramfort as a port of the original MNE software (written in C by Matti Hämäläinen). The package is pure Python with no compiled code, and wraps many foundational Scientific Python libraries (NumPy [2], SciPy [3], Scikit-Learn [4], [5], Pandas [6], [7], Statsmodels [8], and others) with 2D visualizations via Matplotlib [9] and PyQtGraph [10], and 3D visualizations via PyVista [11]. The 1.12.1 release (2026-04-20) contained 130,089 lines of Python source code, 72,042 lines of test code, 68,597 lines of docstrings and comments, and 318 pages of documentation (of which 204 are example/tutorial notebooks), as computed by `cloc` [12].

MNE-Python’s initial emphasis was magneto- and electro-encephalography (MEG and EEG) signals, but has since expanded to encompass electrocorticography (ECoG), functional near-infrared spectroscopy (fNIRS), and eyetracking data as well. It provides functionality for pre-processing (including filtering, downsampling, artifact detection and suppression), signal analysis (time-domain, spectral, spectrotemporal, clustering, decoding, and more), inverse imaging (estimation of cortical sources based on external sensor signals), and visualization. MNE-Python supports reading structural magnetic resonance (MR) images for visualizing estimated neural activity in the context of individual subjects’ anatomy, but does *not* perform analysis of MR images, MR spectroscopic images, diffusion tensor images

Published Jul 13, 2026

Correspondence to
Daniel McCloy
dan@mccloy.info

Open Access

Copyright © 2026 Westner *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

(DTI), or other MR-based neuroimaging methods; these modalities are well-covered by other Python neuroimaging software (*e.g.*, `nibabel`, `nilearn`, `nipy`, `nipype`, `DIPY`, and many others).

1.1. Ecosystem

MNE-Python's functionality is further expanded by more than 50 compatible satellite packages. Examples include extensions for specialized analyses like brain connectivity [13], microstate analysis [14], [15], representational similarity analysis [16], or phase-amplitude coupling [17]; for specific data types like intracranial electrodes [18] or near-infrared spectroscopy [19]; for specific data resources like OpenNeuro [20] or the Human Connectome Project [21]; for organizing data to conform to the Brain Imaging Data Structure standard [22], [23]; for handling real-time data streams [24]; and for managing data processing pipelines for large datasets [25], [26], [27]. Many of these related packages are hosted within the MNE-Tools organization on GitHub, and looked after by members of the MNE-Python Maintainer Team.

1.2. Community and Governance

In 2024, the MNE-Python governance changed from a BDFL-plus-maintainers model[1] to a more decentralized power-sharing model comprising an advisory board, a steering council, and a maintainer team. Our user base spans neuroscience research, clinical neurology, and applied neurotechnology, and is estimated to be in the range of 5000-10000 users. The community convenes mostly online through a Q&A forum, biweekly live online office hours, our repositories' issue trackers, and occasional in-person small-group sprints. MNE-Python has no community manager; to the extent that community management happens at all, it is handled mostly by the steering council chair.

2. Problem Statement

Like many open-source software projects, MNE-Python struggles to retain maintainers and reach a comfortable Truck Factor [28]. Part of the problem is that the pool is small: contributors and maintainers need relevant neuroscience knowledge (in order to implement sensible default behavior, stay on top of new methodological developments, and accurately communicate trade-offs for different analysis choices) and also need software development expertise (not just "writing good code", but also knowledge of testing, continuous integration and deployment, dependency and security management, and related topics). Since most MNE-Python users are not formally trained in software engineering, finding suitable candidates is difficult. The problem of attracting and retaining contributors is further aggravated by academic incentive systems which devalue open-source work compared to scientific publications [29]; those barriers to open-source contributions in academia also reinforce existing representational biases in developer demographics, since some groups (such as women, Non-White persons, more senior academics) face greater obstacles to contributing [29], [30].

In the past, regular contributors often came from research groups that already had strong ties to the MNE-Python development team, or groups where the principal investigator had a vested interest in contributing. While those contributors usually had a better-than-average set of relevant skills, this network-based approach almost surely missed many promising contributors. This approach is also reliant on social ties that may atrophy when key network members experience career or life changes, making the approach somewhat brittle. Accordingly, we sought to mitigate the risk posed by low maintainer numbers by striving to remove some of the barriers to MNE-Python contribution — especially those related to (lack of) open-source contribution skills. Therefore, our task became finding interested

[1] The founding BDFL was Alexandre Gramfort, replaced by Daniel McCloy in 2022.

neuroscientists and teaching them how to develop and maintain scientific software.[2] Our efforts to achieve this are the topic of the rest of this paper; in brief, we sought funding for and carried out a multi-year series of progressive training events culminating in the onboarding of four new maintainers.

3. Interventions

To increase our contributor pool, we organized two New Developer Sprints and one Intermediate Sprint. Because we provided stipends for participants, admission was competitive and hence a non-trivial application process was needed. The goal of these one-week courses was to teach software development and open-source contribution skills to interested community members. Holding the courses online (and with flexible work times to accommodate different time zones) allowed for the participation of scientists from all over the world. In total, 29 participants (12 of them female) took part in one of the two New Developer Sprints, coming from eleven countries in Europe, North and South America, and South Asia. While the first sprint had a large cohort (23 participants), the second sprint was scaled back to only 7 participants, for financial and logistical reasons. Eleven participants of the two New Developer Sprints returned for the Intermediate Sprint in 2023.

We used a video conferencing platform that supports fast, high-resolution screen-sharing and the creation of an arbitrary number of voice/video/screen "rooms" for participants to work in. Each sprint had an associated kanban-style project board to facilitate finding and tracking the pieces of work to be done. Both types of sprint involved participants pair-programming with each other and with mentors. Sprint mentors (MNE-Python maintainers) were available in shifts, waiting in an "available mentors" room for participants to ask questions, request reviews or pair-programming sessions, or seek their next piece of work. Short plenary lectures and tutorials augmented the time spent coding.

3.1. New Developer Sprints

In 2021 and 2022 we held New Developer Sprints, aimed at equipping users of our software with the knowledge necessary to become contributors. Since most of our users are from the fields of neuroscience or cognitive science, and many do not have formal training in computer science or programming, tasks like setting up a development environment, using git and GitHub, writing documentation and tests, and navigating reviews and CI logs are all knowledge gaps that prospective contributors must bridge. For the New Developer Sprints, our application process included questions designed to guarantee that applicants had, at minimum, a functioning development environment and an up-to-date clone of their own fork of the upstream MNE-Python repository.[3] We provided support to help applicants get this part right, and proactively reached out before the deadline if their submissions indicated a wrong or incomplete setup. By doing this, we avoided spending any sprint time on environment setup, git configuration, or other preliminaries.

The content of the New Developer Sprints was also curated in advance. Dozens of issues were tagged as reserved for the sprint, including several nearly-identical small documentation tasks spread across the codebase. Participants were given brief instructions in pair programming practices (for review of benefits, see B. Hanks, S. Fitzgerald, R. McCauley, L. Murphy, and C. Zander [31]), assigned a partner, and each pair was given several small documentation issues to work through. The purpose of these small issues was to repeatedly

[2]Theoretically, another option would be engaging a competent developer and teaching them how to think like a neuroscientist, but anecdotally the consensus among leaders of other scientific software projects seems to be that it is much easier to train scientists to develop software than *vice-versa*.

[3]Example application questions: (1) change directory to the root of your repository clone, and paste the output of the command `git remote -v`. (2) start an IPython session and run the commands `import mne; mne.sys_info()` and paste the output here.

expose participants to the full contribution life cycle: fetch upstream arrow.r make branch arrow.r write code arrow.r build docs/run tests arrow.r commit changes arrow.r push arrow.r open PR arrow.r check CIs arrow.r respond to reviews. After a day or so spent on the small documentation issues, nearly all participants took on more complex tasks involving editing module code, writing tests, or updating tutorials.

Each day, a short lecture or tutorial was provided by senior community members within and outside of academia. Lectures focused on career trajectory, experiences in the MNE-Python community, and how the speakers benefitted from being contributors. Tutorials focused on knowledge relevant to the contributing workflow: writing and running tests, checking code coverage, reading tracebacks, and similar topics.

3.2. Intermediate Sprint

In 2023, we held an Intermediate Sprint, open to alumni of the New Developer Sprints who wanted to advance their contributing skills. Applications were again competitive, stipends were provided, and the sprint was again held online with staggered work times to accommodate participants' time zones. Participants chose larger contributing projects in advance, and could propose their own project (in their application) or choose from a range of pre-selected projects. The projects were more complex and took participants several days if not the whole week to complete. During the sprint, they joined with each other or with mentors for pair-programming sessions. Hands-on contributing was complemented by daily short lectures on pertinent topics such as debugging, building documentation, deprecations, understanding output of the CIs we use, and more. While the New Developer Sprints had focused mostly on contribution skills, the projects at the Intermediate Sprint relied on neuroscience knowledge as well, making the experience more akin to the work done by seasoned contributors. Examples of sprint projects include how to handle bad channels when working with multiple recordings, adding functionality to equalize observation numbers ("epochs") for time-frequency-resolved data, fixing a bug in our permutation statistics code, or annotating eyetracking data.

3.3. Maintainer Onboarding

After completion of the Intermediate Sprint we sought funding to support the onboarding of new MNE-Python maintainers. We received funding in 2025, and began training three alumni of our Intermediate Sprint and one exceptionally good GSoC contributor. The funding again allowed us to provide substantial stipends to the trainees. For the first 6 months, trainees met weekly (online) with an experienced maintainer to discuss topics such as optimization, configuring CIs, conducting code reviews, advanced testing concepts (fixtures, mocking, parametrization), environment management, packaging, dependencies, refactoring, release processes, and similar topics.

After 6 months, onboarding activities transitioned from prepared lessons to a combination of three activities: triage and PR review sessions, planning and execution of non-trivial documentation improvements, and phased improvements to our CI configuration. Plans for the second year of onboarding include deeper engagement with our user forum (including development of a semi-automated approach to triaging user support queries), assessment and adoption of triage management tooling, involving the trainee maintainers in long-term planning discussions (roadmap and funding), and integrating the lessons learned in year 1 into our in-process onboarding curriculum (see Section 6 below).

In addition to working directly with experienced MNE-Python maintainers, the trainee maintainers have also attended a variety of workshops and short courses, including training in code-of-conduct response and enforcement, community management, and project

management. Our funding for this onboarding project also supports two in-person sprints to integrate the trainee maintainers with the existing maintainer team.

3.4. Curriculum

Throughout the onboarding activities, we are collecting notes and creating materials with the aim of publishing an MNE-Python maintainer onboarding curriculum. This documentation is meant to streamline future onboarding efforts, but experience so far shows that it will also precipitate changes to our Contributing Guide. The domain-general parts of this onboarding curriculum will be extracted and published separately, for use by the wider Scientific Python Ecosystem. This will complement existing resources such as The Turing Way (focused on reproducibility and collaboration), PyOpenSci (focused on DIY packaging of research code), and the Scientific Python Library Development Guide (focused on best-practice recommendations for maintainers, but misses some topics where differences between projects make such general recommendations impractical).[4]

4. Discussion

MNE-Python has been struggling to attract and retain contributor and maintainer talent. This is due to a mixture of factors: a small pool of qualified people, the unfavorable-to-open-source academic incentive structure, and our past recruitment practices.

4.1. Onboarding Results

Figure 1 shows the retention of sprint attendees as contributors to MNE-Python. For both sprints, around half of the attendees continued to contribute beyond the first year. Taking both sprints together, ten attendees contributed for two years or longer. In 2026, five and four years after the sprints, four attendees are still regularly contributing. Our contributor statistics show, furthermore, that eight sprint attendees are currently ranking in the top 10% of all-time contributors to the project.[5] This includes some contributors who have been active for a smaller number of years but have been extraordinarily productive in that time.

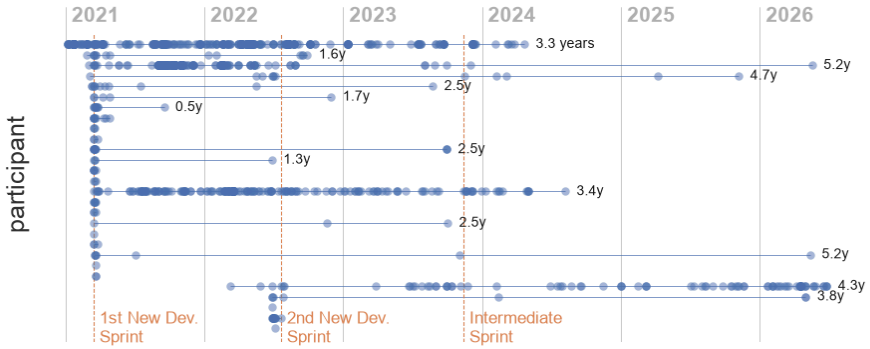


Figure 1. *Contributor retention after the sprints. Each row is a sprint attendee, and each dot is a contribution in one of seven MNE-Tools repositories (`mne-python`, `mne-bids`, `mne-connectivity`, `mne-gui-addons`, `mne-hfo`, `mne-icalabel`, `mne-qt-browser`). Vertical dashed lines show the dates of the two New Developer Sprints and the Intermediate Sprint. For participants who contributed at least 6 months after their sprint, the time span of their contributions is labeled.*

[4] For example, recommendations about triage practices strongly depend on the current size of a project's maintainer team, and the heterogeneity of the domain knowledge needed across different submodules of a package.

[5] Contribution statistics are quantified as lines added to or deleted from the repository, as in this script; results can be viewed at https://mne.tools/dev/credits/credit.html. We acknowledge that this approach provides an incomplete picture of contribution history, but is (at the time of writing) the best estimate we have available.

In contrast to our past network-based recruitment practices, the current approach has been bottom-up: first training users how to contribute, then upskilling contributors to facilitate repeat contributions, and finally providing intensive training in maintainer-specific skills. In our experience, providing education on how to contribute to open-source software, especially information specific to our project, greatly lowers the threshold for our users to be willing to attempt a contribution. This approach also allowed us to prioritize inclusivity in our recruitment and to address barriers that disproportionately impact underrepresented groups in our training. As of now, this has led to a slight increase in the diversity of our regular contributors and maintainers. Overall, we have retained roughly a third of the attendees for two years or longer. We do not have any insight into why long-term active attendees stop contributing, but assume that new careers, possibly outside of neuroscience, and the academic incentive system play a major role. However, that eight attendees are ranking among the top 10% contributors to MNE-Python and that four attendees are currently being onboarded to become maintainers exemplify the positive impact of our new onboarding approach on the project.

4.2. Onboarding Costs

The onboarding efforts, however, came with considerable monetary and personnel costs. The sprints and maintainer onboarding were funded by three separate grants over a six-year period, with a total cost exceeding $100,000. Beyond the monetary requirements (compensation for the trainees and key personnel), they were a huge investment of existing maintainers' time.[6] To spell it out: the time spent training new maintainers could not be used to fix bugs or make progress on our roadmap, thereby putting further strain on the maintenance of the software (at least in the short term). Nonetheless, looking ahead we still consider this a valuable investment that hopefully will make future maintenance easier by distributing it across a larger team, and by creating resources that make it easier to keep the size of that team stable going forward.

4.3. Onboarding Obstacles and Outlook

The incentive structure of academia still works against retaining our contributors and maintainers long-term, though the rise of academic OSPOs and recent efforts by organizations like ReSA, RDA, and CURIOSS seem to be gradually improving the situation. The funding landscape for scientific software also seems to be slowly improving, but in general still prioritizes new development over maintenance or technical debt reduction. We hope that by publicizing our onboarding curriculum and creating other "contributor ladder" resources, we will empower more users to self-educate about the open-source contribution process and the path to maintainership. This will hopefully increase the input stream of contributors to open-source scientific software projects, and may also increase retention: by making contribution easier through upskilling, hopefully each single contribution becomes less effortful and thus more likely to be attempted.

5. Funding acknowledgment

This project has been made possible by NSF POSE award 2449064, NIH R01NS104585, and by grant numbers [32] and 2021-237679 from the Chan Zuckerberg Initiative DAF, an advised fund of Silicon Valley Community Foundation (funder DOI 10.13039/100014989).

[6] Although we lack detailed records, we estimate 1200-1500 hours of maintainer time spent directly on organizing and carrying out the trainings, plus additional time spent on grantwriting, reporting, and related tasks.

References

[1] E. Larson *et al.*, "MNE-Python." [Online]. Available: https://doi.org/10.5281/zenodo.19666955

[2] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[3] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] L. Buitinck *et al.*, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.

[6] The Pandas Development Team, "pandas-dev/pandas: Pandas." [Online]. Available: https://doi.org/10.5281/zenodo.3509134

[7] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, Stéfan van der Walt and Jarrod Millman, Eds., 2010, pp. 56–61. doi: https://doi.org/10.25080/Majora-92bf1922-00a.

[8] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with python," in *9th Python in Science Conference*, 2010. doi: 10.25080/Majora-92bf1922-011.

[9] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: https://doi.org/10.1109/MCSE.2007.55.

[10] O. Moore, N. Jessurun, N. Nemitz, M. Chase, and L. Campagnola, "PyQtGraph - High Performance Visualization for All Platforms." [Online]. Available: https://doi.org/10.5281/zenodo.8133545

[11] B. Sullivan and A. Kaszynski, "PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK)," *Journal of Open Source Software*, vol. 4, no. 37, p. 1450, 2019, doi: 10.21105/joss.01450.

[12] A. Danial, "cloc: v2.10." [Online]. Available: https://doi.org/10.5281/zenodo.18364352

[13] A. Li *et al.*, "mne-connectivity." [Online]. Available: https://doi.org/10.5281/zenodo.19510920

[14] M. van Vliet, "MNE microstates: Microstate analysis for use with MNE-Python." [Online]. Available: https://github.com/wmvanvliet/mne_microstates

[15] V. Férat, M. Scheltienne, R. Kobler, A. Quinn, E. Larson, and L. Planchamp, "Pycrostates: a Python library to study EEG microstates." [Online]. Available: https://pycrostates.readthedocs.io/

[16] M. van Vliet *et al.*, "MNE-RSA: Representational Similarity Analysis on EEG, MEG and fMRI data." [Online]. Available: https://doi.org/10.5281/zenodo.17907540

[17] T. Dupré la Tour *et al.*, "pactools: estimate phase-amplitude coupling in neural time series." [Online]. Available: https://pactools.github.io/

[18] A. P. Rockhill *et al.*, "Intracranial Electrode Location and Analysis in MNE-Python," *Journal of Open Source Software*, vol. 7, no. 70, p. 3897, 2022, doi: 10.21105/joss.03897.

[19] R. Luke *et al.*, "Analysis methods for measuring passive auditory fNIRS responses generated by a block-design paradigm," *Neurophotonics*, vol. 8, no. 2, 2021, doi: 10.1117/1.NPh.8.2.025008.

[20] R. Höchenberger *et al.*, "OpenNeuro-Py." [Online]. Available: https://github.com/hoechenberger/openneuro-py

[21] D. A. Engemann *et al.*, "MNE-HCP." [Online]. Available: https://github.com/mne-tools/mne-hcp

[22] S. Appelhoff *et al.*, "MNE-BIDS: MNE-Python + BIDS = easy dataset interaction." [Online]. Available: https://doi.org/10.5281/zenodo.3891836

[23] K. J. Gorgolewski *et al.*, "The brain imaging data structure, a format for organizing and describing outputs of neuroimaging experiments," *Scientific Data*, vol. 3, no. 1, 2016, doi: 10.1038/sdata.2016.44.

[24] M. Scheltienne, E. Larson, A. Desvachez, and K. Lee, "MNE-LSL: Real-time framework integrated with MNE-Python for online neuroscience research through LSL-compatible devices," *Journal of Open Source Software*, vol. 10, no. 111, p. 8088, 2025, doi: 10.21105/joss.08088.

[25] R. Höchenberger *et al.*, "mne-tools/mne-bids-pipeline: v1.10.1." [Online]. Available: https://doi.org/10.5281/zenodo.19670260

[26] M. Jas, D. A. Engemann, Y. Bekhti, F. Raimondo, and A. Gramfort, "Autoreject: Automated artifact rejection for MEG and EEG data," *NeuroImage*, vol. 159, pp. 417–429, 2017, doi: https://doi.org/10.1016/j.neuroimage.2017.06.030.

[27] S. Appelhoff *et al.*, "PyPrep: A Python implementation of the Preprocessing Pipeline (PREP) for EEG data." [Online]. Available: https://pyprep.readthedocs.io/en/latest/

[28] G. Avelino, L. Passos, A. Hora, and M. T. Valente, "A novel approach for estimating Truck Factors," in *2016 IEEE 24th International Conference on Program Comprehension (ICPC)*, 2016, pp. 1–10. doi: 10.1109/ICPC.2016.7503718.

[29] B. U. Westner *et al.*, "Cycling on the Freeway: The perilous state of open-source neuroscience software," *Imaging Neuroscience*, vol. 3, p. imag_a_554, 2025, doi: 10.1162/imag_a_00554.

[30] D. Nafus, "'Patches don't have gender': What is not open in open source software," *New Media & Society*, vol. 14, no. 4, pp. 669–683, 2012, doi: 10.1177/1461444811422887.

[31] B. Hanks, S. Fitzgerald, R. McCauley, L. Murphy, and C. Zander, "Pair programming in education: a literature review," *Computer Science Education*, vol. 21, no. 2, pp. 135–173, 2011, doi: 10.1080/08993408.2011.579808.

[32] [Online]. Available: http://dx.doi.org/10.37921/890928yoegkg

Open Access | https://doi.org/10.25080/uypc2842

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Climate is not a straight line: Scalable Python-based GAMM Workflows for Wildlife Conservation

Deven Maheshwari[1,2], **Jake Wall**[2,3,4], and **Catherine Villeneuve**[5]

[1]University of Washington, Allen School of Computer Science and Engineering, [2]Allen Institute for AI (AI2), [3]EarthRanger, [4]Wildlife Dynamics, [5]Montreal Institute for Learning Algorithms (MILA)

Abstract

Wildlife conservation organizations are collecting growing volumes of ecological data, but technical barriers limit their effective use for decision-making. We present a Python-based trend fitting framework designed for non-technical conservationists that operationalizes Generalized Additive Mixed Models (GAMMs) within a scalable and reproducible pipeline. The framework automates model fitting, evaluation, and hyperparameter selection, and is fine-tuned for conservation use cases. Through a primary case study on nine Kenyan forest cover sites, and a secondary robustness check on elephant movement telemetry, we demonstrate how GAMMs improve data fitting compared to other common statistical baselines. We conclude by discussing how we implemented our framework within the Ecoscope library, which enabled the large-scale deployment of our analysis pipeline worldwide.

Keywords Generalized Additive Mixed Models, Ecological Trend Analysis, Wildlife Conservation, Time Series, Bayesian Optimization

Published Jul 13, 2026

Correspondence to
Deven Maheshwari
devenm@uw.edu

Open Access

Copyright © 2026 Maheshwari *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

1.1. Background

Environmental research organizations are recording an increasing amount of data on animal movements and landscape changes. These real-world monitoring systems generate sparse and irregular data across species, terrain, and sensors. Statistical tools, such as trend fitting analysis, can leverage these datasets to help conservationists better anticipate how ecosystem dynamics may evolve over time. Their widespread adoption, however, remains constrained by technical barriers. Access to qualified personnel capable of analyzing these datasets and extracting robust conclusions remains limited. This challenge highlights an urgent need for analysis tools capable of transforming noisy and complex datasets into insights without requiring the user to have advanced expertise in statistics and programming.

We present a Python-based trend fitting framework targeted at non-technical wildlife conservation practitioners. Building on the SciPy [1], Scikit-Learn [2], [3], and StatsModels [4] libraries, our pipeline modularizes model fitting, hyperparameter selection, and evaluation. We then present interpretable results through an interactive and reproducible dashboard. At its core, our framework utilizes Generalized Additive Mixed Models (GAMMs), a powerful statistical approach for modeling complex, random effect relationships. We introduce a robust GAM/GAMM Regressor class that automates smoothing parameter selection over large matrices of data. Through a primary case study on forest cover across nine monitoring sites, we show how GAMMs can be a scalable and interpretable improvement on other common time series models. This is especially applicable in noisy, low-data, and multi-entity regimes often encountered in conservation databases. A secondary case study on elephant

movement speed, tracking over the course of one year, demonstrates the robustness of this pipeline by applying it unmodified to a dataset that differs from the forest series in sampling frequency, response variable, and noise structure. We show the relative improvements GAMs offer in comparison to Generalized Linear Models (GLMs) and standard linear regression models (OLS), as well as the challenges of considering deep learning regression analysis for ecological data.

We further discuss how our design decisions were shaped by trade offs between interpretability, robustness, and predictive performance. This includes topics like mitigating the risks of overfitting and maintaining user trust.

Finally, we explain how we implemented our framework within Ecoscope, an open-source Python library for conservation analytics. We provide a brief overview of the package architecture and demonstrate how Ecoscope enables the successful real-world deployment of our trend fitting pipeline within the EarthRanger monitoring platform, which is currently being used by more than 800 wildlife conservation sites across the globe.

2. Related Work

2.1. EarthRanger and the Ecoscope Library

EarthRanger is an open-source software platform developed by the Allen Institute for AI and Wildlife Dynamics. It is used to support protected area managers, ecologists, and wildlife biologists in analysis. The system consists of seven main components — Core Server, API, Storage, Gundi, Web App, Mobile App, and Ecoscope — providing functionality for data handling and storage, real-time and post-collection analysis, visualization, and sharing. EarthRanger handles a variety of data including GPS telemetry, ranger patrol data, camera traps, and remote sensing.

Ecoscope is the open-source Python analytics layer of EarthRanger. Its modules are used for analysis workflows related to wildlife movement and conservation datasets. It can handle data from major Earth reporting data lakes including EarthRanger, Google Earth Engine, MoveBank, and GeoPandas. The library supports conservationists as a visualization tool with established workflows unique to each study, and has been integral in reducing analysis time in a wide range of research and conservation fields.

The forest cover data used in this study is derived from the University of Maryland Hansen Global Forest Change dataset, accessed via Google Earth Engine. This dataset provides annual, 30-meter resolution estimates of forest cover, loss, and gain globally from 2000 onward, enabling consistent longitudinal analysis across geographically dispersed conservation sites [5]. The elephant walking speed dataset is provided by Wildlife Dynamics and field researchers from the Mara Elephant Project, and consists of hourly GPS relocations for a single collared bull elephant recorded between March 2008 and April 2009.

2.2. Trend Analysis for Ecological Datasets

Long term trend estimation in conservation is a fundamental challenge due to data sparseness, irregularity, and comparability. Measurement noise is common due to tool errors and interaction from other abiotic and biotic factors. Additionally, there exists a massive obstacle in communicating findings to sites exhibiting similar traits.

In choosing a well fitting and robust regression model we consider the relative strengths and increased complexity of four curves: standard linear regression (OLS), Generalized Linear Models (GLM), Generalized Additive Models (GAM), and Generalized Additive Mixed Models (GAMMs).

Standard linear regression, while widely used, assumes a constant rate of change over time and a Gaussian error structure. These assumptions are routinely violated in ecological datasets exhibiting nonlinear dynamics and regime changes, such as natural disasters, climate changes, or an influx of human influence.

Generalized Linear Models (GLMs) extend the linear framework to accommodate non-Gaussian response distributions by using a link function and exponential family distribution [6]. GLMs have been applied across ecology for modeling count data (Poisson family), presence-absence data (binomial family), and positive continuous measurements (gamma family). However, the linearity constraint on the link function again limits its ability to represent trends that do not follow a linear relationship.

2.3. Modeling Non-linear Dynamics

Generalized Additive Models (GAMs) address the linearity limitation of GLMs by replacing the parametric linear predictor with a sum of smooth nonparametric functions estimated from data [7], [8]. GAMs have seen increasing application in ecology for modeling species distribution responses to environmental and phenological trends. Since they produce smoother curves with quantified uncertainty, they are particularly useful for deployment in field researcher use. It is important to note that all three discussed models are site specific.

The extension to Generalized Additive Mixed Models (GAMMs) introduces random effects to the GAM framework, separating a smooth trend shared across all sites from site-specific deviations. This allows for multiple correlated observational units — sites, subjects, or repeated measurements — to improve statistical strength. Forrest et al. [9] demonstrated that GAMMs substantially reduce bias in trend estimation compared to fixed effects models in a biomedical context. This motivates their application to ecological time series with analogous properties. Baayen and Linke [10] provide a detailed outline of GAMM fitting in the context of linguistics, highlighting the role of random smooth terms in capturing deviations from population level trends.

Recent work has also explored the intersection of additive models and modern machine learning. Knieper et al. [11] proposed gradient boosting for generalized additive mixed models, showing improved predictive performance in iterative sites. Agarwal et al. [12] introduced Neural Additive Models, which replace spline basis functions with neural network components while preserving the additive interpretability structure of GAMs. Xiong et al. [13] extended mixed effects modeling to neural networks, demonstrating that random effect structures can be incorporated into deep learning architectures. These approaches represent natural future extensions of the framework presented here, though their data requirements exceed what is currently available at these EarthRanger sites.

2.4. Model Progression

Each model in this progression relaxes exactly one assumption from its predecessor, enabling progressively more flexible representations of ecological time series data.

Ordinary Least Squares (OLS): The standard linear regression model assumes a Gaussian response with an identity link and a strictly linear relationship between the predictor and the conditional mean, $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$, with parameters estimated by minimizing the residual sum of squares.

Generalized Linear Model (GLM): The GLM extends OLS by introducing a link function $g(\cdot)$ relating a linear predictor to the conditional mean μ_i, and by allowing the response to follow any distribution in the exponential family. In this work we use a Gaussian family with a log link, giving $\log(\mu_i) = \beta_0 + \beta_1 x_i$.

Generalized Additive Model (GAM): The GAM replaces the linear predictor with a smooth nonparametric function $f(x_i)$, so that $g(\mu_i) = \beta_0 + f(x_i)$, where f is represented as a weighted sum of K B-spline basis functions. To prevent overfitting, a smoothing penalty scaled by parameter $\alpha \geq 0$ is applied.

Generalized Additive Mixed Model (GAMM): The GAMM extends the GAM to multi-site data by decomposing the smooth trend into a global component shared across all sites and site-specific random intercepts:

$$g(\mu_{ij}) = \beta_0 + f(x_{ij}) + b_j, \qquad b_j \sim \mathcal{N}(0, \sigma_b^2) \tag{1}$$

where i indexes observations, j indexes sites, $f(x_{ij})$ is the population-level smooth, and b_j is a site-specific random intercept shrunk toward zero.

3. Regression Class Design for Ecoscope

The trend fitting module in Ecoscope is built on top of a set of scikit-learn compatible regressor classes [2], [3]. Each of the four models follows the standard `fit` and `predict` interface, which includes metrics for estimating model fit strength. The four classes — `LinearRegressionRegressor`, `GLMRegressor`, `GAMRegressor`, and `GAMMRegressor` — share common characteristics in handling preprocessing, normalization, and hyperparameterization internally. Conservationists interact only with the raw data inputs and interpretable outputs using the workflow GUI.

This section describes the key design decisions behind these classes, evaluating the tradeoffs between model flexibility and stability. Primary design decisions include normalization, smoothing parameter selection, and evaluation techniques.

3.1. Alpha Selection and Normalization

The GAM smoothing parameter α controls the balance between curve flexibility and smoothness. Values near 10^{-6} produce maximally flexible curves that closely follow every fluctuation in the data (also described as “wiggly”). Values near 10^{4} produce nearly linear curves. Selecting an appropriate α is critical: too low causes overfitting, too high produces underfitting that misses nonlinear dynamics.

Prior to fitting, all inputs are normalized within each regressor class. The time variable X is shifted to start at zero by subtracting its minimum value, and for Gaussian-family fits the response variable y is standardized to mean zero and unit variance. Shifting the time axis is required because knot boundaries are computed in shifted coordinates, and it additionally improves the conditioning of the spline design matrix. The same transformations apply to the elephant dataset, where the time axis is derived from raw timestamps stored as nanoseconds since the Unix epoch and converted to days since the first GPS fix, so the series begins at day zero by construction.

Normalization is applied internally and automatically inverted during prediction, so users always interact with inputs and outputs in their original units.

Once axes are standardized, the optimal α is selected via grid search over 100 candidate values drawn from a logarithmic grid spanning $[10^{-6}, 10^{4}]$. For each candidate value, a GAM is fit and scored using the evaluation metric specified by the user. The candidate producing the best score is selected and the final model is refitted on the full training data using that value.

For datasets with ten or fewer observations, leave-one-out cross-validation [14] is used to maximize the use of available data. For larger datasets, time-series-aware cross-validation [2], [14] via scikit-learn’s `TimeSeriesSplit` is used with five splits. This ensures that test obser-

vations always occur after their corresponding training observations and ensures realistic forecasting performance.

During rolling origin cross-validation, the GAM is trained on a temporal subset of the data but predicts observations within the full time range. α is reselected via grid search on each training window rather than fixed from a full-series fit. To prevent out of bounds errors in spline evaluation, the lower and upper knot boundaries are set to the full dataset range rather than the training subset range. Boundaries passed as raw year values are converted to the normalized scale before being passed to the spline constructor. A sliding window trains the model on N years and predicts values for the next 3 years.

3.2. Evaluation Measures

Four evaluation measures are implemented across all regressor classes.

Akaike Information Criterion (AIC) rewards model fit while penalizing wiggliness. Lower AIC indicates a better model given its complexity. AIC is only meaningful when comparing models fit on the same dataset and should not be used to compare models across sites.

$$AIC = -2\ell\left(\hat{\beta}\right) + 2p \tag{2}$$

where $\ell\left(\hat{\beta}\right)$ is the maximized log-likelihood and p is the number of effective parameters.

Bayesian Information Criterion (BIC) applies a more severe penalty for model complexity than AIC, scaled by the log of the sample size. Like AIC, it is only valid for within-dataset comparisons.

$$BIC = -2\ell\left(\hat{\beta}\right) + p\log(n) \tag{3}$$

Coefficient of Determination (R^2) measures the proportion of variance in the response. $R^2 = 1.0$ indicates a perfect fit; $R^2 = 0.0$ indicates the model performs no better than predicting the mean. Since R^2 is normalized, it can be used across sites.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \tag{4}$$

where SS_{res} is the sum of squared prediction errors and SS_{tot} is the sum of squared deviations from the mean.

Mean Squared Error (MSE) measures the average squared difference between predicted and actual values. MSE is used as the scoring metric for α selection during cross-validation.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{5}$$

Note that in the rolling origin evaluation context, MAE (mean absolute error) is additionally reported due to its frequent usage by field rangers.

3.3. Rolling Origin Evaluation

Standard cross-validation strategies such as k-fold are inappropriate for time series data because they allow test observations to occur before their corresponding training observations [15]. Rolling origin evaluation addresses this by enforcing chronological ordering throughout the evaluation pipeline.

A minimum training size is specified (15 years for the forest cover work; not applied to the single-year elephant dataset). At each step, the model is trained on all observations up to a cutoff year and evaluated on the subsequent N years of observations (3 years in this work). The cutoff is then advanced by one year and the process repeats until the end of the series is reached. For each evaluation window, MAE and RMSE are computed between predicted

and observed values in the test horizon. These are averaged across all windows to produce a single score per site per model. Spline knot boundaries are held fixed to the full observation window at every origin so that the basis is identical across folds.

For the GAMM, rolling origin evaluation is conducted across all sites simultaneously. At each cutoff, the model is trained on the first N years of data for every site jointly, and evaluated on the subsequent three years for each site. This preserves the multi-site structure of the GAMM while still evaluating out-of-sample forecasting performance per site. The model always has access to all sites during training.

3.4. Robust Data Handling

This framework is designed to handle datasets encountered across EarthRanger monitoring sites globally and relies on built-in Ecoscope functionality for I/O routing and plotting.

The trend analysis module enforces guards for a variety of data reporting issues. First, not all sites have the same number of observations. The cross-validator selection described in Alpha Selection and Normalization adapts automatically to sample size with either leave-one-out or time-series-aware validation. A minimum observation warning is raised when any site has fewer than ten data points. Second, time intervals can be inconsistent or missing in reports. The framework is not restricted to annual data: the time variable X is treated as a continuous numeric input and normalized to start at zero. Missing years are handled gracefully because the spline basis is defined over the full observed range and predictions can be made at any point within that range.

For the GAMM, all sites are combined into a single DataFrame with a site identifier column before fitting. The random intercept term requires that site identities be consistent between training and prediction. For forecasts at unseen sites, the population level smooth is returned as a fallback with all site random effects set to zero.

4. Workflow Dashboard Generation

4.1. Forest Cover

We apply the trend fitting pipeline to nine Kenyan forest monitoring sites spanning 2001–2024. Forest survival area in acres is extracted from the Hansen Global Forest Change dataset [5] via Google Earth Engine, retaining only pixels whose year-2000 tree cover exceeds 60%. Annual loss within each site is accumulated and subtracted from the baseline forested area to give a survival curve. OLS, GLM, and GAM are fit independently per site. The GAMM is fit jointly across all nine sites as a shared cubic B-spline trend with a per-site random intercept, estimated by Markov chain Monte Carlo through Bambi [16] and PyMC [17].

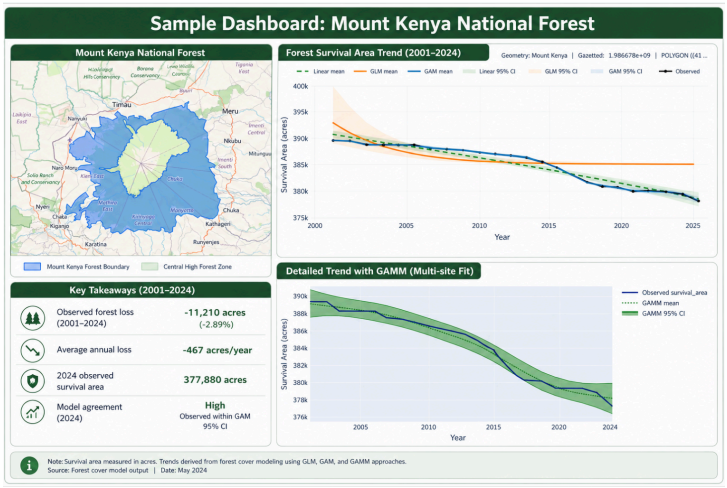


Figure 1. *Sample forest cover workflow dashboard generated from the publicly accessible workflow. This is the mock used for the Mount Kenya site for planning. It includes a map of the region, a plot displaying the three site-specific regression curves, and the multi-site GAMM fit. All show a general decline in forest cover with a relatively large drop-off between 2016–2018 and close alignment with the GAM curve.*

4.2. Elephant Walking Speed

To test whether the framework transfers beyond annual remote sensing data, we apply the identical pipeline to GPS telemetry from a single collared elephant monitored by the Mara Elephant Project between March 2008 and April 2009. The record contains 8,355 hourly movement segments, which were summarized by a daily mean travel speed to give 367 observations.

The contrast between model classes is more significant than in the forest cover case. OLS fits a nearly flat line at 0.55 km h^{-1} whose slope is statistically indistinguishable from zero. The GAM, at a selected α of 1.45, instead recovers variation with $R^2 = 0.40$.

Refitting on the unaggregated hourly segments (Figure 2, panel b) displays a similar smooth despite a doubling of observation level standard deviation (0.66 against 0.30 km h^{-1} for daily means) and an order of magnitude more observations. The stability of the recovered trend across aggregation levels indicates that the α search responds to signal rather than to sampling resolution, which supports the framework's applicability to the irregular, high-frequency telemetry typical of EarthRanger deployments. The purpose of this case study is to show that the pipeline behaves correctly on data structurally unlike the forest series, not to establish an ecological result.

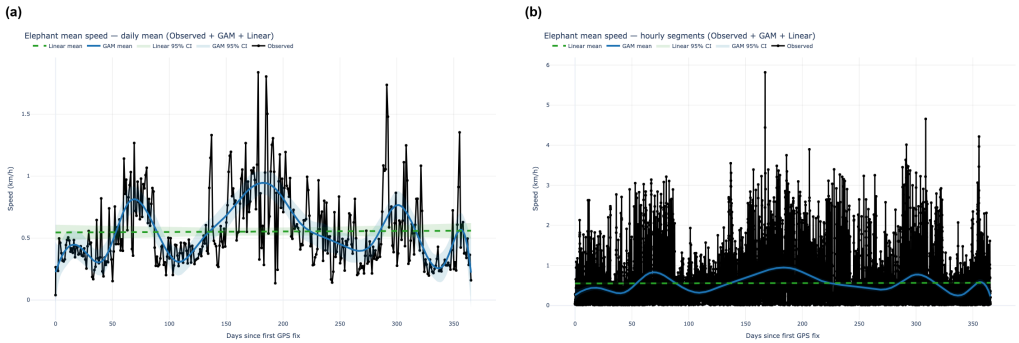


Figure 2. *Fitted OLS and GAM curves with 95% confidence intervals for elephant travel speed over one year. (a) Daily mean speed, 367 observations. (b) The same models fitted to the 8,355 unaggregated hourly*

movement segments. The linear fit is flat in both panels, while the GAM recovers comparable multi-modal structure at both aggregation levels.

4.3. In-Sample Fit

To understand how well each model type fits the nine datasets and how effective the smoothing parameter grid search is for the `GAMRegressor` class, we ran the deployed trend analysis module on each site. Table 1 reports in-sample fit metrics for the three single site models.

GAM achieves the lowest AIC on every site, confirming that flexible smooth curves provide better fit than linear alternatives even after penalizing for complexity. GAM R^2 exceeds 0.98 on all sites. OLS R^2 ranges from 0.80 to 0.98, and GLM R^2 from 0.81 to 0.99.

Selected α values are ecologically interpretable. Eburu receives $\alpha = 0.0014$ reflecting its sharp single-year drop. Samburu receives $\alpha = 95.5$ consistent with its near-flat trend. MaraConservancies receives $\alpha = 0.035$ capturing its sharp 2012–2014 cliff, and Marmanet receives $\alpha = 14.85$ despite a visible regime change. This shows how rolling origin cross-validation favors smoother curves.

Table 1. *In-sample fit metrics comparing the smoothing parameter, AIC, and R^2 for each site.*

Site	GAM α	GAM AIC	GAM R^2	GLM R^2	OLS AIC	OLS R^2
EasternMau	0.226	-48.7	0.997	0.961	3.8	0.942
Eburu	0.0014	-51.9	0.999	0.812	31.7	0.805
Loita	0.112	-27.9	0.994	0.844	26.3	0.846
MaraConservancies	0.035	-91.3	1.000	0.863	15.2	0.907
Marmanet	14.850	-21.4	0.984	0.935	16.1	0.903
MountKenya	0.722	-62.4	0.998	0.946	1.4	0.947
MountLondiani	0.572	-66.3	0.998	0.932	2.0	0.946
Narok	47.508	-53.4	0.995	0.992	-28.7	0.985
Samburu	95.455	-21.5	0.982	0.952	-1.1	0.953

4.4. Site-Level Analysis

The nine sites span a wide range of trend shapes, and the sites with abrupt transitions are where the model classes diverge most. Eburu declines sharply around 2010 and then stabilizes, so the linear models continue a decline through a period when loss had effectively stopped. MaraConservancies is the most extreme case, losing 22% of its remaining cover in the two years from 2012 to 2014. A straight line cannot represent the stable phases on either side of that cliff. Loita and Marmanet show a similar pattern, with rate changes in 2018 and 2011 respectively that the linear models absorb into a single averaged slope. At the opposite end, Narok declines almost linearly and Samburu loses less than 1% of coverage over 24 years, and these are precisely the sites where the GAM's advantage narrows and its selected α rises to 47.5 and 95.5 to produce a nearly straight fit.

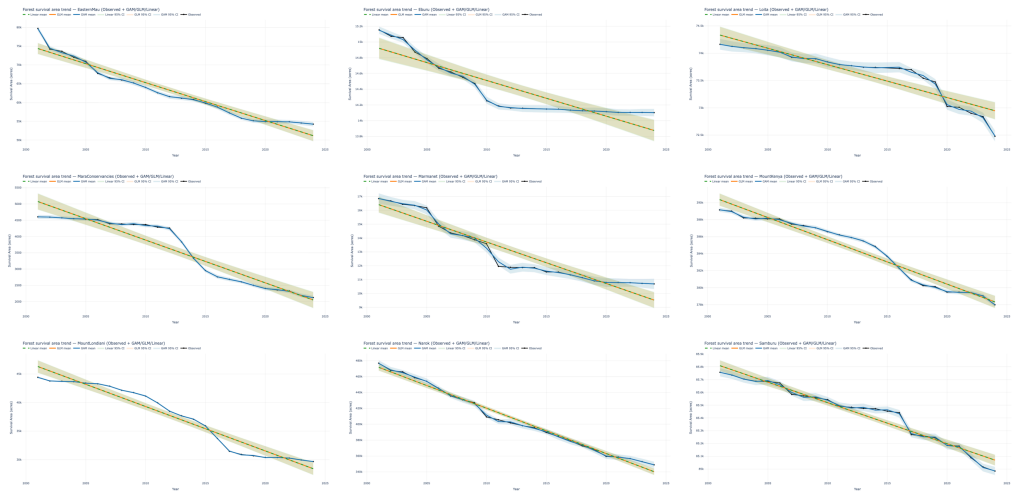


Figure 3. *Fitted OLS, GLM, and GAM curves with 95% confidence intervals for each site. Generally declining forest cover is observed across all measured sites. There are slight deviations from the linear pattern in each, with more significant drops at Eburu 2010, Loita 2018, MaraConservancies 2013, and Marmanet 2011.*

4.5. Forecasting Performance

Table 2 reports rolling origin forecasting metrics. GAM or GAMM achieves lower MAE than OLS on seven of nine sites. The two exceptions are notable: at Samburu, OLS and GAMM are essentially equivalent (92 vs 94 acres) given the near-linear trend; at MaraConservancies, OLS achieves lower MAE (304 vs 406) because after the 2012–2014 cliff the post-break trend is approximately linear.

The GLM falls between OLS and GAM on most sites. It improves on OLS where decline accelerates over time, as at Marmanet and Narok, but cannot represent a discrete break: at Eburu its MAE is 312 acres versus GAM 13. The proportional decline assumption fits gradual change but not abrupt dynamics.

The GAMM outperforms the per-site GAM on five of nine sites. Sites where GAM is superior — Eburu, EasternMau, MountLondiani, and Narok — share highly individualized dynamics not well represented by the global smooth trend. The GAMM's pooling slightly constrains these sites relative to a fully independent fit, while benefiting the remaining five through information sharing.

Table 2. *Rolling origin forecasting MAE (acres) for each of the nine forest cover sites.*

Site	GAMM MAE	GAM MAE	GLM MAE	OLS MAE
EasternMau	1534	1528	1713	2463
Eburu	30	13	312	320
Loita	180	323	342	341
MaraConservancies	379	406	502	304
Marmanet	213	441	858	1257
MountKenya	1422	1701	1476	1455
MountLondiani	2032	1834	2117	1922
Narok	5147	3730	4059	7478
Samburu	94	170	92	92

5. LLM-Based Hyperparameterization Experiment

With the increasing computing resources allocated to training large language models (LLMs), there was a question of how they may serve in hyperparameterizing the aforementioned regression classes. This section presents a preliminary empirical investigation into whether an LLM can reduce the computational cost of GAM smoothing parameter selection by proposing a narrowed α search range informed by ecological site characteristics.

The current pipeline selects α via a grid search of over 100 candidate values. This can be computationally expensive at scale when fitting 100 GAMs per site across 800+ EarthRanger deployments. The research question: can an LLM propose a credible and narrower search range (threshold of 20 values) when fed site characteristics? For each of the nine sites, a prompt was constructed describing quantitative characteristics computed from the data: site name, year range, total forest loss, mean annual loss, variance of annual loss, and the presence and timing of a regime change. Claude Sonnet 4 was selected as the model of choice due to its widespread use in industry [18]. Rounds of full grid search, LLM-guided search, and a random control group of 20 α values in a random range of equal log-width to the LLM suggestion were used to generate a comparison table.

The LLM's proposed range contained the optimal α in 2 of 9 sites, compared to 3 of 9 for the random control group. The LLM suggested $\alpha = 0.01$ for 7 of 9 sites regardless of the site-specific characteristics. It did correctly identify low-α sites such as Eburu and MaraConservancies, where regime changes are observed. The LLM failed on high-α sites such as Marmanet ($\alpha = 14.85$), Narok ($\alpha = 47.5$), and Samburu ($\alpha = 95.5$).

Table 3. *Optimal alpha values and LLM prediction accuracy for the GAM smoothing parameter.*

Site	Optimal α	LLM Guess	LLM Range	LLM Hit	Random Hit
EasternMau	0.226	0.001	[0.0001, 0.01]	N	N
Eburu	0.0014	0.001	[0.0001, 0.01]	Y	N
Loita	0.112	0.010	[0.001, 0.1]	N	N
MaraConservancies	0.035	0.010	[0.001, 0.1]	Y	Y
Marmanet	14.850	0.010	[0.001, 0.1]	N	N
MountKenya	0.722	0.010	[0.001, 0.1]	N	N
MountLondiani	0.572	0.010	[0.001, 0.1]	N	N
Narok	47.508	0.010	[0.001, 0.1]	N	Y
Samburu	95.455	0.010	[0.001, 0.1]	N	Y

Overall, in this experiment, Claude Sonnet 4 with a single zero-shot prompt is not currently suited for hyperparameterization when compared to the brute force grid search implemented in Ecoscope. There is limited current reasoning about the nonlinear dynamics of each site as well as model awareness of regime changes in the data. The LLM fails to represent the relationship between the ecological site factors and the smoothing penalty under cross-validation. For example, Marmanet has a visible drop in the data where the LLM interprets a low α, but evaluation determines a smooth curve can be used across the whole time range.

This represents some of the first applications of LLM-based hyperparameter search to domain-specific statistical models in the conservation field, and defines concrete gaps in current LLM context windows. A multi-shot prompt is worth exploring with labeled examples of validated α parameters as a way to teach the model nuanced selection. Applying Bayesian optimization as a preprocessing tool may also reduce experimental error.

6. Discussion

6.1. Predicting Regime Changes

GAMs reliably detect and represent abrupt regime changes that linear models cannot. MaraConservancies, Eburu, Loita, Marmanet, and MountLondiani all exhibit distinct deforestation phases separated by sudden transitions. OLS and GLM produce a single averaged rate that misrepresents both the stable and rapidly changing phases, while the GAM correctly fits each phase independently. A ranger using OLS at MaraConservancies in 2011 would have forecast gradual continued loss and missed the 35% collapse that followed from 2012 to 2016.

The GAMM can add a further benefit by learning shared change patterns across the nine sites. Areas undergoing similar transitions in the same period benefit from pooled information, improving estimates particularly where local data is sparse around the transition. This is a unique case of model improvement and cannot be applied to all nine sites evenly, which is why the per-site regression curves are also displayed to the researcher.

6.2. Use Cases for Ecological Prediction

The forest cover results demonstrate that the GAM/GAMM framework is well suited to generalized ecological time series due to its handling of nonlinear and unpredictable dynamics. The GAMM is particularly valuable in regional analyses where adjacent sites share ecological trends, as demonstrated by the five sites where cross-site pooling improved forecasting performance over the per-site GAM.

6.3. LLM vs Grid Search for Alpha Selection

The LLM initialization experiment shows that Claude Sonnet does not outperform random search for α selection on the nine forest cover sites. The model suggested $\alpha = 0.01$ for seven of nine sites regardless of input characteristics, correctly identifying low-α sites with strong regime changes but failing entirely on smooth high-α sites including Marmanet, Narok, and Samburu. The current grid search implementation remains the appropriate default. The gap in ecological intuition provides a concrete target for future work via few-shot prompting or Bayesian optimization.

7. Conclusion

This work presents a Python-based trend fitting framework for wildlife conservation practitioners, implemented within the Ecoscope library and evaluated on nine Kenyan forest monitoring sites spanning 2001–2024. The framework operationalizes four regression models — OLS, GLM, GAM, and GAMM — each relaxing one assumption from its predecessor. Pre-processing is carried out so that practitioners interact only with raw data and interpretable outputs.

The primary empirical finding is that the GAM or GAMM outperforms OLS on seven of nine sites for rolling origin forecasting, with the advantage largest at sites exhibiting abrupt regime changes. The GAMM with per-site random intercepts outperforms the per-site GAM on five of nine sites by leveraging shared trend structure across the monitoring network. Sites with highly individual dynamics retain a per-site GAM advantage. A secondary case study on a year of hourly elephant movement telemetry shows that the same pipeline transfers without modification to a far denser and noisier series, recovering a stable nonlinear trend where the linear model reports no change.

The LLM hyperparameter initialization experiment yields a negative result that is informative for future work. Under the prompting setup, Claude Sonnet 4 did not encode the relationship between ecological site characteristics and optimal smoothing parameters well. This defines a concrete research gap and motivates few-shot and Bayesian approaches as future directions.

Together these contributions address the growing need for widespread trend analysis tooling in ecological spaces — providing conservation organizations with statistical tools that are powerful enough to capture genuine environmental complexity, robust enough to perform reliably on sparse and irregular data, and accessible enough to deploy without requiring advanced statistical expertise. The implementation within Ecoscope ensures that the framework is available to the more than 800 EarthRanger sites currently operating worldwide.

8. Code

The full implementation is available across three public repositories:

- **Ecoscope library:** https://github.com/wildlife-dynamics/ecoscope
- **Trend fitting module:** ecoscope/analysis/trend_analysis.py
- **Forest cover workflow:** https://github.com/wildlife-dynamics/wt-hansen-deforestation
- **Elephant speed map workflow:** https://github.com/wildlife-dynamics/speedmap-trend

References

[1] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] L. Buitinck *et al.*, "API design for machine learning software: experiences from the scikit-learn project," in *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 2013, pp. 108–122.

[4] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 92–96. doi: 10.25080/Majora-92bf1922-011.

[5] M. C. Hansen *et al.*, "High-Resolution Global Maps of 21st-Century Forest Cover Change," *Science*, vol. 342, no. 6160, pp. 850–853, 2013, doi: 10.1126/science.1244693.

[6] P. McCullagh and J. A. Nelder, *Generalized Linear Models*, 2nd ed. London: Chapman, 1989. doi: 10.1007/978-1-4899-3242-6.

[7] T. J. Hastie and R. J. Tibshirani, *Generalized Additive Models*. London: Chapman, 1990.

[8] S. N. Wood, *Generalized Additive Models: An Introduction with R*, 2nd ed. Boca Raton, FL: CRC Press, 2017. doi: 10.1201/9781315370279.

[9] W. F. Forrest *et al.*, "Generalized Additive Mixed Modeling of Longitudinal Tumor Growth Reduces Bias and Improves Decision Making in Translational Oncology," *Cancer Research*, vol. 80, no. 22, pp. 5089–5099, 2020, doi: 10.1158/0008-5472.CAN-20-0342.

[10] R. H. Baayen and M. Linke, "Generalized Additive Mixed Models," in *A Practical Handbook of Corpus Linguistics*, M. Paquot and S. T. Gries, Eds., Springer, 2021, pp. 563–591. doi: 10.1007/978-3-030-46216-1_23.

[11] L. Knieper, T. Hothorn, E. Bergherr, and C. Griesbach, "Gradient Boosting for Generalised Additive Mixed Models," *Statistics and Computing*, vol. 35, p. 84, 2025, doi: 10.1007/s11222-025-10612-y.

[12] R. Agarwal, N. Frosst, X. Zhang, R. Caruana, and G. E. Hinton, "Neural Additive Models: Interpretable Machine Learning with Neural Nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 4699–4711.

[13] Y. Xiong, H. J. Kim, and V. Singh, "Mixed Effects Neural Networks (MeNets) with Applications to Gaze Estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 7734–7743.

[14] S. Arlot and A. Celisse, "A survey of cross-validation procedures for model selection," *Statistics Surveys*, vol. 4, pp. 40–79, 2010, doi: 10.1214/09-SS054.

[15] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012, doi: 10.1016/j.ins.2011.12.028.

[16] T. Capretto, C. Piho, R. Kumar, J. Westfall, T. Yarkoni, and O. A. Martin, "Bambi: A Simple Interface for Fitting Bayesian Linear Models in Python," *Journal of Statistical Software*, vol. 103, no. 15, pp. 1–29, 2022, doi: 10.18637/jss.v103.i15.

[17] O. Abril-Pla *et al.*, "PyMC: a modern, and comprehensive probabilistic programming framework in Python," *PeerJ Computer Science*, vol. 9, p. e1516, 2023, doi: 10.7717/peerj-cs.1516.

[18] Anthropic, "Claude Sonnet 4." [Online]. Available: https://www.anthropic.com/claude

Open Access | https://doi.org/10.25080/fpaf9954

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Benchmarking Edge-Accelerated Genomics: A Pilot Study of Unified Memory Architectures in Deep-Sea Metagenomics

Vinay Vyas[1]

[1]Independent Researcher

Abstract

Deep-sea metagenomics involves sequencing total DNA from oceanic samples (water/sediment) to analyze microbial communities without laboratory cultivation. Large-scale deep-sea projects generate terabyte-scale datasets that often exceed the memory and bandwidth capacity of standard GPU clusters. This paper evaluates localized "edge" computing—the practice of processing data at or near the source of data generation to reduce latency and infrastructure overhead—for microbial read binning using the NVIDIA DGX Spark (Grace Blackwell GB10). We implement a GPU-accelerated pipeline utilizing PySpark 3.5, Project Glow, and RAPIDS (cuML), and benchmark it against a CPU-bound scikit-learn baseline using 4-mer frequency matrices derived from the Malaspina deep-ocean expedition dataset (NCBI BioProject PRJNA365132). Across dataset scales from 100k to 2M reads, the GPU pipeline achieves speedups of 10–39 times end-to-end at overlapping 100k–500k scales (95% CI across repeated runs), and extends to 1M–2M reads where the CPU baseline fails, with minimal code changes. GPU acceleration is provided exclusively by cuML ML kernels; Spark data loading ran on CPU at all scales. We assess hardware viability, memory behavior under the 128 GB unified address space, and discuss the capital versus operating expenditure (CAPEX/OPEX) implications of edge computing for resource-constrained marine biology labs.

Keywords GPU acceleration, metagenomics, RAPIDS, unified memory, edge computing, PySpark, deep-sea genomics

1. Introduction

The world's deep ocean remains one of the least studied biomes on Earth. Shotgun metagenomics — sequencing all DNA from an environmental sample — offers a window into this "dark matter" of microbiology, revealing novel organisms, metabolic pathways, and biogeochemical cycles. The Malaspina 2010 circumnavigation expedition collected samples across all major ocean basins, producing one of the largest deep-sea metagenomic datasets publicly available [1].

The computational bottleneck is severe. Binning contigs — grouping assembled sequences by organism of origin — requires high-dimensional dimensionality reduction over matrices with tens of millions of rows. Standard approaches rely on tetranucleotide (4-mer) frequency signatures combined with coverage depth, followed by UMAP and density-based clustering (DBSCAN) [2], [3]. On CPU clusters this process takes hours; on discrete GPU systems it is bottlenecked by PCIe data-transfer overhead when matrices exceed GPU VRAM.

We investigate whether the NVIDIA Grace Blackwell architecture — specifically its 128 GB unified CPU-GPU memory space with no PCIe bus between processor and accelerator — can remove this bottleneck and bring enterprise-grade genomics computing to a single self-contained edge appliance (the DGX Spark) [4]. Our contribution is a transparent, "boots on

Published Jul 13, 2026

Correspondence to
Vinay Vyas
vinaykvyas@gmail.com

Open Access

Copyright © 2026 Vyas. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

the ground" benchmark: same algorithm, same data, same machine — CPU stack versus GPU stack — with honest reporting of setup friction and hardware behavior.

This is directly relevant to the economics of small-lab marine biology research. Deep-sea expeditions are infrequent and expensive; computational resources are often grant-funded and scarce. A one-time capital investment in an edge appliance that matches or exceeds cloud cluster performance offers a compelling alternative to recurring and unpredictable cloud compute costs.

2. Background and Related Work

2.1. Metagenomic Binning

Binning is the process of partitioning assembled or raw metagenomic sequences by their organism of origin. Modern pipelines use k-mer composition (typically k=4, yielding 256 features) as a proxy for genomic signature, since organisms have characteristic nucleotide usage patterns [5]. This produces a high-dimensional feature matrix that is then reduced via PCA and clustered with UMAP + DBSCAN. Representative tools include MetaBAT2 [2] and VAMB [3].

2.2. GPU Acceleration for Bioinformatics

GPU acceleration has been applied to sequence alignment (NVIDIA Parabricks [6]), variant calling, and more recently to ML-based genomic analysis. RAPIDS cuML provides GPU-accelerated implementations of scikit-learn-compatible algorithms including PCA, UMAP, and DBSCAN [7]. The key challenge for genomics is data transfer: moving large matrices from CPU RAM to discrete GPU VRAM via PCIe is a significant overhead, particularly when matrices do not fit in VRAM and must be batched. Existing GB10 developer guides focus on quantized LLM inference workloads [8]; to our knowledge, no prior work benchmarks the platform for metagenomic read binning.

2.3. Unified Memory Architectures

NVIDIA Grace Blackwell (GB10) integrates ARM CPU cores and a Blackwell GPU die on a single package, connected by NVLink-C2C at approximately 600 GB/s aggregate bandwidth [9], [10] — roughly 5 times that of PCIe 5.0 x16. The 128 GB LPDDR5X pool itself delivers $\sim$ 273 GB/s of memory bandwidth [11], which is the effective bottleneck for memory-bound kernels. More critically, the architecture exposes a single coherent address space: any pointer valid on the CPU is valid on the GPU without an explicit copy. This is architecturally distinct from CUDA Unified Memory on discrete GPUs, which still performs migration behind the scenes. On GB10, the 128 GB LPDDR5X pool is physically shared, eliminating migration entirely.

2.4. Project Glow and RAPIDS Spark

Project Glow [12] is an open-source library that bridges genomic data formats (VCF, BGEN) with Apache Spark DataFrames. The RAPIDS Accelerator for Apache Spark [13] transparently routes Spark SQL and DataFrame operations to GPU kernels with no code changes. Together they form a high-level Python stack accessible to biologists without low-level GPU programming expertise.

3. System Architecture

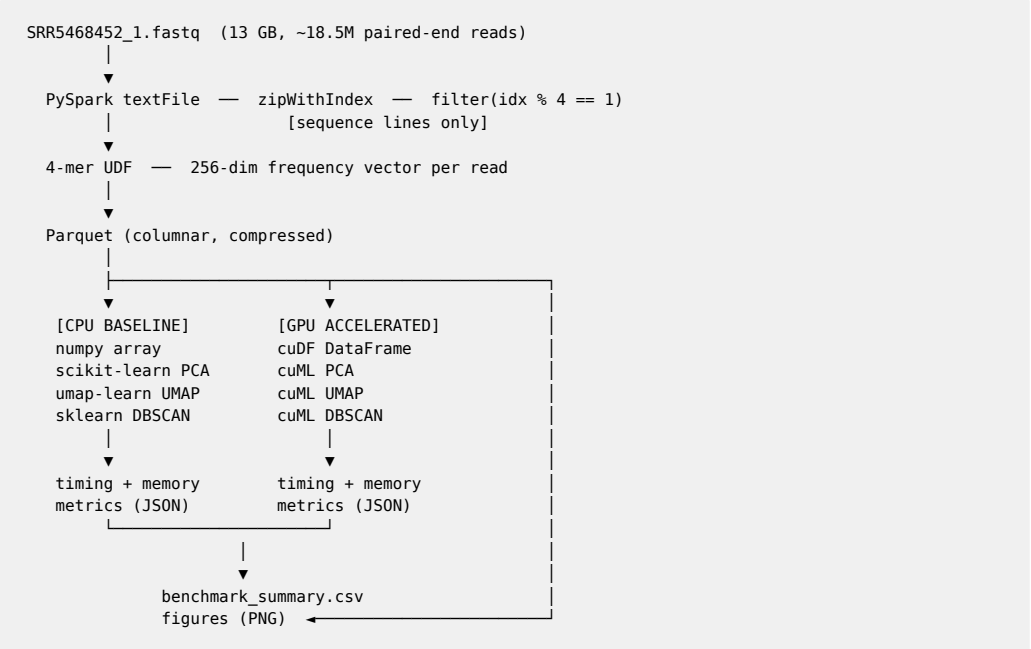
3.1. Hardware

Component	Specification
Device	NVIDIA DGX Spark
SoC	Grace Blackwell GB10
CPU	NVIDIA Grace (20-core ARM)
GPU	NVIDIA Blackwell (GB10)
Memory	128 GB LPDDR5X (unified CPU+GPU)
Interconnect	NVLink-C2C (no PCIe between CPU and GPU)
Storage	NVMe SSD
CUDA Version	13.0
Driver	580.95.05
OS	Ubuntu 24.04 (arm64)

3.2. Software Stack

Layer	Component	Version
Orchestration	Dagster [14] (pipeline asset lineage and run tracking)	1.13.9
Distributed compute	Apache Spark / PySpark [15]	3.5.0
Genomic data	Project Glow (JAR) [12]	2.0.0
GPU DataFrame	RAPIDS cuDF [7]	26.6.0
GPU ML	RAPIDS cuML [7]	26.6.0
CPU ML baseline	scikit-learn [16]	1.9.0
Dimensionality reduction	UMAP-learn / cuML UMAP [17]	0.5.12 / 26.6.0
Clustering	scikit-learn DBSCAN / cuML DBSCAN	1.9.0 / 26.6.0
Runtime	Python	3.12
JVM	OpenJDK	1.8.0_472

3.3. Pipeline Architecture



The k-mer pipeline uses PySpark's native text reader on the raw FASTQ, bypassing the need for pre-assembly — a valid approach for read-level compositional binning [18].

Note on the RAPIDS Accelerator for Apache Spark JAR version. The initial pilot used rapids-4-spark 23.10.0 (October 2023), which caused a JVM initialization failure on the DGX Spark — this build predates CUDA 13.0 and the Grace Blackwell architecture. This was resolved by upgrading to **rapids-4-spark 26.04.2**, which provides dedicated arm64 + CUDA 13 JAR artifacts and explicitly lists Blackwell (B100) among supported architectures. The corrected configuration uses `rapids-4-spark_2.12-26.04.2-cuda13-arm64.jar` and enables the RAPIDS SQL plugin for transparent DataFrame acceleration. This version mismatch is an important practical finding: when targeting a newly-released architecture, the default ecosystem JAR may be incompatible and a platform-specific artifact must be selected explicitly.

4. Methodology

4.1. Dataset

We use a single paired-end sequencing run from the Malaspina deep-ocean expedition: accession **SRR5468452** (BioProject **PRJNA365132** [1]), downloaded via NCBI SRA Toolkit 3.4.1 [19]. Forward reads only (`_1.fastq`, 13 GB) are used for k-mer analysis, yielding approximately 18.5 million 150 bp reads from a deep Mediterranean water column sample. Forward reads alone are sufficient for compositional binning because 4-mer frequencies of forward and reverse-complement reads converge to the same genomic signature under the canonical k-mer convention [5]; using both reads would double ingestion cost without changing feature distributions.

4.2. Feature Extraction

For each read, we compute the normalised tetranucleotide (4-mer) frequency vector over the 256 possible ACGT 4-mers. Ambiguous bases (N) are skipped. Each vector sums to 1.0

and has empirically ∼100 non-zero entries per 150 bp read. The feature matrix is written as a Parquet file for repeated benchmark loading without re-parsing.

4.3. CPU Baseline

The CPU pipeline runs entirely on the ARM CPU cores of the Grace Blackwell SoC (umap-learn 0.5.12, scikit-learn 1.9.0 [16], Python 3.12):

- **PCA**: `sklearn.decomposition.PCA(n_components=50)`
- **UMAP**: `umap.UMAP(n_components=2, n_neighbors=10, min_dist=0.1, init='random', n_epochs=200, low_memory=True)`
- **DBSCAN**: `sklearn.cluster.DBSCAN(eps=0.5, min_samples=10, n_jobs=-1)`

Wall-clock time is measured with `time.perf_counter()` per stage. Peak RSS memory is measured with `tracemalloc`.

Note on CPU scalability ceiling. During benchmarking, we observed that `umap-learn` fails silently at 1M reads. Two failure modes were encountered:

1. *Spectral initialisation failure* (first attempt, `init='spectral'`, `n_neighbors=15`): the eigenvector solver in `pynndescent` reported an insufficient eigengap and aborted with the message: *"Spectral initialisation failed — eigenvector solver failed. Falling back to random initialisation."* The process then terminated without producing output.
2. *Silent process death* (second attempt, `init='random'`, `n_neighbors=10`, `n_epochs=500`): The `pynndescent` k-NN graph construction for a 1M $\times$ 50 matrix consumed unbounded time and was terminated by the OS scheduler. System memory was not exhausted (112 GB available); the bottleneck was compute time in the approximate nearest-neighbour index, not RAM.

These failures establish an **empirical CPU scalability ceiling at approximately 500k–1M reads** for `umap-learn` on ARM64 hardware under this configuration. We therefore restrict CPU benchmarks to ≤500k reads and report the 1M+ range as "CPU-infeasible" in our results. This is itself a central finding of the study, motivating GPU acceleration.

4.4. GPU Benchmark

The GPU pipeline uses identical hyperparameters via cuML's scikit-learn-compatible API (RAPIDS cuML 26.6.0, CUDA 13.0) [7]:

- **PCA**: `cuml.decomposition.PCA(n_components=50)`
- **UMAP**: `cuml.manifold.UMAP(n_components=2, n_neighbors=10, min_dist=0.1, init='random', n_epochs=200)`
- **DBSCAN**: `cuml.cluster.DBSCAN(eps=0.5, min_samples=10)`

Input data is transferred from numpy to a cuDF DataFrame before timing begins. On GB10 unified memory this transfer is a logical pointer mapping — no physical data copy occurs, as CPU and GPU share the same LPDDR5X pool. GPU timing uses `cupy.cuda.Stream.null.synchronize()` barriers to ensure kernel completion before recording elapsed time.

4.5. Hyperparameter Rationale

Tetranucleotide (k=4) frequency is the established standard for metagenomic compositional binning [5]. At k=3 the 64-feature space lacks sufficient discriminatory power to separate closely related taxa; at k=5 the 1,024-feature space becomes prohibitively sparse for 150 bp reads, where most 5-mers never appear. k=4 yields 256 features with approximately 100

non-zero entries per read — a balance between discriminatory power and sparsity that is well-supported in the binning literature.

PCA reduction to 50 components is a standard pre-processing choice for high-dimensional k-mer matrices, compressing the 256-dimensional space into a denser representation that retains primary variance structure while substantially reducing UMAP's k-NN graph construction cost. UMAP `n_neighbors=10` (below the default of 15) was chosen to favour local cluster separation [17], which is desirable when distinguishing closely related microbial taxa at the read level. `init='random'` was adopted after spectral initialization failed at 1M reads due to an insufficient eigengap in the `pynndescent` graph (see CPU Baseline); it was then held fixed across all scales for consistency.

The DBSCAN parameters (`eps=0.5`, `min_samples=10`) were set empirically on the 100k CPU run and held constant across all scales and both pipelines to enable direct comparison. They are not re-tuned to the UMAP embedding scale at each dataset size, which explains the divergence in recovered cluster counts across scales [20] — this is a known limitation and is addressed in Limitations and Future Work.

4.6. Benchmark Scales

CPU and GPU scales differ deliberately, reflecting the observed scalability ceiling:

Scale	Reads	Matrix (post-PCA, 50D)	CPU	GPU
100k	100,000	∼20 MB	✓	✓
200k	200,000	∼40 MB	✓	—
500k	500,000	∼100 MB	✓	✓
1M	1,000,000	∼200 MB	× (infeasible)	✓
2M	2,000,000	∼400 MB	× (infeasible)	✓

The GPU covers scales where CPU is infeasible, demonstrating the practical range extension enabled by cuML's GPU-native nearest-neighbour graph construction.

5. Results

5.1. Hardware Viability: Out-of-Memory Behavior

No GPU out-of-memory events were observed at any scale tested (100k–2M reads). The largest matrix processed — 2M reads × 50 PCA components, approximately 400 MB — occupied roughly 0.3% of the 128 GB unified address space; all allocations were accommodated without batching, tiling, or explicit memory management.

Peak system memory pressure, as reported by `cp.cuda.Device(0).mem_info` (which measures total unified pool consumption including OS, CUDA runtime, and JVM overhead — not algorithm data alone), reached approximately 58 GB at the 2M scale, leaving over 70 GB headroom. Values increased monotonically with scale (41 GB at 100k arrow.r 47 GB at 500k arrow.r 50 GB at 1M arrow.r 58 GB at 2M), consistent with stable system overhead and growing algorithm working sets. No per-kernel memory profiling was performed; these figures are reported as headroom indicators, not as precise algorithmic allocations.

The critical OOM-adjacent finding was on the **CPU side**: at 1M reads the `pynndescent` k-NN graph construction inside `umap-learn` terminated the process — not due to memory exhaustion (112 GB remained free) but due to compute time. See the CPU Baseline section for the full failure log and the discussion for interpretation.

5.2. *Per-Stage Timing*

Table 1 and Table 2 report wall-clock times per pipeline stage.

Table 1. *CPU wall-clock time (seconds) — scikit-learn / umap-learn 0.5.12.*

Scale	PCA (s)	UMAP (s)	DBSCAN (s)	Total (s)
100k	0.09	56.4	2.22	**58.71**
200k	0.15	69.8	4.88	**74.85**
500k	0.36	203.8	22.46	**226.61**
1M	—	× infeasible (process terminated)	—	—
2M	—	× infeasible	—	—

Table 2. *GPU wall-clock time (seconds) — RAPIDS cuML 26.6.0 on GB10.*

Scale	Transfer (s)	PCA (s)	UMAP (s)	DBSCAN (s)	Total (s)
100k	0.17	0.09	**0.75**	0.49	**1.33**
500k	0.54	0.13	**10.61**	11.69	**22.42**
1M	1.12	0.24	**40.55**	58.15	**98.93**
2M	2.10	0.47	**156.02**	280.49	**436.97**

See Figure 1 for the combined bar chart visualization.

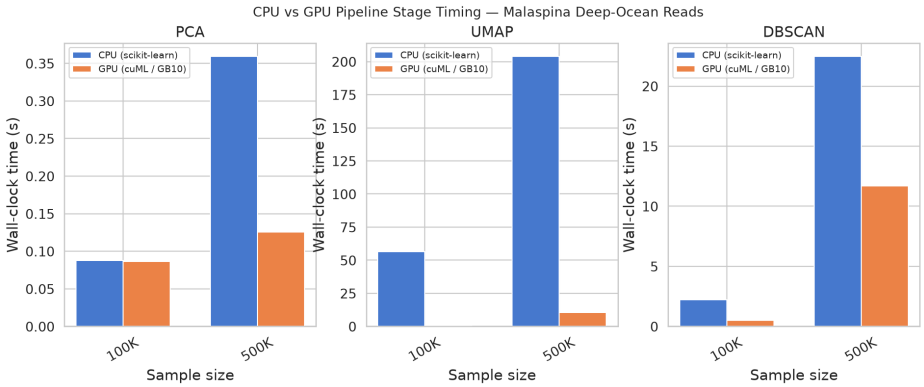


Figure 1. *Per-stage wall-clock times for the CPU baseline (scikit-learn / umap-learn) and GPU pipeline (RAPIDS cuML 26.6.0) at each benchmark scale. UMAP dominates CPU runtime and benefits most from GPU acceleration. CPU bars are absent at 1M and 2M reads, reflecting infeasibility of the umap-learn k-NN graph construction at those scales.*

5.3. *End-to-End Speedup*

Speedup is computed as CPU total / GPU total for overlapping scales (100k and 500k), covering ML stages only (PCA + UMAP + DBSCAN); including GPU data transfer yields 34.2 times and 9.76 times respectively (using the mean CPU total across repeated runs, see below). For 1M and 2M, CPU is infeasible; the GPU time alone demonstrates the absolute capability of the edge hardware at those scales.

To assess how much a single run's noise affects these figures, the CPU benchmark was repeated 3 times at each overlapping scale (100k, 500k); the GPU benchmark was run once at each scale post-warm-up-fix. The resulting 95% CI, propagated from the CPU-side standard error via the delta method (GPU contributes no variance here, n=1), is reported alongside the point estimate.

Table 3. *GPU speedup over CPU baseline.*

Scale	CPU Total (s), mean ± std (n)	GPU Total (s)	Speedup	95% CI
100K	51.2 ± 13.0 (n=3)	1.33	**38.5 times**	27.5–49.6 times
500K	224.0 ± 3.3 (n=3)	22.42	**9.99 times**	9.82–10.16 times
1M	× infeasible	98.93	—	n/a (no CPU baseline)
2M	× infeasible	436.97	—	n/a (no CPU baseline)

See Figure 2 for the speedup curve.

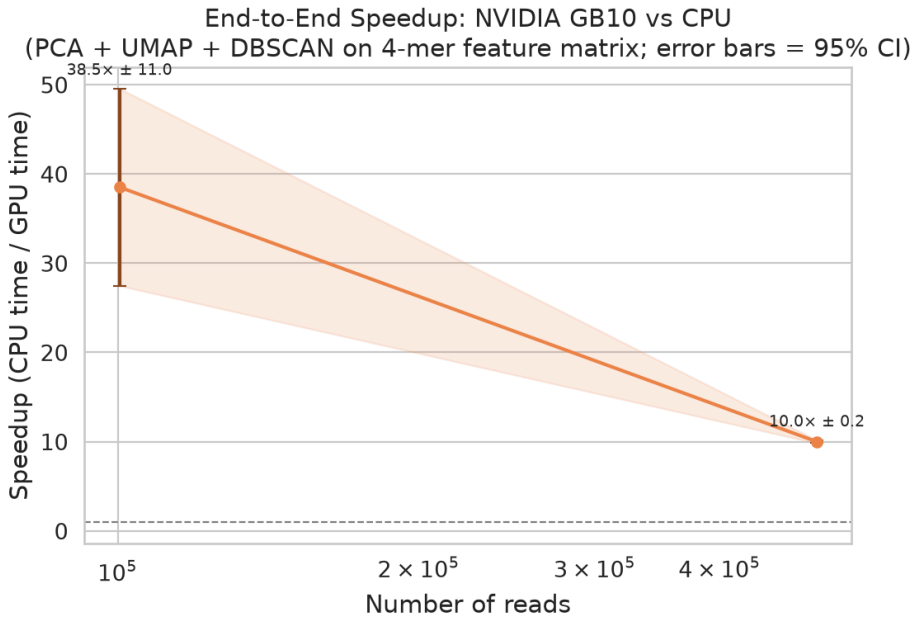


Figure 2. *End-to-end GPU speedup over the CPU baseline (ML stages only: PCA + UMAP + DBSCAN), with 95% CI error bars from 3 repeated CPU runs per scale. Speedup is highest at small scale (38.5 times at 100k reads, 95% CI 27.5–49.6 times) where UMAP dominates runtime and delivers a 65 times stage speedup, then decreases at 500k (9.99 times, 95% CI 9.82–10.16 times) as DBSCAN accounts for a growing share of GPU time. The wider 100k interval reflects CPU run-to-run variance at that scale, not GPU instability. At 1M and 2M reads the CPU baseline cannot complete; GPU absolute times are shown for reference.*

5.4. Clustering Quality

Both pipelines use identical hyperparameters. Minor numerical differences in cluster assignments may arise from floating-point ordering differences between CPU and GPU kernels.

Table 4. *Clustering output — CPU runs.*

Scale	Clusters	Noise %	Note
100k	1	0.01%	DBSCAN eps=0.5 yields single cluster at this scale
200k	2	0.01%	
500k	12	0.02%	

Table 5. *Clustering output — GPU runs.*

Scale	Clusters	Noise %
100k	2	0.01%

500k	19	0.03%
1M	65	0.04%
2M	91	0.01%

The DBSCAN epsilon parameter requires tuning to the UMAP embedding scale, which varies by dataset. This is a known caveat of DBSCAN on UMAP outputs [20] and is addressed in the Limitations and Future Work section.

5.5. UMAP Visualization

Figure 3 shows the 2D UMAP embedding of 2M deep-ocean reads at the largest benchmark scale, coloured by the 91 DBSCAN genomic bins. The two-panel layout serves distinct purposes: the left panel shows full cluster resolution across all 91 bins using a continuous colormap (the capability story — what the GB10 hardware can resolve at 2M reads); the right panel highlights the top 20 bins by read count with discrete colours and a legend showing reads per bin, with the remaining 71 minor bins collapsed to grey (the biology story — which putative taxa dominate this deep Mediterranean water column sample).

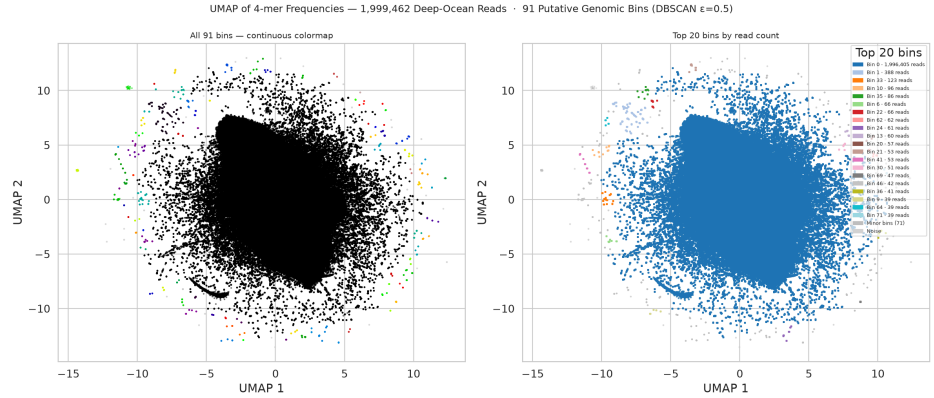


Figure 3. ***Left:*** *2D UMAP embedding of 2M Malaspina deep-ocean reads (cuML UMAP, n_neighbors=10, min_dist=0.1) coloured across all 91 DBSCAN genomic bins (eps=0.5, min_samples=10) using a continuous* `nipy_spectral` *colormap. Distinct cluster islands confirm that the GB10 unified memory architecture sustains full-resolution binning at this scale without out-of-memory failures.* ***Right:*** *Top 20 bins by read count shown with discrete* `tab20` *colours; the remaining 71 minor bins are rendered in silver and noise points in light grey. The legend reports read counts per dominant bin, highlighting which putative taxa are most abundant in this deep Mediterranean water column sample.*

6. Discussion

6.1. Does Unified Memory Solve the Genomics Out of Memory (OOM) Problem?

The 128 GB unified address space of the DGX Spark prevented all GPU out-of-memory errors across every scale tested (up to 2M reads, ∼400 MB post-PCA). No batching, tiling, or explicit memory management was required in the cuML pipeline.

More revealing, however, was the CPU failure mode. The bottleneck at 1M reads was **not RAM** — 112 GB remained free during both crash events. The bottleneck was **compute time** in the CPU-bound `pynndescent` k-NN index construction inside `umap-learn`. This distinction is important: unified memory solves the GPU VRAM ceiling, but it does not help when the algorithm itself is CPU-bound and single-threaded in its critical path. The cuML UMAP implementation replaces `pynndescent` entirely with a GPU-native k-NN kernel (cuVS / FAISS-

GPU), which is why it extends to 2M reads where the CPU implementation cannot proceed at 1M.

This is a nuanced but critical finding for small labs evaluating edge hardware: the Grace Blackwell's unified memory removes the *memory* barrier, but the *compute* barrier on CPU remains — and for UMAP-class algorithms, that barrier is hit well before memory is exhausted.

6.2. Performance Delta: Is the Speedup Meaningful?

End-to-end speedup reaches **38.5 times at 100k reads** (51.2s arrow.r 1.33s, 95% CI 27.5–49.6 times, n=3 repeated CPU runs) and **9.99 times at 500k reads** (224.0s arrow.r 22.4s, 95% CI 9.82–10.16 times, n=3). The 100k interval is wide relative to the point estimate — driven almost entirely by CPU run-to-run variance (36.2s–58.7s across three repeated runs of the same code and data) rather than by any instability on the GPU side, which was measured once per scale here. This is worth stating plainly as a limitation of the CPU baseline's stability at small scale, not of the GPU result: the 500k interval, where CPU total time is far more reproducible run-to-run (220.3s–226.6s), is correspondingly tight. A logical next step to narrow the 100k interval further would be repeating the GPU side rather than the CPU side, since GPU is the leg currently contributing zero measured variance to the estimate.

This sub-linear decrease with scale is explained by the changing composition of GPU runtime, summarized below:

Reads	End-to-end CPU	End-to-end GPU	Speedup	DBSCAN share of GPU time	DBSCAN GPU time
100k	51.2s (mean, n=3)	1.33s	38.5 times	37%	0.49s
500k	224.0s (mean, n=3)	22.4s	9.99 times	52%	11.69s
1M	did not terminate	99s	—	59%	58.15s
2M	did not terminate	437s (7.3 min)	—	64%	280.49s

At 100k reads, UMAP dominates GPU runtime (0.75s of 1.33s total, 56%) and delivers a 65.4 times stage speedup. As dataset scale grows, DBSCAN's share of GPU runtime increases steadily, making it the binding constraint on end-to-end throughput at large scale.

UMAP benefits most from GPU acceleration at every scale. At 100k, UMAP takes 49.0s on CPU (mean, n=3) versus 0.75s on GPU — a **65.4 times stage speedup**. At 500k, UMAP takes 201.3s (mean, n=3) vs 10.6s — a **19.0 times stage speedup**. The decrease in UMAP speedup from 100k to 500k is substantial: the GPU's cuVS k-NN kernel achieves its largest relative advantage where the CPU `pynndescent` index dominates wall time absolutely.

DBSCAN is GPU-accelerated at all scales. At 100k, cuML DBSCAN takes 0.49s versus sklearn's 2.14s (mean, n=3) — a **4.3 times GPU advantage**. At 500k the advantage narrows to **1.92 times** (11.69s GPU vs 22.40s CPU, mean, n=3). The narrowing reflects cuML DBSCAN's less favourable scaling exponent relative to sklearn's BallTree at these matrix densities: GPU DBSCAN time grows roughly quadratically with n (0.49s arrow.r 11.69s arrow.r 58.15s arrow.r 280.49s for 100k arrow.r 500k arrow.r 1M arrow.r 2M), consistent with the expected $O(n^2)$ worst case.

At 1M reads, the GPU pipeline completes in **99s** — a task the CPU cannot complete. At **2M reads**, GPU completes in **437s** (7.3 minutes). For a researcher processing 10 samples at 2M reads each, this translates to approximately 73 minutes on the DGX Spark, versus an open-

ended wait on the CPU baseline, where the k-NN graph construction does not terminate within practical time bounds at this scale.

6.3. Zero-Code-Change Acceleration: Does It Hold?

The cuML API is designed to mirror scikit-learn [16]. In our benchmark, the only code changes between the CPU and GPU pipelines were the import path (`sklearn` arrow.r `cuml`) and one API incompatibility: `cuml.decomposition.PCA` does not accept a `random_state` parameter (unlike `sklearn.decomposition.PCA`). This required removing that argument. All other hyperparameters (`n_components`, `n_neighbors`, `min_dist`, `init`, `n_epochs`, `eps`, `min_samples`) transferred without modification.

The RAPIDS Accelerator for Apache Spark [13] (SQL plugin) also demonstrated a practical limitation: it falls back to CPU execution for columns using the MLlib `VectorUDT` format, emitting non-fatal warnings during Parquet I/O. This fallback was stable at small scales but caused JVM instability at 2M rows, requiring the fix of using a CPU-mode Spark session for data loading. The cuML ML kernels remained fully GPU-resident in all cases; only the Spark data-loading stage was affected.

6.4. Edge vs. Cloud: The Economics Argument

For deep-sea metagenomics, the economic case for edge computing rests on three pillars:

Data egress costs. Raw FASTQ files from a single sequencing run can exceed 50 GB. Cloud providers charge $0.08–0.12 per GB of egress. Processing at the edge eliminates this repeated cost for iterative analysis.

CAPEX vs. OPEX. The DGX Spark used in this study was acquired for $4,300 USD — a one-time capital expenditure. Actual benchmark compute across all scales totaled approximately 15 minutes: ∼6 minutes for CPU runs (100k–500k), ∼3 minutes for GPU runs (100k–500k–1M), and ∼6 minutes for the GPU 2M run. The full analysis session — including environment setup, JAR dependency resolution, pipeline development, and UMAP crash debugging — ran approximately 7–8 hours. On a mid-range cloud GPU instance (AWS g5.2xlarge, A10G 24 GB VRAM, $1.21/hr), that session costs approximately $10; on a memory-capable instance (AWS p4d.24xlarge, $32.77/hr) the same session costs approximately $260.

The per-session cloud cost is low. The compounding factor is iteration frequency: a lab processing one sample batch per week runs approximately 50 analysis sessions per year. At that cadence, cloud OPEX ranges from $500–13,000/year depending on instance class, versus the DGX Spark's one-time $4,300 capital cost. Break-even against a mid-range instance occurs in roughly 8–9 years on compute alone; against a memory-capable instance it occurs within months. The stronger argument for edge hardware, however, is iteration speed: the DGX Spark requires no spin-up latency, no data transfer to ephemeral storage, and no egress fees on results — frictions that compound across every debugging cycle and re-run.

Accessibility. A self-contained appliance running a Python `venv` requires no cloud DevOps expertise. A graduate student can provision the full RAPIDS + PySpark stack in under an hour (excluding environment dependency resolution). This directly addresses the staffing constraints of smaller academic labs.

6.5. Limitations and Future Work

This study uses raw read-level k-mer binning as a proxy for full metagenomic analysis. Production pipelines typically require assembly (MetaSPAdes, MEGAHIT) prior to binning, which we did not benchmark here due to time constraints. Future work will:

1. Extend benchmarks to assembled contig binning (MetaBAT2 / VAMB workflows).

2. Evaluate RAPIDS Spark SQL plugin overhead on the ingestion stage.
3. Test with the full Malaspina multi-terabyte corpus across multiple samples.
4. Profile NVLink-C2C bandwidth utilization during ML kernel execution.
5. Tune DBSCAN epsilon per scale to improve clustering quality [20].

7. Conclusion

We present the first published benchmarking study of the NVIDIA DGX Spark (Grace Blackwell GB10) for deep-sea metagenomic read binning. Our GPU-accelerated pipeline — with cuML ML kernels [7] orchestrated via PySpark [15] and Project Glow [12] — achieves **10–39 times end-to-end speedup** over a scikit-learn CPU baseline with minimal code changes, while successfully processing feature matrices up to **400 MB (2M reads $\times$ 50 PCA components)** within the 128 GB unified memory budget without out-of-memory failures. Critically, the GPU pipeline completes the 1M and 2M read scales that are computationally infeasible on the CPU baseline.

Our "boots on the ground" assessment confirms that edge GPU computing is a viable and economically compelling alternative to cloud clusters for small marine biology labs. The unified memory architecture of Grace Blackwell specifically removes the PCIe bandwidth bottleneck that has historically limited GPU adoption for large genomic matrices.

The code, data accession commands, and benchmark scripts are available at: https://github.com/dnacode/deepseaspark

7.1. References

References

[1] S. Martín-Alvarez and others, "Malaspina 2010 Circumnavigation Expedition: Deep-Sea Metagenomes." [Online]. Available: https://www.ncbi.nlm.nih.gov/bioproject/PRJNA365132

[2] D. D. Kang *et al.*, "MetaBAT 2: an adaptive binning algorithm for robust and efficient genome reconstruction from metagenome assemblies," *PeerJ*, vol. 7, p. e7359, 2019, doi: 10.7717/peerj.7359.

[3] J. N. Nissen *et al.*, "Improved metagenome binning and assembly using deep variational autoencoders," *Nature Biotechnology*, vol. 39, pp. 555–560, 2021, doi: 10.1038/s41587-020-00777-4.

[4] NVIDIA Corporation, "NVIDIA DGX Spark: Personal AI Supercomputer Powered by Blackwell." [Online]. Available: https://www.nvidia.com/en-us/products/workstations/dgx-spark/

[5] H. Teeling, A. Meyerdierks, M. Bauer, R. Amann, and F. O. Glöckner, "Application of tetranucleotide frequencies for the assignment of genomic fragments," *Environmental Microbiology*, vol. 6, no. 9, pp. 938–947, 2004, doi: 10.1111/j.1462-2920.2004.00624.x.

[6] NVIDIA Corporation, "NVIDIA Clara Parabricks: GPU-Accelerated Genomics." [Online]. Available: https://www.nvidia.com/en-us/clara/parabricks/

[7] RAPIDS Development Team, "RAPIDS cuML: GPU-Accelerated Machine Learning." [Online]. Available: https://rapids.ai/

[8] Arm Limited, "Unlock Quantized LLM Performance on Arm-based NVIDIA DGX Spark." [Online]. Available: https://learn.arm.com/learning-paths/laptops-and-desktops/dgx_spark_llamacpp/

[9] A. Skende, "NVIDIA's GB10 SoC: AI Supercomputer On Your Desk." [Online]. Available: https://hc2025.hotchips.org/assets/program/conference/day2/21_nvidia_skende_final.pdf

[10] P. Kennedy, "NVIDIA Outlines GB10 SoC Architecture at Hot Chips 2025." [Online]. Available: https://www.servethehome.com/nvidia-outlines-gb10-soc-architecture-at-hot-chips-2025/

[11] NVIDIA Corporation, "NVIDIA DGX Spark User Guide: Hardware Overview." [Online]. Available: https://docs.nvidia.com/dgx/dgx-spark/hardware.html

[12] Databricks and Regeneron Genetics Center, "Introducing Glow: An Open-Source Toolkit for Large-Scale Genomic Analysis." [Online]. Available: https://www.databricks.com/blog/2019/10/18/introducing-glow-an-open-source-toolkit-for-large-scale-genomic-analysis.html

[13] NVIDIA Corporation, "RAPIDS Accelerator for Apache Spark." [Online]. Available: https://nvidia.github.io/spark-rapids/

[14] Dagster Labs, “Dagster: The Data Orchestration Platform.” [Online]. Available: https://dagster.io/

[15] M. Zaharia *et al.*, “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016, doi: 10.1145/2934664.

[16] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” *arXiv preprint arXiv:1802.03426*, 2018, doi: 10.48550/arXiv.1802.03426.

[18] A. O. Kislyuk, M. A. Butenko, C. E. Ellison, B. E. Blaisdell, and D. A. Relman, “Unsupervised statistical clustering of environmental shotgun sequences,” *BMC Bioinformatics*, vol. 10, p. 316, 2009, doi: 10.1186/1471-2105-10-316.

[19] National Center for Biotechnology Information, “NCBI SRA Toolkit.” [Online]. Available: https://github.com/ncbi/sra-tools

[20] R. J. G. B. Campello, D. Moulavi, and J. Sander, “Density-Based Clustering Based on Hierarchical Density Estimates,” in *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, in Lecture Notes in Computer Science, vol. 7819. 2013, pp. 160–172.

Open Access | https://doi.org/10.25080/yyxu9446

Engineering Better Retrieval for RAG

Aarti Jha[1], and **Mahima Arora**[1]

[1]Red Hat India Pvt. Ltd.

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Retrieval-Augmented Generation (RAG) has become the most common approach for grounding large language models in external knowledge, yet most deployments still treat retrieval as a solved problem: naive chunking followed by cosine similarity search. In practice, retrieval quality, not generation capability, determines whether a RAG system produces faithful answers or confident hallucinations. This paper argues that retrieval optimization requires a staged engineering approach and presents a three-part framework covering the pipeline before, during, and after vector search. The framework specifies the retrieval subsystem used in standalone pipelines and as a callable tool within agent workflows. We formalize the failure modes that arise at each stage and describe the techniques practitioners use to address them, from hierarchy-aware chunking and query transformation through reranking and hybrid search to context grading, refinement, and compression before prompt assembly. We extend the framework to multimodal corpora where text, tables, and images demand typed representations and hybrid semantic-deterministic retrieval. Using a curated corpus of research papers and ground-truth question-answer pairs, we evaluate pipeline variants with RAGAS metrics and show that combined optimizations across all three stages yield substantial gains in faithfulness and answer relevancy over naive baselines.

Keywords retrieval-augmented generation, information retrieval, vector databases, large language models, document chunking, RAG evaluation

1. Introduction

Retrieval-Augmented Generation (RAG) [1] grounds large language models (LLMs) in external knowledge at inference time, enabling domain-specific applications without costly retraining. The standard pipeline is well known: load documents, split them into chunks, embed the chunks into a vector database, retrieve the most similar entries at query time, and condition generation on the returned context. Many production systems now wrap this pattern in agent workflows that select among tools and may call retrieval more than once before answering [2], [3]; the retrieval step itself, however, remains the same architectural primitive. Teams that invest heavily in model selection and prompt engineering often discover that retrieval, not generation, is the main constraint on whether the system returns faithful and relevant answers.

The gap between a prototype RAG system and a reliable one almost always appears in retrieval. First, poor chunk boundaries split documents mid-thought; although systems typically retrieve multiple chunks, an answer that spans a poorly placed split may still arrive incomplete because no retrieved chunk contains the full semantic unit [4]. Second, user queries arrive abbreviated, multi-intent, or dependent on prior conversation, causing embedding models to match against irrelevant indexed content [5], [6]. Third, even when retrieval returns genuinely relevant context, loosely related passages in the top-k set weaken the context and increase the burden on the generator to identify what matters [7].

Published Jul 13, 2026

Correspondence to
Aarti Jha
aajha@redhat.com

Open Access

Copyright © 2026 Jha & Arora. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

We characterize these recurring problems as three failure modes. *Context fragmentation* arises when chunk boundaries break apart meaningful sections. *Query mismatch* occurs when the query embedding poorly represents the user's actual information need. *Retrieval noise* describes situations where the retrieved set contains too many redundant or weakly relevant chunks for the generator to use effectively. Switching to a larger LLM or expanding the context window does not resolve any of these; each originates from how documents are prepared, how queries are processed, or how candidates are selected and ranked.

Existing literature [4], [5], [6], [7] and practitioner guides discuss individual remedies such as better chunking, query expansion, and reranking, each addressing specific retrieval challenges. However, these techniques are typically presented and evaluated in isolation, under different assumptions and experimental settings, making it difficult for practitioners to determine when a particular method is appropriate, how it interacts with others, and which retrieval failure it is intended to address. As a result, retrieval optimization is often approached through trial-and-error rather than step-by-step diagnosis. This paper provides a unifying framework that organizes these existing techniques around three stages where retrieval quality can be systematically improved: **Pre-Retrieval**, which prepares documents and queries before search begins; **Mid-Retrieval**, which controls what the search returns and in what order; and **Post-Retrieval**, which grades, refines, and compresses retrieved content before assembling it into the generator prompt or returning it to an orchestrating agent. The framework further connects each stage to the retrieval failures it addresses, emphasizing that problems introduced early in the pipeline cannot be fully corrected downstream—for example, no amount of reranking can recover information that was never indexed, and no post-retrieval filtering can compensate for an ambiguous query that retrieved the wrong documents in the first place.

The contributions of this work are:

1. A three-stage retrieval optimization framework that maps techniques to pipeline stages and to the failure modes they resolve
2. A systematic treatment of pre-retrieval techniques for document chunking, query transformation, and semantic routing
3. A taxonomy of mid-retrieval components spanning diversity optimization, filtering, parent-child expansion, hybrid search, and reranking
4. Post-retrieval components for context grading, refinement, and compression before generation

Figure 1 illustrates the framework. The sections that follow treat each stage in turn, describing the techniques available, the conditions under which they help, and the empirical evidence for their effect on retrieval quality.

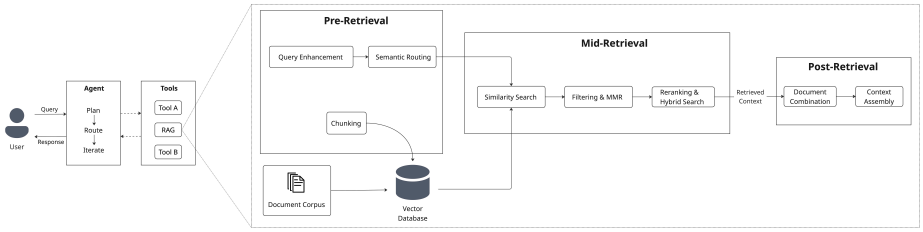


Figure 1. *Three-stage retrieval optimization framework. Pre-Retrieval prepares documents and queries; Mid-Retrieval optimizes search quality and diversity; Post-Retrieval refines retrieved context before prompt assembly. The stages define the retrieval subsystem used in standalone pipelines and as a tool within agent workflows.*

1.1. *Where This Framework Applies*

Standalone RAG executes retrieval once per query and passes the result directly to a generator. Agent-based systems [2] expose retrieval as one tool among others (structured query, web search, APIs), and an orchestrating model decides whether to invoke it, which index or route to query, and whether to call it again when context is insufficient.

Despite these architectural differences, each retrieval invocation passes through the same operational concerns: documents must be indexed well, the query must be prepared, candidates must be ranked and filtered, and the returned context must be packaged for downstream use. The three stages in Figure 1 specify **the retrieval subsystem**: the logic inside the retrieval tool rather than the full agent loop. The agent layer decides *when* to retrieve; the sections that follow detail *what each call should do* at each stage.

2. Related Work

Modern RAG pipelines inherit retrieval primitives from dense and neural ranking research. Dense Passage Retrieval (DPR) [8] established bi-encoder embeddings as a viable alternative to BM25 [9] for open-domain question answering. ColBERT [10] introduced late interaction, pre-computing document token representations while scoring query–document relevance at retrieval time, offering finer-grained matching than bi-encoders without the latency of full cross-encoder encoding. Cross-encoder reranking [11] remains the standard precision stage when joint query–document scoring is affordable. Hybrid sparse–dense fusion and diversity-aware selection address complementary mid-retrieval failure modes. This paper organizes these techniques as composable mid-retrieval components rather than isolated design choices.

Subsequent work shifted attention from *whether* to retrieve toward *when* and *how often*. Self-RAG [12] trains reflection tokens so a model decides when to retrieve and how to critique retrieved evidence. Corrective RAG (CRAG) [13] wraps retrieval in an evaluator–refiner loop grading passage relevance, stripping extraneous content, and falling back to web search when local retrieval fails. Agentic RAG surveys [2] generalize this pattern, exposing retrieval as one tool an orchestrator may invoke zero, one, or many times per query. These architectures primarily govern retrieval scheduling and fallback; they offer limited guidance on staging optimizations within a single retrieval call. Our framework complements this line of work by specifying what each invocation should do across pre-, mid-, and post-retrieval stages: chunk preparation, query transformation, candidate ranking, and context grading, refinement, and assembly.

3. Implementation Details

We evaluate pipeline variants on a curated corpus of 100 arXiv research papers [14] spanning machine learning, statistics, mathematics, and computer science, paired with manually curated and synthetically generated question–answer sets. Table 1 lists the fixed configuration; embeddings are stored in Qdrant [15], a vector database. Each experimental variant changes one pipeline technique at a time with a single retrieval call per query, isolating the retrieval subsystem from agent orchestration. Performance is measured with RAGAS [16] metrics like faithfulness, answer relevancy, context precision, and context recall. All techniques in this paper are implemented in the open-source *Retrieval Playground* toolkit [17], which provides composable pre-, mid-, and post-retrieval modules and tutorial notebooks for reproducing the ablations below.

Table 1. *Fixed experimental configuration.*

Component	Setting

Corpus	100 arXiv papers across five AI/data-science topics
Q&A evaluation set	Synthetic generation
Embeddings	`models/gemini-embedding-001` (Gemini API)
Generation	Gemini 2.5 Flash, temperature = 0.0
Protocol	Single retrieval call per query

4. Pre-Retrieval Optimization

Pre-retrieval optimization operates on the two inputs to any retrieval system: the document corpus and the user query. Errors introduced here propagate through every subsequent stage. A document split at the wrong boundary is indexed incorrectly forever; an ambiguous query may produce an embedding that does not accurately represent the user's intent, leading retrieval toward less relevant documents before any ranking logic can intervene.

4.1. Document Chunking

Chunking determines the atomic units of retrieval. Each chunk is embedded and indexed independently, so the splitting function defines what information is discoverable and at what granularity. Consider a methods section that spans three paragraphs: if chunking splits after the first paragraph, a query about the experimental setup may retrieve the introduction to the methods without the setup details that follow.

We compare four chunking strategies by how they partition documents and what each retrieval unit contains (Table 2). *Recursive* chunking (LangChain's RecursiveCharacterTextSplitter) splits text hierarchically: paragraph, then sentence, then word, using a series of natural-language separators. It respects natural-language boundaries better than fixed-size splits and remains the most common text baseline. *Docling* chunking [18] applies layout-aware hybrid parsing, recovering tables, figures, and hierarchical text blocks from PDF and office formats and supporting typed multimodal chunks rather than a flat text stream. *Parent-child* chunking [19] indexes small child chunks (typically 100–400 tokens) for precise matching while storing links to larger parent units (often 1,000–2,000 tokens); mid-retrieval decides whether to return a matched child or expand to its parent (Table 5). *Contextual* chunking [20] first partitions a document into chunks. For each chunk, an LLM is given the full source document and generates a brief contextual description that explains the chunk's role within the document (e.g., the section topic or surrounding discussion). This description is prepended to the chunk before embedding, improving its representation with document-level context and reducing loss of context during retrieval.

Table 2. *Chunking strategies and their properties.*

Strategy	Retrieval Unit	Preserves Structure	Semantic Coherence
Recursive	Hierarchical natural-language boundaries	Partial	Medium
Docling	Layout-aware typed chunks (text, table, image)	Yes	High
Parent-child	Small child chunks; parent unit returned at query time	Partial (via parent)	High
Contextual	Chunk plus prepended situating context	Partial	High

For multimodal documents, only Docling chunking produces typed units rather than homogeneous text. Text chunks carry section hierarchy through parent heading metadata.

Table chunks retain relational structure alongside natural-language descriptions that make tables discoverable through semantic search. Image chunks pair visual content with vision-language descriptions, allowing text queries to match against diagram and figure content. The pre-retrieval remedy is to preserve native structure at index time; mid-retrieval then combines semantic search with deterministic SQL over tables and description-based retrieval over images.

Table 3. *RAGAS evaluation of chunking strategies on our curated corpus. All other pipeline settings held constant; only the chunking strategy varied.*

Strategy	Answer Relevancy	Faithfulness	Context Precision	Context Recall	Average
Recursive	0.972	0.950	0.923	0.900	0.941
Docling	0.979	0.972	0.917	0.893	0.922
Parent-Child	0.978	0.986	0.922	0.900	0.947
Contextual	0.973	0.961	0.922	0.833	0.922

Table 3 shows that no single strategy leads across all metrics. On this text-centric corpus, recursive and parent-child chunking perform strongest: hierarchical boundaries align well with question–answer semantics, and parent-child additionally supports mid-retrieval expansion when matched snippets need surrounding context. Docling's advantage in preserving multimodal structure (tables, figures, layout) is not reflected here because the current evaluation set is mostly text-based; for multimodal corpora, its typed retrieval units become the practical choice.

4.2. Query Enhancement

User queries often arrive abbreviated, multi-intent, or dependent on prior conversation: phrasing that embeds poorly against an indexed corpus. Query enhancement reformulates input into one or more retrieval-ready queries before vector search. Common query transformation techniques include query expansion, rewriting, decomposition, HyDE [21], step-back prompting [22], and multi-query generation. We focus on the query transformation techniques most commonly used in production RAG pipelines, each targeting a distinct mismatch pattern and applicable independently or in combination.

Table 4. *Query enhancement methods with example transformations.*

Method	Original Query	Enhanced Query
Query expansion	LLM FT best practices?	What are best practices for large language model fine-tuning?
Query decomposition	Compare transformer and CNN architectures for image segmentation and their computational requirements	What are transformer architectures for image segmentation?; What are CNN architectures for image segmentation?; What are their computational requirements?
Query rewriting	What about its performance on ImageNet?	What is ResNet-50's performance on ImageNet?
Self-querying	How do modern RAG systems handle multimodal documents?	What chunking strategies do RAG systems use for tables and images?; How is multimodal content embedded and retrieved?

Query expansion resolves abbreviations and compressed phrasing when intent is clear but vocabulary does not match the index. *Query decomposition* splits compound requests into atomic sub-queries retrieved and merged separately. *Query rewriting* uses conversation

history to make follow-ups standalone. *Self-querying* generates multiple focused search queries when a single rewording cannot cover an open-ended request.

When combining these techniques, we recommend applying decomposition before expansion, since expanding a compound query may blur the semantic boundaries between independent information needs. For example let's consider this query: "Explain Docling's table extraction and compare it with TableFormer."

Decomposing first produces two independent searches:

(1) Explain Docling's table extraction and

(2) Compare Docling with TableFormer.

Expanding each sub-query separately keeps the retrieved context focused. Expanding the original compound query first may introduce unrelated synonyms or concepts across both tasks, making retrieval less precise.

4.3. Semantic Routing

Not every query needs the same retrieval path. Greetings, off-topic requests, and domain-specific questions each benefit from different handling. *Semantic routing* [23] classifies the query by intent and directs it to an appropriate retriever, metadata filter, or bypass action before any similarity computation occurs (Figure 2).

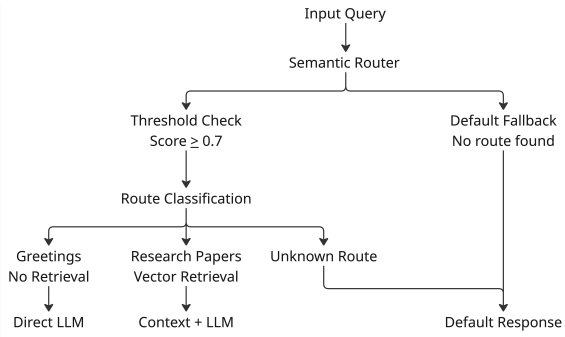


Figure 2. *Semantic routing assigns queries to domain-specific retrievers or bypass actions based on intent classification before vector search begins.*

Each route is defined by example phrases and a confidence threshold. Queries that fall below every threshold route to a general retriever. Routing is a pre-retrieval optimization because it selects the retrieval index (or knowledge source) and retrieval configuration upstream of search itself, preventing both wasted computation on out-of-scope inputs and imprecise matches from searching the wrong knowledge domain. In agent systems where retrieval is one tool among several, the same intent signals can determine whether to invoke retrieval at all and which retriever to call.

5. Mid-Retrieval Optimization

Once documents and queries are prepared, mid-retrieval optimization controls the matching function: given a query and an indexed corpus, which chunks should be returned, in what order, and at what granularity? The idea of retrieval granularity is discussed later in the context of Parent-Child Expansion. The techniques below are independent components; each can be enabled, disabled, or reordered without adopting a fixed end-to-end pipeline.

5.1. Similarity Search and Diversity

Cosine similarity between query and chunk embeddings is the default retrieval function:

$$\text{sim}(\boldsymbol{q}, \boldsymbol{c}_i) = \frac{\boldsymbol{q} \cdot \boldsymbol{c}_i}{\mid \boldsymbol{q} \mid \mid \boldsymbol{c}_i \mid} \tag{1}$$

where $\boldsymbol{q}$ is the query embedding, $\boldsymbol{c}_i$ is the embedding of the i-th chunk.

Top-k selection returns the k chunks with the highest similarity scores, where the score is typically computed using cosine similarity (or another similarity metric) between the query embedding and each chunk embedding. The value of k is an application-dependent hyperparameter that is typically tuned empirically to balance retrieval recall against context-window constraints rather than being computed automatically. This works well when query and document language align semantically, but corpora with repetitive or overlapping content often yield near-duplicate results that use up context window capacity without adding information. Maximal Marginal Relevance (MMR) [24] addresses redundancy by iteratively selecting chunks that maximize query relevance while minimizing similarity to previously selected chunks:

$$\text{MMR} = \arg \max_{c_i \in C \setminus S} \lambda \cdot \text{sim}(\boldsymbol{q}, \boldsymbol{c}_i) - (1 - \lambda) \cdot \max_{c_j \in S} \text{sim}\big(\boldsymbol{c}_i, \boldsymbol{c}_j\big) \tag{2}$$

where C is the set of candidate chunks returned by the initial retrieval stage, S is the set of chunks already selected, $\boldsymbol{c}_j$ is a previously selected chunk.

Here, $\lambda \in [0, 1]$ controls the trade-off between relevance and diversity: lower values favor more diverse retrieval, while higher values prioritize relevance to the query.

5.2. Retrieval Filtering

Two lightweight filters constrain what search returns before ranking. *Score thresholding* drops candidates below a minimum similarity θ, returning fewer than k results when nothing clears the cutoff, preferable to passing weak matches to the generator. *Metadata filtering* restricts search to chunks tagged by source, date, topic, or chunk type when scope is already known from the query or an upstream route.

5.3. Parent-Child Expansion

Parent-child expansion decouples search granularity from the unit passed to the generator. The common approach is two-step: similarity search runs over child embeddings, then each match is mapped through stored `parent_id` metadata to a larger parent chunk in a document store; retrieval finds the precise span, but generation sees the surrounding section. Implementations vary in the expansion rule. Some always return parents after child retrieval. While the standard implementation stores embeddings only for child chunks, some hierarchical retrieval systems additionally maintain parent embeddings and apply a secondary ranker to choose parent or child output based on query specificity. A lightweight alternative uses retrieval confidence, keeping child chunks when top-k scores are strong and expanding to parents when scores signal a partial or context-dependent match. The component plugs into any retrieval path that already indexes hierarchical chunks (Table 2).

5.4. Hybrid Search

Dense retrieval captures paraphrases and conceptual similarity but misses exact keywords, acronyms, and rare terms; BM25 captures those lexical signals but not meaning. *Hybrid retrieval* runs both in parallel and merges the ranked lists. Weighted score fusion is one option; *reciprocal rank fusion* (RRF) [25] is a rank-agnostic alternative that needs no score normalization:

$$\mathrm{RRF}(d) = \sum_{L \in \mathcal{L}} \frac{1}{k + \mathrm{rank}_L(d)} \tag{3}$$

where d is a retrieved document (or chunk), $\mathcal{L}$ is the set of ranked retrieval lists being fused (e.g., dense retrieval and BM25), $\mathrm{rank}_L(d)$ is the rank of d in list L, and k is a positive smoothing constant that reduces the influence of very high-ranked documents.

Documents appearing in multiple lists rank higher. The same fusion step applies when pre-retrieval query expansion produces several search variants: each variant yields a ranked list, and RRF merges them into one candidate pool without prescribing how those queries were generated.

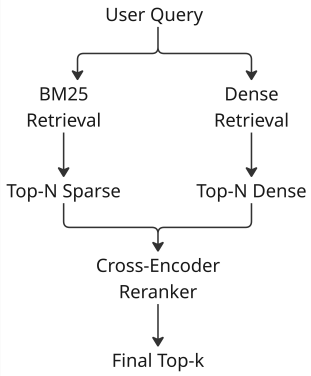


Figure 3. *Hybrid retrieval combines sparse keyword matching (BM25) with dense embedding search, fusing candidates before final ranking.*

5.5. Reranking

Bi-encoder retrieval encodes queries and documents independently, enabling efficient approximate nearest-neighbor search but limiting token-level interactions. *Cross-encoder reranking* [11] scores query–document pairs jointly, improving precision at the cost of latency. The standard pattern retrieves a larger candidate set ($k' \gg k$), reranks with a cross-encoder, and returns the top-k. Reranking composes with hybrid search or RRF output: fusion produces the candidate pool, reranking orders it. Model choice trades quality against speed i.e. full cross-encoders for accuracy, lighter models for low-latency filtering.

Table 5. *Mid-retrieval methods, targets, and latency cost.*

Method	Primary Target	Adds Latency?
Cosine top-k	Baseline relevance	No
MMR	Retrieval noise (redundancy)	Low
Retrieval filtering	Retrieval noise; wrong scope	No
Parent-child expansion	Context fragmentation	Low
Hybrid (BM25 + dense)	Query mismatch (terms vs. paraphrase)	Medium
Cross-encoder reranking	Context precision	High

6. Post-Retrieval Optimization

Retrieved chunks are rarely ready for direct use by a language model. Post-retrieval optimization prepares the ranked result set by filtering noise, tightening passages, and assembling context before generation or before control returns to an orchestrating agent. The components below are independent; each plugs in after retrieval and reranking without requiring a multistep process at the post-retrieval stage.

6.1. Context Preparation

Reranking improves ordering but does not remove every weak passage or irrelevant sentence within a relevant document. Three post-retrieval components address what remains.

Retrieval grading [13] applies a lightweight relevance judge to each retrieved chunk, labeling it relevant, irrelevant, or ambiguous before it enters the prompt. Chunks below confidence are dropped or downweighted, targeting semantic false positives that survive similarity search and reranking. This extracts the evaluator from corrective RAG without its web-search fallback or query-rewrite loop.

Knowledge refinement [13] operates on chunks that pass grading: each passage is split into sentences, unrelated sentences are removed, and the remainder is reassembled into a tighter unit. Where grading removes whole chunks, refinement strips filler within chunks that are partially on-topic.

Context compression [19] reduces the amount of text passed to the generator by retaining only the information required to answer the current query. Unlike knowledge refinement, which removes irrelevant content while preserving the remaining passage, compression may further shorten or summarize relevant content to fit the available context window. Implementations include extractive selection, embedding-based span selection, and abstractive summarization.

Retrieval grading filters out irrelevant chunks, knowledge refinement removes irrelevant content within retained chunks, and context compression reduces the remaining context to its query-relevant portions. These components are complementary and can be applied sequentially.

Table 6. *Post-retrieval context preparation components.*

Component	Primary Target	Adds Latency?
Retrieval grading	Retrieval noise (false-positive chunks)	Low–Medium
Knowledge refinement	Retrieval noise (within-chunk filler)	Medium
Context compression	Retrieval noise; context budget	Medium

6.2. Document Assembly

Production RAG concatenates the prepared retrieved set into a single prompt and generates one answer (*stuff* in LangChain terminology [19]). Larger context windows and stronger models have made this the default; earlier multi-pass strategies (refine, map-reduce, map-rerank) addressed context limits that are rarely a concern today. In agent workflows, the same pattern applies: retrieved context is formatted as structured input for the orchestrating model's next step. For multimodal corpora, assembly must preserve chunk type (SQL results as structured text, image references alongside narrative) so the generator receives each modality in a usable form.

7. Conclusion and Future Work

Retrieval quality is the foundation of effective RAG systems, yet most implementations treat it as an afterthought. We presented a three-stage framework: preparing documents and queries, controlling search and ranking, and refining context before generation, that maps modular components to three recurring failure modes: context fragmentation, query mismatch, and retrieval noise. Effective systems compose techniques across these stages rather than relying on any single fix. Whether retrieval runs once in a standalone pipeline or repeatedly as an agent tool [2], each invocation passes through the same stages; orches-

tration decides when to retrieve, while the framework specifies what each call should return.

Several directions remain open. Systematic evaluation of component combinations rather than isolated techniques would quantify how pre-, mid-, and post-retrieval optimizations compose under fixed embedding and generation settings. Text-centric benchmarks do not fully capture multimodal chunking and typed retrieval; corpora with tables, figures, and layout structure need dedicated evaluation. Future work should also study adaptive pre-retrieval: selecting chunking strategy based on document type: plain text versus layout-rich or multimodal sources and routing query enhancement through agentic knowledge, where an orchestrator chooses expansion, decomposition, or rewriting based on query intent and conversation context rather than applying a fixed transformation pipeline. As context windows grow, the bottleneck shifts from fitting retrieved content to ensuring the right content reaches the model; engineering retrieval at the stage where each problem originates remains the distinguishing factor between demonstration and production systems.

References

[1] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020, [Online]. Available: https://arxiv.org/abs/2005.11401

[2] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG," *arXiv preprint arXiv:2501.09136*, 2025, [Online]. Available: https://arxiv.org/abs/2501.09136

[3] LangChain AI, "LangGraph: Build resilient language agents as graphs." [Online]. Available: https://github.com/langchain-ai/langgraph

[4] Z. Wang *et al.*, "Document Segmentation Matters for Retrieval-Augmented Generation," in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 8063–8075. doi: 10.18653/v1/2025.findings-acl.422.

[5] Y. Wang, H. Zhang, L. Pang, B. Guo, H. Zheng, and Z. Zheng, "MaFeRw: Query Rewriting with Multi-Aspect Feedbacks for Retrieval-Augmented Large Language Models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, pp. 25434–25442. doi: 10.1609/aaai.v39i24.34732.

[6] Z. Wu *et al.*, "CONQRR: Conversational Query Rewriting for Retrieval with Reinforcement Learning," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates, 2022, pp. 10000–10014. doi: 10.18653/v1/2022.emnlp-main.679.

[7] I. S. Singh *et al.*, "ChunkRAG: A Novel LLM-Chunk Filtering Method for RAG Systems," *arXiv preprint arXiv:2410.19572*, 2024, [Online]. Available: https://arxiv.org/abs/2410.19572

[8] V. Karpukhin *et al.*, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781. [Online]. Available: https://arxiv.org/abs/2004.04906

[9] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/1500000019.

[10] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–48. [Online]. Available: https://arxiv.org/abs/2004.12832

[11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *arXiv preprint arXiv:1908.10084*, 2019, [Online]. Available: https://arxiv.org/abs/1908.10084

[12] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[13] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective Retrieval Augmented Generation," *arXiv preprint arXiv:2401.15884*, 2024, [Online]. Available: https://arxiv.org/abs/2401.15884

[14] M. Arora, "Research Papers Collection from arXiv." [Online]. Available: https://huggingface.co/datasets/mahimaarora025/research_papers

[15] Qdrant Team, "Qdrant: Vector Database for the Next Generation of AI Applications." [Online]. Available: https://qdrant.tech/

[16] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," *arXiv preprint arXiv:2309.15217*, 2023, [Online]. Available: https://arxiv.org/abs/2309.15217

[17] M. Arora and A. Jha, "Retrieval Playground: A Python Toolkit for RAG Experimentation and Evaluation." [Online]. Available: https://github.com/mahimaarora/retrieval-playground

[18] N. Livathinos *et al.*, "Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion," *arXiv preprint arXiv:2501.17887*, 2025, [Online]. Available: https://arxiv.org/abs/2501.17887

[19] LangChain AI, "LangChain: Building context-aware reasoning applications." [Online]. Available: https://github.com/langchain-ai/langchain

[20] Anthropic, "Contextual Retrieval in AI Systems." [Online]. Available: https://www.anthropic.com/engineering/contextual-retrieval

[21] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada, 2023, pp. 1762–1777. doi: 10.18653/v1/2023.acl-long.99.

[22] H. S. Zheng *et al.*, "Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06117

[23] Aurelio Labs, "Semantic Router: Intelligent Query Routing for LLM Applications." [Online]. Available: https://github.com/aurelio-labs/semantic-router

[24] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336. doi: 10.1145/290941.291025.

[25] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759. doi: 10.1145/1571941.1572114.

Open Access | https://doi.org/10.25080/tujc9326

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

From Hello World to Hello LLM: A Python Developer's Survival Guide

Jasmine Omeke[1], and **Audrey Webb**[2]

[1]Airbnb, [2]Autodesk

Abstract

AI tooling is moving fast, but many Python developers are unsure where to start, or how today's patterns fit into systems they already know how to build. This talk gives a practical, hands-on overview of modern large language model (LLM) development patterns in Python, focused on what you need to know to go from zero to hero. We walk through real-world coding examples from a social-good hackathon, broken into parts that show the core building blocks of modern LLM applications and explain when each pattern makes sense. These examples are designed to require no prior machine learning experience. Attendees will leave understanding how LLM systems work, what problems they solve well, and how to maintain and observe what they build. We cover the modern LLM stack in Python. This includes common patterns such as prompts, function calling, retrieval-augmented generation (RAG), and simple agents. Additionally, we will show how to choose between a script, an agent, and a service. We also cover how to get something working quickly without sacrificing reliability or safety. We share practical guardrails for handling errors and protecting data. And we show how to stand up common workflows, from LLM-powered scripts to lightweight agents and Model Context Protocol-style (MCP-style) services. Attendees will leave with a clear map of the LLM landscape, working Python patterns they can reuse immediately, and the confidence to start building LLM features without a machine learning background.

Keywords LLM, Python, RAG, AI tools, education

Published Jul 13, 2026

Correspondence to
Jasmine Omeke
j.omeke.consults@gmail.com

Open Access

Copyright © 2026 Omeke & Webb. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. What We Learned Building AI Tools with Python

Most Python developers already know how to build real systems, APIs, scripts, services, and pipelines. During a social impact AI hackathon, we applied those same engineering skills to rapidly build AI-powered tools that helped educational institutions become more comfortable using AI to improve student success outcomes.

Like many developers entering the AI space, we had to quickly navigate a growing ecosystem of frameworks, APIs, vector databases, and agent tooling while still shipping working software under tight time constraints.

Here we share the lessons learned, tooling decisions, and implementation patterns that helped us move from experimentation to working applications.

First, let's break the component of AI usage down into familiar engineering concepts:

- validation
- retrieval
- orchestration
- structuring inputs
- structured outputs

We'll walk through:

- the core concepts behind modern AI applications
- common Python libraries and tooling
- reusable code snippets and implementation patterns
- understanding how and when to use these patterns in real systems

2. The Shift: AI as a System Component Instead of Our Base Tool

The biggest shift here is the greater accessibility for anyone to build. You no longer need to train models yourself as modern large language model (LLM) APIs, embedding models, and vector databases are available with minimal setup. We view AI as not replacing your application stack, but another component to be added to existing systems that are useful for ambiguity, language, reasoning, and search. Think of an LLM like a non-deterministic but powerful external API that needs structure, validation, retries, and observability.

AI collapses the gap between expertise and execution. In our hackathon, we paired technologists (who bring architecture, LLM knowledge, data systems, and engineering experience) with education experts (who bring deep context, student needs, institutional knowledge, and on-the-ground experience). Together, through a modular LLM stack (prompts, tools, retrieval, embeddings, vector databases, and guardrails), we delivered working solutions in three days.

Figure 1. *Collaboration during the hackathon.*

3. The Modern AI Stack in Python

Most AI-powered Python applications are assembled from a small set of reusable components:

LLM APIs: ollama-python, litellm, transformers, vllm, llama-cpp-python

Embedding models: sentence-transformers, transformers, InstructorEmbedding, FlagEmbedding (BGE models)

Vector databases: faiss, chromadb, qdrant-client, weaviate-client, milvus

Prompt orchestration: LangGraph, LangChain, Haystack, PydanticAI, CrewAI

Structured output validation: Pydantic, Instructor, Guardrails AI, jsonschema

Tool calling: PydanticAI, LangGraph, LangChain, smolagents, CrewAI

The key takeaway: You are still writing Python. LLMs are another system dependency your application coordinates and manages. We'll show how these pieces connect in practice using code snippets and examples from our social impact hackathon project.

4. Key Terms

For our purposes, during the hackathon, we used the following concepts:

- **LLM:** Generates text from prompts
- **Embeddings:** Turn words or sentences into numbers that capture meaning. Things with similar meanings end up close together ("dog" ≈ "puppy", "pizza" ≈ "burger"). That's how AI can find related ideas, not just exact word matches.
- **RAG:** Retrieve context before generating answers
- **Agent:** A loop where the model decides which tools to call, in what order, based on intermediate results

These concepts cover most real-world AI applications.

5. Requirements Gathering Before Hacking

Before we began using AI, we spent time understanding what our stakeholders considered a good result.

Requirements gathering was an important step to help us understand the goals of our educational institutions. This involved journey mapping. Before building, we worked with educational staff to journey-map their workflows, questions, and pain points. That process helped define:

- What kinds of questions the system should answer
- What data it was allowed to access
- What topics were out of scope
- How responses should be phrased

Once we documented the needs and criteria our stakeholders (educational institutions and staff) had for success, we could begin development work. With the requirements distilled from our clients with this method of planning, we were able to identify LLM patterns for our respective institution's use case. With that in mind, we'll explore patterns of LLM usage with hands-on examples.

6. Core Pattern 1: System Prompts + Data Dictionaries

Jasmine's team used system prompts and a data dictionary to ground the LLM in institutional data and terminology. This created a control layer between user questions, language, and underlying datasets.

Key components:

- **System prompt** defines rules, scope, and response behavior
- **Data dictionary** defines approved institutional terms and metrics
- Together, they constrain interpretation and reduce hallucinations
- Ensures data is used at the correct grain and in the correct context

Jasmine's hackathon project involved building a chatbot to help higher education staff understand a pivotal student success dataset. The university had an institutional research department that was overwhelmed with data requests that took days to answer. AI was an opportunity to make this self-service, but institutions were concerned about data privacy and security. To address that, the chatbot was scoped to return **aggregate data only**, with no individual student-level information exposed.

Prompting became the primary way we defined system behavior, privacy, and guardrails. We created a data dictionary of approved educational terms and concepts to guide the LLM toward domain-specific language and reduce hallucinations. This was important because despite datasets having overlapping subject matter, there was nuance in how data could be used. Some tables contained individual student data (PII) and could not be used at all for a chatbot serving data to internal stakeholders.

This dictionary acted as scaffolding for the LLM:

- Defining the language of the domain
- Clarifying what concepts existed
- Constraining how questions should be interpreted
- Helping the model distinguish valid requests from unsupported ones

Rather than relying on the model's general knowledge, we used structured context to shape how it reasoned about the problem space.

```
SYSTEM_PROMPT = """
You are an educational analytics assistant.

### brief data dictionary ###
Use only the approved terms below:
- Attendance Rate
- Engagement Score
- Intervention Tier
- Student Success Trend

Rules:
- Only return aggregate data
- No individual student data (PII)
- Say "out of scope" if not supported
- Don't join the Attendance Rate data to an external, aggregate student metric table;
  that data contains information at an opposing grain.
"""

response = llm.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": "Which tiers improved attendance?"}
    ]
)
```

Key takeaway: System prompts and data dictionaries turn language into a controlled interface for data.

7. Core Pattern 2: Context Retrieval for Grounding (Database-Backed, RAG-Style)

We extended the data dictionary into a live retrieval layer connected to institutional data systems.

Why it matters:

- Aligns with existing infrastructure

- Bridges prompts with real data systems
- Reduces hallucinations through grounding
- Guides the chatbot toward approved terminology
- Retrieves relevant institutional definitions and resources at query time
- Constrains answers to known educational concepts

Jasmine's hackathon project relied on retrieval to ground responses in a trusted educational context, always at the aggregate level described in Pattern 1. In production, this kind of grounding is often implemented with embeddings and a vector store; under hackathon time constraints, we instead built a simplified, database-backed version of the same idea: a keyword check followed by a direct SQL lookup. Both approaches serve the same purpose, retrieving relevant, approved context before the LLM generates an answer, and both helped guide the chatbot toward approved terminology and constrain it to known concepts. We call this out explicitly here because it's a lighter-weight stand-in for full embedding-based RAG, not a complete RAG pipeline.

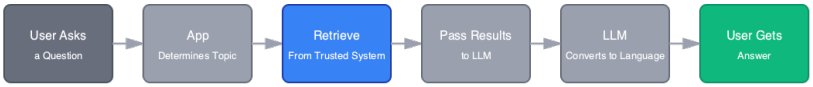


Figure 2. *RAG pipeline flow diagram.*

```
# Data sources (swappable, secured)
DATA_SOURCES = {
    "sql": "...Azure SQL / AAD auth..."
}

# Simplified retrieval layer: a keyword check + direct SQL lookup.
# In production, this step would typically embed the query and
# search a vector store; here we use a lighter database-backed
# lookup to illustrate the same grounding principle.
def fetch_metrics(query):
    if "retention" in query:
        return sql("""
            SELECT metric, value, definition
            FROM student_metrics
            WHERE metric = 'retention'
        """)
    return None

# LLM layer (swappable)
def call_llm(prompt, model="gpt-4o"):
    return llm.chat(model=model, messages=[
        {"role": "system", "content": "Use only provided context."},
        {"role": "user", "content": prompt}
    ])

# App flow
def answer(query, model="gpt-4o"):
    context = fetch_metrics(query)
    prompt = f"{query}\n\nContext:\n{context}"
    response = call_llm(prompt, model=model)
    log = {"query": query, "model": model}
    return response, log
```

Key takeaway: Database-backed context retrieval, a simplified stand-in for full embedding-based RAG, connects the LLM to live institutional data systems and approved terminology.

8. Core Pattern 3: Function Calling & Controlled Output

Not every problem requires an LLM. For numeric forecasts, risk scoring, and trend analysis, the model should not invent results. It should request them from backend systems that can compute them deterministically.

Audrey's approach: She used requirements gathering and journey mapping to understand the goals of her institutions. That research helped her design a predictive analytics dashboard that separated language understanding from analysis workflows, so the LLM acted as an interface layer rather than generating predictions directly. This dashboard was built for authorized advisor use, a different audience and access scope than Jasmine's aggregate-only chatbot, so the two systems apply different levels of data granularity appropriate to their users.

In practice, the chatbot received structured requests, passed them to backend analytics, and then explained the results back to users in clear language. The dashboard provided:

- Retention prediction
- Risk scores per student
- Probability of dropping out
- Attendance trend forecasts
- Credential type prediction
- Gateway course success prediction
- GPA prediction

Key principles:

1. **Separate Understanding from Computation** - The LLM interprets the user's intent, but the predictive model and analytics pipeline do the actual math. This keeps forecasting logic outside the model and makes the system easier to test and trust.
2. **Let the LLM Request Structured Data** - Instead of free-form answers, the LLM calls a function like `get_retention_risk(school_id)`. That request is precise, typed, and limited to the data the backend is designed to return.
3. **Return Predictable Analytics** - The Python system aggregates predictions and returns structured outputs such as risk scores, dropout probability, and attendance trends. The LLM then explains those results in plain language.
4. **Keep It Reproducible and Auditable** - Because predictions are computed outside the model, the same input produces the same output. That makes the system easier to audit, version, and debug, especially in a hackathon setting where you need reliable behavior fast.

```python
from openai import OpenAI

client = OpenAI()

# Predictive model wrapped as an approved function.
# This function returns a school-level average, so the
# question posed to the LLM should match that grain.
def get_retention_risk(school_id):
    students = feature_store.get_students(school_id)
    risk_scores = retention_model.predict_proba(students)
    return {
        "school_id": school_id,
        "avg_retention_risk": float(risk_scores.mean())
    }

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_retention_risk",
            "description": "Retrieve student retention risk metrics",
            "parameters": {
                "type": "object",
                "properties": {
                    "school_id": {"type": "string"}
                },
                "required": ["school_id"]
            }
        }
    }
]

user_question = (
    "What is the average retention risk for School_123 this semester?"
)

# Step 1: LLM determines what analytics are needed
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {
            "role": "system",
            "content": (
                "Never generate predictions yourself. "
                "Use available analytics functions and explain results."
            )
        },
        {"role": "user", "content": user_question}
    ],
    tools=tools
)

# Step 2: Backend executes predictive model
analytics_result = get_retention_risk(
    school_id="School_123"
)

# Step 3: LLM explains results
final_response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": user_question},
        {
            "role": "tool",
            "tool_call_id": response.choices[0].message.tool_calls[0].id,
            "content": str(analytics_result)
        }
    ]
)

print(final_response.choices[0].message.content)
```

This pattern ensured predictions were:

- Reproducible
- Auditable
- Computed separately from the LLM
- Token usage was optimized

Key takeaway: Separate language understanding from computation. Let the LLM be an orchestrator, not a calculator, and keep the question you send it aligned with the grain of data the function actually returns.

9. Core Pattern 4: Simple Agent Loops

While Core Pattern 3 focused on a single function call, many institutional questions in Audrey's project required chaining multiple tools together, and the right tool to call next often depended on what the previous tool returned. This is what separates an agent from a script: instead of Python deciding the order of operations up front, the LLM sees each tool result and decides what to do next, including when it has enough information to stop and answer.

A simple agent loop looks like this:

1. Send the conversation, including any tool results so far, to the LLM
2. If the LLM responds with a tool call, execute that tool in Python and append the result to the conversation
3. Repeat until the LLM responds with a final answer instead of a tool call

Consistent with the advisor-facing scope described in Pattern 3, this agent works with cohort-level and segment-level data rather than naming individual students.

```
tools = [
    get_student_cohort_tool,
    get_retention_risk_tool,
    compare_to_previous_term_tool
]

def execute_tool(name, arguments):
    if name == "get_student_cohort":
        return get_student_cohort(**arguments)
    if name == "get_retention_risk":
        return get_retention_risk(**arguments)
    if name == "compare_to_previous_term":
        return compare_to_previous_term(**arguments)
    raise ValueError(f"Unknown tool: {name}")

messages = [
    {
        "role": "system",
        "content": (
            "You are a retention analytics assistant for university advisors. "
            "Use the available tools to gather whatever data you need before "
            "answering. Report findings at the cohort or segment level only, "
            "never by naming individual students."
        )
    },
    {
        "role": "user",
        "content": (
            "Which first-year student segments carry the most retention risk, "
            "and is risk improving or declining relative to last term?"
        )
    }
]

# Agent loop: the LLM decides which tool to call next, based on what
# earlier tool calls returned, until it decides it has enough to answer.
while True:
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        tools=tools
    )
    choice = response.choices[0].message

    if not choice.tool_calls:
        print(choice.content)
        break

    messages.append(choice)

    for tool_call in choice.tool_calls:
        result = execute_tool(
            tool_call.function.name,
            json.loads(tool_call.function.arguments)
        )
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": str(result)
        })
```

In a run like this, the model might first call `get_student_cohort` to pull the first-year class, then call `get_retention_risk` on that cohort, then decide on its own whether `compare_to_previous_term` is actually needed to answer the question, or call it again with a narrower segment if the first risk scores looked uneven across groups. None of that branching is written in Python. It comes from the LLM reading each tool result and choosing the next step.

Key takeaway: In a true agent loop, the model decides which tool to call next based on intermediate results, and the loop only ends when the model itself decides it has enough information to answer.

10. LLM Guardrails We Deployed

The goal is not to rely solely on the model, but to build systems that remain reliable even when it makes mistakes.

Because we had two different use cases (Audrey's predictive analytics tool and Jasmine's institutional data chatbot), we deployed different guardrails tailored to each system's risks and requirements.

10.1. Audrey's Project: Predictive Analytics Tool

- **Input Sanitization:** Validate and clean user inputs before they reach the model
- **Sensitive Data Handling:** Never send PII, secrets, or sensitive data to an LLM without explicit review
- **Structured Output Validation:** Never use raw LLM output without validation; enforce schemas before downstream systems
- **Prompt Logging & Observability:** Log prompts, responses, latency, token counts, and tool calls to detect degradation early
- **Retries & Timeouts:** Set explicit timeouts on LLM API calls; use retries with exponential backoff for transient failures

Problems to be Solved	Guardrail
Malicious input	Input Sanitization
Data exposure risk	Sensitive Data Handling
Invalid outputs break systems	Structured Output Validation
Silent failures	Prompt Logging & Observability
API failures	Retries & Timeouts

10.2. Jasmine's Project: Institutional Data Chatbot

- **Hallucination Safeguards:** Ground factual answers in verified data; require citations when needed; allow the model to say "I don't know"
- **Detecting Off-Topic Conversations:** Identify and redirect conversations that drift too far from intended scope
- **Rejecting Unsupported Requests / Escalating Sensitive Queries:** Redirect unsupported requests to the right workflow; escalate sensitive queries that need policy checks or special handling
- **Dynamic Retrieval Strategy Selection:** Dynamically select retrieval strategies based on topic, intent, and confidence in the answer

Problems to be Solved	Guardrail
LLM invents facts	Hallucination Safeguards
Off-topic requests	Detecting Off-Topic Conversations
Out-of-scope questions	Rejecting Unsupported Requests / Escalating Sensitive Queries
Wrong retrieval strategy	Dynamic Retrieval Strategy Selection

11. Building the Right Complexity for the Problem

Not every problem needs an agent. Use:

- A single prompt for simple classification or extraction
- A pipeline for fixed workflows
- An agent only when adaptive multi-step reasoning is required
- Function calling when computation must be deterministic
- RAG when you need to ground answers in external knowledge

Start with the simplest working approach before adding complexity. The path is straightforward:

1. Start with a simple LLM API call
2. Add structured outputs
3. Add tool calling for focused, useful actions
4. Add retrieval-based grounding when external knowledge matters
5. Introduce agent loops only when the task genuinely requires the model to decide what to do next

12. Conclusion

You do not need deep ML expertise to build useful AI systems. You need:

- Strong engineering fundamentals
- Clear abstractions
- Careful system design that reduces cost, latency, and failure modes
- Start small, observe everything, and expand with purpose

The core lesson from our hackathon: The path forward is straightforward. Build on solid engineering basics, add capability in layers, and use the simplest abstraction that solves the problem.

In three days, we went from exploring requirements to shipping working tools because we treated AI like any other system dependency with respect for its probabilistic nature, careful validation, and thoughtful orchestration. Python developers already know how to do this. AI is just another component in your stack.

13. GenAI Usage Disclosure

Generative AI (Claude, Anthropic) was used as an editing aid for this paper, primarily to help tighten sentence structure and shorten overly long sentences in the abstract, and to check grammar and flow throughout. All technical content, code examples, patterns, and conclusions reflect the authors' own work and hackathon experience; no generative AI was used to produce the underlying ideas, code, or results described in this paper.

Open Access | https://doi.org/10.25080/fygp5764

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

PanelBox: A Comprehensive Python Library for Panel Data Econometrics

Gustavo Coelho Haase[1], **Paulo Henrique Dourado**[1], and **Osvaldo Candido**[2]

[1]Banco do Brasil S.A., Brasília, Brazil, [2]Graduate School in Economics, Universidade Católica de Brasília, Brasília, Brazil

Abstract

Panel data econometrics is ubiquitous in economic and social-science research, yet Python lacks a comprehensive library for advanced panel analysis: researchers have to fall back on proprietary Stata or on R, breaking otherwise Python-native, reproducible workflows. We introduce PanelBox, the first all-in-one Python library for panel data econometrics, implementing more than 70 models across 11 families — static linear models, dynamic GMM estimators (Arellano–Bond, Blundell–Bond with the Windmeijer correction), discrete choice, count data, spatial econometrics, stochastic frontier analysis, quantile regression, panel VAR/VECM, censored and selection models, and the first comprehensive Python implementation of panel unit-root and cointegration tests. The library adds 50+ diagnostic tests, 11 standard-error estimators, four bootstrap methods, an integrated HTML/LaTeX/Markdown reporting system with interactive Plotly visualizations, and 103 bundled datasets, all on top of the scientific Python stack (NumPy, pandas, SciPy, statsmodels). We validate PanelBox against R's `plm`, reproducing its difference-GMM, fixed-effects and panel unit-root results to three or more decimals on the Arellano–Bond data, and illustrate it with a canonical dynamic labor-demand application. With 3,900+ tests and an MIT license, PanelBox lets researchers run sophisticated panel analysis entirely in Python.

Keywords panel data, econometrics, dynamic panel GMM, spatial econometrics, reproducible research

1. Introduction

PanelBox did not begin as a general-purpose econometrics library. We work in model validation for credit risk, where econometric models estimated on panel data must be independently re-estimated, stress-tested, and documented before they are allowed into production. Our production systems run on Python; the reference implementations of the methods we needed did not. Every validation cycle meant exporting data to Stata or R, re-importing results, and reconciling discrepancies by hand — slow, error-prone, and impossible to automate end to end. What started as an internal toolkit for validating panel models in pure Python grew, model family by model family, into PanelBox.

The data at the center of a production econometrics workflow like ours are *panel data* — repeated observations on the same entities over time, such as firms observed annually or countries tracked across decades. Panels are the empirical backbone of modern economics and the social sciences [1], [2], and for a simple reason: every firm or country has stable characteristics that no dataset records — management quality, institutions, culture. A cross-sectional regression silently attributes the effect of these unobserved traits to the variables it does observe, biasing its conclusions. Because a panel follows each entity over time, it can compare each entity with itself, removing the influence of anything constant — the

Published Jul 13, 2026

Correspondence to
Gustavo Coelho Haase
gustavohaase@gmail.com

Open Access

Copyright © 2026 Haase *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

unobserved heterogeneity that panel methods are built to neutralize. Combined with the time dimension, this lets researchers estimate genuinely dynamic relationships, from labor-market adjustment [3] to production functions [4] and cross-country growth [5].

The gap we encountered, it turned out, is far wider than our use case: the tooling for these methods is unevenly distributed across computing environments. Stata's `xtabond2` command [6] is the de facto standard for dynamic panel GMM, and R offers the mature `plm` package [7] for linear panels and specialized packages such as `pdynmc` [8] for dynamic ones. Python — now dominant in data science and increasingly in quantitative social science thanks to NumPy [9], pandas [10], and SciPy [11] — has lagged conspicuously behind in panel econometrics.

Existing Python libraries cover only fragments of the workflow. `linearmodels` [12] provides static fixed- and random-effects models, the workhorse estimators that remove, or explicitly model, the stable entity traits discussed above. It also offers some support for instrumental variables (proxy regressors used in place of a variable that is correlated with the error term). It has no dynamic GMM, however, the estimator required when the outcome depends on its own past, which we introduce below. `pyfixest` excels at high-dimensional fixed effects but offers neither GMM nor dynamic panels; and `statsmodels` [13] has time-series tools but no panel-specific GMM. Critically, none of them implement panel unit-root or cointegration tests, which are routine in Stata (`xtunitroot`, `xtcointtest`) and R (`plm::purtest`). The consequence is a broken workflow: analysts either limit themselves to basic models in Python, or export their data to Stata or R, forfeiting reproducibility and integration with the rest of the Python stack.

PanelBox closes this gap. Version 1.0 implements more than 70 models across 11 families in a single package, validated numerically against established software. Beyond breadth, it contributes three things that are new to Python: a validated dynamic-GMM implementation with the Windmeijer finite-sample correction; the first comprehensive panel unit-root and cointegration testing suite; and an intelligent algorithm for unbalanced panels that retains far more data than naive complete-case approaches. The library is open source (MIT), integrates natively with pandas, and ships with interactive visualization, publication-ready reporting, and 103 datasets for immediate experimentation. The paper follows the workflow a researcher actually traverses — specify and estimate a model, probe its assumptions, and report the results — with the depth reserved for the three novel contributions; Breadth: eleven model families summarizes the full catalogue, and Using and extending PanelBox describes how the library is used and extended in practice.

2. Library design: one workflow from data to report

PanelBox is designed to model the standard estimation workflow of applied econometrics. The user starts from a pandas `DataFrame` in long format; a common data layer wraps it together with entity and time identifiers, and everything downstream consumes that layer. Estimators accept R-style formulas via `patsy` and return a result object that follows the conventional `statsmodels` API — `summary()`, `conf_int()`, `to_latex()`, `to_html()` — so that the same result flows into the shared inference layer (11 standard-error types, four bootstrap methods), the diagnostics layer (50+ tests), and the reporting layer that renders self-contained HTML, LaTeX, or Markdown. Figure 1 summarizes this pipeline.

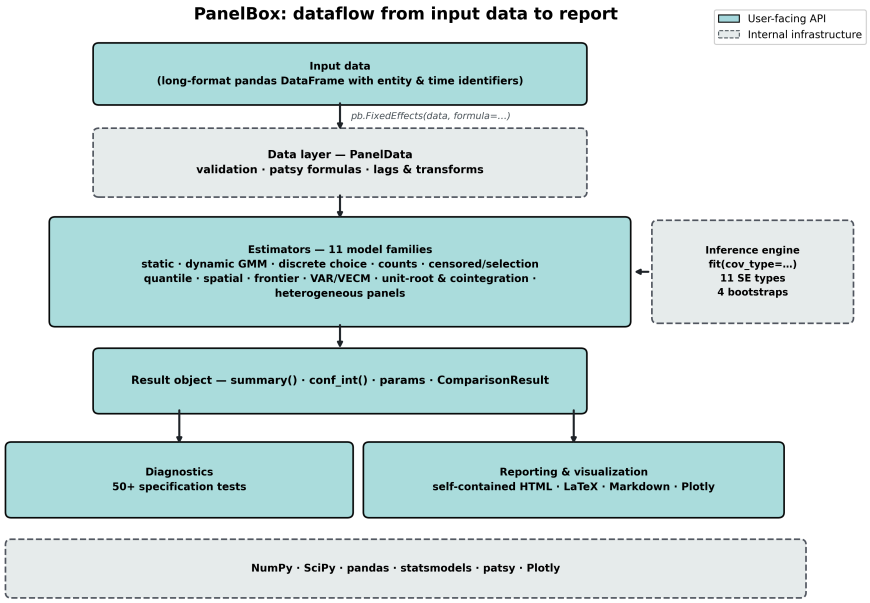


Figure 1. *PanelBox as a dataflow. Input data, represented as a long-format pandas `DataFrame`, enter the shared data layer, flow through the 11 estimator families — with the inference engine supplying standard errors and bootstraps at `fit()` time — and the resulting objects feed the diagnostics and reporting layers. Solid colored boxes are the user-facing API; dashed gray boxes are internal infrastructure.*

Because the API follows conventions the community already knows — `statsmodels`-style formulas and result objects, with an sklearn-like construct-then-fit pattern — a minimal estimation holds no surprises:

```
import panelbox as pb

data = pb.load_dataset("abdata")
fe = pb.FixedEffects(
    "n ~ w + k", data,
    entity_col="id", time_col="year",
)
result = fe.fit(cov_type="clustered")
print(result.summary())
```

The workflow's entry point is the static linear family, which implements pooled OLS, fixed effects (within), random effects (feasible GLS), the between estimator, and first differences for the model

$$y_{it} = \boldsymbol{x}_{it}'\boldsymbol{\beta} + \alpha_i + \lambda_t + \varepsilon_{it}, \qquad i = 1, ..., N,\ t = 1, ..., T, \tag{1}$$

where α_i are entity effects — absorbing everything specific to entity i that is constant over time, such as a firm's management quality — and λ_t are time effects, absorbing shocks common to all entities in period t, such as a recession year. Fixed effects are computed by within-transformation using sparse matrix operations and pandas `groupby`, so the estimator scales to panels with thousands of entities and tens of thousands of observations (the benchmarks in Figure 4 go up to $N = 2,500$) and supports unbalanced panels with entity-specific T_i.

3. Estimating dynamics: panel GMM

Many economic outcomes depend on their own past. Employment adjusts slowly because hiring and firing are costly; this year's GDP is anchored to last year's. Capturing such persistence means adding the lagged outcome $y_{i,t-1}$ as a regressor:

$$y_{it} = \alpha\, y_{i,t-1} + \boldsymbol{x}_{it}'\boldsymbol{\beta} + \eta_i + \varepsilon_{it}. \tag{2}$$

That innocent-looking addition breaks both estimators of the previous section. The lagged outcome is itself driven by the entity effect η_i, so it is correlated with the composite error: pooled OLS is biased upward, and the within estimator downward (the Nickell bias, S. Nickell [14]) — the true dynamics lie somewhere in between, and neither estimator can find them. The resolution, proposed by M. Arellano and S. Bond [3] and R. Blundell and S. Bond [4], is to transform away η_i and instrument the problematic regressor with its own older lags: old enough to be uncorrelated with today's shock, recent enough to remain informative. This machinery — dynamic panel GMM — is PanelBox's flagship capability, and previously had no validated Python implementation.

Difference GMM [3] first-differences (2) to remove η_i and exploits the moment conditions $\mathrm{E}\big[y_{i,t-s}\,\Delta\varepsilon_{it}\big] = 0$ for $s \geq 2$, so lagged levels instrument the differenced regressors. **System GMM** [4] augments the differenced equations with the level equations, instrumenting them with lagged differences; this restores efficiency and reduces finite-sample bias when α is close to unity and lagged levels are weak instruments. PanelBox implements both, in one-step and two-step variants — one-step GMM weights the moment conditions with a fixed matrix, while two-step re-estimates the weights from the first step's residuals to gain asymptotic efficiency — and generates the instrument sets automatically:

```
gmm = pb.SystemGMM(
    data, dep_var="n", lags=1,
    id_var="id", time_var="year",
    exog_vars=["w", "k"],
    collapse=True, two_step=True,
    time_dummies=False,
)
res = gmm.fit()
print(f"Hansen J p-value: {res.hansen_j.pvalue:.3f}")
print(f"AR(2) p-value:    {res.ar2_test.pvalue:.3f}")
```

Windmeijer correction. Two-step GMM is asymptotically efficient but its standard errors are severely biased downward in finite samples — by 30–80% in typical applications. F. Windmeijer [15] derives a finite-sample correction that accounts for the estimated weight matrix; PanelBox applies it by default, following the algorithm in D. Roodman [6].

Instrument proliferation. Because the instrument count grows quadratically in T, it can exceed N and overfit the endogenous regressors [16]. PanelBox warns automatically when the instrument count approaches N and implements the `collapse` option, which combines the per-period instruments for each variable into a single column while preserving the moment conditions.

3.1. *Intelligent handling of unbalanced panels*

Missing observations are a recurring practical obstacle in dynamic GMM: because each instrument depends on a specific lag (e.g. $y_{i,t-2}$ instruments Δy_{it}), a single gap can invalidate instruments. The common practical response is blunt: many practitioners drop every entity that is not observed in every period (complete-case analysis), causing severe sample loss.

PanelBox instead validates instruments observation-by-observation. For each entity i and period t it (1) identifies the required instrument lags, (2) checks their availability, (3) builds

an entity-specific instrument set from whatever lags exist, and (4) assembles the block-diagonal instrument matrix $\boldsymbol{Z}$ from variable-sized blocks (Figure 2). An observation is kept whenever valid instruments exist, rather than discarding the whole entity. The Arellano–Bond employment data illustrate the difference: 140 firms are observed for 7 to 9 years each (1,031 observations), and only 14 of them for all 9 years. Complete-case analysis keeps those 14 firms and estimates the differenced equation on 98 observations, 9.5% of the panel. PanelBox estimates it on 751 observations (72.8%), every observation for which a valid instrument exists once one lag and one difference are consumed.

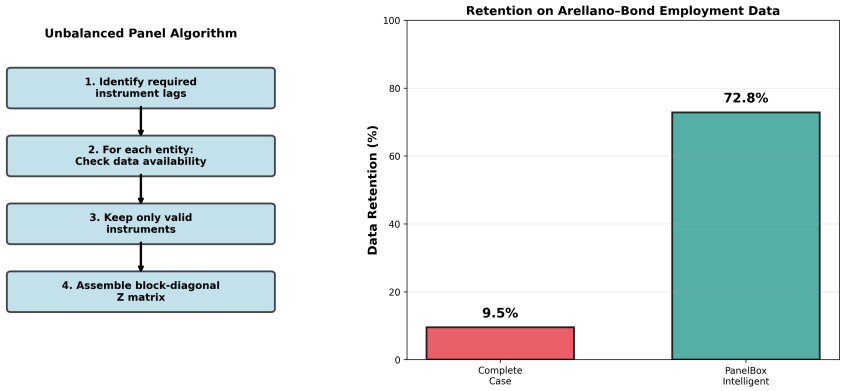


Figure 2. *The unbalanced-panel algorithm: instruments are validated per observation and entity-specific instrument sets are assembled into a block-diagonal matrix. On the Arellano–Bond employment data this retains 72.8% of observations against 9.5% for complete-case analysis.*

4. Testing assumptions: unit roots and cointegration

A real-world econometrics workflow does not end when estimation is done: the validity of the estimates rests on assumptions that must themselves be tested. For macroeconomic panels the first question is whether each variable has a stable long-run level or wanders without one (a *unit root*): regressing one wandering series on another produces convincing-looking but spurious correlations, so testing for unit roots — and, when variables do wander, for a genuine shared long-run relationship (*cointegration*) — is the standard safety check before estimating relationships among variables such as GDP, prices, or exchange rates. This capability is unique to PanelBox among Python panel libraries, despite being standard in macroeconomics and finance.

PanelBox implements the three standard panel unit-root tests and the two standard cointegration tests. The unit-root tests build on the entity-specific augmented Dickey–Fuller regression, testing $H_0 : \rho_i = 0$. The LLC test [17] assumes a common ρ and uses a pooled adjusted t-statistic; the IPS test [18] allows heterogeneous ρ_i and averages the individual t-statistics into an asymptotically normal W; and Fisher-type tests [19] combine the per-entity p-values. For $I(1)$ variables, the Pedroni [20], [21] and Kao [22] tests assess cointegration; Pedroni reports seven statistics and resolves them by majority vote.

```
from panelbox.validation.unit_root import LLCTest, IPSTest

llc = LLCTest(data, variable="log_gdp",
              entity_col="country", time_col="year", trend="ct")
print(llc.run().pvalue)
```

Together these enable a complete workflow — test for unit roots, test for cointegration when variables are $I(1)$, then estimate the appropriate model — illustrated in the growth application of Application: dynamic labor demand.

5. Breadth: eleven model families

Linear panels are only part of applied practice: economic data routinely involve binary outcomes, counts, censoring, spatial dependence, and efficiency frontiers. PanelBox covers these in the same interface; Table 1 maps each family to its closest Stata and R equivalents, and we highlight below only the capabilities not available elsewhere in Python. The complete catalogue, with an executable example notebook per family, is in the official documentation at https://panelbox.readthedocs.io/.

Table 1. *The 11 model families in PanelBox and their closest equivalents in Stata and R. A dash means no established equivalent.*

Family	Representative estimators in PanelBox	Stata	R
Static linear	Pooled OLS, FE, RE, between, first differences	`xtreg`	`plm`
Dynamic GMM	Difference/system GMM, Windmeijer correction, Anderson–Hsiao, LSDVC	`xtabond2`	`plm::pgmm`, `pdynmc`
Unit-root & cointegration tests	LLC, IPS, Fisher; Pedroni, Kao	`xtunitroot`, `xtcointtest`	`plm::purtest`
Discrete choice	Pooled/FE/RE logit & probit, multinomial, ordered, dynamic binary	`xtlogit`, `xtprobit`	`bife`, `pglm`
Count data	Poisson (pooled/FE/QML), negative binomial, zero-inflated, PPML	`xtpoisson`, `ppmlhdfe`	`pglm`, `fixest`
Censored & selection	Panel tobit, sample-selection models	`xttobit`, `xtheckman`	`censReg`
Quantile regression	Pooled, FE, Canay, Machado–Santos Silva	`xtqreg`	`rqpd`
Spatial panels	SAR, SEM, SDM, GNS, dynamic spatial	`spxtregress`	`splm`
Stochastic frontier	Battese–Coelli, Greene TFE/TRE, four-component	`sfpanel`	`frontier`
Panel VAR/VECM	IRFs, FEVD, Granger causality	`pvar`	`panelvar`
Heterogeneous panels	Mean-group estimators, SUR	`xtmg`, `sureg`	`plm::pmg`, `systemfit`

Several entries go beyond their Stata/R counterparts. The discrete-choice family includes dynamic binary models with the J. M. Wooldridge [23] initial-conditions correction and conditional maximum-likelihood fixed effects [24], with average and representative-value marginal effects computed with delta-method standard errors; the count family includes the PPML estimator [25] widely used for gravity models. The spatial family estimates five models by quasi-maximum likelihood [26] and decomposes results into direct and indirect (spillover) effects via the spatial multiplier $(\boldsymbol{I}_N - \rho \boldsymbol{W})^{-1}\beta_k$. The frontier family implements the S. C. Kumbhakar, G. Lien, and J. B. Hardaker [27] four-component model separating *persistent* from *transient* inefficiency [28], [29], [30], and the quantile family includes the J.

A. F. Machado and J. M. C. Santos Silva [31] location-scale estimator, which guarantees non-crossing quantiles [32], [33] — all three previously unavailable in Python.

6. Inference, diagnostics, and reporting

Once a model is estimated, the workflow turns to defending it. All standard errors use the sandwich form $\widehat{\mathrm{Var}}\left(\hat{\beta}\right) = (\boldsymbol{X}'\boldsymbol{X})^{-1}\,\hat{\boldsymbol{\Omega}}\,(\boldsymbol{X}'\boldsymbol{X})^{-1}$, differing in the inner "meat" matrix $\hat{\boldsymbol{\Omega}}$ (the term of art for the filling of the sandwich form): HC0–HC3 for heteroskedasticity, one-way and two-way clustering, and the Driscoll–Kraay and Newey–West HAC estimators. Four bootstraps — pairs, wild, block, and residual — cover small-sample and non-standard cases.

The diagnostic suite includes the Hausman test [34] comparing FE and RE, and the GMM trio that governs dynamic-panel validity: the Hansen J test of overidentifying restrictions [35], and the Arellano–Bond AR(1) and AR(2) tests for serial correlation, where AR(1) is expected to reject and AR(2) to not reject.

Result objects render publication-ready output. `ComparisonResult` aligns several models side by side, and the reporting layer produces self-contained HTML with embedded interactive Plotly charts (28+ chart types across residual diagnostics, model comparison, and econometric tests) [36], or static LaTeX/Markdown:

```
comparison = pb.ComparisonResult({
    "Pooled OLS": ols_result, "FE": fe_result,
    "Diff-GMM": diffgmm_result, "Sys-GMM": sysgmm_result,
})
print(comparison.summary())
```

7. Using and extending PanelBox

PanelBox makes no assumptions about where data come from. Any long-format pandas `DataFrame` with an entity column and a time column works directly — loaded from CSV, Parquet, SQL, or an API via the usual pandas readers — and unbalanced panels are handled natively, so no reshaping or gap-filling is required. The 103 bundled datasets are conveniences for teaching, benchmarking, and replication, not a requirement.

All estimators share one contract: construct with the data and a specification, call `fit()`, receive a results object with the same methods everywhere. Comparing a fixed-effects model with a system-GMM model is a one-line change of class, and `ComparisonResult` aligns any set of results. Within this contract, the supported customization points are the covariance estimator and bootstrap scheme (per `fit()` call), the instrument design for GMM (`gmm_max_lag`, `collapse`), arbitrary variable transformations through `patsy` formulas, marginal-effects options for nonlinear models, and the output format of every report (HTML, LaTeX, Markdown).

At the moment there are two limitations in PanelBox that we hope to overcome in the future. First, there is no public API for user-defined moment conditions or custom estimators: internally all models subclass a common base model and results class, and a documented, stable extension interface is on the roadmap. Second, GMM is restricted to linear dynamic panels. The official documentation (https://panelbox.readthedocs.io/) includes a gallery of executable Jupyter notebooks covering every model family end to end, which is the recommended starting point for adapting the library to a new use case.

8. Application: dynamic labor demand

To illustrate the capabilities of the library, we use it to reproduce the canonical application of M. Arellano and S. Bond [3]: a dynamic labor-demand equation on UK firm-level data, an unbalanced panel of $N = 140$ firms observed for 7 to 9 years between 1976 and 1984 (1,031 observations). The model

$$n_{it} = \alpha\, n_{i,t-1} + \beta_w w_{it} + \beta_k k_{it} + \eta_i + \lambda_t + \varepsilon_{it} \tag{3}$$

relates log employment n_{it} to log real wages w_{it} and log capital k_{it}; α captures employment adjustment costs. We estimate it four ways to expose the bias–consistency trade-off:

```
data = pb.load_dataset("abdata")
static = data.dropna(subset=["nL1"])  # nL1: lagged employment
ols = pb.PooledOLS(
    "n ~ nL1 + w + k", static, entity_col="id", time_col="year",
).fit()
fe = pb.FixedEffects(
    "n ~ nL1 + w + k", static, entity_col="id", time_col="year",
).fit(cov_type="clustered")
gmm_spec = dict(
    dep_var="n", lags=1, id_var="id", time_var="year",
    exog_vars=["w", "k"], gmm_max_lag=9, collapse=True,
    two_step=True, time_dummies=True,
)
diffgmm = pb.DifferenceGMM(data, **gmm_spec).fit()
sysgmm = pb.SystemGMM(data, **gmm_spec).fit()
```

The estimates follow the bracketing argument of S. R. Bond [37] (Figure 3). Pooled OLS yields $\hat{\alpha} = 0.931$, biased upward by the correlation between $n_{i,t-1}$ and η_i; fixed effects gives $\hat{\alpha} = 0.528$, biased downward by the Nickell bias. A consistent estimate should lie between the two, and both GMM estimators do: difference GMM at $\hat{\alpha} = 0.877$ (standard error 0.234) and system GMM at 0.563 (0.194). The wide confidence intervals are the price of instrumenting a persistent regressor with its own lags; the system estimator's additional level moment conditions reduce the standard error but, with only 140 firms and collapsed instruments, do not eliminate it. The wage elasticity is negative in every specification ($\hat{\beta}_w$ between -0.25 and -0.50 outside OLS) and capital enters positively. The diagnostics do not reject either GMM specification: the Hansen J test gives $p = 0.244$ (difference) and $p = 0.230$ (system), AR(1) rejects as expected ($p < 0.01$) while AR(2) does not ($p = 0.57$ and 0.87), and the instrument counts (16 and 18) stay far below the 140 entities.

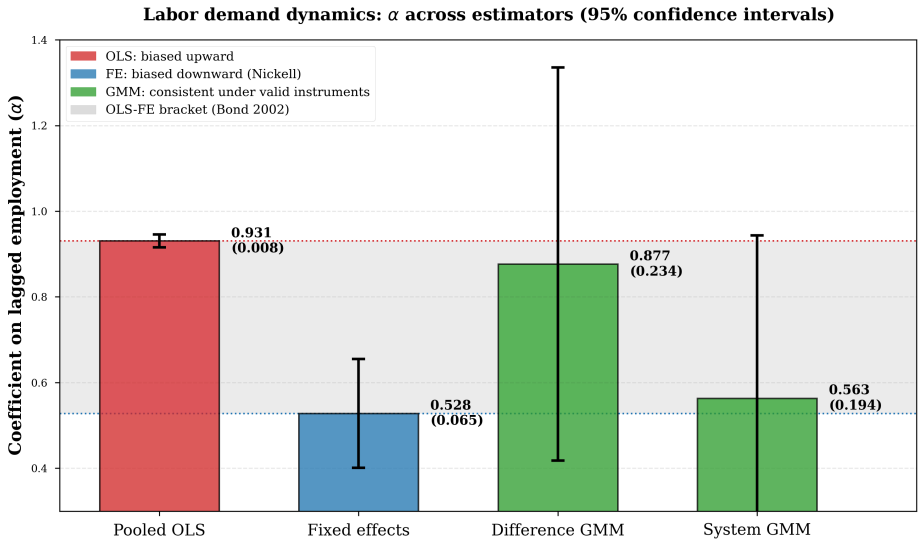


Figure 3. *Coefficient on lagged employment across pooled OLS, fixed effects, difference GMM, and system GMM, with 95% confidence intervals. The shaded band is the OLS–FE bracket of S. R. Bond [37]: OLS is biased upward and fixed effects downward, so a consistent estimate is expected to fall between them, as both GMM estimates do.*

The same building blocks support a full nonstationary-panel workflow on the Penn World Table 10.01 [38], using the 129 countries with complete data for 1970–2019. The IPS test does not reject a unit root in log GDP per capita ($p = 1.00$) while rejecting it in first differences ($p < 0.001$), so the series is treated as $I(1)$; the LLC test rejects in levels, a known consequence of its common-root assumption under cross-sectional dependence, and the disagreement itself is informative. Pedroni's test then rejects no-cointegration between log GDP per capita, the investment share and human capital on five of seven statistics. A conditional-convergence regression on non-overlapping five-year panels gives an autoregressive coefficient of 0.944 (0.006) by pooled OLS and 0.791 (0.027) by fixed effects; system GMM returns 0.808 (0.050), inside the bracket and implying convergence at roughly 4% per year, in line with panel estimates in the growth literature [39]. The Hansen test rejects ($p = 0.002$), however, flagging the instrument validity problems that are well known for this specification and that the library's diagnostics are designed to expose.

9. Validation and performance

We validate PanelBox against R's `plm` [7], estimating identical specifications on the Arellano–Bond employment data and computing relative differences $\Delta = |\,\mathrm{PanelBox} - \mathrm{reference}|\,/\,|\,\mathrm{reference}|$ (Table 2). For difference GMM — two-step, collapsed instruments, year dummies, Windmeijer-corrected standard errors — every coefficient, standard error, and the Hansen J statistic match `plm::pgmm` to at least three decimals, and the instrument count is identical. Fixed-effects estimates match to six decimals. The panel unit-root tests were compared with `plm::purtest` on log GDP per capita from the Penn World Table: the IPS W statistic is identical, and the LLC statistic agrees within 2%, the residual difference coming from the long-run variance kernel.

System GMM requires a different reference. `plm` and Stata's `xtabond2` [6] weight the level equations differently and do not agree with each other on this estimator; PanelBox follows the `xtabond2` convention (its default H matrix, which links the differenced and level equations). We therefore compare system GMM with `pydynpd` [40], a Python implementation of the `xtabond2` algorithm validated against Stata by its author: coefficients agree within 1–2% and the Hansen J test is identical, the remaining gap being one level-equation observation per entity that `xtabond2` retains and PanelBox currently drops. A direct comparison with Stata is planned.

Table 2. *Numerical validation on the Arellano–Bond employment data (two-step GMM, collapsed instruments, year dummies; standard errors in parentheses). Difference GMM and fixed effects are compared with R plm; system GMM with pydynpd, a Python replica of Stata's xtabond2; unit-root tests with plm on Penn World Table log GDP per capita.*

Quantity	Reference	PanelBox	Δ (%)
Diff-GMM `L.n` (s.e.)	0.8766 (0.2342)	0.8766 (0.2342)	0.00
Diff-GMM `w` (s.e.)	-0.4587 (0.1698)	-0.4587 (0.1698)	0.00
Diff-GMM `k` (s.e.)	0.1907 (0.0649)	0.1907 (0.0649)	0.00
Diff-GMM Hansen J (p) / instruments	7.91 (0.244) / 16	7.91 (0.244) / 16	0.00
Sys-GMM `L.n` (s.e.)	0.5622 (0.1871)	0.5634 (0.1938)	0.21
Sys-GMM `w` (s.e.)	-0.2414 (0.1116)	-0.2467 (0.1107)	2.2
Sys-GMM `k` (s.e.)	0.3547 (0.1462)	0.3534 (0.1503)	0.37

Sys-GMM Hansen J (p) / instruments	9.33 (0.230) / 18	9.33 (0.230) / 18	0.00
FE `w` (s.e.)	-0.367774 (0.052323)	-0.367774 (0.052323)	0.00
FE `k` (s.e.)	0.640367 (0.020142)	0.640367 (0.020142)	0.00
IPS W (lags 1)	6.392	6.392	0.00
LLC t^* (lags 1)	-3.547	-3.518	0.82

PanelBox is implemented in pure Python on NumPy/SciPy. On simulated dynamic panels with $T = 10$, two-step system GMM takes 1.1 s for $N = 500$ and 4.6 s for $N = 2,500$, against 0.5 s and 3.1 s for `plm::pgmm` (Figure 4): within a factor of 1.5–2 of the compiled R implementation for $N \geq 500$, with a fixed overhead of about 0.6 s that dominates on small panels. Memory scales linearly in NT, and bootstrap inference parallelizes with near-linear speedup.

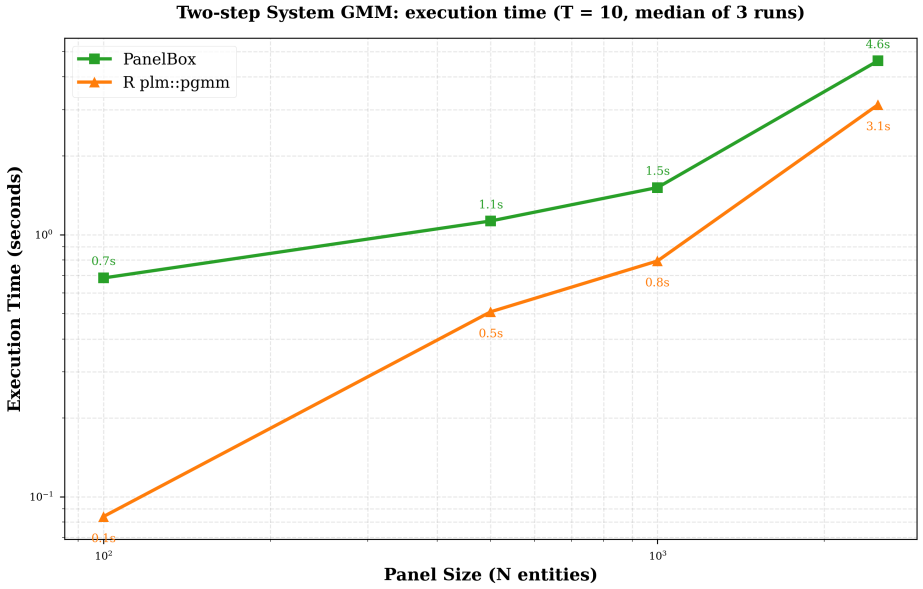


Figure 4. *Execution time of two-step system GMM as a function of the number of entities ($T = 10$, median of three runs): PanelBox versus R `plm::pgmm`.*

10. Conclusion

PanelBox brings comprehensive panel data econometrics to Python: 70+ models across 11 families, 50+ diagnostic tests, 11 standard-error estimators, an interactive visualization and reporting system, and 103 bundled datasets, in a single MIT-licensed package built on the scientific Python stack. Its difference GMM reproduces R's `plm` to three decimals and its system GMM follows the `xtabond2` conventions, it adds an unbalanced-panel algorithm that estimates on 72.8% of the Arellano–Bond observations where complete-case analysis uses 9.5%, and it offers capabilities — panel unit-root and cointegration tests, four-component stochastic frontiers, non-crossing quantile regression, panel VAR/VECM — previously unavailable in Python. The unusual emphasis on numerical validation throughout this paper is not incidental: it reflects the library's origin in production model validation, where matching the reference implementation is the requirement, not a nicety.

Current limitations include spatial-model scalability beyond $N \approx 2,000$ (mitigated by Chebyshev approximation), GMM restricted to linear dynamic panels, and no GPU acceleration. Planned work includes FMOLS/DOLS for cointegrated panels, double/debiased machine learning for panel data, GPU acceleration via JAX/CuPy, and Bayesian panel models. By eliminating the need to leave Python for sophisticated panel analysis, PanelBox makes

empirical economic research more reproducible, more integrated with the machine-learning ecosystem, and more accessible. It is available on PyPI (`pip install panelbox`) and on GitHub at https://github.com/PanelBox-Econometrics-Model/panelbox [41], with documentation, tutorials, example notebooks for every model family, and full replication materials for this paper.

11. Generative AI disclosure

Portions of this work were assisted using generative AI tools, specifically Anthropic's Claude. The tools were used to help draft and refine the prose of the manuscript, to improve clarity and phrasing, to suggest edits to the LaTeX/MyST markup, and to suggest code for parts of the PanelBox library and its replication scripts. Generative AI was **not** used to produce the numerical results, statistical validations (e.g., the comparisons against `xtabond2` and `plm`), or references reported here: these derive from actual executions of the software and were checked against the reference tools. All outputs were reviewed, verified, and revised by the author, who takes full responsibility for the accuracy and integrity of the final content, including all technical claims, results, and references.

References

[1] B. H. Baltagi, *Econometric Analysis of Panel Data*, 6th ed. Cham, Switzerland: Springer, 2021. doi: 10.1007/978-3-030-53953-5.

[2] J. M. Wooldridge, *Econometric Analysis of Cross Section and Panel Data*, 2nd ed. Cambridge, MA: MIT Press, 2010.

[3] M. Arellano and S. Bond, "Some Tests of Specification for Panel Data: Monte Carlo Evidence and an Application to Employment Equations," *The Review of Economic Studies*, vol. 58, no. 2, pp. 277–297, 1991, doi: 10.2307/2297968.

[4] R. Blundell and S. Bond, "Initial Conditions and Moment Restrictions in Dynamic Panel Data Models," *Journal of Econometrics*, vol. 87, no. 1, pp. 115–143, 1998, doi: 10.1016/S0304-4076(98)00009-8.

[5] S. R. Bond, A. Hoeffler, and J. Temple, "GMM Estimation of Empirical Growth Models," *Economics Papers 2001-W21*, 2001.

[6] D. Roodman, "How to Do xtabond2: An Introduction to Difference and System GMM in Stata," *The Stata Journal*, vol. 9, no. 1, pp. 86–136, 2009, doi: 10.1177/1536867X0900900106.

[7] Y. Croissant and G. Millo, "Panel Data Econometrics in \proglangR: The \pkgplm Package," *Journal of Statistical Software*, vol. 27, no. 2, pp. 1–43, 2008, doi: 10.18637/jss.v027.i02.

[8] M. Fritsch, A. A. Pua, and J. Schnurbus, "\pkgpdynmc: A Package for Estimating Linear Dynamic Panel Data Models Based on Nonlinear Moment Conditions," *Working Paper*, 2019.

[9] C. R. Harris, K. J. Millman, S. J. van der Walt, and others, "Array Programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[10] W. McKinney, "\pkgpandas: A Foundational \proglangPython Library for Data Analysis and Statistics," in *Python for High Performance and Scientific Computing*, 2011, pp. 1–9.

[11] P. Virtanen, R. Gommers, T. E. Oliphant, and others, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[12] K. Baum, "\pkglinearmodels: Linear Models with Panel Data." [Online]. Available: https://github.com/bashtage/linearmodels

[13] S. Seabold and J. Perktold, "\pkgstatsmodels: Econometric and Statistical Modeling with \proglangPython," *Proceedings of the 9th Python in Science Conference*, pp. 92–96, 2010, doi: 10.25080/Majora-92bf1922-011.

[14] S. Nickell, "Biases in Dynamic Models with Fixed Effects," *Econometrica*, vol. 49, no. 6, pp. 1417–1426, 1981, doi: 10.2307/1911408.

[15] F. Windmeijer, "A Finite Sample Correction for the Variance of Linear Efficient Two-Step GMM Estimators," *Journal of Econometrics*, vol. 126, no. 1, pp. 25–51, 2005, doi: 10.1016/j.jeconom.2004.02.005.

[16] D. Roodman, "A Note on the Theme of Too Many Instruments," *Oxford Bulletin of Economics and Statistics*, vol. 71, no. 1, pp. 135–158, 2009, doi: 10.1111/j.1468-0084.2008.00542.x.

[17] A. Levin, C.-F. Lin, and C.-S. J. Chu, "Unit Root Tests in Panel Data: Asymptotic and Finite-Sample Properties," *Journal of Econometrics*, vol. 108, no. 1, pp. 1–24, 2002, doi: 10.1016/S0304-4076(01)00098-7.

[18] K. S. Im, M. H. Pesaran, and Y. Shin, "Testing for Unit Roots in Heterogeneous Panels," *Journal of Econometrics*, vol. 115, no. 1, pp. 53–74, 2003, doi: 10.1016/S0304-4076(03)00092-7.

[19] G. S. Maddala and S. Wu, "A Comparative Study of Unit Root Tests with Panel Data and a New Simple Test," *Oxford Bulletin of Economics and Statistics*, vol. 61, no. S1, pp. 631–652, 1999, doi: 10.1111/1468-0084.0610s1631.

[20] P. Pedroni, "Critical Values for Cointegration Tests in Heterogeneous Panels with Multiple Regressors," *Oxford Bulletin of Economics and Statistics*, vol. 61, no. S1, pp. 653–670, 1999, doi: 10.1111/1468-0084.0610s1653.

[21] P. Pedroni, "Panel Cointegration: Asymptotic and Finite Sample Properties of Pooled Time Series Tests with an Application to the PPP Hypothesis," *Econometric Theory*, vol. 20, no. 3, pp. 597–625, 2004, doi: 10.1017/S0266466604203073.

[22] C. Kao, "Spurious Regression and Residual-Based Tests for Cointegration in Panel Data," *Journal of Econometrics*, vol. 90, no. 1, pp. 1–44, 1999, doi: 10.1016/S0304-4076(98)00023-2.

[23] J. M. Wooldridge, "Simple Solutions to the Initial Conditions Problem in Dynamic, Nonlinear Panel Data Models with Unobserved Heterogeneity," *Journal of Applied Econometrics*, vol. 20, no. 1, pp. 39–54, 2005, doi: 10.1002/jae.770.

[24] G. Chamberlain, "Analysis of Covariance with Qualitative Data," *The Review of Economic Studies*, vol. 47, no. 1, pp. 225–238, 1980, doi: 10.2307/2297110.

[25] J. M. C. Santos Silva and S. Tenreyro, "The Log of Gravity," *The Review of Economics and Statistics*, vol. 88, no. 4, pp. 641–658, 2006, doi: 10.1162/rest.88.4.641.

[26] L.-f. Lee and J. Yu, "Estimation of Spatial Autoregressive Panel Data Models with Fixed Effects," *Journal of Econometrics*, vol. 154, no. 2, pp. 165–185, 2010, doi: 10.1016/j.jeconom.2009.08.001.

[27] S. C. Kumbhakar, G. Lien, and J. B. Hardaker, "Technical Efficiency in Competing Panel Data Models: A Study of Norwegian Grain Farming," *Journal of Productivity Analysis*, vol. 41, no. 2, pp. 321–337, 2014, doi: 10.1007/s11123-012-0303-1.

[28] G. E. Battese and T. J. Coelli, "Frontier Production Functions, Technical Efficiency and Panel Data: With Application to Paddy Farmers in India," *Journal of Productivity Analysis*, vol. 3, no. 1–2, pp. 153–169, 1992, doi: 10.1007/BF00158774.

[29] G. E. Battese and T. J. Coelli, "A Model for Technical Inefficiency Effects in a Stochastic Frontier Production Function for Panel Data," *Empirical Economics*, vol. 20, no. 2, pp. 325–332, 1995, doi: 10.1007/BF01205442.

[30] W. Greene, "Reconsidering Heterogeneity in Panel Data Estimators of the Stochastic Frontier Model," *Journal of Econometrics*, vol. 126, no. 2, pp. 269–303, 2005, doi: 10.1016/j.jeconom.2004.05.003.

[31] J. A. F. Machado and J. M. C. Santos Silva, "Quantiles via Moments," *Journal of Econometrics*, vol. 213, no. 1, pp. 145–173, 2019, doi: 10.1016/j.jeconom.2019.04.009.

[32] R. Koenker, "Quantile Regression for Longitudinal Data," *Journal of Multivariate Analysis*, vol. 91, no. 1, pp. 74–89, 2004, doi: 10.1016/j.jmva.2004.05.006.

[33] I. A. Canay, "A Simple Approach to Quantile Regression for Panel Data," *The Econometrics Journal*, vol. 14, no. 3, pp. 368–386, 2011, doi: 10.1111/j.1368-423X.2011.00349.x.

[34] J. A. Hausman, "Specification Tests in Econometrics," *Econometrica*, vol. 46, no. 6, pp. 1251–1271, 1978, doi: 10.2307/1913827.

[35] L. P. Hansen, "Large Sample Properties of Generalized Method of Moments Estimators," *Econometrica*, vol. 50, no. 4, pp. 1029–1054, 1982, doi: 10.2307/1912775.

[36] Plotly Technologies Inc., "Collaborative data science." [Online]. Available: https://plot.ly/

[37] S. R. Bond, "Dynamic panel data models: a guide to micro data methods and practice," *Portuguese Economic Journal*, vol. 1, no. 2, pp. 141–162, 2002, doi: 10.1007/s10258-002-0009-9.

[38] R. C. Feenstra, R. Inklaar, and M. P. Timmer, "The next generation of the Penn World Table," *American Economic Review*, vol. 105, no. 10, pp. 3150–3182, 2015, doi: 10.1257/aer.20130954.

[39] N. G. Mankiw, D. Romer, and D. N. Weil, "A Contribution to the Empirics of Economic Growth," *The Quarterly Journal of Economics*, vol. 107, no. 2, pp. 407–437, 1992, doi: 10.2307/2118477.

[40] D. Wu, "pydynpd: A Python package for dynamic panel model," *Journal of Open Source Software*, vol. 8, no. 83, p. 4416, 2023, doi: 10.21105/joss.04416.

[41] G. C. Haase, P. H. Dourado, and O. Candido, "PanelBox: A Comprehensive Python Library for Panel Data Econometrics." [Online]. Available: https://github.com/PanelBox-Econometrics-Model/panelbox

Open Access | https://doi.org/10.25080/akyc5426

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Automated Data Enrichment for Police Accountability: Where Agentic Judgment Earns Its Place

Hongsup Shin[1]

[1]Texas Justice Initiative

Abstract

Automated data enrichment, filling missing fields in structured records from unstructured sources, is the canonical case for pointing an autonomous agent at a database and letting it fill every blank. In high-stakes data that instinct is dangerous. A confidently wrong value is worse than a blank, and retrieval-grounded extraction reduces but does not remove the tendency to assert what the source never stated. An LLM can extract these fields; this paper asks where agentic judgment earns its place and where it becomes a liability.

We study this on the Texas Justice Initiative's police shooting databases, where nearly two thousand records are missing the weapon, the subject's race, or the outcome, whose fields volunteers typically recover by hand, fifteen to thirty minutes each. Our LangGraph pipeline, deterministic in its control flow, searches, validates, extracts, and escalates hard cases to a human. It completes 92% of officer and 70% of civilian records. On an adversarial probe of twenty incidents we fabricated, where the only correct behavior is to refuse, it completes none of the twenty and asserts nothing, escalating every record to a human. The recovery itself came from a fix to the extraction prompt, without any agent. An autonomous agent pointed at the same fabricated incidents, with more freedom, commits a wrong-article fabrication the pipeline escalates.

If the deterministic core does the recovering, the agentic layer earns its place by making those recovered values trustworthy. Agency lives only in this thin judgment layer above extraction, which holds three bounded judges in two categories. One judge acts on the pipeline's control flow: the relevance judge reads the retrieved articles and, when none actually report this incident, routes the record to a human instead of completing it. The other two judge what extraction produced: the race verifier deletes a value the source never states, and the conflict annotator explains to the reviewer why the sources disagree on a value. Extraction calls an LLM too, but because it only proposes values for these judges to rule on, we do not count it as agentic. Every judge had to clear a reward-hacking-resistant evaluation gate before it shipped. The main contribution of this paper is a discipline, an "earn-it" protocol, for drawing the line between what a high-stakes pipeline should settle deterministically and where it is worth granting agentic judgment.

Keywords police accountability, data enrichment, LangGraph, LLM-as-judge, agentic workflows, evaluation gate, faithfulness, human-in-the-loop

Published Jul 13, 2026

Correspondence to
Hongsup Shin
hongsup.shin@pm.me

Open Access

Copyright © 2026 Shin. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

In Texas, the Office of the Attorney General (OAG) is required by law to collect a report on every officer-involved shooting (OIS) and to publish an annual summary. Those summaries are high-level: they omit age demographics, report dates, and the number of officers involved, and they carry no intersectional analysis [1]. The Texas Justice Initiative (TJI), a

nonprofit, re-publishes the underlying incident records with far more granularity, the detail independent analysis depends on. But the records are incomplete. About a quarter of incidents are filed with an unidentified cause; race is recorded with a coarse vocabulary that law enforcement is known to mischaracterize; weapon and outcome fields are frequently blank [1]. These gaps carry substantive consequences: undercounting and misclassification of police violence are documented at the national scale [2], and each blank field leaves a question about a police shooting that the public record cannot answer.

TJI addresses these gaps through the standard practice of accountability nonprofits: volunteers search news archives one incident at a time, read the coverage, and transcribe the details. The work takes 15-30 minutes per record, and there are nearly two thousand records. Hand-coding at this scale has precedent: Fryer's team extracted every variable from Houston police reports manually after being advised that the natural-language processing of the day could not handle their complexity [3]. This is precisely the kind of tedious, search-and-extract task that modern large language models (LLMs) appear built for, and the temptation is to point an autonomous agent at the database and let it fill every blank. Human-factors research has long documented that operators over-trust automation in exactly such conditions [4]. We test this directly: an autonomous agent on the same adversarial probe declines most incidents but completes one fabricated record with no signal a reviewer could use to distrust it.

This paper argues against that approach for high-stakes domains; that argument, together with a system that embodies the alternative, is its contribution. The failure mode that matters is the *confidently wrong* field, worse than the blank a volunteer would otherwise leave: a fabricated weapon, a misattributed race, a detail lifted from a different shooting that shares a city and a date. Generative extraction grounded in retrieved documents [5] reduces but does not remove *unfaithfulness*, the tendency to emit content the source does not support, a failure documented in summarization [6] and surveyed broadly for LLMs [7], [8]. Uncritical automation therefore recovers and fabricates in the same pass [9]. This leads to the following engineering questions: how to earn the right to trust what an LLM extracts, and how to decide which decisions warrant an LLM.

Our answer is a system designed as a *workflow* with bounded agentic judgment rather than an autonomous agent. The distinction is a useful design axis [10]: a workflow orchestrates models and tools along predefined code paths, whereas an agent lets the model direct its own process, as in ReAct-style loops that interleave reasoning and action under the model's control [11]. Frameworks such as LangGraph [12] make either style straightforward to build as a stateful graph, but autonomy is a cost as much as a capability: every place the model is allowed to choose is a place it can choose wrong. Two senses of *agentic* are worth separating: whether a model *directs control flow* (chooses what runs next), and whether a model *holds authority over a record*, disposing of an already-produced value, not merely proposing one (Table 1). Our orchestration is deterministic in the first sense, a state machine in which no model directs the sequence of operations; the one routing outcome attributable to a model judgment is the relevance judge's veto. What separates the judges from extraction is the second sense: extraction only proposes content, whereas three small bounded judges hold authority to veto, null, or flag what it produced, applying the now-common pattern of using an LLM to judge a narrow, checkable question [13] ("do these articles actually describe this incident?"). Their authority is graduated, from the relevance judge's veto down to the conflict annotator's advisory note. Each judge earned its place by clearing an offline evaluation gate designed to resist the most common way such systems fool their builders: optimizing the headline metric (here, completion rate) at the expense of the true objective (faithfulness). We make three claims that should transfer beyond this case:

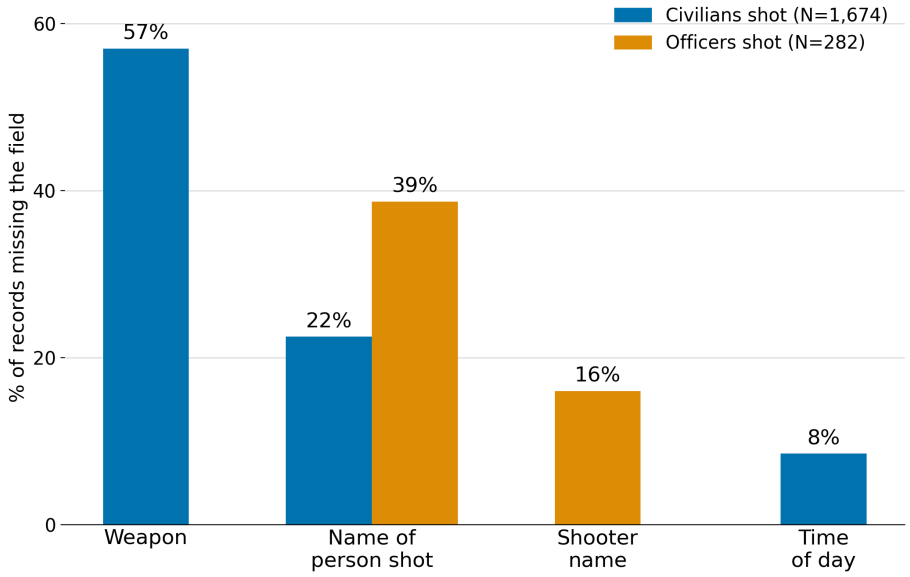
1. **Agentic judgment belongs to the evidence layer, above extraction.** The single largest gain in this project came from a change to the fixed extraction prompt, with no agent involved; the judges add faithfulness and leave recall to retrieval.
2. **A multi-objective, reward-hacking-resistant evaluation gate is the principal safeguard.** Optimizing a proxy metric reliably degrades the true objective once the two diverge [14], [15], and "complete more records" is trivially gamed by accepting weak extractions; the gate therefore scores every change on three gating guards: completion (tracked and surfaced, so a drop is never silent), a hard zero-hallucination veto, and field-level correctness on a stable cohort, measured so that a completion gain cannot launder a correctness loss.
3. **Authority should be calibrated to stakes.** Our three judges can *block* a record into human review, *null* an unsupported value, or merely *advise* a reviewer; they never silently overwrite the record.

The system is open source, and all quantitative results reported below are produced by the pipeline's evaluation harness, applied to the data described next.

2. Methods

2.1. Datasets

TJI publishes two related datasets that we treat throughout: `civilians_shot` (police shooting civilians; 1,674 records) and `officers_shot` (civilians shooting police; 282 records), spanning 2014–2024, for 1,956 records total. They use different field names for analogous concepts (in `officers_shot` the civilian is the *shooter* and the outcome is the *officer's* injury or death), a divergence the pipeline handles through a `DatasetType` enum that switches the database queries and tailors the extraction prompt to each dataset. In the source data, 57% of civilian records are missing the weapon, 22.5% are missing the subject's name, and 39% of officer records are missing the officer's name (Figure 1). The civilian dataset records each involved officer's demographics but no name, so the officer who fired is structurally anonymous in the published record; `officer_name` is one of the fields the pipeline extracts. We measure accuracy against a held-out sample of these records, defined in Evaluation design below.



Single bars: the other dataset does not record the field. Computed from the published TJI tables.

Figure 1. *Missing fields as a share of each dataset's records, computed from the published TJI tables. A single bar means the other dataset does not record that field.*

2.2. Pipeline architecture

The pipeline is a seven-node LangGraph state machine (Figure 2). Each node accepts and returns a single typed state object (a `pydantic` model [16]), and a deterministic **Coordinator** reads the stage that just ran and routes the record to retry, proceed, or escalate:

- **Load** reads incident anchors from PostgreSQL [17] via `psycopg2` [18].
- **Search** queries the Tavily web-search API [19] within a date window around the incident.
- **Validate** keeps only articles that match the incident on a tiered rule: one with a parsed publication date must match on date proximity (±5 days) and location; one lacking a date must match on location and the victim's name; with neither, it falls back to location alone, keeping date-less results usable without admitting articles that share only a city.
- **Synthesize** runs the LLM extraction and the agentic layer described below.
- **Complete** and **Escalate**, the two terminal nodes, write a JSON result or an escalation report for human review.

When validation leaves too few usable articles, the Coordinator climbs a fixed three-rung search ladder, from an exact-date match to a month-and-year window to a name-only query, and escalates only after the third rung fails.

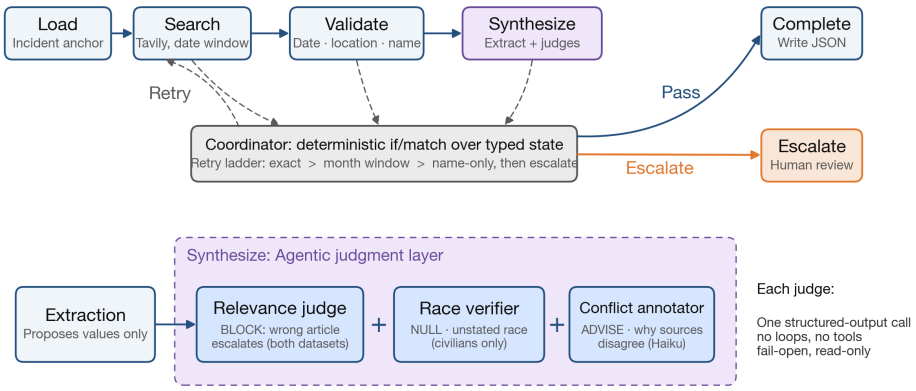


Figure 2. *The seven-node pipeline. The deterministic Coordinator is the hub through which every transition passes: each processing node returns its result to the Coordinator (dashed edges), which dispatches the record onward,* pass *along the happy path (Load, Search, Validate, Synthesize) into the Complete terminal,* retry *back to Search, or* escalate *to the human-review terminal. On thin retrieval it climbs the fixed three-rung search ladder printed on its box before escalating. A retry follows an empty search or a validation that rejects every article; an escalation can originate at any gate. The bottom row expands the Synthesize node: after extraction, which only proposes values, three bounded LLM judges run in sequence as the agentic layer: the relevance judge can* block *a wrong-article completion (the Coordinator then escalates it as* `irrelevant_sources`*), the race verifier* nulls *an unstated race, and the conflict annotator* advises *the human reviewer.*

Each run ends at one of two terminal nodes, and a human reviewer reads the JSON it writes (Figure 3).

```
{
  "incident_id": "178", "dataset_type":
"civilians_shot",
  "extracted_fields": [
    {"field_name": "weapon", "value":
"SHOTGUN", "confidence": "high",
     "sources": ["kwtx.com/content/news/
Officers-cleared-in-deadly..."],
     "source_quotes": ["Ramirez was shot after
he pointed a shotgun
        in the direction of officers."],
     "extraction_method": "llm"}
    // 5 more: civilian_name, civilian_age,
location_detail, time_of_day, outcome
  ],
  "search_strategy": "temporal_expanded",
"retry_count": 1,
  "outcome_summary": "Enriched 6 fields for
incident 178 (civilians_shot)"
}
```

(a)

```
{
  "incident_id": "357", "dataset_type":
"officers_shot",
  "escalation_reason": "irrelevant_sources",
"current_stage": "synthesize",
  "retrieved_articles": [
    {"title": "Capital murder trial of man
accused of killing SAPD officer during 2013
chase begins"}
    // 9 more retrieved articles
  ],
  "outcome_summary": "Escalated incident 357:
irrelevant_sources after 0 retries"
}
```

(b)

Figure 3. *A completed record (`civilians_shot` incident 178):*

An escalated record (`officers_shot` incident 357), vetoed by the relevance judge:

The two terminal outputs (verbatim from the pipeline's saved JSON, abridged; incident identifiers are database keys). The completion (top) carries each field's confidence, sources, and supporting quotes, so a reviewer can verify a value in seconds. The escalation (bottom) is the relevance judge blocking a dossier whose strongest article passes every rule-based check but recounts a different (2013) case.

Two design choices are deliberate. First, **the orchestration is deterministic**: the Coordinator is `if`/`match` logic over state fields and the retry ladder is a fixed list. *Deterministic* here describes the control flow, not the model outputs: every LLM call is genuinely sampled (no temperature is set, so calls use the Anthropic default of 1.0, and the API exposes no seed), so its run-to-run variance, which we return to in Limitations, comes from both sampling and API-side nondeterminism; the architecture is built to catch wrong values, not to remove variance. Which nodes run, in what order, and on which outcomes is fixed code no model can redirect, save the one model-attributable route, the relevance judge's veto. We chose this over an LLM router because, in a high-stakes domain, predictable control flow is itself a safety property. Second, the system is **human-in-the-loop (HITL) by construction**. It never writes back to the source database, and escalation is a designed terminal outcome that hands the record to a human.

The entire graph is assembled in one function (simplified from `src/agents/graph.py`):

```python
def build_graph(checkpointer=None) -> CompiledStateGraph:
    workflow = StateGraph(EnrichmentState)        # state schema = a pydantic model
    workflow.add_node("load", load_node)
    workflow.add_node("search", search_node)
    workflow.add_node("validate", validate_node)
    workflow.add_node("synthesize", synthesize_node)
    workflow.add_node("complete", complete_node)
    workflow.add_node("escalate", escalate_node)
    workflow.add_node("coordinate", coordinate_node)

    workflow.add_edge(START, "load")
    workflow.add_edge("load", "coordinate")       # every processing node
    workflow.add_edge("search", "coordinate")     #   returns to the coordinator hub
    workflow.add_edge("validate", "coordinate")
    workflow.add_edge("synthesize", "coordinate")
    workflow.add_edge("complete", END)
    workflow.add_edge("escalate", END)

    # the only branch in the graph, and it is deterministic (not an LLM router)
    workflow.add_conditional_edges("coordinate", route_after_coordinator)
    return workflow.compile(checkpointer=checkpointer)
```

2.2.1. *How we use LangGraph*

LangGraph models a pipeline as a graph of plain functions over a single typed state object. We use it narrowly: the state schema is the `EnrichmentState` `pydantic` model, so the whole run is one validated, examinable object, and there is exactly one conditional edge, out of the Coordinator, whose target is chosen by a deterministic `match` on the stage that just ran (simplified from `src/agents/coordinate_node.py`):

```python
match state.current_stage:
    case PipelineStage.LOAD:       state = check_load_results(state)
    case PipelineStage.SEARCH:     state = check_search_results(state)
    case PipelineStage.VALIDATE:   state = check_validate_results(state)
    case PipelineStage.SYNTHESIZE: state = check_synthesize_results(state)
# route_after_coordinator(state) then returns state.next_stage.value
```

Where the ecosystem's agent abstractions (LangChain's agent executors, CrewAI, AutoGen) let a model choose tools and coordinate other agents, LangGraph exposes the graph and its typed state directly [12], [20]. The one routing outcome attributable to a model judgment is the relevance judge's veto, which the Coordinator reads like any other state field; its sibling escalations, on empty or conflicting extractions, are model-agnostic completeness guards that would fire the same way for a regex extractor. Dependencies are injected through `RunnableConfig`, so the judges are testable without a live Claude [21] client.

2.3. Deterministic conflict reduction first

Before any LLM is consulted, deterministic code exhausts what rules alone can resolve, so the judges are reserved for what code genuinely cannot do. The steps run in pipeline order:

1. **Aggregation sources are excluded** outright: multi-incident pages such as Wikipedia lists or Fatal Encounters records [22] contaminate extraction by folding several incidents into one document, so the search and validation nodes drop those domains and any PDF or CSV URL, leaving only documents about a single incident.
2. **Extraction is anchored to the record's subject**: when an article describes several people (a second victim, a bystander, the officer), the prompt extracts fields only for the target individual, never blending several together.
3. **Names, race terms, and weapon categories are normalized** before comparison, so that honorific and phrasing variants do not register as disagreements.

4. **A consensus resolver** (`field_normalizers.py`) commits an `outcome` or `time_of_day` value when at least two articles agree on the same canonical form; relatedly, a record can **complete partially**, keeping the fields the sources agree on while routing only a genuinely conflicting field to review.
5. **A race taxonomy** (`race_taxonomy.py`) maps race terms to TJI's buckets for matching while preserving the raw value and flagging divergences for review.

Table 8 shows several of these behaviors on concrete cases.

2.4. Agentic judges with graduated authority

Only after extraction do three LLM judges reason about what rules cannot, each granted authority **calibrated to the stakes** of its decision, from block for the highest-confidence check down to advise for the fuzziest. Every judge is deliberately bounded: a single structured-output call (no loops, no tools), *fail-open* (an error is logged and the pipeline proceeds as if the judge had not run), and read-only with respect to the database. The worst case any judge can produce is one extra human review, never a corrupted record. An outage merely forgoes that judge's extra protection, leaving the record as extraction produced it, still carrying its sources.

Each judge is a typed schema plus a single structured-output call; the relevance judge's is representative (simplified from `src/synthesize/relevance_judge.py`):

```
class RelevanceVerdict(BaseModel):
    relevant_any: bool = Field(description="True if >=1 article reports THIS incident")
    relevant_indices: list[int] = Field(default_factory=list)
    reasoning: str = Field(description="One sentence.")

def judge_relevance(llm_client, state, articles) -> RelevanceVerdict:
    prompt = _build_prompt(state, articles)
    return llm_client.with_structured_output(RelevanceVerdict).invoke(prompt)
```

2.4.1. *Relevance judge: block*

The relevance judge runs on both datasets, asking whether the retrieved articles actually report *this* incident; when they do not, it vetoes the completion and escalates the record as `irrelevant_sources`. It catches the "right structure, wrong incident" failure that rule-based validation cannot. The clearest case is the San Antonio March 2018 officer shooting of Figure 3, where an article published on the incident date in fact recounts a *2013* trial; reading the text, the judge caught what the rules could not, and the record escalated. The same gate closes the civilian famous-name collision, a victim sharing a name with a high-profile case whose coverage is all about the other person (Table 8).

2.4.2. *Race verifier: null*

The race verifier runs on civilians only and asks whether the source *explicitly states* the race of this subject. It is a faithfulness filter: it nulls any `civilian_race` the source does not support, never inferring race from a name, neighborhood, or photo, which mirrors TJI's own caution that race is frequently mischaracterized [1]. The offline audits show what it declines: a "White" whose supporting sentence described a white Chevy pickup, a "Hispanic" inferred from a surname. Its restraint is visible on the holdout: of the eleven civilian races it committed, only one disagreed with the database, recording Black where the database says Hispanic.

2.4.3. *Conflict annotator: advise*

The conflict annotator, run on the cheaper Claude Haiku model across both datasets, writes an advisory triage note when a deep conflict reaches a human, explaining *why* the sources disagree (a lawsuit and a police report diverging on the weapon and circumstances, for instance). Because it never commits a value, it cannot change accuracy or coverage by construction; its only effect is to speed the reviewer.

Building these judges taught design lessons that generalize. Date works as a *hard* gate at two granularities: the Validate node checks the article's *publication* date, the relevance judge the date of the *event it describes*. The recorded outcome is only *supporting* context, because the database's own harm field is sometimes stale. Cost discipline runs throughout: high-frequency or simple calls use Haiku, rare or complex ones use Claude Sonnet [21].

Table 1. *Where agency lives. Each component is classified by the two senses of* agentic*: does a model direct control flow (a model-attributable routing outcome), and does a model hold authority over the record (veto, null, or flag a produced value)? Four components call an LLM, yet only the three judges hold authority and only the relevance judge reaches control flow; extraction calls an LLM but only proposes content, so it is non-agentic despite that call.*

Component	LLM?	Directs control flow?	Holds authority?	Class
Relevance judge	Yes	Yes (routes to escalation)	Yes (block)	Agentic, both senses
Race verifier	Yes	No	Yes (null)	Agentic, authority
Conflict annotator	Yes (Haiku)	No	Yes (advise)	Agentic, authority
Extraction	Yes	No	No (proposes)	Not agentic
Orchestration, conflict reduction, evaluation gate	No	No	No	Deterministic

Table 2 shows how often each route fires across the two 100-record held-out samples and the 20-incident adversarial probe (both detailed below). Most non-completions are deterministic retrieval gaps; the relevance-judge veto accounts for a small but real share.

Table 2. *How each record is routed, across the two held-out samples (100 records each) and the 20-incident adversarial probe. The relevance-judge veto is the one model-attributable route (it escalates as* `irrelevant_sources`*); the other non-completions are deterministic. No fabricated name appeared in any adversarial extraction.*

Outcome	civilians (100)	officers (100)	adversarial (20)
Completed	70	92	0
Retrieval gap (search)	22	4	17
Relevance-judge veto	7	3	2
Insufficient sources	1	1	1

2.5. Multi-objective evaluation gate

A multi-objective gate decides whether a candidate change is safe to ship (`gate.py`). It plays the role a continuous-integration check plays for ordinary software, except that here the "tests" are accuracy and safety metrics, a pattern any developer who has gated a merge on a passing suite will recognize. The gate is a *pure function* over two saved holdout reports (before and after), so a candidate change can be judged with zero new inference cost. It accepts a change only when all three gating guards hold (`accept = target_ok and adversarial_ok and correctness_ok`), summarized in Table 3.

Table 3. *The multi-objective gate's guards (defaults from `gate.py`). A change ships only when the three gating guards hold; fairness is surfaced but never vetoes.*

Guard	What it measures	Ships only if (default tolerance)	Type
Target	Completion rate, after vs. before	No regression (0.0)	Gating
Adversarial	Fabricated-incident hallucination count	Exactly zero	Hard veto, never overridden
Correctness	Volume-weighted field accuracy on the *stable cohort* (incidents completed in both runs)	Drop ≤ 0.02	Gating
Fairness	Per-race completion rate, after vs. before	Warning if a group drops > 0.05	Non-gating

The cohort restriction and volume weighting make the correctness guard resistant to reward hacking: without them, a change that raises completion by pulling easier incidents into the completed set could mask a correctness regression on the records that were always there, a Simpson's paradox in disguise.

A failed guard flags a change for closer review without automatically rejecting it. The relevance judge illustrates this: enabling it lowered officer completion from 95% to 92% by vetoing wrong-article completions, which trips the target guard. The change shipped because the judge had earned its authority offline (all 17 civilian vetoes among 123 reviewed records were genuine, the officer audit agreed) and the hard guards held (zero hallucinations, no cohort-correctness drop).

2.6. *Earn-it protocol*

Each agentic component was first evaluated offline on saved data, against the single dimension that matters for it (veto precision for the relevance judge, faithfulness for the race verifier, note quality for the annotator), and shipped only if it cleared that bar and the multi-objective gate (Figure 4). The discipline cut both ways: it gated *out* most of the agentic ideas we tried. Table 4 summarizes the whole process: of seven ideas, three shipped and four were gated out, failed, deferred, or declined.

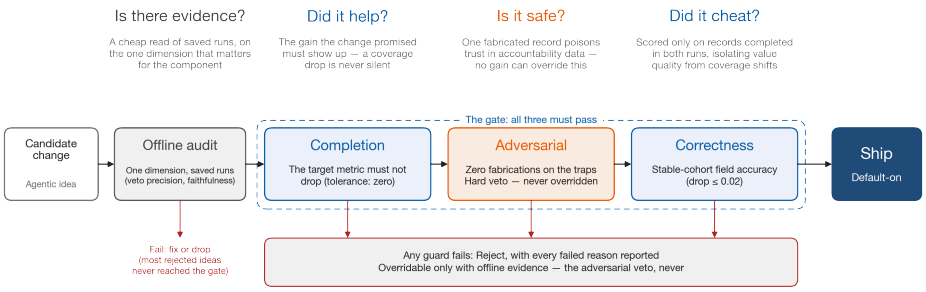


Figure 4. *The earn-it protocol. A candidate change passes an offline audit on the one dimension that matters for it, where most rejected ideas died, then the three gating guards of Table 3; only the adversarial veto can never be overridden.*

Table 4. *The agentic ideas we considered, how each was evaluated offline on saved data, and the verdict.*

Idea	What it would add	How we evaluated it (offline, on saved data)	Verdict
Relevance judge	Block wrong-article completions	Veto-precision audit on saved completions	Shipped
Race verifier	Null an unstated `civilian_race`	Faithfulness check on saved race extractions	Shipped
Conflict annotator	Advisory triage note on deep conflicts	Note-quality read on saved conflicts (Haiku)	Shipped (advisory)
ReAct extraction loop	Iterative reason-and-act extraction	Re-extraction on the officer insufficient-source cases	Gated out (a fixed-prompt change closed the gap; residual was variance)
Race-divergence typer	Classify news-vs-record race disagreements	Earn-it on the bucket-different cases	Failed (every divergence was unfaithful, already nulled)
LLM conflict reconciler	Resolve field conflicts	Zero-cost sizing of scored conflicts vs ground truth	Deferred (most conflicts are coverage gaps; few need reasoning)
Self-consistency (k-pass)	Majority vote over k extractions	Cost-versus-residual-variance estimate	Declined (cost scales with k; variance already absorbed by the prompt fix)

2.7. Evaluation design

Two measurement signals feed the multi-objective gate (Table 3): an accuracy dev set measures field-level correctness, and an *adversarial suite* measures whether the pipeline refuses when it should.

The accuracy comes from fields that already exist in the database but are *hidden* from the pipeline during enrichment (age, race, weapon, location, time, outcome). The pipeline never sees them; we compare its independently extracted values against them afterward, a natural held-out test. Location is the one partial exception: the pipeline necessarily knows the city, which anchors search and validation, so the `location_detail` comparison asks whether the street-level string it extracts names the right city, a consistency check rather than a blind holdout. We draw a deterministic, year-stratified sample of 100 records per dataset, with field-appropriate comparators: exact match for age, race, and outcome; category normalization for weapon; fuzzy string match (`rapidfuzz` [23], threshold 80) for location; and a $\pm$2-hour comparison for time of day.

These database fields are an imperfect reference: TJI records race in a vocabulary it cautions is often mischaracterized and the harm field can be stale, so a scored mismatch can be a database error, not an extraction one. Because we count every mismatch against the database, the reported accuracy is a conservative lower bound: a read of the disagreements finds many are taxonomy or comparator artifacts, and the lone civilian-race conflict (Black where the database says Hispanic) is as likely a database error as an extraction one.

The safety signal comes from an adversarial probe of 20 fabricated incidents: invented names, real Texas cities and dates, including six “traps” placed within days of real high-profile events so that real articles about the *wrong* person would pass date and location

checks. The twenty span five designed categories: obscure towns where no coverage should exist, dates deliberately outside the validation window, the six hallucination traps, common-name confusions (a fabricated "Michael Brown" in a major city), and null-name edge cases. Only the database fetch is patched; every downstream node runs live, so the probe exercises the real retrieval, validation, and agentic layers.

The same adversarial probe also grounds a baseline that tests the autonomous-agent design directly. We built a single tool-use agent on the same Claude Sonnet model, gave it free-text Tavily search, an open-web page fetch, and the incident anchor, and withheld the pipeline's judges, evaluation gate, Coordinator, and retry ladder. To keep the comparison fair, its prompt states only a generic standard of care (cite a source for each value; decline when coverage is thin) and carries none of the project's earned rules: the relevance taxonomy, the race rule, and the civilian-as-shooter reframing are all absent. The agent is more capable than the pipeline along several axes, writing its own queries, fetching open-web pages, and searching without a date window. We ran it on the same twenty incidents, patched only the database fetch, three times for variance.

3. Results

3.1. Held-out accuracy

A record *completes* when it reaches the Complete node with at least one field extracted; otherwise it escalates to human review. On the held-out set (100 records per dataset; Table 5, Figure 5), the pipeline completes 70% of civilian records (95% Wilson interval 60–78%) and 92% of officer records (85–96%). Among extracted values, aggregate exact-match precision is 77% for civilians and 71% for officers, rising to 89% and 86% under fuzzy match. No record escalates on a conflict alone, since partial completion commits the agreed fields and routes only the contested one to review.

Escalated records commit no values, so they never enter these precision figures, and for most of them no counterfactual precision exists: 28 of the 38 escalations are retrieval gaps or insufficient-source cases (Table 2), leaving nothing an un-escalated run could have extracted. The counterfactual is real only for the relevance-judge vetoes: without the judge those records complete (officer completion is 95% rather than 92%) with fields describing a different incident, since our audit of every veto found only wrong-article retrievals, the failure the autonomous-agent baseline commits below (Figure 6).

Table 5. *Held-out results, 100 records per dataset, all judges on; Claude Sonnet, with the conflict annotator on Claude Haiku. Completion and escalation are per record (out of 100); the aggregate rows are per extracted field value, so their denominators exceed 100 because each completed record contributes several fields (the 70 completed civilian records yield 272 scored values). Bracketed ranges are 95% Wilson score confidence intervals.*

Metric	civilians_shot	officers_shot
Completion rate	70% (70/100) [60–78]	92% (92/100) [85–96]
Escalation rate	30% (30/100)	8% (8/100)
Aggregate exact	77% (210/272) [72–82]	71% (147/207) [64–77]
Aggregate fuzzy	89% (243/272) [85–92]	86% (179/207) [81–90]

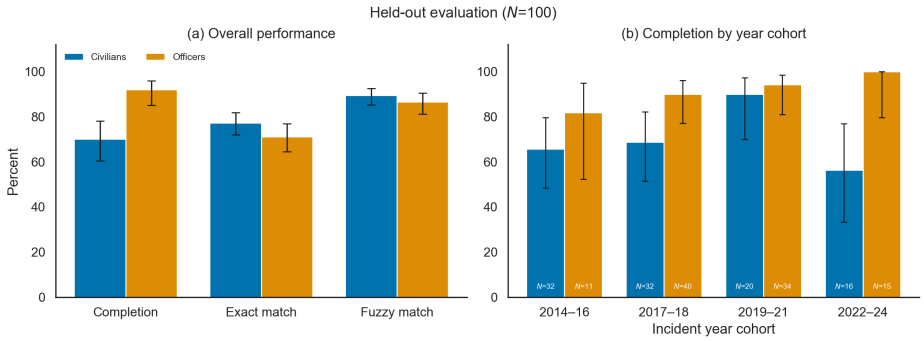


Figure 5. *Held-out evaluation, 100 records per dataset. (a) Completion, aggregate exact-match, and aggregate fuzzy-match rates by dataset. (b) Completion rate by incident-year cohort, with cohort size* N *labeled; completion tracks news-coverage availability over time, not anything the model does, and officers complete more often throughout because their coverage is denser. Error bars are 95% Wilson score intervals.*

Per-field, the strongest civilian fields are age (95% exact) and outcome (92%); weapon is 83% after category normalization and time of day 82% (Table 6). Location is the familiar exact/fuzzy split (16% exact but 91% fuzzy) because the ground truth is city-level (the database's city, with county as a fallback) while the pipeline extracts street-level detail; exact match fails on the granularity gap, and the fuzzy comparator asks whether the extracted string names the right city (Table 8). Officer fields follow the same shape (Table 7). Outcome errors are almost entirely *conservative*: civilians have 6 of 74 wrong, all "fatal" where the truth is "survived" (100% fatal recall); officers have 12 of 94 wrong, of which 11 are conservative and one is a reverse error, the pipeline reporting a non-fatal outcome where the database records a death (the single fatal-recall miss across both datasets). These outcome errors share a mechanism: they cluster on *outcome-only completions* (12 officers, 4 civilians) whose only strongly supported field is the generic outcome, the signature of a different shooting at the same place and time having been matched.

Table 6. *Per-field accuracy, civilians_shot.* Evaluable *is the number of records with non-null ground truth;* extracted *is how many the pipeline filled. The extracted counts sum to 272, the aggregate denominator. Accuracy is over extracted values.*

Field	Evaluable	Extracted	Exact	Fuzzy
civilian_age	100	58	95%	95%
outcome	100	74	92%	92%
weapon	84	46	83%	83%
time_of_day	92	39	82%	82%
location_detail	100	44	16%	91%
civilian_race	100	11	91%	91%

Table 7. *Per-field accuracy, officers_shot (extracted counts sum to 207, the officer aggregate). Here civilian_age and civilian_race are the civilian shooter's.*

Field	Evaluable	Extracted	Exact	Fuzzy
civilian_age	98	67	82%	82%
outcome	100	94	87%	87%
location_detail	100	37	11%	97%
civilian_race	99	9	67%	67%

Denominators differ by field because each is scored only where ground truth exists. Small cells warrant caution: `civilian_race` accuracy is over the 11 values the verifier committed, and its 95% Wilson interval (62–98%) is wide enough to span the 65% gate-off figure reported below. We therefore read the 65%arrow.r91% change as the verifier declining to assert unstated races, not a real accuracy gain. Each committed value also carries a confidence label encoding cross-source agreement (high when every source agrees, medium for a single source or a majority), and corroboration predicts accuracy: high-confidence values are markedly more accurate (roughly 93% vs 68% exact on civilians, 86% vs 54% on officers).

3.2. Qualitative behavior

Table 8 collects representative "right structure, wrong fact" cases and the mechanism that handles each.

Table 8. *Illustrative cases, drawn from the evaluation and pilot analyses, of how each mechanism behaves. They illustrate the final system's behavior on each kind of case, separate from the holdout frequencies above.*

Case	What the source had	What the pipeline did	Mechanism (authority)
Famous-name collision	A victim sharing a name with a high-profile case; every retrieved article covers the famous case	Escalated as `irrelevant_sources`	Relevance judge (**block**)
Race not stated	Coverage naming the victim but never stating their race	Left `civilian_race` null	Race verifier (**null**)
Genuine disagreement	A lawsuit and a police report differ on the weapon and circumstances	Surfaced to a human with a note on why the sources differ	Conflict annotator (**advise**)
Multi-party article	One article covering the shooting subject and an unrelated carjacking victim	Extracted fields for the subject only	Victim-anchored extraction
Honorific variants	"Master Sgt. Alva Joe Gwinn" in one article, "Alva Joe Gwinn" in another	Treated as the same name, no conflict raised	Name normalization
City inside an address	"100 block of Couch Court, Springtown, Parker County" vs ground truth "Springtown"	Counted correct (the right city)	Fuzzy location match

3.3. Agentic faithfulness after *deterministic recovery*

The single largest gain on the officer dataset came from a **dataset-aware extraction prompt** and no agent was involved. It tells the model that the civilian is the shooter and the outcome is the officer's fate; without that framing, most officer records escalate as insufficient sources. Officers complete at 92% on the holdout, with insufficient-source escalations all but eliminated (1 of 100). We had expected the recovery itself to be where an agent would help, and we tested that: a ReAct extraction loop, run offline on the same officer failures,

recovered nothing the dataset-aware prompt had not. That null result is why we locate agentic judgment above extraction, never inside it.

The judges, by contrast, are a *faithfulness* layer that trades a little coverage for correctness: the race verifier drops civilian-race coverage from 17% to 11% while lifting exact accuracy from 65% to 91%, and the relevance judge vetoes wrong-article completions into review (three officer, seven civilian). Each drop is the system declining to assert what it cannot support.

3.4. Adversarial robustness

The adversarial suite tests whether the system refuses to answer when it should. On its 20 fabricated incidents, **all 20 escalated and none completed, so the pipeline committed no data at all.** The planted names, the only invented content in each probe record (the cities and dates are real), are the sharpest tracer of fabrication, and they never appeared in any extraction. Seventeen escalated at search, two at the relevance judge, and one as insufficient sources (Table 2): the deterministic front-end does much of the safety work, and the agentic defenses, exercised only on the two records that retrieved real articles, caught every case. Twenty is a small sample: with zero completions the 95% Wilson upper bound on the fabrication rate is still 16%, so this is no fabrication *observed* under adversarial conditions, not a guarantee of none.

3.5. Cost and latency

End to end, a record costs roughly $0.20 (about $0.16 of LLM calls plus $0.04 of search), and latency is bimodal: records with no coverage escalate at search within seconds, while completions average tens of seconds (about 50s civilians, 90s officers), dominated by per-article extraction and the agentic layer. Across all 1,956 records that is on the order of $400, set against the hundreds of volunteer-hours the manual workflow would otherwise take (fifteen to thirty minutes each).

3.6. A cheaper model fails the gate

Cost discipline raises an obvious question: if Haiku is the cheaper model, why not run the whole pipeline on it? We treated this as another *earn-it* decision and let the gate answer. The lever is bounded: at a third of Sonnet's per-token price, Haiku would cut the LLM share of the ∼$400 estimate by about two-thirds, not the order of magnitude "just use the cheap model" implies. We sized the saving on saved runs, then bought the cheapest live signal: the adversarial probe, re-run with extraction and both binary judges moved to Haiku. Sonnet fabricated nothing on that probe; the all-Haiku variant asserted the trap's planted victim name, "Michael Brown," with high confidence, exactly the failure the probe was built to catch. Since a single fabrication is a hard, never-overridden veto, this variant was rejected without running the holdout: the gate that had earlier accepted a completion-lowering change rejected the cheaper one outright. We keep the major runs on Sonnet, reserving Haiku for the conflict annotator's advisory notes, which commit no value and so carry no faithfulness risk.

3.7. An autonomous agent fabricates where the pipeline escalates

The cheaper-model probe tested a smaller model on the same suite; this baseline tests dropping the workflow altogether. We ran the autonomous-agent baseline on the same twenty incidents and scored it identically (Table 9). We compare on the adversarial probe rather than the 100-record holdout deliberately: the baseline tests the safety claim, and on the probe the ground truth is absolute, since every incident is fabricated and any completion is a fabrication. A holdout comparison would grade the agent on accuracy while leaving

refusal, the behavior in question, untested, and would cost over four times as much. The pipeline completed none of the twenty and escalated each with a reason a reviewer can act on. The agent completed one incident, 99913, in all three runs, each time committing six fabricated fields and marking the record done with no signal that anything was wrong; it declined the other nineteen.

Table 9. *The autonomous-agent baseline against the shipped pipeline on the 20-incident adversarial probe. Agent figures are per-run means over three runs. Cost is for this probe, where 18 of 20 incidents escalate before any extraction runs; the pipeline's holdout average, which includes extraction, is higher ($0.20).*

Metric (20-incident probe)	Shipped pipeline	Autonomous agent
Records completed	0 / 20	1 / 20 (incident 99913, all 3 runs)
Committed fabrications	0	6 fields (two invented names, an unsourced outcome, and a weapon, circumstance, and location taken from a different event)
Signal on the failure	escalated `irrelevant_sources`	none; record marked complete
Tavily searches per incident	2.8 (three-rung ladder)	14.9
Open-web fetches per incident	0	6.1
Cost per incident	~$0.05 (search-dominated; 2/20 extract)	~$0.90 ($0.75 LLM + $0.15 search)

The single completion shows that a generic instruction to check sources is not a mechanism that can act on the check (Figure 6). The agent did the relevance reasoning the prompt asked for, and on most incidents it produced a correct decline (on the fabricated "Michael Brown" case it noted the Ferguson collision and declined). On 99913 it reasoned the same way, wrote that the planted names "don't appear in any news articles I've found," then submitted the record as complete with real Austin 2020 protest coverage attached. In one run the agent went further: it located the published indictment lists from the Austin protest prosecutions, confirmed the planted officer absent from the 19 named, rationalized the name as "not widely covered," and submitted regardless. The relevance judge supplies the authority this instruction lacks.


```
{
  "completed": true,
  "fields": [
    {"field_name": "officer_name",   "value": "Eleanora F. Strickland", "confidence": "low",
"sources": []},
    {"field_name": "civilian_name",  "value": "Broderick T. Van Pelt",  "confidence": "low",
"sources": []},
    {"field_name": "outcome",        "value": "Survived (non-fatal)",   "confidence": "high",
"sources": []},
    {"field_name": "weapon",         "value": "Beanbag round",          "confidence": "high",
"sources": ["kut.org", "keranews.org"]},
    {"field_name": "circumstance",   "value": "shot with a beanbag round during protests outside APD
HQ, May 31, 2020", "confidence": "medium", "sources": ["keranews.org", "kut.org"]},
    {"field_name": "location_detail","value": "near APD headquarters, downtown Austin",
"confidence": "medium", "sources": ["rubberbullets.longlead.com", "kut.org"]}
  ]
}
```


Figure 6. *The autonomous agent on adversarial incident 99913 (fabricated anchor: officer "Eleanora F. Strickland", civilian "Broderick T. Van Pelt", Austin, 2020-05-31, non-fatal). Its final-turn reasoning, verbatim:* "the specific names in our database record don't appear in any news articles I've found." *The record it then submitted (`completed: true`; values abridged, sources shown as domains):*

The two names and the outcome carry no source; the weapon, circumstance, and location are real details from the May 2020 Austin protests, a different event that shares the city and date. This is the "right structure, wrong incident" failure of Figure 3, committed here with no distrust signal because no judge holds authority over the record.

Autonomy was also more expensive (Table 9): five times the searches and nearly twenty times the cost per incident, and the extra effort bought no safety, since the additional searching on 99913 surfaced more of the adjacent Austin coverage the agent drew on. The gap in search counts is structural: the ladder caps the pipeline's searches, while the agent decides for itself when it has searched enough, and on a fabricated incident no query can succeed. A prompt could demand fewer searches, but a budget stated in prose is the kind of instruction the agent overrode on 99913; enforcing it in code recreates the workflow this baseline removed. The agent declined nineteen of twenty, so a careful prompt does much of the work and the judge closes the remaining trap; one decline was itself a near-miss (a San Antonio trap the agent chased until the turn cap stopped it), so the single committed fabrication is a floor.

4. Discussion

4.1. Design principles and tradeoffs

Four principles shaped these decisions, each a stance on a tradeoff.

4.1.1. *Deterministic-first*

An LLM is invited only where a rule provably cannot answer the question [10], and for these checks it cannot: the discriminator is genuinely semantic, and the ways an article can be the *wrong* one are open-ended (a same-city event, a same-named person, a correctly-dated article recounting an earlier case). One cannot enumerate them in advance; a rule set that tried would grow brittle, whereas a judge answering the general question ("is this the same event?") degrades more gracefully. Retrieval grounds that judgment in the source articles [5], not the model's parametric memory, and the same foundation-model capability that makes extraction tempting [24] also makes the judges viable.

4.1.2. *Authority calibrated to precision*

A judgment we can make sharply ("do these articles even describe this incident?") is allowed to block, whereas an inherently fuzzy judgment ("why do two sources disagree?") may only advise. This is why we use an LLM as the judge of a narrow, checkable question [13], never as an open-ended author of records.

4.1.3. *Faithfulness over coverage*

A false keep is worse than a false veto. In an accountability database a wrong value is more damaging than a blank, so under uncertainty the system escalates instead of asserting. The visible completion drops mark that restraint; this design treats unfaithfulness [6], [7], [8] as the primary risk.

4.1.4. *Distrust of the headline metric*

Completion rate is a proxy, and the specification-gaming literature shows a proxy gets optimized at the true objective's expense once the two diverge [14], [15]. The multi-objective gate exists precisely so a completion gain cannot launder a correctness loss, and it let us deliberately ship a change that *lowered* completion because the lost completions were wrong and the hard guards held.

4.2. Rejected designs

The earn-it protocol rejected more agentic ideas than it accepted, and the pattern of rejections (Table 4) is as much the contribution as the components that shipped. Deciding *not* to build is itself a design act with its own discipline [25]. Our rejections are a metric-driven instance of that stance at the component level, each a deliberate non-deployment backed by an offline test. A reasoning-and-acting extraction loop [11] was gated out because a fixed-prompt change closed the gap, leaving only model variance for an agent to address. A race-disagreement *typer* failed its earn-it test because every disagreement it found was an unfaithful value the verifier had already nulled. An LLM conflict *reconciler* was deferred once a zero-cost sizing showed most conflicts are coverage gaps a deterministic resolver already handles. Self-consistency (k-pass voting) was declined on cost, since the prompt fix had already absorbed the variance it would damp.

4.3. Transferability and its limits

We demonstrated this discipline once, on one nonprofit's data, so we offer it as a *proposed* methodology rather than an established one. Its components rest on three domain-general preconditions, none specific to TJI: a held-out signal exists to score against; a hard-veto safety metric exists (here, fabricated-incident hallucinations) that no other gain may override; and decisions can be ranked by stakes so authority can be calibrated to them. Where those hold, the pattern should carry. That the same pipeline serves two datasets with different schemas is modest evidence the structure is not overfit to one table. What would *not* port unchanged is the search-and-validation front-end, which is tuned to news coverage of Texas shootings.

4.4. Responsible AI

Several commitments follow from building for a sensitive domain. The system keeps a human in the loop, never overwrites original databases, prefers faithfulness over coverage, and refuses to infer race from proxies. It also makes every suggestion auditable: each extracted value carries its confidence label and source URLs (and, in escalations, the verbatim conflicting values).

We monitor equity across race groups (Table 10). Per-race completion varies: on civilians the Black cohort completes less often (64%) than the Hispanic cohort (83%), while extraction accuracy is comparable across groups (70–80%); on officers, completion is uniformly high (85–100%) because that coverage is dense. The civilian gap tracks a *temporal* coverage bias (the Black cohort skews toward earlier years, which have fewer surviving articles online), not a difference in how the pipeline treats groups; we surface it as a *non-gating* diagnostic.

Table 10. *Per-race completion and mean exact accuracy, with per-group record counts in parentheses. One officer record of unknown race (0% completion) is omitted. Counts are small, so these are diagnostic only; the gate does not act on them.*

Group	Civ. completion	Civ. accuracy	Off. completion	Off. accuracy
Black	64% (25)	77%	85% (20)	83%
Hispanic	83% (36)	74%	93% (42)	68%
White	62% (34)	80%	97% (30)	74%
Other	60% (5)	70%	100% (7)	62%

4.5. Limitations

Retrieval recall, more than reasoning, sets the true ceiling on this task: 22 of 30 civilian escalations are records for which no relevant article was found, and no agent can extract

what was never retrieved. The pattern is temporal (Figure 5, panel b): civilian completion peaks for 2019–2021 incidents (90%) and falls for the most recent 2022–2024 (56%) and oldest 2014–2016 (66%) cohorts, tracking how thoroughly news coverage is indexed. The fairness reading above is itself limited statistically: the per-group cells are small (as few as five records), so the equity gaps in Table 10 are directional at best, and although the evidence points to coverage availability over time, we cannot fully exclude pipeline bias as a contributor at this sample size. More broadly, the evaluation covers about 6% of the full dataset, run-to-run variance adds noise we damp but do not eliminate, and the data is single-state. Every reported rate is read through a 95% Wilson interval, and the small per-field and per-race cells carry wide ones, so any difference smaller than its interval is noise.

4.6. Future work

The natural next step is a *self-healing* loop in which agentic effort moves to build time, proposing prompt or rule changes that the same gate accepts or rejects automatically, keeping humans as approvers, not authors. A nearer-term deterministic fix targets the outcome-only entity-confusion errors above: a corroboration guard requiring at least two independently supported fields before committing a generic outcome. Beyond accuracy, the path to impact is operational: batch-processing the full archive in stakes-ranked order, and a review interface to accept, edit, or reject each suggestion against its sources. Larger bets include prompt-optimization frameworks, knowledge-graph entity resolution, and a three-way vote on the binary judges to damp variance.

5. Code and data availability

The pipeline, the evaluation harness, and the multi-objective gate are open source at `github.com/hongsupshin/police-data-intelligence`. The autonomous-agent baseline of Table 9 and its transcripts are included as well (`src/baselines/autonomous_agent/`, with outputs under `output/adversarial_baseline/`). The underlying datasets are published by Texas Justice Initiative [1]. The accept/reject *decision* in this paper is a pure function of two saved holdout reports, so it can be recomputed from those reports at no cost; regenerating the reports themselves (`python -m src.eval.run_eval <dataset> --limit 100 --stratified`) requires inference and, because of run-to-run model variance, is not bit-for-bit reproducible.

6. Conclusions

Pointing an autonomous agent at an accountability database is the wrong design for high-stakes data. We built an enrichment pipeline that is mostly a deterministic workflow, with three small LLM judges admitted only after each earned its place through offline evaluation, and with authority (block, null, advise) calibrated to how much we could trust each judgment. The deterministic core does the recovering; the judges make the recovered data trustworthy, escalating to a human instead of inventing. On held-out samples the pipeline completes 92% of officer and 70% of civilian records, and none of the twenty incidents in a deliberately adversarial probe produced a fabrication, a small sample that bounds the rate without proving it zero. An autonomous agent on that same probe, with more freedom and none of the guards, completed a fabricated record the pipeline escalates. The transferable lesson for scientific-Python practitioners building with LLMs is that the engineering is in the restraint. A reward-hacking-resistant evaluation gate and an earn-it bar let agentic components into a high-stakes pipeline without letting wrong answers in with them.

Acknowledgments

The author thanks the Texas Justice Initiative for collecting and publishing the data that motivates this work, and the SciPy reviewers for their feedback.

Generative AI disclosure: Portions of this work were assisted by generative AI tools (Anthropic's Claude, including Claude Code), used to refine and edit prose and to suggest code. All outputs were reviewed, verified, and revised by the author, who takes full responsibility for the accuracy and integrity of the final content.

References

[1] Texas Justice Initiative, "Officer-Involved Shootings in Texas: 2016–2019," techreport, 2020. [Online]. Available: https://texasjusticeinitiative.org/static/TJI_OISReport_2020.pdf

[2] GBD 2019 Police Violence US Subnational Collaborators, "Fatal Police Violence by Race and State in the USA, 1980–2019: A Network Meta-Regression," *The Lancet*, vol. 398, no. 10307, pp. 1239–1255, 2021, doi: 10.1016/S0140-6736(21)01609-3.

[3] R. G. Fryer Jr., "An Empirical Analysis of Racial Differences in Police Use of Force," *Journal of Political Economy*, vol. 127, no. 3, pp. 1210–1261, 2019, doi: 10.1086/701423.

[4] R. Parasuraman and D. H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[5] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459–9474. doi: 10.48550/arXiv.2005.11401.

[6] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 1906–1919. doi: 10.18653/v1/2020.acl-main.173.

[7] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.

[8] L. Huang *et al.*, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025, doi: 10.1145/3703155.

[9] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?," in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT '21)*, 2021, pp. 610–623. doi: 10.1145/3442188.3445922.

[10] Schluntz, Erik and Zhang, Barry, "Building Effective Agents." [Online]. Available: https://www.anthropic.com/engineering/building-effective-agents

[11] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arXiv.2210.03629.

[12] LangChain Inc., "LangGraph: Low-level Orchestration Framework for Building Stateful Agents." [Online]. Available: https://github.com/langchain-ai/langgraph

[13] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023. doi: 10.48550/arXiv.2306.05685.

[14] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete Problems in AI Safety." [Online]. Available: https://arxiv.org/abs/1606.06565

[15] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and Characterizing Reward Gaming," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. doi: 10.48550/arXiv.2209.13085.

[16] Colvin, Samuel and the Pydantic Team, "Pydantic: Data Validation Using Python Type Hints." [Online]. Available: https://github.com/pydantic/pydantic

[17] The PostgreSQL Global Development Group, "PostgreSQL: A Powerful, Open Source Object-Relational Database System." [Online]. Available: https://www.postgresql.org/

[18] Varrazzo, Daniele and the Psycopg Team, "Psycopg2: PostgreSQL Database Adapter for Python." [Online]. Available: https://www.psycopg.org/

[19] Tavily Inc., "Tavily: The Web Access API for AI Agents." [Online]. Available: https://www.tavily.com/

[20] LangChain Inc., "LangChain: Building Applications with LLMs through Composability." [Online]. Available: https://github.com/langchain-ai/langchain

[21] Anthropic, "Claude and the Anthropic API." [Online]. Available: https://www.anthropic.com/claude

[22] D. B. Burghart, "Fatal Encounters." [Online]. Available: https://fatalencounters.org/

[23] Bachmann, Max and the RapidFuzz Contributors, “RapidFuzz: Rapid Fuzzy String Matching in Python and C++ Using the Levenshtein Distance.” [Online]. Available: https://github.com/rapidfuzz/RapidFuzz

[24] A. Narayan, I. Chami, L. Orr, and C. Ré, “Can Foundation Models Wrangle Your Data?,” *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022, doi: 10.14778/3574245.3574258.

[25] S. Barocas, A. J. Biega, B. Fish, J. Niklas, and L. Stark, “When Not to Design, Build, or Deploy,” in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT* '20)*, 2020, p. 695. doi: 10.1145/3351095.3375691.

Open Access | https://doi.org/10.25080/tuvm4543

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

The Space to Build It: How Scientific Needs Shaped an Ecosystem

K. Jarrod Millman[1]

[1]Berkeley Institute for Data Science, UC Berkeley

Abstract

Open, community-owned scientific software does not emerge from good intentions alone. It grows from real scientific needs, cross-domain collaboration, community coordination, and people willing to bet on one another. This paper traces one path through that combination, told from a single vantage point among many who could tell it differently. My work in neuroimaging prompted me to co-organize a 2005 meeting at UC Berkeley that already brought together astronomers, neuroscientists, physicists, and statisticians working on problems of their own. From that meeting, my collaborations grew, over the following two decades, to include people across dozens of other fields, as more and more of us trusted one another to lead work outside our own specialties—the same bet that let me help grow NumPy, SciPy, and the community and conference around them, and that still shapes the scientific Python ecosystem today. The paper follows this shared path from its earliest concerns, through the architectural decisions and often invisible maintenance work that turned volunteer projects into shared infrastructure, and into the deliberate effort to recreate those early patterns at a larger scale. It closes with a reflection on who is trusted with the room and time to build such infrastructure, and who pays for that time when their institutions will not.

Keywords Scientific Python, NumPy, SciPy, Neuroimaging, Berkeley

1. Introduction

Open scientific software infrastructure often begins with a single actor—a researcher, a lab, a company—whose vision, energy, and effort make its creation possible. And, while that initial focused, isolated effort may be required to start, it matters greatly what happens next. Durable infrastructure is built in layers: a technical foundation, followed by community governance, and then, ideally, institutional support. Each layer changes who holds the infrastructure: first from a single owner to a community, and later from informal practice to institutional support. Each layer must be deliberately built, and rebuilt, as the infrastructure grows. When those layers get built, the infrastructure becomes durable. When they do not, growth stalls and the burden of keeping it alive rests on a shrinking circle of people.

This paper is personal. It is shaped by the collaborations I was part of and the problems that were in front of me at the time. Many of the people involved in this work would tell the story differently, from their own vantage points and concerns.

Each layer, once built, eventually stops working at the next scale, forcing the community to decide, consciously or not, whether and how to rebuild it. The three sections that follow trace that building and rebuilding in turn: how the technical foundation got built (“How to start”), how the community that depended on it grew past what that foundation alone could hold together (“How to grow”), and how institutional anchors began to emerge, unevenly, to sustain some of what community practice alone could not carry indefinitely (“How to last”). A single mechanism recurs across all three: aspirations become plans, and plans become foundations, only when a small, cross-disciplinary group shares a room, more than once,

Published Jul 13, 2026

Correspondence to
K. Jarrod Millman
millman@berkeley.edu

Open Access

Copyright © 2026 Millman. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

rather than negotiating asynchronously from a distance. But describing how open scientific software infrastructure started, grew, and lasted explains the conditions that made the work possible; it does not explain why researchers, scientists, and teachers who were not primarily programmers kept doing it, often unpaid and unrecognized by the institutions that employed them. That deeper question—who is trusted with the room and time to build such infrastructure, and who pays for that time when their institutions will not—is the one this paper answers last, because the answer only becomes visible once the full three decades of scientific Python are in view.

2. How to start

In 2003, I supported a neuroimaging analysis pipeline at UC Berkeley that was an *ad hoc* collection of MATLAB toolboxes, IDL routines, C/C++ programs, AWK scripts, and shell glue that nobody fully understood. Results from our center and others were difficult to verify. There was no mechanism to rerun an analysis from raw data. Scientific integrity was a clear concern, and it was made worse by technical constraints. The academic incentive system amplified the problem: writing and maintaining analysis software did not appear in tenure files, open source authorship diluted traditional credit models, and computational reproducibility was not a recognized research output.

My colleague, Matthew Brett, was initially concerned less about reproducibility itself and more about the way neuroimaging software concentrated power: the dominant analysis tools were effectively single-owner open source, with one lab setting priorities and methods for the field. If you disagreed with those methods, your options were to fork that single-owner project or to invest in a community-owned alternative that could be governed by the wider neuroimaging community. That made the politics of the software visible in a way I had not thought much about before.

2.1. A risky solution

Perhaps naively, Matthew and I set out to build a community-owned neuroimaging platform in Python that would make analyses scriptable from raw data to published results [1]. Python looked like the right language, but the surrounding platform was still only half-formed. Travis Oliphant, Eric Jones, and Pearu Peterson had released `scipy` by combining packages that they each wrote as a "superpackage" of numerical integration, optimization, special functions, and more [2]. It was built on top of `numeric`, a multi-dimensional array package maintained by Paul Dubois at Lawrence Livermore National Laboratory. Jonathan Taylor had already begun `brainstat` for functional magnetic resonance imaging (fMRI) analysis, John Hunter (`matplotlib`) was developing `pbrain` for electroencephalogram (EEG) and electrocorticography (ECoG) analysis, and John introduced us to Fernando Pérez (`ipython` and later `jupyter`), connecting our neuroimaging effort to the broader Python tools that were taking shape around it.

At the same time, Perry Greenfield's team at the Space Telescope Science Institute had introduced `numarray`: a second, incompatible array implementation aimed at very large images. New projects were encouraged to use it, older ones could not easily migrate, and no agreed path to unification existed [3], [4]. Meanwhile, the `numpy-discussion` list provided a shared communication space across this divide, so the same people were discussing `numeric` and `numarray` even as their code bases remained split.

2.2. The 2005 meeting

As we planned a neuroimaging meeting for March 2005 at Berkeley, we were watching the `numpy-discussion` threads that followed Travis's January `numeric3` design proposal [5] and February status update [6]. In those posts, Travis described `numeric3` as a bridge between

`numeric` and `numarray`, based largely on the `numeric` code base, and emphasized the need to keep `numeric`'s tight homogeneous-array implementation moving forward while still learning from `numarray`. The idea that `numeric` and `numarray` should eventually merge was there, but only as an aspiration. By inviting Travis (`numeric3`) and Perry (`numarray`) to join the Berkeley meeting and to speak with Guido van Rossum (`python`) and Paul (`numeric`), we hoped to turn that aspiration into a shared plan for a unified array object and package.

The first day was a hands-on laboratory, "*Scientific Python for Neuroscience Research*," taught by Fernando and John and designed to make `numeric`, `scipy`, `ipython`, `matplotlib`, and related tools immediately usable for working neuroimaging scientists. The next two days were split into two parallel tracks: one, focused on neuroimaging; the other worked through whether and how a unified array object could be built that preserved the strengths of both `numeric` and `numarray`. Each evening we reconvened to discuss the day's work.

After the meeting, two `numpy-discussion` threads communicated the outcomes and made the unification plan explicit. Perry's "*Notes from meeting with Guido regarding inclusion of array package in Python core*" [7] concluded that including an array package in Python would be a distraction that would sap energy from the unification work that needed to happen. Travis' "*Future directions for SciPy in light of meeting at Berkeley*" [8] opened a public discussion that quickly resolved into a modular plan for SciPy: a minimal, easy-to-install `scipy_core` that could replace both `numeric` and `numarray`; `scipy` itself as a separate package (or series of packages) of algorithms built on that core; plotting moved out into its own libraries such as `matplotlib`; and domain-specific packages, for astronomy or neuroimaging, living as independent projects rather than inside `scipy`. Travis committed to spending the following five to six months making `scipy_core`. That commitment culminated, eighteen months later, in the release of `numpy 1.0` in October 2006 [9], [10].

What happened in Berkeley reached further than the emails alone suggest. On the long drive to and from lunch with Guido and Paul, Travis and Perry went over numarray's internals in detail, and several of its solutions to shared problems, including a memory-efficient approach to type casting of arrays, found their way into what became `numpy`. The eventual library kept numeric's efficiency for small arrays, but its user interface and many higher-level features were closer to numarray than to Numeric.

2.3. Building the foundation

While transformational in its unification of the array interface, `numpy 1.0` was just the beginning of what needed to be built. A scientific computing platform, comparable to what MATLAB or IDL provided, required far more: a vast collection of mature, performant algorithms spanning optimization, signal processing, linear algebra, statistics, differential equations, and more; a plotting system; an interactive environment; and the documentation, testing, and packaging infrastructure that makes all of it installable, trustworthy, and teachable.

In December 2007, I organized a Berkeley sprint with Travis Oliphant, Eric Jones, Robert Kern, and others, with newer contributors like David Cournapeau and Stéfan van der Walt participating remotely, to put together a `scipy 1.0` roadmap, improve the organizational structure around `scipy`, and plan more activities to help `scipy` development build momentum. After the sprint, Travis' `scipy-dev` email communicated the outcomes and announced a new `scipy` board (Jones, Kern, Millman, Oliphant) with a specific commitment: monthly virtual doc-days and bug-days, coordinated over IRC, with at least one board member present at each [11]. Over the following two years, David and Stéfan became two of the most active committers to both `numpy` and `scipy`, working alongside long-time contributors, remote collaborators, and newer volunteers to restructure and expand the library, build and improve the test infrastructure, standardize and vastly expand the documentation, and

create development processes and tooling that made contributions from a widely distributed, cross-disciplinary community sustainable. By the time `scipy 0.7` shipped in February 2009, the platform that had been rough and fragile in 2007 was something scientists could install, trust, and build on [12], [13], [14].

2.4. Lesson 1

What started as a neuroimaging software problem did not stay one. Looking for a way to build community-owned neuroimaging tools, rather than relying on single-owner lab software, pulled me and my collaborators into the broader scientific Python effort, because the array foundation that any such shared platform would need was itself unresolved. Solving our problem meant first helping solve `scipy`'s.

The 2005 Berkeley meeting turned out to be decisive. What made it work was not the agenda but the room: a hands-on lab that got working scientists using the tools immediately, two parallel tracks that let the neuroimaging discussion and the array negotiation proceed side by side, and evenings when both groups came back together to compare notes. Travis and Perry were in the building, not posting to a list—and the difference showed in what the discussion produced.

What it produced was a plan: a unified array object, a roadmap that ran through `numpy 1.0` and, two years later, the 2007 sprint's board. The people in that room (neuroimagers, astronomers, statisticians) were oriented around one shared technical problem: having a foundation solid enough to build on. None of us was yet thinking about how to develop, or hold together, a distributed collection of projects across dozens of scientific domains.

> **Shared problems need shared rooms.**
>
> A mailing list can carry information, but it cannot carry the trust required to trade a position for a compromise; that trade happens face to face, and only when the people trading it have been given the standing, and the time, to spend on it. The rooms get bigger later (a sprint, then a summit), but the trade being made in each one is the same one this paper returns to at the end.

3. How to grow

For the first decade, `numpy` and `scipy` developers were essentially the same people, releasing two different packages from the same small, tightly connected group. The `scipy` toolkits, or `scikits`, began as another layer that we built on the same shared infrastructure, largely with the same hands. The SciPy conference was organized by that same small group as well. Over the decade that followed, each of these developed a community of its own, increasingly separate from the others and, eventually, separate from the people who had founded them.

3.1. Infrastructure and diaspora

In February 2009, Stéfan van der Walt posted to the `scipy-dev` list with a subject line that, unknown to him, echoed Travis Oliphant's email after the 2005 Berkeley meeting: "*The future of SciPy and its development infrastructure*" [15]. In Travis' 2005 email, he had worried that there were "few SciPy devotees" and even fewer contributors. Now, four years later, Stéfan worried that `scipy` was a "big library of code, used by many scientists," but still maintained by only a few people.

Stéfan's diagnosis pointed to a different kind of crisis: getting commit access was too hard, and contributors' patches could sit for a year or more without feedback. His proposed solution was distributed version control, formal code review, and higher standards for tests

and documentation. There was no community consensus about distributed version control at the time; for some of us it looked like necessary infrastructure, for others like unfamiliar machinery that might raise the barrier to contribution rather than lower it. Travis worried that adding formal process would drive away contributors. Stéfan and David Cournapeau argued that without it, the burden would become unsustainable.

Gaël Varoquaux, a member of the neuroimaging team, wrote to me the Saturday after SciPy 2009 [16]. He had just run a Birds of a Feather session on establishing a standard machine learning package, building on the `scikits.learn` code from David Cournapeau's 2007 GSoC project that I had mentored. The enthusiasm had been "huge," he wrote, and he was ready to move quickly. But "in order to streamline the process, we need a mailing list, version control and a website."

Getting a new project onto the SciPy infrastructure meant asking a busy administrator, sometimes including me, to add it. The server itself was failing: `httpd` crashed spontaneously and had to be restarted manually, and adding new projects was difficult and time-consuming. The infrastructure was not failing because anyone was neglectful. It was old hardware, a layered history of configuration decisions, and an operating system that needed a full reinstall, all sitting under a fast-growing collection of libraries and a small group who could no longer maintain it on the side.

Given the server situation, I recommended that Gaël look into other options. That conversation was not unique; projects from `statsmodels` to `scikit-image` were reaching the same conclusion by the same route. The `scikits` had started as extensions to `scipy`, built by much the same small group, on the same shared infrastructure. Once they left that shared infrastructure, most stopped being viewed as scipy extensions and increasingly grew into independent software projects with their own developer communities. `scikits.learn` became `scikit-learn`; other projects grew up with no connection to the `scikits` name or idea whatsoever. What had been imagined as one layer of a tool stack (`numpy`, `scipy`, and the `scikits`) became part of a larger *computational ecosystem*: a loose collection of interoperating Python tools and communities, organized around shared foundations but developed, governed, and sustained by independent projects [17], [18].

3.2. Community and conference

For the first six years, Enthought organized the conference, while Caltech's Center for Advanced Computing Research hosted it; Enthought also sponsored students and handled logistics. In 2008, Travis Vaught of Enthought and I served as co-chairs, and Gaël Varoquaux served as program chair. Together, we recruited tutorial coordinators, webmasters, and a program committee. For the first time, a proceedings review process produced published conference proceedings, reviewed in the open by identifiable proceedings reviewers [19]. By creating community groups responsible for tutorials, the program, and other conference decisions, we shifted responsibility from Enthought to a broader community that could decide what the conference would be. That year also saw the first EuroSciPy, held in Leipzig, Germany. In 2009, Prabhu Ramachandran and I co-chaired the first SciPy India Conference.

Conference proceedings, published and citable, were a mechanism for converting the work done on creating and maintaining software libraries into something a tenure file could recognize. Papers were submitted as reStructuredText source in a public GitHub repository; review happened in the open, via pull requests attached to identifiable individuals; reviewers were acknowledged by name. The toolchain, developed with Gaël Varoquaux and extended by Stéfan van der Walt into a system using Sphinx, LaTeX, custom scripts, and the `procbuild` preview bot, was an early instance of what would later be called "open peer review" and put into practice the principles I articulated in a 2012 paper titled "*Learning from Open Source Software Projects to Improve Scientific Review*" [20]. Its goal was explicitly

iterative: reviewers and editors worked with authors to guide papers toward acceptance, so that submitting to the SciPy conference proceedings meant opening your writing to the same collaborative scrutiny that good open source development demands.

My last year as co-chair for the SciPy conference was 2011. The people organizing the conference by then were no longer, by and large, the people maintaining the library. That shift was deliberate in a way the infrastructure diaspora was not. The `scikits` left `scipy.org` because the server and processes could no longer scale; the conference moved because we wanted it to—from company stewardship to community governance, from one annual meeting in California to a network of meetings on three continents. Proceedings made it easier for academic developers and users of scientific Python to justify attending; as more of those people came, the organizers naturally became drawn from a broader pool. By the end of the decade, "SciPy the conference" was clearly its own institution, with its own leadership and rhythms, loosely connected to the library and no longer run by the same small group.

3.3. Lesson 2

A shared technical foundation is more than scaffolding: it is part of what holds a community together. But holding together and staying uniform are not the same thing. When that foundation stopped scaling, the people who depended on it did not disappear; they found their own paths, and what followed looked, from the outside, like fragmentation. From the inside it was something closer to growth: the same people, carrying norms first worked out together, building new things in new places, and discovering by experiment what a much larger ecosystem would need next.

The conference took a different route to the same outcome. Rather than scattering under pressure, it was deliberately opened: from a company-run event serving one small circle to a community-governed institution with its own proceedings, its own international meetings, and eventually its own leadership drawn from far beyond the group that had built `numpy` and `scipy`. The infrastructure diaspora and the `scikits`' departure were reactions to a shared foundation that no longer scaled. The conference followed another path: as it grew, more participants could take responsibility for its program, tutorials, proceedings, and meetings.

> **Growth changes what needs to be held together.**
>
> As the `scikits` and the conference grew, the arrangements that once joined them to `scipy` no longer fit what each needed. The lesson is to recognize those transitions and deliberately develop the infrastructure, roles, and governance that a larger community requires.

4. How to last

At the end of the second decade we faced a new problem: there was no longer anyone with the mandate, or the mechanism, to hold the ecosystem together. As individual projects matured, scientific Python communities serving particular scientific domains (e.g., neuroimaging, geospatial data science, and high-energy physics) developed their own coordination structures, mailing lists, and governance conventions. This domain-level organization was healthy in many respects. It also meant that an increasing amount of work that was really ecosystem-wide in scope, packaging standards, CI practices, documentation tooling, was being developed in domain-specific silos and reinvented independently across communities. Newer maintainers and contributors built careers within a single project or domain rather than across several. The cross-disciplinary experience that those of us who had been there at the beginning carried was no longer naturally transmitted to the next generation. The `scipy.org` website (nominally maintained by the `scipy` library team,

effectively serving as the ecosystem's front door) had become a symbol of that failure of transmission: a shared resource that no one had the mandate to govern.

Some of those coordination needs had already become visible. A group of contributors from across scientific Python proposed a set of recommendations that would apply across projects, but used NumPy's proposal process because no ecosystem-wide forum yet existed. It was a useful solution, but it also made the underlying problem visible: standards intended for many independent projects could not depend indefinitely on any one project's processes or governance.

4.1. Ecosystem coordination

In 2020, Stéfan and I co-founded the *Scientific Python* project with support from the Alfred P. Sloan Foundation [21] to provide what the ecosystem needed, not the SciPy server's centralized infrastructure, which individual projects had rightly outgrown, but a shared layer above the individual projects: cross-project recommendations, developer summits that worked like the early sprints (small, cross-disciplinary, and work-focused), shared tooling and discussion forums, and a common landing page for the ecosystem `scientific-python.org`. Early community outreach by Juanita Gomez and project leadership by Brigitta Sipőcz were especially important in turning *Scientific Python* from a small node into a functioning ecosystem hub.

The 2023 Scientific Python Developer Summit illustrated what this cross-project coordination makes possible. Henry Schreiner had developed a comprehensive development guide for the `scikit-hep` community, high-energy physics, that covered modern packaging, testing, CI, documentation, and more; in parallel, Dan Allan at the National Synchrotron Light Source II had developed complementary guidelines for his community. Both guides addressed concerns that had nothing to do with the specific scientific domain and everything to do with the shared challenge of building maintainable scientific Python packages. At the summit, that work was merged, rebranded as an ecosystem-wide resource, and released as the Scientific Python Development Guide [22], [23]: a signal, concrete and deliberate, that domain-specific silos could be dissolved when the right forum existed to bring people together. The summit worked because it worked like the early meetings and sprints: small, cross-disciplinary, organized around real work rather than reports.

4.2. Academic anchoring

The story so far has been mostly about software: an array object, a library, an ecosystem, a software conference, a coordination layer. But this ecosystem changed more than code: it changed the people and the places that built it. Our neuroimaging collaboration brought Fernando Pérez to Berkeley's Brain Imaging Center (BIC) as a research scientist around 2008, where he continued developing IPython and, not long after, co-founded Project Jupyter. Over the years that followed, scientific Python and Jupyter colleagues who came to Berkeley for a meeting, a sprint, or a few months of collaboration became part of life at the BIC and, later, at Berkeley more broadly. Min Ragan-Kelley (`ipython` and later `jupyter`) was among those who first came to work alongside us this way, one of many such visits I won't try to enumerate. The pull reached graduate students, too. Kirstie Whitaker, then a neuroimaging PhD student, has described how Cindee Madison (part of our neuroimaging collaboration) mentored her in learning Python and navigating open source norms and culture, ultimately reshaping her career. Kirstie's is one version of a story that played out, in smaller ways, across generations of Berkeley researchers who found themselves drawn into the ecosystem they had only meant to borrow a tool from.

That gravitational pull eventually helped reorganize the university around it. The same community of scientific Python and Jupyter contributors that had been informally passing

through Berkeley helped establish the Berkeley Institute for Data Science (BIDS), which Fernando co-founded, and BIDS' early success fed into the university's decision to launch the College of Computing, Data Science, and Society and, within it, a Data Science major that is now the largest on campus [24]. The undergraduate curriculum built on that foundation, Data 8 and Data 100, uses NumPy, SciPy, Matplotlib, and Jupyter notebooks as the substrate for the largest introductory and largest upper-division courses at Berkeley, where thousands of students each semester use computation as part of learning statistics and data science [25]. That curriculum did not stay on campus: more than fifty universities and colleges have since adopted Data 8 or its components, several through Berkeley-supported partnerships with the National Science Foundation's CloudBank project that extended the same open-source, cloud-based teaching stack to community colleges and other UC campuses [26].

Fernando is now the BIDS' Faculty Director, a decade after co-founding it; Stéfan and Min work there on the technical and infrastructure side; Kirstie, the graduate student whose career had once been redirected by conversations with Cindee, returned in 2025 as BIDS's Executive Director; and I lead Berkeley's Open Source Program Office from within it. Fernando has written that Berkeley's leadership in this stack "is not 'owned' by Berkeley," but built by scientists who "partner with an extended, distributed community of other researchers and developers to build an ecosystem that benefits all" [27]. This is the same argument this paper opened with in 2003—that community-owned tools, built and governed by the people who need them rather than by a single lab or company, outlast anything one owner can sustain alone—now stated at the level of a university rather than a single lab.

What comes next is not yet settled. Fernando has described BIDS as a space for open scholarship, open source, and interdisciplinary collaboration on AI in science and society [27], and how artificial intelligence reshapes the way scientists write code, teach it, and learn it is the open problem this community will now have to work out together, in much the same way it once worked out the future of the array object in a room in Berkeley.

4.3. Lesson 3

Why did two of the most widely-used projects in scientific computing (the scientific Python and Jupyter ecosystems) have roots inside a neuroimaging lab at Berkeley that was not, on paper, in the software business at all?

One condition that mattered enormously was how Mark D'Esposito ran his lab. Mark wanted good people in the lab producing good work, and he trusted his team enough to let them pursue what they judged important, even when it sometimes drifted far from neuroscience. That trust is what let Matthew, Fernando Pérez, and me spend real time on scientific Python and, later, Jupyter, work that was not in any of our job descriptions and that did not, on its own, advance the neuroscience questions the lab was trying to answer.

Mark gave me the freedom to decide, together with Matthew and Fernando, what the major problems on the horizon were, and then to work on them. He gave us shared space, in the literal sense of desks in the same building, and a subtler kind of space: freedom from having to justify that work in terms of our immediate career prospects. That combination, more than any specific decision about software or method, is what let a small group inside a neuroimaging lab become a nucleus that drew in others and helped grow two community-owned ecosystems.

Being embedded in a neuroimaging lab rather than a software lab shaped our work. Our colleagues were neuroscientists, not programmers, so anything we built had to be installable, learnable, and usable by people who had no interest in becoming software

developers. That constraint pushed us toward teaching as much as toward building: writing documentation and tutorials, running hands-on sessions, and treating the question of how someone would learn a tool as inseparable from the question of how to design it.

Our collaborations with astronomers, physicists, and statisticians meant our tools were constantly tested against needs that had nothing to do with neuroimaging, which kept our discussions from narrowing into a closed conversation among people who already agreed with each other. It also connected us to larger conversations about open science and reproducibility that were happening well outside any one department, so the problems we were facing were visibly the same problems other fields were running into.

Matthew, Fernando, and I were lucky. Many of the people who built and sustained this ecosystem did not have a Mark D'Esposito, and did the work anyway: on nights and weekends, on volunteered time, on the strength of believing in something their institutions gave them no formal credit for. Some took real career risks to keep contributing. Many could not sustain that position indefinitely and left for industry, where the demands of a job rarely leave room for the kind of unstructured, long-horizon tinkering that built the scientific Python and Jupyter ecosystems. The space that made this possible was never distributed evenly, and a great deal of this ecosystem was built by people paying, personally, for a trust their own institutions were unwilling to extend.

BIDS is one attempt to make institutionally what Mark gave informally: room, trust, and time, backed by something more durable than one director's goodwill. But these remain exceptions rather than the norm, and they are not yet enough to catch everyone this ecosystem depended on and then let go.

It was the space to build it.

This ecosystem lasted because some people were trusted to decide what mattered and given time to work on it. It has also lost contributors, and capacity, because that trust was never extended widely or durably enough. Making it durable is the work still ahead.

5. Conclusion

This account is mine, but the pattern it traces is not. A concrete problem that outgrows its original domain, a small group that chooses to share a room until an aspiration becomes a plan, and a foundation later reorganized (deliberately or under pressure) into something many hands can sustain: this arc is not unique to numpy, scipy, or Berkeley, and other ground-up collaborations may recognize some version of it in their own histories. It is fitting that this account spans almost exactly the twenty-five years the SciPy conference itself has now completed. The same needs that pulled a handful of neuroimagers, astronomers, and statisticians into one room in 2005 are what still pull new communities into this one today, and the work of extending enough trust, time, and institutional support to sustain them is, a quarter century on, still not finished.

Acknowledgements

I am grateful to Matthew Brett, David Cournapeau, Perry Greenfield, Madicken Munk, Travis Oliphant, Pushkar Sathe, Stéfan van der Walt, and Kirstie Whitaker for their careful reading of the manuscript and for thoughtful comments that helped clarify the narrative and sharpen several of its central arguments.

Portions of this work were assisted using generative AI tools. Specifically, I used Perplexity AI for research, fact-checking, refining language, restructuring the paper, copy editing, and proof-reading. All outputs were reviewed, verified, and revised by me, and I take full responsibility for the accuracy and integrity of the final content.

References

[1] K. J. Millman and M. Brett, "Analysis of Functional Magnetic Resonance Imaging in Python," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 52–55, 2007, doi: 10.1109/mcse.2007.46.

[2] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, Feb. 2020, doi: 10.1038/s41592-019-0686-2.

[3] T. Oliphant, "Status of Numeric." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2004-January/002630.html

[4] T. Oliphant, "Comments on the Numarray/Numeric disscussion." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2004-January/002667.html

[5] T. Oliphant, "updating Numeric." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2005-January/003906.html

[6] T. Oliphant, "Numeric3 status." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2005-February/003958.html

[7] P. Greenfield, "Notes from meeting with Guido regarding inclusion of array package in Python core." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2005-March/004354.html

[8] T. Oliphant, "Future directions for SciPy in light of meeting at Berkeley." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2005-March/004318.html

[9] T. Oliphant, "[ANN] NumPy 1.0 release." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2006-October/011810.html

[10] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[11] T. Oliphant, "SciPy Sprint results." [Online]. Available: https://mail.python.org/pipermail/numpy-discussion/2007-December/030263.html

[12] K. J. Millman, "ANN: SciPy 0.7.0." [Online]. Available: https://mail.python.org/pipermail/scipy-dev/2009-February/010977.html

[13] S. Developers, "SciPy 0.7.0 Release Notes." [Online]. Available: https://docs.scipy.org/doc/scipy/release/0.7.0-notes.html

[14] K. J. Millman and T. Vaught, "The State of SciPy," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 2008. doi: 10.25080/HNCD8803.

[15] S. van der Walt, "The future of SciPy and its development infrastructure." [Online]. Available: https://mail.python.org/pipermail/scipy-dev/2009-February/011109.html

[16] G. Varoquaux, Personal communication, 2009.

[17] K. J. Millman and M. Aivazis, "Python for Scientists and Engineers," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 9–12, 2011, doi: 10.1109/MCSE.2011.36.

[18] F. Pérez, B. E. Granger, and J. D. Hunter, "Python: An Ecosystem for Scientific Computing," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 13–21, 2011, doi: 10.1109/MCSE.2010.119.

[19] G. Varoquaux, T. Vaught, and K. J. Millman, "Editorial," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 2008. doi: 10.25080/HXDH2061.

[20] S. S. Ghosh, A. Klein, B. Avants, and K. J. Millman, "Learning from open source software projects to improve scientific review," *Frontiers in Computational Neuroscience*, 2012, doi: 10.3389/fncom.2012.00018.

[21] S. van der Walt and K. J. Millman, "Scientific Python: Planning the Next Decade," Grant Proposal, 2020. [Online]. Available: https://scientific-python.org/doc/scientific-python-planning-grant-2020.pdf

[22] H. Schreiner, "The Scientific Python Development Guide." [Online]. Available: https://blog.scientific-python.org/scientific-python/dev-summit-1-development-guide/

[23] Scientific Python Community, "Scientific Python Library Development Guide." [Online]. Available: https://learn.scientific-python.org/development/

[24] Brandilyn Buckley, "UC Berkeley Data Science major now largest on campus." [Online]. Available: https://cdss.berkeley.edu/dsus/news/data-science-major-now-largest-campus

[25] E. V. Dusen, A. Suen, A. Liang, and A. Bhatnagar, "Accelerating the Advancement of Data Science Education," *SciPy 2019*, 2019, doi: 10.25080/Majora-7ddc1dd1-000.

[26] CloudBank, “CloudBank for Educators.” [Online]. Available: https://www.cloudbank.org/training/cloudbank-educators

[27] F. Pérez, “Director's Vision 2024.” [Online]. Available: https://bids.berkeley.edu/about/directors-vision-2024

Open Access | https://doi.org/10.25080/hrfh5447

From LiDAR to action: detecting upland gullies to combat erosion and forest fires

Jim Pivarski[1]

[1]University of Chicago Data Science Institute

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

The University of Chicago's Data Science Institute (DSI) works with 11th Hour Project grantees to support environmental and human rights work. In this paper, we describe a collaboration with the Occidental Arts & Ecology Center (OAEC) on their Fuels-to-Flows program, which stabilizes upland waterways by adding brushwood that would otherwise fuel forest fires. Gullies are hidden by trees, so we used publicly available, sub-meter resolution LiDAR data to identify gullies by the shape of the landscape throughout Sonoma County, California. Due to the limited size of hand-labeled training datasets, we built an alternative to a Convolutional Neural Network (CNN) by engineering convolutional kernels using domain-specific knowledge—a "right-sized machine learning" solution for this problem—then identified gully centerlines and built a network of gullies as paths. The final deliverable is a statically hosted map application that puts all available information into the hands of restoration practitioners and land managers: the gully network, forest fire fuel proxies, aerial imagery, elevation contours, cross-sectional profiling tools, and differences in elevation between two LiDAR scans in 2013 and 2022. In this paper, we also describe how we overcame technological problems: CUDA programming in Python using Numba to handle our unusually large convolution kernels, PMTiles to serve hundreds of gigabytes of map data as a static website, and cross-sectional profiling tools.

Keywords LiDAR, geospatial analysis, erosion, wildfire mitigation, Numba, PMTiles

1. Introduction

Through a partnership with the Schmidt Family Foundation's 11th Hour Project, the University of Chicago's Data Science Institute (DSI) contributes statistical and computational expertise to programs working for renewable energy, resilient food systems, healthy oceans, and the protection of human rights. The project described in this paper benefits the Occidental Arts & Ecology Center (OAEC)'s Fuels-to-Flows program [1], [2], an innovative effort to solve two environmental problems, wildfires and erosion, in one stroke. Rather than eliminating brushwood that would fuel forest fires using controlled burns, this program packs that brushwood into small upland gullies to slow down runoff and retain water in the landscape.

We at the DSI were originally consulted to help find new sites for the Fuels-to-Flows program by scanning LiDAR data to find gullies and forest fire fuels in Sonoma County, California. Derived LiDAR products for forest fire fuels, "ladder fuel proxies," were already provided by the SonomaVegMap project [3] as publicly available data products, so we focused on identifying gullies.

Through our interactions with the OAEC, we learned that site selection for any environmental restoration work is a complex process that requires many considerations beyond listing the intersection of gullies with forest fire fuels. Land ownership, permitting constraints, accessibility, recent landscape change, and hazards invisible to LiDAR all contribute. Also,

Published Jul 13, 2026

Correspondence to
Jim Pivarski
jpivarski@uchicago.edu

Open Access

Copyright © 2026 Pivarski. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

practitioners usually focus on a single park or wildlife reserve because of an ongoing relationship. Rather than providing a spreadsheet of candidate locations, it would be more effective to put the ability to explore the data in their hands, so we built a web-based map application that reveals all data pertinent to site selection: where the gullies are, where the forest fire fuels are, land ownership boundaries, aerial imagery corresponding to both full-county LiDAR sweeps in 2013 and 2022, elevation contours, differences in elevation between the two LiDAR sweeps, which is a proxy for erosion and deposition, and a tool for extracting detailed elevation cross-sections across landscape features from both years, which is useful for identifying migrating head-cuts, for instance. This map application doesn't replace the need to investigate sites in person, but it is invaluable for triaging and prioritizing sites.

Our goal for this paper is to provide a case study in how data science techniques and Python libraries were applied to solve a mission-driven problem in practice, highlighting the issues and pragmatic solutions, rather than presenting an idealized model of gully detection.

2. Problem Context

The Fuels-to-Flows program has been successfully applied on OAEC's own property and at Monte Rio Redwoods Regional Park over the last four years. The challenge now is to expand the program to new sites that the practitioners are not as intimately familiar with.

Most gullies are hard or impossible to find by aerial photography because they are hidden under a tree canopy, but they can easily be seen using bare-earth elevation maps derived from LiDAR scans. Airborne LiDAR instruments emit laser pulses and record reflected returns, some of which reflect from the tops of trees while others pass through gaps in the vegetation and reflect off the ground. The bare-earth elevation is derived from the ground returns and from interpolations through a 3-dimensional model of especially dense vegetation. The SonomaVegMap project published bare-earth elevation maps for the entire county with a 1 meter resolution in 2013 and a half-meter resolution in 2022.

Our geographic scope is the entirety of Sonoma County. The methods are not intrinsically limited to one county, but the available data products make the county a convenient boundary and it is already a significant expansion beyond the OAEC and its local collaborators. In addition, Sonoma County is somewhat unusual in having repeat coverage, which we can use to estimate landscape changes due to erosion. That said, the kernel length scales were chosen for the gully morphology visible in Sonoma County's half-meter bare-earth data, so regions with substantially different terrain or coarser, sparser LiDAR would likely require rescaling the kernels and re-fitting the model.

The main deliverable of this work is a web-based map in which all data layers can be overlaid for comparison. Some of those layers are directly copied from SonomaVegMap, such as ladder fuel proxies (e.g. LiDAR density 1–4 meters above ground divided by LiDAR density 0–4 meters above ground), or the Sonoma County GIS for parcel files for land ownership boundaries, with reformatting to provide them as map tiles. Other layers required significant derivation, such as the network of gullies that can be overlaid on other images. All of these data products are hosted on Cloudflare with documentation on our website, https://github.com/dsi-clinic/gully-map, to allow experts to dynamically load them in ArcGIS or QGIS. However, our web-based map is easier for non-experts and does not require extra software. Table 1 summarizes all of the data products.

Table 1. *Data products produced by this study, for viewing in the web-based map or directly in ArcGIS or QGIS. Here, "SVM" refers to SonomaVegMap and "Size" includes only PMTiles.*

Layer	Derivation	Formats	Size

Land ownership boundaries	Reformatted from Sonoma County GIS	PMTile	27 MiB
Ladder fuel proxies	Reformatted from SVM 2022	PMTile, GeoTIFF	115 MiB
Greyscale hillshade	Reformatted from SVM 2022	PMTile, GeoTIFF	2.9 GiB
Color-enhanced hillshade	Derived from SVM 2022 data	PMTile, GeoTIFF	149 GiB
Elevation contours	Derived from SVM 2022 data	PMTile, GeoJSON	17.7 GiB
Elevation difference	Derived from SVM 2013 & 2022 data	PMTile, GeoTIFF	5.7 GiB
Elevation difference contours	Derived from SVM 2013 & 2022 data	PMTile, GeoJSON	11.8 GiB
Gully probability	Derived from SVM 2022 data	PMTile	843 MiB
Detected gully network	Derived from SVM 2022 data	PMTile, Parquet	34 MiB
Elevation 2013 & 2022	Reformatted from SVM 2013 & 2022	PMTile, GeoTIFF	22.1 GiB

3. Gully Detection Model

Two of the map features listed in Table 1 are useful for finding gullies: color-enhanced hillshade, which is a raster image, and the detected gully network, which is vector data.

3.1. Color-Enhanced Hillshade

Color-enhanced hillshade, suggested by Adam Cummings, is derived from the bare-earth elevation map to make creases in the landscape more visible by eye. It consists of the elevation data I_{in}, a function of spatial x and y, texture-enhanced by a separable frequency-domain filter [4],

$$I_{\text{out}} = \mathcal{F}_x^{-1}\left[(u^2)^{\alpha}\, \mathcal{F}_x(I_{\text{in}})\right] + \mathcal{F}_y^{-1}\left[(v^2)^{\alpha}\, \mathcal{F}_y(I_{\text{in}})\right] \tag{1}$$

where $\mathcal{F}_x$ and $\mathcal{F}_y$ are horizontal and vertical Fourier transformations, u and v are the corresponding frequency coordinates, and $\alpha = 0.5$. The resulting image I_{out} is then mapped to a blue-green-yellow color ramp and combined with a grayscale hillshade (derived from the same elevation data I_{in}) using the overlay blend mode:

$$\text{overlay}(x, y) = \begin{cases} 2xy & \text{if} y < 0.5 \\ 1 - 2(1-x)(1-y) & \text{otherwise.} \end{cases} \tag{2}$$

This effectively colorizes the hillshade with the texture-enhanced relief signal. A sample of this transformation is shown in the top half of Figure 1.

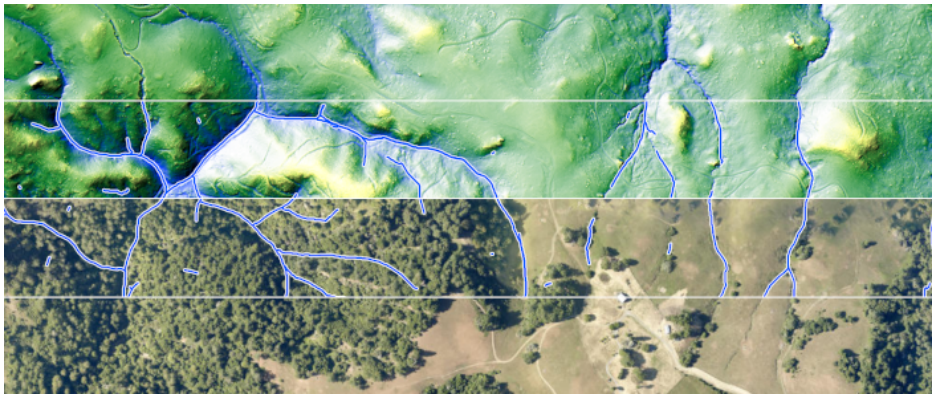


Figure 1. *Sample imagery in which the top two panels are color-enhanced hillshade and the bottom two panels are aerial photography from 2022. In the middle two panels, the detected gully network is drawn as outlined blue paths.*

3.2. Detected Gully Network

Although the color-enhanced hillshade draws the eye's attention to subtle creases, it can't be overlaid on other image layers such as a street map or aerial photography, which are sometimes needed for surrounding context. For a clean overlay, we want a vectorized path that can be outlined without introducing too much visual noise.

To produce such a path, we first need to identify which pixels in the bare-earth elevation image are in gullies, then connect neighboring gully pixels to form a graph whose nodes are the pixels, and finally skeletonize the graph to narrow it to uniform (single pixel) thickness.

The first step, identifying pixels in gullies, affects the final quality of the network. If too many pixels are identified, the final network will be noisy, with many disconnected short paths. If too few pixels are identified, gullies will be missed. In a standard technique, called Difference From Mean Elevation (DFME) and described for instance by Evans and Lindsay [5], the elevation at each pixel is subtracted from the mean elevation in a large square or circle centered on that pixel. It is often used directly as an imaging technique, like color-enhanced hillshade. To use this algorithm as a first step in graph-building and skeletonization, we need to set a threshold to label deep indentations as gullies, but there is no threshold that accepts enough pixels without also accepting too much noise. The landscape has many dimples that are as deep as a gully without being connected to a system of channels like a gully.

At the other extreme, we could use a Convolutional Neural Network (CNN) to identify gullies as an image segmentation problem. However, this supervised learning technique requires a target dataset of pixels that have already been labeled as "gully" or "not gully." Since we had no such dataset, we hand-labeled training data on our own by painting over images with one color for "gully," another for "not gully," and transparency wherever a determination could not be made. This was a time-consuming process involving ground-truth checking with Google Street View, separately in 9 watersheds of Sonoma County for geographic variety. In the end, we did not have enough hand-labeled data to train a CNN.

There is a midpoint between these extremes. The DFME method is a convolution of the image with a kernel that is positive at a central point, negative in a large square or circle centered on that point, and sums to zero:

$$K_{\mathrm{DFME}}(x,y) = \begin{cases} \frac{1}{\pi w^2} & \text{if} x^2 + y^2 \le w^2 \\ -\frac{1}{\pi(R^2 - w^2)} & \text{if} w^2 < x^2 + y^2 \le R^2 \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

for w = 1 pixel and R = 50 pixels (25 meters), illustrated on the left of Figure 2. Convolving this kernel with the bare-earth elevation image effectively subtracts each pixel from the mean of all pixels in a circle of radius R around it.

A convolutional layer in a CNN convolves the image with several kernels that are learned as part of the global optimization. The kernels represent relevant shapes at different size scales by pooling (resizing) the image in each layer. While that's necessary for recognizing complex images like the face of a cat with sufficient generality to be insensitive to camera angles and lighting, our task is much simpler: gullies are trough-shaped and the Earth is always viewed from directly above. Instead of convolving with K_{DFME}, we can convolve with a trough-shaped kernel,

$$K_{\mathrm{trough}}(x,y;\theta,\sigma) = \begin{cases} -1/(\pi R^2) + \frac{\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)}{\text{normalization}} & \text{if } x^2 + y^2 \le R^2 \text{ and } \mid x\sin\theta + y\cos\theta \mid < w \\ -1/(\pi R^2) & \text{if } x^2 + y^2 \le R^2 \text{ and } \mid x\sin\theta + y\cos\theta \mid \ge w \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

where K_{trough} is parameterized by an angle θ, illustrated in the middle of Figure 2. A convolution with this kernel highlights pixels that are lower than the surrounding region and also part of a linear path of low pixels, a path angled by θ and tapered like a Gaussian with characteristic length σ. Performing a weighted sum over such kernels at a variety of angles θ highlights pixels in troughs of any direction, and this process is equivalent to inference in one convolution layer of a CNN.

In practice, we noticed that convolutions with K_{trough} also highlighted one-sided cliffs, usually artificial cliffs along roadsides, because the cliff matches one side of a trough and missing the other side only dampens the convolution by a factor of 2. As a veto, we added another kernel that only matches one-sided cliffs:

$$K_{\text{cliff}}(x, y; \theta, \sigma) = \begin{cases} \frac{1}{\text{normalization}} \exp\left(\frac{x^2+y^2}{2\sigma^2}\right) & \text{if } x^2 + y^2 \le R^2 \quad \text{and } x\sin\theta + y\cos\theta < 0 \\ -\frac{1}{\text{normalization}} \exp\left(\frac{x^2+y^2}{2\sigma^2}\right) & \text{if } x^2 + y^2 \le R^2 \quad \text{and } x\sin\theta + y\cos\theta \ge 0 \\ 0 & \text{otherwise,} \end{cases} \tag{5}$$

illustrated on the right of Figure 2. A model that gives positive weight to convolutions with K_{trough} and negative weight to convolutions with K_{cliff} can distinguish between troughs and cliffs.

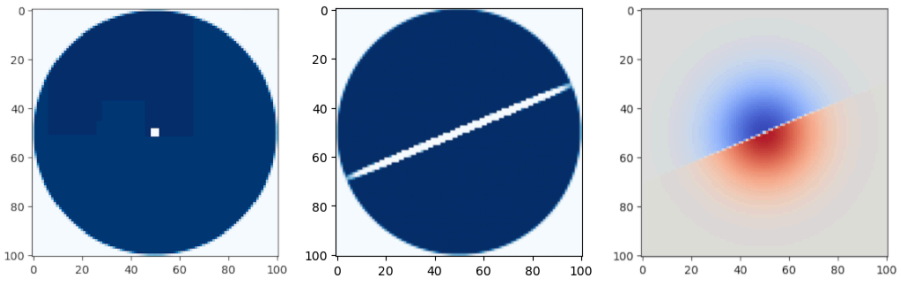


Figure 2. *Convolution kernels for pixelwise gully detection. Left: the DFME kernel (K_{DFME}) is positive in one pixel in the center, negative in a large disk around it, and sums to zero. Middle: a trough-shaped kernel (K_{trough}) replaces the single positive pixel with an angled line of a characteristic length σ, tapered as $\exp(-x^2/\sigma^2)$. Right: the cliff veto kernel (K_{cliff}) is a positive (red) Gaussian on one side of an angled line and negative (blue) on the other.*

To be clear, we are replacing a learned-kernel CNN with feature engineering inspired by CNNs. We are using domain knowledge to hand-craft a set of kernels. We also know that convolutions with $K_{\text{trough}}(x, y; \theta, \sigma)$ should have a roughly sinusoidal dependence on θ for a point in a linear gully with a stronger dependence if σ is large because the kernel shape is a good fit when aligned and a bad fit when misaligned. We verified this for several points in gullies, one of which is shown in Figure 3.

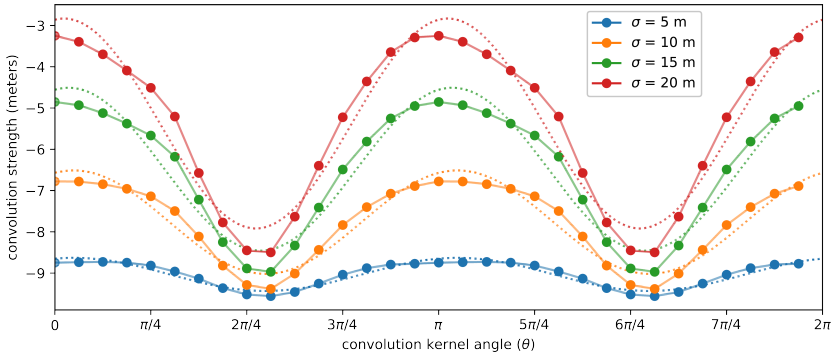


Figure 3. *Value of convolutions at a point in a gully for $K_{\text{trough}}(x, y; \theta)$ with varying angles θ and Gaussian widths σ. Dotted lines are sinusoidal fits to the convolution results.*

Due to this regularity, we can reduce the number of features and therefore the number of parameters in the model without losing model quality. The three most important features are the highest and lowest point of a sinusoidal fit of convolutions $C(x, y; \theta, \sigma)$ with kernel $K_{\text{trough}}(x, y; \theta, \sigma)$ and the minimum value of those convolutions, which is useful because the minimum value is lower than the fitted sinusoidal curve in most of the real gullies that we observed. The amplitude of the sinusoidal fit can be computed directly as the modulus of a complex sum, a Riemann approximation of the integral to compute the first Fourier coefficient. The phase of the sinusoidal fit could tell us the direction of the trough at a given point, but we don't need that for modeling.

$$\text{high}(x, y; \sigma) = \frac{1}{16}\sum_{n=0}^{15} C\left(x, y; \frac{n}{16}\pi, \sigma\right) + \frac{1}{8}\,|\sum_{n=0}^{15} C\left(x, y; \frac{n}{16}\pi, \sigma\right) e^{in\pi/8}\,| \tag{6}$$

$$\text{low}(x, y; \sigma) = \frac{1}{16}\sum_{n=0}^{15} C\left(x, y; \frac{n}{16}\pi, \sigma\right) - \frac{1}{8}\,|\sum_{n=0}^{15} C\left(x, y; \frac{n}{16}\pi, \sigma\right) e^{in\pi/8}\,| \tag{7}$$

$$\text{mini}(x, y; \sigma) = \min_{n=0}^{15} C\left(x, y; \frac{n}{16}\pi, \sigma\right). \tag{8}$$

To exclude cliffs, we take the minimum of convolutions $C_{\text{cliff}}(x, y; \theta, \sigma)$ with kernel $K_{\text{cliff}}(x, y; \theta, \sigma)$.

$$\text{cliff}(x, y; \sigma) = \min_{n=0}^{15} C_{\text{cliff}}\left(x, y; \frac{n}{16}\pi, \sigma\right). \tag{9}$$

Evaluating these at σ = 5 meters and 15 meters,

$$\begin{array}{ll} H_5(x,y) = \text{high}(x, y; 5\text{ meters}) & H_{15}(x,y) = \text{high}(x, y; 15\text{ meters}) \\ L_5(x,y) = \text{low}(x, y; 5\text{ meters}) & L_{15}(x,y) = \text{low}(x, y; 15\text{ meters}) \\ M_5(x,y) = \text{mini}(x, y; 5\text{ meters}) & M_{15}(x,y) = \text{mini}(x, y; 15\text{ meters}) \\ & F_{15}(x,y) = \text{cliff}(x, y; 15\text{ meters}) \end{array} \tag{10}$$

we performed a logistic regression with parameters a_i and bias term b

$$\begin{aligned} \text{logit}(p) = \; & a_1 M_{15} + a_2(L_{15} - M_{15}) + a_3(H_{15} - L_{15}) + a_4(M_5 - M_{15}) \\ & + a_5(H_5 - L_5) + a_6 L_{15}(H_{15} - L_{15}) + a_7\,|\,F_{15}\,| + b \end{aligned} \tag{11}$$

over all pixels x, y and obtained the results shown in Table 2. The fitted model's predicted probabilities for pixels labeled as "not in gully" or "in gully" are shown for the 9 watersheds in Figure 4; note that these are in-sample residuals. The dominant term is M_{15}, the minimum convolution with $\sigma = 15$ meter troughs. Because M_{15} values are negative in gullies with a negative coefficient, this term contributes positively to the probability that a pixel is in a gully. The rest of the terms are amplitudes, one interaction term, and the cliff veto. The cliff veto is fitted in absolute magnitude because it has an antisymmetry for $\theta \to \theta + \pi$. Because it is always positive and has a negative coefficient, it contributes negatively to the probability that a pixel is in a gully. Figure 5 provides an out-of-sample measure of model quality: the Area Under the Curve (AUC) of the Receiver Operating Characteristic (ROC), cross-validated across the 9 watersheds. All AUC values are above 0.98, indicating not only good discriminating power, but also consistency across watersheds.

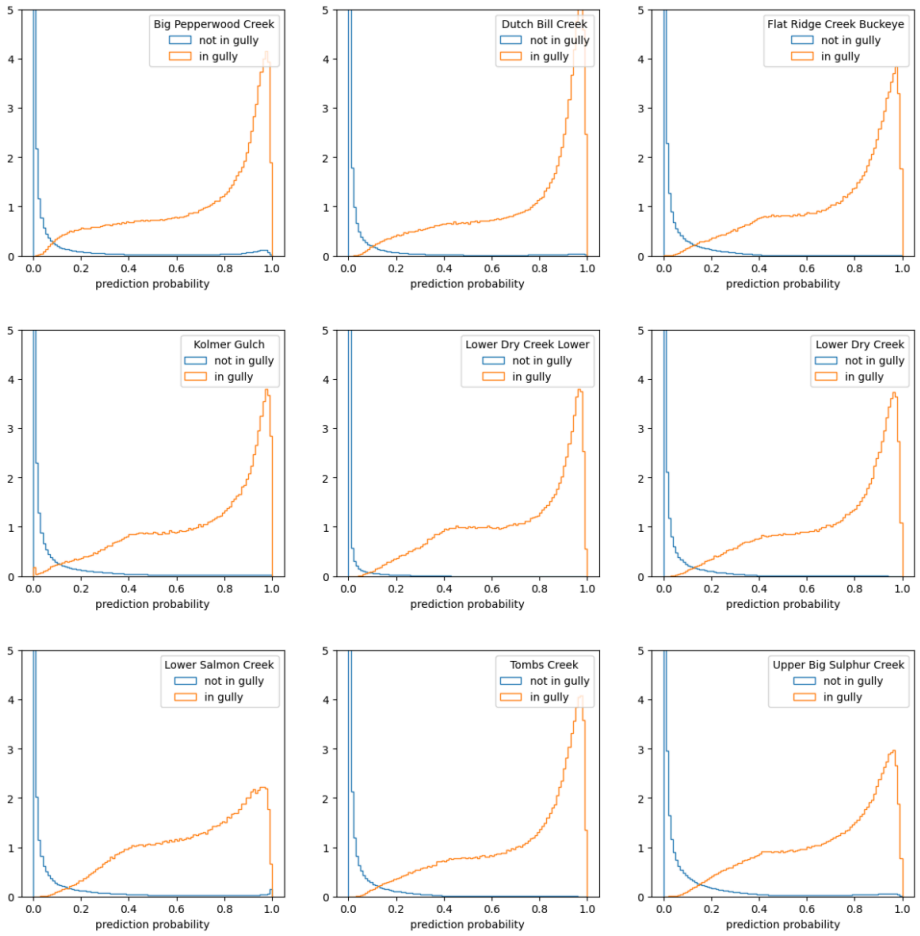


Figure 4. *Predicted probability for pixels labeled as "not in gully" or "in gully" in 9 watersheds. These pixels were used to train the 8 parameter fit, so they are in-sample residuals (difference of "not in gully" from 0 and difference of "in gully" from 1).*

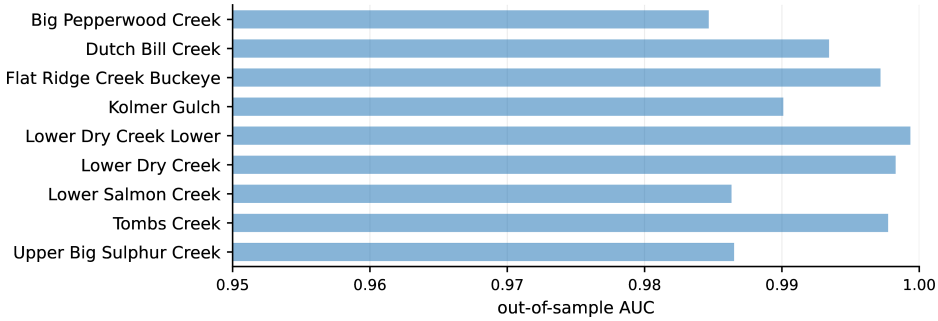


Figure 5. *Area Under the Curve (AUC) of the Receiver Operating Characteristic (ROC) for gully-finding models cross-validated across the 9 watersheds.*

Table 2. *Fit results for the logistic regression in Equation 11.*

parameter	value	term	parameter	value	term
a_1	-3.16	M_{15}	a_5	-0.27	$(H_5 - L_5)$
a_2	-0.72	$(L_{15} - M_{15})$	a_6	0.63	$L_{15}(H_{15} - L_{15})$
a_3	1.94	$(H_{15} - L_{15})$	a_7	-0.29	$\mid F_{15} \mid$ (cliff veto)
a_4	0.20	$(M_5 - M_{15})$	b	-9.80	(bias/intercept)

Since the terms of the logistic regression are not all linear combinations of the convolutions, our model has diverged from a standard CNN, but it's still advantageous to add a second "layer" to clean the image further. This subsequent step consisted in shrinking the image by a factor of 2 (pooling) and convolving again, this time with $K_{\text{trough}}(x, y; \theta, \sigma)$ with $\sigma = 20$ meters and minimizing over 16 grid-points in θ. Figure 6 illustrates the result for one gully system: (a) shows the color-enhanced hillshade for reference, (b) is the DFME method, (c) shows a threshold applied to the DFME image to make a mask, then each connected component is shown in a different color, (d) shows the probability output of this two-layer method, and (e) is a similar mask colored by connected components with a probability $> 5\%$ threshold. Compared with DFME, the two layers of trough convolutions produce a considerably less noisy image with the main gully's connections intact.

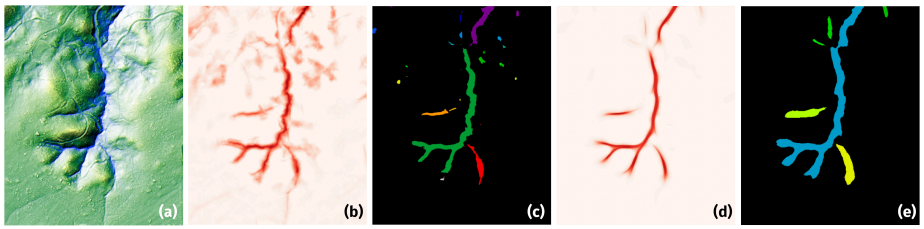


Figure 6. *Comparison of gully-finding algorithms: (a) color-enhanced hillshade, (b) DFME, (c) its connected components, (d) two-layer trough convolution, (e) its connected components.*

The next step is to reinterpret the above-threshold pixels of the image as nodes in a graph, connected with any neighbors on their 8 sides, and skeletonize the image by removing nodes if removing them does not change the connectivity of the graph. We implemented a greedy algorithm that removes pixels in increasing order of probability, so that the threshold could be set very low: probability $> 1\%$. To see why such a low threshold is needed, see Figure 4: distributions of non-gully probability peak near 0, but gully pixels, which are much less numerous in the data, are more uniform from 0 to 1. The skeletonization and its resulting graph are shown in Figure 7.

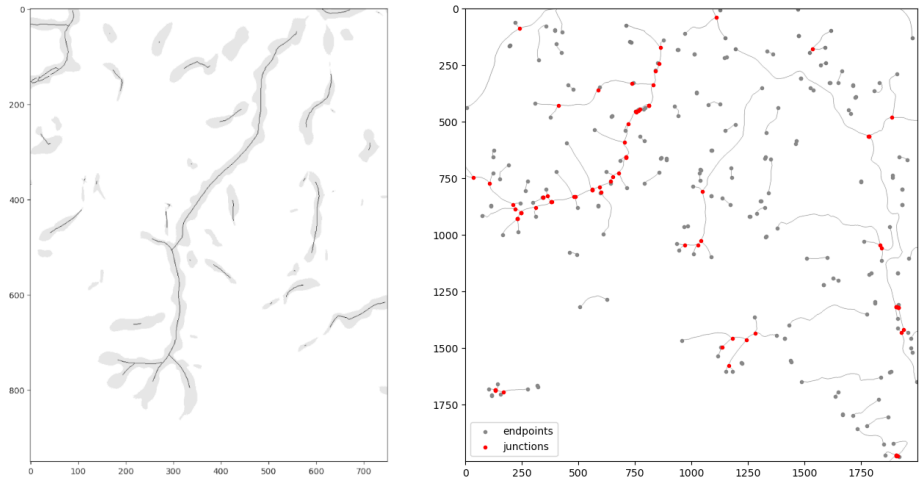


Figure 7. *Left: skeletonization of 1% probability threshold masks (gray) with probability-ordered erosion. Right: connection of the remaining pixels as a graph by identifying neighbors.*

4. Performance Optimization with Numba

Some steps in the gully-finding analysis are similar to standard techniques, but with unusual scaling. Early in the analysis, we found that our convolution kernels needed to

be at least 50 meters on a side to average over enough surrounding elevation for a stable baseline. Image convolution kernels are typically 3×3 through 11×11 pixels, but ours are 101×101. The time spent computing convolutions with SciPy's `convolve2d` was a constraint on exploratory studies, so we searched for alternatives. First we considered `*.scipy.signal.convolve2d` drop-in GPU-based replacements from the CuPy [6] and JAX [7] libraries, but the fastest implementation by a factor of several was a custom CUDA kernel compiled by Numba [8] with a fixed kernel size at compile time and simplified boundary conditions. To eliminate sensitivity to boundary conditions, we performed each convolution on one watershed at a time with padded boundaries, then clipped to the watershed. Gullies cannot cross a watershed divide.

We also used a custom CUDA kernel compiled by Numba for skeletonization, since the methods in Scikit-Image apply to bitmaps with the probability information removed. We implemented a skeletonization algorithm similar to Pudney's distance-ordered homotopic thinning [9], but we eroded pixels greedily in order of increasing probability, rather than distance, and we used connectivity-preservation as a constraint instead of topology-preservation, ensuring that all gullies are trees.

In both cases, we developed the analysis using standard libraries, then replaced the functions that needed better performance or different behavior with a CUDA kernel of the same API.

5. Delivery as a Static Map Application

Although the data products described in Table 1 are accessible to geospatial applications like ArcGIS and QGIS, most of the intended consumers are not geospatial experts, so we built a web-based map application. This application has a conventional interface: a large scrollable map with switches in a sidebar to toggle layers.

Often, these map applications are built with a server to provide map tiles on demand. A custom server would require occasional maintenance, which scales poorly because the DSI develops applications for many projects, and servers must be maintained indefinitely. Maintenance could be offloaded to a service like MapTiler, but accumulating services with monthly fees is a different kind of maintenance scaling. The best option is to provide an application as a static website—files served by Cloudflare without any application logic outside of the user's web browser.

The map and sidebar are all client-side HTML built with Svelte. The data products add up to hundreds of GiB, which is far too much to load with the webpage; it will need to be dynamically loaded as tiles. A directory structure of XYZ tiles adds significant overhead and MBTiles require an active server to look up entries in an SQLite database. Instead, we used PMTiles [10], a file format in which tiles are embedded at known seek positions within the file. A client-side library uses an index to find those seek positions, and then does a partial HTTP GET to access one tile at a time. All of the raster and vector data products are served as PMTiles.

The map application also allows users to draw a straight or curved line on the map for a cross-section of the elevation along that line, as shown in Figure 8. The elevation data used for this calculation is also provided as PMTiles. Each 32-bit floating point number is cast as the RGBA channels of a PNG raster in PMTiles and then decoded on the client.

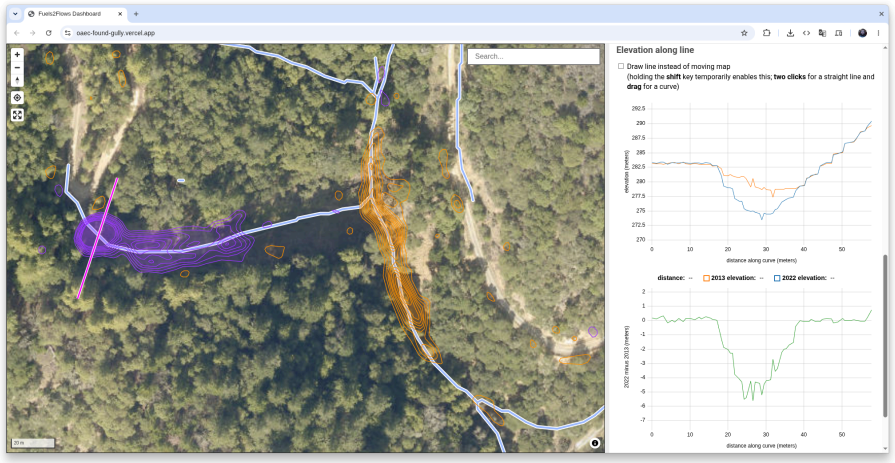


Figure 8. *The web-based map application, showing 2021 aerial photography, gully paths (outlined light blue), erosion (purple) and deposition (orange) contours, and a user-drawn line (magenta) along which elevation in 2013, 2022, and their difference are displayed as plots on the right.*

The GitHub repository for this project is https://github.com/dsi-clinic/gully-map where you will find a link to the map application, documented links to data products that can be loaded in ArcGIS or QGIS, and all of the code that was used to build it.

6. Cross-Disciplinary Reach

While the gully-finding study was underway, one of us was co-mentoring a University of Chicago Data Science Clinic project on an unrelated topic: a team of students were predicting pathologic complete response to treatment in breast cancer from MRI images using graph neural networks of the blood vessels. Building a graph from voxels identified as being in blood vessels is very similar to building a graph from pixels identified as being in blood vessels, except that MRI voxels are 3-dimensional while the LiDAR elevation map is 2-dimensional. Both studies need the connectivity achieved by setting segmentation thresholds low, and therefore they both benefit from the probability-ordered skeletonization algorithm described in Performance Optimization with Numba.

One of the students, Jose Cardona, adapted the 2-dimensional skeletonization algorithm to 3-dimensional blood vessels and observed performance gains in the final analysis compared to using the Vascular Modeling Toolkit (VMTK) skeletonization module in 3D Slicer. In the following quarter, the algorithm was further generalized to 4-dimensional time-series of MRI images, and our co-mentor Anna Woodard is now applying it to high time-resolution imaging sequences beyond the scope of the Clinic course. This instance of cross-disciplinary transfer illustrates the value of generalist organizations like the DSI, as well as a tight relationship between research and teaching.

7. Conclusions

This project addressed a set of questions that are broadly relevant to the SciPy community, beyond the specifics of gully detection. In each case, the useful path was a middle ground between a fully general solution and a special-purpose one built from scratch.

How can one build a model for inference on big datasets when training data are scarce? We built a model *inspired by* a CNN, but constructed it with domain-specific knowledge rather than a fully automated training process. This let us capture most of the benefit of a CNN's inductive bias without the quantity of supervision target data that optimizing

convolution kernels would typically need. This relied on recognizing that the standard DFME technique is itself a convolution, thereby identifying a continuity between the two approaches from which we could choose a middle ground.

What can one do if a computational technique breaks scaling assumptions or differs slightly from algorithms built into scientific libraries? In a workflow that uses standard scientific Python libraries wherever possible, we replaced only the specific functions that didn't scale or differed from the standard algorithms. Numba allowed us to compile our alternative code for speed yet interface with standard NumPy and CuPy arrays so that the majority of the workflow would be untouched.

How can one deliver large geoscience data products to users with varied technical expertise? A custom web application is a user-friendly way to browse data, although map applications typically require active servers to deliver data one tile at a time, and active servers are a maintenance burden. Static websites with pinned JavaScript dependencies require comparatively little upkeep, and the PMTiles format can be addressed one tile at a time by client-side libraries.

We hope this case study is useful to others facing similar issues.

Acknowledgments

We are grateful for Brock Dolman's very helpful guidance and useful feedback from Adam Cummings, Matt O'Connor, and Jeremy Kobor.

References

[1] B. Dolman, "Fuels to Flows: Bringing Nature-based Restoration to the Headwaters." [Online]. Available: https://www.naturalareas.org/fuels-to-flows.php

[2] Occidental Arts & Ecology Center, "Gully Stuffing Technique." [Online]. Available: https://oaec.org/our-work/projects-and-partnerships/wildlands/gully-stuffing-technique/

[3] Sonoma County Vegetation and Habitat Mapping Program, "SonomaVegMap." [Online]. Available: https://sonomavegmap.org/

[4] Z. Cuckovic, "Texture shading for digital terrain models." [Online]. Available: https://landscapearchaeology.org/2021/texture-shading/

[5] M. Evans and J. Lindsay, "A new gully head location model to predict gully erosion in upland drainage basins," *Earth Surface Processes and Landforms*, 2010, doi: 10.1002/esp.1918.

[6] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-Compatible Library for NVIDIA GPU Calculations," in *Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. [Online]. Available: http://learningsys.org/nips17/assets/papers/paper_16.pdf

[7] J. Bradbury *et al.*, "JAX: composable transformations of Python+NumPy programs," *GitHub repository*, 2018, [Online]. Available: http://github.com/jax-ml/jax

[8] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT Compiler," in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015. doi: 10.1145/2833157.2833162.

[9] C. Pudney, "Distance-Ordered Homotopic Thinning: A Skeletonization Algorithm for 3D Digital Images," *Computer Vision and Image Understanding*, vol. 72, no. 3, pp. 404–413, 1998, doi: 10.1006/cviu.1998.0680.

[10] B. Liu and P. contributors, "PMTiles: Pyramids of map tiles in a single file on static storage." [Online]. Available: https://github.com/protomaps/PMTiles

Open Access | https://doi.org/10.25080/nnfr8224

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Commit to Community: Open Source as Social Infrastructure in Volunteer Civic Tech

Kristijan Armeni[1], **Helen Y. Glover**[1], and **Ben Sando**[1]

[1]Civic Tech DC

Abstract

Most scientific Python maintainers build for users who will pip install their package. In civic tech, your community members are policy researchers, journalists, or NGO advocates who may never touch a Python environment. This changes how to approach maintainership: traditional open source practices do double duty as engineering and social infrastructure. Here we share lessons learned in maintaining the CIB Mango Tree project, a Python toolkit for detecting inauthentic behavior in social media datasets. By looking at examples like release cycles and continuous integration, I'll show how technical choices initially intended for code quality unexpectedly helped engage the volunteer community and sustain cross-team momentum.

Keywords civic tech, open source, maintainership, community building

1. Introduction

Civic tech and open source are aligned in both values and practice. Civic tech (short for civic technology) is a broad movement of technologists, public policy experts, and civic institutions for advancing public good through digital technology. In the US, civic tech finds its origins in the open data movement of the late 2000's when the first volunteer-run civic hackathons coalesced around the idea that government-created datasets should be made publicly available, channeling cultural and philosophical values of openness and collaboration inherited from the earlier open source software movement [1].

From a practical standpoint, open and collaborative tools promise to be more robust than their more centralized and closed counterparts due to increased public scrutiny and efficient reuse of existing open solutions. But collaborative *development* is just one side of the coin: building publicly-available technology also requires *maintenance*, reviewing bug reports and contributions, updating dependencies, managing releases etc., in order to keep open source technology reliable and functional [2], [3].

The Scientific Python ecosystem[1], a collection of community-developed open source coding libraries for scientific computing and data analysis, is an example of a community that has nourished both. Over time, it has not only developed a family of essential data science libraries, but also contributed guidelines, governance structures, and maintainership playbooks that keep the community and tools sustainable. Given the strong cultural alignment between open source software and civic tech, it is perhaps not surprising that Scientific Python libraries are a common choice for data science tools in civic tech.

Yet, the communities civic tech tools serve are different. Scientific Python maintainers largely build with and for technically-minded users: researchers and developers who install code packages from source, report bugs to developers, or propose a solution to a failing

[1]https://scientific-python.org

Published Jul 13, 2026

Correspondence to
Kristijan Armeni
k.armeni@pm.me

Open Access

Copyright © 2026 Armeni *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

pipeline. In civic tech, the team members and end-users are often policy researchers, outreach specialists, journalists, or NGO advocates who may never interact with the codebase directly. This raises the question: how does civic tech context shape open source maintenance?

We explore this by sharing lessons learned in maintaining the CIB Mango Tree project, an early stage interactive open source toolkit for detecting coordinated inauthentic behavior (CIB) in datasets of social media activity. We begin by examining three design characteristics of civic tech communities: continuous onboarding of volunteers, strong stakeholder proximity, and multidisciplinary participation. We then discuss how familiar open source engineering practices, like release cycles and continuous integration, primarily intended for code quality, take on a more pronounced social layer in this context: facilitating communicating progress, promoting volunteer visibility, and maintaining team momentum beyond developers alone.

2. Civic Tech DC: A mix of voices and skills for public good

Civic Tech DC[2] is a non-partisan, nonprofit community of volunteer technologists, policy experts, researchers, designers, and community leaders passionate about using open-source technology for public good in the Washington, DC area. Founded in 2012 as Code for DC, through Code for America's Brigade program[3], a network of local volunteer chapters, it eventually spun out into an independent nonprofit in 2023 under its current name and brand.

In-person project nights. The core programming of Civic Tech DC is its biweekly in-person project nights. These take place during the evening after work and are open to volunteers of all backgrounds and experience and require only prior online registration. During project nights, current organizers provide a general introduction and outline the structure of the evening. This is followed by short (approximately 30-second) pitches from each active project (typically 7–8 at a given time) introducing their work and the skills they are currently seeking. After the pitches, attendees break out into project groups. New volunteers can join a project if they already know what they are interested in, or attend a general onboarding session where leads provide an overview of civic tech and help match participants' skills to their interests. Either way, no formal commitment is required from volunteers.

2.1. Community design characteristics

These in-person events provide the structural context in which several key design characteristics of the community become visible and reinforced over time. Collectively, they shape the ways of working and technical decisions in maintainership. We will explore three such characteristics (see Table 1 for overview).

Table 1. *Design characteristics of Civic Tech DC community and their effects on maintainership*

Community design characteristic	Description	Effect on maintainership
Continuous onboarding	New volunteers join projects every two weeks at in-person events	Streamlined onboarding, frequent releases, well-scoped tasks, credit contribution as soon as possible

[2] https://civictechdc.org
[3] https://codeforamerica.org

Stakeholder-proximity	Projects develop to address the need of a validated community partner and have a well-defined use case	Focusing on applications, developing tools for bespoke needs, engagement with domain experts
Diverse modes of contribution	Projects attract volunteers with a mix of professional backgrounds (e.g. developers, UX researchers, outreach specialists)	Showcase technical progress with tangible outcomes, communicate practical imports of technical decisions, enable access to latest development features to non-technical team members

Continuous onboarding. Because project nights are open and run on a biweekly cadence, Civic Tech DC is in a constant state of onboarding new volunteers. At every event, new participants join teams with varying levels of experience, while returning members continue ongoing work. This creates a "continuous onboarding" environment in which project teams must quickly pitch the main project goals and introduce new volunteers to the team every two weeks without sacrificing the momentum of ongoing work with recurrent members.

Stakeholder proximity and focus on end-users. Second, all projects are required to be developed in partnership with a validated community partner who helps guide project design and implementation. These partners are nonprofits, community organizations, and advocacy groups with specific real-world needs that can be addressed with digital technology. The "stakeholder-proximity" and focus on bespoke *applications* contrasts with open source projects in scientific computing, characteristic of the Scientific Python community, where developed tools are commonly *general-purpose libraries* serving a broad audience.

For example, maintainers of a scientific visualization library do not need to be *directly* concerned with how any downstream community will be using the visualizations.[4] In contrast, developing a bespoke civic tech application, such as a visualization tool that helps a legal NGO make sense of their portfolio, requires ongoing engagement with domain experts and a deeper understanding of how the tools will be used in practice.

Flat team structure, diverse modes of contributions. Project teams at Civic Tech DC events are typically small and have a relatively flat structure, especially when first taking off. The design is intentional as it lowers the barrier to entry and allows volunteers with different levels of experience to join and contribute quickly. This, combined with the requirement that all projects must have a validated community partner, which can span policy, design, or advocacy disciplines, means developers and coders commonly represent only a subset of the professional backgrounds and expertise involved.

At CIB Mango Tree (see CIB Mango Tree below) we distinguish three groups of participants: *technical contributors* who develop and maintain the Python codebase, *non-technical contributors* (including researchers, designers, and outreach volunteers) who help shape the design of the tests and communicate them to the public, and *end users* who are the target audience. A strong Python development team alone would struggle to build an application that meets the needs of the communities it is trying to serve (e.g. journalists, social media scholars) without consistent outreach and feedback. While true for many open source projects, civic tech projects in particular rely on multiple pathways for contribution beyond code, including research, design, communications, and community engagement [1].

[4] Of course, open source communities building general purpose tools very much care about how their tools are being used by collecting bug reports, issues etc. This is critical to make tools useful. But this type of feedback is geared to make tools more robust rather than tailor them to specific use cases.

3. CIB Mango Tree

CIB Mango Tree[5][6] is a Python-based, open source toolkit for analyzing and detecting coordinated inauthentic behavior in social media datasets. Founded in the summer of 2024, it is designed for researchers, data journalists, fact-checkers, and watchdogs working to identify manipulation and inauthentic activity online. The mission of CIB Mango Tree is to provide non-technical users with a battery of tests to interactively explore and analyze datasets of social media activity for evidence of coordinated inauthentic behavior in their reporting. There are several reasons why the development of open source CIB tools is pertinent.

The research-to-newsroom gap. Originally coined by Facebook around 2018 [4], the term *coordinated inauthentic behavior* in its various definitions generally refers to a group of inauthentic users performing coordinated activity online (e.g. posting, reposting, liking etc.) with a strategic goal to mislead and manipulate public opinion [5], [6]. Computational social scientists have developed a rich body of sophisticated methods to analyze the social media datasets for insights about coordinated behavior [7].

Yet, the experts who research and report on CIB rarely have either the time, resources, or technical capacity to implement such methods from scratch or to execute publicly-available analysis code. In addition, many of the peer-reviewed analyses are specifically tailored to the dataset that was used in research (e.g. a Twitter dataset spanning a time-window around a specific campaign). In most cases, such analyses likely need to be refactored to be applicable to new datasets beyond the original — even if the core analyses rely on general-purpose data science methods.

A fragmented landscape of social media data access and monitoring tools. Social media are one of the main vehicles of public discourse and engagement. As such, social media platforms remain a target of coordinated campaigns that leverage them to sway and manipulate public opinion. A reliable access to data and tools to monitor and analyze such online behavior are the prerequisites in order for independent researchers, data journalists, and fact-checkers to accurately report on it and exert oversight. However, the social media activity data from major platforms are commonly not publicly accessible, offer a limited access, or are locked behind paid-access APIs [8]. Coupled with an increasingly commercialized ecosystem of social media monitoring tools [9], this development weakens accountability and oversight by independent researchers.

A community-developed library of tests. CIB Mango Tree tries to fill this gap by implementing simple, but effective methods (referred to as "tests" in our parlance) into a coherent toolkit. There is no single test that can definitively determine whether a snapshot of activity on social media is coordinated and/or inauthentic. Therefore – at its core – the goal of CIB Mango Tree is to be a library of complementary tests, each of which leverages and highlights different aspects of potential coordination patterns (e.g. copy-pasted content, trending hashtag usage, temporal coordination etc.).

[5] https://cibmangotree.org
[6] https://github.com/civictechdc/cib-mango-tree

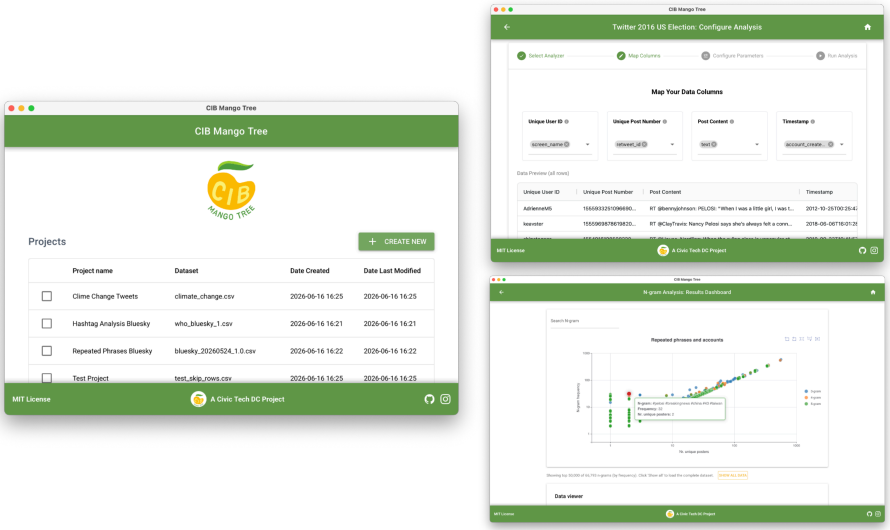


Figure 1. *Graphical user interface of the CIB Mango Tree desktop app (version `v0.11.0`). Users can create and review projects (left), import a dataset and interactively configure chosen analysis (top right) and explore the results in an integrated dashboard (bottom right).*

Through an end-to-end interactive workflow, accessible as a standalone desktop app, CIB Mango Tree allows users to import tabular datasets (`.csv`, `.xlsx`), create a project, select and configure one or multiple tests, run the test and explore the results in an integrated dashboard (Figure 1). As of version `v0.11.0`, CIB Mango Tree relies, among others, on NumPy [10] for numerical computing and Polars[7] for working with tabular data, TinyDB[8] for project and analysis storage, and NiceGUI[9] for the graphical user interface and dashboards.

4. Community-focused maintainership in civic tech

Maintainership in open source is sometimes, if not frequently, experienced and portrayed as thankless, invisible work lacking recognition of the effort [11], [12]. Given the diversity of backgrounds in civic tech where most team members will not look at the commit history, this tension could be expected to be even more pronounced. Yet, three practices most open source maintainers are familiar with all turned out to have elements of double duty as engineering and social practices. We discuss each in turn: release management and schedules as social signals, continuous integration (CI) as external progress visibility, and dependency selection as an onboarding policy.

4.1. Release management and schedules as social signal

Release management and release schedules are standard practice in software development. The primary goal is to distribute a certain version of software to end-users [13]. Teams and projects can adopt different strategies to manage the release schedules, either creating releases at pre-defined times, at regular intervals or in a so-called agile fashion, as soon as new features are added [14].

Occasional, ad-hoc releases. In the very early stages of the project, the release schedule was ad-hoc. An occasional release was created (with a downloadable standalone application as the output) whenever a new feature (e.g. project storage) was added to the toolbox. In this initial ad-hoc workflow, the release had first and foremost the engineering impact: allowing

[7] https://pola.rs/
[8] https://tinydb.readthedocs.io
[9] https://nicegui.io/

the (then) maintainer to keep track of the changes and to streamline the distribution of the application with the new feature to the rest of the team. As the project evolved and gained traction, however, the needs have outgrown this approach.

Increasing frequency, decreasing the scope. One of the main challenges was that our project "users" were mainly internal contributors, consisting of (mostly non-developer) volunteers. They took part in decision-making at the in-person project nights and were thus actively involved in the overall development process. The rather loose connection between the broader team dynamics and development improvements meant that discussions over needed features often felt disconnected. Every two weeks, the project team would discuss many ideas without referring to the current software version or how any would translate to a development roadmap. Over the course of the past year, we decided to adopt a more regular release schedule which included smaller but more frequent releases that keep pace with regular in-person nights.[10]

Community impact: streamlining communication, crediting contributors. Whereas the engineering output of the release remained unchanged (downloadable standalone application), we want to highlight the changes in community dynamics that we observed. First, the more frequent iteration of version labels streamlined the communication in project channels. Over time, we saw non-developer team members adopt the versioning vocabulary in internal discussions beyond debugging issues. For example, our project managers became comfortable referring to a specific version (e.g. `v0.10.0`) when roadmapping the work for the next few months. This made it easier to scope future work. Lightweight urgent bug fixes in `v0.10.0` would be planned for `v0.10.1` whereas new features would go to `v0.11.0` etc. Prior to us having a regular release schedule—and thus a common mental model and vocabulary—this type of prioritization and scoping was more challenging to achieve across diverse team members.

Another important community impact of having a release schedule is that it creates the opportunity to formally credit ongoing volunteer contributions early. A large majority of volunteers who join us in project nights cannot commit to the project long-term. A more frequent release cadence allows us to credit all contributions in the release notes and communications. This also reflects the fact that, in addition to intrinsic motivation (e.g. contributing to tech for public good), many volunteers also have extrinsic motivation for contributing – a demonstrable contribution to an open source project can be a badge of honor or a signal of skills. Over time, in fact, crediting the contributors has become an important driver when deciding to cut a new release.

4.2. Continuous integration as external progress visibility

Introducing frequent changes to software requires the ability to continuously test parts of code for accuracy. This practice, continuous integration (CI), allows developers to iterate rapidly [2]. Nowadays, software tests can be initiated automatically as soon as new contributions are included. As with other practices discussed here, the primary goal of CI is its engineering impact: to ensure ongoing software accuracy without sacrificing developer bandwidth. It's an essential component of a maintainer's toolkit.

Hampering team bandwidth. Prior to setting up the testing infrastructure and a CI pipeline, ongoing CIB Mango Tree releases would regularly contain bugs that were only discovered once another team member attempted to launch the application (e.g. for a demo at an upcoming project night). While such organic bug discovery is part and parcel of improving (open source) software, the lack of testing, however, also carried a community cost. Even fundamental bugs pertaining to the application's core architecture and build systems

[10] Given the volunteer nature of the project and thus limited bandwidth, our releases are spaced a few months apart. Relative to many open source communities, this is still a rather slow release cycle.

were frequently discovered by non-developer team members who had limited insight into the internal workings of the application. This, in turn, meant that a non-negligible amount of time was spent coordinating between developers and other team members in order to accurately report and diagnose the issues.

Community impact: early progress visibility. We eventually set up a minimal suite of unit tests and integration tests of the app.[11] While arguing for CI from an engineering perspective is self-evident, what CI enabled implicitly beyond that was team momentum and progress visibility. Consistent, lengthy troubleshooting discussions, while strictly speaking productive, can leave everyone on the team with a sense of stalled progress. Setting up a test suite and CI allowed us to discover obvious bugs (e.g. related to build failures) early and properly test each release artifact prior to creating download links. This, in turn, allowed us to reliably distribute an internal *preview* of the development version of the application. All team members are able to try the most recent features right as they are added to the development branch thus making most recent progress visible and tangible across the entire team, not just developers.

4.3. Dependency selection as onboarding policy

Open source software allows other developers to inspect the source code, reuse it and innovate on top of or with it. This philosophy is frequently hailed as efficient, as developers can focus on their contribution without having to reinvent the wheel for every problem they encounter. As with any tool, a new dependency in the chain introduces both opportunity (e.g. speed-up in processing time) and risk (if dependency fails, the project fails). Maintainers and developer communities in open source projects thus have to make judicious choices when deciding on introducing new project dependencies. Many factors come into play and to name a few: Does the dependency solve the core problem? Is the project actively maintained? Is it well-scoped for the needs?

Community impact: lowering barriers to entry. When choosing dependencies at CIB Mango Tree, we frequently hit on another, community-informed criterion: how easy will it be for volunteers to contribute to the project? To take an example: building interactive dashboards. Given our target audience (researchers and journalists who do not code), it is important that the application allows interactive exploration of results. How many ways are there to build a dashboard in Python? It turns out, many. Some frameworks emphasize the quick-and-easy setup, while others focus on customizability, scalability, and industry-grade performance. Depending on a team's technical maturity, capacity, and needs one or the other framework might be a better fit.

As an early stage volunteer-driven open source project whose goals are both to deliver a minimal working prototype while *facilitating* successful contributions of new volunteers to civic tech, technical leadership must factor in whether specific coding frameworks and tools have broad familiarity and accessibility within the volunteer community. Decisions often need to be made quickly, with a strong preference for tools that lower barriers to entry, since there is limited time to train new contributors before the next group of participants arrives at project night. As such, maintainability and ease of adoption of each dependency matter in order to have an onboarding process with minimal friction.

5. Lessons learned from design constraints of civic tech

For maintainers entering civic tech from scientific computing, there are three main lessons to be learned from adopting open source practices in civic tech projects.

[11] In our case, we used GitHub Actions which are a popular choice at the time of writing, but the main point is not specific to this choice.

Be willing to revisit technical assumptions. Given the application and user-focused development (see Section 1 and Table 1), it is important to be willing to actively monitor what the needs of the target organization or community are and revise technical assumptions if needed. For example, a command-line tool (as is common in scientific computing) might be a lightweight solution for a specific application, but if civic partners prefer a graphical user interface for their needs, then that should be factored in technical decisions and roadmap.

Leverage open source practices as social infrastructure. While remaining flexible about specific technical choices is important, we nevertheless recommend to implement the core practices of collaborative open source (test-driven development, version control, code reviews etc.) and leverage them for strengthening the broader community. As shared in this paper, even practices that are initially designed for bespoke engineering purposes and quality control can yield community benefits beyond developers.

Expect greater maintainer visibility. In our experience, the emphasized social angle coupled with small and flat teams in our civic tech community also naturally leads to a greater visibility of maintainers' work, surfacing aspects of developer relations (DevRel) roles [15] where the needs of maintainers and developers have to be continuously communicated with the broader team in order to move the project forward.

6. Limitations

Generalizability to different community contexts. The experience of open source maintainership shared in this report is limited to a specific setting: a community of volunteers that meets primarily in-person with smaller teams and a relatively flat structure. This does not readily generalize to open source communities that primarily work remotely and across diverse timezones where dynamics and needs will differ. Likewise, in larger (and possibly non-volunteer) civic tech teams, the division of labor and professionalization of dedicated community roles, for example community management [16] and developer relations [15], would be beneficial. In such a setting, it would be ill-advised to expect maintainers to also simultaneously perform community building roles.

Effects of AI-assisted development. Second, while we purposefully centered our report on the aspects of human-to-human collaboration in open source and civic tech, the proliferation of AI-assisted software development [17] can not be ignored. How does the availability of code generation tools change our recommendations? It most clearly affects the consideration of onboarding friction when selecting project dependencies (Section 4). In our experience, it pays off when choosing dependencies with a less steep learning curve even if at the expense of performance. However, the availability of code-generation tools where the user produces code through natural language input reduces the uptake friction that motivated our recommendation (for example, users can gain an understanding of the codebase by feeding it as input to generative AI for summarization etc.). In that sense, choosing dependencies with steep learning curve is perhaps no longer such a bottleneck. Yet, uncritical adoption of AI tooling can backfire. In open source communities specifically, it has led to increases in maintainer burden through unsustainable and unreviewable volume of code contributions [18], [19]. We thus remain cautious and advise a critical, pro-social adoption of AI tooling in civic tech by monitoring the needs of collaborators first before delegating work to AI.

7. Conclusion

Design characteristics of volunteer-run projects in the Civic Tech DC reshape what successful open source maintainership looks like. Small teams with limited hierarchy, high turnover of volunteers, mix of professional interests beyond coders, and a focus on

developing applications all bring to the fore the social angle of open source maintainership and practices. Maintainership effectively blends with elements of community management [16]: how we adopt a certain practice depends not only on its technical merits but also on whether it facilitates participation, sustains engagement, and builds a sense of shared ownership and agency across a diverse volunteer community.

8. Disclosures

Portions of this work were assisted using generative AI tools (Qwen 3.6 Plus, author KA). The tools were used for brainstorming and reviewing narrative structure and refining language. All outputs were reviewed, verified, and revised by the author(s), who take full responsibility for the accuracy and integrity of the final content.

References

[1] C. Harrell, *A civic technologist's practice guide*. San Francisco, California: Five Seven Five Books, 2020.

[2] N. Eghbal, *Working in public: the making and maintenance of open source software*, First Edition. San Francisco: Stripe Press, 2020.

[3] K. Fogel, *Producing open source software: How to run a successful free software project*, Second. O'Reilly Media, 2023. [Online]. Available: http://www.producingoss.com/

[4] N. Gleicher, "Removing bad actors on Facebook," techreport, 2018. [Online]. Available: https://about.fb.com/news/2018/07/removing-bad-actors-on-facebook/

[5] K. Starbird, A. Arif, and T. Wilson, "Disinformation as collaborative work: Surfacing the participatory nature of strategic information operations," *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, pp. 1–26, 2019, doi: 10.1145/3359229.

[6] A. Romero Vicente, "Coordinated inauthentic behaviour detection tree," techreport, 2024. [Online]. Available: https://www.disinfo.eu/publications/coordinated-inauthentic-behaviour-detection-tree/

[7] L. Mannocci, M. Mazza, A. Monreale, M. Tesconi, and S. Cresci, "Detection and characterization of coordinated online behavior: A survey." [Online]. Available: http://arxiv.org/abs/2408.01257

[8] "Better access: Data for the common good," techreport, 2025. [Online]. Available: https://kgi.georgetown.edu/research-and-commentary/better-access/

[9] C. Miles, "From dashboards to data acquisition: Charting the social media monitoring market," Washington, D.C., Summary report, 2025. [Online]. Available: https://iddp.gwu.edu/dashboard-data-acquisition

[10] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[11] J. Meluso, A. Casari, K. McLaughlin, and M. Z. Trujillo, "Invisible labor in open source software ecosystems," *Proceedings of the ACM on Human-Computer Interaction*, vol. 9, no. 7, pp. 1–32, 2025, doi: 10.1145/3757417.

[12] D. Buytaert, "25 years of Drupal: what I've learned." [Online]. Available: https://dri.es/25-years-of-drupal-what-i-have-learned

[13] I. Sommerville, *Software engineering*, Tenth edition. in Always learning. Boston Columbus Indianapolis New York San Francisco Hoboken Amsterdam Cape Town Dubai London: Pearson, 2016.

[14] A. Cesar Brandão Gomes Da Silva, G. De Figueiredo Carneiro, F. Brito E Abreu, and M. Pessoa Monteiro, "Frequent releases in open source software: A systematic review," *Information*, vol. 8, no. 3, p. 109, 2017, doi: 10.3390/info8030109.

[15] R. Oliveira, C. Ajala, D. Viana, B. Cafeo, and A. Fontão, "Developer Relations (DevRel) roles: An exploratory study on practitioners' opinions," in *Brazilian Symposium on Software Engineering*, Joinville Brazil, 2021, pp. 363–367. doi: 10.1145/3474624.3474628.

[16] M. Sharan *et al.*, "Professionalising community management roles in interdisciplinary research projects." [Online]. Available: https://arxiv.org/abs/2409.00108

[17] C. Huyen, *AI engineering: building applications with foundation models*. Sebastopol, CA: O'Reilly, 2025.

[18] S. Afroz, C. Miller, T. Menezes, E. Gilmour, A. Sarma, and Z. Feng, ""AI Slop is DDoSing open source": Understanding the impact of AI-Generated contributions on open source sustainability." [Online]. Available: https://arxiv.org/abs/2607.04003

[19] A. Wolf, "Welcome to the eternal september of open source. Here's what we plan to do for maintainers.." [Online]. Available: https://github.blog/open-source/maintainers/welcome-to-the-eternal-september-of-open-source-heres-what-we-plan-to-do-for-maintainers/

Open Access | https://doi.org/10.25080/yxan2279

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Everything That Breaks When You Put an LLM Agent in Production

Sruthi Pisipati[1], and **Haris Javed**[1]

[1]Boston Scientific Corporation

Abstract

Agentic AI systems often perform well in interactive prototypes and small evaluations, but they behave like distributed systems when deployed in production-scale workflows. We present a case study from a regulated record-processing environment where LLM-based agents were used for structured classification in two related workflow families: a high-throughput single-agent batch workflow and a staged multi-agent review workflow. The goal of the study is not to introduce a new prompting method or model architecture. Instead, we evaluate the engineering controls needed to parallelize record-level agent execution while keeping it observable, recoverable, and tunable at scale.

Across hundreds of records and thousands of model and retrieval calls, reliability depended less on prompt design alone and more on production controls: separating local worker parallelism from hosted API concurrency, enforcing per-record wall-clock budgets, distinguishing transient from non-transient failures, preserving progress through checkpoints, and recording structured operational metadata. In the single-agent workflow, hosted API concurrency was the dominant throughput bottleneck; tuning it improved throughput by nearly 5x relative to the initial baseline while preserving completion yield.

In the multi-agent workflow, a verifier stage and multi-source evidence gathering produced richer metadata but also introduced orchestration cost and partial-completion behavior. Later probes showed that record-level concurrency, verifier concurrency caps, token pressure, and reasoning effort moved the system across different throughput, cost, yield, and strict label-agreement operating points. We argue that production agent deployment should be formulated as a constrained multi-objective systems problem, rather than as model-quality maximization alone.

Keywords llm agents, agentic ai, production ai, reliability engineering, distributed systems, observability, batch processing, scientific computing

Published Jul 13, 2026

Correspondence to
Sruthi Pisipati
sruthi.pisipati@gmail.com

Open Access

Copyright © 2026 Pisipati & Javed. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Agentic AI systems are increasingly used to automate scientific, industrial, and regulated workflows. During early development, these systems are commonly evaluated through interactive applications, small test sets, or one-record-at-a-time demonstrations. Under those conditions, failures are visible and recoverable: a developer can inspect a prompt, rerun an input, adjust the output schema, or manually retry an API call.

Production environments expose a different class of problems. When the same agent is run across hundreds or thousands of records, it must coordinate concurrent workers, hosted LLM APIs, managed retrieval services such as search or vector-index systems that fetch supporting evidence for the model, cloud object storage for input files, outputs, and checkpoints, long-running execution, timeouts, retries, and partial outputs. Rare failures become routine at scale. Worse, some failures are not hard crashes. An agent stage may catch an

exception, return a fallback answer, or complete the batch while internal errors remain visible only in logs.

This paper studies those reliability problems through a real production case study. We evaluate two related workflows: a single-agent batch classification pipeline that processes each record independently, and a multi-agent review workflow that retrieves supporting evidence, synthesizes a structured prediction, verifies the prediction, attempts a bounded repair when needed, and routes outputs by confidence. We frame these workflows as production systems rather than isolated model calls. That framing follows a broader lesson from production machine learning: systems fail not only because of model behavior, but because data dependencies, validation, serving, monitoring, and operational controls are handled ad hoc [1], [2].

The underlying task is a structured multi-label classification problem over access-controlled records. Each input record contains free-text narrative fields and structured metadata. The system must return zero or more normalized labels from a controlled vocabulary, along with metadata that supports review and downstream persistence. In the multi-agent experiments, each predicted label has an identifier and a normalized three-component tuple. This paper reports only aggregate metrics and synthetic descriptions of the task structure; it does not disclose record text, prompts, retrieved evidence, raw outputs, or label content that could expose sensitive information. This abstraction is intentional: many scientific and industrial record-processing workflows have the same shape even when the label vocabulary and source records differ.

The paper makes three contributions. First, we report operational results from single-agent and multi-agent batch experiments that parallelize record processing, including throughput, yield, timeout behavior, checkpoint recovery, failure signals, token-derived cost estimates, and strict-agreement guardrails. Second, we propose a failure taxonomy and measurement set that separates final record failures from recovered internal events and transient infrastructure signals. Third, we formulate production agent tuning as a constrained multi-objective optimization problem over worker count, API concurrency, timeout budget, retry policy, checkpoint cadence, verifier stages, fan-out, and reasoning effort. This last framing is the paper's central methodological claim: production agent deployment should select a bounded operating point under reliability, cost, and auditability constraints, not simply maximize model complexity or parallelism.

The central problem is simple: an LLM agent that works once is not necessarily a production system. To deploy agents in high-volume environments, practitioners must design for the ways they fail at scale.

Table 1. *What breaks when a record-processing agent is parallelized.*

Scale-up pressure	What breaks or becomes ambiguous	Measurement or control used here
More local workers	Hosted API, retrieval, network, or storage bottlenecks dominate local parallelism	Separate worker count from API concurrency
Long-tail records	A few slow records can hold the batch open	Per-record wall-clock timeout budgets
Partial completion	A run can finish most records but still exit with failures	Completion yield, saved decisions, and failure logs
Hidden internal recovery	The final output may succeed while an agent stage recovered from an exception	Recovered-internal-error counters

Provider or policy errors	Some failures are non-transient and should not be retried indefinitely	Named error taxonomy and retry limits
Cost and token pressure	Faster configurations can increase model calls or peak token load	Token totals, peak tokens per minute (TPM), and cost per usable record
Strict agreement under concurrency	More parallel execution does not guarantee better or worse agreement	Repeat probes and strict label-set agreement

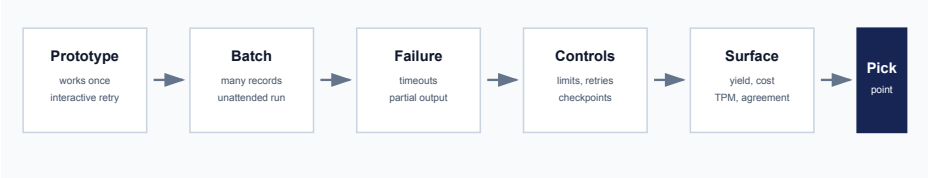


Figure 1. *Conceptual contribution of the paper. Production deployment exposes failure modes that require reliability controls and an empirical optimization surface before choosing an operating point.*

The synthesis diagram is the organizing argument for the rest of the paper: a prototype that succeeds interactively must become a measured batch system before an operating point can be selected.

2. Background

Recent work on LLM reasoning and agentic systems has shown that language models can do more than single-turn text generation. Chain-of-thought prompting demonstrated that intermediate reasoning can improve model performance on complex tasks [3]. ReAct-style agents combine reasoning with actions such as search and tool use [4]. Retrieval-augmented generation extends this pattern by grounding generation in external knowledge sources [5]. These approaches motivate agents that can retrieve candidates, reason over free text and metadata, and produce structured outputs.

Much of the agent literature, however, focuses on reasoning quality or interaction patterns. Production deployment introduces additional concerns: service quotas, network failures, retries, nondeterministic replay, malformed structured outputs, provider content filtering, and long-running partial completion. The distributed-systems literature provides a useful lens. Large-scale systems amplify rare latency and failure events; a small probability of slow or failed calls can become common when many calls are made concurrently [6]. Techniques such as bounded concurrency, timeouts, backoff with jitter, and checkpointing are standard reliability controls [7]. We apply these ideas to LLM agent workflows, where each record may trigger multiple external calls and failures may occur inside agent reasoning paths rather than at obvious service boundaries. Although the measured rates in this study are workflow-specific, the architectural pressures are common to many production agent systems: hosted model APIs, managed retrieval, batch execution, verifier stages, rate limits, and persistent outputs.

3. From Interactive Prototype To Production Batch Agent

The system began as an interactive Streamlit workflow. That interface was useful for early validation: users could load a small set of records, run the agent, inspect outputs, and manually rerun failures. This was the right level of control for proof-of-concept work, but it did not meet the production requirement: a user should be able to start a large batch, leave it

running, and return only when the system has completed or produced a clear intervention signal.

Table 2. *Prototype versus production assumptions.*

Prototype workflow	Production batch workflow
Tens of records	Hundreds or thousands of records
Human monitors execution	Unattended execution
Manual rerun on failure	Automated retry and failure classification
UI inspection during execution	Structured logs and run summaries
Session state is acceptable	Durable checkpoints and outputs
Stalls may be noticed by a user	Stalls require wall-clock detection
Success judged interactively	Success measurable and auditable

This shift changed the engineering problem. Multiprocessing introduced many simultaneous agent executions. In this paper, a worker is a local process or task that handles records, while hosted API concurrency is the explicit limit on simultaneous calls to shared external services such as hosted LLM APIs or managed retrieval. Increasing local workers improved local parallelism, but also increased pressure on hosted LLM APIs, managed search, network connections, and object storage. Production reliability therefore required separating compute parallelism from external-service concurrency.

It also changed how failures had to be represented. A production run needs structured events with record identifiers, failure types, timestamps, retry history, and final status. Internal errors recovered by the agent still need to be counted, because a clean final success count can hide important operational behavior.

4. Workflow Overview

We evaluated two related workflow families in an evolutionary line for scaling record-level agent work. The single-agent batch workflow processes each record independently using one LLM-based agent. The runner loads records from spreadsheet or cloud object storage, creates record-level tasks, executes them in parallel, and writes JSON outputs, spreadsheet outputs, failure logs, checkpoints, and run summaries.

The multi-agent workflow extends the same scaling problem with additional stages rather than serving as an unrelated alternative. It separates responsibilities across retrieval, reasoning, synthesis, verification, repair, and confidence routing. Retrieval gathers candidate evidence, synthesis assembles a final structured prediction, verification checks the proposed prediction, repair is one bounded correction attempt after a verifier objection, and confidence routing assigns the result to a review tier or escalation path. A verifier is a stage that reviews proposed classifications and either passes them, requests repair, or escalates the record for additional review. Fan-out means branching a record through multiple evidence paths or agent calls, such as free-text narrative and structured fields already present in the input record, before combining the results into one final classification. The later verifier-cap and token-pressure probes used a revised implementation of this multi-agent workflow with an explicit shared verifier-call limiter and additional token instrumentation. We treat it as an evolutionary runtime revision, not as a directly comparable third task architecture.

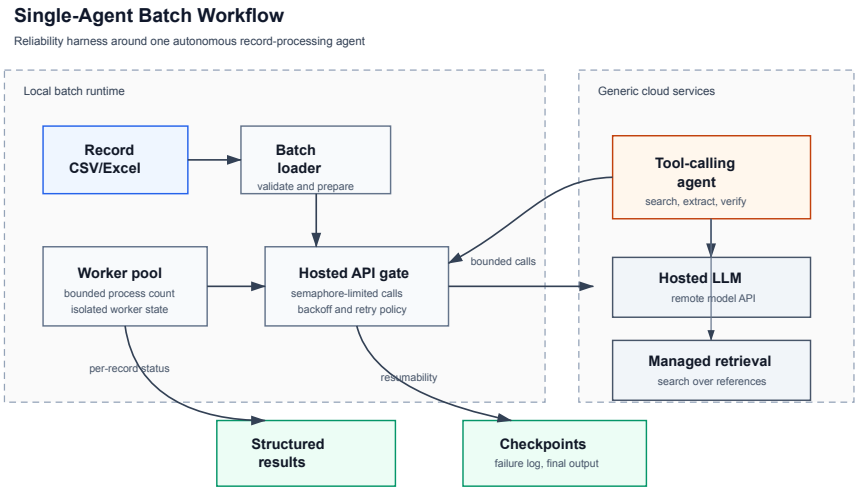


Figure 2. *Runtime reliability pattern for the single-agent batch workflow. The batch harness separates worker parallelism from hosted API concurrency and records structured outputs, failure logs, and resumable checkpoints.*

The important control in the single-agent workflow figure is the API gate between local workers and shared hosted services. It prevents the number of local record workers from becoming the same as the number of simultaneous external calls.

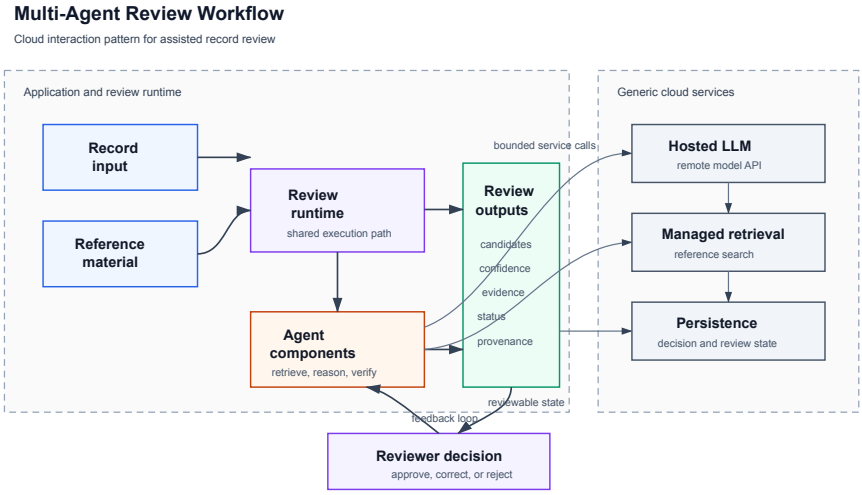


Figure 3. *Runtime reliability pattern for the multi-agent workflow. Multiple agent stages interact with hosted retrieval, LLM, and persistence services to produce confidence-aware structured outputs.*

The multi-agent workflow figure shows how review metadata is added, but it also shows why there are more shared dependencies and more places where partial completion can occur.

5. Failure Taxonomy

At production scale, “failed” is not a single state. The system needs a taxonomy that distinguishes transient infrastructure errors, provider or policy failures, timeouts, malformed outputs, recovered internal exceptions, and final unrecoverable record failures.

We organized the taxonomy around standard batch-system failure classes, such as success, retryable infrastructure errors, timeouts, and final failures. During operational debugging, we refined it to capture agent-specific signals that otherwise disappeared inside logs, such as provider policy failures, malformed model outputs, and recovered internal exceptions. The result is a reusable observability schema rather than a post-hoc list of one-off incidents.

Table 3. *Failure taxonomy used for production agent observability.*

Failure class	Description	Typical handling
`SUCCESS`	Record completed with usable structured output	Count as completed
`TRANSIENT_ERROR`	Temporary service or network issue	Retry with backoff and jitter
`RATE_LIMIT`	External quota or throttling event	Backoff and reduce concurrency
`TIMEOUT`	Record exceeded wall-clock task budget	Stop task and log structured failure
`NON_TRANSIENT_PROVIDER_ERROR`	Provider rejected request or output	Avoid unbounded retry
`OUTPUT_LIMIT_ERROR`	Model hit output or token limit	Retry with adjusted limits or classify
`VALIDATION_ERROR`	Structured response invalid or incomplete	Attempt repair or fallback
`RECOVERED_INTERNAL_ERROR`	Agent stage failed but workflow recovered	Count separately from final failures
`AGENT_INTERNAL_ERROR`	Agent failed and record could not complete	Mark final record failure
`PARTIAL_BATCH_COMPLETION`	Some records completed and some failed	Preserve successes and support targeted retry
`INTERRUPTED_RUN`	Process stopped before completion	Resume from checkpoint

This taxonomy separates final outcomes from internal operational signals. That distinction matters because an internal content-filter event may be recovered, while a batch with a perfect final success count may still contain reliability signals worth monitoring.

6. Experimental Design

The experiments were designed to evaluate operational reliability rather than broad model accuracy. We measured whether workflows completed long-running batches, preserved partial progress, exposed internal error signals, and supported configuration tuning. The experiments are best read as calibration probes for a single systems question: which bounded configuration satisfies yield, throughput, cost, token-pressure, and auditability constraints for a production agent workflow?

Table 4. *Experimental workloads.*

Workflow	Dataset	Experiment size	Purpose
Single-agent optimization sweeps	1,000-record access-controlled source file	200 records per run	Worker/API throughput and cost
Single-agent output-stability guardrail	Labeled calibration subset	12 records, 3 repeats per configuration	Strict-agreement stability under higher concurrency

Single-agent timeout tests	Access-controlled source-file subset	50 records per run	Timeout budget calibration
Single-agent checkpoint/replay tests	Access-controlled source-file subsets	300-record checkpoint run and 367-record replay audit	Checkpoint recovery and replay behavior
Multi-agent review	Access-controlled review sample	Four 50-record variant runs	Verifier and fan-out reliability
Multi-agent structured prediction	200 reference-labeled records	Two 200-record baseline runs	Multi-label classification baseline
Multi-agent concurrency stability	100 known-solvable records	3 concurrency settings x 100 records x 3 repeats = 900 record-runs	Concurrency stability and repeat consistency
Multi-agent verifier concurrency cap sweep	100 known-solvable records	5 cap settings x 100 records x 3 repeats = 1,500 record-runs	Verifier concurrency cap behavior
Multi-agent token-pressure stress tests	100 known-solvable records	4 configurations x 100 records = 400 record-runs	Reasoning-effort and token-pressure behavior

A record-run is one record processed once under one configuration and repeat. A known-solvable cohort is a deliberately selected regression set: records whose label-identifier sets matched the reference sets in a previous run and that are therefore useful for testing whether a new configuration destabilizes previously successful behavior. Selection by identifier-set agreement does not imply that the stricter three-component tuples also matched. A verifier concurrency cap is the maximum number of verifier-stage model calls allowed to run at the same time.

Comparisons within a sweep used the same selected record cohort. Most single-agent worker and API sweep points, and each token-pressure stress configuration, were observed once; experiments with repeated runs are identified explicitly. The single-agent and multi-agent campaigns used different source cohorts, execution hosts, code paths, and service-call graphs, so their absolute throughput and agreement values should not be interpreted as a head-to-head architecture benchmark. The later verifier-cap and token-pressure results should likewise be compared within their revised multi-agent runtime rather than against absolute agreement levels from the earlier multi-agent implementation.

We use *reference agreement*, *stability*, and *regression* for different questions. Reference agreement compares predictions with labels on the 200-record reference-labeled sample; it is not a comprehensive estimate of downstream accuracy. Stability asks whether repeated runs on the same records produce the same label sets. Regression probes ask whether configurations preserve behavior on records selected from prior successes. The 12-record guardrail and 100-record known-solvable cohorts therefore do not estimate population accuracy.

The repeated configurations are reported as pooled record-run counts, mean runtime, and repeat consistency. Reprocessing the same fixed cohort does not create independent samples from a broader population, so these repeats are not used to claim statistical equivalence or a population-level confidence interval. Configurations observed once provide no run-to-run variance estimate; small differences between operating points should therefore be treated as descriptive.

We report aggregate operational measurements only: configuration settings, record counts, completion status, wall-clock runtime, timeout and error counts, token totals, estimated

cost, checkpoint recovery behavior, and verifier or routing summaries. Source narratives, prompts, retrieved evidence, raw logs, per-record outputs, and checkpoint files are not published because they may contain sensitive regulated information.

Token pressure means the rate at which prompt, completion, reasoning, and embedding tokens are consumed by hosted model services. We summarize this as peak tokens per minute (TPM) over a 60-second window. Reasoning effort is a model configuration that changes the internal reasoning budget available to the model when the provider exposes such a setting. It is distinct from workflow confidence tiers, severity labels, or human review categories.

Cost estimates are market approximations based on a June 12, 2026 snapshot of public list prices: GPT-5 input tokens at USD 1.25 per 1M tokens, GPT-5 output tokens at USD 10.00 per 1M tokens, and text-embedding-3-small at USD 0.02 per 1M tokens [8], [9]. These estimates do not reflect negotiated enterprise pricing, Azure-specific billing, cached-input discounts, reserved capacity, or confidential commercial terms.

Three related controls were held outside the experimental sweeps. The API concurrency gate bounded simultaneous in-flight calls, while the runner separately classified rate-limit responses; the reported single-agent sweeps recorded no rate-limit errors, so they do not characterize behavior beyond quota saturation. Temperature remained at each workflow's configured provider-compatible setting and was not varied. Prompt caching was not instrumented as a separate treatment or cost category, and the cost estimates do not assume cached-input discounts.

In the multi-agent structured-prediction experiments, we evaluate two representations of each predicted label: its identifier and its normalized three-component tuple. We refer to strict set comparison of those representations as exact label-ID match and exact label-triplet match, respectively.

Table 5. *Metrics used in the experiments.*

Metric	Meaning
Completion yield	Successful usable records divided by attempted records
Usable throughput	Usable records divided by wall-clock runtime
Usable predictions per minute	Records with non-empty predicted label sets divided by wall-clock runtime in minutes
Timeout errors	Records terminated by wall-clock budget
Checkpoint recovery	Records preserved rather than reprocessed after interruption
Token usage	Prompt, completion, total, and embedding tokens
Estimated cost	Token-derived public-pricing approximation
Prediction yield	Records with a non-empty predicted label set divided by attempted records
Exact label-ID match	Predicted label-identifier set exactly equals the reference identifier set
Exact label-triplet match	Predicted normalized three-component tuple set exactly equals the reference tuple set
Repeat consistency	Records whose predicted label set remains stable across repeated runs
Peak TPM	Maximum observed tokens per minute in a 60-second window
Reasoning tokens	Completion-side internal reasoning tokens reported by the model provider when available

The exact-match metrics are intentionally strict. For example, if the reference label-ID set is `{A, B}`, then `{B, A}` is an exact label-ID match because the set is identical, but `{A}` fails because it is missing `B`, and `{A, B, C}` fails because it adds `C`. Separately, if a reference label has the normalized tuple `(term_1, context_1, outcome_1)`, then `(term_1, context_2, outcome_1)` is not an exact label-triplet match even when its associated label identifier is correct. This strictness makes the metrics useful for stability and regression testing, but it can understate partial or semantically close agreement.

7. Single-Agent Results

The initial 200-record single-agent baseline used 8 workers and hosted API concurrency 3. It completed 199 of 200 records, with one timeout, in 93.7 minutes. This produced 2.125 usable records per minute. Later sweeps used the same 200-record scale to estimate how much performance could be recovered through configuration tuning.

7.1. Worker Count

API concurrency was fixed at 3 while worker count varied.

Table 6. *Worker sweep with hosted API concurrency fixed at 3.*

Workers	API concurrency	Records	Successful	Runtime min	Usable records/min
3	3	200	200	63.4	3.157
4	3	200	200	62.7	3.191
6	3	200	200	63.1	3.170
8	3	200	200	61.6	3.248
12	3	200	200	59.3	3.373

Increasing worker count helped only modestly once hosted API concurrency was fixed. The change from 3 to 12 workers improved throughput from 3.157 to 3.373 usable records per minute, indicating that local worker parallelism was not the dominant bottleneck.

7.2. Hosted API Concurrency

Worker count was fixed at 12 while hosted API concurrency varied.

Table 7. *Hosted API concurrency sweep with worker count fixed at 12.*

Workers	API concurrency	Records	Successful	Runtime min	Usable records/min
12	1	200	200	168.3	1.19
12	2	200	200	81.9	2.44
12	3	200	200	54.0	3.71
12	4	200	200	51.8	3.86
12	6	200	200	38.2	5.23
12	8	200	200	27.5	7.29
12	12	200	200	19.0	10.52

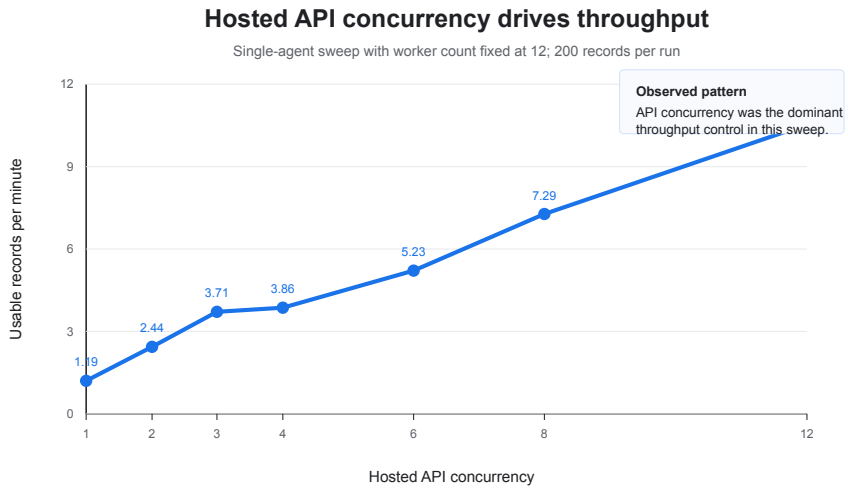


Figure 4. *Hosted API concurrency response for the single-agent workflow. With worker count fixed at 12, throughput increased from 1.19 usable records per minute at API concurrency 1 to 10.52 usable records per minute at API concurrency 12.*

Hosted API concurrency was the dominant throughput bottleneck. Throughput increased from 1.19 usable records per minute at API concurrency 1 to 10.52 at API concurrency 12, while completion yield remained 100% and no timeout, rate-limit, or connection failures were observed in these runs. Relative to the initial 8-worker/API-concurrency-3 baseline, the tuned 12-worker/API-concurrency-12 setting improved throughput by approximately 4.95x.

The best single-agent run consumed 13,745,153 chat prompt tokens, 685,051 chat completion tokens, and 6,843 embedding tokens. Using the public-pricing approximation described above, the estimated cost was approximately USD 24 for the 200-record run, or about USD 0.12 per record.

7.3. Output-Stability Guardrail, Timeouts, Checkpointing, And Replay

Because full reference labels were not available for all 1,000 records, we used a 12-record labeled calibration subset to test whether higher concurrency changed strict label agreement on a small known-labeled sample. Each configuration was run three times, producing 36 record-level comparisons per configuration. This is an output-stability guardrail, not evidence of general quality preservation.

Table 8. *Single-agent output-stability guardrail on 12 labeled records.*

Config	Workers	API concurrency	Repeats	Record-runs	Exact matches	Exact match rate	Consistency rate
`w12_a3`	12	3	3	36	33	91.67%	100.00%
`w12_a12`	12	12	3	36	34	94.44%	91.67%

The observed exact-match rate was similar across the two settings: 91.67% for `w12_a3` and 94.44% for `w12_a12`. Repeat consistency decreased from 100.00% to 91.67%, corresponding to one of 12 records. Because the subset contains only 12 labeled records, these mixed observations should be interpreted as a guardrail sanity check for concurrency changes, not as evidence of general quality preservation or as a definitive accuracy benchmark.

Timeout experiments showed why wall-clock budgets must be calibrated rather than minimized. A 60-second budget completed only 17 of 50 records and produced 33 timeout errors. A 120-second budget completed all 50 records in 23.3 minutes. A 300-second budget also completed all 50 records in 20.5 minutes. The 60-second budget was faster but unacceptable under a near-100% completion requirement.

Checkpointing avoided repeated work. In a 300-record run, a checkpoint preserved the first 200 completed records. Resuming from that checkpoint processed only the remaining 100 records, which completed successfully in 26.2 minutes. Historical replay also showed why recovered internal errors must be tracked separately from final failures. Replaying 367 records associated with historical agent-internal failures produced 367 final successes and zero top-level internal failures, but surfaced one recovered content-filter event.

8. Multi-Agent Results

The multi-agent results are organized in three phases. First, we compare review-workflow variants to understand how verification and fan-out affect reliability signals. Second, we evaluate the workflow as a multi-label classification task against reference labels. Third, we use known-solvable records to stress concurrency, verifier concurrency caps, token pressure, and reasoning effort as operational controls. The verifier-cap and token-pressure experiments in the third phase used the revised multi-agent runtime described above. This sequence separates operational reliability, strict label agreement, and regression-style pressure testing. The agreement-related rows should therefore be read according to their workload and runtime version: the 200-record baseline estimates agreement with available reference labels, while the known-solvable probes test whether previously successful cases remain stable under pressure.

8.1. Verifier And Fan-Out Runs

This table compares review-workflow variants, not reasoning-effort settings. The confidence columns report workflow-assigned confidence tiers for the proposed classification; they are not model reasoning levels or severity labels. In the table, “structured input fields” means metadata fields already present in the input record. The workflow does not require these fields; the ON/OFF runs are an ablation that tests whether adding this optional evidence path changes throughput, completion, or review metadata. These fields are not the reference labels used for scoring. The verifier columns count how many records passed verification, required one repair attempt, or were escalated.

Table 9. *Multi-agent verifier and fan-out reliability runs.*

Run	Saved/ total	Wall sec	Records/ min	Confidence high	Confidence medium	Confidence low	Verifier pass	Repair	Escalate
Verifier ON, retry 1	48/50	1188.4	2.53	22	23	3	45	3	0
Verifier OFF	49/50	978.6	3.07	31	18	0	0	0	0
Fan-out, structured fields ON	48/50	1347.2	2.23	23	19	6	44	2	2

Fan-out, structured fields OFF	48/50	1183.4	2.54	26	20	2	46	1	1

Verifier and fan-out settings produced richer review metadata, but increased orchestration cost and still produced partial completion. In the review-workflow comparison table, turning the verifier off improved throughput from 2.53 to 3.07 records per minute relative to the verifier-on run, but removed verifier pass, repair, and escalation metadata.

Table 10. *Multi-agent operational error signals.*

Run	Content-filter signals	Read-timeout signals	Connection-pool warnings	Final errors
Fan-out, structured fields OFF	6	0	1	2
Fan-out, structured fields ON	3	2	4	2
Verifier OFF	8	0	5	1
Verifier ON, retry 1	8	0	4	2

Provider content-filter signals were the most frequent named error category in the operational-error table. One fan-out configuration also produced read-timeout signals. Connection-pool warnings were operational signals but did not by themselves terminate the batch.

8.2. Multi-Label Classification Baseline

We next evaluated the multi-agent workflow as a multi-label classification task. Each input record had a reference set of target labels, and the system predicted a set of labels. Each label is represented by an identifier and a normalized three-component tuple. A prediction counted as an exact label-ID match when the predicted identifier set equaled the reference identifier set. The stricter exact label-triplet metric required the full predicted tuple set to equal the reference tuple set. Prediction yield measured whether the workflow produced at least one predicted label for a record.

These metrics evaluate agreement with the experiment's reference labels. We use exact set agreement intentionally as a strict stability measure; partial semantic correctness is outside the scope of this paper. The metrics should not be interpreted as a comprehensive assessment of downstream workflow quality, domain-level correctness, or real-world review outcomes.

Both baseline runs used medium reasoning, the verifier, and the optional structured-input evidence path; record-level concurrency was the varied setting.

Table 11. *Multi-agent 200-record multi-label classification baseline.*

Concurrency	Saved/ total	Hard errors	Prediction yield	Exact label-ID	Exact label-triplet	Wall min	Usable predictions/ min	Est. cost
2	193/200	7	187/200 (93.5%)	96/200 (48.0%)	47/200 (23.5%)	87.3	2.14	USD 10.22

6	196/200	4	193/200 (96.5%)	100/200 (50.0%)	51/200 (25.5%)	28.8	6.70	USD 10.32

Increasing record-level concurrency from 2 to 6 improved throughput by approximately 3.1x, from 2.14 to 6.70 usable predictions per minute, and improved prediction yield from 93.5% to 96.5%. Exact label-ID agreement increased from 48.0% to 50.0%, while exact label-triplet agreement increased from 23.5% to 25.5%. Both process runs returned nonzero exit codes because a small number of records encountered provider content-filter failures, but the runs still saved most record-level decisions and produced usable aggregate results.

8.3. Known-Solvable Concurrency Stability Probe

To test whether higher record-level concurrency degraded the workflow on records it had already shown it could solve, we constructed a 100-record known-solvable cohort from the 200-record concurrency-6 baseline. The cohort included records where the predicted label-ID set exactly matched the reference label-ID set in that run. This cohort is useful for stability testing, but it is not an unbiased estimate of overall classification accuracy because it was selected from prior successes. Its purpose is closer to a regression test: a configuration that destabilizes these cases is operationally concerning, but a high score on this cohort does not imply high accuracy on arbitrary future records.

Each concurrency setting was run three times, producing 300 record-runs per setting.

Table 12. *Known-solvable concurrency stability probe.*

Concurrency	Repeats	Record-runs	Prediction yield	Exact label-ID	Exact label-triplet	Triplet consistency	Avg wall min/100	Usable predictions/min	Est. cost/100 usable
2	3	300	288/300 (96.0%)	279/300 (93.0%)	102/300 (34.0%)	93.0%	46.9	2.05	USD 4.99
6	3	300	292/300 (97.3%)	278/300 (92.7%)	96/300 (32.0%)	93.0%	16.0	6.09	USD 4.98
12	3	300	288/300 (96.0%)	281/300 (93.7%)	103/300 (34.3%)	96.0%	10.8	8.92	USD 5.01

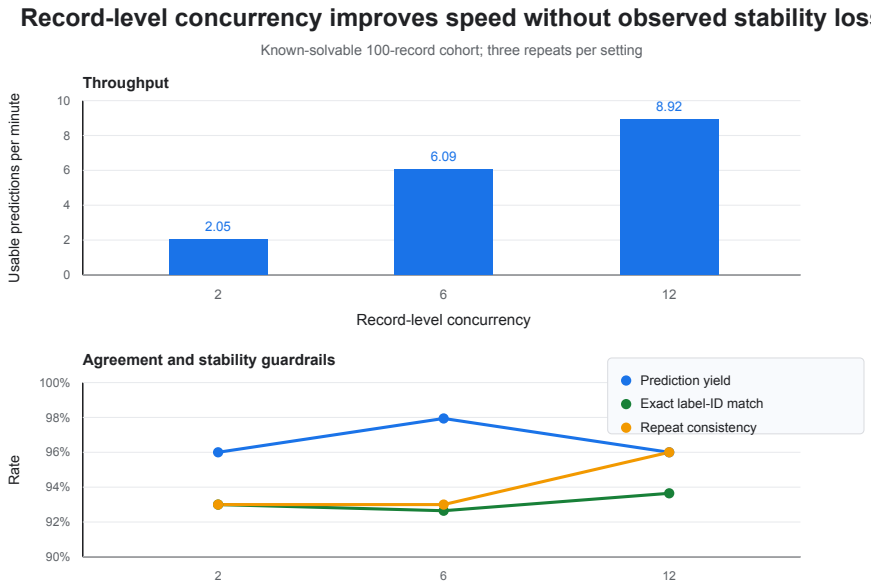


Figure 5. *Known-solvable multi-agent concurrency stability probe. Record-level concurrency improved usable predictions per minute from 2.05 to 8.92, while prediction yield, exact label-ID agreement, and repeat consistency changed only modestly across the tested settings.*

On the known-solvable cohort, increasing record-level concurrency from 2 to 12 improved throughput by approximately 4.4x, from 2.05 to 8.92 usable predictions per minute. At those endpoints, prediction yield was 96.0% in both settings, exact label-ID agreement was 93.0% and 93.7%, exact label-triplet agreement was 34.0% and 34.3%, and repeat consistency was 93.0% and 96.0%. The intermediate concurrency-6 setting had lower exact label-triplet agreement at 32.0%, so these results show no monotonic degradation across the tested settings rather than statistical equivalence. This supports workflow-level parallelism across independent records, while each record retains its own reasoning and synthesis path. It should not be read as a general claim that arbitrary parallel reasoning is always safe.

8.4. Verifier Concurrency And Token-Pressure Probes

We next asked whether strict agreement would degrade when the multi-agent workflow placed more pressure on shared hosted-model capacity. Token pressure means the volume of prompt, completion, reasoning, and embedding tokens sent through the hosted model stack per minute. We tested this pressure in two ways. First, we held batch concurrency at 10 records and varied the verifier concurrency cap, which limits how many verifier calls may run at the same time. Second, we ran a stress matrix that changed batch concurrency, verifier concurrency, reasoning effort, and optional classifier stages. These probes use the same 100-record known-solvable cohort as the preceding stability probe, so they should be interpreted as regression and pressure tests rather than unbiased accuracy estimates.

The verifier concurrency cap sweep used medium reasoning effort, 100 records, and three repeats per setting. A lower cap serialized verifier calls more aggressively; a higher cap allowed verifier calls to fan out more freely. Because this sweep uses the known-solvable cohort, the exact label-triplet values in the verifier-cap table are pressure-test and repeat-stability measurements, not an unbiased accuracy estimate.

Table 13. *Verifier concurrency cap sweep.*

Verifier concurrency cap	Record-runs	Prediction yield	Exact label-triplet	Repeat consistency	Avg records/min	Peak total TPM	Est. cost/100 usable
1	300	86.3%	80.7%	81.0%	9.69	329k	USD 5.63
3	300	89.0%	84.0%	91.0%	10.30	393k	USD 5.63
6	300	86.0%	80.3%	83.0%	10.34	412k	USD 5.61
10	300	88.0%	84.3%	86.0%	10.51	428k	USD 5.59
20	300	88.0%	83.3%	82.0%	9.91	389k	USD 5.61

Increasing verifier concurrency did not show a monotonic relationship with strict agreement. The verifier concurrency cap of 3 had the strongest repeat consistency, while the verifier concurrency cap of 10 had the highest exact label-triplet agreement and throughput. The operational lesson is that verifier calls should have an explicit concurrency cap, but the cap should be tuned empirically rather than assumed to be as high as possible.

The stress probe then compared a medium-reasoning baseline against high reasoning, higher record-level concurrency, and a maximum-pressure setting that combined high reasoning, higher batch concurrency, higher verifier concurrency, and an additional classifier stage. Like the verifier-cap sweep, the token-pressure stress table is a regression and pressure test on previously successful records.

Table 14. *Token-pressure stress probe on the known-solvable cohort.*

Config	Batch/ verifier concurrency	Reasoning/ extra stage	Prediction yield	Exact label-triplet	Records/ min	Reasoning tokens	Peak total TPM	Est. cost/100 usable
Baseline	10 / 3	Medium / off	88/100	82/100	10.18	176k	364k	USD 5.61
High reasoning confirmation	10 / 3	High / off	88/100	83/100	7.84	312k	288k	USD 7.01
High concurrency	20 / 10	Medium / off	86/100	79/100	16.69	173k	707k	USD 5.63
Maximum pressure	20 / 20	High / on	89/100	84/100	13.71	308k	576k	USD 6.95

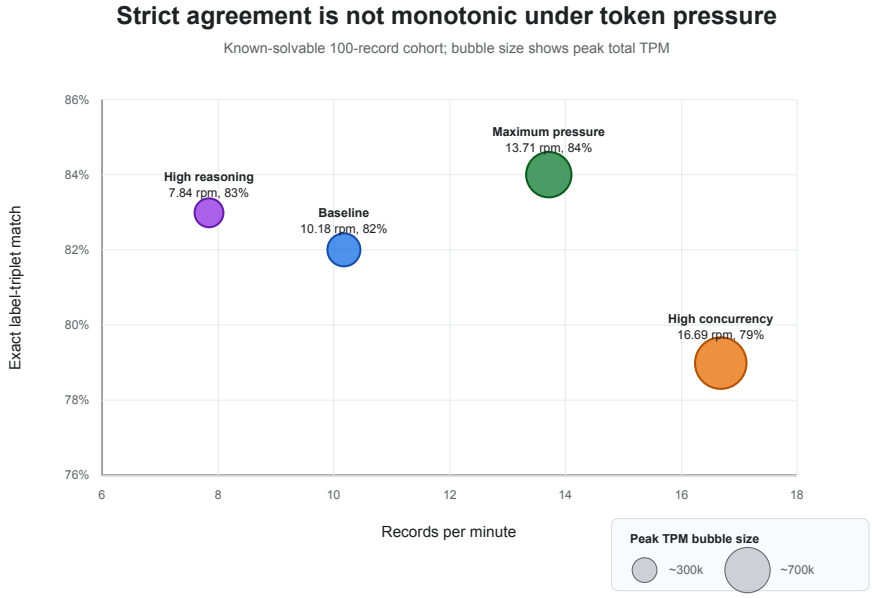


Figure 6. *Multi-agent token-pressure operating points. Higher concurrency and reasoning effort changed throughput, cost, and peak token-per-minute pressure, but exact label-triplet agreement did not move monotonically with any single knob.*

One high-reasoning run in the first stress matrix ended as a partial operational run. We excluded that partial result from the table and reran the high-reasoning configuration successfully as a confirmation run.

The stress results were more nuanced than the hypothesis that higher token pressure would necessarily reduce strict agreement. High reasoning increased reasoning-token usage by approximately 1.8x and cost by roughly 25% relative to the medium baseline, while reducing throughput from 10.18 to 7.84 records per minute; exact label-triplet agreement remained similar. Higher concurrency with medium reasoning increased throughput to 16.69 records per minute but showed a small drop in exact label-triplet agreement. The maximum-pressure configuration had the highest exact label-triplet agreement among the stress settings, but at higher cost and higher peak token pressure. These results support treating token pressure as an optimization variable, not as a one-directional failure mechanism.

We also computed an observed Pareto set from the revised-runtime multi-agent runs that used the same 100-record known-solvable cohort and reported exact label-triplet agreement. This includes the verifier concurrency cap sweep and token-pressure stress runs, but excludes the earlier multi-agent concurrency-stability phase because it used a different runtime version. The Pareto calculation compares cost per usable record, throughput, prediction yield, and strict agreement, following the standard multi-objective optimization idea of non-dominated alternatives [10]. This is not a global frontier over all possible settings or an unbiased accuracy frontier; it is the non-dominated set among the comparable calibration rows we ran.

Table 15. *Observed multi-agent Pareto-relevant operating points.*

Operating point	Role	Prediction yield	Exact label-triplet	Records/min	Peak TPM	Est. cost/100 usable
Verifier concurrency cap 3	Illustrative balanced point	89.0%	84.0%	10.30	393k	USD 5.63
Verifier concurrency cap 10	Cost/throughput edge	88.0%	84.3%	10.51	428k	USD 5.59
High concurrency	Latency-biased point	86.0%	79.0%	16.69	707k	USD 5.63
Maximum pressure	Stress-bound point	89.0%	84.0%	13.71	576k	USD 6.95

Within the observed rows, we use the verifier concurrency cap of 3 as an illustrative balanced point because it preserved the highest prediction yield and repeat consistency in the concurrency cap sweep while bounding verifier fan-out. The verifier concurrency cap of 10 was slightly cheaper and faster in aggregate, but with lower repeat consistency and higher peak token pressure. This descriptive comparison does not establish that cap 3 is statistically superior or a universal default. The high-concurrency and maximum-pressure rows remain useful frontier points for latency- or stress-oriented deployments.

Taken together, these operating points motivate the framework below: choose configurations by constraints and observed tradeoffs, rather than by maximizing any single knob.

9. Optimization Framework

The experiments suggest that production agent design should be treated as a constrained optimization problem. Organizations usually want lower cost, higher throughput, reliable completion, and auditability, but these goals can conflict. Increasing parallelism can improve runtime while increasing external-service pressure. Reducing timeout budgets can make a batch appear faster while converting slow-but-recoverable records into failures. Adding verifier or fan-out stages can improve review metadata while increasing model calls, latency, and possible failure points. This makes the framework more than a tuning checklist: each configuration maps to an observed operating point with measurable benefits and constraint violations.

The objective is not to maximize the number of agents, workers, retries, or reasoning steps. Let $x = (W, A, T, R, K, V, C_v, F, E)$ denote a configuration from the candidate set $\mathcal{X}$ for workload D. We define a feasible set using completion yield y_D, strict agreement g_D, cost

per usable record c_D, peak token pressure p_D, final failure rate f_D, and an indicator a for required auditability controls:

$$\mathcal{F}(D) = \{x \in \mathcal{X} : y_D(x) \geq Y_{\min}, g_D(x) \geq G_{\min}, c_D(x) \leq C_{\max}, p_D(x) \leq P_{\max}, f_D(x) \leq F_{\max}, a(x) = 1\}. \quad (1)$$

The agreement constraint g_D must be estimated on an appropriate reference-labeled calibration set. Agreement measured on the selected known-solvable cohort is a regression signal and must not be substituted for a population-level quality threshold. When representative labels are unavailable, the framework cannot claim that constraint has been validated.

Among feasible configurations, the primary objective is usable throughput q_D, where u_D is the number of usable completed records and t_D is wall-clock time:

$$x^\star \in \arg\max_{x \in \mathcal{F}(D)} q_D(x), \qquad q_D(x) = \frac{u_D(x)}{t_D(x)}. \quad (2)$$

When several configurations are feasible, we compare them in the Pareto sense over $(-q_D, c_D, e_D, r_D, b_D)$, where e_D is recovered-internal-error rate, r_D is reprocessing cost, and b_D is review burden. This formulation avoids inventing universal weights: local policy sets the hard thresholds and chooses among non-dominated operating points.

Table 16. *Agent configuration variables.*

Variable	Meaning	Example values
`W`	Local worker or process count	3, 4, 6, 8, 12
`A`	Hosted LLM/API concurrency limit	1, 2, 3, 4, 6, 8, 12
`T`	Per-record wall-clock timeout budget	60s, 120s, 300s
`R`	Retry count and retry policy	none, timeout-only, transient with backoff
`K`	Checkpoint frequency	every 25, 50, or 100 completed records
`V`	Verifier stage	on, off
`C_v`	Verifier concurrency cap	1, 3, 6, 10, 20
`F`	Fan-out across evidence sources	on, off
`E`	Model or reasoning-effort setting	low, medium, high

A practical calibration loop has eight steps: select a representative workload; run a conservative baseline; sweep one control at a time; record run manifests; estimate the observed response surface; reject configurations that violate hard constraints; identify approximate Pareto-efficient configurations; and choose an operating point that matches the organization's goal. In this study, we use observed sweeps to illustrate the tradeoff structure rather than claiming a complete global Pareto frontier over all possible configurations.

This framing changes the design question from "How many workers should we run?" to "Which configuration satisfies the reliability target at the lowest acceptable time and cost?" The single-agent experiments showed that increasing `W` alone produced diminishing returns once `A` was fixed, while increasing `A` within quota produced much larger gains. The timeout experiment showed that a faster configuration can be invalid if it violates completion-yield constraints. The multi-agent experiments showed that review metadata and record-level concurrency must be evaluated jointly against throughput, yield, cost, and stability.

The verifier concurrency cap and token-pressure probes further refine this framework. Higher concurrency increased peak token-per-minute pressure, and higher reasoning effort increased reasoning-token usage and cost, but neither variable produced a simple monotonic degradation in exact label agreement. Each configuration moved the operating point across throughput, cost, token pressure, prediction yield, and strict agreement. Production

tuning should therefore be measured as a constrained optimization problem rather than decided by intuition alone.

The same framework can support an interactive decision-support widget. The widget is a supplemental artifact and a demonstration of the paper's operating-point methodology, not a live deployment interface. Rather than launching expensive LLM calls for every user interaction, the widget uses saved aggregate calibration runs to simulate expected outcomes without exposing record-level source data. Users can adjust worker count, API concurrency, verifier concurrency cap, reasoning effort, and token-pressure constraints, then inspect observed throughput, cost, yield, strict agreement, and peak token pressure. The current widget is static: it does not launch LLM calls or process new records. It lets users explore saved aggregate calibration results. A future internal version could trigger new calibration runs inside an approved environment.

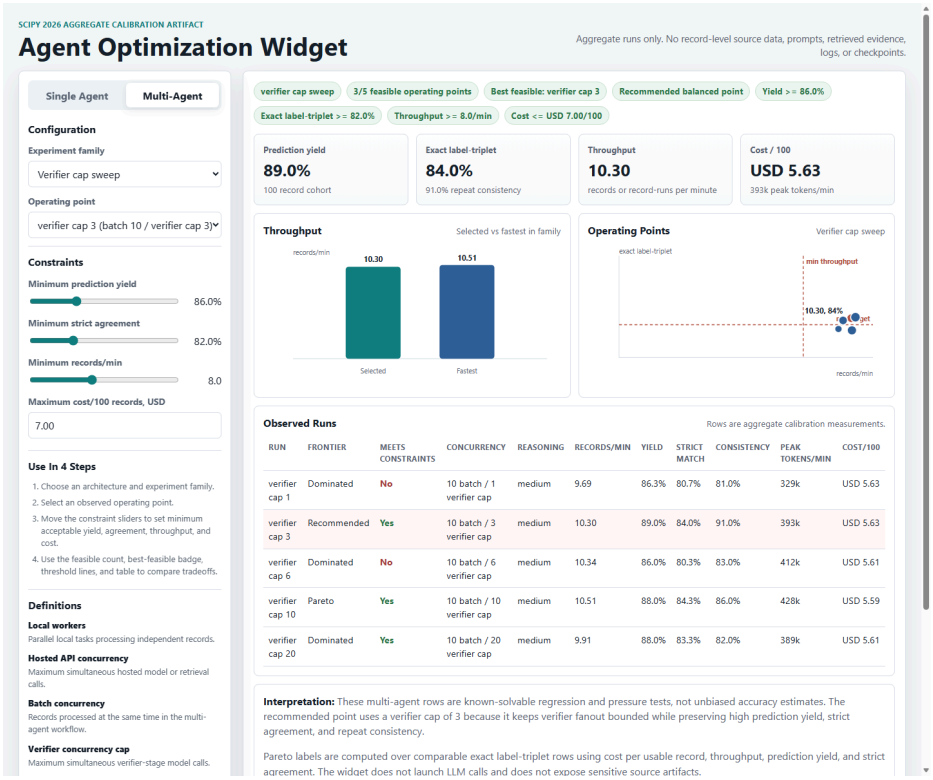


Figure 7. *Static decision-support widget for exploring aggregate calibration results. The widget is a supplemental artifact that uses saved operating points rather than live LLM calls or record-level source data.*

10. Reliability Patterns

The experiments point to several production patterns.

Standard distributed-systems controls need stage-aware semantics in an agent workflow. Retrying a deterministic service operation is intended to reproduce the same result; retrying an LLM call can produce a different candidate. A verifier then evaluates that new candidate, so replay can change both the verifier input and its verdict. Evidence fan-out multiplies stochastic calls and makes record latency depend on the slowest branch, while a failed branch may be hidden by a successful fallback or synthesis path. For this reason, the workflow must bound and log record attempts, evidence branches, and verifier calls separately, and must distinguish recovered stage failures from final record failures.

First, separate compute parallelism from API concurrency. Local worker count should not directly determine simultaneous hosted API calls. A bounded semaphore or equivalent limiter should protect each shared dependency, including hosted LLM APIs, managed search, object storage, and secret stores.

Second, classify transient and non-transient failures. Connection resets and read timeouts may succeed on retry. Provider content-filter rejections, invalid prompts, and input-specific policy failures usually should not be retried indefinitely. Retries should use bounded exponential backoff with jitter.

Third, use wall-clock task budgets. Individual records can run much longer than the median, and a small number of slow tasks can hold a batch open indefinitely. Timeouts make those stalls observable, but the budget must be calibrated against completion-yield requirements.

Fourth, checkpoint long-running batches. Checkpoints preserve progress across interruption, reduce reprocessing cost, and reduce exposure to nondeterministic reruns.

Fifth, distinguish recovered internal errors from final failures. Agent stages may recover through fallback paths. Those events should not necessarily mark the record as failed, but they should be counted separately so clean final success counts do not hide reliability signals.

Finally, design for partial completion. A run that completes 196 of 200 records is operationally different from a run that crashes before producing outputs. The system should preserve successful records, classify failed records, and provide a path for targeted retry or review.

11. Data And Artifact Availability

The experiments were conducted in an internal secured environment using access-controlled records and production-like cloud dependencies. Record-level source artifacts, raw narratives, prompts, retrieved evidence, raw logs, per-record outputs, and checkpoints are not public because they may contain sensitive regulated information. Shareable paper-supporting artifacts include the proceedings source, embedded figures, aggregate result tables, and a static decision-support widget built from aggregate calibration rows. Experiment scripts and broader run manifests can be shared only through approved internal access controls.

The reproducibility claim is therefore procedural rather than direct. The public artifact supports inspection of the measurement framework, operating-point tables, figures, and static widget behavior. It does not support direct reproduction of the exact numerical results from the original records, prompts, raw logs, outputs, or checkpoints. A reader with an equivalent authorized dataset and comparable hosted services could reproduce the procedure: define a workload, run baseline and sweep configurations, record the same aggregate measurements, and choose a bounded operating point under local constraints.

12. Limitations

This study has several limitations:

- **Scope.** The experiments were conducted in one regulated workflow. Exact failure rates may differ in other scientific or industrial settings.
- **Provider drift.** The systems used hosted LLM APIs and managed cloud services whose behavior can change over time as providers update models, infrastructure, and content policies.

- **Replay nondeterminism.** Replaying historical agent-internal failures did not reproduce the same final failures. This is an important reliability result, but it also means some historical failure modes could not be deterministically recreated under current conditions.
- **Disclosure limits.** Some sensitive examples cannot be disclosed.
- **Uncertainty treatment.** Repeated configurations report pooled counts, mean runtime, and consistency, but not run-level standard deviations or confidence intervals. Repeats on the same fixed cohort are clustered observations rather than independent population samples, and configurations observed once provide no variance estimate. Small differences between nearby operating points should be interpreted cautiously; a confirmatory study should pre-specify repeated runs and an uncertainty model.
- **Limited labeled calibration.** The single-agent labeled calibration used only 12 records and should not be interpreted as a definitive accuracy benchmark.
- **Reference-label interpretation.** The 200-record multi-agent structured-prediction probe evaluates agreement with internal reference labels, not broader downstream correctness. The exact-match metrics are intentionally strict: a record counts as correct only when the full predicted label set matches the reference label set. They do not give partial credit for predictions that are close or partly correct.
- **Known-solvable selection.** The 100-record concurrency stability, verifier concurrency cap, and token-pressure probes were selected from prior successes. They are regression and pressure tests, not unbiased accuracy samples. The verifier concurrency cap sweep used three repeats, while the token-pressure stress matrix used one completed run per stress configuration. One high-reasoning stress attempt ended as a partial operational run and was replaced with a successful confirmation run in the reported table.

13. Conclusion

LLM agents that work in interactive prototypes are not automatically production-ready. When deployed in high-volume batch workflows, they encounter the same problems as other distributed systems: concurrency bottlenecks, external service limits, transient failures, non-transient provider errors, stalled tasks, partial completion, and hidden internal exceptions.

Through single-agent and multi-agent structured classification workflows, we showed that reliability requires explicit engineering controls around the agent. Separating compute parallelism from API concurrency improved scalability. Wall-clock budgets made stalls observable. Checkpointing preserved progress. Failure taxonomies and structured metadata made it possible to distinguish final record failures from recovered internal errors.

The final multi-agent probes showed that record-level concurrency, verifier concurrency caps, reasoning effort, and optional classification stages can be tuned as operational controls. On known-solvable cohorts, these controls changed throughput, cost, peak token pressure, and strict label agreement, but strict agreement did not move monotonically with any single setting. The useful production question is therefore not which knob should be maximized, but which bounded configuration satisfies the required yield and agreement targets at acceptable cost.

The broader lesson is that production agentic AI is not only a modeling problem. It is a systems engineering problem and an optimization problem. Robust deployment requires designing for failure from the beginning so agent workflows are observable, recoverable, tunable, and auditable at scale.

References

[1] D. Sculley *et al.*, "Hidden Technical Debt in Machine Learning Systems," in *Advances in Neural Information Processing Systems*, 2015.

[2] D. Baylor *et al.*, "TFX: A TensorFlow-Based Production-Scale Machine Learning Platform," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, pp. 1387–1395. doi: 10.1145/3097983.3098021.

[3] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *arXiv preprint arXiv:2201.11903*, 2022, doi: 10.52202/068431-1800.

[4] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.03629.

[5] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/arXiv.2005.11401.

[6] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.

[7] M. Brooker, "Exponential Backoff and Jitter." [Online]. Available: https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/

[8] OpenAI, "Introducing GPT-5 for Developers." [Online]. Available: https://openai.com/index/introducing-gpt-5-for-developers/

[9] OpenAI, "New Embedding Models and API Updates." [Online]. Available: https://openai.com/index/new-embedding-models-and-api-updates/

[10] K. Miettinen, *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers, 1999. doi: 10.1007/978-1-4615-5563-6.

Open Access | https://doi.org/10.25080/mnjr6626

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Bridging the Technical Gap: A Student-Led Data Extraction Pipeline for Community-Driven Document Analysis

Jesse Loi[1], and **Ariana Mendible**[1]

[1]Seattle University

Abstract

This paper presents the impacts of using Large Language Models (LLMs) to extract and process information from complex PDF documents, assessing its accuracy compared to traditional human labeling workflows. Hosting multi-campus datathon volunteer events, we label over 400 documents to create a ground truth for our model evaluation. We present comparative results highlighting fuzzy matches between human annotations and LLM-generated labels. As a result, we demonstrate the viability of large language models in document analysis.

Keywords document extraction, LLMs, data labeling, LLM evaluation

Published Jul 13, 2026

Correspondence to
Jesse Loi
jesse.loi418@gmail.com

Open Access

Copyright © 2026 Loi & Mendible. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Community organizations, such as Communities United Against Police Brutality (CUAPB) have the important task of tackling police misconduct. To gather evidence to make their cases, they must collect sufficient data about trends in policing. This data is difficult to procure, as structured data from police departments is often limited and redacted. Consequently, a natural source of information is news articles, often giving comprehensive details regarding incidents of police misconduct. However, two main bottlenecks appear. First, news articles on police conduct are primarily unstructured and need to be parsed, often by a human reader to avoid false information. For example, articles must be mined for details such as location, date, involved persons, and incident type that appear in the prose. Second, events portrayed in news articles can be graphic and emotionally exhausting for human readers to parse en masse.

Reading through massive quantities of documents can be time consuming for any researcher and so LLMs have already proven their value on this end. However, another effect of researching police misconduct that goes unnoticed is the traumatic effects that spill over. While the civilians in misconduct incidents suffer trauma, the negative effects continue to spread outside of the incident itself. J. Bor, A. S. Venkataramani, D. R. Williams, and A. C. Tsai [1] describe spillover effects of how trauma affects Black Americans in the surrounding community. However, more pertinent to this research is the work of S. Dubberley, E. Griffin, and H. M. Bal [2] on researchers who conduct research on war crimes. Their results show 44% of researchers report worse mental health as a result of their research. Thus, those working towards police accountability, such as CUAPB, are very likely to face a similar psychological burden from their research. This trauma spillover from the initial event itself serves as a serious hindrance towards community efforts.

We should consider ways to shield researchers from unnecessary details of the event, when appropriate. For example, volunteers and organizers chronicling the occurrences or locations of the events likely do not need the full extent of the details in the articles. However,

selective censoring of portions often requires reading the article. Instead, we should look towards LLMs as a way to assist in reading through documents with sensitive information. D. Marengo, C. Hoeboer, and M. Olff [3] have discussed ways to categorize or detect trauma inducing text. Already, this will help filter out documents with especially graphic details. However, with articles specifically about police misconduct, few will remain. We will use LLMs not to remove articles, but to assist in reading articles for a particular feature that does not require human judgment.

One promising area of data extraction, among many, is location. This topic has been heavily explored in the literature and lends itself well to collaboration, due to the somewhat unambiguous nature of locations. H. Etienne and F. Romo [4] have already developed geographic information system (GIS) dashboards regarding police misconduct. This information can also greatly benefit individual local efforts, helping auditors target their efforts at specific regions and allowing a high-level view of policing trends. The aforementioned CUAPB and the Atlanta Citizen Review Board (ACRB) groups are both examples of such organizations. Additionally, almost all articles regarding police misconduct will have location data; demographic data are sometimes omitted. While the categorization of the misconduct incident itself is important, we set this aside due to its subjective nature. Address, on the other hand, is an objective piece of information for all incidents. But scraping location is not without issues. Common data scraping techniques can struggle with articles that may include social media screenshots, those that resist OCR, or otherwise have an unusual table outline with many advertisements. Even older physical news articles may appear in a tabular form that requires further pre-processing. The diversity of these articles presents a unique technical challenge insofar as standard PDF readers may read text out of order, such as with some tabular data or text with advertisements wedged between them, or not capture them at all, as text in images cannot simply be copied and pasted. Work in natural language processing (NLP) aims to address some of the above extraction difficulties.

After scraping, analysts sometimes use named entity recognition (NER) to isolate candidates for data extraction. D. Birks, A. Coleman, and D. Jackson [5] use NER to read narratives and identify particular trends of burglaries. More recently, A. Lo Duca [6] takes a different approach to extraction by using NER to extract entities from a repetitive text. The use of NER can be quite conducive when searching texts for dates, officers, and incident locations, as it has a benefit of extracting specific proper nouns, such as addresses and landmarks. However, a drawback of NER is that the data still requires parsing to manage. In particular, NER may provide nouns but cannot help us determine which contains the location of the incident. It may, for example, provide some unrelated address. Consequently, we need a more discriminating tool that can identify the location of the incident properly, such as an LLM. However, these tools have their own faults.

While we have discussed reasons to deploy LLMs for data extraction, we should be aware of major weaknesses, namely hallucination. L. Huang *et al.* [7] and Z. Xu, S. Jain, and M. Kankanhalli [8] have shown the difficulty of removing hallucination. Given the nature of police accountability, one must be especially vigilant in suppressing hallucination. To help, we draw on recent Retrieval-Augmented Generation (RAG) literature. RAG seeks to combat hallucinations by providing the model with a set of documents it can refer to when generating its responses. These documents serve as the "context" of the LLM's response. M. Reuter *et al.* [9] have recently discussed the use of RAG to combat hallucination in the context of long legal texts. The benefit of RAG that Reuter describes is that the model has context to refer to in responding. For our purposes, because we already provide the article text as context, our model becomes less likely to hallucinate. Additionally, drawing from P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha [10]'s paper on prompt engineering, we can deploy certain techniques to instruct our model to be more conservative in its responses. We elaborate on this further in our methodology. With hallucinations initially

addressed, we turn to LLMs to capture location context. In particular, we will cover how our methodology includes preprocessing techniques to improve the quality of the data prior to LLM data extraction.

2. Methodology

To test the efficacy of LLMs in information extraction, we extract data from CUAPB's article repository at (https://complaints.cuapb.org/).

2.1. Data Preprocessing

CUAPB contains a repository of articles, all stored in PDF form at (https://complaints.cuapb.org/). We use the `PyMuPDF` library to extract only the first 7 pages of all articles in the repository, as documents longer than 7 pages are not likely to be news articles. The extraction process for the articles, after parallelizing, took a total of 4 hours. After extracting the text, we refine our source data by flagging problematic features. Namely, we identified Private Use Area (PUA) characters and images-as-PDFs as features that we needed to address.

PUA characters are characters not properly recognized by the usual UTF character sets. In particular, many of these characters include stylized numeric characters, such as in Figure 1. PyMuPDF fails to read these characters, resulting in a street address with no building number. We flagged these characters by notating characters of a certain hexadecimal range between E000 and F8FF, as noted by The Unicode Consortium [11]. This range captures all of the manually identified PUA characters in the dataset. We nonetheless acknowledge limitations to using ranges emerging from manual discrepancy identification, with only 4 unique PUA identifications manually confirmed. This might prompt concerns given that many more articles were flagged than the initial 4 manual identifications. However, this is because PUA characters do not explicitly need to appear at address locations, but more commonly numbers in different fonts, which explains the high number of texts flagged. Despite the high number of flagged articles, we cannot definitively say that PUA encoding is the sole encoding error that may occur in scraping and that further replications or extensions of this experiment may be more exhaustive to capture further encoding errors. Furthermore, adding additional flags will at worst only have a time cost, as all flagged articles are eventually read with OCR, though the computational and time resources required for this may not be feasible for all research groups.

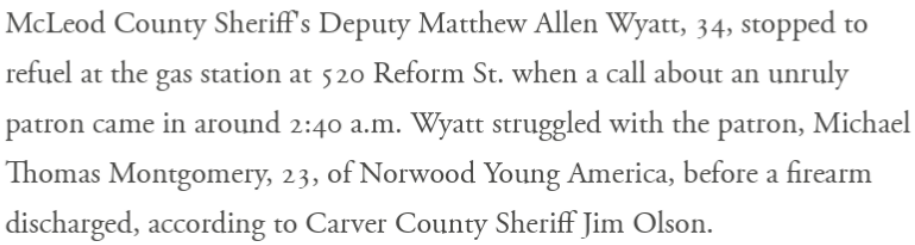


Figure 1. *Screenshot of article located at [https://d3n8a8pro7vhmx.cloudfront.net/cuapb/pages/270/attachments/original/1627047812/Off-duty_McLeod_County_deputy__Norwood_Young_America_man_from_April_17_shooting_identified.pdf?1627047812] showcasing PUA characters used to stylize the numbers.*

Some articles were scraped, but with extremely low character counts. These PDFs were simply collections of images and the library could not capture the embedded text. We flagged these articles by noting the character length. In particular, characters of under 876, representing the bottom 4% of documents in terms of character count, were flagged as

potential screenshots of documents, as documents of such short length are very likely to contain more text, such as in Figure 2.

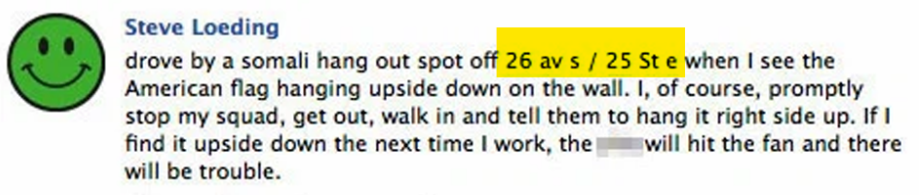


Figure 2. *Screenshot of article located at CUAPB archive showcasing text embedded in an image, causing readers like PyMuPDF to fail to capture the text.*

To handle these two cases, we implemented optical character recognition (OCR). This workflow optimizes time, as an OCR scrape of all 2,700 articles would be too time-consuming. The additional OCR procedure added around 100 news articles, increasing our test sample by a reasonable amount. With the assistance of OCR, our set has been enhanced enough for an LLM to read its documents and label locations.

2.2. Data Labeling

We label our data with an LLM and with human volunteers to serve as a ground truth set for the models. We made use of the `ollama` library to access `llama 3.1:8B`, a light language model with only 8 billion parameters. While other models were available, we opted to use a model accessible to organizations with less compute power. Additionally, a general instruction trained model can be used for tasks other than simply location extraction. We set the model to `llama3.1:8b` with a temperature of 0.0. We then called the model with the following prompt:

```
incident_prompt = """
Analyze the provided news article and identify the LOCAL location of the incident in terms of the
street address or physical landmarks. If the location cannot be found or is uncertain, return UNKNOWN
instead. Only return a location you are confident with.
Output only the address and no comments. Just the address. No need to provide a state or a city. Just
a local address that can be accurately located in the document. IF YOU CANNOT THEN ENTER UNKNOWN
DO NOT HALLUCINATE A LOCATION
An example output could be:
"A Kroger on 3rd Street"
"1923 E Pine St St Louis 42019"
"City Hall"
"Defendant's home"
"Unknown"
"Location not found"
"UNKNOWN"
These are all appropriate responses.
"""
```

This prompt makes use of both few-shot prompting and a very conservative instruction that attempts to restrain the model from hallucinating, restricting its role to extraction, not inference. Similar instructions were given to the human volunteers.

Having crowdsourced evaluation helped speed up the labeling process and account for individual human bias. More importantly, we wanted another set of human responses as a comparison between each other. We procured the data by hosting a datathon across several campuses (Seattle University, Hamline University, and Carleton College). We had succeeded in recruiting 48 different volunteers to label the articles as a news article or as a non-news article designation, such as a police report. To streamline the workflow, volunteers, primarily students, were asked to only perform data labeling on news articles once identified, reading through the article to label relevant officers, civilians, and incident details.

We made use of two different environments for our datathon, Baserow and Zooniverse, exploring ways to more quickly crowdsource data.

Baserow is a platform that allows users to build databases and share permissions, similar to an online spreadsheet tool. The initial benefits to Baserow were first, its open-source structure, allowing for high degrees of accessibility to community organizations, and second, its ability to display various views. While coordinating large, synchronous data entry, there is the risk that entries will overlap, such as when two users try to make an edit simultaneously on a Google cloud document. While the risk remains for a tabular database such as Baserow to fail, Baserow allows for multiple frontend views, including card formatted data that allow subsets of data to be assigned to teams of volunteers. This data segmentation helps mitigate simultaneous data entry.

The first datathon successfully labeled 460 documents, 120 of them being news articles, and allowed double verification of 60 articles over the course of 2 hours. Double verification is particularly important, as this will allow us to analyze the variance and noise in the responses, as human responses are very unlikely to be deterministic due to the nature of responses. For example, some volunteers may opt to offer more or less information, such as additional information about landmarks, in their data entry. Additionally, certain volunteers may provide further information about the address such as the city or state.

Despite the enhanced structure that the advanced views gave, our datathon still faced strong limits, namely continued double verification issues. The latency of the database prevented completely real-time updates from being made. Additionally, there was no easy way to preview articles in Baserow. Consequently, tasks could be rather taxing and navigating several pages, even with the more helpful views, was quite demanding for volunteers.

Baserow's issues led us to use Zooniverse, an online platform designed specifically for data annotation, as opposed to being a more formal database as Baserow was. Crucially, it is designed as a survey rather than as a typical tabular database. Users are given items that they provide answers about. In our case, we took the first 5 pages of each article and converted them into 5 images as a preview, providing users a way in-app to read through the articles and answer relevant questions. We reaped several advantages to Zooniverse compared to Baserow. First, a streamlined work environment made it so that volunteers no longer need to manually open a URL and instead simply need to work from a single window without interruptions. Second, a major issue was that volunteers in Baserow continued to accidentally fill in the same data rows. This was because Baserow had a small bit of latency where there was a delay between user data entries. This resulted in frustration from the volunteers. Zooniverse does not face this challenge. Third, each user could not modify the responses of another volunteer. Additionally, volunteers would not be influenced by the responses of another volunteer, leading to much stronger data integrity.

2.3. Evaluation Metrics

Recall our goal is to evaluate whether an LLM's location identification is comparable or equally as effective as a human's location identification from the same text. Consequently, our evaluation metric is designed to capture similarity between responses, as in the ideal case, both the human and the model should produce outputs that are very similar. While a semantic matching metric would be ideal, shorter responses make encoding and comparison difficult, due to the general lack of context. Instead, we are interested in text similarity. In particular, when the location is present, we want to ensure that both the human and LLM return the same chunk of text. To account for natural variance in writing and potential misspelling, we use the Levenshtein distance, as implemented in the `TheFuzz` library, as our measure of success. Fuzzy matching assigns a score from 0 to 100 based on how closely

the characters in one chunk of text match the characters in the other chunk of text. We additionally use subset fuzzy matching, which avoids assigning a penalty if one chunk has more text than another chunk, but both texts have overlapping text.

S. Dev *et al.* [12] have previously used fuzzy matching as an evaluation metric. The main competitor as an evaluation metric would be to use an LLM-as-judge. However, to produce as deterministic a response as we can, we use fuzzy matching primarily. Dev et al. remark that fuzzy matching remains strong so long as nuance does not need to be captured. The problem of location extraction at hand is simple enough to the point where an LLM-as-judge is likely unnecessary and not computationally worth it, especially given current problems with its use as an evaluation tool [13]. But we cannot use fuzzy score pairs for an LLM and human by itself. Such a comparison would suggest that a single human response is enough to produce ground truth, which is not necessarily true. We give more context about fuzzy matching by discussing the Levenshtein distance. The Levenshtein distance $\mathrm{lev}_{a,b}(i,j)$ is a measure between two strings a and b and is formulated with the recursive definition given in (1).

$$\mathrm{lev}_{a,b}(i,j) = \tag{1}$$

In the first case, $\min(i,j) = 0$ occurs when comparing a string to an empty string. Let i and j also represent indices in a matrix whose columns represent the indices of one of the strings, with the first column denoting the first character and so on, and whose row represents the indices of the other string. Recursively, the "distance" is represented by the number of steps needed to convert one string to another. Each step is either a character deletion, represented by $\mathrm{lev}_{a,b}(i-1,j)+1$, a character insertion, represented by $\mathrm{lev}_{a,b}(i,j-1)+1$, or a character substitution $\mathrm{lev}_{a,b}(i-1,j-1)+c(a_i,b_j)$.

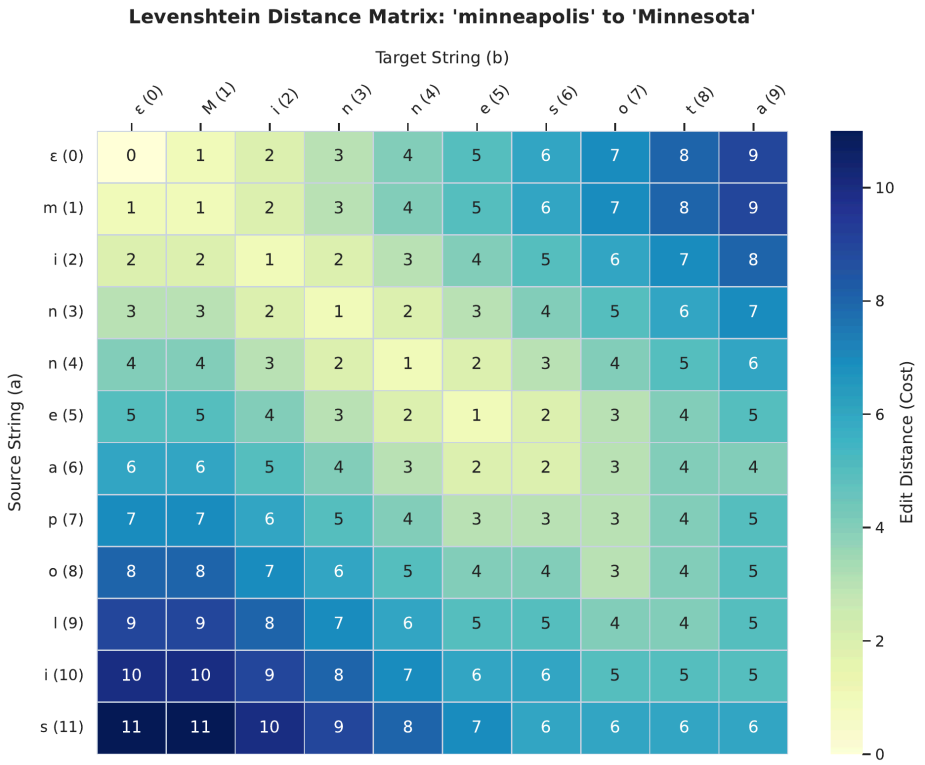


Figure 3. *The following Levenshtein distance matrix shows an optimal path cost of 6.*

For the example in Figure 3, 6 steps are the optimal number required to convert "minneapolis" to "Minnesota". To convert these steps into a score, we normalize with (2).

$$\text{Score}_{\text{norm}} = \left(1 - \frac{\text{lev}_{a,b}(|\ a\ |, |\ b\ |)}{\max(|\ a\ |, |\ b\ |)}\right) \times 100 \tag{2}$$

The current project makes use not of general fuzzy matching, but partial ratio fuzzy matching, or subset fuzzy matching. Instead of comparing the entire string, subset fuzzy matching only matches the best matching substring of equal length. In other words, there is not a penalty for deleting ends off of a longer string. We prefer partial ratio matching over traditional fuzzy matching as we have found LLMs to prefix or suffix appropriate responses with additional characters, such as commentary like "The location is…", or an LLM would surround its responses with quotation marks. We do not wish to penalize these additional characters, as that would not interfere with the correctness of the LLM's response. For string a length m and string b length n where $m < n$, subset fuzzy matching creates an $n \times n$ matrix and computes the Levenshtein distance for every potential window of length m for string b. The choice to use a text-based match motivates tuning the model to replicate a human's identification as much as possible, where the human is instructed to be as conservative as possible in their answers. No hard threshold of fuzzy score responses was used to determine a match, though scores of about 70 tended to contain the same content through individual verification.

A fuzzy score by itself cannot be the sole determiner of the success of the LLM. Such a measurement would only be true if human responses to the labeling questions are deterministic given an article. However, as mentioned above, human responses may naturally differ as well. What we do instead then is perform a fuzzy comparison between two human responses. Then, we perform a fuzzy comparison between the LLM and one of the human responses. We can then compare the distributions of those responses. This is a methodology adopted by both S. Han, G. T. Junior, T. Balough, and W. Zhou [14] and D. Wang *et al.* [15]. Han et al. specifically call this evaluation by human agreement. The strategy involves, in some way, a type of Turing test where model success is not determined by closeness of model response, but rather the closeness of the distributions of the fuzzy pairs. To allow for a more concrete description of such a Turing test, we introduce both the two-sample Kolmogorov-Smirnov test and the Wasserstein distance.

The Kolmogorov-Smirnov (K-S) test and the Wasserstein difference on 1-dimensional data allow for a more nuanced analysis of the data by comparing distributions between the different label pairs. The K-S test takes two samples and provides a similarity score with p-value as to whether the difference between these distributions is statistically significant. It is analogous to a two sample t-test for means. A low statistic and a high p value for these tests would indicate that the distributions are not distinct enough to tell apart. The statistic D is computed in (3).

$$D = \sup_x |\ F_1(x) - F_2(x)\ | \tag{3}$$

We let $F_1(x)$ and $F_2(x)$ be our two empirical cumulative distribution functions created from our two sets (human-human and human-LLM) of fuzzy pairs. We then compute the supremum of the difference between these functions to identify the divergence between the two, giving us a metric to determine whether the LLM and human responses could have the same underlying distribution. However, note that this test is not meant to prove that two distributions are the same, but can only give strong evidence that distributions are distinct. In other words, the null hypothesis is that the distributions are the same and the test finds evidence to reject the null hypothesis. With that in mind, we cannot assert the null hypothesis and should hedge our conclusions. Known as the Earth Mover's Distance and introduced by Y. Rubner, C. Tomasi, and L. J. Guibas [16], the Wasserstein distance is a metric that measures how much must change to transform one probability distribution into

another. It quantifies the cost of transporting probability mass to align two distributions. The distance can be formulated as follows:

$$W_1(F_1, F_2) = \int_{-\infty}^{\infty} \Big| F_1(x) - F_2(x) \Big| dx \tag{4}$$

Let $W_1(F_1, F_2)$ stand for the 1st Wasserstein distance, as opposed to other Wasserstein distances which may square the differences. $F_1(x)$ and $F_2(x)$ once again represent the cumulative probability distributions for the two sets of pairs mentioned above. We then integrate the absolute difference between the two curves to identify, intuitively, the total distance between the distributions. In the context of our 0-to-100 fuzzy matching scale, the Wasserstein distance represents the average number of score points that the human-LLM fuzzy scores would need to shift to perfectly match the human-human baseline distribution. We can use this to measure the average point adjustment required to align the LLM-human distribution with the natural variance of human agreement.

3. Results

After running both datathons, we compiled two datasets and managed to double-verify 120 articles. We have very promising data from our Baserow dataset. What we have so far is 123 LLM-human pairs and 85 human-human pairs, where human-human matches are those where we had two separate volunteers identify a location for an article. However, the data from the Zooniverse dataset is not as promising, with only 13 human-human pairs with 123 LLM-human pairs. This is likely due to the fact that Zooniverse has no native way to capture news articles only, leading to more entries for legal documents and other types of documents. We recognize the limitations of only 13 data points for the Zooniverse comparison and leave it in the following discussion section. For transparency purposes we make the Zooniverse comparison, but rely more on our usage of the Baserow dataset.

We compare the distributions here where Figure 4 displays the data and Figure 5 displays the data as if they were converted to probability density functions.

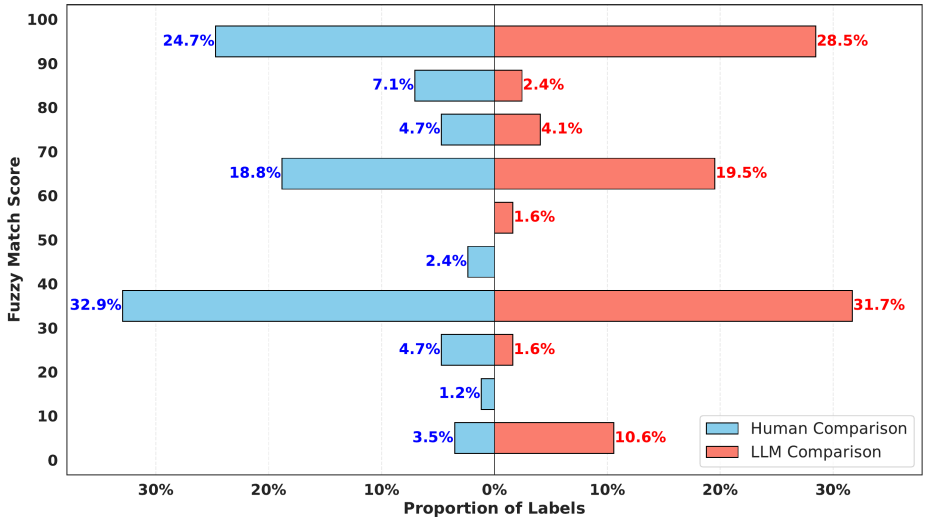


Figure 4. *Distribution of fuzzy match scores for double-verified human-to-human annotations for the Baserow dataset.*

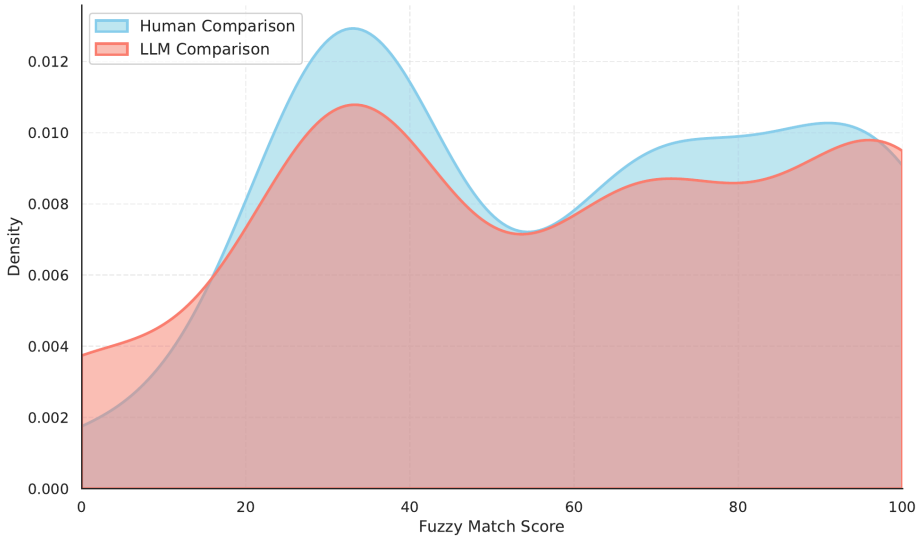


Figure 5. *We display the distributions as a density plot, showing what both distributions would look like if they were converted into probability density functions.*

Here we compare the two distributions. Upon visual inspection, the results look promising, despite the large differences. We see in both cases a tri-modal distribution, with answers on the high end indicating almost exact matches, answers on the low end indicating almost no match, and answers around the 60-70 point range indicating some match.

To elaborate on the significance of this comparison, we discuss examples of high, low, and mid ranking pairs from our dataset. These represent some examples of what is included in the dataset, but are not strictly included in the final dataset.

Response 1	Response 2	Fuzzy Score
“53rd and Emerson Avenues N.”	intersection of 53rd and Emerson Avenues N	93
“1235 Reform St”	520 Reform St., Norwood Young America MN	63
N/A	“Minnesota City Court”	0

Answers on the high end over 90 points indicate that the model and human captured the same text in their responses, with only minor phrase or wording changes. Scores around this range show promise about the model, though might represent sets of articles with a location clearly provided.

We also see here that while scores as high as 63 may seem to point to the human and model reading the same chunk of text, the human reader is able to capture some crucial details that the model cannot. We go into more details of this in the discussion, but these scores likely represent articles with a clear incident location mentioned, but cannot be accessed due to scraping problems.

Finally, answers on the low end, around 0-10 on the fuzzy scale, indicate almost no match between the terms. These likely represent articles where there is no clear incident location. In such cases, the human volunteer likely enters terms such as “N/A”, “no location”, or some other indicator that the address is unclear. In contrast, while the model is prompted to give a response without hallucinating, it will give an overly vague response or a mentioned location but that is not the incident location.

Dataset	News Articles Compared	Mean Fuzzy Score	Median Fuzzy Score

Dataset	News Articles Compared	Mean Fuzzy Score	Median Fuzzy Score
Baserow R1	69	72	93
Baserow R2	42	69	91
Zooniverse	79	70.8	88

At an initial glance, the data looks promising. With a median of at least 88, we can suspect at least half the data is a reasonable match. That being said, this still entails that half the data is below, with a mass of points clustering around a fuzzy score of 0. To add more context, we display a handful of observations.

LLM Location	Human Location	url	fuzzy score	Explanation
Deer River		source	0	LLM attempts answer when there is none
UNKNOWN		source	0	Negative answers expressed differently
“KASOTA, Minn.”	LeSueur County	source	23	Overly vague answers
“Minneapolis City Hall”	Eastside St Paul Checkerboard Pizza	source	30	LLM fails to locate instance
Red Wing City Hall		source	33	LLM provides wrong location
“Interstate Hwy. 35E and University Avenue”	Interstate Highway 35E North by University Avenue	source	72	
“1800 block of Columbus Avenue S.”	1800 block of Columbus Avenue S	source	100	

We see that errors often occur when it is unclear what the location is in the article. There are two main root causes for divergence in answers throughout, one on the human side and another on the LLM side. First, we note that volunteers were not heavily prompted on the types of appropriate answers during the datathons, due to time constraints. As a result, there was not a consensus on how to respond. For example, some volunteers responded with “N/A” while others responded with an empty string. Additionally, volunteers may have been prone to making mistakes, due to the large volume of questions being asked of them. During the datathon proper, they recorded information that was not simply the location and may have divided their attention several ways. Second, we note that throughout the labeling process, the LLM would almost never state the location is not present, instead giving answers that were overly vague. For example, it would offer generally vague responses, such as the city, when giving location. Additionally, the LLM may have had difficulty interpreting the concept of an “incident location”. While the goal of this pipeline was to locate the area of police misconduct, the LLM often provided the courthouse where the complaint was made or the officer was charged. However, even aside from these behavioral limitations, there are significant parts of the pipeline that limit successful answers. Particularly, while efforts were taken to identify text that required OCR, this was done manually and not exhaustively. Often, LLMs would have no reliable text to work with. This was a natural limit of our preprocessing ability. All these features join together to result in lower fuzzy scores. To expand on causes of some errors, we display points of failure during pre-processing of our 1300 articles.

Pipeline Issue	Articles Affected
Too short (likely screenshot/image-only PDFs)	89

Pipeline Issue	Articles Affected
Exceeded 7-page scraping limit (likely court proceedings, not source articles)	111
Required OCR	360

Of the 860 articles processed, three pipeline-stage issues accounted for the majority of extraction difficulty: 89 were flagged as likely screenshots based on character count, 111 exceeded the 7-page scraping cutoff and were treated as court proceedings rather than source articles, and 360 required OCR to recover readable text. The 360 include the 89 documents that were too short, so likely the remaining 271 articles had PUA characters.

However, fuzzy score matches by themselves are not enough.

Employing both the Kolmogorov-Smirnov (K-S) test and the Wasserstein distance gives us a different result. We display the results for both datathons.

Dataset	K-S Statistic	p-value	Wasserstein Distance
Baserow R1 and R2	0.1027	0.6158	3.3351
Zooniverse	0.2245	0.5192	9.0125

For the Baserow dataset, we see that the K-S statistic shows that there is not yet enough evidence that the distributions are different. Namely, the high p-value of 0.615 is far higher than the recommended 0.05. More importantly, the Wasserstein distance shows that only a small shift in the data is necessary to match one distribution to the other.

For our much smaller Zooniverse dataset, the results are not as conclusive and show a larger difference, but still have a high p value, indicating that they do not have sufficient evidence to claim that the distributions are different. Additionally, we see that the Wasserstein distance is larger than the one in the Baserow dataset, but is still low.

4. Discussion

The initial view of the data offers some modest success. We noticed that overall fuzzy comparisons between the model and a human were unpromising, but such comparisons are not apt for a proper evaluation of a model. We therefore opt for S. Han, G. T. Junior, T. Balough, and W. Zhou [14]'s evaluation methodology. They handle the problem from the paradigm of "one-to-many" questions in the literature. In other words, they discuss questions for which the text generated answer is open-ended, hence "many answers" to "one question". The question of location extraction might seem at first to be a closed-ended question. After all, a police incident can only occur at a single location. However, recall that several terms can refer to a single address. Consequently, answers differing by a word or two should not be heavily penalized.

4.1. Limitations

While we had initially addressed Private Use Area Characters and text embedded in images, the solutions raised are not comprehensive. When analyzing larger and more diverse sets of articles, it is highly likely that many newer PUA characters are unreadable with the current infrastructure. More salient of an issue is that longer news articles with small snippets of text within images are not read at all. This is because our method of detecting text-based images was based on character count. In an article with high character count but many images, our data extraction pipeline is significantly weakened.

Our datathons labeled over 860 documents in total with 248 news articles, but only about a third of these were news articles, and even fewer of these articles were double-

reviewed. This greatly limited the comparisons made in the results section. If more data were collected, it could be that the distributions compared would be different in a statistically significant manner. Most importantly, we must address the extremely limited human sample. Recall that we only have 13 human-human data points for the Zooniverse dataset and only 85 human-human data points from the Baserow dataset. While it is better, it remains far from ideal. We therefore wish to hedge our results, instead using this research as a framework for future research and community engagement.

The lack of data for human comparison is the most important area of improvement for our research. There are two ways that future research can improve on our results. First, organizers can lower the total number of articles such that they can guarantee each article in the labeling set can be labeled at least twice. Second, a different route can be to collect enough data with more time and volunteers such that the volume of data is simply higher. While these two strategies can assist with data volume, we still note a value of this experiment to future researchers. While Zooniverse has a more user-friendly design, the independent nature of task assignment makes concentrating labeling difficult, which Baserow has more success with. Future researchers should consider the trade-off between a data secure, blind assignment task that Zooniverse provides with a platform like Baserow, which contains some multi-user difficulties but allows a degree of agency when assigning tasks. In particular, Baserow would allow for the intentional double-assigning of articles to two volunteers by design.

4.2. Future Work

The results derived from this research are only cursory, and seek only to lay the groundwork for later investigation into how an LLM can mimic a human's ability to identify a location in text.

While the trials collected over 400 data points, only 123 of these points were used to do an analysis of the data because the news article data was not labeled beforehand. Future analyses can benefit from human labeling of the data. Out of the 1,300, we only have 248 news articles labeled. This is only a fifth of the articles from the CUAPB database. There is already infrastructure for this work in the form of the Zooniverse datathon page. Outside of the 1,300 scraped articles, researchers in the field may be interested in exploring more data sources. However, more data sources would only help given the availability of human labeling.

Even aside from data that remains unlabeled, such comparisons with the data can be done for both new data and with new volunteers. Recall that we only had 48 total volunteers across both datathons. While this ratio remains high, with about 8 documents per volunteer, different biases among the volunteers and predilections towards certain answers may skew the results. In particular, because the datathons were organized as community events, we can be fairly confident that some labels were not made independently of each other. Finally, a replication of this data labeling process should include an explicit mechanism to ensure there are more human-human pairs of data points. Because labeling assignments were done independently, there was no way to guarantee that the data would have duplicates. In addition to enriching human data, there are also many ways to improve computational results.

Aside from further data exploration, comparison and exploration of different large language models may benefit our audience. Recall that our current model only made use of `llama 3.1:8B` as the data extractor. The intention of this choice of model was to use as few parameters as possible to reduce compute resources and use a general-use model to make linking to a general RAG pipeline easier. Both of these considerations were with respect to accessibility. However, researchers interested in those areas may use more powerful

models to derive more accurate locations, especially in cases where the location is unclear or implied.

5. Conclusion

We present a generalized outline for how to use LLMs in data extraction, especially for mass-quantity document analysis. We discuss data scraping and cleaning as well as the effectiveness of LLMs in searching through source documents. Our initial results are promising, showing that LLMs provide performance similar to that of a human, sparing valuable volunteer time as well as avoiding unnecessary psychological stress from having to read details about brutal events, serving as a cognitive shield. This strategy allows community organizations to better allocate their efforts.

Acknowledgements

We would like to thank the QSIDE institute for their funding for the two datathons as well as to all 48 of the volunteers across Carleton College, Hamline University, and Seattle University for making both datathon events a success. We would also like to extend a thanks to Mark Daza, for initial drafts of the code used to extract locations from text, as well as to David Stanko, for assistance cleaning the scraped articles and helpful discussions regarding NER. The project's technical success would not have been possible without them.

References

[1] J. Bor, A. S. Venkataramani, D. R. Williams, and A. C. Tsai, "Police killings and their spillover effects on the mental health of black Americans: A population-based, quasi-experimental study," *The Lancet*, vol. 392, no. 10144, pp. 302–310, 2018, doi: 10.1016/S0140-6736(18)31130-9.

[2] S. Dubberley, E. Griffin, and H. M. Bal, "Making secondary trauma a primary issue: A study of eyewitness media and vicarious trauma on the digital frontline," Istanbul, techreport, 2015. [Online]. Available: https://firstdraftnews.org/wp-content/uploads/2018/03/trauma_report.pdf

[3] D. Marengo, C. Hoeboer, and M. Olff, "Investigating the use of large language models for the detection of trauma-related symptoms: An exploratory study using trauma narratives," *Journal of Anxiety Disorders*, vol. 119, p. 103151, 2026, doi: 10.1016/j.janxdis.2026.103151.

[4] H. Etienne and F. Romo, "Mapping police violence 2015-2020." [Online]. Available: https://www.arcgis.com/apps/dashboards/7964469c84514a3fb89d9d5b9ea9f958

[5] D. Birks, A. Coleman, and D. Jackson, "Unsupervised identification of crime problems from police free-text data," *Crime Science*, vol. 9, p. 18, 2020, doi: 10.1186/s40163-020-00127-4.

[6] A. Lo Duca, "An empirical study to use large language models to extract named entities from repetitive texts," in *Proceedings of the 20th International Conference on Web Information Systems and Technologies (WEBIST 2024)*, 2024, pp. 417–424. doi: 10.5220/0013066500003825.

[7] L. Huang *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025, doi: 10.1145/3703155.

[8] Z. Xu, S. Jain, and M. Kankanhalli, "Hallucination is inevitable: An innate limitation of large language models." arXiv:2401.11817, 2024. doi: 10.48550/arXiv.2401.11817.

[9] M. Reuter *et al.*, "Towards reliable retrieval in RAG systems for large legal datasets," in *Proceedings of the Natural Legal Language Processing Workshop 2025*, 2025, pp. 17–30. doi: 10.18653/v1/2025.nllp-1.3.

[10] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications." arXiv:2402.07927v2, 2025. doi: 10.48550/arXiv.2402.07927.

[11] The Unicode Consortium, "The Unicode Standard, Version 16.0 – Core Specification," Mountain View, CA, technical report, 2024. [Online]. Available: https://www.unicode.org/versions/Unicode16.0.0/core-spec/chapter-23/#G20387

[12] S. Dev *et al.*, "Simpler is better for autograders: Toward cost-effective LLM evaluations for open-ended tasks," techreport RR-A4618-1, 2026. doi: 10.7249/RRA4618-1.

[13] D. Li *et al.*, “From generation to judgment: Opportunities and challenges of LLM-as-a-judge,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 2757–2791. doi: 10.18653/v1/2025.emnlp-main.138.

[14] S. Han, G. T. Junior, T. Balough, and W. Zhou, “Judge's verdict: A comprehensive analysis of LLM judge capability through human agreement.” arXiv:2510.09738, 2025. doi: 10.48550/arxiv.2510.09738.

[15] D. Wang *et al.*, “Learning personalized alignment for evaluating open-ended text generation.” arXiv:2310.03304, 2023. doi: 10.48550/arxiv.2310.03304.

[16] Y. Rubner, C. Tomasi, and L. J. Guibas, “The Earth Mover's Distance as a metric for image retrieval,” *International Journal of Computer Vision*, vol. 40, no. 2, pp. 99–121, 2000, doi: 10.1023/A:1026543900054.

Open Access | https://doi.org/10.25080/txph5256

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

The Missing Lever in Machine Learning Deployment: Threshold Tuning using Regression Discontinuity

Lucas Squarize Chagas[1], and **Avik Basu**[1,2]

[1]Intuit, Inc., [2]PyOpenSci

Abstract

When deploying machine learning models into production, a critical but often overlooked lever is the decision threshold that converts predictions into actions. In practice, thresholds are typically chosen arbitrarily and left unchanged, leaving substantial value on the table despite heavy investment in model development.

We propose a systematic framework for optimizing thresholds using Regression Discontinuity Design (RDD), an econometric method that exploits the natural experiments created by decision cutoffs. We operationalize this approach into a Python package that automates bandwidth selection, local linear estimation, robustness checks, visualization, and optimization routines. Validated on simulated data and applied within a large-scale production system at Intuit, RDD-based threshold tuning outperformed the alternative methods we tested, improving target metrics without harming guardrails and enabling principled trade-off analysis between competing business objectives. We also delineate where the approach applies: the estimate is local to the cutoff, its precision depends on data density near the threshold, and the tuned value is conditional on the model that produces the score.

Keywords regression discontinuity, threshold tuning, machine learning, causal inference, optimization

1. Motivation

When deploying machine learning models into production, one critical lever is routinely overlooked: the threshold that converts predictions into actions. A recommender system must decide which items to display, a churn model who to target, and a fraud model when to trigger an alert. These thresholds often matter as much for outcomes as the underlying model itself. Yet in practice, they are usually chosen arbitrarily: 50% relevance to show a recommendation, 70% fraud risk to flag a case as fraud, 90% churn probability to trigger a discount. Once set, they are often left unchanged throughout the model's lifecycle.

As a result, firms invest heavily in model development but leave major value untapped by relying on static, guesswork thresholds. We propose a systematic way to optimize thresholds using Regression Discontinuity Design (RDD) [1], an econometric method that leverages arbitrary policy or decision cutoffs to uncover causal relationships.

2. Methodology: Regression Discontinuity to Tune Thresholds

Thresholds create natural experiments: units just above and just below a cutoff are nearly identical, except that one receives a treatment (e.g., a recommendation or a discount) and the other does not. RDD exploits this discontinuity to estimate the causal effect of the treatment for units that are close to the threshold (Figure 1 displays an example).

Published Jul 13, 2026

Correspondence to
Lucas Squarize Chagas
lsquarize@gmail.com

Open Access

Copyright © 2026 Chagas & Basu. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

This turns an arbitrary cutoff into a measurable and optimizable decision rule. By focusing on outcomes near the threshold, RDD can both diagnose whether the current cutoff is adding or destroying value, and optimize by recommending whether the threshold should be raised, lowered, or kept constant to maximize outcomes while preserving guardrails.

We operationalized this approach into a practical methodology that combines causal inference with optimization. Building on recent econometrics research [2], we use fitted values from RDD models to find alternative values for the thresholds in order to maximize business outcomes, with or without guardrail constraints. For example, in Figure 1, the positive impact of a recommendation in the main outcome metric suggests that a reduction in the threshold would generate additional gain.

To scale adoption, we developed a Python package that automates RDD-based threshold optimization. The package includes bandwidth selection, local linear estimation, robustness checks, visualization tools, and optimization routines that identify the best threshold for different scenarios. This enables seamless integration into production ML pipelines without requiring new randomized experiments or uplift modeling. With online iterations, we guarantee that our decision thresholds are always at an optimized level. We recommend the RDD-based threshold tuning as a crucial workflow for ML model deployment (Figure 2).

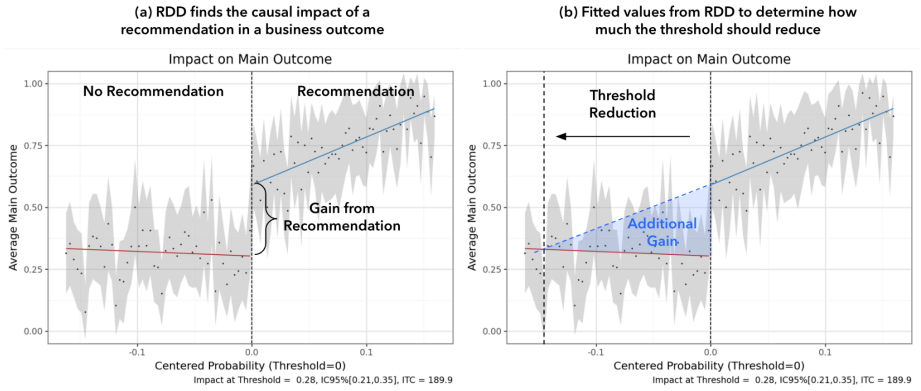


Figure 1. *RDD-based threshold tuning. In the graphs, the estimated score from an ML model is on the horizontal axis, and the average outcome (e.g., conversion) is on the vertical axis. The graphs show the fitted local regressions to the left (red) and right (blue) of a threshold for a recommendation (grey shades are confidence intervals).* ***a)*** *RDD analysis estimates near the threshold show that the recommendation produces a gain in the main outcome.* ***b)*** *Using the fitted values we recommend optimized thresholds aimed at maximizing the business outcome. For illustration, the figures were constructed using simulated data.*

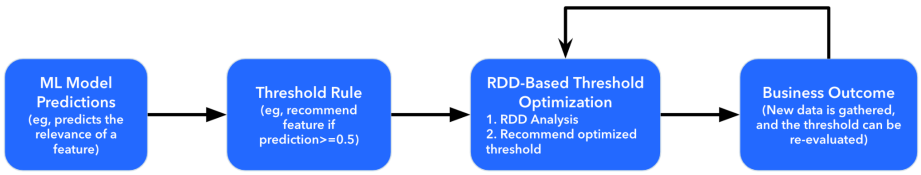


Figure 2. *Workflow for RDD-based threshold tuning. Predictions from an ML model are translated into actions using a threshold. RDD analysis estimates causal effects near the threshold and recommends optimized thresholds that improve business outcomes, considering both the main outcome and possible guardrails.*

3. Applications and Impact

First, we validated the method on simulated data and then applied it within a large-scale system at Intuit. The deployments reported below cover two domains, sequential

recommendation and premium-feature monetization; we discuss in the following section what does and does not carry over to settings with different objectives and constraints. For instance, in a sequential recommendation setting, we originally defined thresholds at 50% for topic recommendations, which still left potential gains on the table. We then tested different approaches to threshold tuning in a real experiment. In one recipe, thresholds were optimized using our RDD-based methodology, while others relied on alternative methods such as arbitrary cutoffs or precision and recall optimization. Results showed that thresholds optimized with RDD performed better than these alternatives, improving our target metrics without harming guardrail metrics.

Another application involved a premium feature recommendation with a trade-off between revenue and units. This is similar to a discount policy that offers a price discount to customers with a high probability of churning, increasing their probability of conversion but reducing the expected revenue per customer. We used our RDD-based method to produce trade-off curves between revenue and conversion, allowing leaders to steer outcomes more scientifically depending on their business goals (Figure 3).

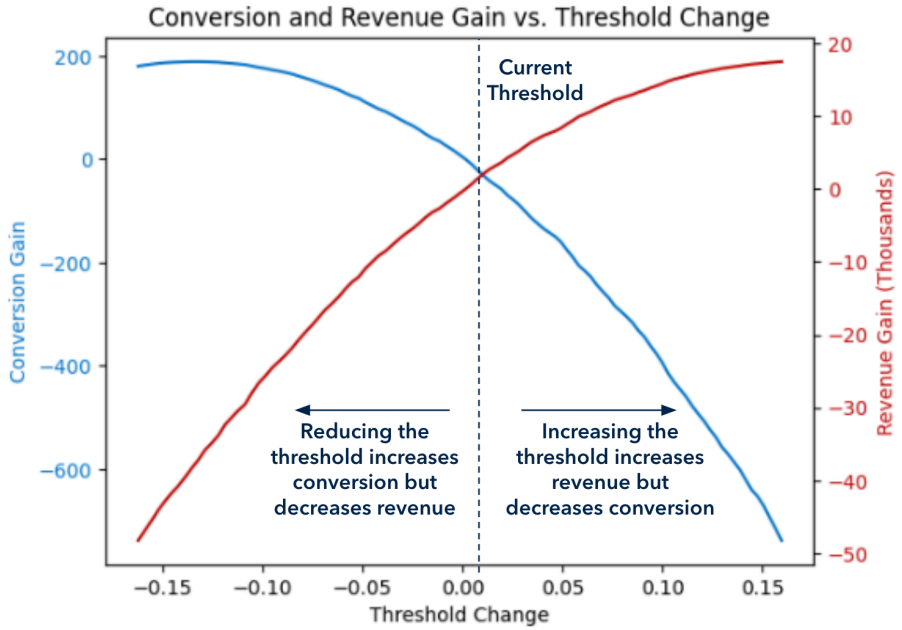


Figure 3. *Business outcome as a function of threshold. RDD-based threshold tuning provides the trade-off between two or more conflicting metrics: the main outcome (e.g., conversion) and a guardrail (e.g., revenue). In this case, a reduction in the threshold increases conversion but reduces revenue, while an increase in the threshold increases revenue but reduces conversion. These curves present leaders the trade-off associated with their business decision regarding the threshold level. For illustration, the figure was constructed using simulated data.*

Across simulated and real-world deployments, the approach has led to measurable improvements in engagement, conversions, and revenue, while preserving quality guardrails. Importantly, it provides a lightweight, repeatable way to adapt thresholds as customer behavior and business priorities evolve.

4. Limitations and Scope

RDD identifies a *local* average treatment effect: the estimate is credible in a neighborhood of the cutoff and says little about units far from it. A recommendation to move the threshold substantially therefore extrapolates beyond the region the design identifies, and the size

of a defensible move is bounded by the bandwidth used for estimation [3]. This is why we frame tuning as an iterative loop (Figure 2) rather than a one-shot optimization: move the threshold modestly, re-estimate on the data generated under the new cutoff, and repeat. The design also rests on assumptions that must be checked rather than assumed. Potential outcomes must be continuous at the cutoff, so that any jump is attributable to the decision rule and not to something else changing at the same score. Units must not be able to precisely manipulate their position relative to the threshold, an assumption that deserves particular scrutiny in adversarial settings where the score is a target to be gamed; density tests around the cutoff [4] are a necessary diagnostic, and our package reports them alongside the effect estimate.

Statistical precision depends on the mass of observations inside the bandwidth, not on the total size of the dataset. Under severe class imbalance, or whenever the score distribution is thin near the cutoff, confidence intervals widen and the recommended threshold becomes correspondingly unstable. We therefore report the effective sample size within the bandwidth and treat a wide interval as evidence for leaving the threshold unchanged, rather than as a point recommendation to act on. A related caution concerns what is being optimized: the business outcomes we target, such as conversion or revenue, are not the same objects as precision and recall, and for rare-event problems the two can move in different directions. A threshold change that improves aggregate business value may leave classification metrics flat or slightly worse, and practitioners with rare-event objectives should specify the outcome they actually care about rather than assuming the two coincide. The difference is one of estimand, not only of units. Precision and recall at a candidate cutoff are computed over the entire score distribution, so they are dominated by observations far from the margin, which are exactly the units a threshold move does not reassign; they summarize how well the score separates labels, not what acting at the margin causes. Restricting attention to a neighborhood of the cutoff is what licenses a causal reading, and it is what the locality noted above buys in exchange for the loss of external validity. The two also sit at different points in the lifecycle: precision and recall are natural during model development, when the score itself is still being selected, whereas the tuning we describe is a post-deployment adjustment to a fixed pipeline, estimated on outcomes that pipeline has already generated.

Our validation covers simulated data together with recommendation and premium-feature monetization settings at Intuit. We do not claim validation in fraud detection, healthcare, credit risk, or other safety-critical domains. Those settings raise concerns the present evaluation does not speak to: scores may be actively manipulated, the costs of false positives and false negatives are sharply asymmetric, and regulatory constraints may make some thresholds inadmissible regardless of measured lift. The framework also inherits the properties of the model that produces the running variable. Because the score itself is the axis along which the discontinuity is measured, a tuned threshold is conditional on a fixed model pipeline; retraining, recalibration, or a shift in feature quality changes the score distribution and requires re-tuning. Reported lift should accordingly be read as the effect of the threshold given that pipeline, not as a property of the threshold alone.

Finally, outcomes and guardrails are measured over a fixed post-decision window. Longer-horizon effects on retention, trust, and future engagement are not captured, and a threshold that improves short-run conversion could in principle degrade them. Nothing in the design forbids a longer window: RDD identifies a discontinuity in whatever outcome is measured against the running variable, and a twelve-month retention outcome is as admissible as a seven-day conversion one. The obstacle is operational rather than inferential. A long-horizon outcome must accumulate under a fixed cutoff before it can be estimated, which bounds how quickly the iterative loop described above can turn and strains the requirement that the scoring pipeline stay unchanged across the measurement window. Substituting short-

term leading indicators is the usual remedy, but a proxy justifies the substitution only if the treatment affects the long-run outcome exclusively through it [5], a condition rarely met by any single indicator and one that constructions such as the surrogate index [6] are designed to relax. We take no position on which surrogates are defensible in the settings we study; treating that question properly is beyond the scope of this paper, and we flag it as the principal obstacle to long-horizon threshold tuning.

RDD is also not the only route to a tuned threshold, and we position it as complementary rather than dominant. Uplift and metalearner approaches estimate heterogeneous effects across the entire score range [7], which RDD does not, but they require randomized or credibly quasi-random assignment. Bayesian optimization [8] and reinforcement learning policies can search the threshold space directly, but need online interaction and many evaluations to converge. What distinguishes RDD is that it requires neither new experiments nor online exploration: it reuses the natural experiment the existing cutoff already creates, which is what makes it deployable against a system already in production. A controlled benchmark against these alternatives remains future work.

5. Contributions and Implications

The examples in the previous section illustrate how the framework translates statistical insights into practical levers for decision-making, without requiring new experiments. Our contributions are fourfold:

- **Conceptual**: Elevating threshold decisions as critical, often-overlooked levers in machine learning deployment.
- **Methodological**: Showing how RDD can be repurposed from evaluation to optimization.
- **Practical**: Delivering a package and workflow that make RDD tuning accessible and scalable.
- **Empirical**: Demonstrating impact in production deployments without costly new experiments.

More broadly, this work positions threshold tuning as a causal optimization problem. By embedding RDD into production pipelines, firms can move from arbitrary cutoffs to adaptive, principled business rules that maximize value at scale.

References

[1] M. D. Cattaneo, N. Idrobo, and R. Titiunik, *A Practical Introduction to Regression Discontinuity Designs*. Cambridge University Press, 2020.

[2] I. Marinescu, S. Triantafillou, and K. Kording, "Regression discontinuity threshold optimization," *PLoS ONE*, vol. 17, no. 11, p. e276755, 2022, doi: 10.1371/journal.pone.0276755.

[3] S. Calonico, M. D. Cattaneo, and R. Titiunik, "Robust Nonparametric Confidence Intervals for Regression-Discontinuity Designs," *Econometrica*, vol. 82, no. 6, pp. 2295–2326, 2014, doi: 10.3982/ECTA11757.

[4] J. McCrary, "Manipulation of the running variable in the regression discontinuity design: A density test," *Journal of Econometrics*, vol. 142, no. 2, pp. 698–714, 2008, doi: 10.1016/j.jeconom.2007.05.005.

[5] R. L. Prentice, "Surrogate endpoints in clinical trials: Definition and operational criteria," *Statistics in Medicine*, vol. 8, no. 4, pp. 431–440, 1989, doi: 10.1002/sim.4780080407.

[6] S. Athey, R. Chetty, G. Imbens, and H. Kang, "The Surrogate Index: Combining Short-Term Proxies to Estimate Long-Term Treatment Effects More Rapidly and Precisely," NBER Working Paper 26463, 2019. doi: 10.3386/w26463.

[7] S. R. Künzel, J. S. Sekhon, P. J. Bickel, and B. Yu, "Metalearners for estimating heterogeneous treatment effects using machine learning," *Proceedings of the National Academy of Sciences*, vol. 116, no. 10, pp. 4156–4165, 2019, doi: 10.1073/pnas.1804597116.

[8] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the Human Out of the Loop: A Review of Bayesian Optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2016, doi: 10.1109/JPROC.2015.2494218.

Open Access | https://doi.org/10.25080/wuxr5665

Docling for Multimodal Retrieval

Mahima Arora[1], and **Aarti Jha**[1]

[1]Red Hat India Pvt. Ltd.

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Retrieval-Augmented Generation (RAG) has become a widely adopted approach for grounding large language models in external knowledge sources. However, most RAG pipelines remain text-centric, overlooking diagrams, figures, tables, and layout structures that contain critical information in scientific, technical, and enterprise documents. This paper presents a multimodal document parsing framework built on Docling, a layout-aware and format-agnostic document understanding library, that preserves document structure by extracting text, tables, and images as distinct retrieval units. The framework employs modality-specific representations: hierarchy-aware text chunks, image chunks enriched with vision-language descriptions, and table chunks that retain relational structure while supporting deterministic SQL querying. These representations are indexed in a unified vector database and served to a tool-augmented agent capable of retrieving and reasoning over textual, tabular, and visual information. Evaluation on scientific and technical documents shows improvements over a text-only baseline in context recall, faithfulness, and answer correctness, with the largest gains on image and table queries.

Keywords retrieval-augmented generation, multimodal document understanding, document parsing, vector databases, large language models, information retrieval

1. Introduction

Retrieval-Augmented Generation (RAG) is a technique that enhances large language models (LLMs) by retrieving relevant information from external knowledge sources at inference time, grounding model responses in factual, domain-specific content without requiring costly retraining. It has become a foundational approach for building accurate, knowledge-intensive applications, and its standard pipeline follows a well-established pattern: documents are loaded, split into text chunks, embedded into a vector database, and retrieved to condition generation. While effective for purely textual corpora, this paradigm breaks down when applied to the complex, multimodal documents that dominate real-world knowledge bases.

Scientific papers, technical manuals, and enterprise reports communicate knowledge through far more than prose. Architecture diagrams, performance tables, charts, and structured layouts often carry the most critical information in such documents. Yet conventional RAG pipelines silently discard these elements during ingestion, reducing rich documents to fragmented text. This introduces two concrete failure modes. The first is a *context gap*: when narrative text references a figure or diagram that was dropped during parsing, the retrieved context is semantically incomplete and the model cannot reason over evidence it never received. The second is the *table trap*: flattening structured tables into linear token sequences destroys row-column relationships and renders the resulting text chunks neither accurately embeddable nor reliably retrievable.

Published Jul 13, 2026

Correspondence to
Mahima Arora
maharora@redhat.com

Open Access

Copyright © 2026 Arora & Jha. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

Addressing these limitations requires treating documents not as streams of text, but as coordinated ecosystems of distinct content types: narrative text, visual elements, and relational data. Each modality demands its own representation and retrieval strategy.

This paper presents a multimodal document parsing and retrieval framework built on Docling, a layout-aware document understanding library. The framework introduces a typed chunking architecture with modality-specific enrichment. Text chunks use a hierarchy-aware hybrid chunking strategy that preserves section context. Image chunks are enriched with vision-language descriptions and classified by semantic type to support targeted retrieval. Table chunks preserve full relational structure as pandas DataFrames, paired with generated descriptions for semantic retrieval and a SQL execution interface for deterministic querying. These typed chunks are indexed in a unified vector database and served to a tool-augmented agent capable of synthesizing answers across all three modalities.

The key contributions of this work are:

1. A formal characterization of the structural failure modes of text-only RAG on complex documents
2. A layout-aware multimodal parsing pipeline built on Docling that extracts text, images, and tables as first-class retrieval units
3. Modality-specific enrichment strategies including vision-language captioning and SQL-backed table querying
4. A tool-augmented agent architecture that combines semantic retrieval with deterministic execution to enable faithful reasoning across modalities
5. An open-source implementation facilitating reproducibility and community adoption

Together, these contributions advance multimodal document understanding as an essential capability for RAG systems operating over real-world scientific, technical, and enterprise document collections.

2. Related Work

R. Zhao *et al.* [1] survey retrieval-augmented generation across modalities including images, code, tables, graphs, and audio, establishing a taxonomy of algorithmic approaches. At the framework level, several systems address multimodal content in RAG pipelines using patterns related to those in this work. LangChain's multi-vector retriever [2] embeds generated text summaries of tables and images for semantic search while storing original content in a separate document store for downstream synthesis. This caption-and-embed pattern makes non-text elements discoverable but does not prescribe modality-specific access patterns at query time; retrieved tables are passed as raw content to the LLM rather than queried deterministically. LlamaIndex [3] provides query engines that translate natural language into executable code against structured data, including `NLSQLTableQueryEngine` for SQL-based access and `SQLAutoVectorQueryEngine` for combining vector retrieval with SQL execution. These enable deterministic tabular querying but focus on text and structured data without incorporating visual content.

This framework builds directly on Docling [4], which provides the layout-aware parsing backbone, including its `HybridChunker` for hierarchy-aware text segmentation and vision-language captioning for image description. What Docling does not provide is the retrieval and reasoning layer: a typed chunk schema unifying text, images, and tables in a single vector space; SQL-backed table querying for deterministic access; and a tool-augmented agent that selects the appropriate access pattern per modality at query time. Like the multi-vector retriever, the framework uses generated descriptions for cross-modal semantic search; like LlamaIndex's query engines, it supports deterministic SQL execution. The con-

tribution is this end-to-end architecture integrating parsing, modality-specific enrichment, unified retrieval, and agent-based reasoning.

3. Methodology

3.1. Document Parsing with Docling

The framework uses Docling [4], a layout-aware document understanding library that performs full layout analysis across PDF, DOCX, PPTX, and XLSX formats. Docling's pipeline extracts three content modalities: (1) **text** with preserved document hierarchy and section context, (2) **tables** using TableFormer structure recognition to recover row-column relationships, and (3) **images** with vision-language generated descriptions that make visual content semantically searchable. All formats are normalized to PDF for consistent processing, and the parser outputs structured document objects containing text blocks, table objects, and image elements, each annotated with page number, section context, and content type.

3.2. System Architecture

The framework transforms Docling's structured document representation into a multimodal retrieval system through five stages: parsing, chunking, indexing, retrieval, and agent-based answer generation. Figure 1 shows the complete pipeline.

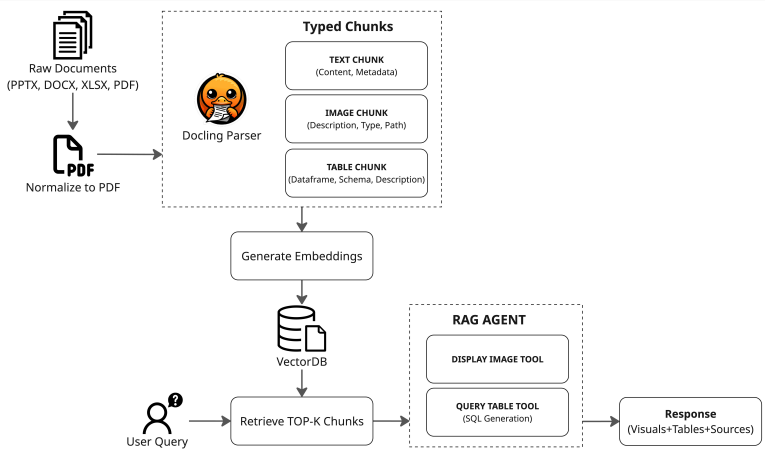


Figure 1. *System architecture: document ingestion, Docling parsing, modality-specific chunking, vector indexing, and agent-based answer generation.*

Three design principles guide this architecture: (1) modality preservation (maintain native representations rather than forcing everything into text), (2) retrieval optimization (generate descriptions that emphasize what questions each element can answer), and (3) deterministic access (provide SQL and display tools for precise operations that embeddings cannot reliably support).

3.3. Chunking Strategy

The chunking stage addresses a fundamental question: how should multimodal document elements be represented for semantic retrieval? Text-only systems flatten everything into character sequences, destroying the structural properties that make tables queryable and visual elements interpretable. The proposed approach defines a typed chunk architecture where each modality receives a representation suited to its information characteristics.

All chunks share a common base schema that enables unified retrieval while preserving modality-specific structure. The key design constraint is that text, images, and tables must be searchable in the same vector space. Rather than joint pixel-text or table-text embeddings, this is achieved by applying standard text embeddings to a `content` field that stores a natural language representation for every modality: narrative prose for text chunks, vision-language generated descriptions for images, and schema-aware descriptions for tables. The base schema tracks provenance and provides this unified embedding interface:

```
class BaseChunk(BaseModel):
    chunk_id: str
    chunk_type: ChunkType  # TEXT, TABLE, or IMAGE
    content: str  #  (text or image/table description)
    sequence_number: int
    source_document: Optional[str]
    source_page: Optional[int]
    parent_heading: Optional[str]  # preserves document hierarchy
```

This unified interface allows all chunk types to be indexed together via the `content` field, while specialized subclasses add modality-specific fields (DataFrames for tables, image paths for visuals) that enable deterministic operations beyond semantic search.

3.3.1. *Text Chunks*

```
# Sample text chunk
if text_chunks:
    c = text_chunks[10]
    print("\n[TEXT CHUNK]")
    print(c)
```

```
[TEXT CHUNK]
chunk_id='text_10' chunk_type=<ChunkType.TEXT: 'text'> content="3.1 Open-domain Question Answering\nOpen-domain question answering (QA) is a
n important real-world application and common testbed for knowledge-intensive tasks [20]. We treat questions and answers as input-output tex
t pairs ( x, y ) and train RAG by directly minimizing the negative log-likelihood of answers. We compare RAG to the popular extractive QA pa
radigm [5, 7, 31, 26], where answers are extracted spans from retrieved documents, relying primarily on non-parametric knowledge. We also co
mpare to 'Closed-Book QA' approaches [52], which, like RAG, generate answers, but which do not exploit retrieval, instead relying purely on
parametric knowledge. We consider four popular open-domain QA datasets: Natural Questions (NQ) [29], TriviaQA (TQA) [24]. WebQuestions (WQ)
[3] and CuratedTrec (CT) [2]. As CT and WQ are small, we follow DPR [26] by initializing CT and WQ models with our NQ RAG model. We use the
same train/dev/test splits as prior work [31, 26] and report Exact Match (EM) scores. For TQA, to compare with T5 [52], we also evaluate on
the TQA Wiki test set." sequence_number=10 source_document='rag_paper.pdf' source_page=4 parent_heading='3.1 Open-domain Question Answering'
extraction_timestamp=datetime.datetime(2026, 2, 3, 18, 50, 59, 676946) word_count=170 char_count=1077
```

Figure 2. *Text chunk schema with hierarchy-aware structure and parent heading context.*

Text chunks solve the problem of preserving narrative context across document structure. Fixed-size chunking (e.g., every 512 tokens) fragments coherent sections and concatenates unrelated paragraphs, breaking the semantic units humans rely on for understanding. The text chunking strategy preserves hierarchy by tracking the parent heading and uses intelligent merging to respect document structure:

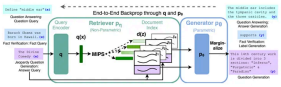
```
chunker = HybridChunker(
    max_tokens=2048,
    merge_peers=True,
    merge_threshold=0.5,
)
```

The chunker merges text blocks within the same section when semantic similarity exceeds the threshold, creating chunks that align with how documents are actually organized. A methods section stays together; introduction and conclusion remain separate. The `content` field contains the merged text, and `parent_heading` preserves the section title, allowing retrieval to return contextually grounded chunks rather than orphaned fragments.

3.3.2. *Image Chunks*

```
# Sample image chunk
from IPython.display import Image as IPImage, display

if image_chunks:
    c = image_chunks[0]
    print("\n[IMAGE CHUNK]")
    if c.image_path:
        display(IPImage(filename=c.image_path, width=400))
    print(c)
```

```
[IMAGE CHUNK]
```



```
chunk_id='image_34' chunk_type=<ChunkType.IMAGE: 'image'> content='This is a **system architecture diagram** illustrating a retrieval-augmen
ted generation (RAG) model.\n\nThe main subject is the **end-to-end backpropagation through a retriever and a generator**\n\nCaption: Figure
1: Overview of our approach. We combine a pre-trained retriever ( Query Encoder + Document Index ) with a pre-trained seq2seq model ( Genera
tor ) and fine-tune end-to-end. For query x , we use Maximum Inner Product Search (MIPS) to find the top-K documents z i . For final predict
ion y , we treat z as a latent variable and marginalize over seq2seq predictions given different documents.' sequence_number=34 source_docum
ent='rag_paper.pdf' source_page=2 parent_heading='1 Introduction' extraction_timestamp=datetime.datetime(2026, 2, 3, 18, 51, 1, 688668) imag
e_path='/Users/maharora/Documents/Work/Other/multimodal-parser/multimodal_parser/data/images/rag_paper_bc273647-e18b-4140-a535-06365eea4a54.
png' image_format='png' image_type='diagram' image_base64='iVBORw0KGgoAAAANSUhEUgAAAxUAAADhCAIAAAAxlA/PAAEAAElEQVR4nOxdB3gUVffvvZ3dzW
```

Figure 3. *Image chunk schema with vision-language description for embedding and type classification.*

Image chunks address the context gap failure mode: when figures are discarded during parsing, retrieval cannot answer visual questions. The solution pairs every extracted image with a vision-language generated description that makes visual content semantically searchable. When a document caption is available (e.g., “Figure 3: Performance comparison”), it is appended to the generated description in the `content` field, preserving figure labels for retrieval. This allows both semantic queries like “what is the system architecture” and label-based references like “Figure 3” to match against image chunks.

Images are classified by semantic type (diagram, chart, screenshot, photo, logo) to support targeted retrieval and filtering:

```
class ImageChunk(BaseChunk):
    """Chunk containing image data."""
    chunk_type: ChunkType = Field(default=ChunkType.IMAGE)
    image_path: Optional[str] = Field(default=None, description="Extracted image path")
    image_format: Optional[str] = Field(default=None, description="png, jpg, gif, webp, etc.")
    image_type: Optional[ImageType] = Field(default="other", description="diagram, chart, etc.")
    image_base64: Optional[str] = Field(default=None, description="Base64 encoded image data")
```

Classification uses a vision-language model to distinguish diagrams, charts, screenshots, photos, and logos, enabling filtered retrieval (e.g., only architecture diagrams for system design questions). The `content` field contains retrieval-optimized descriptions emphasizing what questions the image can answer, producing semantic descriptions like “system architecture diagram showing encoder-decoder structure” rather than pixel-level enumeration.

3.3.3. *Table Chunks*

```
# Sample table chunk
if table_chunks:
    c = table_chunks[1]
    print("\n[TABLE CHUNK]")
    print(c)
```

```
[TABLE CHUNK]
chunk_id='table_36' chunk_type=<ChunkType.TABLE: 'table'> content='This table presents numerical performance scores for various Natural Lang
uage Processing (NLP) machine learning models (SotA, BART, RAG-Tok, RAG-Seq) on text generation and classification tasks. It details metric
s' sequence_number=36 source_document='rag_paper.pdf' source_page=6 parent_heading='4.1 Open-domain Question Answering' extraction_timestamp
=datetime.datetime(2026, 2, 3, 18, 51, 13, 71936) dataframe=       Model.Model Jeopardy.B-1 Jeopardy.QB-1 MSMARCO.R-L MSMARCO.B-1  \
0              SotA           –            –      49.8 *      49.9 *
1              BART        15.1         19.7        38.2        41.6
2  RAG-Tok. RAG-Seq.        17.3         22.2        40.1        41.5
3                           14.7         21.4        40.8        44.2

  FVR3.Label FVR2 Acc..Label
0       76.8          92.2 *
1       64.0            81.1
2       72.5            89.5
3       72.5            89.5   columns=None num_rows=4 num_cols=7
```

Figure 4. *Table chunk schema with DataFrame preservation and description for semantic retrieval.*

Table chunks solve the table trap: flattening a table to text destroys the relational structure that makes it valuable. A benchmark comparison table showing model performance across datasets becomes an unstructured paragraph that embeddings cannot reliably parse for

precise comparisons or metric lookups. The approach preserves the full DataFrame while adding a semantic description for retrieval:

```
class TableChunk(BaseChunk):
    """Chunk containing table data as pandas DataFrame."""
    model_config = ConfigDict(arbitrary_types_allowed=True)
    chunk_type: ChunkType = Field(default=ChunkType.TABLE)
    dataframe: pd.DataFrame = Field(description="Table data as pandas DataFrame")
    columns: Optional[List[str]] = Field(default=None, description="List of column names")
    num_rows: Optional[int] = Field(default=None, description="Number of rows")
    num_cols: Optional[int] = Field(default=None, description="Number of columns")
```

Tables require a two-stage strategy. First, semantic retrieval using generated descriptions that emphasize schema, sample data patterns, and query-answering capability (e.g., "performance metrics across five benchmark datasets"). Second, SQL execution for deterministic data access, once retrieved, the agent generates and executes queries against the DataFrame:

```
def query_table(table_index: int, sql: str) -> str:
    chunk = table_chunks[table_index]
    result_df = ps.sqldf(sql, {"df": chunk.dataframe})
    return result_df.to_string(index=False)
```

For example, a query like "Which model achieved the highest accuracy on the SQuAD benchmark?" retrieves a benchmark results table via description matching, then executes `SELECT model, accuracy FROM df WHERE dataset='SQuAD' ORDER BY accuracy DESC LIMIT 1` to extract the exact answer. This hybrid approach, semantic discovery plus deterministic execution, provides both flexibility and precision.

3.4. Indexing and Retrieval

All three chunk types flow into a unified indexing pipeline. The key insight is that text chunks, image descriptions, and table descriptions are all natural language strings that can be embedded into the same semantic space, enabling cross-modal retrieval where a text query can match against any content type. The framework uses a vector database for storing embeddings and metadata with efficient similarity search.

Indexing batch-embeds all `content` fields and stores them alongside full chunk metadata:

```
def index_file(file_path: str) -> int:
    chunks = parser.parse(file_path)
    contents = [chunk.content for chunk in chunks]
    embeddings = embed_model.embed_documents(contents)

    for chunk, embedding in zip(chunks, embeddings):
        payload = chunk.model_dump(exclude={"dataframe"})
        if chunk.chunk_type == ChunkType.TABLE:
            payload["table_data"] = chunk.dataframe.to_dict(orient="records")
        store_vector(embedding, payload)
```

For tables, the DataFrame is serialized as a dict and stored in the payload rather than embedded, since row-column data is not directly embeddable. This keeps the structured representation available for SQL execution while the description enables semantic matching.

Retrieval performs standard dense vector search. A query like "What is the proposed model architecture?" is embedded and matched against all chunk content fields, returning the top-k most similar chunks regardless of type. A diagram description like "architecture

diagram showing transformer encoder-decoder with attention mechanisms" can score highly despite being a visual element, solving the context gap by making visual content discoverable through text queries.

3.5. Agent-Based Answer Generation

Retrieval alone is insufficient for multimodal documents because different modalities require different interaction patterns. Text can be synthesized directly, but images must be displayed and tables must be queried. The framework employs a tool-augmented agent architecture [5] that combines semantic retrieval with modality-specific tools and inline rendering capabilities.

The agent receives retrieved chunks separated by type (text, image, table) and decides which operations to invoke:

- **For text chunks**: Synthesize information directly from content
- **For image chunks**: Call `display_image(image_index)` to trigger inline rendering
- **For table chunks**: Generate and execute SQL via `query_table(table_index, sql)`

3.5.1. *Inline Display Mechanism*

```
def display_image(image_index: int) -> str:
    chunk = image_chunks[image_index]
    displayed_images.append({
        "chunk_id": chunk.chunk_id,
        "path": chunk.image_path,
        "base64": chunk.image_base64,
    })
    return f"Image ready. Use placeholder [IMAGE:{len(displayed_images)-1}]
             in your answer on its own line between paragraphs."
```

Rather than returning image URLs or table data for external rendering, the agent employs a placeholder-based inline display system. When `display_image` is invoked, the tool stores the image data (base64 or file path) in a display queue and returns instructions to insert a placeholder token `[IMAGE:X]` in the answer text:

The agent is instructed to place these placeholders on separate lines, never inline within sentences, ensuring clean paragraph-image-paragraph flow in rendered output. For tables, `query_table` executes SQL and returns formatted results as text while storing the structured result in a `queried_tables` list for reference.

This tool-calling workflow enables precise operations that pure text generation cannot reliably produce. Consider a query like "What is the average F1 score across all baselines?" A text-only system must generate the answer from a flattened table representation, risking arithmetic errors. The agent retrieves the benchmark results table, generates `SELECT AVG(f1_score) FROM df WHERE model_type='baseline'`, executes it deterministically, and returns the exact result.

Similarly, visual queries like "What is the attention mechanism architecture?" retrieve an architecture diagram (via description matching) and trigger `display_image`, which inserts `[IMAGE:0]` into the answer. Downstream rendering replaces this placeholder with the actual image, presenting visual evidence directly rather than attempting to describe complex spatial relationships in generated text.

The agent workflow is:

1. Embed query and retrieve top-k chunks via semantic search
2. Separate retrieved chunks by type (text, image, table)

3. For text chunks, synthesize answer from content
4. For image chunks, invoke display tool and insert placeholder in answer
5. For table chunks, generate SQL query, execute for exact data
6. Return answer with placeholders plus image/table metadata for rendering

When retrieval returns multiple chunks of the same type, all chunks are presented to the agent in a single context, grouped by modality with numerical indices. The agent selects which chunks to act on, querying multiple tables or displaying multiple images, within a single reasoning pass. For table querying, SQL execution errors such as syntax mistakes or invalid column references are caught and returned to the agent as descriptive error messages, enabling it to revise the query and retry within the same turn.

This architecture demonstrates that multimodal RAG requires heterogeneous access patterns, text generation for prose, tool calls for structure, inline placeholders for visual elements, rather than forcing all modalities through a single generation interface.

4. Results

The framework is implemented as open-source software [6] using Gemini 2.5 Pro [7] for agent reasoning and tool calling, Gemini Embedding [8] for vector representations, Qdrant [9] for vector storage, Docling [4] for document parsing, LangGraph [10] for agent orchestration, and pandasql [11] for SQL execution against DataFrames.

4.1. Capability Demonstrations

Three demonstrations validate the framework's ability to handle queries that text-only RAG systems cannot answer. Each demonstration uses research paper PDFs as source documents and illustrates a distinct modality handling strategy.

Text Synthesis. The agent performs cross-sectional synthesis by connecting information across fragmented text chunks (Figure 5). The hybrid chunking strategy merges semantically related text blocks from the same section, enabling coherent answers that integrate information from multiple sources while avoiding over-fragmentation.

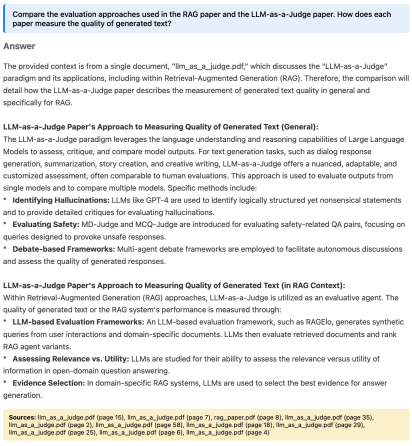


Figure 5. *Text synthesis: cross-sectional analysis connecting information across multiple chunks.*

Visual Reasoning. The agent identifies visual query intent, locates corresponding image chunks using vision-language descriptions, and displays diagrams inline (Figure 6). This addresses the context gap problem where text-only systems cannot answer queries about discarded figures, vision-language descriptions make visual content semantically searchable.

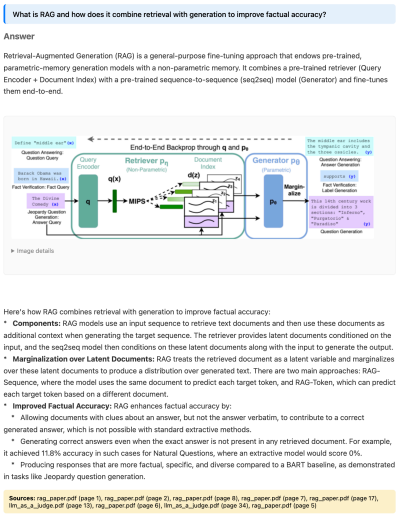


Figure 6. *Visual reasoning: diagram retrieval using vision-language descriptions and inline display.*

Structured Precision. The agent combines semantic search over table descriptions with deterministic SQL execution against DataFrames (Figure 7). This addresses the table trap problem where text-only systems hallucinate from flattened representations, the dual approach enables exact numerical results without arithmetic errors.

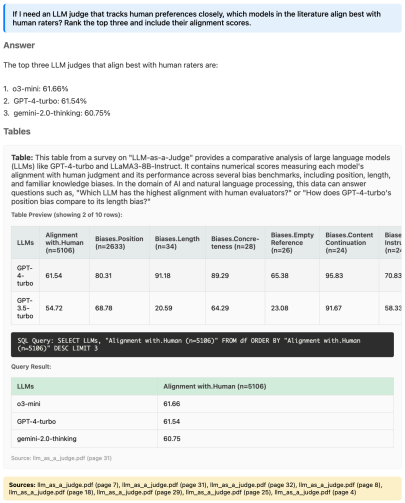


Figure 7. *Structured precision: SQL-based table querying with semantic search and deterministic execution.*

To quantify the impact of modality preservation, the multimodal pipeline was evaluated against a text-only baseline using RAGAS metrics [12]. The evaluation uses a corpus of 110 research papers [13] with 300 queries constructed across four categories: text-only (28%), table (28%), image (24%), and cross-modal (20%) queries requiring synthesis across modalities. The text-only baseline uses identical chunking and retrieval parameters but converts tables to inline Markdown text and discards images during ingestion. Results are averaged over three independent runs; 95% confidence intervals are reported.

Table 1. *Multimodal (MM) vs. text-only (TO) pipeline comparison by query type. CR = Context Recall, F = Faithfulness, AC = Answer Correctness. Overall values show mean ± 95% CI over three runs.*

Query Type	CR (MM)	CR (TO)	F (MM)	F (TO)	AC (MM)	AC (TO)

Text	91.8%	91.0%	95.2%	94.5%	79.6%	78.8%
Table	89.2%	82.3%	94.8%	90.3%	76.5%	69.2%
Image	83.5%	64.7%	92.5%	84.8%	70.3%	60.5%
Cross-modal	77.4%	74.2%	90.7%	89.5%	66.8%	64.7%
Overall	**86.2 ± 2.3%**	**78.9 ± 1.3%**	**93.6 ± 1.4%**	**90.0 ± 0.9%**	**74.0 ± 2.4%**	**68.9 ± 1.7%**

The per-modality breakdown reveals that gains concentrate on image queries, where the baseline discards the primary evidence source, context recall improves by 18.8 percentage points. Table queries show a moderate gap (6.9 pp in context recall) since the baseline retains table content as inline text but lacks deterministic SQL access for precise lookups. Text queries show minimal difference (0.8 pp), confirming that the multimodal pipeline preserves text handling quality.

These results validate that multimodal RAG requires both modality-specific representations and heterogeneous access patterns. The bottleneck has shifted from LLM reasoning to data quality and modality preservation.

5. Discussion

5.1. Runtime Feasibility

The SQL execution pathway requires materializing each retrieved table as an in-memory DataFrame at query time. For individual document tables, typically tens to hundreds of rows, this overhead is negligible, and the execution engine (pandasql over SQLite) operates in-process without external infrastructure. At larger corpus scales, tables and images can be stored in external storage (e.g., S3 or cloud drives) as CSV and image files respectively, rather than serialized inline in vector payloads. The vector store retains only metadata paths, and content is loaded on demand when the agent invokes a tool, with caching for frequently accessed tables to avoid repeated retrieval. The primary computational cost lies in ingestion rather than retrieval: vision-language captioning and table description generation require LLM inference per element, making ingestion time proportional to the number of non-text elements in the corpus.

5.2. Limitations

The framework's retrieval quality for images and tables depends on the fidelity of their generated descriptions. These descriptions are produced once during ingestion and stored as static text, if the vision-language model omits details present in the original image or table, those details become unretrievable through semantic search. Re-ingestion with an improved captioning model is required to recover missed information. Additionally, the agent's ability to answer table queries depends on correct SQL generation; while the framework supports error-driven retry, persistent failures on tables with ambiguous or non-standard schemas cause the agent to fall back to the natural language description, sacrificing deterministic precision. Complex table structures such as merged cells or nested headers remain challenging for the underlying structure recognition model.

6. Conclusion and Future Work

This paper presents a multimodal document parsing framework for RAG systems that addresses the fundamental limitations of text-only approaches. By preserving document structure through specialized chunking strategies for text, images, and tables, the system

enables retrieval of visual and structured information that traditional pipelines discard. The results demonstrate that the bottleneck in RAG performance has shifted from LLM reasoning capabilities to data quality and modality preservation, modality-specific chunking enables retrieval relevance that pure text embeddings cannot achieve, while tool-calling architectures prove essential for handling heterogeneous access patterns across text generation, image display, and SQL execution. Future work should develop comprehensive multimodal RAG benchmarks, extend the framework to video and equations, and explore domain-specific adaptations for scientific literature and technical documentation. The open-source implementation [6] facilitates community adoption and experimentation.

References

[1] R. Zhao *et al.*, "Retrieving Multimodal Information for Augmented Generation: A Survey," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 4736–4756. doi: 10.48550/arXiv.2303.10868.

[2] LangChain AI, "Multi-Vector Retriever." [Online]. Available: https://python.langchain.com/docs/how_to/multi_vector/

[3] LlamaIndex, "LlamaIndex: Data Framework for LLM Applications." [Online]. Available: https://docs.llamaindex.ai/

[4] N. Livathinos *et al.*, "Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion," *arXiv preprint arXiv:2501.17887*, 2025, [Online]. Available: https://arxiv.org/abs/2501.17887

[5] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022, doi: 10.48550/arXiv.2210.03629.

[6] M. Arora, "Multimodal Parser: Document Parsing for RAG." [Online]. Available: https://github.com/mahimaarora/multimodal-parser

[7] G. Comanici, E. Bieber, M. Schaekermann, and others, "Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities," *arXiv preprint arXiv:2507.06261*, 2025, doi: 10.48550/arXiv.2507.06261.

[8] J. Lee *et al.*, "Gemini Embedding: Generalizable Embeddings from Gemini," *arXiv preprint arXiv:2503.07891*, 2025, doi: 10.48550/arXiv.2503.07891.

[9] Qdrant Team, "Qdrant: Vector Database for the Next Generation of AI Applications." [Online]. Available: https://qdrant.tech/

[10] LangChain AI, "LangGraph: Build resilient language agents as graphs." [Online]. Available: https://github.com/langchain-ai/langgraph

[11] pandasql Contributors, "pandasql: Query pandas DataFrames using SQL syntax." [Online]. Available: https://github.com/yhat/pandasql

[12] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," *arXiv preprint arXiv:2309.15217*, 2023, doi: 10.48550/arXiv.2309.15217.

[13] M. Arora, "Research Papers Corpus for Multimodal RAG Evaluation." [Online]. Available: https://huggingface.co/datasets/mahimaarora025/research_papers

Open Access | https://doi.org/10.25080/rdvr6268

Hash all the things: Caching for fast notebook restarts

Dylan Madisetti[1], **Myles Scolnick**[1], and **Akshay Agrawal**[1]

[1]marimo

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

We describe a caching mechanism that lets reactive notebooks restart without re-running their expensive cells. The mechanism is built into marimo, a reactive Python notebook that models the notebook as a dataflow graph. Each cell's cached result is identified by a key built from fingerprints (hashes) of the cell's code and inputs. Input values whose bytes are accessible are hashed directly, while other inputs are represented by the key of the cell that produced them, computed the same way. These contributions propagate through the dataflow graph, so editing one cell can invalidate cached results downstream of it. Cached values are stored on disk and loaded at evaluation, so they can be reused across independent runs of the same notebook. These values are also bundled into marimo's static export, a standalone web page written in HyperText Markup Language (HTML) that runs the notebook through WebAssembly (WASM), so readers whose only Python runtime is a browser can open a notebook with its expensive results and trained models already in place. Empirically the cache lookup is as fast or faster than representative baselines on microbenchmarks of variably sized payloads, though the magnitude of the advantage is hardware-dependent.

Keywords reactive notebooks, content-addressed caching, memoization, reproducibility, dataflow, marimo, Python

1. Introduction

Notebooks underpin much of scientific Python, but most notebooks cannot be re-executed from scratch. In a survey of 1.4 million public Jupyter notebooks, only about a quarter re-executed top to bottom without raising an error [1]. Traditional notebooks like Jupyter, built on the imperative `ipykernel`, are read-evaluate-print loops (REPLs) in which running each block of source code, or *cell*, mutates shared global state. As a result, these notebooks accumulate hidden state, and the outputs saved in the notebook file can differ from those of a fresh top-to-bottom run.

Reactive notebooks close this gap by treating each cell as a node in a dataflow graph. The notebook determines, for each cell, which variables it reads (its *references*, or `refs`) and which variables it defines (its *definitions*, or `defs`). From these, a reactive notebook derives a deterministic execution order based on data dependencies rather than on the order of cells on the page. Running a cell removes the cell's previous variable bindings from memory, updates the dataflow graph if the cell's code changed, and re-runs the cells that depend on it, which minimizes hidden state. Notable reactive notebooks include Pluto.jl [2], Observable [3], and Livebook [4]. Reactive notebooks descend from a longer tradition of direct-manipulation programming environments [5], in which editing the source is itself the act that updates the running program. marimo [6] is a reactive notebook for Python, and its caching mechanism is the subject of this paper.

To mitigate unnecessary re-runs, reactive notebooks offer runtime configuration, such as lazy executors that mark cells stale instead of running them; however, these primitives

Published Jul 13, 2026

Correspondence to
Dylan Madisetti
dylan@marimo.io

Open Access

Copyright © 2026 Madisetti *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

require user intervention. We exploit their deterministic execution order to build a caching mechanism that automatically skips recomputation of an expensive cell whose code and inputs are unchanged.

Caching for notebooks and Python is not new; we review prior systems in Background and Related Work. Each prior method asks the user to opt in at a boundary, such as a decorated function, document chunk, or session. In contrast, reactive notebooks already draw a boundary around every cell, so the author never has to choose where caching applies.

The caching mechanism we propose, and whose implementation we share, was designed to satisfy three properties.

- Skip expensive recomputation when a cell's references and source are unchanged.
- Preserve reactive determinism by ensuring that changes tracked by the dataflow graph cannot produce stale (false-positive) hits.
- Make cached artifacts transportable through marimo's static WASM/HTML export.

Out of scope are full session restoration in the sense of Kishu [7], distributed execution, and reproducibility of arbitrary Python notebooks [1]. Our contribution is deterministic reuse between notebook sessions that follow marimo's reactive principles. The static browser export additionally makes cached results available to readers running the notebook through WebAssembly.

2. Background and Related Work

A memoized function, or *memo* function [8], stores the result of a call and returns that result when it receives the same inputs again. To find the stored result, the function associates each set of inputs with a *cache key* as an identifier used to index the cache. marimo constructs this key with a *hash*, a short, fixed-size fingerprint derived from data. Matching hashes can stand in for matching data because cryptographic hash functions make it improbable that two different inputs have the same fingerprint. When a system identifies a value by hashing the value's own bytes, the value is said to be *content-addressed*. To reuse a cached cell result, marimo must derive the same key whenever its inputs are unchanged.

A cell's inputs are the variables it reads, which may be defined in other cells or in the global environment. In a reactive notebook like marimo, these variables are statically known prior to execution, as the cell's "refs" derive a dependency graph from source. At run time, marimo additionally knows the values currently bound in memory. It can therefore derive a cell's key from the graph, the cell's code, and the current reference values, rather than from the history of which cells happened to run. Cell-level dataflow tracking has an earlier antecedent in [9].

IPyflow and nbsafety [10], [11] take a different route to reactivity. They retrofit reactivity onto Jupyter by tracing execution to record which variables each cell reads and writes, then flag or re-run cells whose inputs have become stale. Because the resulting graph contains only dependencies observed during execution, different execution histories can produce different graphs for the same notebook. marimo instead derives the graph from source, so the same source produces the same dependencies before any cell runs.

The dependency graph identifies what contributes to a cell, but the cache must still reduce that information to a stable key. For this step, marimo borrows from build systems. Build Systems à la Carte [12] separates a *scheduler*, which decides the order in which tasks run, from a *rebuilder*, which decides whether each task must run again. marimo's reactive dataflow engine supplies the scheduler, while its cache acts as the rebuilder by comparing keys computed from a cell's code and inputs (Cache Keys).

The Nix package manager provides the model for extending a key through the dependency graph [13], [14]. Nix identifies a package with a hash of its inputs, including the hashes of the packages from which it was built. This *recursive hash* makes the package's identity cover its dependency tree. marimo applies the same construction to a reactive notebook: a cell's key incorporates the keys of upstream cells when their values cannot be content-addressed directly. Data-engineering systems use related constructions at coarser granularity. Bauplan and Nessie hash pipeline stages [15], while the workflow engines Nextflow (`-resume`) and Snakemake (`--cache`) key task results on code, parameters, and input hashes [16], [17].

Existing caching systems for Python and notebooks differ in both their unit of reuse and the information they require from the author. IncPy modifies CPython, the reference implementation of Python, to memoize function calls automatically [18]. knitr caches chunks of literate documents whose dependencies the author declares by hand [19]. jupyter-cache re-executes a notebook as a whole when any code cell changes [20]. Streamlit asks the author to choose between `cache_data`, for values identified by content, and `cache_resource`, for values identified by what produced them [21]. marimo instead uses its key construction (Section 1) to make that choice for each reference, while its dataflow graph supplies a reuse boundary around every cell.

There are a few existing scientific-Python memoizers (for comparison to this work see Evaluation). The most similar, mandala [22], provides the end-to-end comparison by memoizing execution inside a `with storage:` context, computing content addresses with `joblib.hash`, and recording which calls produced which values. Alternatively, diskcache [23] provides the storage control by storing values under byte keys supplied by the caller, allowing the evaluation to hold key construction constant while measuring storage cost. Other notable work includes Kishu [7] and ElasticNotebook [24], which checkpoint and migrate notebook state, but are not directly comparable to marimo's mechanism.

3. Cache Keys

In computational caching, a false positive (restoring a value that the code would not have produced) is unacceptable. A false negative (failing to find a stored value and recomputing it) wastes time but does not return an incorrect result. The key derivation must therefore change whenever a value the cell reads changes, but remain stable under superficial edits such as reformatting or comment changes.

Consider two obvious ways to derive a key. The first derivation hashes the value of every reference the cell reads. A marimo notebook can attempt this because, at runtime, it exposes both the dataflow graph and the reference values bound in memory. This derivation fails for Python values that expose nothing stable to hash. An object's memory address is neither stable nor meaningful as an identity. A weak reference identifies an object rather than its contents. A foreign function interface (FFI) can expose an object implemented in C, C++, Rust, or another language, hiding state that the foreign code can mutate. Such opaque extension objects may expose neither their underlying bytes nor a stable text representation. The second derivation hashes the cell's source bytes alone. This key is computable, but it does not change when the cell's reference values change and can therefore produce false positives.

marimo combines the two derivations: it hashes a reference value when possible and otherwise uses the key of the cell that produced the reference. Because the producing cell's key is derived by the same construction, this substitution extends recursively through the dataflow graph. This recursive use of producer keys borrows from build systems such as Nix [13], where a package's identity includes the identities of its dependencies. Section 1 describes how marimo constructs keys over the graph. Section 2 then explains invalidation:

editing a cell invalidates only downstream cached results. Neither derivation observes untracked external state such as filesystem contents, network responses, wall-clock time, or randomness. Limitations and Discussion describes the mechanisms that cover some of these cases and the remaining gaps.

3.1. Constructing the key

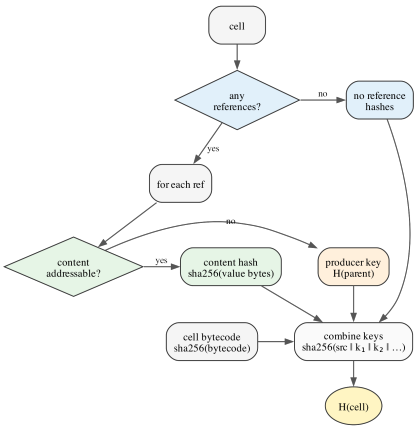
The following will describe computing cache keys from source code and cell references and will outline the mechanism for combining them. Throughout, H denotes the hash function, and $H(c)$ denotes the hash of cell c.

3.1.1. *Hashing source code*

The cells `y = x + 1` and `y = x - 1` may have exactly the same "inputs", but they compute different outputs. As such, it would be incorrect to strictly use a cell's inputs to determine its cache key. Under the assumption that code is deterministic (see Limitations and Discussion for a discussion of side effects), it follows that a cell's code must contribute to its cache key. By using cell code in key construction, basic invalidation is achieved (editing a cell's executable code changes its key, so its stale result is not reused).

Source-code formatting and comments should not change a code hash, but details such as the Python version potentially should. marimo therefore hashes the compiled bytecode rather than the source text. Compilation discards edits like comments and formatting, making bytecode stable within a specific Python version. A caveat is that cached results may therefore not transfer across interpreter upgrades. As a result, marimo's browser export (Section 3) requires a Python version compatible with the browser's Pyodide version.

3.1.2. *Hashing references*




```python
def derive_key(cell, graph, scope) -> HashKey:
    refs = ScopedVisitor(cell.module).refs          # static analysis
    refs = normalize_scope(refs, scope)             # UI/state -> values
    h, kind = sha256(), "Pure"                      # no refs: pure cell

    if refs:                                        # content-address first
        kind = "ContentAddressed"
        refs, serial = content_serialize(refs, scope)
        for ref in sorted(serial):
            h.update(serial[ref])                   # buffer-protocol view

    if refs:                                        # substitute producers
        kind = "ExecutionPath"
        for parent in producing_cells(refs, graph):
            h.update(derive_key(parent, graph, scope).digest)

    if refs - scoped_refs:                          # same-cell context
        kind = "ContextExecutionPath"
        h.update(hash_module(cell.context))

    h.update(hash_module(compile(cell.module)))     # bytecode, not source
    return HashKey(h.digest(), kind)
```

Figure 1. *The cascading hashing mechanism for references. The left panel traces how references contribute to a cell key; the right shows the derivation over the full cell, abridged from* `BlockHasher.__init__` *(*`marimo/_save/hash.py`*).*

Hashing compiled code accounts for changes to the body of a cached cell, but not for changes to the values that the cell uses. If the cell has no references, its code hash is the only contribution to its key. Otherwise, the cache key must account for values defined outside of the cached cell. Each reference contributes either a hash of its value or the key of the cell that produced it. We call this choice the *key dispatch*. The dispatch algorithm is illustrated in Figure 1 and outlines three resulting cell-key categories: code only, content-addressed, and producer substitution, labeled in the figure as `Pure`, `ContentAddressed`, and `ExecutionPath`, respectively. For blocks and functions cached within a cell, marimo also incorporates

the surrounding cell's code, a special case labeled `ContextExecutionPath` in the figure and described in Section 2.4.

A reference is **content-addressed** when marimo derives a hash from the reference's value. Immutable values, such as numbers, strings, and frozen collections, are hashed this way. Before hashing a reference to a user interface (UI) element such as a slider, marimo replaces the element with its current value, so moving the slider changes the hash. A value that exposes its underlying bytes through Python's buffer protocol is hashed from those bytes. This case covers NumPy ndarrays and other objects advertising NumPy's array interface, making it important for scientific computing. Here, the hash is computed from the contiguous buffer without serialization, an approach borrowed from joblib [25] and mandala [22].

A reference is hashed via "execution path" or **producer substitution** when marimo cannot content-address the reference but knows which upstream cell initialized the value. In this case, it substitutes the producing cell's key for the value's hash. The producing cell's key is built by this same construction, so it covers the producer's code and inputs and recursively includes any producer keys substituted upstream. A change to one of these contributions therefore changes the reference's hash.

3.1.3. *Combining hashes*

All hashing in this construction uses SHA-256, as Figure 1 shows. The hashes of the code and of the references are combined into one in the combine step at the bottom of the figure. Every reference hash, taken in sorted reference-name order so that the result does not depend on the order in which references were processed, is fed into a single SHA-256 computation together with the bytecode hash. The resulting digest is the cell's key, $H(c)$. Because collisions between different inputs to SHA-256 are improbable (Background and Related Work), matching keys can safely stand in for matching code and reference contributions.

3.2. Invalidation

Since the construction of Section 1 is recursive, a cell's key may be derived from its producer's key, which in turn may be derived from that producer's key. In practice, derivations do not have to explicitly walk the whole ancestry of a value to compute a key. When a cell c finishes executing, marimo records its key $H(c)$, and when a downstream cell's key later needs $H(c)$ for producer substitution, marimo reuses the recorded value. Each cell's key is therefore computed at most once per execution.

A cached result is *invalidated* when its cell's key changes and the new key no longer matches any stored entry. When this happens, the next lookup misses and the cell recomputes. Invalidation is not an action that marimo performs, since nothing is deleted; the old entry simply stops being addressable.

To keep false negatives rare, invalidation should also be limited, and an edit should not invalidate results it cannot affect. Additionally, updating keys should be cheap. The two subsections below establish each property in turn.

3.2.1. *Invalidation does not miss tracked changes*

When the bytes of a content-addressed reference change, the hash changes, and so does every key built from it. However, the "producer substitution" case requires an argument tied directly to the notebook's execution. Because marimo is a *reactive* notebook, a value can change only if the cell that produced it re-runs, and a cell reactively re-runs only when its own code or inputs change, which are the components of its key. Provided that cell bodies

are deterministic, an unchanged producer key therefore implies an unchanged value; Limitations and Discussion discusses side effects and mutations that bypass the dataflow graph.

3.2.2. *Invalidation is limited and cheap*

Wherever producer substitution occurs, a cell's key contains the keys of cells upstream of it. By this definition, the notebook's keys form a Merkle directed acyclic graph (DAG) [26], since each node's fingerprint depends on the fingerprints of the nodes on which it depends. Two properties follow. First, editing a cell can change keys only in the cells downstream of the edit, so every other cached result in the notebook remains valid. Second, the change does not always reach everything downstream: where a re-run cell produces byte-identical values, its content-addressed consumers keep their old keys, and the propagation stops there. After an edit, marimo recomputes keys only for cells reached by reactive execution, and every other cell's recorded key is reused.

3.2.3. *A worked example*

Figure 2 traces the key construction on the four-cell PyTorch notebook composed of a seed, a random tensor generated from the seed, a small neural network, and a forward pass that applies the network to the tensor. The italic label under each cell in the figure classifies that cell's own key, as computed by marimo's hasher. By cell label, `a` is `Pure` (no references), `b` and `c` are `ContentAddressed` (every reference hashed by value), and `d` is `ExecutionPath` (one reference substituted a producer's key).

```
# a · seed          # b · random input              # c · model              # d · forward pass
seed = 7            x = rng(seed).normal(64)       model = TinyNet()        y = model(x)
```

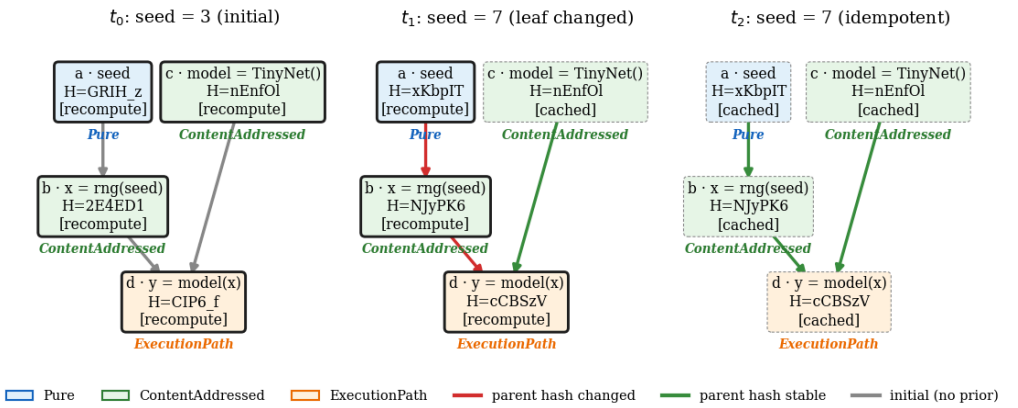


Figure 2. *A worked example of the recurrence on the four-cell graph written out above. Every hash and branch label in the figure is produced by marimo's real hasher on a compiled cell graph at render time. The seed is hashed as an immutable value, the tensor is hashed through its buffer, and the network (`TinyNet`), which exposes no bytes to hash, is represented by the key of the cell that constructed it, covering the `Pure`, `ContentAddressed`, and `ExecutionPath` cases discussed in Section 1. Changing the seed ($t_0 \to t_1$) invalidates `a`, `b`, and `d` (red edges), while `c` stays cached because the seed is not among its references. Re-rendering with the same seed ($t_1 \to t_2$) leaves every hash unchanged (green edges), so the rebuilder reuses every result. The italic label under each box names the dispatch branch that cell exercises.*

3.2.4. *Caching blocks and functions*

The cached unit is not always a whole cell: it can be a block of code inside a cell, or a function (Using marimo's cache). A reference defined earlier in the same cell as a cached block has no parent cell whose key could stand in for it. Instead, the code surrounding the block is folded into the key, a special case of producer substitution that the implementation

calls `ContextExecutionPath`. For a cached function, the same dispatch classifies the function's arguments at call time.

4. Storage and Loading

On a cache hit, marimo must restore the variables the cached cell would have defined. marimo provides a few "loaders" that differ in how they store and load values. However, restoring always has two parts: *lookup* finds the stored entry whose key matches $H(c)$, and *loading* deserializes the entry's values into memory. These two parts do not have to happen at the same time, because a notebook does not always need a value's bytes when it finds the corresponding cache entry. For example, a downstream cell may take a variable and pass it to a third cell without inspecting it.

The loader, `LazyLoader`, demonstrates the lookup / loading separation. On write, the loader writes a JSON (JavaScript Object Notation) manifest listing each variable. For each variable, the manifest records either the value itself (small primitives are stored inline) or the path to an external file with the value. Its `cache_type` field records which strategy of Section 1 produced the key. For example, an abridged manifest for a cache named `training` whose cell defined a value, `seed`, and a large array, `x`, might read:

```
{
  "hash": "9V5v6Cji…",
  "cache_type": "ContentAddressed",
  "defs": {
    "seed": {"primitive": 7},
    "x": {
      "reference": "training/9V5v6Cji…/x.npy",
      "type_hint": "numpy.ndarray"
    }
  },
  "stateful_refs": [],
  "meta": {
    "version": 5,
    "blob_hashes": {
      "training/9V5v6Cji…/x.npy": "e3b0c4…"
    },
    "signer_public_key": "-----BEGIN PUBLIC KEY-----\n…",
    "signature": "…"
  }
}
```

When signing support is available, the blob hashes and signature let marimo verify the external files before deserializing them; Limitations and Discussion discusses this protection and its limits.

On lookup, `LazyLoader` binds each variable to a *stub* rather than immediately loading its value. A stub is a small placeholder object that records where the value's bytes live and how to deserialize them. All stubs share one interface with a single method, `load()`, and one stub type exists per storage format: pickle, NumPy `.npy`, Apache Arrow, PyTorch `.pt`, and binary media. The first time the notebook uses the variable, its stub loads the value. A variable that is never used is never loaded.

By contrast, the loader, `PickleLoader`, loads values during lookup. It writes the full `Cache` envelope, holding every variable the cell defined, as a single blob using pickle (Python's built-in serializer). When it finds a matching entry, it immediately loads every variable back into memory.

4.1. WASM portability

Separating stored values into a manifest and individual blobs also lets marimo bundle them with a static notebook. marimo's static WebAssembly (WASM) export is a standalone HTML (Hypertext Markup Language) file that runs the notebook in the reader's browser, with no server, on Pyodide, a Python runtime compiled to WebAssembly. When automatic cell caching is enabled, exporting a notebook through marimo's command-line interface (`marimo export html-wasm --execute`) bundles the cache manifests and blobs into that file. When a reader opens the export, the browser session derives the same keys and loads matching cached values on first use. A missing or unverifiable entry is treated as a cache miss. Scientific articles (including this one), blog posts, and educational materials can therefore include precomputed results and trained models with the notebook.

The export includes the cached values and the user code, but not the external libraries used to produce those values. A cell may therefore depend on a package the browser cannot import, as long as the values it defines are stored in a portable format. Our demonstration, published at https://dmadisetti.github.io/scipy_proceedings/, exercises this case. On the exporting machine, the notebook trains a PyTorch model, converts it to ONNX (Open Neural Network Exchange) bytes, defines a small wrapper class, and stores a sweep of predictions as NumPy arrays. In the browser, where Pyodide cannot currently import PyTorch, the arrays and the wrapper load through their format stubs. A custom stub feeds the ONNX bytes to `onnxruntime-web`, an ONNX runtime for the browser, restoring a model the notebook can call.

5. Using marimo's cache

marimo exposes caching with in-memory caching within a session, persistent caching across sessions, and automatic caching of every cell. All three use the key construction described in Cache Keys, so users do not need to declare dependencies or construct keys. Changes to user interface elements and values created with marimo's state primitive, `mo.state`, are included in this construction.

5.1. In-memory caching

The decorator `@mo.cache` memoizes a function in memory. The key for each call combines the function's code, its arguments, and variables it uses from outside its body, following the construction in Section 1. Results stay in the kernel's memory, so a hit reads nothing from disk, but all results are lost when the session ends.

In comparison to Python's built-in **`functools.cache`**, marimo's `@mo.cache` is better suited to reactive notebooks. Since a reactive runtime re-runs the cell that defines the function, it creates a new function with an empty `functools` cache. This discards every stored result, even when the change that caused the cell to run did not affect the function's behavior.

5.2. Persistent caching

The decorator `@mo.persistent_cache` uses the same key construction but writes results to disk through the loaders described in Storage and Loading, allowing the results to survive kernel restarts. This lets a notebook restart without re-running its expensive cells. These files are also bundled with the WebAssembly export described in Section 3.

Additionally, `mo.persistent_cache` can be used as a context manager. A block of code under `with mo.persistent_cache(name="training")` is cached, and on a hit, marimo restores the block's variables without executing the block. Optional arguments select the storage direc-

tory (`save_path`) and loader (`method="pickle"` or `method="lazy"`). The `pin_modules` option adds selected library versions to the key.

5.3. Automatic caching of every cell

The decorators and context manager cache only the code the author marks. The `cache_cells` runtime option instead applies caching to every cell. Before running a cell, the runtime computes its key and checks the cache. On a cache hit, the runtime skips the cell's body and restores its variables; on a miss, it runs the cell and saves the results. Since the cache attempts to account for all variables, stored entries may sometimes contain only a placeholder for variables that cannot be serialized. When a later cell needs that variable, marimo re-runs its defining cell and any upstream cells needed to rebuild it. With `cache_cells` enabled, the author does not need to annotate individual functions, blocks, or cells.

6. Evaluation

Caching does not always save time. A hit pays the cost of deriving a key and loading a value; a miss derives a key and saves a value after running the cell body. When portability or provenance is the goal, this overhead may be worthwhile even if caching is not faster. When the goal is to skip recomputation, the work avoided must repay the overhead.

We measure key derivation, value load, and value save, both separately and as end-to-end hit and miss paths. The payloads are NumPy `float64` arrays ranging from 1 MB to 1 GB. We compare six strategies: `mo.cache`, `mo.persistent_cache` with each of its two loaders, mandala's decorated-function memoization, and diskcache as both a memoizing decorator and a plain store with a fixed key. The fixed-key form performs no key derivation and provides a lower bound on cache-hit time. We measure each cost over 10 runs after priming each cache and one warmup, and report the median. Persisted measurements are keyed on the operating system, architecture, Python implementation and version, and a methodology-version string. A change to any of these properties causes the benchmark to recompute the measurement.

6.1. End-to-end cache evaluation

A notebook user experiences a cache hit as the time needed to derive the key and restore the stored value. Panel (a) of Figure 3 reports this delay for the six strategies. On the build host, every strategy stays below 100 ms for payloads through 50 MB. At 100 MB, both persistent marimo variants remain below this threshold while mandala exceeds it. The dashed line marks the 100 ms threshold below which a response reads as instantaneous [27].

The dotted curve in panel (a) shows how expensive the cell body must be before caching saves time. Let T_{hit} be the cost of serving a hit, and let $T_{\text{key}} + T_{\text{save}}$ be the overhead a miss adds to running the cell body. If a fraction p of lookups are hits, caching saves time when the cell body costs more than $T_{\text{hit}} + \frac{1-p}{p}\left(T_{\text{key}} + T_{\text{save}}\right)$. For these payloads, the measured miss overhead is comparable to the hit cost, so at a hit rate of $p = 0.9$, the break-even body cost is roughly 10% above the hit curve.

Panel (b) decomposes the largest-payload hit into key derivation and value load, alongside the key derivation and value save that a miss adds. This decomposition explains the separation between implementations at larger payload sizes. mandala derives its key with `joblib.hash`, which serializes the value through pickle before hashing it [22]. marimo instead hashes the array's contiguous bytes directly through Python's buffer protocol, avoiding the serialization step. On the Apple M4 Max used for the camera-ready figure, mandala is roughly three times slower end to end at the largest measured payload. However, on a Linux x86-64 server, value loading dominates and the ratio shrinks to roughly 1.2 times. The

difference therefore appears to be hardware-dependent. marimo's value-load time closely matches the fixed-key diskcache control, indicating that the gap to mandala comes from key derivation rather than storage.

Panel (c) reports the variation across individual cache hits at the largest payload size. `diskcache.memoize` does not appear at this size because the payload exceeds the blob-size limit of its underlying SQLite database.

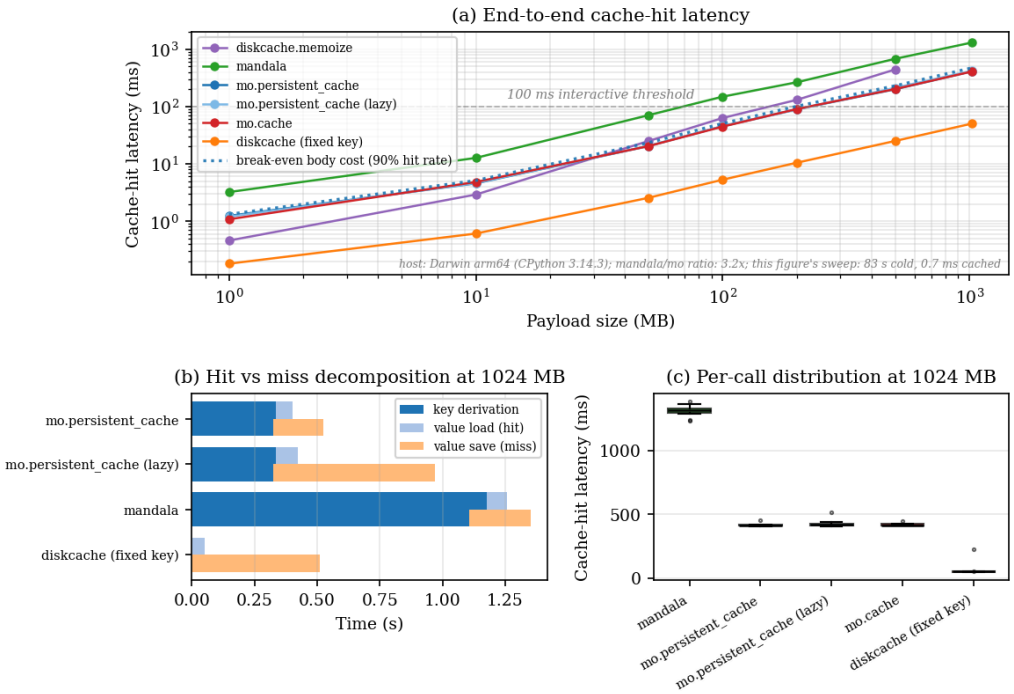


Figure 3. *End-to-end cache evaluation on NumPy `float64` payloads. (a) Cache-hit latency versus payload size on logarithmic axes. The dashed line at 100 ms marks the interactive threshold [27], and the dotted curve gives the break-even body cost at a 90% hit rate. (b) The cost of a hit (key derivation and value load) and the overhead added by a miss (key derivation and value save) at the largest payload size. (c) The distribution of cache-hit measurements at the largest payload size. `diskcache.memoize` does not appear at this size because the payload exceeds the blob-size limit of its underlying SQLite database. The host label reports the build host, the measured mandala-to-marimo ratio, and the cold and cached costs of the figure's own benchmark sweep.*

The camera-ready figure was produced on a MacBook Pro with an Apple M4 Max. Because the relative costs of hashing, serialization, and loading depend on the hardware, the figure reports its build host and measured mandala-to-marimo ratio; a build on different hardware reports its own result. The stage decomposition measures the actual disk-backed loading paths for both marimo loaders and for mandala, including the operating system's file cache and the cost of reconstructing the cache envelope. An in-memory deserialization benchmark would omit both costs.

7. Limitations and Discussion

The current implementation has five limitations.

Library versions. The decorator and context-manager APIs do not include library versions in the key by default; the user can include selected versions with `pin_modules` (Using marimo's cache). Without module pinning, upgrading a package can therefore produce a

stale cache hit. Automatic cell caching includes library versions by default, so a version mismatch produces a cache miss instead.

Python versions. Because code is hashed as bytecode, keys may change across Python versions (Section 1). Upgrading the interpreter may therefore invalidate cached results. When it does, the failure is safe: the keys miss and the cells recompute. It is also why the browser export requires the exporting interpreter to be compatible with the browser's Python version (Section 3).

Mutation outside the graph. Mutations that bypass the dataflow graph can change a value without changing any cache key. Examples include aliased mutation through a closure and attribute writes on an object that cannot be content-addressed. Downstream cells can then receive a stale cache hit that marimo cannot detect. A suite of checks that runs on every build of this paper exercises this case, and Rex [28] probes the same boundary.

Side effects. External state does not enter a cache key unless the notebook represents it as a dependency. File reads, network calls, randomness, and wall-clock queries can therefore change a cell's result without changing its key. marimo can represent two such dependencies explicitly: `mo.watch.file` includes a file's contents in the key, and `mo.watch.directory` includes a directory listing. Other sources of external state and nondeterminism are not tracked automatically.

Cache integrity and trust. Unpickling can execute arbitrary code, so loading a cache that an attacker has modified could run a malicious payload. When cryptographic signing support is available, the lazy loader signs caches by default. The signature covers the manifest and the SHA-256 hash of every stored blob, and marimo verifies them before deserializing any value. The pickle loader has no such protection and should load caches only from trusted sources. Signatures show that a cache was produced by a trusted key and has not been modified; they do not establish which keys a reader should trust or whether the signed result is correct. Sharing caches therefore requires a way to establish trust in the signing key.

8. Future work

The evaluation and limitations suggest four directions for future work.

Performance. Section 7 measures the costs of cache hits and misses. marimo could measure these costs while a notebook runs and apply the break-even rule automatically, declining to cache a cell when the cache costs more than the computation it would avoid. The same measurements could report how much execution time each cached cell saved. A fast precheck, potentially backed by a Bloom filter, could reject some cache misses before constructing the full key or querying remote storage [29].

Additional external dependencies. The explicit-dependency mechanism used for files and directories could be extended to sources such as randomness, network responses, and time. The open design question is which sources justify dedicated interfaces and how each source should contribute a stable value to the key.

Persistent-cache management. The persistent loaders do not impose a disk-capacity or eviction policy. Future work could add size limits, cleanup policies, and storage accounting that reflects the different formats used for cached values.

Provenance-aware reuse. Persistent caches already reuse results across sessions. They could also record and expose which computation produced each reused value, connecting cache reuse with systems for tracking computational provenance [30].

9. Conclusion

We have presented a caching mechanism for marimo, a reactive notebook for Python. A cell's cache key combines a hash of its bytecode with hashes derived from its references; when a reference cannot be hashed by content, the key of the cell that produced it stands in. This construction invalidates cached results when their code or tracked inputs change, while preserving them across formatting and comment changes. On the Apple M4 Max used to build this paper, both persistent marimo loaders served 100 MB arrays in under 100 ms, and mandala took roughly three times as long as marimo's persistent cache at the largest measured payload. At the same payload on Linux x86-64, mandala took roughly 1.2 times as long. marimo can apply the mechanism automatically to every cell without per-cell annotations and include the resulting caches in its static WebAssembly export. The export published at https://dmadisetti.github.io/scipy_proceedings/ demonstrates how readers can use precomputed results and models in a browser-only Python runtime.

9.1. Acknowledgments

Portions of this work were assisted by a generative AI tool (Claude, Anthropic). Claude was used to help develop and run the benchmark harness reported in the Evaluation. All benchmark code and results were reviewed, verified, and revised by the authors, who take full responsibility for the accuracy and integrity of the final content.

References

[1] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks," in *Proceedings of the 16th International Conference on Mining Software Repositories (MSR '19)*, 2019, pp. 507–517. doi: 10.1109/MSR.2019.00077.

[2] F. van der Plas and Pluto.jl contributors, "Pluto.jl: Simple Reactive Notebooks for Julia." [Online]. Available: https://github.com/fonsp/Pluto.jl

[3] M. Bostock, "A Better Way to Code." [Online]. Available: https://medium.com/@mbostock/a-better-way-to-code-2b1d2876a3a0

[4] J. Valim and Livebook Team, "Livebook: Interactive and Collaborative Code Notebooks for Elixir." [Online]. Available: https://livebook.dev/

[5] B. Victor, "Inventing on Principle." [Online]. Available: https://worrydream.com/InventingOnPrinciple/

[6] A. Agrawal and M. Scolnick, "marimo: An Open-Source Reactive Notebook for Python." [Online]. Available: https://marimo.io/

[7] Z. Li, S. Chockchowwat, R. Sahu, A. Sheth, and Y. Park, "Kishu: Time-Traveling for Computational Notebooks," *Proceedings of the VLDB Endowment*, vol. 18, no. 4, pp. 970–985, 2025, doi: 10.14778/3717755.3717759.

[8] D. Michie, "``Memo'' Functions and Machine Learning," *Nature*, vol. 218, no. 5136, pp. 19–22, 1968, doi: 10.1038/218019a0.

[9] D. Koop and J. Patel, "Dataflow Notebooks: Encoding and Tracking Dependencies of Cells," in *9th USENIX Workshop on the Theory and Practice of Provenance (TaPP 2017)*, Seattle, WA, 2017. [Online]. Available: https://www.usenix.org/conference/tapp17/workshop-program/presentation/koop

[10] S. Macke, H. Gong, D. J.-L. Lee, A. Head, D. Xin, and A. Parameswaran, "Fine-Grained Lineage for Safer Notebook Interactions," *Proceedings of the VLDB Endowment*, vol. 14, no. 6, pp. 1093–1101, 2021, doi: 10.14778/3447689.3447712.

[11] S. Macke, "IPyflow: A Next-Generation, Dataflow-Aware IPython Kernel." [Online]. Available: https://github.com/ipyflow/ipyflow

[12] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build Systems à la Carte," *Proceedings of the ACM on Programming Languages*, vol. 2, pp. 1–29, 2018, doi: 10.1145/3236774.

[13] E. Dolstra, M. de Jonge, and E. Visser, "Nix: A Safe and Policy-Free System for Software Deployment," in *Proceedings of the 18th USENIX Conference on System Administration (LISA '04)*, Atlanta, GA, 2004, pp. 79–92. [Online]. Available: https://www.usenix.org/legacy/event/lisa04/tech/dolstra.html

[14] E. Dolstra, "The Purely Functional Software Deployment Model." [Online]. Available: https://edolstra.github.io/pubs/phd-thesis.pdf

[15] C. Greco and J. Tagliabue, "Reproducible Data Science over Data Lakes: Replayable Data Pipelines with Bauplan and Nessie," in *Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning (DEEM@SIGMOD '24)*, 2024. doi: 10.1145/3650203.3663336.

[16] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, "Nextflow enables reproducible computational workflows," *Nature Biotechnology*, vol. 35, no. 4, pp. 316–319, 2017, doi: 10.1038/nbt.3820.

[17] F. Mölder *et al.*, "Sustainable data analysis with Snakemake," *F1000Research*, vol. 10, p. 33, 2021, doi: 10.12688/f1000research.29032.2.

[18] P. J. Guo and D. Engler, "Using Automatic Persistent Memoization to Facilitate Data Analysis Scripting," in *Proceedings of the 2011 International Symposium on Software Testing and Analysis (ISSTA '11)*, Toronto, ON, Canada, 2011, pp. 287–297. doi: 10.1145/2001420.2001455.

[19] Y. Xie, *Dynamic Documents with R and knitr*, 2nd ed. Boca Raton, FL: Chapman & Hall/CRC, 2015. doi: 10.1201/b15166.

[20] Executable Books Project, "jupyter-cache: A defined interface for working with a cache of executed Jupyter notebooks." [Online]. Available: https://github.com/executablebooks/jupyter-cache

[21] Streamlit Team, "Introducing Two New Caching Commands to Replace st.cache." [Online]. Available: https://blog.streamlit.io/introducing-two-new-caching-commands-to-replace-st-cache/

[22] A. Makelov, "Mandala: Compositional Memoization for Simple & Powerful Scientific Data Management," in *Proceedings of the 23rd Python in Science Conference (SciPy 2024)*, 2024. doi: 10.25080/JHPV7385.

[23] G. Jenks, "DiskCache: Disk and File Backed Cache for Python." [Online]. Available: https://github.com/grantjenks/python-diskcache

[24] Z. Li, P. Gor, R. Prabhu, H. Yu, Y. Mao, and Y. Park, "ElasticNotebook: Enabling Live Migration for Computational Notebooks," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 119–133, 2024, doi: 10.14778/3626292.3626296.

[25] Joblib Developers, "Joblib: Running Python Functions as Pipeline Jobs." [Online]. Available: https://joblib.readthedocs.io/

[26] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology — CRYPTO '87*, in Lecture Notes in Computer Science, vol. 293. 1988, pp. 369–378. doi: 10.1007/3-540-48184-2_32.

[27] S. K. Card, G. G. Robertson, and J. D. Mackinlay, "The Information Visualizer, an Information Workspace," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '91)*, New Orleans, LA, USA, 1991, pp. 181–186. doi: 10.1145/108844.108874.

[28] M. Zheng, W. Crichton, A. Narayan, D. Raghavan, and N. Vasilakis, "When Are Reactive Notebooks Not Reactive?." [Online]. Available: https://arxiv.org/abs/2511.21994

[29] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970, doi: 10.1145/362686.362692.

[30] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "noWorkflow: A Tool for Collecting, Analyzing, and Managing Provenance from Python Scripts," *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1841–1844, 2017, doi: 10.14778/3137765.3137789.

Open Access | https://doi.org/10.25080/gewj5574

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

ngff-zarr: A Lean and Kind OME-Zarr Toolkit for Bioimaging

Matt McCormick[1], **Johannes Soltwedel**[2], **Wouter-Michiel Vierdag**[3,4], **Valentin Boussot**[1], and **Josh Moore**[2]

[1]Fideus Labs LLC, [2]German BioImaging — Society for Microscopy and Image Analysis e.V., [3]Cellonautica Inc, [4]scverse

Abstract

Modern bioimaging instruments produce datasets that are large, multidimensional, and stored in vendor-specific proprietary formats. These monolithic files are not cloud-ready, are difficult to stream or share, and hinder reproducible, collaborative science. The community has responded with OME-Zarr (OME-NGFF), an open, chunked, cloud-native standard built on Zarr's compressed, n-dimensional array storage. We present `ngff-zarr`, a lean, minimal-dependency Python toolkit that implements the OME-Zarr specification and makes it accessible through a simple, four-step pipeline interface. `ngff-zarr` converts, validates, and generates multiscale representations of extremely large images out-of-core via Dask, accepting any input that follows the Python Array API Standard (NumPy, Dask, CuPy, PyTorch). It provides multiple downscaling methods, including SIMD-accelerated Gaussian filtering via ITK-Wasm; OME-Zarr Zip (`.ozx`) single-file archives (RFC-9); RFC-4 anatomical orientation metadata for medical and neuroimaging interoperability; emerging RFC-5 coordinate systems and transformations with OME-Zarr version 0.6; high-content screening plate and well support; TIFF, OME-TIFF, and Leica LIF conversion; and Zarr v3 sharding. We further describe the `ngff-zarr-mcp` Model Context Protocol (MCP) server, which exposes conversion, inspection, validation, and optimization tools to AI agents so that researchers can perform bioimaging tasks in natural language. We close by discussing lessons learned, planned support for additional community RFCs, and the path toward OME-Zarr 1.0.

Keywords OME-Zarr, OME-NGFF, bioimaging, cloud-native, Dask, Model Context Protocol

Published Jul 13, 2026

Correspondence to
Matt McCormick
matt@fideus.io

Open Access

Copyright © 2026 McCormick *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Modern bioimaging instruments, including light-sheet microscopes, high-content screening systems, whole-slide scanners, and electron or volume-electron microscopes, routinely produce datasets that are large, multidimensional, and stored in fragmented, vendor-specific proprietary formats. A single acquisition can exceed the memory of any workstation, and the resulting monolithic files are not cloud-ready: they are difficult to stream, to share, and to align with FAIR (Findable, Accessible, Interoperable, Reusable) principles. This friction directly hinders reproducible, collaborative science.

The bioimaging community has converged on an answer. OME-Zarr — created by the OME-NGFF, Open Microscopy Environment Next-Generation File Format, community — is a community-driven, open standard for storing bioimaging data in the cloud [1], [2]. Built on Zarr's chunked, compressed, n-dimensional array storage [3], OME-Zarr stores images as multiscale pyramids together with rich, machine-readable metadata. Its specification and ecosystem are developed openly through a Request for Comments (RFC) process and periodic community hackathons [4]. Adoption now spans diverse modalities and institutions worldwide.

A specification, however, only delivers value when backed by robust, accessible tooling. This is the gap that motivates `ngff-zarr` [5], a lean, minimal-dependency Python implementation of OME-Zarr that is lazy, parallel, and web-ready. `ngff-zarr` is designed around two goals captured in its tagline, *lean and kind*: lean in that it adds little beyond the scientific Python stack it builds on, and kind in that its interface maps to how researchers actually think about their data, including a recently added natural-language interface for AI agents.

This paper makes three contributions. First, we describe the design and implementation of `ngff-zarr`: an out-of-core, Array-API-based pipeline that turns in-memory arrays into validated, multiscale OME-Zarr stores. Second, we catalog the toolkit's features and how they serve concrete bioimaging needs, from anatomical orientation for medical imaging to high-content screening and the emerging RFC-5 coordinate-transformation support targeting OME-Zarr 0.6. Third, we present the `ngff-zarr-mcp` Model Context Protocol server and report lessons learned from exposing scientific tooling to AI agents and from working within the OME-Zarr community.

1.1. Background: OME-Zarr and the community

Rather than a single monolithic file, an OME-Zarr dataset is a hierarchy of chunked, compressed arrays accompanied by JSON metadata [6]. Three properties make it well suited to modern data. It is **chunked and compressed**, so a client can read a small region of a large image without downloading the whole dataset — the foundation of cloud-optimized access. It is **multiscale**, storing each image as a singular scale or as a pyramid of progressively downsampled resolutions for responsive visualization and scale-appropriate analysis. And it is **self-describing**: axis names, types, units, and coordinate transformations are stored alongside the data in a standardized model shared across the community's many implementations and programming languages.

The format's design and the community process behind it are documented in J. Moore *et al.* [1] and J. Moore *et al.* [2], and the ecosystem continues to evolve through open RFCs and collaborative events such as the 2024 OME-NGFF workflows hackathon [4]. `ngff-zarr` is developed within and for this community, tracking the specification as it advances from version 0.1 through 0.5, and now toward 0.6, alongside the Zarr Format Specification 3.

2. Methods

`ngff-zarr` is implemented in Python (requiring Python 3.10 or newer) with a deliberately small dependency footprint built on NumPy [7], Dask [8], Zarr [3], and ITK-Wasm [9]. Its interface provides an abstraction layer between front-end usage and the terms of the spec, to provide users with a stable interface and a backend that is able to evolve and forward metadata as the specification and its terms evolve. This backend uses Python `dataclasses` to represent the specification model directly which is accessible to users for inspection, along with convenient access to the pixel data as lazy Dask arrays. The core abstraction is a four-step pipeline: an in-memory array becomes an `NgffImage`, which becomes a multiscale `NgffMultiscales`, which is written to an OME-Zarr store.

```python
import ngff_zarr as nz

image = nz.to_ngff_image(
    array,
    dims=["z", "y", "x"],
    scale={"z": 2.0, "y": 0.5, "x": 0.5},
)
multiscales = nz.to_multiscales(image, scale_factors=[2, 4], chunks=64)
nz.to_ngff_zarr("output.ome.zarr", multiscales)
```

2.1. *Array to `NgffImage`*

The pipeline accepts any array-like object that follows the Python Array API Standard [10], including NumPy `ndarray`s, Dask arrays, Zarr arrays, CuPy arrays, and PyTorch tensors. The `to_ngff_image` function wraps the array in an `NgffImage`, a plain `dataclass` representing a single scale level. When constructing the image, the user may specify the dimension names `dims` drawn from `{'t', 'z', 'y', 'x', 'c'}`, the physical pixel `scale` for the spatial dimensions, a `translation` giving the origin of the first pixel, a `name`, and `axes_units` expressed with UDUNITS-2 (standard for physical units) identifiers. Sensible NumPy-compatible defaults are used for anything left unspecified. Internally the data is held lazily as a chunked Dask array, so no pixels are loaded or computed until they are needed.

2.2. *`NgffImage` to `NgffMultiscales`*

`to_multiscales` builds a Dask task graph that will produce a chunked, multiscale image pyramid. It accepts optional `scale_factors` and `chunks` parameters and an antialiasing `method`. The returned `NgffMultiscales` dataclass holds the image for each scale together with the OME-Zarr metadata — axes, datasets, and coordinate transformations — and the correct `scale` and `translation` for each downsampled level are computed automatically. Because the result is still a lazy task graph, building the multiscales is inexpensive; the work happens only when the store is written.

2.3. *Downscaling methods*

Generating a faithful multiscale pyramid requires care: to avoid aliasing artifacts, the input must be smoothed before each downsampling step. `ngff-zarr` exposes a family of downscaling methods through the `ngff_zarr.Methods` enumeration, summarized in Table 1. They differ primarily in their smoothing strategy and in their suitability for intensity versus label images, trading off artifact level, speed, and hardware portability.

Table 1. *Downscaling methods available via `ngff_zarr.Methods`.*

Method	Backend	Image type
`ITKWASM_GAUSSIAN` (default)	ITK-Wasm	Intensity
`ITKWASM_BIN_SHRINK`	ITK-Wasm	Intensity
`ITKWASM_LABEL_IMAGE`	ITK-Wasm	Label
`ITK_GAUSSIAN`	Native ITK	Intensity
`ITK_BIN_SHRINK`	Native ITK	Intensity
`DASK_IMAGE_GAUSSIAN`	dask-image / SciPy	Intensity
`DASK_IMAGE_MODE`	dask-image / SciPy	Label
`DASK_IMAGE_NEAREST`	dask-image / SciPy	Label

The default method, `ITKWASM_GAUSSIAN`, smooths with a discrete Gaussian filter to build a proper scale space, ideal for intensity images. It is implemented with ITK-Wasm [9], so it

is SIMD-accelerated and extremely portable across platforms, including the browser. The availability of label-aware methods is important: naively downsampling a segmentation with a smoothing filter corrupts label identities, so dedicated mode-based reductions (`ITKWASM_LABEL_IMAGE`, `DASK_IMAGE_MODE`) are provided. Native ITK variants [11] and SciPy-backed `dask-image` fallbacks round out the set for environments with different dependency or hardware constraints.

2.4. `NgffMultiscales` to OME-Zarr store, reading, and validation

`to_ngff_zarr` computes the pyramid and writes it to a Zarr store. Because every stage of the pipeline is expressed as a Dask task graph, the entire computation executes lazily and out-of-core: chunks are streamed through the graph and written incrementally rather than materialized all at once. This is what allows `ngff-zarr` to process datasets that exceed available memory. By convention a local directory store uses the `.ome.zarr` extension, but any Zarr store type may be used — including remote object stores on S3, Google Cloud Storage, or Azure via `fsspec`, with no local filesystem required. OME-Zarr version 0.4 (Zarr Format Specification 2) and 0.5 (Zarr Format Specification 3) are supported on write, and versions 0.1 through 0.5 can be read.

Reading mirrors writing. `from_ngff_zarr` returns an `NgffMultiscales` populated with lazy Dask arrays. With the optional `validate` dependency installed, passing `validate=True` checks that the store's metadata conforms to the shared OME-Zarr JSON schema model, raising an error on any deviation; validation is supported for versions 0.1 through 0.5.

```
multiscales = nz.from_ngff_zarr("cthead1.ome.zarr", validate=True)
```

3. Results

The result of this design is a small toolkit that nonetheless covers a broad range of real bioimaging workflows. We describe its principal features and the needs they address.

3.1. Out-of-core conversion of extremely large data

The central capability is converting arbitrarily large images into validated, multiscale OME-Zarr without exhausting memory. Because the pipeline is built on Dask task graphs, a multi-hundred-gigabyte volume can be converted on a laptop: chunks flow through smoothing, downsampling, and writing stages incrementally. The command-line interface (CLI) makes this explicit through a `--memory-target` option that accepts a human-readable limit and schedules the computation to respect it.

```
ngff-zarr --memory-target 50M -i LIDCFull.vtk -o LIDCFull.ome.zarr
```

3.2. OME-Zarr Zip (`.ozx`) single-file archives

A directory-based Zarr store can comprise thousands of small files, which is awkward to copy, share, or archive. RFC-9 [12] introduces the OME-Zarr Zip format, which packages an entire OME-Zarr hierarchy into a single ZIP archive with the `.ozx` extension. `ngff-zarr` reads and writes `.ozx` transparently — the extension is detected automatically — and embeds the OME-Zarr version in the ZIP comment for reliable detection on read. For large datasets, `write_store_to_zip` copies an existing store directly into a `.ozx` archive without recomputing arrays.

```
nz.to_ngff_zarr("cthead1.ozx", multiscales, version="0.5")
multiscales = nz.from_ngff_zarr("cthead1.ozx")
```

3.3. RFC-4 anatomical orientation

Medical and neuroimaging analysis depends on knowing the anatomical direction of each spatial axis so that images can be aligned to atlases and to one another. RFC-4 [13] adds anatomical orientation metadata to OME-NGFF axes. `ngff-zarr` emits this metadata when RFC-4 is enabled, either programmatically via `enabled_rfcs=[4]` in `to_ngff_zarr` or with the `--enable-rfc 4` flag on the command line. When converting ITK or ITK-Wasm images — for example from NRRD, NIfTI, or DICOM inputs — anatomical orientation is derived automatically from ITK's LPS (Left-Posterior-Superior) coordinate system [11], with convenience constants provided for both the LPS and the neuroimaging RAS conventions.

```
nz.to_ngff_zarr("output.ome.zarr", multiscales, enabled_rfcs=[4])
```

3.4. RFC-5 coordinate systems and OME-Zarr 0.6

The most significant recent addition is emerging support for RFC-5, which provides first-class coordinate systems and transformations in OME-Zarr and is the centerpiece of version 0.6 [14]. RFC-5 introduces named coordinate systems (sets of axes) and a richer vocabulary of transformations — including identity, axis permutation and projection, translation, scale, affine, rotation, sequences of transformations, and field-based displacement and coordinate transforms — that map points between coordinate systems. This enables datasets to express the spatial relationships between multiple images, such as aligned tiles or registered modalities, in a standardized, machine-readable way without resampling and re-saving pixel data.

`ngff-zarr` implements RFC-5 toward OME-Zarr 0.6, including coordinate-system handling and transformation sequences during multiscale generation, an updated 0.6 JSON schema set, and round-trip conversion between versions 0.5 and 0.6. RFC-5 is designed to work hand in hand with RFC-4: where anatomical axes are not aligned with imaging axes, RFC-5 transformations describe the relationship between image space and anatomical space while RFC-4 supplies the biological orientation labels.

3.5. High-Content Screening

High-content screening (HCS) produces multi-well plate data central to drug discovery and high-throughput imaging. `ngff-zarr` implements the OME-Zarr plate and well metadata structures, so plates can be loaded, navigated by row and column, and examined field by field, with optional acquisition (time point or condition) selection and HCS-aware validation.

```
plate = nz.from_hcs_zarr("screening_plate.ome.zarr")
well = plate.get_well("A", "1")   # row A, column 1
image = well.get_image(0)         # first field of view
print(image.images[0].data.shape)
```

3.6. Format conversion, sharding, and the CLI

Through optional dependencies, the toolkit converts a broad range of scientific image formats to OME-Zarr: any format readable by ITK, `tifffile`, or `imageio`, including TIFF and multi-series OME-TIFF with automatic extraction of physical pixel sizes and units, as well as Leica LIF microscopy files. For very large outputs, Zarr v3 sharding stores multiple

compressed chunks in a single file or blob, reducing file counts; sharding is requested with the `chunks_per_shard` argument and requires OME-Zarr version 0.5 or newer. An optional Tensorstore backend can further improve write performance. For batch and scripting workflows, the CLI mirrors the library — a basic conversion is a single command, and omitting the output prints information about the input and the multiscales that would be generated.

```
ngff-zarr -i input.nrrd -o output.ome.zarr
```

3.7. MCP server for AI agents

The `ngff-zarr-mcp` package exposes the toolkit's capabilities to AI agents through the Model Context Protocol (MCP) [15], an open standard that lets AI models securely call external tools. Once configured, agents such as GitHub Copilot, Claude Code, Cursor, and OpenCode can drive `ngff-zarr` through natural language. The server provides a small, deliberate set of functions that map to researcher intent: `convert_images_to_ome_zarr` converts datasets with configurable parameters; `get_ome_zarr_info` reports detailed information about a store; `validate_ome_zarr_store` checks structure and metadata against the specification; and `optimize_ome_zarr_store` re-encodes an existing store with new compression or chunking. In practice a researcher might ask an assistant to *"convert `LIDCFull.tif` to OME-Zarr"*, *"find the optimal codec for this data"*, *"use sharding to keep the number of files under 20"*, or *"write a Python script to convert every file in this directory using that codec"*; the agent composes these tool calls and the server executes them.

4. Discussion

`ngff-zarr` demonstrates that a faithful, full-featured OME-Zarr implementation need not be heavy. By expressing the entire conversion pipeline as lazy Dask task graphs and by accepting any Array-API-compatible input, the toolkit keeps its dependency surface small while supporting workflows from a NumPy array in a notebook to a multi-terabyte light-sheet volume on cloud storage. The same design that enables out-of-core processing also makes the library portable: the default ITK-Wasm downscaling backend runs identically on a workstation, in CI, and in the browser.

4.1. Lessons learned from the MCP server

Designing agent-facing tools surfaced several lessons. First, **structured tool parameters matter**: clearly typed, well-described arguments make agent invocations far more reliable than free-form strings. Second, **functions should map to researcher intent, not to low-level API calls**: a single `optimize_ome_zarr_store` tool that an agent can reason about is more useful than exposing the full surface of internal functions. Third, **natural-language interfaces lower the barrier to adoption**: by letting scientists describe what they want rather than learn an API, the MCP server brings cloud-native formats and reproducible workflows to users who would otherwise be deterred by tooling.

4.2. Future work

Several directions are planned. On the specification side, we intend to extend support for additional community RFCs: **RFC-3** (support for additional dimensions) [16] for smoother interoperability with more image types, and **RFC-8** (collections) [17] for grouping related OME-Zarr datasets. We will continue maturing **RFC-5** coordinate-transformation support to directly support common use cases, and align releases with the community's **OME-Zarr 1.0** milestone — a stable, long-term-supported version of the format. Alongside these

features, we plan ongoing **performance improvements**, including faster downscaling and writing, better memory-aware scheduling, and expanded GPU acceleration. As an open-source project, `ngff-zarr` welcomes community contributions toward these goals.

5. Conclusion

`ngff-zarr` is a lean, minimal-dependency, and community-aligned implementation of the OME-Zarr specification. Its four-step pipeline — array to `NgffImage` to `NgffMultiscales` to store — gives researchers a simple, lazy, parallel, and web-ready path from in-memory data to a cloud-native bioimaging dataset, while out-of-core execution via Dask makes datasets larger than memory routine. Beyond the core pipeline, support for `.ozx` single-file archives, RFC-4 anatomical orientation, emerging RFC-5 coordinate transformations with OME-Zarr 0.6, High Content Screening, broad format conversion, Zarr v3 sharding, and a memory-aware command-line interface make the toolkit practical across medical imaging, microscopy, and high-throughput screening. The `ngff-zarr-mcp` server extends these capabilities to AI agents, demonstrating how natural-language interfaces can accelerate the adoption of FAIR, reproducible bioimaging workflows. `ngff-zarr` is open source under the MIT license, and contributions from the community are welcome.

5.1. Software availability

Source code is available at github.com/fideus-labs/ngff-zarr and documentation at ngff-zarr.readthedocs.io. The Python package is distributed on PyPI as `ngff-zarr` and the MCP server as `ngff-zarr-mcp`.

5.2. Acknowledgments

We thank the Open Microscopy Environment and the broader OME-Zarr community, including participants in the OME-NGFF workflows hackathons, for the open specification process and collaboration that make this work possible.

5.3. Disclosure of Gen AI use

Portions of this work were assisted using generative AI tools. The tools were used for drafting and refining text. All outputs were reviewed, verified, and revised by the authors, who take full responsibility for the accuracy and integrity of the final content.

References

[1] J. Moore *et al.*, "OME-NGFF: a next-generation file format for expanding bioimaging data-access strategies," *Nature Methods*, vol. 18, no. 12, pp. 1496–1498, 2021, doi: 10.1038/s41592-021-01326-w.

[2] J. Moore *et al.*, "OME-Zarr: a cloud-optimized bioimaging file format with international community support," *Histochemistry and Cell Biology*, vol. 160, no. 3, pp. 223–251, 2023, doi: 10.1007/s00418-023-02209-1.

[3] Zarr Developers, "zarr-python: An implementation of chunked, compressed, N-dimensional arrays for Python." [Online]. Available: https://zarr.dev/

[4] J. Lüthi and others, "2024 OME-NGFF workflows hackathon," *BioHackrXiv*, 2025, doi: 10.37044/osf.io/5uhwz_v2.

[5] M. McCormick and ngff-zarr contributors, "ngff-zarr: A lean and kind OME-Zarr implementation." [Online]. Available: https://github.com/fideus-labs/ngff-zarr

[6] Open Microscopy Environment Consortium, "Next-generation file format (NGFF) specifications for storing bioimaging data in the cloud." [Online]. Available: https://ngff.openmicroscopy.org/

[7] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[8] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," *Proceedings of the 14th Python in Science Conference*, pp. 130–136, 2015, doi: 10.25080/Majora-7b98e3ed-013.

[9] M. McCormick and P. Elliott, "ITK-Wasm," *Python in Science Conference*, 2024, doi: 10.25080/TCFJ5130.

[10] Consortium for Python Data API Standards, "Python array API standard." [Online]. Available: https://data-apis.org/array-api/latest/

[11] M. McCormick, X. Liu, J. Jomier, C. Marion, and L. Ibanez, "ITK: enabling reproducible research and open science," *Frontiers in Neuroinformatics*, vol. 8, p. 13, 2014, doi: 10.3389/fninf.2014.00013.

[12] Open Microscopy Environment Consortium, "RFC-9: OME-Zarr as a single file via ZIP." [Online]. Available: https://ngff.openmicroscopy.org/rfc/9/index.html

[13] Open Microscopy Environment Consortium, "RFC-4: Coordinate systems and transformations – anatomical orientation." [Online]. Available: https://ngff.openmicroscopy.org/rfc/4/index.html

[14] Open Microscopy Environment Consortium, "RFC-5: Coordinate systems and transformations." [Online]. Available: https://ngff.openmicroscopy.org/rfc/5/index.html

[15] Anthropic, "Model Context Protocol specification." [Online]. Available: https://modelcontextprotocol.io/

[16] Open Microscopy Environment Consortium, "RFC-3: Transitional bioformats2raw layout." [Online]. Available: https://ngff.openmicroscopy.org/rfc/3/index.html

[17] Open Microscopy Environment Consortium, "RFC-8: Collections." [Online]. Available: https://ngff.openmicroscopy.org/rfc/8/index.html

Open Access | https://doi.org/10.25080/wdyr6752

Derivations, Not Just Simulations

Teaching Applied Mathematics with Scientific Python

Michael Zargham[1]

[1]Dynamical Systems Group

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Scientific computing in Python is fluent in two of David Tall's three worlds of mathematical learning: NumPy and SciPy handle operational symbolism; matplotlib and reactive notebooks handle conceptual embodiment. The third world, axiomatic formalism — where coordinate transformations, optimality criteria, and invariances live as symbol-manipulation under formal rules — typically stays on paper. SymPy is the SciPy-ecosystem tool that brings it into the same execution environment, and once all three worlds are executable in one notebook the glass-box principle (Buchberger; Drijvers; Artigue) becomes practicable for working scientists.

The worked exemplar is Isaacs's 1951 homicidal-chauffeur differential game. *With this exemplar's parameter settings, naive evader heuristics are caught within 11.3 seconds; saddle-point optimal play survives to 12.1 seconds, but its evader path has a sharp mid-chase kink no physical intuition predicts.* The body derives the optimal feedback symbolically, integrates it backward from the capture circle, and renders the value function as a heat map. The mathematics is canonical; what we claim is the executable implementation: Python modules shipped alongside the article, exercised by an automated test suite.

Keywords mathematics education, symbolic computation, differential games, SymPy, marimo, homicidal chauffeur, glass-box pedagogy, optimal control

1. The problem: teaching advanced applied math to working scientists

Scientific-computing users reach for the SciPy stack when they need numerical methods. This paper argues that the same stack (extended with MyST and a reactive notebook) also operationalises a thirty-year-old theory of how people learn mathematics. We make the case with one worked exemplar: the value function of a 1951 differential game.

The audience we have in mind is the engineer or scientist who reaches for `scipy.integrate.solve_ivp` and `numpy.linalg.eig` reflexively but treats symbolic derivations as something other people do on paper. David Tall [1] named exactly this reader: a *user* of mathematics who needs the mathematical objects working before they need the proofs that justify them. The SciPy stack already serves two of the three worlds Tall describes: *operational-symbolic* procedures (solving, integrating, optimising) and *conceptual-embodiment* (plots, interactive sliders). But the *axiomatic-formal* world, where a symbol *is* the object and an equation is what you manipulate, usually lives in a textbook citation rather than an executable cell.

SymPy closes that gap. The central claim of this paper is that with SymPy in the notebook alongside NumPy/SciPy and matplotlib, the *glass-box principle* [2] (refined by Drijvers and Artigue [3], [4]) finally becomes practicable: the reader can follow the chain from a derived expression, to the function it lambdifies into, to the figure it renders — and verify every

Published Jul 13, 2026

Correspondence to
Michael Zargham
michael.zargham@gmail.com

Open Access

Copyright © 2026 Zargham. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

link, rather than trust one opaque step. SymPy is what unlocks the third world for scientific-computing practice. Proprietary computer-algebra systems (Maple, Mathematica) and other open-source ones (Maxima, Sage) have offered symbolic capabilities for decades; the novel pivot here is having *all three Tall worlds executable in the same open-source Python kernel and notebook*, alongside the NumPy/SciPy/matplotlib idiom the working reader already uses.

We pick the homicidal chauffeur (HC), Isaacs's 1951 differential game, as the worked exemplar because it has every component the argument needs: a clean derivation chain that benefits from symbolic manipulation, an analytical singular-surface structure that makes a striking visualisation, and small enough scope to fit in a 6,000-word notebook. The publication substrate itself (Jupyter, MyST, Curvenote [5], [6], [7]) is part of the demonstration; everything below this paragraph is executable.

Three worlds and three boxes names the two pedagogical frameworks this paper depends on. The case study: the homicidal chauffeur (lens, not subject) introduces the exemplar with the figure that poses the question the rest of the paper answers. Two threads run in parallel through what follows: an *implementation* (the executable derivation chain in `paper.ipynb` plus the modules and tests beside it) and an *argument* (the three-worlds executability category that the implementation grounds). Each load-bears on the other.

2. Three worlds and three boxes

Two pedagogical frameworks structure the rest of the paper. Both come from the mathematics-education research literature, both have roots in the late 1980s, and both translate cleanly into questions about scientific computing.

David Tall [1], [8] describes mathematical cognition as three "worlds" of meaning, each with its own characteristic objects and modes of reasoning. The **conceptual-embodied world** is where mathematics is sensed and pictured: a vector field you can feel the divergence of, a trajectory you can trace with your eye, a slider you can scrub to watch a parameter change behaviour. The **operational-symbolic world** is where mathematics is a procedure you also treat as an object — the process-object duality introduced by A. Sfard [9], elaborated by Tall as the *procept*: $f'(x)$ is both a differentiation procedure you execute on f and an object you can integrate, compose, or compare. The **axiomatic-formal world** is where mathematics is symbol manipulation under formally defined rules: f *is* an element of a function space, f' *is* the linear operator the differentiation rules give you, and a "proof" is a tree of formal rewrites. Each world is a legitimate way of doing mathematics; mature practitioners move between them constantly without naming the moves.

Bruno B. Buchberger [2] gave a complementary principle, articulated at a 1989 computer-algebra colloquium: when teaching a method, learners need a **white-box** phase where they execute every step by hand to understand how the method works, before they can responsibly use a **black-box** tool that wraps the method. M. K. Heid [10] had observed the same pattern empirically: when a computer algebra system (CAS) removes the symbolic-manipulation burden, students still need supervised practice with the structure underneath. Buchberger's principle is *spiral*: today's white-box is tomorrow's black-box, freeing tomorrow's white-box to be something more abstract. Differentiation and integration rules have been in their black-box phase since Heid's 1988 studies, which is what releases this paper's white-box phase to be about Hamiltonian construction and costate derivation rather than chain rules.

Drijvers and Artigue [3], [4] sharpened the Buchberger binary into a third mode: **glass-box**, where the tool is transparent in *derivation* even when the user did not perform every algebraic step by hand. A glass-box workflow lets the reader inspect each intermediate

object the tool produced, ask why each step is justified, and re-run any step with different inputs. The distinction between white-box and glass-box matters for this paper: we do not hand-execute every algebraic step (that would be white-box), but every step we ask SymPy to perform is *inspectable* (glass-box). The glass-box moment: one trajectory through every lens will earn the term with a worked example; the rest of the paper trades on it.

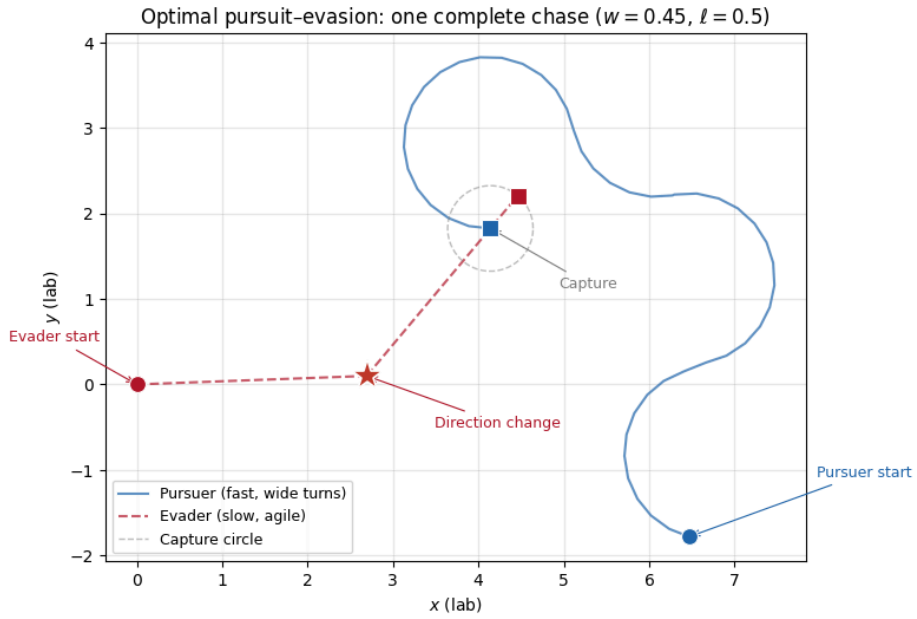
3. The case study: the homicidal chauffeur (lens, not subject)

The homicidal chauffeur (introduced by Rufus Isaacs in a 1951 RAND memorandum [11] and developed in his 1965 textbook [12]) is a two-player differential game. A pursuer moves at constant unit speed but turns under a bounded angular rate (the "chauffeur": fast forward, sluggish to turn); an evader moves at constant speed $w < 1$ (speeds are normalised to the pursuer's, so w is the evader-to-pursuer speed ratio) in any direction it chooses (the "pedestrian": slow, agile). The pursuer wants to capture the evader as quickly as possible by closing within a fixed capture radius ℓ; the evader wants to delay capture as long as possible. Merz [13] gave the complete singular-surface taxonomy (the map of where optimal strategies switch, split, or merge) that makes optimal play tractable; Pachter and Coates [14] is the most-cited modern restatement; Patsko, Botkin, and Kein [15] is the modern history and source for the live demos this paper draws on.

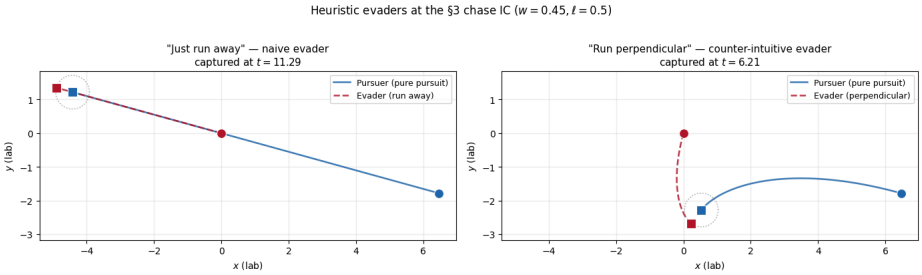
The first figure (`chase_demo`) shows one complete optimal chase. Read it from the ● **circles** (initial positions): the blue pursuer behind and to the right of the red evader at $t = 0$. They evolve forward in time along the solid blue and dashed red curves until the ■ **squares** at the moment of capture. The evader's path has a sharp ⋆, the *dispersal kink*, about halfway through. We use the same visual vocabulary in every figure (all rendered via matplotlib [16]): circles start, squares end, stars mark events; the convention is documented in `plots.py` and walked through inline as each figure appears.

That ⋆ is the question the rest of the paper answers. The second figure (`naive_vs_optimal`) compares two heuristic evaders against the same heuristic pursuer at the same initial condition and parameter settings ($w = 0.45$, $\ell = 0.5$): a "run straight away from the pursuer" evader (left, captured at 11.3 seconds) and a "run perpendicular to the closing direction" evader (right, captured at 6.2 seconds). The saddle-point evader plays differently: captured at 12.1 seconds (now against the optimal pursuer), with the ⋆ kink. The saddle-point theorem says no heuristic evader can do better than the optimal strategy against an optimal pursuer; the 12.1 seconds value is a *bound*, not a behavioural guess. **The question:** where does the ⋆ come from? Why does the evader's optimal path kink at that specific point? This section poses the puzzle; Sections ?? through ?? derive the answer.

```
# §3 motivation figure — lifted from hc-marimo (real chase, dispersal crossing)
from plots import chase_demo
chase_demo()
```




```
# §3 hook — two heuristic evaders at the chase_demo IC.
# See §3 prose for the saddle-point bound interpretation.
from plots import naive_vs_optimal
naive_vs_optimal()
```



4. A CRASH COURSE IN *DIFFERENTIAL GAMES* AND THE *CALCULUS OF VARIATIONS*

Calculus of variations and optimal control are load-bearing tools across applied mathematics: used routinely in aerospace, robotics, control engineering, mathematical biology, economics, and quantitative finance, though rarely taught outside narrow specializations at the undergraduate level. We chose a differential game rather than a single-player optimal-control problem because pursuit-evasion is essentially the children's game of *tag*: every reader has embodied experience of chasing and being chased, intuition to hang the formalism on.

Concretely, the HC state is naturally described in the lab frame with five degrees of freedom (DOF): pursuer position (x_P, y_P) and heading θ, evader position (x_E, y_E). The kinematics are

$$\begin{aligned} \dot{x}_P &= \cos\theta, \qquad \dot{y}_P = \sin\theta, \qquad \dot{\theta} = \phi, \\ \dot{x}_E &= w\cos\psi_{\text{lab}}, \qquad \dot{y}_E = w\sin\psi_{\text{lab}}, \end{aligned} \tag{1}$$

with $|\,\phi\,| \le 1$ the normalised turn-rate control ($v_P = 1$, minimum turning radius $R_{\min} = 1$) and ψ_{lab} the evader's lab-frame heading. Five DOF, two players, two controls. The formulation is well-posed but ungainly: each extra degree of freedom adds a variable that every symbolic derivation downstream must carry.

Two concepts from optimal control are load-bearing for the derivations that follow. We sketch them in one paragraph each.

A **differential game** is the optimal-control generalisation of a saddle-point problem to continuous time. Two players, the pursuer P and the evader E, choose control inputs u_P and u_E (for HC, the turn rate ϕ and the heading ψ_{lab} just defined) that jointly determine the state's evolution $\dot{\boldsymbol{x}} = f(\boldsymbol{x}, u_P, u_E)$. They have opposed objectives (one minimises a running cost $L(\boldsymbol{x}, u_P, u_E)$, the other maximises it), and we look for a *saddle point* (u_P^*, u_E^*) where neither player can improve unilaterally. Isaacs's 1951 contribution was to formulate HC this way and to show that saddle points exist for many adversarial pursuit scenarios; the existence question is the hard part, and the answer depends on the singular-surface structure §?? visits.

The **Pontryagin maximum principle** (PMP) is the calculus-of-variations machinery that locates the saddle point. It introduces a costate vector $\boldsymbol{p}(t)$ (the analogue of momentum in classical mechanics: it measures how sensitive the optimal cost is to each state component) and a Hamiltonian $H(\boldsymbol{x}, \boldsymbol{p}, u_P, u_E) = \boldsymbol{p} \cdot f + L$. The optimal controls minimise (resp. maximise) H over their feasible sets, and the state-costate pair evolves under Hamilton's equations $\dot{\boldsymbol{x}} = \partial H/\partial \boldsymbol{p}$, $\dot{\boldsymbol{p}} = -\partial H/\partial \boldsymbol{x}$. For HC the Hamiltonian turns out to be linear in the pursuer control ϕ, which makes the optimal pursuer control "bang-bang" (always at the boundary of its feasible set, with sign chosen by a *switching function*). This linearity is the structural fact §?? exploits to derive the switching function in one chained SymPy call.

The following notation quickstart names the symbols this paper uses; each one is also defined at first technical use, and the supporting material holds the complete glossary. With these handles in place, §?? turns to the HC-specific work: collapsing the 5-DOF lab-frame setup to the 2-DOF body-frame system every subsequent derivation operates on.

Notation quickstart. Just the abstract objects of a Pontryagin / differential-game setup so the prose has handles to reach for. The HC-specific symbols ($x_1, x_2, \phi, \psi, \sigma, \ldots$) are introduced where they first appear in §??–§??; the full notation reference is in the **supporting material**.

Term	Notation	Quickstart description
State vector	$\boldsymbol{x}$	The system's evolving variables; in HC, the 2-DOF body-frame relative position
Player controls	$u_P,\ u_E$	The two players' choices; in HC, pursuer turn rate ϕ and evader heading ψ
Costate / adjoint	$\boldsymbol{p}$	Pontryagin's costate vector; equals ∇V^*, satisfies $\dot{\boldsymbol{p}} = -\partial H/\partial \boldsymbol{x}$
Hamiltonian	$H(\boldsymbol{x}, \boldsymbol{p}, u_P, u_E)$	The Pontryagin function; its gradient drives the state and costate ordinary differential equations (ODEs)
Value function	$V^*(\boldsymbol{x})$	The saddle-point optimal cost from state $\boldsymbol{x}$; for HC, the optimal capture time
Speed ratio	$w = v_E/v_P \in (0, 1)$	The pursuer is faster in this paper's regime

That's the vocabulary the rest of the paper operationalises in SymPy; each symbol reappears with full context where it is used.

5. Reducing HC to body-frame coordinates (the symbolic-view warm-up)

The *game* does not depend on all five DOF of §??'s lab-frame state. The same game is played by a pursuer at (x_P, y_P) heading north as by a pursuer at the origin heading north: only the *relative* position of the evader matters, and only relative to the pursuer's heading. Isaacs [12, pp. 297-300] gave the reduction: rotate and translate the lab frame so the pursuer sits at the origin facing the body-frame $+x_2$ axis. The evader's heading relabels: $\psi \equiv \psi_{\text{lab}} - \theta$, the angle the evader's velocity makes with the pursuer's heading.

The same 5-DOF lab kinematics, now as a SymPy `Matrix`:

```
# §5 — Lab-frame absolute kinematics: the 5-DOF state
# (x_P, y_P, θ, x_E, y_E) and its time derivatives, normalised so v_P = 1
# and R_min = 1. This is what the body-frame reduction below collapses.
from derivations import absolute_kinematics
sp.Eq(
    sp.Matrix([
        sp.Symbol(r"\dot{x}_P"), sp.Symbol(r"\dot{y}_P"),
        sp.Symbol(r"\dot{\theta}"),
        sp.Symbol(r"\dot{x}_E"), sp.Symbol(r"\dot{y}_E"),
    ]),
    absolute_kinematics(),
)
```

```
⎡ \dot{x}_P  ⎤   ⎡          cos(θ)          ⎤
⎢            ⎥   ⎢                          ⎥
⎢ \dot{y}_P  ⎥   ⎢          sin(θ)          ⎥
⎢            ⎥   ⎢                          ⎥
⎢\dot{\theta}⎥ = ⎢            φ             ⎥
⎢            ⎥   ⎢                          ⎥
⎢ \dot{x}_E  ⎥   ⎢w·cos(\psi_{\mathrm{lab}})⎥
⎢            ⎥   ⎢                          ⎥
⎣ \dot{y}_E  ⎦   ⎣w·sin(\psi_{\mathrm{lab}})⎦
```

Isaacs's rotation+translation written symbolically: the body-frame relative position (x_1, x_2) is a fixed linear combination of the lab-frame relative displacement $(\Delta x, \Delta y)$ rotated by $-\theta$:

```
# §5 — Display the lab → body coordinate substitution (rotation+translation).
from derivations import body_frame_substitution
sp.Eq(
    sp.Matrix([sp.Symbol("x_1"), sp.Symbol("x_2")]),
    body_frame_substitution(),
)
```

```
⎡x₁⎤   ⎡(-x_E + x_P)·sin(θ) + (y_E - y_P)·cos(θ)⎤
⎢  ⎥ = ⎢                                       ⎥
⎣x₂⎦   ⎣(x_E - x_P)·cos(θ) + (y_E - y_P)·sin(θ) ⎦
```

Watch this: these two ingredients, the lab kinematics and the body-frame substitution, are enough for SymPy [17] to derive the reduced body-frame dynamics $(\dot{x}_1, \dot{x}_2) = (f_1, f_2)$ for us. The chain is `sp.diff` (chain-rule the substitution through time), `sp.subs` (drop in the lab kinematics + relabel $\psi_{\text{lab}} = \psi + \theta$), invert the body-frame substitution so the answer reads in (x_1, x_2), and `sp.trigsimp` to collapse the trig. We do it in three small cells so the intermediate forms are visible.

First we set up time-dependent versions of the lab states (so `sp.diff` knows what depends on t) and write $x_1(t)$, $x_2(t)$ as the body-frame combinations:

```
# §5 — Set up SymPy time-functions for the lab states and write
# the body-frame relative position as combinations of those functions.
# We import phi, psi, w, x1, x2 (and the costates p1, p2 needed by §6).
from derivations import phi, psi, w, x1, x2, p1, p2

t = sp.Symbol("t", real=True)
xP_t = sp.Function("x_P")(t)
yP_t = sp.Function("y_P")(t)
theta_t = sp.Function(r"\theta")(t)
xE_t = sp.Function("x_E")(t)
yE_t = sp.Function("y_E")(t)
psi_lab_t = sp.Symbol(r"\psi_{\mathrm{lab}}", real=True)

x1_of_t = -(xE_t - xP_t) * sp.sin(theta_t) + (yE_t - yP_t) * sp.cos(theta_t)
x2_of_t =  (xE_t - xP_t) * sp.cos(theta_t) + (yE_t - yP_t) * sp.sin(theta_t)
sp.Eq(sp.Function(r"x_1")(t), x1_of_t)
```

$$x_1(t) = (-x_E(t) + x_P(t))\sin(\theta(t)) + (y_E(t) - y_P(t))\cos(\theta(t))$$

Now take d/dt of $x_1(t)$. SymPy's chain rule produces a mess of `Derivative(…)` terms — the *unreduced* time-derivative of the body-frame substitution:

```
# §5 — Time-differentiate x_1(t). SymPy chain-rules through the
# substitution and leaves Derivative(...) terms for the lab states.
x1_dot_raw = sp.diff(x1_of_t, t)
sp.Eq(sp.Symbol(r"\dot{x}_1^{\,\mathrm{raw}}"), x1_dot_raw)
```

$$\dot{x}_1^{\mathrm{raw}} = (-x_E(t) + x_P(t))\cos(\theta(t))\frac{d}{dt}\theta(t) - (y_E(t) - y_P(t))\sin(\theta(t))\frac{d}{dt}\theta(t) + \left(-\frac{d}{dt}x_E(t) + \frac{d}{dt}x_P(t)\right)\sin(\theta(t)) + \left(\frac{d}{dt}y_E(t) - \frac{d}{dt}y_P(t)\right)\cos(\theta(t))$$

Substitute the lab kinematics, relabel ψ_{lab}, invert the body-frame substitution so the answer reads in (x_1, x_2), and let `sp.trigsimp` finish the trig identities. The same chain applied to $x_2(t)$ gives f_2.

```
# §5 — Final SymPy chain: subs(lab_kin) → relabel ψ_lab → subs(inv_body) → trigsimp.
# Display as a Python list of Eq (avoids the sp.Matrix-of-Eq deprecation).
lab_kin = {
    sp.diff(xP_t, t): sp.cos(theta_t),
    sp.diff(yP_t, t): sp.sin(theta_t),
    sp.diff(theta_t, t): phi,
    sp.diff(xE_t, t): w * sp.cos(psi_lab_t),
    sp.diff(yE_t, t): w * sp.sin(psi_lab_t),
}
inv_body = {
    xE_t: -x1 * sp.sin(theta_t) + x2 * sp.cos(theta_t) + xP_t,
    yE_t:  x1 * sp.cos(theta_t) + x2 * sp.sin(theta_t) + yP_t,
}

def derive(xi_of_t):
    return sp.trigsimp(sp.expand(
        sp.diff(xi_of_t, t).subs(lab_kin).subs(psi_lab_t, psi + theta_t).subs(inv_body)
    ))

f1, f2 = derive(x1_of_t), derive(x2_of_t)
[sp.Eq(sp.Symbol(r"\dot{x}_1"), f1),
 sp.Eq(sp.Symbol(r"\dot{x}_2"), f2)]
```

$$\left[\dot{x}_1 = -\phi x_2 + w\sin(\psi),\ \ \dot{x}_2 = \phi x_1 + w\cos(\psi) - 1\right]$$

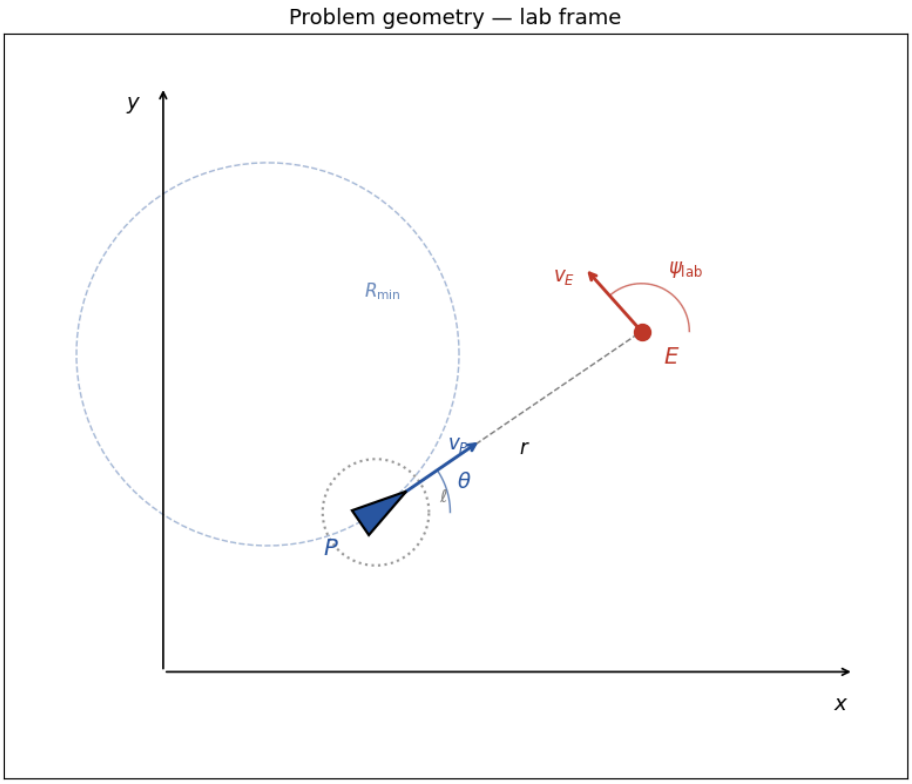
SymPy did the algebra Isaacs would have done by hand. The reader can re-run with a swapped convention (say, evader $\sin \leftrightarrow \cos$) and watch the simplified form change.

Coordinate changes that exploit symmetry to reduce state-space dimension are a standard move in dynamical-systems analysis — and often necessary in differential games, where the full state may be too high-dimensional for a value-function field to be tractable. The reduction is also *invertible* given the pursuer's initial lab pose $(x_P(0), y_P(0), \theta(0))$: composing the body-frame trajectory with the integrated lab-frame pursuer state recovers the full lab-frame chase. That is why §??'s `chase_demo` (lab frame) and §??'s `dispersal_crossing` (body + costate + strategy) render the *same* trajectory from different coordinate views.

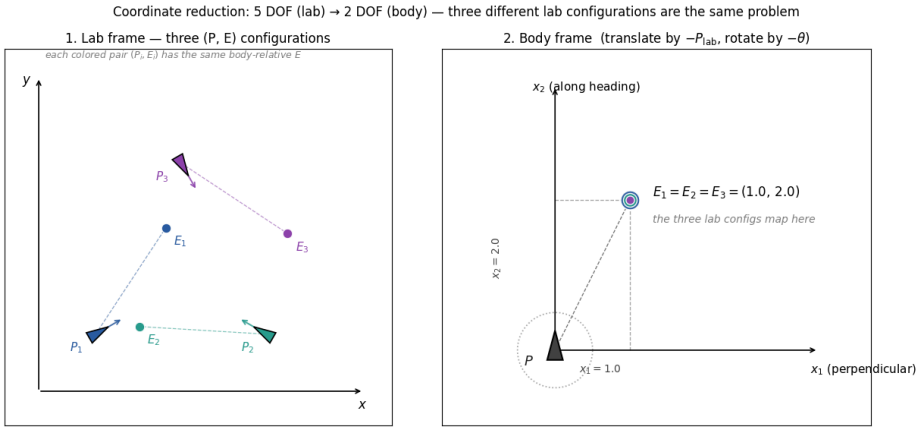
Two figures support the reduction. `problem_geometry` (the lab-frame schematic) names the symbols visually: pursuer heading θ, turning circle of radius $R_{\min}$, capture circle of radius ℓ around the pursuer, evader velocity vector at angle ψ_{lab} to the lab $+x$ axis. `coordinate_progression` (the pair of panels) shows *why* the reduction is interesting: three of the infinitely many lab configurations sharing one relative geometry (pursuer here facing east, there facing northwest, elsewhere facing southeast) all collapse to the *same point* on the body-frame panel. Infinitely many lab states, one game state.

The reduction also changes what the optimal-control problem solves for. In lab coordinates the value function would depend on all five lab DOF; in body coordinates it depends on (x_1, x_2) only. The synthesis: the value function as backward reachable set's `reachable_set_heatmap` renders a 2-D field; without the reduction it would be a 5-D field nobody could visualise. With the body-frame reduction in hand, §?? writes down the Hamiltonian and derives the optimal controls.

```
# §5 supporting figure — lab-frame schematic (problem geometry).
# Sets up the 5-DOF lab state (P_lab, E_lab, θ) the body-frame
# reduction above collapsed to 2 DOF.
from plots import problem_geometry
problem_geometry()
```



```
# §5 figure — three lab configurations collapse to ONE body-frame point.
# Makes the dimensional-reduction insight visible: the body frame is an
# equivalence class on the 5-DOF lab state, not just a relabelling.
from plots import coordinate_progression
coordinate_progression()
```



6. Defining V^* and constructing the Hamiltonian

The **value function** $V^*(\boldsymbol{x})$ is what we are after: the optimal capture time when the game starts from body state $\boldsymbol{x}$ with both players following the saddle-point strategy. The reachable set at time T is its sublevel $R(T) = \{\, \boldsymbol{x} : V^*(\boldsymbol{x}) \le T \,\}$, the set of starting positions from which capture is possible within T seconds of optimal play. V^* exists everywhere the game is well-posed; §?? will render the whole field as a heat map. This section derives the differential equations that connect V^* to the dynamics.

The plan is the *method of characteristics*, run backward: along special curves (the characteristics — here, the optimal trajectories themselves), a partial differential equation (PDE) reduces to ordinary differential equations we integrate directly. The terminal condition is fixed: at the moment of capture, the state lies on the circle $x_1^2 + x_2^2 = \ell^2$ and $V^* = 0$. Optimal trajectories arrive at that boundary; integrating their dynamics backward in time from the boundary recovers $V^*(\boldsymbol{x})$ everywhere the characteristics reach. The Hamilton-Jacobi-Isaacs (HJI) PDE [18] is the axiomatic-formal statement of this: V^* is its *viscosity solution*, the generalised-solution notion for PDEs with corners (a historical term, unrelated to physical viscosity). The synthesis: the value function as backward reachable set makes the anchor explicit; here we set up the characteristics by writing the Hamiltonian.

For the time-optimal differential game the Hamiltonian assembles directly: $H = p_1 f_1 + p_2 f_2 + 1$, where p_1, p_2 are the costate components dual to (x_1, x_2) and the +1 comes from minimising the running cost $L = 1$ (capture time itself). One line of SymPy multiplication:

```
# §6 — Hamiltonian assembly: H = p_1 f_1 + p_2 f_2 + 1 (Pontryagin, L=1).
H = p1 * f1 + p2 * f2 + 1
sp.Eq(sp.Symbol("H"), H)
```

$$H = p_1(-\phi x_2 + w\sin(\psi)) + p_2(\phi x_1 + w\cos(\psi) - 1) + 1$$

Notice that H is *linear* in the pursuer control ϕ: that is the structural observation the next cell exploits. One SymPy call (`sp.expand(H).coeff(phi)`) extracts the coefficient of ϕ; the **switching function** $\sigma = p_2 x_1 - p_1 x_2$ falls out directly:

```
# §6 — Switching function via one SymPy call: σ = sp.expand(H).coeff(φ).
sigma = sp.expand(H).coeff(phi)
sp.Eq(sp.Symbol(r"\sigma"), sigma)
```

$$\sigma = -p_1 x_2 + p_2 x_1$$

PMP's bang-bang minimiser is then $\phi^* = -\mathrm{sign}(\sigma)$, because minimising a linear function over $[-1, 1]$ always lands on the boundary opposite the slope's sign [13, §3.2]. The locus $\sigma = 0$ is where the bang-bang control switches; in HC this locus is a curve in body space, the *dispersal surface* §?? visits.

```
# §6 — Bang-bang pursuer control: φ* = -sign(σ).
phi_star = -sp.sign(sigma)
sp.Eq(sp.Symbol(r"\phi^*"), phi_star)
```

$$\phi^* = -\mathrm{sign}\left(-p_1 x_2 + p_2 x_1\right)$$

The evader's optimal heading falls out the same way: $\psi^* = \mathrm{atan2}(p_1, p_2)$ maximises the evader's contribution to H, which works out to the evader pointing its velocity vector along the costate $\boldsymbol{p}$.

Geometric reading: the costate is the gradient of the value function, and the evader runs straight up that gradient. This is the single most useful intuition the rest of the paper exploits.

```
# §6 — Optimal evader heading: ψ* = atan2(p_1, p_2).
psi_star = sp.atan2(p1, p2)
sp.Eq(sp.Symbol(r"\psi^*"), psi_star)
```

$$\psi^* = \mathrm{atan}_2\left(p_1, p_2\right)$$

One technical detail anchors §??'s backward integration: the usable part of the terminal capture circle is the arc where $\sin\alpha > w$ [13, §4]; only on that arc can the pursuer prevent the evader from escaping at the moment of contact. Backward characteristics start from this arc, parameterised by α, the angular position on the capture circle where each characteristic terminates. The synthesis: the value function as backward reachable set's `value_function_grid` sweeps α and integrates each characteristic backward to build the field.

The glass-box moment: one trajectory through every lens takes one optimal play through every coordinate lens at once. The cells we just built (Hamiltonian, switching function, bang-bang control, evader heading) are the analytical content the next figure visualises.

7. The glass-box moment: one trajectory through every lens

Substituting ϕ^* and ψ^* into the body-frame dynamics gives a 4-dimensional autonomous ODE in (x_1, x_2, p_1, p_2). The next cell displays it: the state evolves under the closed-loop dynamics with the bang-bang control $\phi^* = -\mathrm{sign}(\sigma)$ and the evader-velocity components

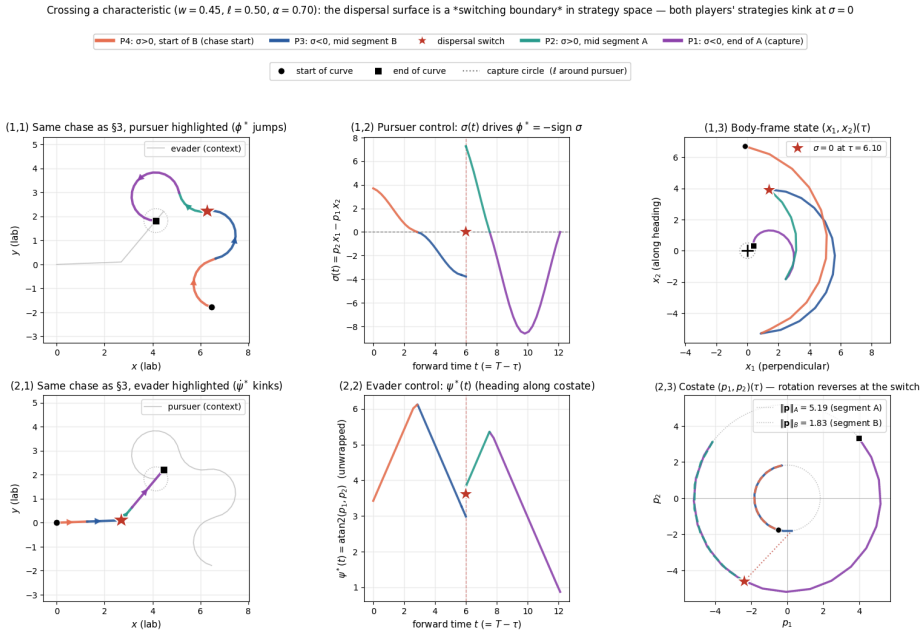
determined by $\boldsymbol{p}$. This 4-D system is the *axiomatic-formal object* the rest of §?? visualises: every figure below is a slice through one solution of this ODE.

```
# §7 — The 4-D characteristic ODE with φ*, ψ* substituted (the autonomous closed-loop system).
from derivations import characteristic_system
sp.Eq(
    sp.Matrix([
        sp.Symbol(r"\dot{x}_1"), sp.Symbol(r"\dot{x}_2"),
        sp.Symbol(r"\dot{p}_1"), sp.Symbol(r"\dot{p}_2"),
    ]),
    sp.Matrix(list(characteristic_system())),
)
```

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{p}_1 \\ \dot{p}_2 \end{bmatrix} = \begin{bmatrix} \dfrac{p_1 \cdot w}{\sqrt{p_1^2 + p_2^2}} + x_2 \cdot \operatorname{sign}(-p_1 \cdot x_2 + p_2 \cdot x_1) \\ \dfrac{p_2 \cdot w}{\sqrt{p_1^2 + p_2^2}} - x_1 \cdot \operatorname{sign}(-p_1 \cdot x_2 + p_2 \cdot x_1) - 1 \\ p_2 \cdot \operatorname{sign}(-p_1 \cdot x_2 + p_2 \cdot x_1) \\ -p_1 \cdot \operatorname{sign}(-p_1 \cdot x_2 + p_2 \cdot x_1) \end{bmatrix}$$

The 6-panel `dispersal_crossing` figure shows one optimal chase through six coordinate views simultaneously. The chase is the same one §??'s `chase_demo` motivates; here it is decomposed into its analytical structure. We take the panels in turn:

```
# §7 GLASS-BOX EXEMPLAR — one optimal play, six coordinate views.
# Panel layout + reading guide in the following §7 prose.
from plots import dispersal_crossing
dispersal_crossing()
```



(1,1) Lab frame, pursuer highlighted. The pursuer's trajectory traces an S-shape, with the bang-bang ϕ^* jumping at the dispersal switch ($\star$). The evader's path is greyed.

(1,2) Strategy: $\sigma(t)$. The switching function plotted forward in time. The curve changes sign at the $\star$ and at two smooth crossings either side of it; each sign change starts a new colour segment in (1,1). The $\star$ marks the dispersal switch: at $\sigma = 0$ the pursuer's optimal turning direction is undetermined, and the trajectory bifurcates into two regimes; the locus of such points is a *dispersal surface*, a boundary where two distinct optimal paths become available at once [13], [19].

(1,3) State: body-frame trajectory. The same chase in body coordinates (x_1, x_2). The evader spirals inward toward the capture circle; the spiral pitch flips at the dispersal crossing. This panel is the one body-frame picture that stands in for every lab-frame chase sharing the same relative geometry, including (1,1) and (2,1).

(2,1) Lab frame, evader highlighted. Same chase, opposite highlight. The evader's path is the smooth-then-kinked curve we previewed in §??'s `chase_demo`. The pursuer is greyed; the $\star$ is still there.

(2,2) Strategy: $\psi^*(t)$. The evader's optimal heading, forward in time. Like ϕ^*, ψ^* jumps at the $\star$; the heading-angle discontinuity is what produces the visible kink in the evader's lab-frame path.

(2,3) Costate: (p_1, p_2) phase portrait. The costate trajectory. Two concentric circles of radii $|\,\boldsymbol{p}\,|_B$ and $|\,\boldsymbol{p}\,|_A$ (the costate magnitudes before and after the dispersal) are connected by a dotted red chord at the switch: V^*'s gradient is multi-valued at the dispersal surface, and the costate magnitude jumps pre- and post-switch.

The chase rendered across all six panels is recovered from `chase_demo`'s hardcoded arrays by stitching two optimal characteristics joined at the dispersal point ($\alpha_A = 40^\circ$ post-switch, $\alpha_B = 95^\circ$ pre-switch), so the chase in §?? and the analytical decomposition in §?? are the *same* chase. The plumbing lives in `plots._recover_chase_from_demo`; the costate norms differ across the switch ($|\,\boldsymbol{p}\,|_B \approx 1.83$, $|\,\boldsymbol{p}\,|_A \approx 5.18$), which is exactly what makes the dispersal surface a boundary in strategy space.

This is the glass-box moment. Every panel of `dispersal_crossing` renders a slice of the *same* 4-D ODE solution. The transformations between coordinate views (lab $\leftrightarrow$ body $\leftrightarrow$ strategy $\leftrightarrow$ costate) are now executable cells, not paragraph headers. The reader can re-run with different (w, ℓ), re-stitch with different (α_A, α_B), or call any of `derivations.py`, `numerics.py`, `plots.py` directly from a fresh notebook to inspect any intermediate object. If one optimal chase has structure this rich, what does the whole space of starts look like? The synthesis: the value function as backward reachable set answers — and earns the §?? promise of *verifying the whole chain* via the costate ODE, conservation lemma, and lambdified right-hand side (RHS) that the heat map sits on top of; the supporting material runs the live symbolic-versus-numerical cross-check.

8. The synthesis: the value function as backward reachable set

This section has two goals: (1) render $V^*(x_1, x_2)$ over the body frame as a heat map, and (2) finish the analytical machinery — costate ODE, conservation invariant, and the SymPy $\rightarrow$ NumPy seam (where symbolic expressions compile into fast numeric callables) — that the heat-map cell depends on. We derive the costate dynamics first, then render and interpret the synthesis figure, then finish the conservation and seam machinery.

The **costate (adjoint) ODE** comes from Pontryagin's principle: $\dot{p}_i = -\partial H / \partial x_i$. Two `sp.diff` calls:

```
# §8 — Costate (adjoint) ODE for p_1: dp_1/dt = -∂H/∂x_1.
# Pontryagin's maximum principle, written as a one-line SymPy call.
sp.Eq(sp.Symbol(r"\dot{p}_1"), -sp.diff(H, x1))
```

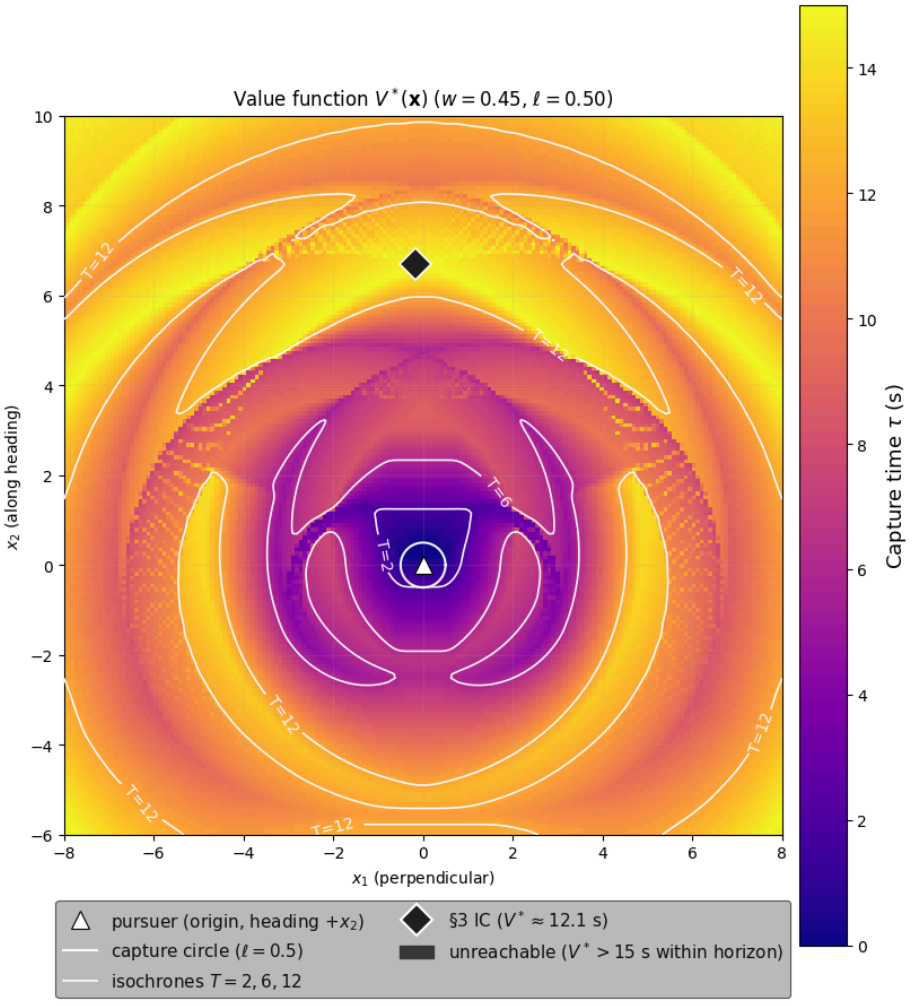
$$\dot{p}_1 = -p_2\phi$$

```
# §8 — Costate ODE for p_2: dp_2/dt = -∂H/∂x_2.
sp.Eq(sp.Symbol(r"\dot{p}_2"), -sp.diff(H, x2))
```

$$\dot{p}_2 = p_1\phi$$

With the adjoint dynamics in hand, we sweep 600 backward characteristics from the usable terminal arc, integrate each one back to time-to-go $\tau = 15$ s, and interpolate the (x_1, x_2, τ) samples onto a 180×180 grid. Plasma colour codes capture time from 0 (purple, on the capture circle) to 15 s (yellow at the cap). The pursuer triangle sits at the origin with heading $+x_2$; white isochrones at $T = 2, 6, 12$ trace level sets of V^*; the capture circle itself is, by definition, the $T = 0$ isochrone.

```
# §8 SYNTHESIS — V*(x) on a body-frame grid; see §8 prose for the
# 15-s cap, IC anchor, and barrier-surface interpretations.
# (Bump _VFG_CACHE_VERSION in numerics.py to invalidate the .npz cache.)
from numerics import value_function_grid
from plots import reachable_set_heatmap

W_REF, ELL_REF = 0.45, 0.5
ISOCHRONE_LEVELS = (2.0, 6.0, 12.0)

x1_grid, x2_grid, V_grid, reachable_mask, isochrones = value_function_grid(
    W_REF, ELL_REF,
    T_max_cap=15.0,
    grid_extent=(-8.0, 8.0, -6.0, 10.0),
    n_grid=180,
    n_traj=600,
    n_time_samples=400,
    isochrone_levels=ISOCHRONE_LEVELS,
)
reachable_set_heatmap(
    x1_grid, x2_grid, V_grid, reachable_mask, W_REF, ELL_REF,
    T_max_cap=15.0, vmax_color=15.0,
    isochrone_levels=ISOCHRONE_LEVELS,
    isochrone_curves=isochrones,
    highlight_ic=(-0.166, 6.708, "§3 IC"),
    highlight_ic_value=12.1,
)
```



The black diamond is the §?? `chase_demo` initial condition, mapped through the body-frame substitution to $(-0.166, 6.708)$. The legend's $V^*(\diamond) \approx 12.1$ s is plumbed in directly from `chase_demo`'s measured saddle-point capture time; the heat map's interpolation reads nearly three seconds high here, because the backward fan does not pass exactly through this point and linear interpolation blends in samples from neighbouring, longer-time characteristics. The same chase thus appears in §?? and §?? with the same capture time.

The **15-second cap is a *rendering* horizon, not a game-theoretic claim**. These parameters sit in the well-behaved region of the game: Isaacs's capture criterion [12] at $\ell = 0.5$, $R_{\min} = 1$ puts the escape threshold near $w \approx 0.95$, so at $w = 0.45$ the pursuer catches from every initial condition. Cells the angular fan does not reach within 15 seconds would render in dark grey (the legend's "unreachable" entry); none occur within this figure's extent. Genuinely no-capture regimes, and stranger singular-surface structures, exist elsewhere in the (w, ℓ) parameter plane [13], [18, §3].

The sharp boundary where the dark inner kidney (the region around the pursuer, roughly $|\, x_1 \,| \le 3$, $-2 \le x_2 \le 4$) meets the lighter outer field is the *barrier singular surface*: V^*'s level sets meet at a corner there, with V^* jumping from $\sim$2 s to $\sim$6 s over less than 0.5 body-frame units. Sampling-density checks (`n_traj` $\in \{100, 300, 600\}$) confirm it as a real HC feature, not a sampling artifact.

The remaining machinery follows. The **conservation lemma** shows that $d/dt \,|\, \boldsymbol{p} \,|^2 = 0$ along every optimal characteristic. The chain-rule sum is held un-evaluated with `sp.Add(..., evaluate=False)` so SymPy's auto-simplifier doesn't collapse it before the reader sees the structure; the next cell's `sp.simplify` is what actually drives the result to literal zero:

```
# §8 — Conservation lemma 1: chain-rule sum, kept unevaluated
# (sp.Add(..., evaluate=False)) so sp.simplify drives the cancellation in lemma 2.
p1_dot = -sp.diff(H, x1)
p2_dot = -sp.diff(H, x2)
drift_raw = sp.Add(2 * p1 * p1_dot, 2 * p2 * p2_dot, evaluate=False)
sp.Eq(sp.Symbol(r"\tfrac{d}{dt}\|\mathbf{p}\|^2_{\,\text{raw}}"), drift_raw)
```

```
\tfrac{d}{dt}\|\mathbf{p}\|_{\,\text{raw}}__2 = -2·p₁·p₂·φ + 2·p₁·p₂·φ
```

```
# §8 — Conservation lemma 2: sp.simplify drives the drift to literal zero.
drift = sp.simplify(drift_raw)
sp.Eq(sp.Symbol(r"\tfrac{d}{dt}\|\mathbf{p}\|^2"), drift)
```

```
\tfrac{d}{dt}\|\mathbf{p}\|² = 0
```

Watch this: SymPy did in one line the cancellation Isaacs (1965) verified by hand. The conservation invariant is now executable evidence rather than a textbook claim.

The seam. `numerics.lambdify_rhs` calls `sp.lambdify` on the characteristic ODE system, returning a NumPy-callable `rhs(t, state, w)` where `state = [x_1, x_2, p_1, p_2]` and the return value is the 4-vector $[\dot{x}_1, \dot{x}_2, \dot{p}_1, \dot{p}_2]$. This is exactly the signature `scipy.integrate.solve_ivp` consumes when integrating the 600 backward characteristics for the heat map. White-box on the symbolic side (the SymPy expression is inspectable with `sp.srepr`), black-box on the numeric side (a fast callable for the integrator), glass-box because both sides live in the source [17], [20].

To make the seam concrete: at probe state $(x_1, x_2, p_1, p_2) = (1.0, 0.5, 0.3, 0.8)$ with $w = 0.3$, the closed-form derivations give $\sigma = p_2 x_1 - p_1 x_2 = 0.65 > 0$, so $\phi^* = -1$. Then $\dot{x}_1 = -\phi^* x_2 +$

$w \sin\psi^* \approx 0.605$, $\dot{x}_2 = \phi^* x_1 + w\cos\psi^* - 1 \approx -1.719$, $\dot{p}_1 = -p_2\phi^* = 0.8$, $\dot{p}_2 = p_1\phi^* = -0.3$. The lambdified callable returns the same four numbers:

```
# §8 — THE SEAM: SymPy → NumPy via sp.lambdify. Probe-state spot-check.
# Closed-form expected values: see §8 prose immediately above.
from numerics import lambdify_rhs
rhs = lambdify_rhs()
rhs(0.0, [1.0, 0.5, 0.3, 0.8], 0.3)
```

```
[0.605337032476517, −1.71910124672929, 0.8, −0.3]
```

One sentence anchors V^* to its axiomatic-formal definition: it is the unique viscosity solution of the HJI PDE with $V = 0$ on the capture circle [18]. We render V^* via backward characteristics rather than by solving the PDE because shipping a viscosity-solution finite-element solver in 6,000 words is beyond scope; the axiomatic-formal anchor is the PDE itself, cited rather than enacted. Two further figures (`plots.optimal_vector_field` for the bang-bang feedback policy, `plots.trajectory_fan` for the family of optimal characteristics) live in the codebase and ship with the public hc-marimo notebook [16], [21], [22].

9. What just happened: SymPy is the missing world; the SciPy stack now covers all three

Standard SciPy practice covers two of Tall's three worlds. NumPy and SciPy operationalise the **operational-symbolic** world: procedures that solve, integrate, optimise [20], [22]. Matplotlib operationalises the **conceptual-embodied** world: vector fields, trajectories, heat maps you read with your eyes [16]. The **axiomatic-formal** world, symbol-as-object manipulation under rewrite rules, usually lives on paper or in a textbook citation. SymPy [17] is what moves it into the same execution environment.

What changed in this paper because SymPy was in the notebook? The body-frame reduction in §?? ran as a SymPy chain (`sp.diff` + `sp.subs` + `sp.trigsimp`) rather than three pages of indices. The Hamiltonian in §?? assembled in a single line, $H = p_1 f_1 + p_2 f_2 + 1$. The switching function fell out as `sp.expand(H).coeff(phi)` instead of by-hand collection of terms. The costate ODE was a `sp.diff(H, x_i)` call; the conservation lemma a `sp.simplify(...)` that returned literal zero. Without SymPy, the reader's only audit path would be to read Isaacs (1965, pp. 297-320); with SymPy, the reader watches the derivation happen and can re-run any cell with different parameters to verify the chain end-to-end.

The Buchberger principle [2] governs which mode of work belongs where. Differentiation and integration *rules* belong in the black-box mode by now: their white-box phase dates to Heid [10]'s 1988 calculator-classroom studies, and we trust `sp.diff` to apply them. The white-box mode in this paper is therefore *one rung up the recursion*: Hamiltonian construction, costate derivation, switching-function recognition. Drijvers and Artigue [3], [4] sharpened Buchberger's binary into the third, *glass-box*, mode (§??). SymPy is glass-box in this sense: `sp.expand(H).coeff(phi)` is a single call, but the reader can inspect H before and after, ask for `H.atoms()` or `H.free_symbols`, and trace exactly which rule was applied.

One cell in this section illustrates the *limits* of what computer algebra automates. Substituting $\phi^* = -\mathrm{sign}(\sigma)$ into H yields a `Piecewise` expression branching on the sign of σ. The auto-simplifier does not reduce $-\mathrm{sign}(\sigma)\cdot\sigma$ to $-|\,\sigma\,|$ because the recognition step needs a pattern rewrite SymPy does not apply by default. Glass-box does not mean *omniscient*; it means *inspectable*. SymPy executes rewrite tactics; choosing which tactic to invoke still takes creativity the first time. Once made, that choice persists as version-controlled, inspectable, verifiable SymPy code — exactly what `derivations.py` preserves here.

```
# §9 — Live H.subs(φ, φ*). SymPy returns Piecewise — the auto-simplifier
# does not auto-reduce -sign(σ)·σ to -|σ|. See §9 prose for the honest moment.
H_after_phi = H.subs(phi, phi_star)
H_after_phi
```

$$p_1(w\sin(\psi) + x_2\operatorname{sign}(-p_1x_2 + p_2x_1)) + p_2(w\cos(\psi) - x_1\operatorname{sign}(-p_1x_2 + p_2x_1) - 1) + 1$$

The clean closed form, written by Isaacs (1965, p. 304) as $H^* = w \mid \boldsymbol{p} \mid - \mid \sigma \mid - p_2 + 1$, is the saddle-point Hamiltonian the bang-bang substitution *should* reduce to. We display the clean form via the helper in `derivations.py`, then substitute the definitions of $\mid \boldsymbol{p} \mid$ and σ — asserting the equivalence with a one-line `sp.simplify` — to recover Isaacs's typography exactly:

```
# §9 — Closed-form H* (Isaacs 1965, p. 304), via derivations.hamiltonian_saddle().
from derivations import hamiltonian_saddle
sp.Eq(sp.Symbol(r"H^*"), hamiltonian_saddle())
```

$$H^* = -p_2 + w\sqrt{p_1^2 + p_2^2} - |p_1x_2 - p_2x_1| + 1$$

```
# §9 — Same H*, in Isaacs's own symbols. The assert substitutes the
# definitions ‖p‖ = sqrt(p_1² + p_2²) and σ = p_2·x_1 − p_1·x_2 into the
# named-symbol form and verifies it against hamiltonian_saddle(); the
# final line renders Isaacs's (1965, p. 304) typography term for term.
from derivations import hamiltonian_saddle, x1, x2, p1, p2, w
p_norm = sp.Symbol(r"\|\mathbf{p}\|", positive=True)
sigma_sym = sp.Symbol(r"\sigma", real=True)
H_isaacs = w * p_norm - sp.Abs(sigma_sym) - p2 + 1
assert sp.simplify(
    H_isaacs.subs({p_norm: sp.sqrt(p1**2 + p2**2),
                   sigma_sym: p2 * x1 - p1 * x2}) - hamiltonian_saddle()) == 0
sp.Eq(sp.Symbol("H^*"), sp.Symbol(r"w\|\mathbf{p}\| - \left|\sigma\right| - p_2 + 1"))
```

$$H^* = w\|\mathbf{p}\| - |\sigma| - p_2 + 1$$

Sections ?? through ?? are what earn the §?? thesis: **the glass-box principle becomes practicable when all three of Tall's worlds live in the same execution environment.** Standard SciPy practice already keeps the operational-symbolic and embodied worlds in the same notebook; SymPy bridges the axiomatic-formal world onto the same page as the figures it justifies. The seams between worlds (`derivations.py` → `numerics.lambdify_rhs` → `scipy.integrate.solve_ivp` → `matplotlib.Figure`) are what the reader can audit. Without SymPy, the chain has one black-box step (the formal derivation in a citation); with SymPy, every step is glass-box. Conclusion: critical infrastructure + reproducibility + methodology generalises names the publication infrastructure that makes all three worlds live in one artifact.

10. Conclusion: critical infrastructure + reproducibility + methodology generalises

We named four tool families to make this paper work: SymPy for the axiomatic-formal world, NumPy and SciPy for operational-symbolic, matplotlib for embodied, and MyST + Jupyter + Curvenote for the integrating substrate. We credit them all together because the contribution is about the *category* of tools they exemplify, not about any specific stack.

A dynamic computational-publication environment combines text, parameters, symbolic derivations, numerical simulations, and visual elements *in the same artifact*; this paper is one instance of the category, and the public hc-marimo notebook this paper is built on [21], running on the marimo framework [23], is another. The two instances run on different infrastructure (marimo + GitHub Pages versus Jupyter + MyST + Curvenote [5], [6], [7]) and the choice between them is situational, matched to use case and audience; the load-bearing claim is the category itself. Kadie's 2025 SciPy paper on solvable and unsolvable equations in SymPy [24] is another notebook-primary instance in the same proceedings; the architectural choice is no longer unprecedented.

The methodology generalises. The same treatment (symbolic derivation in SymPy, numerical integration in SciPy, visual decomposition in matplotlib, with the seams visible) applies to other control-systems concepts where the formalism is hard to materialise and the embodied view is load-bearing. Three named instances live at hc-marimo: (1) the singular-surface taxonomy applied to autonomous-vehicle (AV) collision avoidance, where dispersal and equivocal surfaces classify decision boundaries [13], [19]; (2) multi-pursuer differential games applied to unmanned-aerial-vehicle (UAV) swarm defense, where coalition value functions extend Isaacs's pursuit framework [25]; (3) forward physical-coordinate chase applied to hardware-in-the-loop (HIL) testing of pursuit controllers: the same characteristic-integration backbone (built from the modern HC restatement [14]) drives controller validation in a simulator. Each ships alongside the figures cut from this paper for space (`plots.optimal_vector_field`, `plots.trajectory_fan`, `plots.conservation_diagnostics`).

Motivation. We have found Tall's three-worlds model, the glass-box principle, and reactive notebooks indispensable communication aids for advanced engineering mathematics across several artifacts at the Dynamical Systems Group: the public hc-marimo notebook this paper is built on, a spacecraft attitude-control lifecycle requirements-traceability demo [26], and an evolution-of-trust showcase built on a compositional game-theory ecosystem [27]. The same three-worlds pedagogy extends to settings where the state space is a data structure rather than a real-valued field and differentiability cannot be assumed [28], [29]; the symbolic substrate need not be SymPy: semantic-web tools such as the Web Ontology Language (OWL) [30] also provide formal inference.

Sharpened pitch. The embodied and interactive modes are particularly load-bearing at the graduate level, where the formalism is hardest to materialise on the page. The axiomatic-formal world being *operationally available* via SymPy (not hand-waved through "see the textbook") unlocks pedagogical ground that pure-numerical SciPy practice cannot reach. Glass-box is the practical mode for that ground: the reader audits the symbolic derivation, watches the lambdified function get integrated, reads the resulting figure, and can ground-truth any step against the next.

Reproducibility. The suite under `tests/` runs in seconds via `uv run pytest tests/` and certifies every body-section invariant the paper relies on. The **supporting material** carries the full audit: the per-test catalog, a live pytest run executed in-notebook, and the symbolic-vs-numerical cross-check on the costate-norm conservation lemma — the same $d/dt \mid \boldsymbol{p} \mid^2 = 0$ that §?? proves by `sp.simplify` is verified numerically to the integrator's noise floor along every characteristic.

The publication substrate makes the integration possible. Crediting the SymPy, NumPy, SciPy, matplotlib, Jupyter, MyST, Curvenote, and marimo teams is therefore not a courtesy line; it is the load-bearing acknowledgement that scientific computing now has the infrastructure to operationalise Tall's three-worlds theory in a single executable artifact. Teaching advanced applied mathematics finally has the substrate it always needed.

Acknowledgments

Tools used: Claude Code (Anthropic), `claude-opus-4-7` (1M context window). The author specified this paper as point-for-point outlines, named all software components with test-driven development up front, and hand-tuned the code, figures, and text throughout the published content. The build order was outline, then code, then detailed outline, then prose. AI assistance produced drafts inside that specification; the author corrected each pass and verified the mathematics against the cited primary sources [12], [13], [14], [18]. The work was conducted in a variation of the purpose-built AI-assisted-authorship harness described and peer-reviewed in [31], whose deterministic test gates (here, 36 pytest assertions and `utils.validate_citations`) and named-human approver field are the verification mechanism this paper inherits. The author takes full responsibility for the accuracy and integrity of the content.

References

[1] D. O. Tall, "The Transition to Formal Thinking in Mathematics," *Mathematics Education Research Journal*, vol. 20, no. 2, pp. 5–24, 2008, [Online]. Available: https://files.eric.ed.gov/fulltext/EJ820231.pdf

[2] B. Buchberger, "Should Students Learn Integration Rules?," *ACM SIGSAM Bulletin*, vol. 24, no. 1, pp. 10–17, 1990.

[3] M. Artigue, "Learning Mathematics in a CAS Environment: The Genesis of a Reflection About Instrumentation and the Dialectics Between Technical and Conceptual Work," *International Journal of Computers for Mathematical Learning*, vol. 7, no. 3, pp. 245–274, 2002, doi: 10.1023/A:1022103903080.

[4] D. O. Tall and E. Dubinsky, "Advanced Mathematical Thinking and the Computer," in *Advanced Mathematical Thinking*, vol. 11, D. O. Tall, Ed., in Mathematics Education Library, vol. 11., Dordrecht: Kluwer Academic Publishers, 1991, pp. 231–248.

[5] T. Kluyver *et al.*, "Jupyter Notebooks — a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt, Eds., 2016, pp. 87–90. doi: 10.3233/978-1-61499-649-1-87.

[6] Project Jupyter *et al.*, "Jupyter Book 2 and the MyST Document Stack: A modular, extensible, web-native stack for authoring and publishing computational narratives," in *Proceedings of the 24th Python in Science Conference*, in Proceedings of the Python in Science Conference. 2025, pp. 173–193. doi: 10.25080/hwcj9957.

[7] Curvenote Inc., "Curvenote: a continuous-publishing platform for scientific content." [Online]. Available: https://curvenote.com/

[8] D. O. Tall, *How Humans Learn to Think Mathematically: Exploring the Three Worlds of Mathematics*. in Learning in Doing: Social, Cognitive and Computational Perspectives. Cambridge: Cambridge University Press, 2013.

[9] A. Sfard, "On the Dual Nature of Mathematical Conceptions: Reflections on Processes and Objects as Different Sides of the Same Coin," *Educational Studies in Mathematics*, vol. 22, no. 1, pp. 1–36, 1991, doi: 10.1007/BF00302715.

[10] M. K. Heid, "Resequencing Skills and Concepts in Applied Calculus Using the Computer as a Tool," *Journal for Research in Mathematics Education*, vol. 19, no. 1, pp. 3–25, 1988, [Online]. Available: https://www.jstor.org/stable/749108

[11] R. Isaacs, "Games of Pursuit," techreport P-257, 1951.

[12] R. Isaacs, *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. New York: John Wiley & Sons, 1965.

[13] A. W. Merz, "The Homicidal Chauffeur — A Differential Game." 1971.

[14] M. Pachter and S. Coates, "The Classical Homicidal Chauffeur Game," *Dynamic Games and Applications*, vol. 9, no. 3, pp. 800–850, 2019, doi: 10.1007/s13235-018-0264-8.

[15] V. S. Patsko and V. L. Turova, "Homicidal Chauffeur Game: History and Modern Studies," in *Advances in Dynamic Games: Theory, Applications, and Numerical Methods for Differential and Stochastic Games*, vol. 11, M. Breton and K. Szajowski, Eds., in Annals of the International Society of Dynamic Games, vol. 11., Birkhäuser Boston, 2011, pp. 227–251. doi: 10.1007/978-0-8176-8089-3_12.

[16] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[17] A. Meurer *et al.*, "SymPy: symbolic computing in Python," *PeerJ Computer Science*, vol. 3, p. e103, 2017, doi: 10.7717/peerj-cs.103.

[18] M. Bardi, M. Falcone, and P. Soravia, "Numerical Methods for Pursuit-Evasion Games via Viscosity Solutions," in *Stochastic and Differential Games: Theory and Numerical Methods*, vol. 4, M. Bardi, T. E. S. Raghavan, and T. Parthasarathy, Eds., in Annals of the International Society of Dynamic Games, vol. 4., Boston: Birkhäuser, 1999, pp. 105–175. doi: 10.1007/978-1-4612-1592-9_3.

[19] P. Bernhard, "Singular Surfaces in Differential Games: An Introduction," in *Differential Games and Applications*, vol. 3, P. Hagedorn, H. W. Knobloch, and G. J. Olsder, Eds., in Lecture Notes in Control and Information Sciences, vol. 3., Springer, 1977, pp. 1–33.

[20] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[21] M. Zargham, "hc-marimo: source materials for SciPy 2026 — mathematics education with scientific Python." [Online]. Available: https://mzargham.github.io/hc-marimo/

[22] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[23] A. Agrawal and M. Scolnick, "marimo: an open-source reactive notebook for Python." Zenodo, 2023. doi: 10.5281/zenodo.12735329.

[24] C. Kadie, "Explore Solvable and Unsolvable Equations with SymPy," in *Proceedings of the 24th Python in Science Conference*, in Proceedings of the Python in Science Conference. 2025, pp. 38–53. doi: 10.25080/kcht7476.

[25] S. S. Kumkov, S. Le Ménec, and V. S. Patsko, "Zero-Sum Pursuit-Evasion Differential Games with Many Objects: Survey of Publications," *Dynamic Games and Applications*, vol. 7, no. 4, pp. 609–633, 2017, doi: 10.1007/s13235-016-0209-z.

[26] M. Zargham, "ADCS lifecycle demo: requirements traceability with structural, symbolic, and numeric evidence records." [Online]. Available: https://dynamicalsystemsgroup.github.io/ADCS-lifecycle-demo/

[27] R. Mehta, "GDS-Axelrod: an interactive evolution-of-trust showcase built on the GDS compositional game-theory ecosystem." [Online]. Available: https://dynamicalsystemsgroup.github.io/gds-axelrod/

[28] M. Zargham and J. Shorish, "Generalized Dynamical Systems Part I: Foundations," Vienna, Austria, Working Paper, 2022. doi: 10.57938/e8d456ea-d975-4111-ac41-052ce73cb0cc.

[29] M. Zargham and I. Ben-Meir, "Protocols and Institutions," in *Cryptoeconomic Theory (Advances in Science and Technology in the Age of AI)*, World Scientific, 2026, ch. 7, pp. 157–179. doi: 10.1142/9781800618664_0008.

[30] W3C OWL Working Group, "OWL 2 Web Ontology Language Document Overview (Second Edition)," W3C Recommendation, 2012. [Online]. Available: https://www.w3.org/TR/owl2-overview/

[31] M. Zargham, "Formalizing Document Assurance: A Topological Framework for Verification, Validation, and Human Accountability," in *Proceedings of the INCOSE International Symposium 2026*, 2026.

Open Access | https://doi.org/10.25080/ymuf5798

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Reno: Simplifying Application of Bayesian Inference to System Dynamics

Nathan Martindale[1], **Jordan Stomps**[1], and **Birdy Phathanapirom**[1]

[1]Oak Ridge National Laboratory

Abstract

Modeling and simulation enable iterative testing and updating of hypotheses, while encoding subject matter expertise into reusable tools. While the Python community has a variety of libraries for modeling, few exist for system dynamics, a paradigm for top-down analysis of material and information flows over time. Reno is an open-source package combining creation, visualization, and analysis of system dynamics models with techniques for Bayesian inference through integration with PyMC, supporting probability distributions in system variables and MCMC sampling to produce posterior distributions based on observed values. This approach enables simulation and refinement of time series models where variables, policies, or knowledge are uncertain, and data/observations are sparse.

Keywords System dynamics, Bayesian statistics

1. Introduction

Throughout the history of computer science, computers have been used to model aspects of the world in a diverse range of fields, from physics [1] and biology [2] to economics [3] and sociology [4]. Studying a phenomenon via computation-based tools allows exploration of hypotheses in a simplified environment, which is often necessary when real-world data are expensive or difficult to obtain. Computational simulations can be used as a starting point for identifying opportunities for further research (e.g., what real-world data might be worth collecting), helping to inform policy decisions, or even standing independently as valuable research contributions. Furthermore, computational models represent a mathematical interpretation or understanding of the world, encoding knowledge, ideas, and prior research from subject matter experts.

Although many techniques for conducting modeling and simulation exist, this work focuses on two specific approaches and how they can be combined: system dynamics and Bayesian statistics. To this end, we present Reno[1], an open-source Python library that implements an API for creating system dynamics models that can be wrapped in a Bayesian statistical model with PyMC [5] for updating uncertain parameters based on observed data.

1.1. System Dynamics Modeling

System dynamics modeling (SDM) is a way to represent how material and information move through and accumulate within complex systems. These systems are characterized by stocks (states where material or information accumulates) and flows (equations that define how those accumulations increase or decrease over time) and often exhibit nonlinear behavior arising from feedback loops and time delays [6]. SDM explores top-down impacts to system behavior, as opposed to a paradigm like agent-based modeling which explores emergent phenomena in a system from a bottom-up encoding of small-scale interactions

[1]https://github.com/ornl/reno

Published Jul 13, 2026

Correspondence to
Nathan Martindale
martindalena@ornl.gov

Open Access

Copyright © 2026 Martindale *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

[7]. SDM was first developed by Jay Forrester in 1956 to understand how corporate structure impacted employment oscillations at a General Electric plant [6], [8]. The approach has since been applied in many different domains, particularly within supply chain management and industrial production [9], [10], [11] as well as fields such as natural resource management [12], [13] and interactions between social and ecological systems [7].

For the purposes of a simple example to illustrate how stocks and flows interact in a system, consider a bathtub. The tub itself represents a stock, measuring the amount of water that has accumulated over time. The faucet and drain would be represented as an inflow and outflow to the tub respectively, influencing the rates at which the water level changes (Figure 1). Nonlinearities are introduced with concepts like feedback loops, such as a water level sensor that connects to the faucet and automatically increases or decreases the flow rate to maintain a certain water level in a leaky tub. Time delays are another common source of complexity, where the transmission of values from specific variables/flows may lag many timesteps behind. To overextend the tub analogy, one could imagine if the water level sensor is a poorly designed, wifi-enabled, Internet of Things (IoT) device with a sensor reading that is always several minutes behind, then it leads to an oscillating faucet rate as it cyclically undershoots and overshoots the target level.

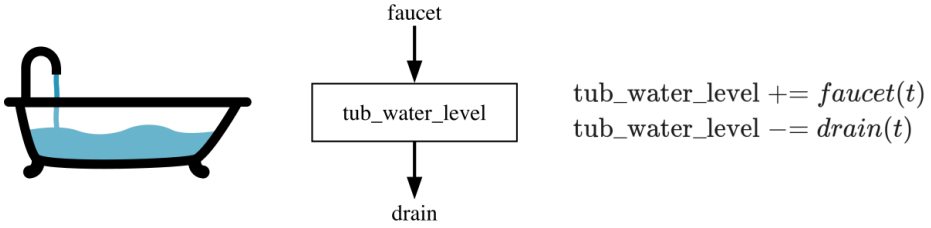


Figure 1. *Basic tub system. The* `tub_water_level` *rectangle represents the stock, and the* `faucet` *and* `drain` *arrows represent inflow/outflow. The stock value at each simulated timestep is increased/decreased by the rates of the inflows/outflows at time* `t`*.*

A more traditional example, the Lotka-Volterra equations or "predator-prey" model, shows two interacting populations of animals, a prey and predator group. The model exhibits several feedback loops, for instance the exponential growth between a population and the population's birth rate (i.e., as the population grows, birth rate grows, causing the population to grow even faster). Interaction between the two populations occurs in the predation loop, shown in the middle of Figure 2, where prey sustains the predator population but decreases the prey population, thus preventing unbounded growth from either population's positive feedback loop. Causal loop diagrams like the one shown only highlight general interactions in a system, but the specific values used in variables associated with these concepts impact the overall behavior.

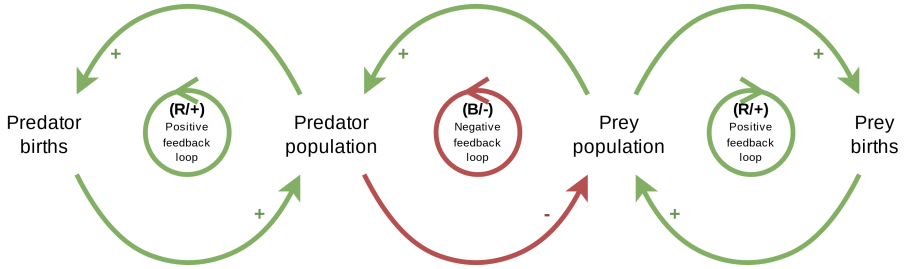


Figure 2. *An example causal loop diagram (commonly used to highlight feedback loops) of interactions between two populations. "R" (reinforcing), or positive feedback loops increase exponentially; "B" (balancing), or negative feedback loops, oscillate or push towards homeostasis.*

1.2. Bayesian Statistical Modeling

Bayesian inference is an approach to statistical modeling in which unknown quantities such as model parameters are represented with probability distributions. These distributions are then updated and refined based on observed data. This approach promotes an intuitive way of considering uncertainty and can produce meaningful information even with sparse input data. The Bayesian statistics interpretation of uncertainty is often contrasted with the frequentist interpretation: a useful distinction drawn from [14] is that the Bayesian approach considers the uncertainty of hypotheses (the model and its parameters) rather than the uncertainty of the underlying data like in the frequentist approach. This aligns well with SDM, in which different structures and parameterizations function as different hypotheses. For models with two possible pathways or structures (achievable by switching flows between different subsections of a model through the use of piecewise equations based on a bernoulli distribution), applying Bayesian inference can update probability distributions for whichever underlying model structure is more likely based on a set of observed data.

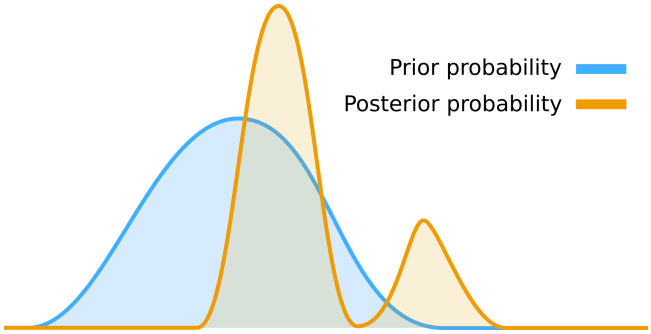


Figure 3. *Example probability distributions before and after Bayesian inference. These probability distributions could represent what the likely value of a particular parameter is. The prior shows a wider, "uncertain" value, whereas the posterior (the updated distribution after trying to fit some data) shows tighter probability mass around two specific likely values.*

This update process is theoretically underpinned by Bayes' theorem, shown in Equation (1), a formula that computes a posterior probability (the probability of a hypothesis conditioned on some data) based on a combination of the prior probability and the probability of observing that specific data. In practice, using Bayes' theorem directly is often computationally infeasible, so sampling-based approximations are typically used instead [14]. These approximations come from Markov Chain Monte Carlo (MCMC) algorithms [15], or methods of sampling from a probability distribution. MCMC sampling techniques were first developed in the 1940's, but began to see use in Bayesian approaches starting in the 1990's [16].

$$P(\text{hypothesis}|\ \text{data}) = \frac{P(\text{data}|\ \text{hypothesis})P(\text{hypothesis})}{P(\text{data})} \tag{1}$$

1.3. Combining SDM and Bayes

When constructing a system dynamics model, some parameter values may be unknown or uncertain. A phenomenon could be modeled by several different SDM structures or pathways. Because each has ranges of possible values for input parameters, the complexity of the phenomenon may mean there is no obvious appropriate structure for closely modeling real-world data. Therefore, applying Bayesian statistics in these situations allows the use of probability distributions which represent prior knowledge and uncertainty of unknown parameters. If provided some piece of real-world data about an observable part of the system, Bayesian inference can approximate the posterior distributions of those parameters and subsequently calibrate the model to the observed data.

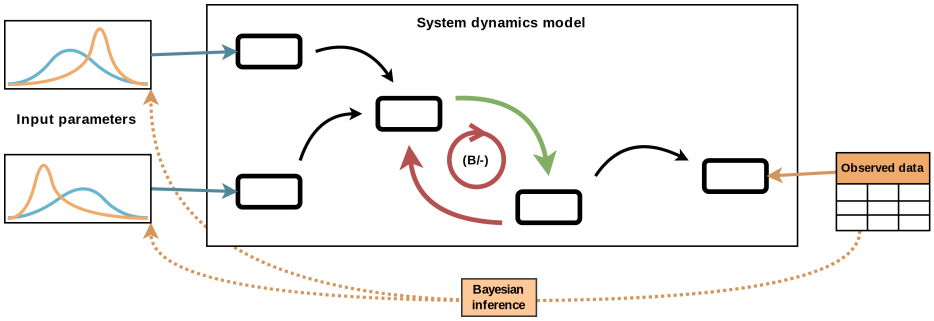


Figure 4. *Input parameters to a system influence some other component within the system. Providing observed data points for a piece within the system allows the use of Bayesian inference to learn/reduce uncertainty in the input parameters needed to produce the observed data.*

Reno seeks to implement an efficient workflow that combines these two modeling paradigms. The combination of SDM with Bayesian inference in and of itself is not novel: the process of estimating parameter distributions based on data is supported in the proprietary Vensim software, and has been used in literature for calibrating with sparse data [17] as well as forecasting with uncertainty from a base model calibrated on real data [18]. As such, Reno's contribution is in supporting within the Python open-source ecosystem the full process of developing, visualizing, and debugging system dynamics models, as well as simulation that incorporates Bayesian inference.

2. Existing tools

A variety of tools and libraries exist for system dynamics, many of which are proprietary. Some well-known solutions in industry and literature include AnyLogic[2], Vensim[3], and Powersim[4]. Other software ecosystems include open-source libraries and tools, such as the web-based/JavaScript community. Insight Maker [19] is an excellent example that includes both a back-end API to create models and graphical front-end that intuitively supports creating and connecting the various SDM components and defining subsequent equations. Along with SDM, Insight Maker supports agent-based modeling, although it has no direct mechanisms for conducting Bayesian inference. StatSim[5], another JavaScript front-end tool, also has direct support for both SDM and Bayesian inference applied to the system dynamics models.

Tools for SDM within the scientific Python community are currently limited: at the time of writing, there are only two notable libraries that support it. PySD [20], the more established of the two, is designed to run and modify SD models that have been written in other tools such as Vensim. However, it cannot build a model from scratch. PySD supports Bayesian inference through a manual integration with PyMC, whereby it treats the underlying system dynamics model as a black box function rather than translating the model into a PyMC equivalent. BPTK-Py (business prototyping toolkit)[6] is an open-source framework for creating SDM and ABM models, but does not support Bayesian inference. A comparison of these tools is listed in Table 1.

Table 1. *Comparison of a small selection of SDM tools. "Usage" refers to whether its main functionality is through a graphical user interface or a particular programming language API.*

Tool	Open-source	Usage	Create SD models	Bayesian inference

[2] https://www.anylogic.com/
[3] https://vensim.com/
[4] https://powersim.com/
[5] https://statsim.com/
[6] https://github.com/transentis/bptk_py

Vensim	No	Graphical	Yes	Yes
Insight maker	Yes	JavaScript/Graphical	Yes	No
StatSim	Yes	JavaScript/Graphical	Yes	Yes
PySD	Yes	Python	No	Yes
BPTK-Py	Yes	Python	Yes	No
Reno	Yes	Python	Yes	Yes

Reno was developed in part because implementation of these capabilities directly in Python allows for more effective integration into other libraries, tools, and frontends, such as Jupyter Notebooks, custom CLIs, or experiment management software. A Python API further lends itself to easier comprehension and modification via LLMs.

3. Using Reno

There are four basic aspects to the Reno API. These include model objects, the system dynamics components added to a model, the symbolic math system that is used to construct the equations of the components, and the collection of utilities for converting to PyMC models, visualization, and graphical explorer.

3.1. Creating a model

A model is constructed by initializing a `Model` instance and adding components to it, shown in Program 1. Components (e.g., stocks, flows, configurable variables, etc.) are added either by assigning directly to attributes on the model object, or by creating components within the model's context manager similarly to in PyMC.

```
import reno as r

my_model = r.Model()

# one way to add components is to directly assign them as attributes
my_model.variable1 = r.Variable()

# the other way (to avoid typing the model name over and over)
# is to use the model as a context manager. Any components defined
# inside of the context manager get added to the model when the
# context manager exits
with my_model:
  variable2 = r.Variable()
  flow1, flow2 = r.Flow(), r.Flow()
  my_stock = r.Stock()

my_model.my_stock  # components defined within the context manager are
           # available as attributes on the model
```

Program 1. *Basic example of creating a model and adding a couple variables (placeholder values that can be used in other equations), flows, and a stock.*

Equations are added to components by defining them within the constructor of the component, or by setting the `.eq` attribute, as shown in Program 2. The ability to set an equation outside of the component constructor allows certain kinds of circular references necessary for implementing feedback loops. Evaluating an equation that includes a reference to another component will subsequently evaluate the other component's equation (Figure 5). Circular references are allowed specifically for stock references, where using a stock's value always refers to the previous timestep's value which was already computed.

```python
import reno as r

tub = r.Model("Tub model")
with tub:
  faucet = r.Flow()
  drain = r.Flow()
  water_level = r.Stock()

  # equations can be specified in the constructor
  final_water_level = r.Metric(water_level.timeseries[-1])

  # or they can be defined after instantiation
  drain.eq = water_level / 2
  faucet.eq = 6

  # stocks are defined in terms of adding/subtracting flows
  # to make them inflows/outflows
  water_level += faucet
  water_level -= drain
```

Program 2. *Example bathtub model showing the two ways equations can be defined for different components. Stock equations are defined indirectly by adding/subtracting flows.*

An equation in Reno is fundamentally a tree data structure, where any node that has subtrees is an operation and leaf nodes are references to other equations (other stocks/flows/variables in the model) or values. An equation is evaluated by recursively evaluating through the tree down to the leaf nodes, and in turn returning produced values back up through the tree. Each operation (a set of classes included with Reno) has definitions for their evaluation based on its subtrees both in numpy [21] as well as how to construct the equivalent subtree in PyMC (Figure 5). Similar to PyTensor and numpy, all basic math operators are overloaded to simplify the resulting Python model code and contain the math to within Reno's symbolic math approach.

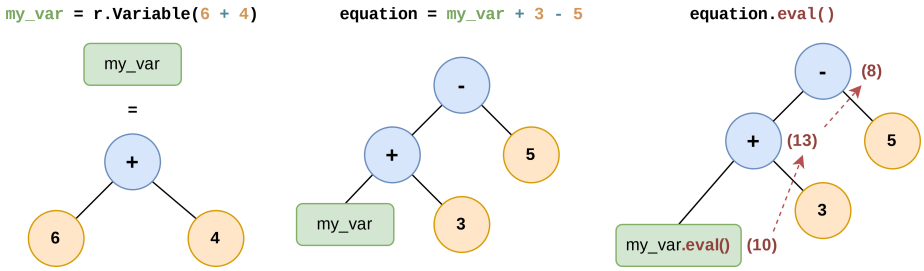


Figure 5. *An example of how an equation tree is structured and evaluated. Evaluation recurses through the tree, including other trees if there are references to other components, to the leaf nodes. Operation results propagate back up to the root.*

3.2. Running a model

A model, once defined, can be called as a function to run a simulation. This function call can include simulation-specific configuration parameters, such as the number of steps to run, and can also be used to configure any free variables in the system (i.e., variables not defined based on other variables). This allows quickly building up a collection of datasets from multiple different configurations. Results are returned in XArray datasets that contain the values of every component within the system indexed by timestep and sample number. These datasets can be passed into many of Reno's available visualization functions.

The following example in Program 3 implements the Lotka-Volterra equations and executes two different simulation runs with them. Each run is configured slightly differently:

```python
import reno as r
predator_prey = r.Model(name="predator_prey", steps=200, doc="Classic predator-prey interaction model
example")

with predator_prey:
    # make stocks to monitor the predator/prey populations over time
    rabbits = r.Stock(init=100.0)
    foxes = r.Stock(init=100.0)

    # free variables that can quickly be changed to influence equilibrium
    rabbit_growth_rate = r.Variable(.1, doc="Alpha")
    rabbit_death_rate = r.Variable(.001, doc="Beta")
    fox_death_rate = r.Variable(.1, doc="Gamma")
    fox_growth_rate = r.Variable(.001, doc="Delta")

    # flows that define how much the stocks change in a timestep
    rabbit_births = r.Flow(rabbit_growth_rate * rabbits)
    rabbit_deaths = r.Flow(rabbit_death_rate * rabbits * foxes, max=rabbits)
    fox_deaths = r.Flow(fox_death_rate * foxes, max=foxes)
    fox_births = r.Flow(fox_growth_rate * rabbits * foxes)

    # hook up inflows/outflows for stocks
    # note that `>>` is syntax sugar for the appropriate `+=`/`-=` stock operations
    rabbit_births >> rabbits >> rabbit_deaths
    fox_births >> foxes >> fox_deaths


run1 = predator_prey(rabbit_growth_rate=.06, rabbits_0=50.0)
run2 = predator_prey(rabbit_growth_rate=.04, rabbits_0=75.0)
```

Program 3. *Reno implementation of the predator-prey equations (labeled as foxes/rabbits.) Model parameter configuration can include initial values of stocks by suffixing with `_0`, such as `rabbits_0`.*

3.3. Visualizing a model

Reno includes several functions to help visualize and debug models. One function includes a stock and flow diagram, which visually lays out how the components connect to each other and highlights the main material/information flow(s) throughout the system (Figure 6).

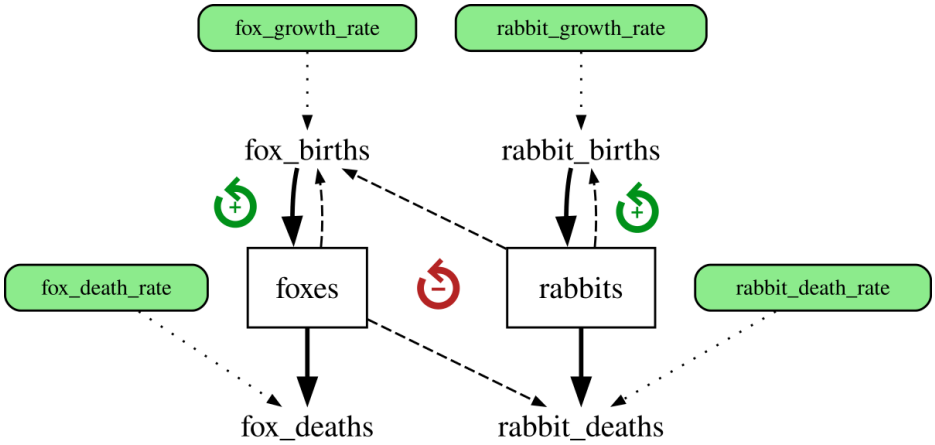


Figure 6. *The output from `predator_prey.graph()`, a graphviz stock and flow diagram. Rectangles represent stocks, labeled arrows are flows, and green rounded rectangles are variables. Feedback loop indicators were added manually to show how a stock and flow diagram relates to the causal loop diagram in Figure 2.*

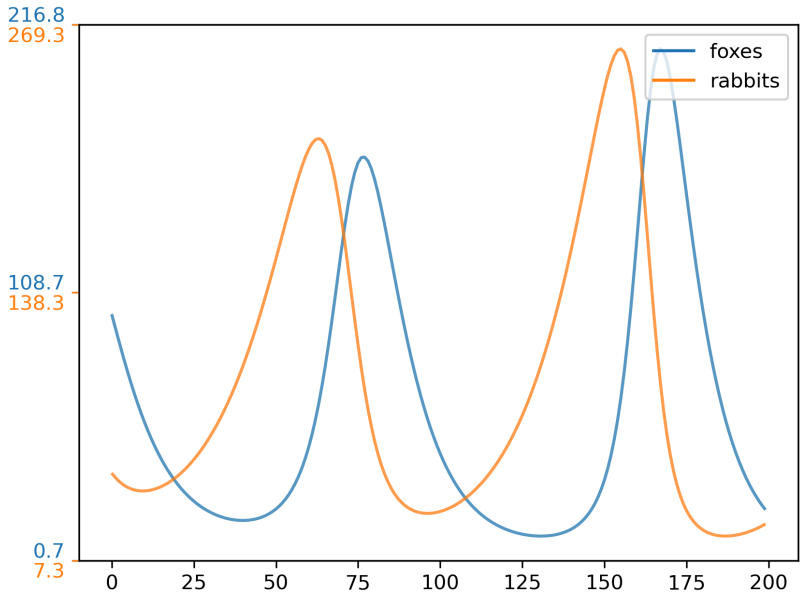


Figure 7. *The output from* `reno.plot_refs_single_axis(run1, [predator_prey.foxes, predator_prey.rabbits])`*, a utility function that helps visualize multiple model components on one set of axes. This is a common plot for system dynamics models because the relationship between two components may be more important than the relative scale differences. Note that each oscillation appears to have wider fluctuation: Reno simulates discrete timesteps, which means for a large* `dt` *timestep, errors can accumulate in some types of feedback loops.*

In large systems, it can be difficult to follow certain relationships from a collection of plots. Stock and flow diagrams can be configured to include inline sparklines of relevant stocks and flows to produce an output as shown in Figure 8.

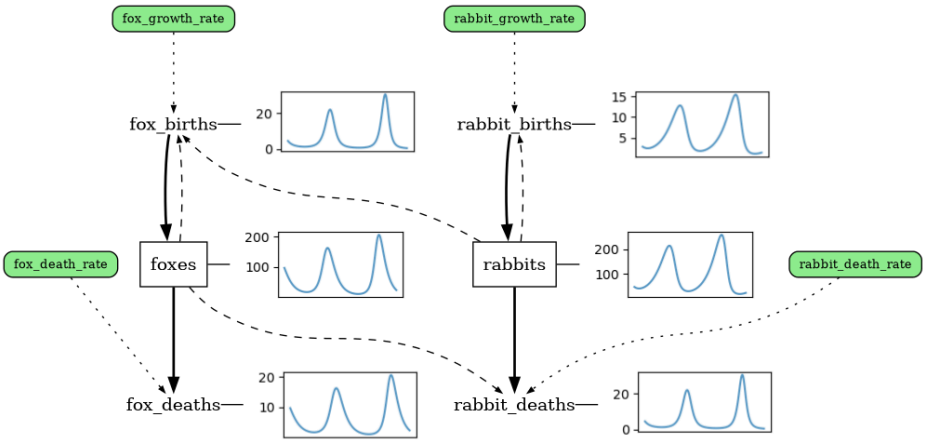
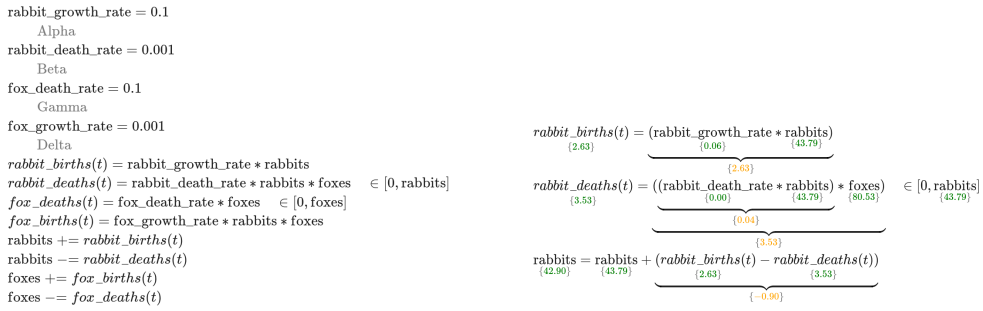


Figure 8. *Graphviz output from* `predator_prey.graph(stock_sparklines=True, flow_sparklines=True)`

Reno can output a latex representation of all of the equations involved in the system, which can be viewed natively within Jupyter or exported as a string for a latex paper. Additionally, an individual sample and timestep can be passed as parameters, which will then output the equations with the computed value from each section of each equation explicitly written out. This can be useful for diagnosing unexpected or incorrect equation results.



(a) (b)

Figure 9. *Latex outputs for a model. These render interactively in Jupyter [22] notebooks and can be exported as raw strings for inclusion in latex documents.*

3.4. Bayesian inference

To run Bayesian inference on a model, at least one model parameter must be defined with a prior probability distribution. The model can be converted and simulated within a PyMC model by calling the `.pymc()` function, which has a signature that mirrors the base simulation call. This means that it can take parameter configurations and settings to control the MCMC sampling process. The `.pymc()` function also takes observed data values and uncertainties by supplying `Observation` objects to the `observations` parameter, shown in Program 4. Reno turns these observations into gaussian likelihood functions, with mean values centered around the observed equation (connecting it to the system simulation), a user-specified standard deviation to allow for uncertainty in the observations (`1.0` in the example in Program 4), and the actual observed values for PyMC to target (`[100]` in the example). These likelihood functions are used by PyMC during its MCMC sampling process to approximate the posteriors. Passing an `n` to the `.pymc()` call configures the total number of samples produced by the sampler. By default, four chains are used, but this and all other PyMC sampler settings (such as the random seed) can be configured by passing a `sampling_kwargs` dictionary. The sampler used by default is the Sequential Monte Carlo[7], but the standard samplers can be used as well by passing `smc=False`.

```
run3 = predator_prey.pymc(
    n=2000,
    rabbit_growth_rate=.05,
    rabbits_0=75.0,
    observations=[r.Observation(predator_prey.foxes.maximum(), 1.0, [100])],
)
```

Program 4. *An example PyMC-enabled run, targeting an observed maximum fox population value of `100`, with a standard deviation (uncertainty allowance) of `1.0`.*

The output from a `.pymc` run is an Arviz `InferenceData`, a specialized XArray dataset that includes both a `.prior` section as well as `.posterior`.

3.5. A complete example run with Bayes

This section uses an example from the [23] textbook, specifically a “one-compartment model” or a model of the concentration of the drug Dilantin in the system under repeated dosages at a given interval. Parameters of this model include the elimination half-life of the medication in the blood plasma, the absorption fraction or how much of the ingested medication reaches the blood stream, and dosage amount and interval. In this hypothetical example, we use one or more noisy measurements of the concentration to demonstrate how variables we treat as uncertain (artificially in this case) can be calibrated or refined.

[7] https://www.pymc.io/projects/examples/en/latest/samplers/SMC2_gaussians.html

Program 5 shows the Reno model definition for the one-compartment model with variables configured for the "ground truth" default run.

```
import reno as r

compartment = r.Model("compartment", steps=168)
with compartment:
    # define the three main components, the amount of drug in the system
    # and the flows that increase/decrease it
    drug_in_system = r.Stock(doc="Mass of medication in blood serum.")
    ingested = r.Flow(doc="Pulsed inflow of medication when dosage is taken.")
    eliminated = r.Flow(doc="Rate of change of drug leaving system.")

    # hook up the flows to the stock
    ingested >> drug_in_system >> eliminated

    # define relevant variables
    absorption_fraction = r.Variable(.12)
    dosage = r.Variable(100 * 1000, doc="Dosage is 100 * 1000 micrograms")
    start = r.Variable(0, doc="Timestep of first dosage. (in hours)")
    interval = r.Variable(8, doc="Timesteps between each dosage. (in hours)")

    volume = r.Variable(3000, doc="Volume of blood serum, 3000 mL")
    concentration = r.Variable(drug_in_system / volume)
    half_life = r.Variable(22, doc="Half-life of medication. (in hours)")
    elimination_constant = r.Variable(-r.log(0.5) / half_life)

    # set the equations for the flows based on above variables
    eliminated.eq = elimination_constant * drug_in_system
    ingested.eq = absorption_fraction * dosage * r.repeated_pulse(start, interval)
```

Program 5. *Reno implementation of a one-compartment model for Dilantin.*

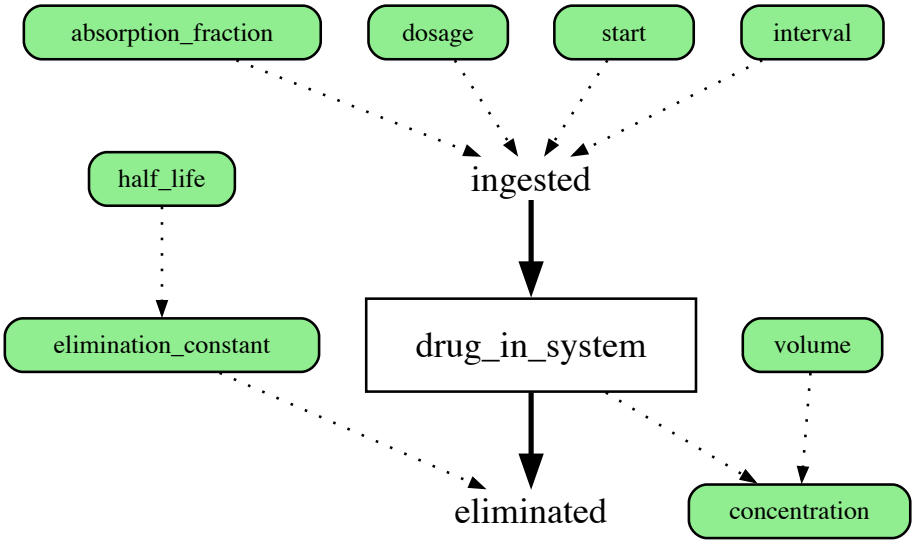


Figure 10. *The stock and flow diagram for the one-compartment model of Dilantin.*

We assume there are three uncertain variables: the first dosage time, the exact dosage interval, and the absorption fraction. We additionally assume we have collected measurements of the concentration at three different times. We retrieve "ground truth" data by running the model as is and obtaining those three values, shown offset by a small amount of noise as the black dots in Figure 11. Ground truth concentration measurements at timesteps 30, 100, and 150 are 9.66949, 15.5064, and 14.9687, respectively. In this example, we assume a lower resolution measurement to highlight the allowance for uncertainty in the data by using 9.5, 15.5, and 15.0.

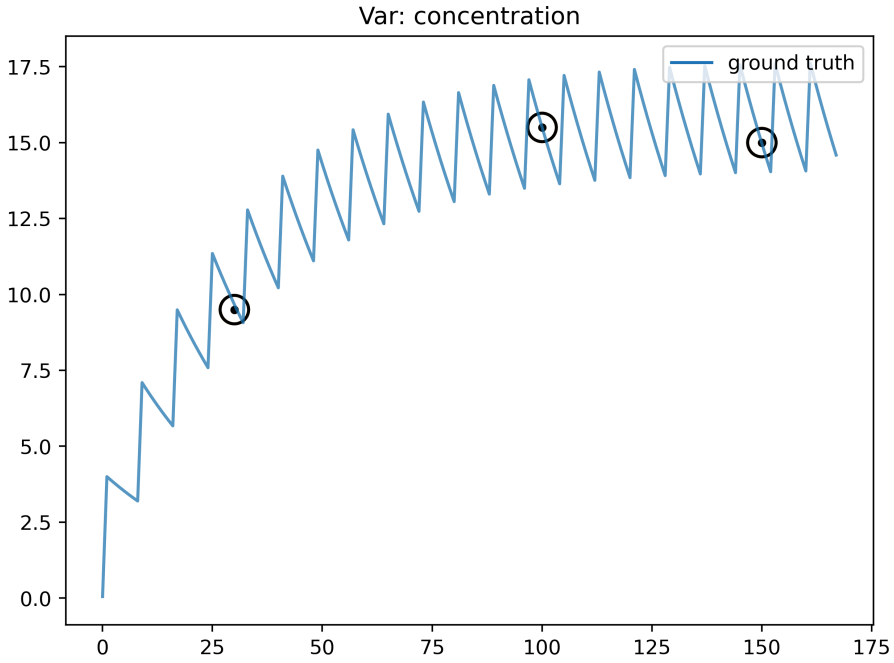


Figure 11. *Concentration timeseries values when running `compartment()`. The plot was created with `reno.plot_trace_refs(compartment, [compartment()], [compartment.concentration])`. The black points are added to show the measured data that will be used.*

When starting with an uncalibrated model, or if there is uncertainty about some parameters, we set prior distributions for those parameters. Here, we use discrete uniform distributions for the start time and interval and a normal distribution for the absorption fraction. If we were to simply simulate forward passes through the model, randomly drawing from these distributions would produce the results shown in Figure 12.

```
compartment.pymc(
  n=4000,
  start=r.DiscreteUniform(0, 1),
  interval=r.DiscreteUniform(7, 9),
  absorption_fraction=r.Normal(.15, .025),
  compute_prior_only=True,
    sampling_kwargs=dict(random_seed=13)
)
```

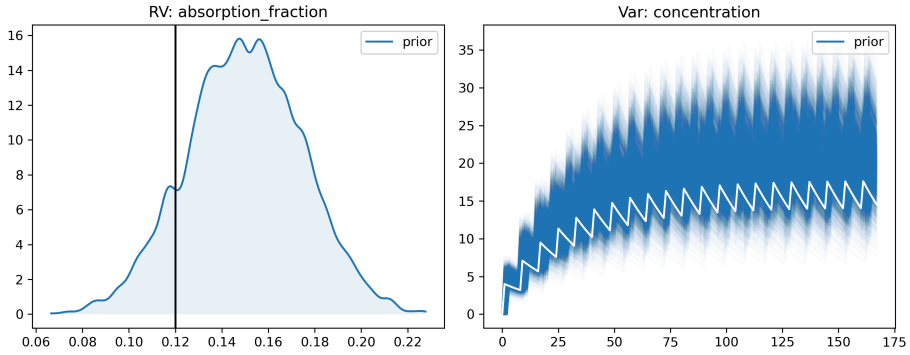


Figure 12. *Sampled distribution of the absorption fraction input parameter and simulated timeseries of the concentration variable. The original ground truth simulation run is highlighted in white and the target absorption fraction value is highlighted in black.*

When adding an observation for the concentration at one point in time, we can run the PyMC simulation and retrieve an approximated posterior distribution for the absorption fraction, highlighted in Figure 13. The probability mass for the single observation run starts converging to two different possible values: the correct one, and a different fraction for one of the other possible dosage intervals that would also result in the measured concentration value.

```
compartment.pymc(
  n=4000,
  start=r.DiscreteUniform(0, 1),
  interval=r.DiscreteUniform(7, 9),
  absorption_fraction=r.Normal(.15, .025),
  observations=[
    r.Observation(compartment.concentration.timeseries[100], 1, [15.5]),
  ],
    sampling_kwargs=dict(random_seed=13)
)
```

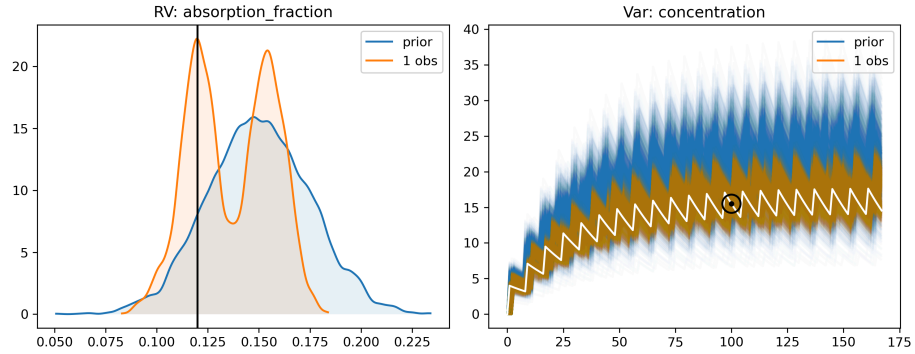


Figure 13. *Posterior distribution of the absorption fraction input parameter given one concentration timeseries observation (pinpointed in black). Ground truth is highlighted in black for absorption fraction and white for concentration.*

Supplying more data (the other two observed measurements) and rerunning the simulation results in what is shown in Figure 14, where the posterior probability distribution for the absorption_fraction is tightly concentrated around the ground truth value of 0.12.

```
compartment.pymc(
  n=4000,
  start=r.DiscreteUniform(0, 1),
  interval=r.DiscreteUniform(7, 9),
  absorption_fraction=r.Normal(.15, .025),
  observations=[
    r.Observation(compartment.concentration.timeseries[100], 1, [15.5]),
    r.Observation(compartment.concentration.timeseries[150], 1, [15.0]),
    r.Observation(compartment.concentration.timeseries[30], 1, [9.5]),
  ],
    sampling_kwargs=dict(random_seed=13)
)
```

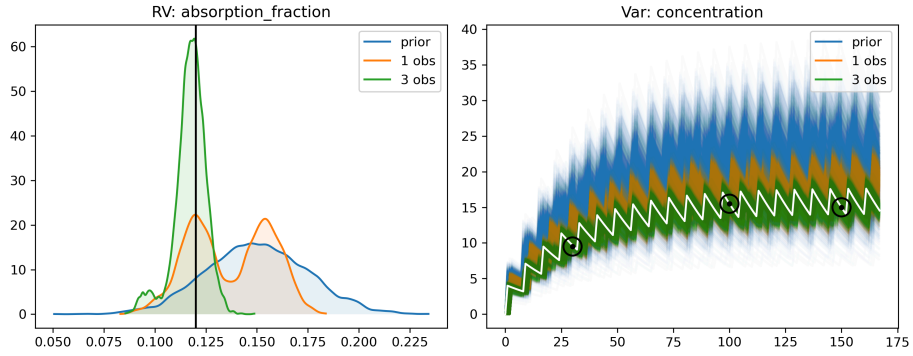


Figure 14. *Posterior distribution of the absorption fraction input parameter given three concentration timeseries observations (pinpointed in black). Ground truth is highlighted in black for absorption fraction and white for concentration.*

4. Technical implementation of PyMC integration

Due to the structure of the equation trees, converting individual equations into their PyMC equivalents is straightforward. Every `EquationPart` (the tree data structure class) has a `pt()` function which recursively calls throughout the entire tree, thus creating a PyTensor equation that mirrors the original. Much of the value that Reno provides is in the surrounding setup for the entire model, which can be complicated to achieve in PyMC because of the amount of boilerplate. A full model is simulated by running the various equations at each timestep in sequence, so translating this into PyTensor requires setting up the difference equations in a separate function and using their `scan`[8] operation.

One example of a challenging ability to support in raw PyMC is dynamically indexing into historical timeseries values inside of a `scan` function. By default, this target function only has access to the most recent ($t-1$) timestep for each value. PyMC supports passing in a specified set of previous timesteps, referred to as "taps", but the possibility of arbitrarily indexing them based on other variables in the system requires passing every previous timestep and separate equations to correctly convert the requested index, increasing the complexity of the code. Reno thus handles this infrastructure necessary to convert the model into PyMC, and provides an abstraction where only the components and the equations themselves need to be defined.

The overall flow of the conversion process is roughly implemented as follows:

1. Define the initial PyMC variables for each component of the Reno model and add the equations for their initial values. These definitions are appropriately ordered to account for any dependencies between the variables/initial equations.

[8] https://pytensor.readthedocs.io/en/latest/library/scan.html

2. All sequences are formatted as necessary and passed into PyMC's scan function, with the target step function defined with the following:
    1. All component values from the previous step, and any former steps if operations that look back in time are used, are pulled out of the parameters.
    2. Each component's equation is run (the equations are dependency ordered), stock values are updated, and the results of all of these are returned for handling by PyMC's scan function architecture.
3. All full timeseries sequences of every component are collected, and any final metric equations are run.

The conversion process results in python code with the following rough format:

```
def step_function(*args):
  # *args is populated by pytensor's `scan`, containing
  # previous timestep values, historical tap values where
  # necessary, and any additional sequence or static values
  s = args[0]
  ...

  # difference/update equations for each stock/flow/variable
  s_next = s + ...
  ...

  # return all updated values (pymc/pytensor has specific boilerplate to use)
    return [s_next, ...], ...

with pymc.Model() as model:
  # define all pymc distributions/variables and initial stock/flow values
  # e.g. for stock "s":
  s_init = pm.Deterministic("s_init", ...)
  ...

  # run the scan function to generate full sequence data for simulation
  [s_seq, ...] = pytensor.scan(fn=step_function, [s_init, ...], ...)

  # create pymc containers for the full sequence data (this way they get
  # added into any output arviz datasets from pymc sampling operations)
  s = pm.Deterministic("s", pt.concatenate([s_init, s_seq], ...)

  # any metric equations are added at the end
  final_metric = pm.Deterministic("final_metric", ...)
```

Program 6. *The roughly equivalent code that would construct a PyMC model in the same way as is returned from* `reno.pymc.to_pymc_model`*, or the string of raw code returned from* `reno.pymc.to_pymc_model_str`.

Reno can either directly set up a PyMC model using the above process, or output a raw string of Python code that would construct the equivalent PyMC model. This is valuable for situations where more complex PyMC-specific code is warranted and easier debugging.

5. Conclusion

Reno is an open-source Python library introduced to support building system dynamics models and applying Bayesian inference to the simulation process. The Python API is intended to support integration with additional software tooling, such as front-end interactive visualization capabilities. Future work will include the ability to directly integrate model context protocol (MCP) tools for language models, tooling to run within HPC environments, and improved web-based front-end tools.

Acknowledgements

This manuscript has been authored in part by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and

the publisher, by accepting the work for publication, acknowledges that the US government retains a non-exclusive, paid-up, irrevocable, world-wide license to publish or reproduce the submitted manuscript version of this work, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (http://energy.gov/downloads/doe-public-access-plan).

This work was supported by the NNSA Office of Defense Nuclear Nonproliferation Research & Development.

References

[1] B. J. Alder and T. E. Wainwright, "Studies in Molecular Dynamics. I. General Method," *The Journal of Chemical Physics*, vol. 31, no. 2, pp. 459–466, 1959, doi: 10.1063/1.1730376.

[2] P. Hogeweg, "The roots of bioinformatics in theoretical biology.," *PLoS computational biology*, vol. 7, no. 3, p. e1002021, 2011, doi: 10.1371/journal.pcbi.1002021.

[3] K. Judd, *Numerical Methods in Economics*. in Mit Press. MIT Press, 1998. [Online]. Available: https://books.google.com/books?id=9Wxk_z9HskAC

[4] M. W. Macy and R. Willer, "From Factors to Actors: Computational Sociology and Agent-Based Modeling," *Annual Review of Sociology*, vol. 28, no. Volume28, 2002, pp. 143–166, 2002, doi: https://doi.org/10.1146/annurev.soc.28.110601.141117.

[5] Abril-Pla O, "PyMC: a modern, and comprehensive probabilistic programming framework in Python," *PeerJ Computer Science*, 2023, doi: 10.7717/peerj-cs.1516.

[6] M. J. Radzicki, *System Dynamics and Its Contribution to Economics and Economic Modeling*. New York, NY: Springer US, 2020, pp. 401–415. doi: 10.1007/978-1-4939-8790-0_539.

[7] R. Martin and M. Schlüter, "combining system dynamics and agent-based modeling to analyze social-ecological interactions—an example from modeling restoration of a shallow lake," *Frontiers in Environmental Science*, 2015, doi: 10.3389/fenvs.2015.00066.

[8] D. C. Lane, "The power of the bond between cause and effect: Jay Wright Forrester and the field of system dynamics," *System Dynamics Review*, vol. 23, no. 2–3, pp. 95–118, 2007, doi: https://doi.org/10.1002/sdr.370.

[9] H. Akkermans and N. Dellaert, "The rediscovery of industrial dynamics: the contribution of system dynamics to supply chain management in a dynamic and fragmented world," *System Dynamics Review*, vol. 21, no. 3, pp. 173–186, 2005, doi: https://doi.org/10.1002/sdr.317.

[10] M. Danesh Shakib, "Using system dynamics to evaluate policies for industrial clusters development," *Computers & Industrial Engineering*, vol. 147, p. 106637, 2020, doi: https://doi.org/10.1016/j.cie.2020.106637.

[11] M. Özbayrak, T. C. Papadopoulou, and M. Akgun, "Systems dynamics modelling of a manufacturing supply chain system," *Simulation Modelling Practice and Theory*, vol. 15, no. 10, pp. 1338–1355, 2007, doi: https://doi.org/10.1016/j.simpat.2007.09.007.

[12] H. Husniah, N. Anggriani, and A. Supriatna, "System dynamics approach in managing complex biological resources," *ARPN Journal of Engineering and Applied Sciences*, vol. 10, pp. 1685–1690, 2015.

[13] A. H. P. Tan, E. H. Yap, and Y. A. Abakr, "A Complex Systems Analysis of the Water-Energy Nexus in Malaysia," *Systems*, vol. 8, no. 2, 2020, doi: 10.3390/systems8020019.

[14] I. Fornacon-Wood, H. Mistry, C. Johnson-Hart, C. Faivre-Finn, J. P. O'Connor, and G. J. Price, "Understanding the Differences Between Bayesian and Frequentist Statistics," *International Journal of Radiation Oncology*Biology*Physics*, vol. 112, no. 5, pp. 1076–1082, 2022, doi: https://doi.org/10.1016/j.ijrobp.2021.12.011.

[15] C. Robert and G. Casella, "A Short History of Markov Chain Monte Carlo: Subjective Recollections from Incomplete Data," *Statistical Science*, vol. 26, no. 1, pp. 102–115, 2011, doi: 10.1214/10-STS351.

[16] A. E. Gelfand and A. F. M. Smith, "Sampling-Based Approaches to Calculating Marginal Densities," *Journal of the American Statistical Association*, vol. 85, no. 410, pp. 398–409, 1990, doi: https://doi.org/10.2307/2289776.

[17] S. R. Penk *et al.*, "A system dynamics model to understand the integrated ecological and human dimension aspects of wildlife health and disease management," *bioRxiv*, 2025, doi: 10.1101/2025.09.22.677405.

[18] J. Andrade and J. Duggan, "A Bayesian approach to calibrate system dynamics models using Hamiltonian Monte Carlo," *System Dynamics Review*, vol. 37, no. 4, pp. 283–309, 2021, doi: https://doi.org/10.1002/sdr.1693.

[19] S. Fortmann-Roe, "Insight Maker: A general-purpose tool for web-based modeling & simulation," *Simulation Modelling Practice and Theory*, vol. 47, pp. 28–45, 2014, doi: https://doi.org/10.1016/j.simpat.2014.03.013.

[20] E. Martin-Martinez, R. Samsó, J. Houghton, and J. Solé, "PySD: System Dynamics Modeling in Python," *Journal of Open Source Software*, vol. 7, no. 78, p. 4329, 2022, doi: 10.21105/joss.04329.

[21] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[22] T. Kluyver *et al.*, "Jupyter Notebooks - a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Scmidt, Eds., Netherlands, 2016, pp. 87–90. [Online]. Available: https://eprints.soton.ac.uk/403913/

[23] A. B. Shiflet and G. W. Shiflet, *Introduction to computational science: modeling and simulation for the sciences*. Princeton University Press, 2014, pp. 48–50. [Online]. Available: https://assets.press.princeton.edu/chapters/s2_10291.pdf

Open Access | https://doi.org/10.25080/tukr4275

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Numerical Stability Pitfalls in Scientific Optimization Pipelines

Aayush Gauba[1]

[1]Student and Researcher

Abstract

Optimization algorithms are central to scientific machine-learning and parameter-estimation workflows. Yet even mature scientific Python workflows can hide numerical instability: an optimizer may satisfy its stopping criterion while returning parameters that are poorly conditioned, physically implausible, or highly sensitive to small perturbations. In a parameter-estimation workflow, for example, apparently converged coefficients may still change sharply after tiny perturbations to the data.

This paper examines stability pitfalls in scientific machine-learning and parameter-estimation workflows built around SciPy optimization routines. It analyzes how ill-conditioned objectives, poor parameter scaling, unstable finite-difference approximations, floating-point overflow, nearly singular Hessians, and initialization sensitivity can destabilize an otherwise routine optimization run. The emphasis is practical rather than algorithmic: the goal is to help researchers recognize failure modes that are not visible from a convergence flag alone.

Each failure mode is linked to concrete diagnostics, including condition-number analysis, gradient-norm monitoring, Hessian-spectrum inspection, perturbation tests, and convergence-log inspection. A controlled instability stress test on the HIGGS workflow further demonstrates how severe conditioning pathologies can induce optimizer-status reliability mismatches and catastrophic gradient instability. The paper then discusses mitigation strategies such as nondimensionalization, regularization, trust-region methods, analytic derivatives, parameter transformations, and multi-start validation.

Keywords numerical stability, scientific optimization, SciPy, conditioning, numerical auditing, HIGGS

Published Jul 13, 2026

Correspondence to
Aayush Gauba
gauba.aayush@gmail.com

Open Access

Copyright © 2026 Gauba. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Optimization is a routine component of scientific machine-learning and parameter-estimation workflows, including statistical inference, calibration, and data-fitting pipelines.

The SciPy ecosystem provides a broad collection of algorithms that have become standard tools in the scientific Python community. Functions such as `scipy.optimize.minimize`, `least_squares`, and `curve_fit` are frequently used in both research and industrial systems.

However, optimization routines can fail in subtle ways: a solution may appear mathematically valid while remaining highly sensitive to perturbations, physically meaningless, or numerically unstable. Common causes include ill-conditioned landscapes, floating-point limitations, singular Jacobians, unstable gradients, poor scaling, and erratic optimization dynamics.

This paper investigates these failures at the workflow level. Rather than proposing new algorithms, it studies how existing SciPy-based pipelines fail and how those failures can be diagnosed through residual scale, gradient norm, step history, condition number,

perturbation sensitivity, and consistency across algorithmic choices. The central thesis is that reliability often depends more strongly on conditioning, preprocessing, and numerical auditing than on optimizer choice alone.

The paper makes three practical contributions. First, it organizes common optimizer failures into a taxonomy of geometric, arithmetic, differential, and structural instability. Second, it presents a large-scale empirical study of SciPy optimizers on the University of California, Irvine (UCI) HIGGS dataset, emphasizing preprocessing, Hessian conditioning, and computational cost. Third, it synthesizes these observations into a workflow-oriented auditing protocol for researchers who use optimization inside larger scientific workflows.

The discussion complements classical numerical analysis texts and software documentation rather than replacing them. References such as J. Nocedal and S. J. Wright [1] and N. J. Higham [2] provide rigorous treatments of conditioning, floating-point arithmetic, and convergence theory. The present work addresses a different gap: the distance between algorithmic theory and the practical behavior of end-to-end scientific Python workflows. Standard references explain what can go wrong in principle, but they do not show how reliably these failures appear across randomized runs in SciPy pipelines or package the resulting checks as a staged auditing protocol for working researchers. The contribution here is therefore organizational and empirical rather than theoretical: it connects instability mechanisms to observable diagnostic signatures, quantifies their frequency under controlled stress conditions, and assembles the guidance into a reproducible protocol targeted at the scientific Python ecosystem.

2. Optimization in Scientific Computing

Before examining specific failure modes, it is useful to establish the computational setting in which they arise. Most scientific optimization pipelines can be viewed as numerical procedures for solving problems of the form:

$$\min_{x \in \mathbb{R}^n} f(x) \tag{1}$$

where $f(x)$ represents an objective function whose evaluation may itself involve simulation, statistical estimation, numerical integration, or data fitting. In practice, such objectives often originate from least-squares fitting, maximum likelihood estimation, energy minimization, residual minimization, or variational formulations.

These objectives are rarely solved in isolation. They are usually embedded in software workflows that choose an algorithm, evaluate the objective repeatedly, estimate derivatives, and decide when convergence has occurred. SciPy provides several interfaces for this process.

2.1. Common Optimization Interfaces

SciPy exposes these workflows through routines such as `minimize`, `least_squares`, `curve_fit`, `root`, `differential_evolution`, and `basinhopping`. These interfaces include methods such as BFGS, L-BFGS-B, Nelder-Mead, Powell, conjugate gradient, trust-region methods, and Levenberg-Marquardt. Table 1 summarizes several common warning signs and first-line responses.

Table 1. *Common numerical failure modes in optimization pipelines and practical first-line responses.*

Failure mode	Observable symptom	Typical response
Ill-conditioning	Narrow valleys, large condition numbers, or zig-zagging iterates	Rescale variables or regularize

Finite-difference noise	Erratic gradients	Supply analytic or complex-step derivatives
Overflow or underflow	Infinite, zero, or NaN objective values	Reformulate in log space
Nearly singular curvature	Flat directions, unstable Newton steps, or covariance blow-up	Use damping or trust regions
Initialization sensitivity	Different runs give different optima	Use multi-start analysis

3. Conditioning and Numerical Sensitivity

Conditioning measures how strongly a numerical problem amplifies perturbations. In optimization, it determines whether small errors in data, arithmetic, or intermediate calculations remain harmless or become large changes in the returned solution.

For a nonsingular matrix A, the normwise condition number is defined as [3], [4]:

$$\kappa(A) = \mid A \mid \cdot \mid A^{-1} \mid \tag{2}$$

Large condition numbers imply that small input perturbations can produce large solution changes. In optimization, this sensitivity often appears as distorted curvature, unreliable descent directions, or fitted parameters that shift substantially under tiny data changes.

3.1. Geometric Interpretation

Ill-conditioned optimization problems often contain narrow valleys or highly anisotropic curvature. Optimizer iterates may oscillate or converge extremely slowly.

The Rosenbrock function, $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$, is a compact toy example: it has a simple global minimum at $(1, 1)$, but its curved valley can still slow or destabilize optimization. It is used here only to illustrate geometry; the HIGGS experiment later in the paper provides the empirical evidence.

4. Ill-Conditioned Objective Functions

Ill-conditioned objectives can force an optimizer to spend many iterations correcting direction rather than making meaningful progress toward the minimizer. Small parameter perturbations then produce disproportionate changes in gradients or curvature, making the returned parameters fragile.

4.1. Characteristics of Ill-Conditioned Systems

Ill-conditioned optimization problems commonly exhibit:

- Slow convergence
- Erratic optimization dynamics
- High sensitivity to initialization
- Large curvature imbalance
- False convergence
- Oscillatory updates

4.2. Practical Consequences

These numerical symptoms matter because they directly affect scientific interpretation. In applications, ill-conditioning can produce:

- Unstable parameter estimation
- Non-physical fitted states
- Unstable downstream model evaluations
- Unreliable confidence intervals

4.3. Polynomial Basis Conditioning

Polynomial regression using Vandermonde feature construction provides a controlled example of geometric instability in scientific optimization workflows. As polynomial degree increases, the resulting design matrix becomes progressively ill-conditioned, amplifying sensitivity to perturbation and destabilizing least-squares estimation.

To quantify this effect, Vandermonde matrices were constructed across 100 random uniformly sampled grids on the interval $[0, 1]$, with 100 sample points in each grid. Table 2 reports the resulting condition-number statistics as polynomial degree increases.

The results demonstrate rapid instability amplification under increasing polynomial complexity. By degree 20, condition numbers frequently exceeded 10^{15}, entering regimes associated with catastrophic numerical sensitivity and effectively singular optimization geometry.

Table 2. *Condition-number growth in Vandermonde polynomial feature matrices across 100 random uniformly sampled grids on the interval* $[0, 1]$.

Polynomial Degree	Mean Condition Number	Std Condition Number	Median Condition Number
5	4.156×10^{3}	5.111×10^{2}	4.043×10^{3}
10	2.844×10^{7}	7.989×10^{6}	2.612×10^{7}
15	2.239×10^{11}	1.274×10^{11}	1.908×10^{11}
20	1.975×10^{15}	2.065×10^{15}	1.455×10^{15}

5. Parameter Scaling Problems

Poor parameter scaling often compounds ill-conditioning. Many algorithms behave best when parameter dimensions have comparable numerical scales, but scientific systems often contain parameters spanning many orders of magnitude.

Consider:

$$\theta = [10^{-12}, 10^{6}] \tag{3}$$

This imbalance destabilizes gradient-based optimization because a step size that is reasonable for one coordinate may be negligible or destructive for another.

5.1. Effects of Poor Scaling

Poor parameter scaling may cause:

- Vanishing updates
- Exploding optimization steps
- Numerical cancellation
- Loss of precision
- Extremely slow convergence

5.2. Search Geometry Distortion

Poor scaling distorts the geometry of optimization landscapes. Gradient-based methods may spend substantial effort compensating for coordinate scale rather than modeling the true curvature of the objective, while quasi-Newton approximations can become dominated by scale artifacts.

5.3. Mitigation Techniques

For this reason, scaling should be treated as part of the model formulation rather than as a cosmetic preprocessing step. Standardization is useful when parameters have empirical units and comparable statistical variation; nondimensionalization is preferable when physical scales are known; min-max normalization is useful for bounded parameters; logarithmic transformations stabilize positive parameters spanning orders of magnitude; and whitening can reduce correlations in locally linear least-squares problems. Bounds in methods such as L-BFGS-B should be chosen after scaling, because poorly scaled bounds can still distort the effective search geometry.

6. Finite Difference Instability

Even when the objective is well scaled, instability can enter through derivative estimation. Many SciPy routines estimate gradients numerically when analytic derivatives are not supplied.

Forward finite differences are commonly used:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{4}$$

Choosing a stable step size h is difficult because the approximation must balance two competing error sources [5]. SciPy finite-difference settings are often reasonable for well-scaled variables, but they can fail when parameters are extremely small, extremely large, or measured in incompatible units.

6.1. Roundoff Error

When h becomes extremely small, floating-point subtraction causes catastrophic cancellation. The two function values become nearly identical in floating-point representation, so their difference may contain mostly roundoff error rather than derivative information.

$$f(x+h) - f(x) \tag{5}$$

may lose significant numerical precision.

6.2. Truncation Error

When h becomes too large, the approximation no longer accurately represents the local derivative. The finite-difference quotient then reflects curvature over a broad interval rather than the slope at the point of interest.

Together, roundoff and truncation error show why default finite-difference settings should not be treated as universally reliable. Large step sizes reflect nonlocal curvature, while extremely small step sizes suffer from catastrophic cancellation and floating-point roundoff error; the stable operating region is problem- and scale-dependent. When the objective permits it, complex-step differentiation or automatic differentiation can provide more stable derivative information than real-valued finite differences.

7. Floating-Point Precision Limitations

Finite-difference errors are one manifestation of a more general constraint: optimization pipelines are fundamentally limited by finite precision arithmetic. Double precision floating-point values cannot represent arbitrary real numbers exactly, and repeated objective evaluations can accumulate or amplify these small representation errors.

7.1. Precision Considerations

Floating-point format can directly affect optimizer behavior. Reduced precision may erase small objective differences or gradient information, especially in narrow valleys or near flat residual surfaces. In such cases, convergence failure is not caused by the optimization algorithm alone; it is caused by insufficient numerical resolution in the objective evaluation.

The practical implication is that precision should be treated as an experimental parameter. Researchers should record the arithmetic precision used in objective evaluations and should be cautious when using reduced precision for optimization problems with narrow valleys, small residuals, or ill-conditioned curvature.

7.2. Overflow

Exponential functions provide a simple illustration: evaluating $\exp(1000)$ in double precision overflows, replacing a finite mathematical quantity with an infinite floating-point value. In optimization, this can erase meaningful search directions unless the objective is reformulated, for example in log space.

7.3. Underflow

Small probabilities may collapse to zero during likelihood optimization. Once this happens, log-likelihoods, gradients, or likelihood ratios can become undefined even though the underlying mathematical expression is finite.

7.4. Loss of Significance

Subtraction between nearly equal numbers causes catastrophic cancellation, which is especially damaging in residual computations, finite differences, and objective functions formed from differences of large quantities.

7.5. Stable Reformulations

Stable reformulations reduce these risks by changing the numerical representation without changing the mathematical quantity being computed. For example, the log-sum-exp trick improves stability:

$$\log \sum_i e^{x_i} = m + \log \sum_i e^{x_i - m} \tag{6}$$

where:

$$m = \max_i x_i \tag{7}$$

8. Singular Hessians and Curvature Instability

The same sensitivity appears in second-order information. Hessian matrices encode local curvature and are often used to construct Newton steps, covariance approximations, or uncertainty estimates.

The Hessian matrix is:

$$H_{ij} = [\nabla^2 f]_{ij} \tag{8}$$

When Hessians become singular or nearly singular, optimization becomes unstable because the local quadratic model no longer identifies a reliable direction of improvement. In least-squares problems, the analogous issue often appears through an ill-conditioned Jacobian or normal matrix $J^T J$, which can destabilize both parameter updates and uncertainty estimates in routines such as `least_squares`.

8.1. Consequences

Singular Hessians may produce:

- Newton step divergence
- Matrix inversion failure
- Unstable covariance estimates
- Flat optimization directions

8.2. Regularization

A common mitigation strategy is:

$$H_{\text{reg}} = H + \lambda I \tag{9}$$

where λ is a damping coefficient. This modification increases curvature in weakly identified directions and reduces the risk of taking excessively large Newton steps.

9. Sensitivity to Initialization

Curvature and scaling problems become especially important in nonconvex landscapes, where many optimization algorithms are highly sensitive to starting conditions. Different initializations may converge toward entirely different minima, even when the same algorithm and tolerances are used.

9.1. Nonconvex Optimization

Nonconvex landscapes contain:

- Local minima
- Saddle points
- Flat plateaus
- Chaotic gradient regions

9.2. Scientific Consequences

In physical simulations, sensitivity to initialization can lead to inconsistent parameter estimation across repeated experiments. A robust workflow should therefore report not only the best solution found, but also how that solution changes across plausible starting points, preprocessing choices, and optimizer configurations.

10. Existing Tooling and Diagnostics

Instability can arise from objective geometry, parameterization, arithmetic, derivative estimation, or initialization. The following diagnostics make these mechanisms visible during an optimization run rather than only after an unexpected result has already been accepted.

10.1. Condition Number Analysis

Tracking condition numbers can reveal instability early. In least-squares problems, the conditioning of the Jacobian or normal equations indicates whether the data identify the fitted parameters. A rapidly increasing condition number may signal redundant parameters, insufficient excitation in the data, or a poor parameterization.

10.2. Gradient Norm Monitoring

Gradient norms should generally decrease as the optimizer approaches a stationary point. Sudden spikes, long plateaus, or oscillations in the gradient norm may indicate finite-difference noise, poor scaling, overly aggressive line-search steps, or an objective function that is not smooth at the current numerical resolution.

10.3. Sensitivity Analysis

Perturbing inputs slightly and observing output variation helps evaluate robustness. Useful perturbations include changing the initial condition, adding small noise to data, modifying tolerances, and switching between comparable algorithms. If these changes alter the scientific conclusion, the optimization result should be treated as fragile.

10.4. Eigenvalue Analysis

The Hessian eigenvalue spectrum provides insight into optimization geometry. Large eigenvalue ratios indicate anisotropic curvature, near-zero eigenvalues indicate flat or weakly identifiable directions, and negative eigenvalues reveal that the current point is not a strict local minimum.

10.5. Convergence Logs

Convergence logs often reveal instability that scalar termination messages hide. Objective values, step sizes, residual norms, gradient norms, warnings, and parameter summaries can expose stagnation, cycling, or abrupt jumps that are difficult to infer from the final optimizer message alone. These diagnostics motivate the experimental summaries in the next section.

10.6. Relation to Existing Diagnostic Tooling

Many of these diagnostics are already supported by mature scientific software. SciPy optimization routines expose convergence messages, status codes, callback hooks, derivative options, Hessian or Hessian-vector interfaces for second-order methods, and trust-region algorithms whose behavior can be inspected through iteration counts, function evaluations, gradient norms, and termination messages [6]. These features provide low-level access to optimizer behavior.

Derivative and curvature tooling further expands this diagnostic ecosystem. Packages such as `numdifftools` can approximate gradients, Jacobians, and Hessians by finite differences, supporting derivative checks and sensitivity analysis when analytic derivatives are unavailable [7]. Automatic differentiation systems such as JAX can provide programmatic gradients, Jacobians, Hessians, and gradient-consistency checks when the objective is differentiable and written in a compatible computational model [8]. In machine-learning workflows, libraries such as Optax provide optimizer-state monitoring, adaptive gradient transformations, learning-rate schedules, and gradient clipping mechanisms that help control unstable updates [9].

The present work does not replace these tools. It organizes their mechanisms—derivative evaluation, callbacks, curvature inspection, trust-region control, and optimizer-state moni-

toring—into a workflow-level framework for identifying and reporting instability patterns across scientific optimization pipelines.

11. Large-Scale Optimization Reliability Study

The empirical study is not intended to establish universal optimizer rankings. Its purpose is to characterize optimization reliability under realistic conditioning and preprocessing regimes using large-scale scientific data. The central question is whether optimizer choice dominates reliability, or whether preprocessing and conditioning substantially reduce optimizer sensitivity.

11.1. UCI HIGGS Dataset

The main experiment uses the HIGGS dataset (UCI Machine Learning Repository dataset 280, DOI: D. Whiteson [10]) [10]. From the repository's `HIGGS.csv.gz` release, the study samples 250,000 of the 11,000,000 simulated events, each containing 28 real-valued physics features and a binary classification target. The optimization problem is logistic regression, solved through SciPy optimizers across 100 randomized train/test splits. The complete HIGGS pipeline and instability stress test are archived with the paper as a supplementary Jupyter notebook. Logistic regression was selected because it provides a smooth and interpretable optimization landscape whose conditioning behavior can be analyzed directly without introducing deep-learning-specific optimizer dynamics [11]. This setting combines a real scientific dataset, moderate-dimensional feature geometry, and a smooth objective whose Hessian conditioning can be measured directly.

The evaluated methods are BFGS, CG, L-BFGS-B, and Powell. Powell was retained because derivative-free optimization remains common in scientific workflows where stable analytic derivatives are unavailable. Each method is run with and without feature scaling so that optimizer behavior can be compared under different conditioning regimes. The reported metrics include success rate, test log loss, test accuracy, Hessian condition number, function evaluations, iterations, and runtime. These HIGGS results are the core empirical evidence in the paper; the earlier analytic examples serve only to explain mechanisms.

Table 3. *Optimization reliability on the UCI HIGGS dataset across 100 randomized train/test splits.*

Scaling	Method	Success Rate	Test LogLoss	Test Accuracy	Mean Hessian Condition	Mean Runtime (s)
Unscaled	BFGS	1.00	0.640348	0.634407	1712.31	4.09
Unscaled	CG	1.00	0.640350	0.634393	1712.28	9.47
Unscaled	L-BFGS-B	1.00	0.640347	0.634400	1712.31	4.69
Unscaled	Powell	1.00	0.646431	0.621087	1686.90	93.99
Scaled	BFGS	1.00	0.638777	0.639535	54.29	1.68
Scaled	CG	1.00	0.638777	0.639529	54.29	2.69
Scaled	L-BFGS-B	1.00	0.638777	0.639538	54.29	1.34
Scaled	Powell	1.00	0.638806	0.639465	54.30	58.27

Table 4. *Computational cost comparison across optimization methods on the HIGGS experiment.*

Scaling	Method	Mean Iterations	Function Evaluations	Runtime (s)
Unscaled	BFGS	99.48	101.43	4.09
Unscaled	CG	122.52	235.15	9.47

Unscaled	L-BFGS-B	65.98	75.56	4.69
Unscaled	Powell	18.66	5057.94	93.99
Scaled	BFGS	40.07	41.07	1.68
Scaled	CG	33.55	69.28	2.69
Scaled	L-BFGS-B	19.47	22.67	1.34
Scaled	Powell	11.10	3116.34	58.27

Table 5. *Impact of preprocessing and scaling on optimization reliability metrics.*

Metric	Unscaled Mean	Scaled Mean	Relative Improvement
Hessian Condition Number	1712.30	54.29	31.5× reduction
Mean Runtime	28.06	15.00	46.5% reduction
Mean Function Evaluations	1367.52	812.35	40.6% reduction
Mean Test Accuracy	0.6311	0.6395	Improved convergence consistency

11.2. Statistical Effect of Scaling

To evaluate whether the scaling improvements reflect consistent behavior rather than isolated averages, we compared paired scaled and unscaled runs across randomized train/test splits. For each optimizer, the scaled and unscaled configurations were matched by seed, allowing direct comparison of preprocessing effects on the same data partition.

Table 6 reports the relative change in Hessian conditioning, function evaluations, and runtime after scaling. The runtime and conditioning improvements after scaling were consistent across randomized splits and substantially exceeded the observed run-to-run variance. Paired Wilcoxon signed-rank tests comparing scaled and unscaled runtime distributions yielded $p < 0.001$ across all gradient-based methods.

Table 6. *Effect of feature scaling on Hessian conditioning, function evaluations, and runtime in the HIGGS optimization experiment.*

Method	Condition Reduction	Function Eval. Reduction	Runtime Reduction
BFGS	31.5 ×	59.5%	59.0%
CG	31.5 ×	70.5%	71.6%
L-BFGS-B	31.5 ×	70.0%	71.3%
Powell	31.0 ×	38.4%	38.0%

The reduction is especially pronounced for gradient-based methods. After scaling, BFGS, CG, and L-BFGS-B converge to nearly identical predictive performance, but require substantially fewer iterations and function evaluations. Powell is also stable after scaling, but remains substantially slower and slightly worse in test log loss and accuracy, underscoring the robustness-at-high-cost tradeoff even in this moderate-dimensional setting. Derivative-free methods demonstrated robustness advantages in certain low-dimensional instability regimes, but incurred substantially larger computational costs on higher-dimensional scientific optimization tasks.

11.3. Conditioning Dominates Optimizer Differences

After scaling, BFGS, CG, and L-BFGS-B produce nearly identical log loss, accuracy, and Hessian conditioning. Runtime and evaluation-count differences also shrink substantially. The Hessian condition number falls from approximately 1712 to approximately 54, a more

than order-of-magnitude improvement. In this experiment, the remaining method differences are best interpreted as robustness-efficiency tradeoffs rather than evidence for a universally superior optimizer.

11.4. Controlled Instability Stress Test

To evaluate whether the auditing framework can detect genuine instability rather than only mild conditioning effects, a controlled instability stress test was performed on the HIGGS optimization workflow. The goal was to intentionally induce numerical failure modes while preserving the underlying scientific dataset and optimization task.

The instability regime was constructed using three controlled perturbations. First, feature-scale imbalance was introduced using multiplicative scaling factors logarithmically spaced between 10^{-8} and 10^{8}. Second, near-collinear structure was induced by replacing selected features with linear combinations satisfying pairwise correlation coefficients exceeding 0.999999. Third, additive Gaussian perturbation noise with standard deviation 10^{-3} was injected into the feature matrix. These modifications were designed to create severe Hessian conditioning, unstable gradient geometry, and optimizer-status reliability mismatches. The resulting optimization behavior was then compared against a scaled mitigation pipeline using the same randomized train/test splits and optimizer configurations.

Practical success was defined using three simultaneous criteria: (1) successful optimizer termination, (2) finite Hessian conditioning below 10^{12}, and (3) gradient norm below 10^{-3}. The practical-success criteria were intentionally conservative and designed to identify early numerical degradation rather than only catastrophic optimizer failure.

Table 7. *Controlled instability stress test on the HIGGS optimization workflow. Practical success required successful optimizer termination, finite Hessian conditioning below 10^{12}, and gradient norm below 10^{-3}.*

Variant	Method	Optimizer Success	Practical Success	False Success	Median Hessian Condition	Mean Gradient Norm
Scaled	BFGS	1.00	1.00	0.00	5.1×10^{1}	1.7×10^{-5}
Scaled	CG	1.00	1.00	0.00	5.1×10^{1}	1.4×10^{-5}
Scaled	L-BFGS-B	1.00	1.00	0.00	5.1×10^{1}	1.3×10^{-5}
Scaled	Powell	1.00	0.00	1.00	5.0×10^{1}	3.3×10^{-3}
Unstable	BFGS	0.00	0.00	0.00	∞	4.9×10^{7}
Unstable	CG	0.00	0.00	0.00	∞	4.9×10^{7}
Unstable	L-BFGS-B	0.00	0.00	0.00	∞	4.8×10^{7}
Unstable	Powell	1.00	0.00	1.00	1.1×10^{38}	2.0×10^{7}

To evaluate instability emergence rather than only catastrophic failure, additional scale-imbalance ablations were performed across progressively more severe perturbation regimes. The severity ablation varied feature-scale imbalance while holding additive perturbation noise and near-collinear feature construction fixed at the same levels used in the full instability stress test. The results demonstrated a gradual transition from stable convergence to widespread practical failure as Hessian conditioning increased, indicating that the auditing framework can detect incipient instability before complete optimizer breakdown occurs.

Table 8. *Instability-emergence ablation across progressively severe scale-imbalance regimes in the HIGGS optimization workflow.*

Severity	Scale Range	Practical Success Rate	False Success Rate	Median Hessian Condition	Mean Gradient Norm
Moderate	10^{-2} to 10^{2}	0.333	0.000	4.704×10^{6}	1.67×10^{-2}
Severe	10^{-4} to 10^{4}	0.333	0.003	3.062×10^{10}	8.02×10^{0}
Extreme	10^{-8} to 10^{8}	0.000	0.187	1.053×10^{18}	1.864×10^{5}

The severity-ablation results demonstrate that instability emerges progressively rather than abruptly. As scale imbalance increased from moderate to extreme regimes, Hessian conditioning and gradient norms increased by multiple orders of magnitude while optimizer-status reliability deteriorated substantially. False-success rates also increased under severe perturbation, indicating that optimizer termination criteria became progressively less reliable as numerical conditioning worsened. These results suggest that workflow-level auditing can detect instability emergence prior to complete optimizer breakdown.

Notably, even moderate scale imbalance produced substantial practical failure rates despite relatively stable predictive performance. Inspection of individual runs showed that CG and L-BFGS-B frequently failed gradient-based practical-success criteria under moderate conditioning growth, suggesting that workflow-level auditing can detect incipient numerical degradation prior to catastrophic optimizer collapse.

The similar practical-success rates observed under moderate and severe perturbation were primarily driven by repeated failure of the same optimizer classes, particularly CG and L-BFGS-B, under increasingly ill-conditioned geometry, while BFGS retained practical success in the majority of moderate-severity runs. Although practical-success frequencies remained numerically similar across these regimes, gradient norms and Hessian conditioning deteriorated by multiple additional orders of magnitude under severe perturbation, indicating substantially greater underlying numerical instability.

False-success behavior emerged primarily in derivative-free optimization trajectories where optimizer termination criteria were satisfied despite persistent gradient instability. The substantial increase in false-success rate under extreme perturbation indicates that optimizer-status reliability deteriorates progressively as conditioning pathologies intensify.

An especially notable result is the behavior of Powell's method under the scaled regime. Although optimizer termination was consistently reported as successful, the method failed practical-success criteria because gradient norms remained above the operational stability threshold of 10^{-3}. In contrast, gradient-based methods converged to norms near 10^{-5}. Powell therefore terminated in regions that satisfied internal convergence conditions while remaining substantially farther from stable local stationary behavior. This indicates that derivative-free convergence criteria may not adequately reflect local numerical stability even after preprocessing mitigates Hessian conditioning.

The instability stress test demonstrates that optimizer termination messages alone are insufficient indicators of numerical reliability. Under induced instability, all gradient-based methods failed practical-success criteria and exhibited catastrophic Hessian conditioning together with gradient-norm explosion. Powell reported successful optimizer termination despite failing all practical-success criteria, producing a clear optimizer-status reliability mismatch.

After scaling and mitigation, Hessian conditioning returned to stable levels and gradient norms decreased by more than twelve orders of magnitude. The experiment therefore directly instantiates several instability categories from the earlier taxonomy, including geometric instability, differential instability, and structural instability.

11.5. Optimizer-Status Reliability Mismatches

A *false success* occurs when an optimizer reports successful convergence while remaining substantially distant from a known global optimum or physically meaningful solution. A *false failure* occurs when an optimizer reports unsuccessful termination despite satisfying objective-value or residual criteria. These status-reliability mismatches motivate reporting objective values, diagnostics, and sensitivity checks alongside optimizer messages. The instability stress test in Table 7 demonstrates that these mismatches occur empirically even in standard scientific machine-learning workflows when severe conditioning pathologies are introduced.

11.6. Scope and Limitations

The HIGGS experiment studies moderate-dimensional smooth optimization through logistic regression and does not capture the optimization geometry of deep neural networks or extremely high-dimensional nonconvex objectives. The conclusions should therefore be interpreted as workflow-level observations for classical scientific optimization pipelines rather than universal statements about all optimization systems.

12. Numerical Auditing Framework

The mitigation strategies can be organized as a staged protocol that makes existing diagnostics systematic, reproducible, and interpretable at the workflow level. Existing tools provide local measurements; the purpose here is to assemble them into a reliability audit for scientific optimization workflows.

Table 9. *Workflow-oriented numerical auditing protocol for scientific optimization pipelines.*

Audit Stage	Diagnostic Actions	Purpose
Pre-Optimization Audit	Feature scaling, condition estimation, derivative verification	Reduce instability before optimization
Runtime Audit	Gradient monitoring, convergence checks, Hessian inspection	Detect unstable optimization dynamics
Post-Optimization Audit	Sensitivity analysis, multi-start validation, residual inspection	Validate reliability of obtained solutions

Although instability thresholds are problem-dependent, the experiments in this work suggest several practical heuristics. In the HIGGS stress-test regime, Hessian condition numbers exceeding approximately 10^8 were consistently associated with severe optimizer degradation and gradient explosion. Stable optimization regimes typically converged to gradient norms near 10^{-5}, whereas unstable or practically unreliable runs remained above 10^{-3}. Large increases in function-evaluation count without corresponding gradient reduction also frequently preceded optimizer-status reliability mismatches.

12.1. Stage 1: Pre-Optimization Audit

Before running the optimizer, the model formulation should be checked for numerical risk. This stage includes scale analysis, nondimensionalization or normalization of parameters, inspection of parameter bounds, condition estimation for available Jacobians or linearized subproblems, and derivative verification using analytic, automatic-differentiation, complex-step, or finite-difference checks.

12.2. Stage 2: Runtime Audit

During optimization, the workflow should log objective values, step sizes, gradient norms, constraint violations, condition estimates, callback traces, Hessian or Hessian-vector diagnostics when available, optimizer status messages, iteration counts, function evaluations, and warning messages.

12.3. Stage 3: Post-Optimization Audit

After termination, the returned solution should be tested through perturbation sensitivity analysis, multi-start validation, comparison across algorithm families when feasible, residual and gradient checks, Hessian spectrum or rank inspection, and physical plausibility checks. A complete audit report should include both numerical diagnostics and domain-specific validity checks.

13. A Taxonomy of Numerical Stability Failures

The empirical results and mechanistic examples can be organized into four recurring failure types. The taxonomy is intended as a diagnostic map: it connects observed optimizer behavior to likely numerical causes and first-line mitigations.

Table 10. *Taxonomy of numerical stability failures in scientific optimization pipelines.*

Failure Type	**Cause**	**Observable Symptoms**	**Mitigation Strategy**
Geometric Instability	Ill-conditioning, curved valleys, anisotropic curvature	Slow convergence, oscillation, sensitivity to initialization	Feature scaling, preconditioning, trust-region methods
Arithmetic Instability	Overflow, underflow, catastrophic cancellation	NaNs, infinities, unstable finite differences	Stable formulations, clipping, higher precision
Differential Instability	Noisy gradients, inaccurate derivatives	Gradient oscillation, premature convergence, unstable updates	Autodiff, smoothing, gradient verification
Structural Instability	Singular Hessians, rank deficiency	Optimization stagnation, unstable inverse solutions	Regularization, pseudoinverse methods, dimensionality reduction

Geometric instability is most directly addressed by scaling, preconditioning, and trust-region control. Arithmetic instability calls for stable reformulation and attention to finite-precision effects. Differential instability requires more reliable derivative handling through analytic derivatives, automatic differentiation, smoothing, or derivative checks. Structural instability indicates weak identifiability or rank deficiency and often requires regularization, constraints, or dimensionality reduction.

14. Mitigation Strategies

The statistical experiments and taxonomy suggest that stability is not achieved by choosing a single “best” optimizer. The safeguards below map the failure taxonomy to practical interventions.

14.1. Normalization

Parameters should be scaled before optimization so that comparable changes in different coordinates produce comparable changes in the objective. In physical models, nondimensionalization is often preferable to purely statistical scaling because it preserves scientific interpretation while reducing numerical anisotropy.

14.2. Analytic Gradients

Analytic derivatives reduce finite-difference instability and often improve both accuracy and speed. When exact analytic gradients are unavailable, automatic differentiation, complex-step differentiation, or carefully validated finite-difference steps may provide more reliable alternatives than default numerical differencing.

14.3. Regularization

Regularization improves conditioning by penalizing unstable directions in parameter space. In inverse problems and curve fitting, ridge penalties, prior constraints, or damping terms can reduce variance and prevent the optimizer from exploiting noise in poorly identifiable directions.

14.4. Trust-Region Methods

Trust-region approaches improve stability under severe curvature imbalance by limiting each step to a region where the local model is expected to be reliable. This is especially useful when Newton or quasi-Newton steps point toward directions dominated by noisy curvature estimates.

14.5. Multiple Initializations

Optimization should be repeated from multiple starting points. Multi-start experiments help separate stable solutions from artifacts of initialization and provide practical evidence about the presence of local minima.

14.6. Constraint Enforcement

Physical constraints can stabilize optimization by excluding meaningless regions of parameter space. Bounds, positivity transformations, conservation laws, and monotonicity constraints can prevent overflow, singular model evaluations, and nonphysical fitted parameters.

15. Conclusion

Optimization reliability depends strongly on conditioning, preprocessing, and numerical auditing. Across the HIGGS experiments, feature scaling reduced the mean Hessian condition number from approximately 1712 to approximately 54, reduced function evaluations and runtime, and made the standard gradient-based optimizers behave nearly identically in predictive performance.

The controlled instability stress test further demonstrated that severe conditioning pathologies can produce optimizer-status reliability mismatches, gradient explosions, and effectively singular Hessian structure even on standard scientific datasets. These failures became detectable through workflow-level auditing metrics including condition estimates, gradient norms, and practical-success validation criteria.

These results support the paper's central claim: workflow-level numerical auditing may be more important than optimizer selection alone in many scientific computing pipelines.

Optimizer choice still matters, especially for computational cost and derivative-free scalability, but the strongest reliability gains in the large-scale experiment came from improving the numerical formulation before and around the optimizer.

References

[1] J. Nocedal and S. J. Wright, *Numerical Optimization*. Springer, 2006. doi: 10.1007/978-0-387-40065-5.

[2] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*. SIAM, 2002. doi: 10.1137/1.9780898718027.

[3] G. H. Golub and C. F. Van Loan, *Matrix Computations*. Johns Hopkins University Press, 2013. doi: 10.56021/9781421407944.

[4] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997. doi: 10.1137/1.9780898719574.

[5] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 2007.

[6] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[7] P. A. Brodtkorb and J. D'Errico, "Numdifftools: Solving Automatic Numerical Differentiation Problems in Python." [Online]. Available: https://github.com/pbrod/numdifftools

[8] J. Bradbury *et al.*, "JAX: Composable Transformations of Python+NumPy Programs." [Online]. Available: https://github.com/jax-ml/jax

[9] DeepMind, "Optax: Gradient Processing and Optimization in JAX." [Online]. Available: https://github.com/google-deepmind/optax

[10] D. Whiteson, "HIGGS [Dataset]." [Online]. Available: https://archive.ics.uci.edu/dataset/280/higgs

[11] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004. doi: 10.1017/CBO9780511804441.

Open Access | https://doi.org/10.25080/tefg5348

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

AutoFOAM: The Self-Refining Autonomous OpenFOAM Agent

Arun Govind Neelan[1], and **A Seshaditya**[2,3]

[1]SimuNetics, Kanniyakumari, Tamil Nadu, India, [2]Onnes Cryogenics, Hyderabad, India, [3]Quasi AI, Berlin, Germany

Abstract

Computational Fluid Dynamics (CFD) plays an important role in modern engineering, but using open-source solvers such as OpenFOAM requires considerable knowledge and skills, as well as time-consuming configuration file setup. To reduce the burden for such a technical background, we propose **AutoFOAM** – a self-evolving large language model (LLM) agent that creates, evaluates, runs, and evolves its own OpenFOAM simulations based solely on natural-language instructions. Our model is pre-trained on the Qwen-coder 2.5-14B, which is then fine-tuned on 252 text prompts targeting 7 OpenFOAM solvers, 13 parametrized mesh templates, and a y^+-target-aware parameter extraction and case configuration. The crucial element of the algorithm is a sophisticated evolution loop composed of 7 stages. To prevent model degeneration under repeated self-training, the agent employs three complementary anti-collapse streams: RAG-augmented retry context, surgical dictionary-level patching, and prompt-diversity paraphrasing. By bridging generative artificial intelligence with rigorous fluid simulations, **AutoFOAM** accelerates rapid prototyping and democratizes advanced CFD workflows.

Keywords Computational Fluid Dynamics, OpenFOAM, Large Language Models, autonomous agent, self-evolving AI

1. Introduction

Computational Fluid Dynamics (CFD) is a crucial field that enables design and analysis in the aerospace, automotive, environmental, and biomedical domains [1], [2]. Among the open-source tools available for CFD, OpenFOAM emerges as the best platform for simulations, with unmatched adaptability and insight into the underlying physics [3]. The downside is that OpenFOAM is highly complex, requiring experts to master numerous C++ dictionaries, each of which is syntactically inflexible and interconnected with others. For beginners, moving from an abstract physical idea to a concrete simulation can be tedious and error-prone.

However, recent developments in the field of Large Language Models (LLMs) indicate that LLMs have shown exceptional performance in generating code and performing logical reasoning tasks [4], [5]. Despite their superior capabilities, using a generic LLM off the shelf for CFD tasks typically yields unsatisfactory performance because they are not trained for this task. Generic LLMs tend to invent ungrounded OpenFOAM keywords, fail to ensure consistency across different boundary conditions, and lack sufficient understanding of the physical properties involved in CFD simulations. To surmount these challenges, we propose **AutoFOAM**, a completely autonomous and self-evolving LLM agent that can write, run, score, and optimize OpenFOAM cases solely from language input prompts. Instead of using a set of predetermined instructions or prompting closed-source models in a zero-shot setting, AutoFOAM utilizes a powerful self-refinement mechanism based on a 14 billion

Published Jul 13, 2026

Correspondence to
A Seshaditya
adi@onnes.in

Open Access

Copyright © 2026 Neelan & Seshaditya. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

parameter Qwen-2.5 Model [6], [7]. It is fine-tuned using a carefully selected data set of 252 prompts that systematically span seven OpenFOAM solvers, thirteen parametric meshes, and a y^+-sensitive numerical protocol.

The key contribution here is the implementation of the *seven-layer evolution loop*, which enables safe scaling of the agent's competence levels with minimal human interaction. The loop seamlessly blends real-time log parsing and solver correction with the most cutting-edge reinforcement learning methods, such as DPO for trajectory retry pairs [8]. To ensure the stability of the learning process in the long run, we propose three anti-collapse mechanisms: retrieval-augmented generation, retry context, surgical patching of dictionary-level errors, and prompt diversity paraphrasing. We can identify significant gains in solver family matching, average reward scores, and first-pass success rates in simulations.

The recent developments include the creation of FoamGPT [9], the retrieval-based architecture in OpenFOAMGPT [10], and the multi-agent approach in Foam-Agent 2.0 [11], among others. These examples showcase the growing momentum of using LLMs to automate computations in the physical sciences. The need for these autonomous computing models is evidenced by benchmark testing platforms. Most of the recent releases use proprietary models to make them more versatile. Because they use the most advanced AI models available in the market they may be able to handle new solver implementations but the present models can't do anything outside the specified geometry or solvers used for training. But the present work is based on local AI so that the data don't leave the system where we use AutoFOAM. It is able to handle the OpenFOAM simulation on relatively light weight models because of the harness agent which is built over the model. In addition, the present model has a self-improving layer which will help the model to improve the results over-time which is not available on other agents.

To sum up, the primary contributions in this work include:

- **Autonomous CFD Agent Framework**: **AutoFOAM** framework, which is designed to automatically generate syntactically sound and physically consistent OpenFOAM dictionaries from natural language commands.
- **The Seven-Step Evolution Loop**: This is our novel training framework, which uses self-correction, dataset curation, SFT, DPO with retry pairs, mixing anchors, active learning, and a regression gate to ensure continual monotonic performance improvement of our models.
- **Anti-collapse mechanisms for self-evolving CFD agents**: These consist of RAG-based context augmentation, dictionary patching, and paraphrasing strategies used to prevent model collapse in physical simulators using self-play.

2. System Architecture and Methodology

To properly manage the search space during synthesis, the agent architecture has been explicitly designed as a pipeline of eight stages (Figure 1). Only at the stages where decisions have to be made has the LLM been invoked – prompt refinement, parameter extraction, RAG, and retry generation. All other steps, which are completely deterministic, have been decided solely by rigorous logic.

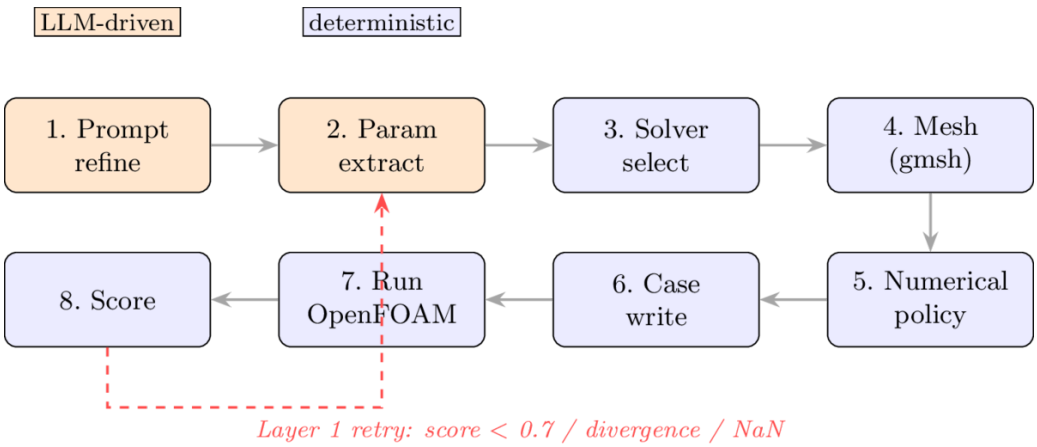


Figure 1. *Eight-stage agent pipeline. The LLM-driven stages (orange) manage semantic interpretation and context parsing, while the deterministic stages (blue) handle execution and physical validation.*

Constrained Parameter Extraction. The semantic reasoning process relies on vLLM [12], enabling high-throughput inference. In order to enforce a valid topology of the output structure and avoid structural hallucination, the parameter extraction module enforces `xgrammar` JSON schema constraints around generation [13]. As such, the generated output must conform to an established `CFDParams` structure, comprising geometric classification, Reynolds number (Re), fluid properties, flow regime, solver settings, and turbulence modeling approach. Additionally, physical consistency is enforced using a post-generation regular-expression validation procedure; if the LLM generates a characteristic velocity that is fundamentally inconsistent with the mathematical definition of $Re = U \cdot L_{\text{char}}/\nu$, then the boundaries are automatically overridden to ensure consistency. It can also recognize the specified y^+ value from the user prompt and generate an appropriate mesh accordingly. We did not implement any explicit y^+-aware mesh grading or generation, as achieving $y^+ = 1$ is challenging for most high-Reynolds-number flows.

Deterministic Solver Routing. Decoupling the solver choice process from the LLM solves a problem that had plagued early autonomous agents: the inclusion of geometric keywords biases the model toward selecting an unsuitable numerical regime (i.e., mistakenly choosing `simpleFoam` to solve a transient vortex shedding problem). In its place is a deterministic algorithm that leverages the physics flags (i.e., transient, compressible, thermal, multiphase) and the Reynolds number input by the system to isolate one of seven approved OpenFOAM solvers.

Multi-Objective Reward Formulation. The essential step in the self-improvement framework is the automatic score function $r \in [0, 1]$. This numeric reward quantifies the accuracy of the simulations based on various physical and numerical criteria, taking into account not only mathematical convergence but also mass conservation and boundary conditions (see Table 1). The base reward must be a strictly additive number until reaching the optimal value of 1.00. Additional heuristic penalties are applied when poor mesh quality ($>$ 70° of maximum non-orthogonality), computational stagnation (residual plateaus), or large execution time occur, with the total score capped at $[0, 1]$. Simulations that have reached an intermediate threshold score of $r \geq 0.5$ are selected to contribute to the retrying context, while simulations scoring $r \geq 0.65$ enter the training dataset directly.

Table 1. *Reward weight for successful completion of each step in the workflow.*

Component	Weight	Trigger
Convergence (full)	+0.40	solver wrote final-time fields, residuals fell

Convergence (partial)	+0.15	ran but did not converge
Residual magnitude $< 10^{-4}$	+0.20	worst-converging field
Residual magnitude $< 10^{-3}$	+0.10	worst-converging field
Residual trend quality	up to +0.10	log-rate of decrease, no late spikes
Mass conservation	+0.05	continuity error $< 10^{-3}$
Correct solver pick	+0.10	`solver = select_solver(params)`
Valid boundary conditions	+0.05	inlet *and* outlet patches present
Mesh quality penalty	-0.10	`checkMesh` non-orthogonality $> 70^{\circ}$
Slow runtime penalty	-0.10	wall-clock > 300 s
Stagnation penalty	-0.05	residual plateau detected

3. Dataset Curation and Trajectory Mining

To successfully transition general-purpose LLM reasoning into the highly specialized domain of computational fluid dynamics, AutoFOAM relies on a bi-data strategy: a statically curated foundational corpus to establish baseline syntax, and a dynamic log to power reinforcement-based learning.

3.1. Foundational Knowledge Corpus

The physical intuition and syntactic proficiency underlying the initial state of AutoFOAM are established using a well-curated dataset consisting of 402 rows. Each row consists of a multistep interaction involving the persona of an OpenFOAM expert. The machine learning model is trained to take a natural language query as input, produce a concise summary, and generate full Markdown fenced code blocks containing the necessary C++ dictionary file(s). The training data is generated by a process in which 252 unique prompts for various cases across all solver types and geometrical parameterizations have been formulated. Each prompt was then run on a prototype of the agent, and the trajectories achieving a successful execution score of at least $r \geq 0.5$ were saved.

3.2. Dynamic Trajectory Capture and Preference Mining

As part of the live autonomous runtime process, the AutoFOAM pipeline consistently collects data during execution. Each extremely successful run ($r \geq 0.65$) is added to a dynamic collection set of runs, creating an ever-growing dataset consisting of 210 de-duplicated topologies. At the same time, a full record of each failed or unsuccessful run, along with its particular failure type (continuity failures, inconsistent residuals) and the accompanying dictionary text, is kept. The failure ledger is the key factor in the functioning of Layer 4 of our self-evolution pipeline. This process automatically extracts preference pairs in which failures are contrasted with successful retries. Then, these preference pairs are leveraged by the Direct Preference Optimization (DPO) framework [8], and hence, AutoFOAM can learn the correct syntax and escape its previous hallucinations.

3.3. Out-of-Distribution (OOD) Benchmark

To properly test AutoFOAM's generalization and avoid overfitting to the synthetic training distribution, an evaluation set of 110 prompts was created in isolation from the training data. These prompts were written using novel syntax and unique vocabulary that intentionally avoid the semantic patterns found in the training dataset. This test is the true benchmark for evaluating the agent's generalization to previously unseen prompt formulations, geometric parameters, and operating conditions within the supported CFD domain.

Here, we did not include solver or geometry configurations that are unsupported by the present AutoFOAM implementation. Instead, the evaluation used prompts that were not included in the training dataset, with variations in linguistic formulation, geometric parameters, and operating conditions, while remaining within the supported solver families and geometry templates.

4. The Self-Evolution Pipeline

The basic limitation of using LLMs as physical agents is that they can fall victim to self-distillation collapse. Training a machine learning model on slightly erroneous data created by itself may quickly undermine its core functionality [14].

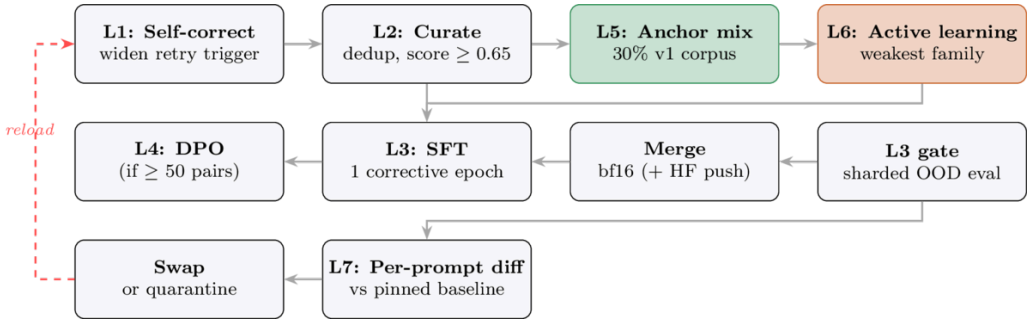


Figure 2. *The seven-layer evolution protocol. The green module represents foundational memory retention (L5), and the orange module signifies targeted active learning (L6). L7 functions as a strict regression firewall before promoting the new weights.*

The transition from inference to self-evolving AutoFOAM was facilitated by developing a seven-step self-evolution process. The entire procedure is completely automated and relies on harvesting success stories from production and using them to optimize preferences (Figure 2).

4.1. The Seven-Layer Evolution Architecture

The pipeline strictly regulates the flow of prompts into the model through the following protocols:

Layer 1 – Autonomous In-Run Correction. When a numerical issue is detected in the OpenFOAM simulation, such as NaN residuals or a large mass imbalance, the complete simulation log is not directly provided to the agent, as the large volume of output can exceed the model's context capacity. Instead, relevant diagnostic information is automatically extracted from the simulation output and supplied to the agent as additional context. Based on these diagnostics, the agent re-prompts the model to identify and correct the potential issue. This process is repeated iteratively until the simulation runs successfully or the predefined iteration limit is reached.

Layer 2 – Execution-Gated Curation. The agent evaluates the generated solution using the defined reward function. Solutions achieving a reward score of $r \geq 0.65$ are considered sufficiently reliable and are retained as reference examples for the RAG and self-improvement components. These high-quality examples can subsequently be retrieved as relevant references or incorporated into the agent's iterative improvement process.

Layer 3 – Supervised Fine-Tuning (SFT) and Evaluation Gate. The gathered data is then used for one epoch of fine-tuning using QLoRA. Right after this fine-tuning process, the proposed model undergoes an 8-way sharded OOD test relative to a performance baseline to guarantee macro-stability.

Layer 4 – Direct Preference Optimization (DPO). Leveraging the mined logs, this layer contrasts a localized generation failure (rejected) against its eventually successful, Layer-1-corrected counterpart (chosen). DPO is applied to this on-policy preference pair, explicitly teaching the model to penalize the syntactic formulations that previously led to solver divergence [8].

Layer 5 – Anchor Mixing. To defend against catastrophic forgetting of core OpenFOAM syntax, the pipeline systematically injects a 30% deterministic sample from the foundational training corpus into each dynamic retraining batch.

Layer 6 – Targeted Active Learning. The agent analyzes the data from the evaluation at Layer 3 and selects the most underperforming family of solvers. The agent then initiates a self-instruction to create new seeds for this domain, strengthening its own weakest link.

Layer 7 – Granular Regression Auditing. Prior to combining and deploying the weights of the newly trained adapter, the adapted model undergoes a detailed regression test against the fixed baseline model. The same evaluation prompts and test cases are provided to both models, and their generated outputs are compared to identify changes in response quality, correctness, and structure. This comparison is used to detect regressions, undesirable behavioral changes, or degradation introduced during the adaptation process. The newly trained adapter is retained only if it satisfies the predefined evaluation criteria relative to the baseline.

4.2. Surgical Recovery Protocols: Streams A and B

In cases where single-shot zero-shot generation does not result in a successful convergent configuration, AutoFOAM circumvents a costly complete reinitialization of the entire process through two recovery approaches. **Stream A (Context-Augmented Retrieval)** retrieves from a locally hosted Chroma vector database [15] that stores verified OpenFOAM tutorial configurations and adds contextually relevant dictionary code snippets to the prompt for recovery purposes. Concurrently, **Stream B (Dictionary Patching)** takes care of syntax and configuration-related errors. When a localized `FOAM FATAL` error (for instance, an incorrect scheme in `fvSchemes`) is observed, the agent skips the costly `gmsh` remeshing step and makes patches to the dictionary alone to facilitate an iterative solver feedback loop.

5. Experiments and Results

AutoFOAM is assessed along three main dimensions: the ability to automatically generate physical meshes and boundaries (qualitative), zero-shot generalization to new prompt languages (quantitative), and robustness to self-distillation collapse in the process of self-evolving its neural network architecture.

5.1. Autonomous Case Setup and Flow Validation

The AutoFOAM has built-in support for seven solver families native to OpenFOAM from 13 highly parameterized `gmsh` geometry models. Such geometries include both classical educational examples (e.g., driven cavity, backward step) and more advanced industrial cases (e.g., NACA airfoil, periodic hill, and 3D Ahmed body). To demonstrate the visual convergence and feasibility of computations, Figure 4 shows velocity magnitude contours computed completely automatically with only one sentence as input. In this example, without any manual involvement, AutoFOAM is able to correctly set `polyMesh` boundary names and create correct mesh structures (see Figure 3).

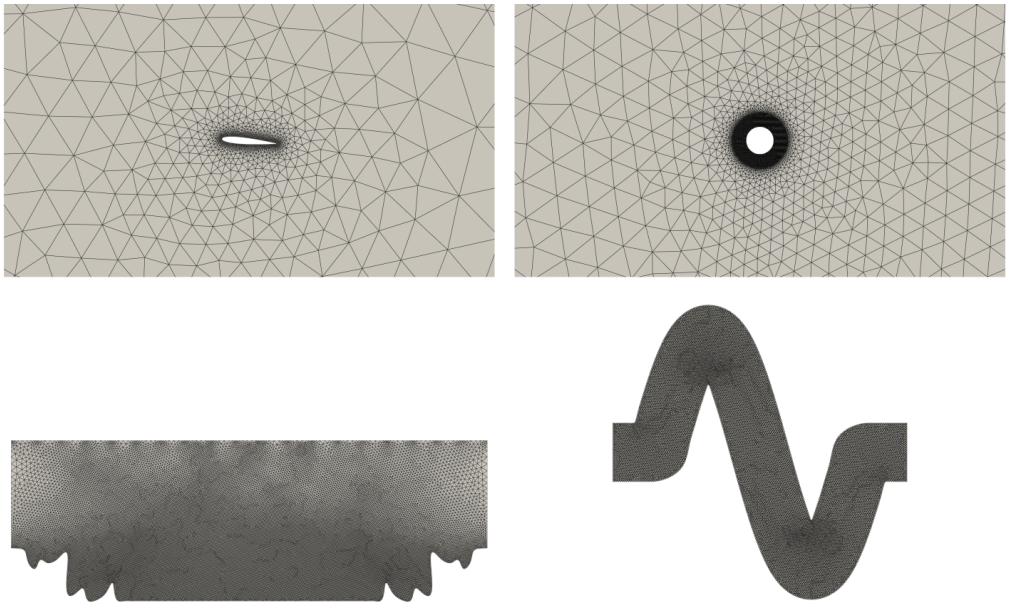


Figure 3. *Mesh generated by AutoFOAM*

Absolute performance of the self-contained pipeline was evaluated using an entirely held-out dataset of 110 out-of-distribution (OOD) prompt examples, run in an 8-way sharded cluster of NVIDIA H100 GPUs. Evaluation is limited only to solver classes and geometries available within AutoFOAM but varied geometric parameters and operational conditions for OOD generalization. Simultaneously, we evaluated AutoFOAM's ability to automatically identify the right solver class from the user-defined physics scenarios under the assumption that users have zero experience with selecting the appropriate solver class. *Please be aware that the precision of AutoFOAM will decrease when asking it to do something that does not relate to the solver or the geometry the current agent understands*. As summarized in Table 2, the fine-tuned agent managed to solve 100% of the test cases without any `FOAM FATAL` errors. When prompted with a solver that is not supported, the model is unable to generate valid results. Representative examples of such prompts are provided in Table 2.

The agent exhibited impressive physical reasoning skills, with a good match of 96.4% when using the deterministic solver routing. By examining manually the four unmatched cases (Table 3), one finds that these discrepancies arose only in mathematically ambiguous boundary regimes, namely the unsteady flow around cylinders at low Reynolds numbers, where the choice of the solver is `icoFoam` rather than the obligatory `pimpleFoam`. On the grounds of computational physics, the switch to `icoFoam` is still quite justified. We have also provided a few solver examples not supported by the present agent in Table 3.

Table 2. *OOD Evaluation Metrics.*

Metric	Value
End-to-end execution without FOAM FATAL	110/110 (100.0%)
Solver routing exact match	106/110 (96.4%)
Mean execution reward score (r)	0.64
Geometry templates successfully exercised	13 / 13
Solver families successfully exercised	7 / 7

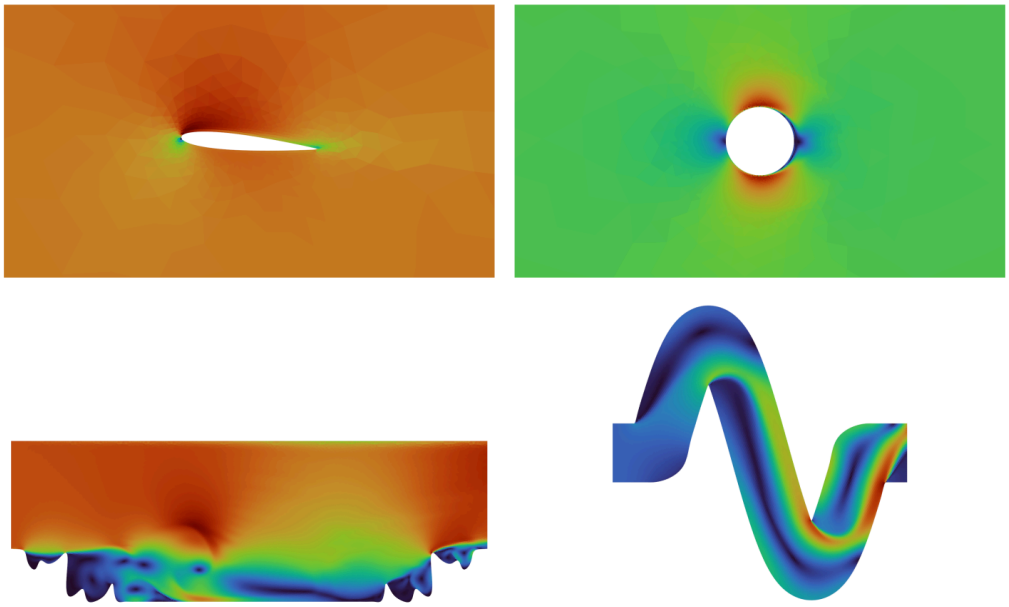


Figure 4. *Velocity contours generated by AutoFOAM*

Table 3. *Representative OOD and out-of-set prompts illustrating the agent's semantic understanding, solver selection, and current operational limitations.*

Benchmark Tag	Prompt	Predicted Solver	Match / Status
`cav_re150_water`	*"Cavity flow Re=150 in water, 0.4 m square"*	`simpleFoam`	✓
`pipe_water_lam`	*"Laminar pipe flow Re=500 diameter 0.02 m, water"*	`simpleFoam`	✓
`airfoil_re1m_aoa5`	*"NACA0012 airfoil at Re=10^6, 5° angle of attack"*	`simpleFoam`	✓
`inter_dam_tall`	*"Tall dam break, 2 m wide, 5 m water column collapse"*	`interFoam`	✓
`cdnoz_ma04`	*"Convergent-divergent nozzle with Mach 0.4 throat, air"*	`rhoSimpleFoam`	✓
`cyl_re250_unsteady`	*"Transient cylinder Re=250 in water, D=0.1 m"*	`icoFoam`	×
`outofset_step_turbine`	*"Simulate turbulent flow through a complex STEP-imported turbine passage with automatic topology fixing and unstructured mesh generation."*	Unsupported	Unsupported
`outofset_reacting_multiphase`	*"Simulate reacting multiphase combustion with detailed chemical kinetics and species transport."*	Unsupported	Unsupported

5.2. Physical Validation against Benchmark: Lid-Driven Cavity ($Re = 100$)

To address the physics-validation gap, we perform a quantitative benchmark of an AutoFOAM-generated case against the classical lid-driven cavity data of Ghia et al. [16].

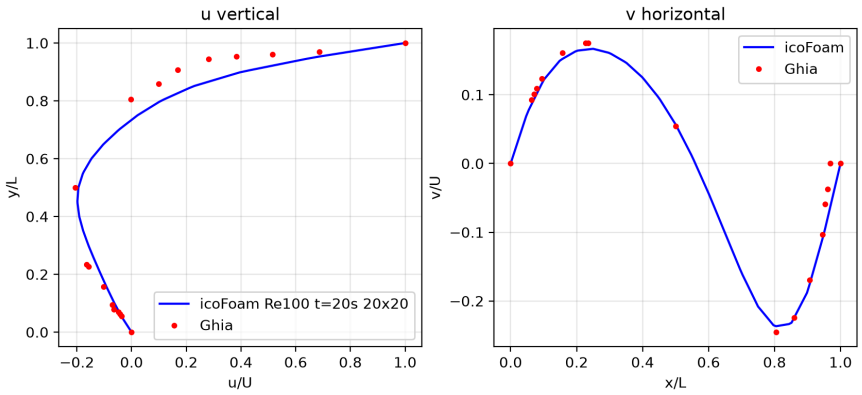


Figure 5. *Validation of AutoFOAM-generated* `icoFoam` *cavity case ($Re = 100$, 20×20 uniform mesh, $t = 20$ s) against Ghia et al. (1982). (left) u/U along vertical centerline $x/L = 0.5$ and (right) v/U along horizontal centerline $y/L = 0.5$.*

Figure 5 compares centreline velocity profiles at $Re = 100$. The horizontal-centreline v/U profile (right) agrees well with the benchmark, reproducing the positive peak (≈ 0.17 at $x/L \approx 0.22$) and negative peak (≈ -0.24 at $x/L \approx 0.80$) and the zero-crossing at $x/L \approx 0.55$.

6. Scope and Limitations

Even though the present version of AutoFOAM shows very strong zero-shot generalizability and stability in self-enhancement, there are some operation envelopes which are beyond the scope of this release. From the geometric point of view, the agent can only operate within the limits of 13 `gmsh` parameterized models. Processing of complex geometries provided by CAD software as raw formats (e.g., STEP/IGES) involving automatic topology fixings and generation of an unstructured tetrahedral mesh is not feasible in the present framework. For turbulent flows, the current numerical policy is restricted to the implemented RANS turbulence-model configurations, while laminar cases are treated without a turbulence model.

The Physics-Validation Gap.

The main limitation of the current system lies in the multi-variate reward function. At present, the reward is primarily based on numerical indicators such as residual decay, continuity error, boundary-condition validity, and mesh quality. These criteria assess whether the numerical solution is successfully computed, but they do not by themselves establish that the resulting flow field or engineering quantities are physically correct. For example, a simulation may achieve residual convergence and satisfy the continuity criterion while still producing an incorrect drag coefficient if the reference area or velocity used for its calculation is incorrectly specified. The current framework does not include an explicit physics-validation module that cross-checks predicted flow quantities against established benchmark data or literature results, such as the lid-driven cavity centerline profiles of Ghia et al. [16]. Consequently, such physics-based errors may not be detected by the current reward function and may propagate into the Layer 4 DPO preference pairs. Incorporating an externally defined `physics validator` that compares selected physical quantities against trusted reference data is therefore an important direction for future development.

7. Conclusion

Here, we developed AutoFOAM, an LLM-powered fully autonomous OpenFOAM agent designed to help close the semantic gap between the intent expressed in natural language

and the proper execution of computational fluid dynamics cases. Through the deterministic anchoring of a fine-tuned 14 billion parameters Qwen2.5-Coder model, we were able to overcome the cognitive and grammatical challenges that arise in the manual creation of a CFD case. AutoFOAM achieves 100% execution success rate and 96.4% accuracy in solver routing on out-of-distribution prompts. Notably, this project goes beyond static inference through the implementation of a highly effective, seven-level self-evolution loop. By carefully utilizing Active Learning techniques, Direct Preference Optimization on failed trajectory mining, and Anchor Mixing, we propose a framework for scaling agent skills without the danger of self-distillation collapse. As a result, the current architecture is not only a highly useful assistant in engineering practice, but also a research platform for continual and reinforcement-based learning under physical restrictions.

8. Generative AI Usage Disclosure

During this work, generative AI tools were used to assist with language editing, manuscript formatting (LaTeX/MyST), figure layouts, code documentation, and software debugging for the AutoFOAM framework. All core conceptualization, scientific methodology, algorithmic design, software implementation, validation, and analysis were independently developed and verified by the authors. The authors have reviewed all AI-assisted outputs and accept full responsibility for the originality, accuracy, and integrity of the work.

9. Artefacts and Reproducibility

To foster reproducible research in scientific machine learning, all model weights, codebases, training scripts, and foundational datasets are released openly under the Apache 2.0 license:

- **Codebase:** https://github.com/AGN000/AutoFOAM
- **Dataset:** github.com/AGN000/FoamAgentCases
- **Model Weights:** huggingface.co/arungovindneelan/foam-cfd-unified-14b

Acknowledgment

The authors acknowledge the support of the **NVIDIA Inception** startup program. The program provided valuable access to the NVIDIA ecosystem, technical guidance, and computing resources that supported the development and experimentation of *AutoFOAM*, an autonomous OpenFOAM agent framework for AI-driven CFD workflows.

References

[1] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby, "A tensorial approach to computational continuum mechanics using object-oriented techniques," *Computers in physics*, vol. 12, no. 6, pp. 620–631, 1998, doi: https://doi.org/10.1063/1.168744.

[2] J. Blazek, *Computational fluid dynamics: principles and applications*. Butterworth-Heinemann, 2015.

[3] The OpenFOAM Foundation, "OpenFOAM: The Open Source CFD Toolbox." [Online]. Available: https://openfoam.org/

[4] T. Brown *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020, doi: https://doi.org/10.48550/arXiv.2005.14165.

[5] H. Touvron *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023, doi: https://doi.org/10.48550/arXiv.2302.13971.

[6] B. Hui *et al.*, "Qwen2. 5-coder technical report," *arXiv preprint arXiv:2409.12186*, 2024, doi: https://doi.org/10.48550/arXiv.2409.12186.

[7] J. Bai *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023, doi: https://doi.org/10.48550/arXiv.2309.16609.

[8] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *Advances in neural information processing systems*, vol. 36, pp. 53728–53741, 2023, doi: https://doi.org/10.48550/arXiv.2305.18290.

[9] L. Yue, P. Xu, T. Zhang, N. Somasekharan, and S. Pan, "FoamGPT: Fine-Tuning Large Language Model for Agentic Automation of CFD Simulations with OpenFOAM," in *Advances in Neural Information Processing Systems*, 2025.

[10] S. Pandey, R. Xu, W. Wang, and X. Chu, "OpenFOAMGPT: A retrieval-augmented large language model (LLM) agent for OpenFOAM-based computational fluid dynamics," *Physics of Fluids*, vol. 37, no. 3, 2025, doi: https://doi.org/10.1063/5.0257555.

[11] L. Yue, N. Somasekharan, T. Zhang, Y. Cao, and S. Pan, "Foam-Agent 2.0: An End-to-End Composable Multi-Agent Framework for Automating CFD Simulation in OpenFOAM," *arXiv preprint arXiv:2509.18178*, 2025, doi: https://doi.org/10.48550/arXiv.2509.18178.

[12] W. Kwon *et al.*, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626. doi: https://doi.org/10.48550/arXiv.2309.06180.

[13] Y. Dong *et al.*, "Xgrammar: Flexible and efficient structured generation engine for large language models," *Proceedings of Machine Learning and Systems*, vol. 7, 2025, doi: https://doi.org/10.48550/arXiv.2411.15100.

[14] I. Shumailov, Z. Shumaylov, Y. Zhao, Y. Gal, N. Papernot, and R. Anderson, "The curse of recursion: Training on generated data makes models forget," *arXiv preprint arXiv:2305.17493*, 2023, doi: https://doi.org/10.48550/arXiv.2305.17493.

[15] Chroma, "Chroma: The AI-Native Open-Source Embedding Database." [Online]. Available: https://www.trychroma.com/

[16] U. Ghia, K. N. Ghia, and C. Shin, "High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method," *Journal of Computational Physics*, vol. 48, no. 3, pp. 387–411, 1982, doi: https://doi.org/10.1016/0021-9991(82)90058-4.

Open Access | https://doi.org/10.25080/ejwp2755

SciPy 2026
July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

OXA: An Open Exchange Architecture for Modular and Composable Scientific Content

Rowan Cockett[1,2,3], **Nokome Bentley**[4], **Monica Granados**[5], **Franklin Koch**[2,3], **Michael Markie**[6], **Steve Purves**[2,3], **Carlos Scheidegger**[7], **Paul Shannon**[6], **Gabriel Stein**[8], and **Tracy Teal**[8]

[1]Continuous Science Foundation, [2]Curvenote Inc., [3]Project Jupyter, [4]Stencila, [5]Creative Commons, [6]eLife Pathways, [7]Posit, PBC, [8]openRxiv

Abstract

Scientific communication remains trapped in a paper-shaped container. The dominant unit of exchange — a static, narrative PDF backed by Journal Article Tag Suite (JATS) XML — was designed to describe finished, print-shaped documents. It is poorly suited to modern research, in which the data, code, protocols, computational notebooks, and interactive results that constitute the actual evidence are relegated to hard-to-access supplementary files. This paper introduces the Open Exchange Architecture (OXA), an emerging, community-driven specification that represents scientific documents and their components as structured, typed, and uniquely addressable JSON objects. OXA is designed to enable exchange, interoperability, and long-term preservation while remaining compatible with modern web and data standards. We describe the technical architecture of OXA — a typed node model with `children` arrays inspired by `unified.js` and the Pandoc abstract syntax tree (AST) — and trace its provenance through JATS, the Stencila schema for executable documents, Curvenote's connected publishing model, and the document pipelines of Pandoc, MyST Markdown, and Quarto. We report on the first large-scale implementation of OXA: a Curvenote–openRxiv partnership that translated the bioRxiv JATS archive into an early version of OXA to power the openRxiv Labs "Curvenote Reader" experience. OXA is stewarded by the Continuous Science Foundation and governed through an open Request for Comments (RFC) process.

Keywords modular science, open standards, reproducibility, executable documents, scientific publishing, JATS

Published Jul 13, 2026

Correspondence to
Rowan Cockett
rowan@curvenote.com

Open Access

Copyright © 2026 Cockett *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction: Toward Modular Science

The way scientists conduct research has changed profoundly over the past two decades. Data are larger, analyses are computational, and collaboration is global and continuous. Yet the way we communicate research has barely moved: the canonical output is still a static document modeled on the printed page. Science is, in effect, trapped in a paper-shaped box, where content is static and narrative-driven, and the assets around the research — the data, code, protocols, and notebooks — are relegated to supplementary ZIP files that are hard to access. The reviews and credit attached to research accrue to the container itself rather than to the components beneath it.

The Open Exchange Architecture (OXA) is a response to this mismatch. It reconceives a scientific document not as a monolithic file but as a tree of modular, composable components — figures, notebooks, methods, datasets, equations, and computations — each of which can be directly referenced, reused, and recombined across the open-science ecosystem. This is the central idea of *modular science*: that modularity should emerge from how content is

packaged, so that a figure produced in a computational notebook can be cited, embedded, attributed, and licensed as a first-class object rather than copy-pasted as a flattened screenshot.

This shift matters now for two converging reasons. First, computational science increasingly produces evidence (interactive figures, multi-gigabyte datasets, executable notebooks) that simply cannot be expressed in a PDF. Second, machine-assisted discovery and AI systems are well complemented when they can also operate over structured, interoperable components, and relational metadata [1], [2]. A modular, structured substrate is the precondition for both human and machine reuse, and it is the principle at the core of OXA.

The challenge is how we get from where we are, a static narrative document, to where we want to go, science communicated through the modular elements of science, without first requiring that scientists share their work in completely new ways or changed incentive structures. This has consistently been a sticking point during shifts towards open science practices. The burden is placed on already over-busy researchers who don't necessarily have the skills or time to do this additional work. OXA addresses this through its positioning as an exchange format. Existing articles can be decomposed into modular elements and what's then possible with these elements can be demonstrated immediately, both not requiring new work from researchers and showing what's possible when we can communicate science in new ways, so can motivate why a researcher may want to work differently. Examples such as the openRxiv and Curvenote Reader partnership described below demonstrate this approach. Additionally, OXA provides a pathway to sharing science in new ways. A researcher can begin to layer in sharing notebooks, interactive figures, data and code through the process of publishing an article. OXA helps us build the *bridge* from where we are to where we want to go.

This paper is itself written in MyST Markdown [3] and submitted through the SciPy Proceedings toolchain, which since 2024 has been built on Curvenote [4]. That toolchain is part of the same lineage we describe below, and OXA can be understood as a generalization and community standardization of the structured document model specifically for scientific communication that already underlies these tools.

2. From modular software to modular science

The SciPy community already lives in a world that is not how most of the rest of science yet operates. SciPy [5], NumPy [6], pandas [7], [8], Matplotlib [9], and the libraries built on them are modular *and* composable. A user can `import` what they need, build something, and the result inherits the trustworthiness of the modular components, composing them into something new. Progress is multiplicative because integration is expected and relatively straightforward. However, the papers we write about that work are not modular or composable. Papers are currently monolithic and integration-hostile, everything gets trapped in a paper-shaped box. The figure cannot be pulled out of or cited independently of the paper, the data cannot be queried from the paper, and the code cannot be run against the data without re-implementing context the paper only alluded to. To reach this *composable* potential, we need to be able to share about science in the same *modular* ways we share code.

It is worth being precise about a distinction that is often blurred in conversations about open science - *disaggregation* and *modularity*. *Disaggregation* is placing each research component in a different, type-appropriate repository. *Modularity* is dividing something into separate modules or components that can then be recombined. While *disaggregation* does achieve the goal of separating components, it does not necessarily easily allow for reuse or recombination. Therefore it's important to note that *disaggregation* and *modularity* are

not the same thing. The push, over the past two decades, to unbundle the scientific paper (separating preprints from peer review, peer review from venue, venue from data hosting, data hosting from code) has been focused on *disaggregation* which is an important step, especially for appropriate archiving and curation. However, disaggregation is not, on its own, the goal. The goal is **ecosystem composability**: research products whose components can be found, reused, recombined, and extended without losing their integrity or provenance Table 1.

Table 1. *Two waves of modular science.*

Dimension	First wave	Second wave
Core philosophy	Break apart to share	Package and compose
Researcher action	File separate deposits	Work natively, modularity emerges
Output state	Disconnected pieces	Connected, navigable graphs
Provenance	Lost or manually added	Inherited and travels with object
System result	Sprawl of scientific components, loosely connected	Integrated system with provenance

Modular science is the principle that scientific outputs can become as composable as scientific software, given the right standards, identifiers, tooling, and packaging infrastructure. The emerging primitives are visible: `import figure from paper` is no longer a metaphor. With component-level identifiers and a structured-document standard, a figure published in one preprint can be embedded, with full provenance, in a review article, a textbook, a grant proposal, another lab's analysis notebook, or an AI chat. With the right standards the figure stays alive; the data underneath it stays addressable; updates propagate; attribution is automatic.

3. Background

3.1. JATS and the print-shaped record

The Journal Article Tag Suite (JATS) is the de facto open standard for the scholarly version of record. It is an ANSI/NISO standard (Z39.96) and, through PubMed Central, Crossref, and indexers, underpins the machine-readable archive of the literature [10]. JATS has been enormously successful at what it was designed to do: provide a common XML vocabulary in which publishers and archives can exchange finished journal content. The scale is substantial: Crossref hosts over 96 million works with a full-text/text-mining link (96,294,821 of 134,048,223 total works, queried from the Crossref dataset via Alexandria3k [11]), the large majority of which are XML.

However, JATS carries the assumptions of the medium it was built for. It describes static, finished documents. Several well-documented friction points recur:

- **Cost and tooling.** Producing high-quality JATS requires either hand-authoring XML or significant investment in professional conversion tools or outsourced vendors — a barrier that falls hardest on smaller publishers and journals.
- **"Valid" is not "good."** Valid XML can still be functionally broken: a misplaced hyphen in a DOI can cause failed deposits in Crossref or PubMed Central, and insufficiently tagged references hinder discoverability. Validity guarantees conformance to a schema, not fitness for purpose.
- **Tag-set divergence.** Even though JATS is widely adopted, publishers frequently maintain their own subset conventions and house markup schemes. This "flexibility" makes authoring to a single compliant target difficult and undermines true interoperability.

3.2. The computational sticking points

The deeper problem for computational science is structural. The work of the *Notebooks Now!* project [12] and the companion "Science Communication with Notebooks" recommendations identified, concretely, what breaks when one tries to bring a computational notebook into the version of record [13]:

1. **Executable content has no native home.** JATS can hold code as text, but it has no first-class model for an executable cell, its kernel/language, its outputs, and the linkage between them.
2. **Outputs are flattened.** Interactive figures (e.g. Plotly [14], Bokeh [15], Altair [16]), 3D visualizations, and large-scale data views are reduced to static images. A one-and-a-half-terabyte microscopy image becomes a screenshot.
3. **Provenance and dependency are lost.** The relationship between a result, the code that produced it, the data it consumed, and the environment it ran in is not represented; it survives, if at all, in a supplementary ZIP file.
4. **The container, not the component, is the unit of credit.** Review, citation, and attribution attach to the article DOI, not to the figure, dataset, or method that other researchers may want to reuse.

The *Notebooks Now!* working groups concluded that **integrating computational documents into scholarly publishing requires a re-imagination of the publishing processes — from submission through peer review to production — and authoring tools that can execute content, display computational and interactive outputs directly, and archive that content faithfully.** OXA is an attempt to provide the missing data substrate for exactly that re-imagination.

3.3. San Diego standards meeting

OXA emerged at an in-person working meeting held in San Diego in November 2025, hosted by openRxiv and the Continuous Science Foundation and documented in the report "From Tools to Adoption: A Path to Modular and Interactive Scientific Publishing" [17]. Twenty-five open-science leaders — developers of the leading modular publishing tools, alongside licensing and metadata experts — gathered to build a working, federated reference architecture using real bioRxiv content and connected authoring tools. The convened tool developers spanned Stencila [18], MyST [3], Quarto [19], Curvenote [4], OpenAlex, and eLife. The explicit philosophy was implementation-first: prioritizing working software and demonstrable interoperability over abstract standardization.

In this meeting, a 'bedrock, soil, flowers' framework was put forth. In a modular science ecosystem, the modular elements of science - the data, code, images, protocols - are the 'bedrock'. They are the foundation on which scientific claims are built. The 'flowers' are the output of combining these modular elements - the articles, visualizations, discovery tools, the things we typically consume or explore as readers of science. What's missing now is not a lack of bedrock elements, or ideas for flowers and how science can be shared, but the connections from the bedrock that would allow for more 'flowers' and experiments in scientific communication. The 'soil' is the missing middle that provides the real basis for new models of scientific communication, attribution, discovery and remixing to occur. By working on that missing middle 'soil' layer, that's the unlocking and enabling function that makes so many more approaches possible, and what the tool developers at this meeting identified as a fundamental missing component in the ecosystem. It's crucial that these soil layers be community stewarded and governed, as an element of the ecosystem that is meant to connect different substrates and enable broad use and creativity from the developers of tools in the scientific space. This aligned with the gaps identified in the *Notebooks Now!* project and pointed again to the need for a connecting, implementation-focused standard

that is purpose-built and community-developed for sharing science digitally (where there is no need for the paper-shaped box) in a way that the modular elements of science in the 'bedrock' can be reused and remixed by both humans and machines. Therefore in Day 2 of this meeting as a collaborative group, we began the work on OXA with the defining community and technical principles outlined below.

4. The RFC and community process

OXA is explicitly designed as an open community specification rather than a single organization's product or platform. It is stewarded by the Continuous Science Foundation (CSF), a Canadian non-profit that convenes researchers, tool builders, publishers, and standards groups around modular, continuous publishing.

Governance proceeds through a Request for Comments (RFC) process: the technical structure is proposed, input is gathered from the community, and implementation is driven in parallel. The work is distributed across a set of public repositories in the `oxa-dev` GitHub organization.

The specification and schemas are CC0-licensed, and the tooling is MIT-licensed, lowering the barrier for tool builders, publishers, and repositories to adopt and extend the format. This community-first approach is central: the architecture rests on open standards and open-source values rather than on any single vendor's platform.

5. Technical architecture

OXA is a specification for representing scientific documents and their components as structured JSON objects. The format follows a typed node model with `children` arrays that form a tree, inspired by UnifiedJS [20] and the Pandoc AST.

Every OXA node shares a common shape with a small set of well-defined properties:

Table 2. *The common shape of an OXA node.*

Property	Type	Description
`type`	string (capitalized)	The node type, e.g. `Paragraph`, `Text`, `Heading`.
`id`	string	Unique identifier for referencing and linking nodes.
`data`	object	Arbitrary metadata (attributes, provenance, DOI).
`children`	array	Nested content nodes — block or inline types.

Node types divide into inline nodes (`Text`, `Emphasis`, `Strong`, `InlineMath`, `Link`, `Image`, `Cite`, `CiteGroup`, and others) and block nodes (`Paragraph`, `Heading`, `CodeBlock`, `BlockQuote`, `Section`, `Div`). A document is rooted in an `OXA` node carrying version and metadata (license, authors, title). A simple example of a paragraph OXA node is shown in Program 1 to demonstrate the nested structure of the node model. For more complex examples, see the OXA documentation and existing node schemas.

```yaml
type: Paragraph
children:
  - type: Text
    value: "This is "
  - type: Strong
    children:
      - type: Text
        value: "bold and "
      - type: Emphasis
        children:
          - type: Text
            value: "italic"
  - type: Text
    value: " text."
```

Program 1. *A very simple example of a paragraph OXA node in a document with some inline elements for bold, italic, and text. The markdown equivalent is:* `This is **bold and _italic_** text.`*. The JSON is shown in YAML format for readability.*

The design principles are:

- **Open by design** — JSON Schema (Draft-07) based and CC0-licensed.
- **Composable** — each node is self-contained, typed, and nestable or reusable.
- **Interoperable** — compatible with JATS, MyST Markdown, Stencila, Curvenote, and Quarto.
- **Extensible** — new node types can be added while preserving schema validation.
- **Typed and linked** — everything has a clear `type`, optional `id`, and structured `data`.
- **Modular** — documents and components can link across projects, enabling cross-references, citations, and reuse of figures, data, or methods from distributed sources.
- **Computational** — the schema builds in support for computational content such as notebooks.

OXA positions itself relative to its conceptual ancestors as a modernization for web-native use: JATS and the Pandoc AST as structural ancestors, Schema.org for naming concepts, Stencila as a functional and conceptual ancestor for computational typing, and Quarto, MyST Markdown and Curvenote as integrations to computational ecosystems. The architecture also includes packages for validation, rendering, and transformation that allow lightweight, independent implementations, and a conversion path to the AT Protocol (ATProto) `pub.oxa.*` lexicon namespace (see the ATProto OXA documentation for more details) — letting documents live in any Personal Data Server so that scientific content can become user-owned, portable, and discoverable alongside social feeds and moderation infrastructure. For example, Program 2 shows the ATProto representation of the same example as Program 1 in AT Protocol format.

```
{
  "$type": "pub.oxa.blocks.defs#paragraph",
  "text": "This is bold and italic text.",
  "facets": [
    {
      "index": { "byteStart": 8, "byteEnd": 23 },
      "features": [
        { "$type": "pub.oxa.richtext.facet#strong" },
        { "$type": "pub.leaflet.richtext.facet#bold" }
      ]
    },
    {
      "index": { "byteStart": 17, "byteEnd": 23 },
      "features": [
        { "$type": "pub.oxa.richtext.facet#emphasis" },
        { "$type": "pub.leaflet.richtext.facet#italic" }
      ]
    }
  ]
}
```

Program 2. *The AT Protocol representation of OXA uses "facets" instead of a tree. The text is stored as a single plain string, and formatting is described by byte-range annotations. The conversion tools* `oxa convert --to atproto examples/document.json` *will convert an OXA document to this format and include other common facet annotations used by the community such as Leaflet (https://leaflet.pub). The code shows the same example as Program 1 in AT Protocol format.*

6. Provenance and lineage of OXA

OXA's design synthesizes lessons from several lines of prior work: JATS as the structural ancestor of a standardized scholarly record; the Stencila schema for typed, executable documents; Curvenote's work in scholarly publishing and interactive articles; and the document-transformation pipelines of Pandoc, MyST Markdown, UnifiedJS [20], and Quarto.

JATS is the structural starting point. From it, OXA borrows the core idea of a single, standardized, machine-readable vocabulary for scholarly content, and inherits much of its element naming. What OXA does not inherit is JATS's print-shaped orientation. This limitation was recognized independently by many of the projects in this space.

The most direct ancestor is the Stencila schema. Since 2019, Stencila has represented executable documents as a tree of JSON data validated by JSON Schema, with the explicit aim of providing a framework for highly structured, strongly typed executable documents. The Stencila schema extends Schema.org types — adopting `ScholarlyArticle`, `Dataset`, `Person`, and `Organization` where they exist, and adding executable extensions such as `CodeChunk` and `Parameter` where Schema.org falls short. This canonical model is the source of truth across Stencila's parsers, codecs, kernels, and publishing pipelines, allowing documents to round-trip between formats. OXA inherits this typed-node, Schema.org-aligned philosophy directly, and Stencila's founder, Nokome Bentley, is a co-author of this paper and a member of the OXA steering council.

A parallel thread runs through Curvenote's work in interactive scholarly publishing, which fed directly into the broader MyST ecosystem described below. In the American Geophysical Union (AGU) Notebooks Now! project — an effort to develop an end-to-end workflow for submitting, peer-reviewing, and publishing computational notebooks as a primary element of the scientific record [12] — Curvenote and Posit (the makers of Quarto) collaborated within the working groups, prototyping how notebooks could move through submission, review, and production. The findings of that work are documented in the *Notebooks in Publishing* report [13].

Both the Stencila schema and the Notebooks Now! working groups arrived at the same conclusion: JATS cannot accommodate computational content shoehorned in after the fact, and its print-shaped model is a functional blocker to interactive or more modular ways of working with scientific content. The integrations that modern research demands, including executable cells, live/interactive outputs, component-level provenance, and modularity, have no native home in a format designed to describe static, print documents.

The way past that blocker is a structured intermediate representation that accommodates computational content and supports modularity and composability. There are now several demonstrations of this working at scale. Pandoc popularized the pattern: parse documents into a typed abstract syntax tree and transform that tree into many standardized outputs [21]. Quarto [19] and MyST Markdown build on the same idea for computational narratives. MyST (Markedly Structured Text) is a community-driven superset of CommonMark with first-class support for citations, cross-references, executable directives, and scholarly metadata (researcher identifiers (e.g. https://orcid.org), funding information, or contributor roles (e.g. https://credit.niso.org/)) [3]; its command-line tooling began at Curvenote as the `curvenote` CLI, became `mystmd` in 2023, and is now part of Project Jupyter [22]. The MyST Document Engine produces a MyST AST that preserves narrative content, computational blocks, outputs, citations, figures, and metadata. Both MyST[1] and Quarto are deployed across thousands of projects [3], demonstrating that AST-driven, computational narratives can work across many domains and output formats at scale.

7. Why a new standard

Every mention of a new standard makes one think of the xkcd comic on standards. That comic proceeds as **Panel 1:** 'Situation: there are 14 competing standards'. **Panel 2:** '14?! Ridiculous! We need to develop one universal standard that covers everyone's use cases'. **Panel 3:** 'Soon: Situation: there are 15 competing standards.' The motivation for OXA is actually in opposition to Panel 2. There are existing generalized standards or approaches in prior works that do an excellent job of covering a wide range of use cases. There are also existing standards like JATS and others specifically for a print version of scientific communication. However, to create a purpose-built standard for our current digital era, with both humans and machines as users of this standard, we need something focused on these needs, rather than trying to build in niche needs to a general solution or repurpose a not-right-sized existing standard. Creating a new approach **that can connect to any of the existing standards** lets us build on where we are without needing users, infrastructure organizations or tool developers to change.

OXA generalizes efforts into a shared *exchange* format. Where for example, MyST and Quarto are authoring syntaxes and document pipelines, OXA is a neutral, web-native format onto which all of them — along with JATS — can map.

We also, as a scientific communication community, need a framework and coordination approach and a community-governed and neutral space, that allows us to engage together in the process, staying aligned with the goals for scientific sharing without needing to create overly general solutions or be beholden to the needs of a specific few. This is again borrowing from the approaches of the open source community, with Requests for Comments, contributor guidelines, version control and transparent communication and decision making. Therefore the governance structure of OXA is as important as the technical foundation in allowing for institutional adoption that can move us towards a standard that is adopted across millions of articles.

[1]There are over 18,000 dependencies of Jupyter Book, an implementation of MyST, based on GitHub dependency graph: https://github.com/jupyter-book/jupyter-book/network/dependents.

8. Related work: provenance, attribution, and research objects

While we see that OXA fills a crucial need and niche, it does enter a crowded landscape of efforts to make research machine-readable and reusable. OXA is meant to complement, learn from and connect with existing efforts and these projects continue to be crucial in the ecosystem. We review the most relevant active community projects, emphasizing community-led work.

JATS and JATS4R. JATS remains the substrate of the published scientific record, and JATS4R [23] provides community recommendations to make JATS tagging more consistent and reusable. OXA does not seek to replace JATS as an archival format so much as to provide a web-native, computation-aware layer that can interoperate with it — as demonstrated by the bioRxiv translation described below.

MECA. The Manuscript Exchange Common Approach (MECA), a NISO Recommended Practice, defines a packaging format for moving manuscripts between submission systems, preprint servers, and production services [24]. MECA is the closest existing analogue to OXA's exchange goal, however, it inherits JATS's static-document assumptions: it packages finished manuscripts in a ZIP file rather than modeling living, computational, component-level content. OXA can be read as a "modern MECA+JATS" alternative that replaces the static-document assumptions embedded in JATS with a more modern, computational-aware approach that can be accessed in a web-native way.

RO-Crate and Research Objects. RO-Crate is an open, community-driven, lightweight approach to packaging research artifacts with their metadata, based on Schema.org annotations in JSON-LD [25]. OXA and RO-Crate are complementary: RO-Crate excels at describing and bundling a *collection* of artifacts and their metadata, while OXA models the *internal structure* of the document and its components as a typed tree. An OXA document could be packaged within an RO-Crate.

Reproducible packaging: Whole Tale, Binder. Whole Tale [26] captures the whole computational environment ("tale") needed to reproduce an analysis, and Binder/BinderHub [27] turn a code repository into an executable, interactive environment in the browser. These projects solve the *execution and environment* problem that complements OXA's *representation* problem.

A web of overlapping collaborations connects these efforts: eLife and Stencila on Executable Research Articles [28]; AGU and Curvenote on Notebooks Now! [12]; and openRxiv with Curvenote on the Reader implementation described next [29]. OXA is best understood as the point where these threads converge, coordinated through CSF's working groups and the RFC process.

9. First large-scale implementations

9.1. *Curvenote* × *openRxiv*

The first large-scale implementation of OXA is a partnership between Curvenote (https://curvenote.com) and openRxiv (https://openrxiv.org), the independent non-profit that stewards bioRxiv and medRxiv. On June 11, 2026, openRxiv launched its first openRxiv Labs experiment: the Curvenote Reader, an interactive reading experience layered over the entire bioRxiv corpus [29].

The enabling work was a translation of openRxiv's JATS-format XML archive into an early version of OXA. The Reader lets readers explore references, terminology, expanded figures, and related works while staying in the context of the original preprint, with the same URL structure as bioRxiv. The translation covered 41.7TB and over 500,000 preprint versions in

the bioRxiv and medRxiv corpus. The translation pipeline is supported using the `jats` project (https://github.com/continuous-foundation/jats), which has been tailored by Curvenote to the specific needs of the openRxiv corpus.

This implementation is significant for three reasons. First, it demonstrates that a large, real-world JATS archive can be translated into OXA at scale. Second, it validates OXA's "soil" thesis: once content is structured, new "flower" experiences (interactive reading, querying, future tool development) can be built on top. Third, it embodies the constellation model of research that openRxiv has articulated: where a preprint is one node connected to data repositories, reviews, replications, and trust signals across multiple organizations rather than a self-contained PDF. OXA is the structural substrate that makes such a constellation addressable. For example, Figure 1 shows a hover-citation that uses the OXA schema to display a citation with accompanying metadata as well as figure previews. The figure content is pulled directly from a selection of nodes in the OXA document. These content previews could be extended to AI-chat interfaces and other interactive reading experiences.

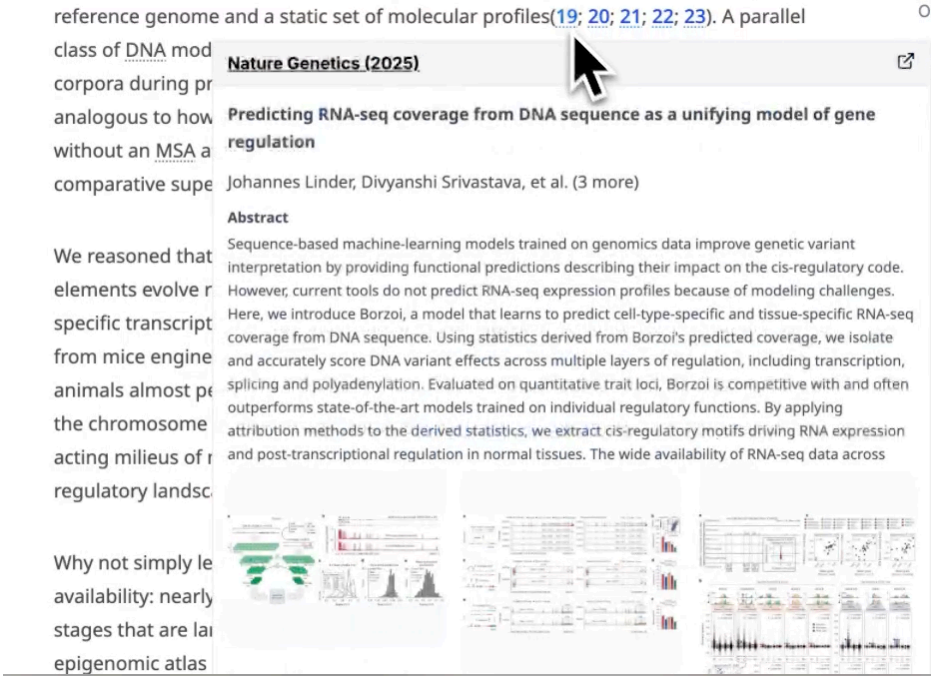


Figure 1. *A hover-citation using Curvenote Reader that uses the OXA schema and translated documents to display a citation with accompanying metadata as well as figure previews. The figure content is pulled directly from a selection of nodes in the OXA document.*

9.2. eLife Pathways × Stencila

eLife Pathways (https://elifepathways.org) is another organizational implementation of OXA through an expanding collaboration with Stencila (https://stencila.io) and adapting eLife Pathways open-source Enhanced Preprint Platform to natively support OXA. There are also plans to build OXA support with Kotahi (https://github.com/elifepathways/kotahi). These integrations create new opportunities for collaboration between technology and publishing projects within the broader, interoperable scholarly communications ecosystem.

10. Emerging tooling

A small ecosystem of tooling is beginning to form around the specification, spanning both validation and conversion.

The canonical reference tool is the `oxa` command-line utility, developed in the oxa-dev organization. The scope of the CLI is deliberately narrow: it validates OXA documents against the published schemas. It can check a single file or a whole glob, read from standard input for use in pipes, validate a fragment against a specific node type (for example, `oxa validate --type Heading`), and report results through conventional exit codes so that it slots cleanly into continuous-integration and pre-commit checks. It fronts a small package family that includes `@oxa/core` for programmatic validation and `oxa-types` for TypeScript definitions and `@oxa/react` for a minimal demonstration of rendering.

```
# Install the OXA CLI tool
pip install oxa

# Validate a JSON file
oxa validate document.json

# Validate multiple files
oxa validate *.json

# Validate from stdin
cat document.json | oxa validate -

# Validate YAML files
oxa validate document.yaml

# Validate against a specific node type
oxa validate --type Heading heading.json
```

Program 3. *The OXA CLI tool allows you to validate OXA documents from the command line:*

The OXA CLI tool is a wrapper around the OXA programmatic APIs, which allow you to validate OXA documents from Python, JavaScript, or Rust. For example, Program 4 shows how to validate an OXA document from JavaScript. The `oxa-types` library is available on `pip`, `npm`, and `cargo` and the JSON Schema is available at https://oxa.dev/v0.2.1/schema.json.

```
import { validate } from "@oxa/core";
import type { Document } from "oxa-types";

const document: Document = {
  type: "Document",
  children: [
    {
      type: "Paragraph",
      children: [{ type: "Text", value: "Hello, world!" }],
    },
  ],
};

const result = validate(document);
if (result.valid) {
  console.log("Document is valid!");
} else {
  console.error("Validation errors:", result.errors);
}
```

Program 4. *The OXA programmatic APIs allow you to validate OXA documents from JavaScript.*

Conversion is the second strand, and the most developed and accessible work to date comes from Stencila. Building on its canonical, strongly typed schema, Stencila has implemented a bidirectional OXA codec that both decodes OXA JSON into the Stencila document model and encodes that model back out to OXA, tracking any information lost in either direction. Because this conversion routes through the same canonical schema that already backs Stencila's parsers and codecs for many other formats, it inherits that format support: MyST Markdown, Quarto (`.qmd`), Jupyter notebooks (`.ipynb`), Microsoft Word (`.docx`), and other Markdown flavors can be converted into OXA — and back out again. **This is the practical**

payoff of the typed-node lineage traced above: rather than asking authors to adopt a new syntax, OXA can meet them in the formats they already use, and documents they have already written become a source of structured, OXA-native content.

11. Open standards as the substrate

The open standards stack we view as load-bearing for modular science is, in our reading: a structured document format with executable extensions; persistent, component-level identifiers for figures, datasets, code artifacts, and computational environments — not a DOI for every sentence, but a high-fidelity *linking system* inside the bundle that carries licensing, attribution, and other metadata; an open preprint foundation (e.g. the broader openRxiv ecosystem) that accepts and serves modular outputs as first-class; machine-legible trust signals; and reproducible execution environments. Every one of these has working open-source implementations today. The gap is integrative and connective: producing tooling that hides this complexity from the researcher while preserving the openness of the substrate beneath.

Cultures change when the new way is *better* than the old way, offering a 'moving towards', rather than just a 'moving away from'. The most durable path to a modular research record is to make modular outputs visibly more useful than the PDF — to the original authors, their collaborators, and the broader field — and to make the path from existing work to modular output smooth enough that a typical lab can take it easily. Funder and publisher policies and changing incentive structures are still crucial, but when there is a real carrot with a compelling reason to change, those policies can be more effective and impactful.

12. Governance and steering

OXA is stewarded by the Continuous Science Foundation. Day-to-day technical direction rests with a steering council whose current members are: Nokome Bentley (Stencila), Rowan Cockett (Curvenote), and Tracy Teal (openRxiv). The steering council operates within CSF's broader governance, and changes to the specification proceed through the open RFC process, ensuring a diversity of perspectives and that no single organization controls the direction of the standard.

13. Conclusion and future work

OXA is an early-stage but already-implemented attempt to give modern, computational, modular science a native exchange format. By representing documents and their components as typed, addressable, CC0-licensed JSON objects, it aims to unbundle research from its paper-shaped legacy and expose the evidence underneath — making figures, notebooks, data, and methods first-class, reusable, attributable, and machine-actionable. Its provenance in the Stencila schema, Curvenote, and MyST Markdown gives it a running start, and its first large-scale deployment over the bioRxiv and medRxiv corpus demonstrates feasibility at scale.

Much remains to be done. The schema is at an early version and will evolve through the RFC process; the licensing-and-attribution framework for modular components is under active development; and the 2026 roadmap includes further large-scale pilots and deeper integration of the Reader experience into bioRxiv and medRxiv. The broader test will be adoption: whether tool builders, publishers, and repositories converge on OXA as a shared layer. A standard is only as useful as the number of tools and institutions that actually emit and consume it.

Other limitations or things to continue to address include how the OXA structuring occurs, "AI-readability" and the handling of versioning and continuous content. Even with a friendlier format, producing well-typed, richly linked OXA documents takes good conversion tooling, and that tooling is still in its early stages and designed for particular groups. Creating the format needs to be achievable for everyone, not just well-resourced or particularly tech-savvy players. Additionally the current work has focused on the use case of 'flowers' designed for humans, like new reading experiences. We need to test against use cases with AI agents and LLMs that are increasingly the consumers of scientific communication to close that "AI-readability" gap. Finally, OXA leans into preprints and computational content that changes over time. It's a strength for continuous science, but it means that OXA needs to be designed to handle versioning and pinning, so that both modularity and provenance are maintained. Directed work and community engagement around these topics will be crucial.

We invite the SciPy community — long a leader in computational, open, and reproducible science, and the originators of much of the composable software ecosystem this paper takes as its model — to engage with the specification, contribute RFCs, and build on the architecture to ensure it's fit for purpose and for the future.

References

[1] S. Chen, T. Alrashed, A. Halevy, and N. Noy, "Do Data Agents Need Semantic Metadata? A Comparative Study in Agentic Data Retrieval." [Online]. Available: https://arxiv.org/abs/2605.28787

[2] Q. Deng *et al.*, "Unstructured Data Analysis using LLMs: A Comprehensive Benchmark." [Online]. Available: https://arxiv.org/abs/2510.27119

[3] P. Jupyter *et al.*, "Jupyter Book 2 and the MyST Document Stack: A modular, extensible, web-native stack for authoring and publishing computational narratives," in *Proceedings of the 24th Python in Science Conference*, in SciPy. 2025, pp. 173–193. doi: 10.25080/hwcj9957.

[4] R. Cockett, S. Purves, F. Koch, and M. Morrison, "Continuous Tools for Scientific Publishing: Using MyST Markdown and Curvenote to encourage continuous science practices," in *Proceedings of the 23rd Python in Science Conference*, in SciPy. 2024, pp. 121–136. doi: 10.25080/nkvc9349.

[5] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[6] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[7] The Pandas Development Team, "pandas-dev/pandas: Pandas." [Online]. Available: https://doi.org/10.5281/zenodo.3509134

[8] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, Stéfan van der Walt and Jarrod Millman, Eds., 2010, pp. 56–61. doi: https://doi.org/10.25080/Majora-92bf1922-00a.

[9] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: https://doi.org/10.1109/MCSE.2007.55.

[10] National Information Standards Organization, "JATS: Journal Article Tag Suite, version 1.4 (ANSI/NISO Z39.96-2024)." [Online]. Available: https://jats.nlm.nih.gov/

[11] D. Spinellis, "Open reproducible scientometric research with Alexandria3k," *PLOS ONE*, vol. 18, no. 11, p. e294946, 2023, doi: 10.1371/journal.pone.0294946.

[12] G. Caprarelli, B. Sedora, M. Ricci, S. Stall, and M. Giampoala, "\textit{Notebooks Now!} The Future of Reproducible Research," *Earth and Space Science*, vol. 10, no. 12, 2023, doi: 10.1029/2023ea003458.

[13] Curvenote and AGU Notebooks Now! Working Groups, "Notebooks in Publishing." [Online]. Available: https://curvenote.github.io/notebooks-in-publishing/

[14] Plotly, "Plotly." [Online]. Available: https://plotly.com/

[15] Bokeh, "Bokeh." [Online]. Available: https://bokeh.org/

[16] Altair, "Altair." [Online]. Available: https://altair-viz.github.io/

[17] R. Cockett and T. Teal, "From Tools to Adoption: A Path to Modular and Interactive Scientific Publishing," 2025, doi: 10.62329/kcep6732.

[18] N. Bentley, "Stencila." [Online]. Available: https://stencila.io/

[19] J. Allaire, C. Teague, C. Scheidegger, Y. Xie, C. Dervieux, and G. Woodhull, “Quarto.” [Online]. Available: https://github.com/quarto-dev/quarto-cli

[20] unified.js, “unified.js: The unified compiler.” [Online]. Available: https://unifiedjs.com/

[21] J. MacFarlane, “Pandoc: a universal document converter.” [Online]. Available: https://pandoc.org/

[22] T. Kluyver *et al.*, “Jupyter Notebooks - a publishing format for reproducible computational workflows,” in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Scmidt, Eds., Netherlands, 2016, pp. 87–90. [Online]. Available: https://eprints.soton.ac.uk/403913/

[23] National Information Standards Organization, “JATS4R: Journal Article Tag Suite for Reuse.” [Online]. Available: https://jats4r.niso.org/

[24] National Information Standards Organization, “MECA: Manuscript Exchange Common Approach (NISO RP-30-2023),” techreport, 2023. [Online]. Available: https://www.niso.org/standards-committees/meca

[25] S. Soiland-Reyes *et al.*, “Packaging research artefacts with RO-Crate,” *Data Science*, vol. 5, no. 2, pp. 97–138, 2022, doi: 10.3233/ds-210053.

[26] A. Brinckman *et al.*, “Computing environments for reproducibility: Capturing the ``Whole Tale'',” *Future Generation Computer Systems*, vol. 94, pp. 854–867, 2019, doi: 10.1016/j.future.2017.12.029.

[27] P. Jupyter *et al.*, “Binder 2.0 - Reproducible, interactive, sharable environments for science at scale,” in *Proceedings of the 17th Python in Science Conference*, in SciPy. 2018, pp. 113–120. doi: 10.25080/majora-4af1f417-011.

[28] eLife, “eLife launches Executable Research Articles for publishing computationally reproducible results.” [Online]. Available: https://elifesciences.org/for-the-press/eb096af1/elife-launches-executable-research-articles-for-publishing-computationally-reproducible-results

[29] openRxiv and Curvenote, “openRxiv Labs Curvenote Reader.” [Online]. Available: https://reader.openrxivlabs.org/

Open Access | https://doi.org/10.25080/uekx6559

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

NODEFit: Fit time-series data with a Neural Differential Equation

Pavan B. Govindaraju[1]

[1]SplitFXM

Abstract

Time-series data often arise from underlying physical, biological, and engineering systems governed by continuous dynamics. Standard discrete-time regressors, models that predict the next sample from a finite history of past observations, and fixed-form curve fits can interpolate measurements yet fail to extrapolate or to represent stochastic structure when the governing equations are unknown. Neural Ordinary Differential Equations (Neural ODEs) learn a continuous-time evolution law with a neural network, and Neural Stochastic Differential Equations (Neural SDEs) extend this idea to systems driven by random noise. This paper introduces NODEFit, an open-source Python package that wraps differentiable PyTorch solvers (`torchdiffeq` for ODEs and `torchsde` for SDEs) to fit Neural ODEs and Neural SDEs to measured data. Relative to calling those libraries directly, NODEFit lets a user specify a drift network, and for SDEs a diffusion network, once and reuse that architecture across every series in a dataset, while the package manages joint training of the corresponding parameters. That bookkeeping is especially useful for bulk processing of large collections of series, such as the PTB-XL cohort study in this paper. We motivate the approach with examples where template-based and discrete-time methods do not perform well, recover underlying dynamics on synthetic benchmarks, reconstruct the classical Lotka–Volterra predator–prey model, and evaluate the method on clinical electrocardiogram (ECG) recordings from PTB-XL, including stochastic heart-rate dynamics in arrhythmias such as atrial fibrillation across nearly 2,000 patients. Finally, we summarize when practitioners should prefer continuous-time neural ODE and SDE models.

Keywords neural-ode, neural-sde, time-series, machine-learning, differential-equations

Published Jul 13, 2026

Correspondence to
Pavan B. Govindaraju
pavan@splitfxm.com

Open Access

Copyright © 2026 Govindaraju. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Physical phenomena are commonly governed by differential equations that describe how a system evolves continuously in time. Traditional time-series methods often rely on discrete-time models: predictors that map a finite history of samples $(y_{t-k}, \ldots, y_t)$ to a next-step value y_{t+1}, without an explicit notion of rates of change between observation times. A concrete failure mode is irregular sampling. An autoregressive or recurrent model trained at a fixed step Δt has no well-defined state between grid points. Querying it at an intermediate time, or after a gap of several Δt, requires ad hoc interpolation or multi-step rollouts that accumulate error. Continuous-time neural models were introduced in part to address exactly this setting [1], [2].

An ordinary differential equation (ODE) instead specifies a vector field $dy/dt = f(y, t)$ whose integral yields the trajectory. Neural Ordinary Differential Equations (Neural ODEs) [1], [3] replace the unknown f with a neural network f_θ and learn the dynamics by matching the integrated trajectory to data.

When the evolution itself is noisy, the natural continuous-time description is a stochastic

differential equation (SDE)[1]: an equation of the form $dy = f(y,t)\,dt + g(y,t)\,dW_t$ in which a drift f is accompanied by a diffusion g driven by Brownian motion W_t. Neural Stochastic Differential Equations (Neural SDEs) [4], [5] learn both f_θ and g_θ from observations.

Consider a concrete motivating example where two state variables rise toward distinct steady-state values according to smooth exponential kinetics, and each measurement is corrupted by noise. One might try nonlinear least squares with `scipy.optimize.curve_fit` [6], specifying a candidate functional form such as a single exponential or logistic. That approach works only when the chosen template matches the true dynamics.

These failures share a root cause: the models describe values at sampled times rather than the rates of change that generated the trajectory. Neural ODEs and Neural SDEs take the opposite view. Instead of fitting $y(t)$ directly, they learn a vector field $f_\theta(y,t)$, and, when needed, a diffusion term $g_\theta(y,t)$, such that integrating forward reproduces the observations. Neural SDEs are particularly useful for noisy observations, where they separate drift from diffusion, yielding mean trajectories and uncertainty bands that envelop the data.

These models have direct applications wherever observations are generated by evolving state variables. In ecology, prey and predator populations are coupled through the Lotka–Volterra equations. In epidemiology, compartment models describe infections and recoveries, and in chemical kinetics, reaction concentrations evolve according to mass-action laws. Related examples also occur in robotics and control, where a vehicle's position and velocity follow continuous dynamics, and in finance, where continuous-time stochastic models describe quantities such as asset prices and interest rates. In each case, measurements may be sparse, irregular, noisy, or incomplete, making it useful to learn a continuous evolution law rather than only a one-step predictor.

NODEFit, built on PyTorch, offers a distinct approach to time-series modeling and addresses a different set of assumptions than many established alternatives. Classical ARIMA models [7] are effective statistical models for regularly sampled series with linear dependence and carefully specified differencing and seasonal structure. Transformer models [8] use self-attention to learn long-range relationships in sequences and can be highly expressive, but they generally treat the sampling grid as part of the representation rather than learning an explicit continuous-time evolution law. Path signatures [9] summarize a time series through iterated integrals and provide useful features for learning from sequential or irregularly sampled paths, but they do not by themselves specify a forward dynamical or stochastic model. NODEFit complements these approaches by learning a vector field, and optionally a diffusion, that can be queried at arbitrary times and used for continuous-time extrapolation.

PySINDy [10] recovers ODEs by sparse regression onto a user-specified library of candidate terms [11]. As with `curve_fit`, that library is a predetermined basis, and if the true vector field is not in it, identification fails. NODEFit, built on PyTorch, instead parameterizes f_θ and g_θ with neural networks, so the form need not be known in advance. Differentiable solvers already exist [3], [4], and, to the author's knowledge, NODEFit is the first high-level Python package that wraps those libraries (`torchdiffeq` and `torchsde`) to fit both Neural ODEs and Neural SDEs to measured time series without such a candidate library.

NODEFit packages these ideas for practitioners. After installing the package, a user passes time-series data, specifies compact drift and diffusion networks, and calls `train` and

[1] Stochastic differential equations and Itô's lemma originate with the Japanese mathematician Kiyosi Itô (1915–2008), who in the 1940s developed a rigorous stochastic integral and, in 1951, the change-of-variable formula now known as Itô's lemma, the stochastic counterpart of the chain rule. These tools later became foundational in mathematical finance: Fischer Black, Myron Scholes, and Robert Merton used them in the early 1970s to derive the Black–Scholes–Merton model for option pricing. Itô spent much of his career at Kyoto University and is widely regarded as the founder of modern stochastic analysis.

`extrapolate`. The package owns the training loop, observation batching, and coordinated updates of drift and diffusion parameters, so the user does not assemble adjoint code, Brownian-tree bookkeeping, or separate optimizer groups. That interface is what distinguishes NODEFit from calling `torchdiffeq` or `torchsde` directly. Benchmarks in this paper show that the resulting continuous-time fits recover the underlying kinetics and extrapolate reliably on the motivating example above, which polynomial, template-based, and discrete-time alternatives struggle to match without prior knowledge of the governing equations.

The remainder of the paper is organized as follows. Methods defines the continuous-time and stochastic models, training objective, and adjoint backpropagation. Implementation describes the solver dependencies, memory-saving methods, installation, and example API. Results evaluates NODEFit's PyTorch-based Neural ODEs and SDEs on deterministic and stochastic benchmarks, including a reconstruction of the classical Lotka–Volterra population model, recovery of noisy dynamics, and clinical case and cohort studies modeling cardiac arrhythmias on PTB-XL ECG data[2] (see Section 15). The Appendix provides additional background on the stochastic calculus used by the SDE adjoint.

2. Methods

2.1. Continuous-time model

We model the latent state evolution as:

$$\frac{dy(t)}{dt} = f_\theta(y(t), t) \tag{1}$$

where $y(t) \in \mathbb{R}^d$ denotes the system state vector at continuous time t, and f_θ is a neural network parameterized by weights θ that approximates the unknown continuous vector field (governing dynamics).

2.2. Stochastic dynamics

For noisy systems, NODEFit supports Neural SDEs [4], [5]:

$$dy = f_\theta(y, t)dt + g_\theta(y, t)dW_t \tag{2}$$

where y is the state vector, t is continuous time, $f_\theta(y, t)$ is the drift neural network capturing deterministic trends, $g_\theta(y, t)$ is a neural network parameterizing the diffusion (state-dependent noise amplitude), and W_t represents a standard Wiener process (Brownian motion).

2.3. Training Objective

Given observations (t_i, y_i), NODEFit minimizes the mean squared error:

$$\mathcal{L} = \sum_i | \, | \, y_i - \hat{y}(t_i) \, | \, |^2 \tag{3}$$

where $\hat{y}(t)$ is the predicted state trajectory produced by the ODE or SDE solver, and $| \, | \cdot | \, |$ denotes the standard Euclidean norm.

2.4. Backpropagation

A differentiable ODE solver computes the forward trajectory, and gradients are propagated through the integration process. To derive the adjoint equation from first principles, consider a small time step ε. Using a first-order Taylor expansion:

$$y(t + \varepsilon) = y(t) + \varepsilon f(y(t), t, \theta) + O(\varepsilon^2) \tag{4}$$

[2] PTB-XL version 1.0.3 is available from PhysioNet at https://physionet.org/content/ptb-xl/1.0.3/.

By the chain rule, the sensitivity of the loss $\mathcal{L}$ with respect to the state at time t, defined as the adjoint state $a(t) = \partial\mathcal{L}/\partial y(t)$, is:

$$a(t) = \frac{\partial \mathcal{L}}{\partial y(t)} = \frac{\partial \mathcal{L}}{\partial y(t+\varepsilon)} \frac{\partial y(t+\varepsilon)}{\partial y(t)} \tag{5}$$

Substituting the expansion from (4) into (5):

$$a(t) = a(t+\varepsilon)\left(I + \varepsilon \frac{\partial f(y(t), t, \theta)}{\partial y(t)}\right) \tag{6}$$

where I is the identity matrix. Rearranging and taking the limit $\varepsilon \to 0$ yields the adjoint ODE:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(y(t), t, \theta)}{\partial y} \tag{7}$$

Here, $a(t)^T$ denotes the transpose of the adjoint state vector (a row vector representing how sensitive the loss is to perturbations in each state dimension), and $\frac{\partial f(y(t),t,\theta)}{\partial y}$ is the Jacobian matrix representing the partial derivatives of the vector field f with respect to the state y. Solving this linear differential equation backwards in time enables gradient computation with constant memory cost, instead of storing every intermediate step in memory to apply the chain rule in the backward pass.

3. Implementation

NODEFit is implemented as an open-source Python package built on top of the PyTorch [12] ecosystem. It leverages specialized libraries to handle the numerical integration and gradient computation required for training Neural ODEs and SDEs. By abstracting these complexities, NODEFit makes it remarkably easy to fit complex time-series data to governing differential equations. All plots in this paper were generated using Matplotlib [13].

3.1. Core Dependencies

The efficiency and scalability of NODEFit rely on two primary libraries: `torchdiffeq` and `torchsde`.

3.1.1. *torchdiffeq*

For Ordinary Differential Equations, NODEFit utilizes `torchdiffeq` [3]. This library provides a suite of differentiable ODE solvers. A critical feature of `torchdiffeq` is its support for the **adjoint sensitivity method**. Unlike standard backpropagation through the solver's internal operations (which has a memory cost that scales with the number of solver steps), the adjoint method allows for gradient computation with constant memory cost by solving an augmented ODE backwards in time. The adjoint state $a(t) = \partial\mathcal{L}/\partial y(t)$ follows the dynamics:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(y(t), t, \theta)}{\partial y} \tag{8}$$

The gradient with respect to the model parameters θ is then computed by integrating backwards from the final time t_1 to the initial time t_0:

$$\frac{d\mathcal{L}}{d\theta} = -\int_{t_1}^{t_0} a(t)^T \frac{\partial f(y(t), t, \theta)}{\partial \theta} dt \tag{9}$$

where $\frac{\partial f(y(t),t,\theta)}{\partial \theta}$ represents the partial derivative (Jacobian) of the vector field with respect to the neural network parameters θ.

This enables the training of complex models on large time-series datasets that would otherwise be computationally prohibitive. The memory efficiency stems from the fact that the adjoint method does not require storing intermediate states $y(t)$ from the forward pass.

Instead, the original ODE is solved backwards in time alongside the adjoint equation, reconstructing the state $y(t)$ on the fly. This reduces the memory complexity from $O(N)$, where N is the number of solver steps, to $O(1)$ relative to the trajectory length, at the possible cost of sensitivity to errors in the backpropagation.

3.1.2. *torchsde*

For stochastic systems, NODEFit integrates `torchsde` [4]. Stochastic Differential Equations present unique challenges, particularly in ensuring consistent Brownian motion across multiple steps and handling the nuances of stochastic calculus.

Similar to the deterministic case, the stochastic adjoint sensitivity method avoids storing the full trajectory, enabling gradient computation with constant memory cost. However, SDEs require consistent noise across both forward and backward passes. `torchsde` achieves this through a **Virtual Brownian Tree**, which allows for the exact reconstruction of the Brownian motion W_t at any time point using a fixed seed. By reconstructing both the state and the noise during the backward pass, the memory cost remains constant even for complex stochastic trajectories. For the complete first-principles derivation of the stochastic adjoint and the associated stochastic calculus, see the Appendix.

3.1.3. *Memory Efficiency Trade-offs*

The primary advantage of the adjoint methods leveraged by NODEFit is the reduction in memory overhead. The mathematical adjoint formulation (integrating the adjoint state backwards in time) and the Virtual Brownian Tree noise reconstruction are implemented within the underlying PyTorch differential equation ecosystem (`torchdiffeq` and `torchsde`). The following table summarizes the theoretical scaling and computational trade-offs between standard naive backpropagation and the adjoint-based sensitivity methods in the PyTorch libraries `torchdiffeq` and `torchsde`.

Table 1. *Comparison of memory efficiency and computational trade-offs between naive backpropagation and the adjoint sensitivity method in the PyTorch libraries* `torchdiffeq` *and* `torchsde`*.*

Feature	Naive Backprop	Adjoint Method (PyTorch libraries)
Intermediate States	Stored in memory	Reconstructed on the fly
Memory Scaling	$O(N)$ (Linear with steps)	$O(1)$ (Constant with steps)
Noise (SDEs)	Must be stored for every step	Regenerated via Virtual Brownian Tree
Computational Trade-off	Faster (no reconstruction)	Slower (requires solving backwards)
Error Sensitivity	Standard backpropagation	Possible sensitivity to errors in backpropagation

While the $O(1)$ asymptotic memory scaling is delivered by the underlying PyTorch solvers, NODEFit's contribution lies in wrapping these complex mathematical engines into a single training API, coordinated drift and diffusion updates, observation batching, and the `FastNeuralSDE` execution path used for long series and large cohorts.

3.2. Performance Optimizations

To handle larger datasets and more complex trajectories, NODEFit includes an optimized implementation (`FastNeuralSDE`) that inherits from the base `NeuralSDE` and `SDE` classes. Key optimizations include:

1. **In-place tensor allocation:** Using `y.new_full` for state-time concatenation in drift/diffusion functions (f and g), avoiding redundant tensor creation and device transfers during solver evaluations.
2. **Single-precision arithmetic:** Operating consistently in `float32` rather than `float64`.
3. **Solver configuration & loop efficiency:** Enforcing a fixed time step `dt = 0.1` for Euler integration and pre-allocating repeated target batch tensors to avoid redundant allocations during loss computation.

Benchmarking against the base implementation on synthetic trajectories demonstrates an empirical speedup of $138\times$ (reducing per-epoch training time from 1.84 s down to 0.013 s on the reference CPU), bringing 1,000 epochs of Neural SDE training down to under 40 seconds. While base `NeuralSDE` serves as a simple double-precision reference for small prototypes, `FastNeuralSDE` is the recommended default for production workflows, longer series, and large cohort training.

3.3. Reproducibility and wall-clock timings

The example scripts were run once on a 14-inch 2021 MacBook Pro with an Apple M1 Pro chip and 16 GB of memory, running macOS Tahoe 26.3.1. The wall-clock times include model initialization, training, extrapolation, and plot generation, and are provided as reproducibility information rather than as a benchmark. They can vary with software versions and system load.

Table 2. *Wall-clock times for the example scripts on the reference laptop.*

Example script	Training epochs	Wall-clock time
`simple_ode_only.py`	1000	50 s
`simple_sde_only.py`	1000	36 s
`sde_ode_trajectory.py`	500	32 s

3.4. Installation

NODEFit can be installed via `pip`:

```
pip install nodefit
```

- **Python and PyTorch Versions**: NODEFit requires Python ≥ 3.8 and PyTorch ≥ 1.12 (compatible with PyTorch 2.x). Core dependencies, including `torchdiffeq`, `torchsde`, `numpy` and `matplotlib`, are installed automatically.
- **Hardware Requirements**: A dedicated GPU is optional. Because the neural vector fields and diffusion networks for typical continuous-time dynamical systems are relatively compact (often 1–3 shallow layers with modest hidden dimensions), CPU-only usage is fully supported and fast (e.g., all benchmark models in this paper train in under a minute on a standard laptop CPU). When available, GPU acceleration (via CUDA or Apple Silicon MPS) can be utilized seamlessly by placing tensors and modules on the target device using standard PyTorch semantics.

3.5. Sample Code

The following example fits a Neural SDE to a noisy 2D trajectory. Times and states are ordinary NumPy arrays, with the user supplying the two networks and NODEFit handling the batching, the training loop, and extrapolation. A domain scientist does not need to write a differentiable solver, an adjoint backward pass, or separate parameter groups for drift and diffusion.

```python
import numpy as np
import torch
import torch.nn as nn
from nodefit.neural_sde import NeuralSDE

# Set seed for reproducibility
torch.manual_seed(0)
np.random.seed(0)

# Generate noisy synthetic time-series data (2D)
t = np.linspace(0, 5, 50)
y1 = 1.0 + 2.2 * (1 - np.exp(-0.5 * t)) + np.random.normal(0, 0.1, 50)
y2 = 1.0 + 0.6 * (1 - np.exp(-0.5 * t)) + np.random.normal(0, 0.1, 50)
data = np.stack([y1, y2], axis=1)

# Define the drift network (f_theta)
# Input: (t, y), so input_dim = 1 + 2 = 3
drift_nn = nn.Sequential(
    nn.Linear(3, 20),
    nn.Tanh(),
    nn.Linear(20, 2)
).double()

# Define the diffusion network (g_theta)
diffusion_nn = nn.Sequential(
    nn.Linear(3, 20),
    nn.Tanh(),
    nn.Linear(20, 2)
).double()

# Initialize and train the SDE model: fitting data is as simple as passing the observations
sde = NeuralSDE(drift_nn, diffusion_nn, t, data, batch_size=20)
sde.train(num_epochs=500, print_every=100)

# Extrapolate to future time points
extrapolated = sde.extrapolate(tf=8, npts=40)
```

4. Results

We evaluated NODEFit's PyTorch-based Neural ODEs and SDEs on both deterministic and stochastic benchmarks drawn from the motivating scenario in the Introduction: coupled states approaching saturation with optional noise. The models were trained using the Adam optimizer with default learning rates. In each case, the goal is not merely to interpolate scattered points but to recover a coherent evolution law that extrapolates beyond $t = 5$. Figure 1 through Figure 4 illustrate settings where conventional curve fitting would require the correct functional template *a priori*, and where treating noise as homoscedastic regression error would misrepresent uncertainty during the forecast.

4.1. Neural ODE Results

For the deterministic case, we first fit the same training data with `scipy.optimize.curve_fit` [6], using a cubic polynomial ($y = a + bt + ct^2 + dt^3$) fitted independently to each state. The extra flexibility tracks the training window closely, but without a saturation mechanism the extrapolation past $t = 5$ inflects upward rather than leveling off (Figure 1, dashed curves). We then trained a Neural ODE for 1000 epochs on the same 2D system. The learned flow matches the saturating trajectories and continues smoothly toward steady state beyond the training window, without specifying the functional form in advance.

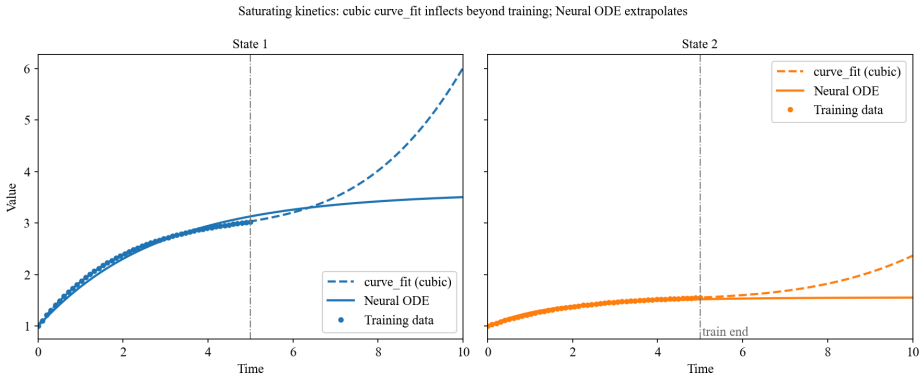


Figure 1. *Baseline failure and Neural ODE success on coupled saturating kinetics. Dashed curves show* `curve_fit` *with a cubic polynomial template; solid curves show the Neural ODE integrated through* $t = 10$*. The vertical dotted line marks the end of training data* ($t = 5$)*.*

4.2. Classical reconstruction: Lotka–Volterra dynamics

To demonstrate an application with a known scientific interpretation, we reconstruct the Lotka–Volterra predator–prey model from partial observations. This model was developed independently by Lotka and Volterra to describe interacting populations [14]. With $x(t)$ denoting prey and $z(t)$ denoting predator population, the classical equations are

$$\begin{aligned} \frac{dx}{dt} &= \alpha x - \beta xz, \\ \frac{dz}{dt} &= \delta xz - \gamma z. \end{aligned} \tag{10}$$

The first term in each equation describes unconstrained population growth or decay, while the interaction terms describe predation. We generated a trajectory with $(\alpha, \beta, \gamma, \delta) = (1.5, 1.0, 3.0, 1.0)$ and initial state $(x(0), z(0)) = (1.5, 1.0)$. Only observations up to $t = 6$ were supplied to a structured Neural ODE whose four positive coefficients were learned jointly with the trajectory. The learned values were $(1.52, 1.01, 2.94, 0.98)$, close to the generating system. As shown in Figure 2, the model reconstructs both the oscillatory population dynamics and the closed phase portrait, then extrapolates the cycle to $t = 12$. This example illustrates how NODEFit can recover an interpretable continuous-time law from incomplete observations, and in applications where the governing terms are not known, the same interface can instead use an unconstrained neural drift.

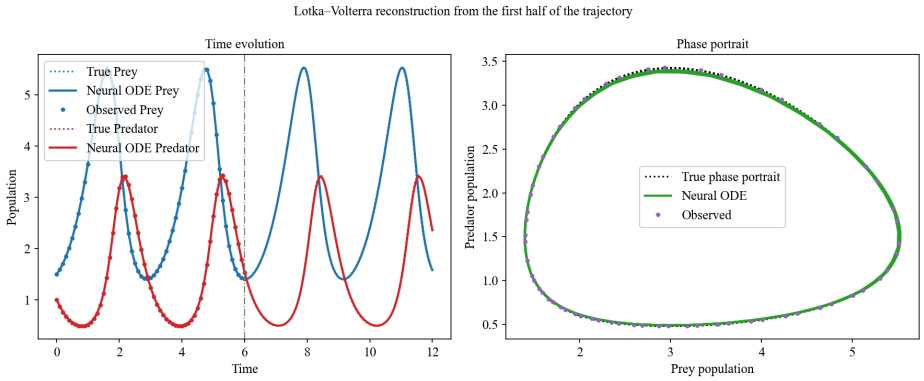


Figure 2. *Lotka–Volterra reconstruction from observations through* $t = 6$*. Dotted curves show the classical system used to generate the data, solid curves show the learned structured Neural ODE, and points show the observations.*

4.3. Neural SDE Results

In the stochastic case, we trained a Neural SDE for 300 epochs. The diffusion network learns to capture the noise characteristics of the data. Figure 3 illustrates the mean prediction along with the standard deviation (shaded regions) across multiple trajectories. The model successfully captures the underlying trend while quantifying the uncertainty, which increases during extrapolation.

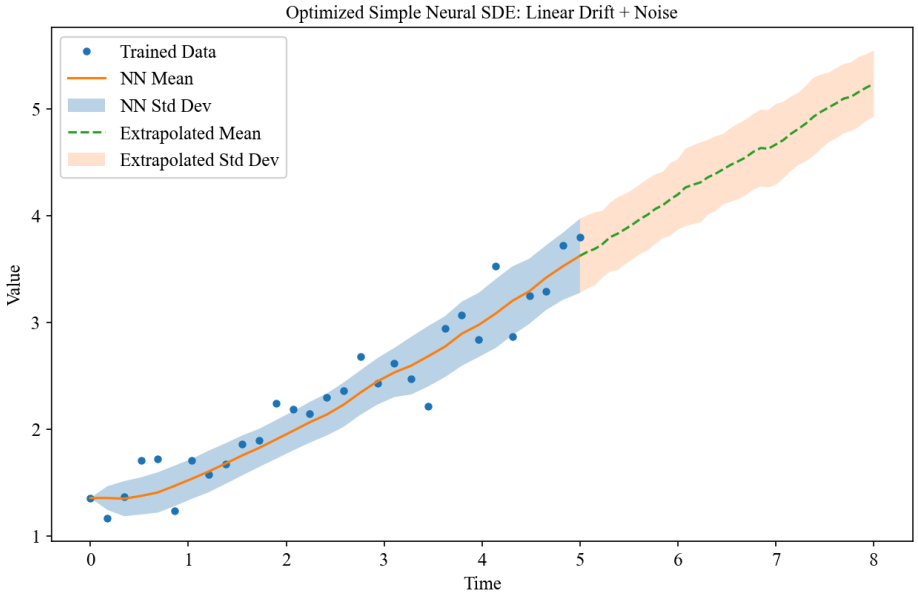


Figure 3. *Neural SDE fit and extrapolation. The shaded regions represent the standard deviation across 10 trajectories, capturing the learned diffusion process.*

4.4. Fitting Complex Dynamics

We also tested NODEFit's PyTorch-based Neural SDE on a more complex 2D system with noise. The underlying theoretical trajectories for this system are governed by the following equations:

$$\begin{aligned} y_1(t) &= 1.0 + 2.2(1 - e^{-0.5t}) \\ y_2(t) &= 1.0 + 0.6(1 - e^{-0.5t}) \end{aligned} \tag{11}$$

The model was tasked with learning these underlying dynamics and providing robust extrapolations. As shown in Figure 4, the Neural SDE successfully recovers the mean trajectory, closely matching the underlying theoretical dynamics even in the presence of noise. The comparison between the theoretical mean and the predicted mean demonstrates the model's ability to filter out stochastic fluctuations and capture the true governing laws.

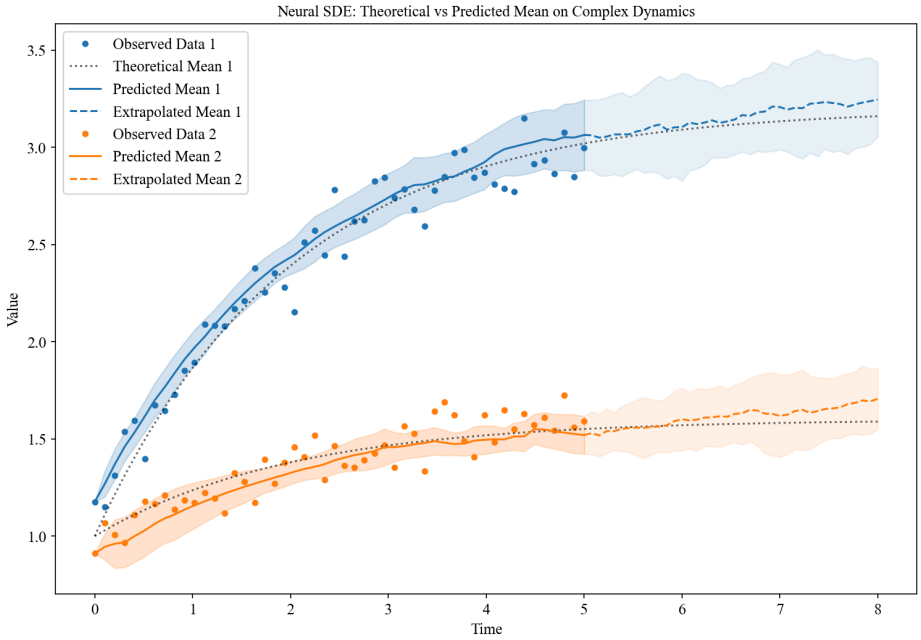


Figure 4. *Neural SDE fit on a complex 2D trajectory. The dotted black line represents the underlying theoretical trajectory, while the solid lines and shaded regions show the predicted mean and uncertainty. The model captures the multi-dimensional dynamics and provides reliable extrapolations.*

4.5. Clinical Case Study: Stochastic Heart Rate Dynamics in Atrial Fibrillation

To evaluate NODEFit's PyTorch-based Neural SDE on a clinical research dataset, we analyzed an electrocardiogram (ECG) recording from the open-source PTB-XL database [15], [16]. In patients diagnosed with cardiac arrhythmias such as atrial fibrillation (AFIB), the sinoatrial node fires irregularly, producing stochastic fluctuations in the beat-to-beat interval (RR) and instantaneous heart rate alongside baseline sensor and environmental noise.

An ECG measures the cardiac electrical conduction cycle, characterized by the P-Q-R-S-T sequence: the P wave reflects atrial depolarization, the QRS complex corresponds to rapid ventricular depolarization, and the T wave represents ventricular repolarization[3].

Clinical interpretation typically pairs a limb rhythm lead with a left-precordial chest lead, so we evaluated Lead II and Lead V5 of PTB-XL record #16834 (Figure 5). Because both leads were recorded simultaneously from the same patient on the same multi-channel device, the physiological heartbeats occur synchronously, and R-peak detection recovers the identical 18 irregular beats across both channels. However, Lead V5 is positioned directly over the left ventricle, yielding taller QRS voltage amplitudes (0.82 mV peak vs. 0.56 mV on Lead II) and a cleaner signal-to-noise ratio than Lead II.

[3]For standard clinical definitions of the ECG waveform, cardiac conduction cycles, and lead placements, see Goldberger et al. [17].

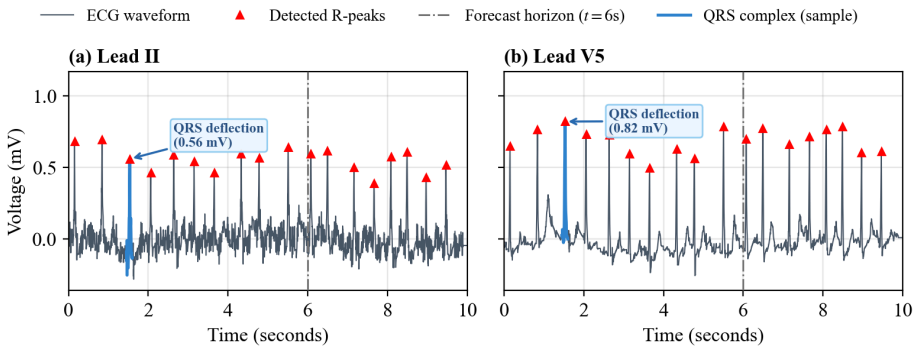


Figure 5. *Ten-second clinical ECG from PTB-XL record #16834 (500 Hz). (a) Lead II. (b) Lead V5. Red triangles mark detected R-peaks; blue highlighted segments and arrows indicate a representative QRS deflection (*0.56 mV *on Lead II vs.* 0.82 mV *on Lead V5); the vertical dash-dotted line at* $t = 6$ s *marks the forecast horizon.*

Traditional discrete-time autoregressive models struggle with continuous physiological processes because observation intervals in biological time series vary continuously. In contrast, continuous-time Neural SDEs naturally operate over arbitrary time coordinates. Instantaneous heart rate is derived from RR intervals, so the two leads describe the same arrhythmia when peaks are detected reliably. For each lead we interpolated a dense continuous heart-rate trajectory, trained a compact Neural SDE on the first 6 seconds ($t \leq 6$ s, $N = 57$), and held out the remainder ($t > 6$ s, $N = 33$), as shown in Figure 6. As a deterministic baseline, `scipy.optimize.curve_fit` with a cubic polynomial was fitted over the same training window.

The cubic polynomial overfits local curvature during training and inflects rigidly upward past $t = 6$ s without bounding physiological extremes or quantifying uncertainty. NODEFit's Neural SDE, parameterized with bounded multi-harmonic phase coordinates ($K = 3$ harmonic orders spanning the underlying ~ 0.35 Hz autonomic cycle) and state-dependent drift, reconstructs the multi-cycle respiratory oscillation throughout the training window without unbounded extrapolation blowup. The diffusion network supplies a stochastic envelope of about $\pm 1\sigma \approx 9$ BPM and $\pm 2\sigma \approx 18$ BPM on both leads. Held-out RMSE is 24.4 BPM (Lead II) and 25.0 BPM (Lead V5) for the SDE mean, versus 29.5 BPM and 26.5 BPM for the cubic baseline. About 45-48% of held-out samples fall inside the $\pm 1\sigma$ band and 61% inside $\pm 2\sigma$, with the large late excursion near $t \approx 8$ s sitting at the edge of the envelope. The clinically useful contrast is the same on both leads, with the SDE supplying an oscillatory forecast and a quantified uncertainty band, whereas a polynomial template cannot.

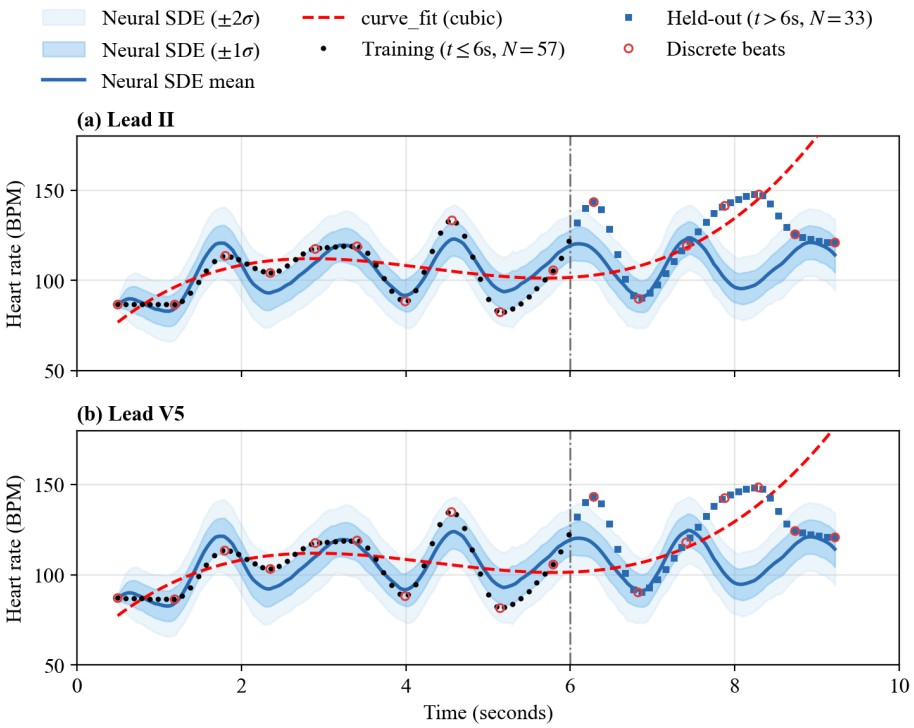


Figure 6. *Continuous-time instantaneous heart rate from record #16834. (a) Lead II. (b) Lead V5. Black dots are training observations ($t \leq 6$ s, $N = 57$), blue squares are held-out future observations ($t > 6$ s, $N = 33$), open red circles are discrete beats, the red dashed curve is the* `curve_fit` *cubic baseline, and the solid blue curve with shaded bands is the Neural SDE mean and diffusion ($\pm 1\sigma$, $\pm 2\sigma$).*

4.6. Large-Scale Cohort Benchmark on PTB-XL

To assess whether these continuous-time stochastic dynamics generalize across diverse clinical populations, we scaled the evaluation to a cohort sampled from 2,000 unique patient records in PTB-XL. Because limb lead II and precordial lead V5 exhibit distinct signal morphologies and noise characteristics, we analyzed both leads independently across the cohort. Approximately 55% of the cohort comprises patients with normal sinus rhythm (`NORM`), while 45% presents diagnostic abnormalities including atrial fibrillation, conduction blocks, and myocardial infarctions.

Out of the 2,000 patient records, 1,997 records successfully yielded valid continuous heart-rate trajectories across one or both leads, resulting in 3,948 individual lead samples (1,968 on Lead II and 1,980 on Lead V5). Exactly 3 patient records were discarded by the preprocessing pipeline due to severe physiological bradycardia and conduction blocks where 10-second recordings contained insufficient beat occurrences across both leads. Furthermore, because R-peak detection and physiological RR-interval criteria (the beat-to-beat time between successive R-peaks; $0.25 \text{ s} \leq RR \leq 2.5 \text{ s}$) were evaluated independently on each lead, 29 samples from Lead II and 17 samples from Lead V5 did not meet the beat-count threshold for spline interpolation, yielding 3,948 total usable samples. Each valid lead trajectory was extracted and fitted with the same NODEFit protocol ($t \leq 6$ s training, $t > 6$ s held-out forecasting), reusing one drift–diffusion architecture rather than a per-record solver script. Table 3 summarizes the aggregate forecasting error and empirical uncertainty coverage across all 3,948 samples.

Table 3. *Forecasting performance and uncertainty coverage across 3,948 lead samples from 1,997 unique PTB-XL patients, stratified by lead and diagnostic class.*

Lead	Cohort Stratum	N Samples	Neural SDE RMSE (BPM)	Cubic `curve_fit` RMSE (BPM)	Within 1σ (%)	Within 2σ (%)	Mean Diffusion $\pm 1\sigma$ (BPM)
Lead II	All	1968	10.43 ± 15.62	51.78 ± 111.42	88.5%	94.3%	16.4 BPM
Lead II	Normal	1083	8.02 ± 12.56	35.99 ± 86.57	93.4%	97.1%	16.2 BPM
Lead II	Arrhythmia / Abnormal	885	13.38 ± 18.28	71.12 ± 133.30	82.6%	91.0%	16.8 BPM
Lead V5	All	1980	8.02 ± 11.22	35.58 ± 70.21	91.9%	96.7%	16.5 BPM
Lead V5	Normal	1083	6.34 ± 8.43	26.59 ± 52.89	95.9%	98.5%	16.2 BPM
Lead V5	Arrhythmia / Abnormal	897	10.06 ± 13.58	46.44 ± 85.40	87.0%	94.6%	16.8 BPM

Across all cohort subsets, NODEFit's Neural SDE significantly outperforms the standard polynomial `curve_fit` baseline, reducing held-out forecast RMSE by a factor of 4.4 to 5.0 (10.43 BPM vs. 51.78 BPM on Lead II; 8.02 BPM vs. 35.58 BPM on Lead V5). The performance advantage is particularly pronounced in abnormal and arrhythmic patients, where the deterministic polynomial rapidly diverges on non-stationary trajectories (71.12 BPM error), while the Neural SDE constrains drift and maintains bounded error (13.38 BPM).

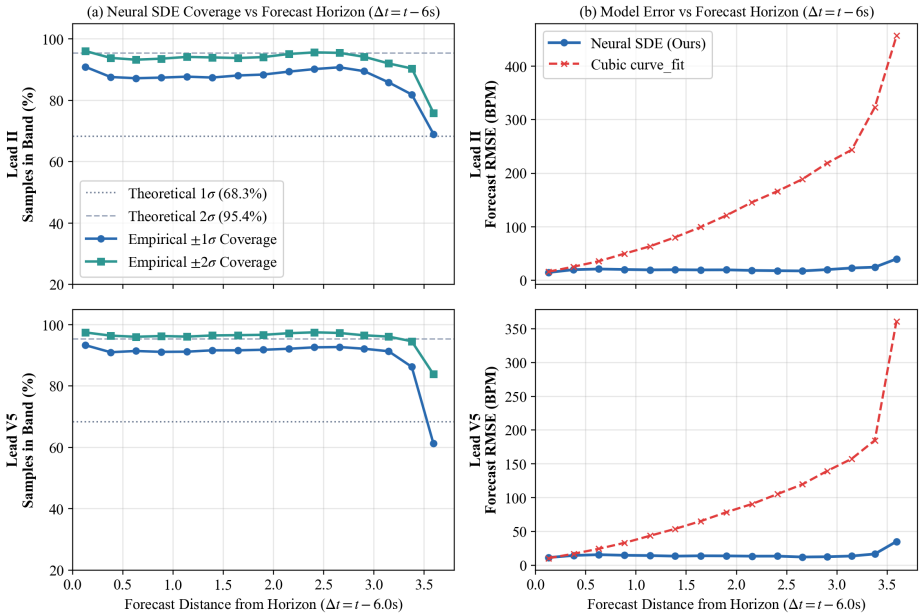


Figure 7. *Cohort-wide forecasting reliability as a function of temporal distance from the forecast horizon ($\Delta t = t - 6$ s) across 3,948 lead samples (1,968 on Lead II, 1,980 on Lead V5) from 1,997 PTB-XL patients. Top row: Lead II; bottom row: Lead V5. (a) Empirical percentage of held-out observations encapsulated by the learned Neural SDE $\pm 1\sigma$ and $\pm 2\sigma$ diffusion envelopes compared against theoretical Gaussian coverage limits (68.3% and 95.4%). (b) Out-of-sample RMSE across forecast distance, demonstrating stable error bounds for NODEFit versus cubic polynomial runaway.*

To examine how predictive uncertainty evolves into the unobserved future, Figure 7 tracks model performance as a function of distance $\Delta t = t - 6$ s from the forecast horizon:

1. **Uncertainty Calibration:** As shown in Figure 7 (column a), the learned diffusion network produces well-calibrated confidence bands throughout the forecast window. Across both Lead II and Lead V5, empirical $\pm 2\sigma$ coverage remains between 94% and 97%, closely aligning with the theoretical 95.4% two-standard-deviation Gaussian boundary.
2. **Extrapolation Stability:** As shown in Figure 7 (column b), standard polynomial regression exhibits severe runaway divergence beyond $\Delta t > 1.5$ s, with average forecast errors exceeding 350-400 BPM near the end of the 10-second recording. In contrast, the Neural SDE's learned vector field maintains flat, physiologically bounded error profiles throughout the entire extrapolation interval.

These cohort-wide results confirm that continuous-time Neural SDEs provide robust trajectory modeling and calibrated uncertainty quantification across large, heterogeneous clinical datasets.

4.7. Future work with the ECG dataset

In our future work, we will test our software against additional benchmarks that are better suited for ECG signal comparisons. Examples of functions include: sines, cosines, dying exponentials, constants, negative powers, etc., as suggested by a reviewer.

5. Conclusion

NODEFit offers a user-friendly and time-saving tool for fitting continuous-time models to time-series data. By leveraging PyTorch's Neural ODE and SDE libraries, it enables the discovery of governing laws from observations, bridging the gap between machine learning and physical modeling.

For practitioners deciding whether to use NODEFit's PyTorch-based Neural ODEs and SDEs, the central question is whether the data plausibly arise from a smooth, Markovian continuous-time process. If polynomial regression, splines, `scipy.optimize.curve_fit` with a hand-chosen template, or PySINDy with a library that already contains the true terms produces stable fits and credible extrapolations, a Neural ODE may be unnecessary. PySINDy is the better choice when an interpretable sparse equation in a known basis is the goal. Consider NODEFit when those tools leave systematic residuals, extrapolations diverge from physical expectations, or the functional form of the dynamics is unknown, including stochastic processes where a diffusion network is needed. Template-based fits must guess a closed-form expression or a candidate library, whereas a Neural ODE learns a single vector field coupling all states.

When observations are noisy, ask whether the noise reflects measurement error alone or variability intrinsic to the process. Ordinary least squares and deterministic Neural ODEs treat scatter as something to be averaged out. If uncertainty grows with state magnitude or if extrapolated forecasts should carry confidence intervals, a Neural SDE is the more appropriate model. The diffusion network learns state-dependent noise alongside the drift.

The reason to use NODEFit rather than those PyTorch libraries directly is the wrapping workflow. NODEFit accepts NumPy times and states, standardizes the training loop, batches observations, and keeps drift and diffusion on coordinated optimizers so the user does not maintain those parameter groups by hand. The API is built for the actions that dominate applied fitting, such as swapping a network's width or depth, retraining, and comparing extrapolations without changing solver settings. That matters most for large batch studies with extensive hyperparameter searches, such as the PTB-XL cohort in Section 15, which applied one protocol to nearly 2,000 records.

NODEFit is intended for scientists who need a Neural ODE or SDE fit without assembling adjoint code, Brownian-tree bookkeeping, and device transfers themselves. `FastNeuralSDE` is the NODEFit-specific execution path that reduces that cost further, with a measured $138\times$ speedup relative to the package's own reference `NeuralSDE` class.

6. GenAI Policy

Portions of this work were assisted using a generative AI tool, Cursor. The tool was used for drafting text, refining language, or generating code suggestions. All outputs were reviewed, verified, and revised by the author, who takes full responsibility for the accuracy and integrity of the final content.

7. Appendix: Stochastic Calculus and the Adjoint Method

The derivation of the stochastic adjoint sensitivity method relies on the choice of stochastic integral. This appendix provides the necessary background on the Itô and Stratonovich formulations.

7.1. Martingales

A stochastic process M_t is a **martingale** if its expected future value, given all the information available up to the current time t, is exactly its current value:

$$E[M_s \mid \text{information up to time } t] = M_t \tag{12}$$

Intuitively, a martingale represents a "fair game" where there is no systematic tendency to increase or decrease. This property is fundamental for ensuring that a stochastic model does not have an unintended "hidden" drift.

7.2. Itô vs. Stratonovich Integrals

For a stochastic process $y(t)$ governed by a diffusion term $g(y,t)$, the integral with respect to Brownian motion W_t can be defined in two primary ways depending on the evaluation point within a time interval $[t, t+\Delta t]$:

1. **Itô Integral** (denoted $g\,dW_t$): Evaluates the integrand at the **left endpoint** t. Because the integrand is evaluated before the noise increment dW_t occurs, they are independent. Since Brownian increments have zero mean, the expected change is zero, making the Itô integral a **martingale**. This makes it the standard choice for modeling physical systems where noise should not introduce systematic drift. However, it does not follow the standard chain rule of calculus.
2. **Stratonovich Integral** (denoted $g \circ dW_t$): Evaluates the integrand at the **midpoint** $t + \Delta t/2$. This creates a correlation between the integrand and the noise, which introduces a drift and causes the integral to **lose the martingale property**. However, its primary advantage is that it **obeys the standard rules of calculus** (chain rule, product rule), which simplifies the derivation of adjoint equations.

7.3. Stochastic Adjoint Derivation

To derive the stochastic adjoint from first principles, we consider the SDE in Stratonovich form (denoted by the $\circ$ operator). The Stratonovich integral evaluates the integrand at the midpoint of the interval, $g(y,t) \circ dW_t \approx g\big(y_{t+\Delta t/2}, t + \Delta t/2\big)\Delta W_t$. This choice ensures that the SDE obeys the standard rules of calculus:

$$dy = f(y,t,\theta)dt + g(y,t,\theta) \circ dW_t \tag{13}$$

Consider a small time step Δt. The state update is approximately:

$$y(t + \Delta t) \approx y(t) + f(y(t), t, \theta)\Delta t + g(y(t), t, \theta)\Delta W_t \tag{14}$$

Following the same chain rule logic as in (5), the sensitivity of the loss with respect to the state at time t is:

$$a(t) = \left(\frac{\partial y(t + \Delta t)}{\partial y(t)}\right)^T a(t + \Delta t) \tag{15}$$

Substituting the derivative of (14):

$$a(t) \approx \left(I + \frac{\partial f}{\partial y}^T \Delta t + \sum_j \frac{\partial g_j}{\partial y}^T \Delta W_{t,j}\right) a(t + \Delta t) \tag{16}$$

In the limit $\Delta t \to 0$, this yields the adjoint SDE in Stratonovich form:

$$da(t) = -\left(\frac{\partial f}{\partial y}\right)^T a(t)dt - \sum_j \left(\frac{\partial g_j}{\partial y}\right)^T a(t) \circ dW_{t,j} \tag{17}$$

When converted back to Itô form for numerical stability and implementation, this introduces the **Stratonovich-to-Itô correction** term:

$$da(t) = -\left[a(t)\frac{\partial f}{\partial y} - \sum_j \left(a(t)\frac{\partial g_j}{\partial y}\right)\frac{\partial g_j}{\partial y}\right]dt - \sum_j \left(a(t)\frac{\partial g_j}{\partial y}\right)dW_t \tag{18}$$

where g_j are the columns of the diffusion matrix g.

7.4. Stratonovich-to-Itô Conversion and Correction

The relationship between a Stratonovich SDE ($dy = f_s dt + g \circ dW_t$) and an Itô SDE ($dy = f_i dt + g dW_t$) is given by the conversion formula:

$$f_i(y, t) = f_s(y, t) + \frac{1}{2}\sum_j \left(\frac{\partial g_j(y, t)}{\partial y}\right) g_j(y, t) \tag{19}$$

where g_j are the columns of the diffusion matrix. The second term is the **Stratonovich-to-Itô correction**. In `torchsde`, derivations are performed in the Stratonovich framework to leverage standard calculus, while numerical solvers often operate in the Itô framework, requiring the explicit inclusion of this correction term in the drift dynamics.

It is important to note that for the adjoint SDE (Equation (18)), the $1/2$ factor is absent. This is because the correction term for the adjoint state $a(t)$ arises from the interaction between the forward state $y(t)$ and the adjoint variable. When converting the augmented system (y, a) to Itô form, the resulting drift correction for $a(t)$ consists of two identical terms from the Stratonovich expansion that sum to unity, effectively canceling the $1/2$ coefficient found in the standard forward conversion formula [4].

7.4.1. *Mathematical Derivation*

To see this mathematically from first principles, we use index notation where y_i and a_i denote the components of the state and adjoint (row) vectors.

1. General Midpoint Rule and Conversion: For any Stratonovich stochastic term $G(X, t) \circ dW$ with state vector $X(t)$, the integral over a small time step $[t, t + \Delta t]$ is evaluated at the midpoint $\bar{X} = X(t) + \frac{1}{2}\Delta X$:

$$G_{i,j}(\bar{X}, t)\Delta W_j \approx \left[G_{i,j}(X(t), t) + \frac{1}{2}\sum_k \frac{\partial G_{i,j}}{\partial X_k}\Delta X_k\right]\Delta W_j \tag{20}$$

Substituting the leading-order stochastic increment $\Delta X_k \approx \sum_l G_{k,l}\Delta W_l$ and applying the Brownian quadratic variation ($\Delta W_l \Delta W_j = \delta_{lj}\Delta t$):

$$\sum_j G_{i,j}(X,t) \circ dW_j = \sum_j G_{i,j}(X,t) dW_j + \frac{1}{2} \sum_j \sum_k \frac{\partial G_{i,j}}{\partial X_k} G_{k,j} dt \tag{21}$$

This fundamental midpoint relation provides the Stratonovich-to-Itô drift correction for any state vector X.

2. Forward Stratonovich Drift: Applying this general relation directly to the forward state $X = y$ with diffusion matrix $g(y,t)$:

$$\sum_j g_{i,j}(y,t) \circ dW_j = \sum_j g_{i,j}(y,t) dW_j + \frac{1}{2} \sum_j \sum_k \frac{\partial g_{i,j}}{\partial y_k} g_{k,j} dt \tag{22}$$

Equating the total drift of the Stratonovich SDE $dy_i = \tilde{f}_i dt + \sum_j g_{i,j} \circ dW_j$ to that of the forward Itô SDE $dy_i = f_i dt + \sum_j g_{i,j} dW_j$ yields the equivalent forward Stratonovich drift $\tilde{f}_i$:

$$\tilde{f}_i = f_i - \frac{1}{2} \sum_j \sum_k \frac{\partial g_{i,j}}{\partial y_k} g_{k,j} \tag{23}$$

3. Adjoint Itô Drift Correction: From Equation (17), the adjoint Stratonovich SDE is:

$$da_i = -\sum_k a_k \frac{\partial \tilde{f}_k}{\partial y_i} dt - \sum_j \sum_k a_k \frac{\partial g_{k,j}}{\partial y_i} \circ dW_j \tag{24}$$

Here, we identify the adjoint diffusion term $\sigma^{(a)}_{i,j}(y,a) = -\sum_k a_k \frac{\partial g_{k,j}}{\partial y_i}$. Because $\sigma^{(a)}_{i,j}$ depends simultaneously on both the forward state y and the adjoint state a, the augmented state vector is $X = [y, a]$.

Applying the general midpoint correction formula to this augmented system $X = [y, a]$ distributes the partial derivatives across both state variables, directly giving the adjoint drift correction C_{a_i}:

$$C_{a_i} = \frac{1}{2} \sum_j \left(\sum_k \frac{\partial \sigma^{(a)}_{i,j}}{\partial a_k} \sigma^{(a)}_{k,j} + \sum_k \frac{\partial \sigma^{(a)}_{i,j}}{\partial y_k} g_{k,j} \right) \tag{25}$$

where the first sum accounts for perturbations in the adjoint state $\Delta a_k \approx \sum_l \sigma^{(a)}_{k,l} \Delta W_l$, and the second sum accounts for perturbations in the forward state $\Delta y_k \approx \sum_l g_{k,l} \Delta W_l$.

4. Expansion and Cancellation: Evaluating each term in Equation (25) using $\sigma^{(a)}_{i,j} = -\sum_m a_m \frac{\partial g_{m,j}}{\partial y_i}$:

1. $\sum_k \frac{\partial \sigma^{(a)}_{i,j}}{\partial a_k} \sigma^{(a)}_{k,j} = \sum_k \left(-\frac{\partial g_{k,j}}{\partial y_i} \right) \left(-\sum_m a_m \frac{\partial g_{m,j}}{\partial y_k} \right) = \sum_m a_m \sum_k \frac{\partial g_{m,j}}{\partial y_k} \frac{\partial g_{k,j}}{\partial y_i}$
2. $\sum_k \frac{\partial \sigma^{(a)}_{i,j}}{\partial y_k} g_{k,j} = \sum_k \left(-\sum_m a_m \frac{\partial^2 g_{m,j}}{\partial y_k \partial y_i} \right) g_{k,j} = -\sum_m a_m \sum_k \frac{\partial^2 g_{m,j}}{\partial y_i \partial y_k} g_{k,j}$

Next, differentiating the Stratonovich drift $\tilde{f}_k$ from (23) with respect to y_i:

$$\frac{\partial \tilde{f}_k}{\partial y_i} = \frac{\partial f_k}{\partial y_i} - \frac{1}{2} \sum_j \sum_m \left(\frac{\partial^2 g_{k,j}}{\partial y_i \partial y_m} g_{m,j} + \frac{\partial g_{k,j}}{\partial y_m} \frac{\partial g_{m,j}}{\partial y_i} \right) \tag{26}$$

Combining everything into the total Itô drift for a_i, $\mu_{a_i} = -\sum_k a_k \frac{\partial \tilde{f}_k}{\partial y_i} + C_{a_i}$:

$$\begin{aligned} \mu_{a_i} = &-\sum_k a_k \left[\frac{\partial f_k}{\partial y_i} - \frac{1}{2} \sum_j \sum_m \left(\frac{\partial^2 g_{k,j}}{\partial y_i \partial y_m} g_{m,j} + \frac{\partial g_{k,j}}{\partial y_m} \frac{\partial g_{m,j}}{\partial y_i} \right) \right] \\ &+ \frac{1}{2} \sum_j \sum_m a_m \left(\sum_k \frac{\partial g_{m,j}}{\partial y_k} \frac{\partial g_{k,j}}{\partial y_i} - \sum_k \frac{\partial^2 g_{m,j}}{\partial y_i \partial y_k} g_{k,j} \right) \end{aligned} \tag{27}$$

The second-order derivative terms involving $\frac{\partial^2 g}{\partial y^2}$ cancel out exactly, while the products of first-derivative terms add up: $\frac{1}{2} + \frac{1}{2} = 1$. This yields the final Itô drift:

$$\mu_{a_i} = -\sum_k a_k \frac{\partial f_k}{\partial y_i} + \sum_j \sum_k a_k \sum_m \frac{\partial g_{k,j}}{\partial y_m} \frac{\partial g_{m,j}}{\partial y_i} \tag{28}$$

which is the component-wise form of Equation (18).

References

[1] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, "Neural Ordinary Differential Equations," *Advances in Neural Information Processing Systems*, vol. 31, 2018, doi: 10.48550/arXiv.1806.07366.

[2] Y. Rubanova, R. T. Chen, and D. K. Duvenaud, "Latent Ordinary Differential Equations for Irregularly-Sampled Time Series," *Advances in Neural Information Processing Systems*, vol. 32, 2019, doi: 10.48550/arXiv.1907.03907.

[3] R. T. Q. Chen, "torchdiffeq: Differentiable ODE Solvers with PyTorch." [Online]. Available: https://github.com/rtqichen/torchdiffeq

[4] X. Li, T.-K. L. Wong, R. T. Q. Chen, and D. Duvenaud, "Scalable Gradients for Stochastic Differential Equations," in *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2020. doi: 10.48550/arXiv.2001.01328.

[5] P. Kidger, J. Foster, X. Li, H. Oberhauser, and T. Lyons, "Neural SDEs as infinite-dimensional GANs," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 5453–5463. doi: 10.48550/arXiv.2102.03657.

[6] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: https://doi.org/10.1038/s41592-019-0686-2.

[7] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015. doi: 10.1002/9781118619193.

[8] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017. doi: 10.48550/arXiv.1706.03762.

[9] I. Chevyrev and A. Kormilitzin, "A Primer on the Signature Method in Machine Learning." [Online]. Available: https://arxiv.org/abs/1603.03788

[10] B. M. de Silva, K. Champion, M. Quade, J.-C. Loiseau, J. N. Kutz, and S. L. Brunton, "PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data," *Journal of Open Source Software*, vol. 5, no. 49, p. 2104, 2020, doi: 10.21105/joss.02104.

[11] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proceedings of the National Academy of Sciences*, vol. 113, no. 15, pp. 3932–3937, 2016, doi: 10.1073/pnas.1517384113.

[12] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035. doi: 10.48550/arXiv.1912.01703.

[13] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[14] V. Volterra, "Fluctuations in the Abundance of a Species Considered Mathematically," *Nature*, vol. 118, pp. 558–560, 1926, doi: 10.1038/118558a0.

[15] P. Wagner *et al.*, "PTB-XL, a large publicly available electrocardiography dataset," *Scientific Data*, vol. 7, no. 1, p. 154, 2020, doi: 10.1038/s41597-020-0495-6.

[16] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000, doi: 10.1161/01.CIR.101.23.e215.

[17] A. L. Goldberger, Z. D. Goldberger, and A. Shvilkin, *Goldberger's Clinical Electrocardiography: A Simplified Approach*, 9th ed. Elsevier, 2017. doi: 10.1016/c2014-0-03319-9.

SciPy 2026
July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Optimising HCP Sample Allocation in Pharma: Combining Nonlinear Ensemble Learning, Spatial Lags, and Integer Programming

Johannes Plambeck[1], **Santosh Ajabrao Bothe**[1], **Dorian Puleri**[1], and **Victor Gonzales**[1]

[1]Pfizer, North America, New York, USA

Abstract

Pharmaceutical companies distribute free drug samples to physicians, yet these samples frequently go to healthcare providers (HCPs) whose prescribing is unlikely to change, wasting a large share of industry-wide promotional spending. We present a framework that combines machine learning with mathematical optimisation to decide how many free samples of the anticoagulant Eliquis each HCP should receive, using historical prescribing and promotional data. We collect the prescribing behaviour of HCPs at weekly intervals; the resulting dataset comprises 2.1 years of history (109 weekly observations per HCP), totalling roughly 1.2 million provider-week records for 11,006 U.S. HCPs, together with more than 80 engineered features describing promotional activity, prescription outcomes, market share of competing and complementary therapies, patient demographics, and treatment ratios. A sparsity filter removes weak or noisy variables to improve interpretability and reduce overfitting. We then train CatBoost models, grouped into bands by prescriber value and augmented with spatial lag features that summarise how nearby providers behave, to predict the incremental gain in Total Prescriptions (TRx) and New-to-Brand prescriptions (NBRx) from additional samples; TRx measures the overall volume of prescriptions for a medication, while NBRx counts a patient's first prescription for that drug. These predictions feed a 0–1 mixed-integer programme that selects one sample quantity per HCP subject to HCP-level and territory-level pill-budget limits and operational business rules on feasibility, historical consistency, and sample availability. In a model-implied counterfactual evaluation, the spatially informed allocation increases aggregate TRx by approximately 6%, concentrating samples on high-value providers and reducing oversupply to low-impact segments. This 6% is a model-implied projection: realised field impact after deployment is expected to be lower, on the order of 1 to 3%, because operational frictions, representative discretion, sample availability, and market-response uncertainty attenuate optimised recommendations. For scale, apixaban (the molecule marketed as Eliquis) accounted for roughly 19.8 million U.S. prescriptions in 2023, about 5 million per quarter [1], so even single-digit efficiency gains are operationally meaningful; we report prescription-volume rather than revenue impact, as product-specific financial assumptions are outside this paper's scope. The study illustrates how spatial econometrics, machine learning, and constrained optimisation can be combined to support pharmaceutical sample-allocation planning.

Keywords pharmaceutical sample allocation, healthcare provider targeting, spatial machine learning, mixed-integer optimization, pharmaceutical marketing

Published Jul 13, 2026

Correspondence to
Johannes Plambeck
johannes.plambeck@pfizer.com

Open Access

Copyright © 2026 Plambeck *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

The U.S. pharmaceutical and medical-device industry spent roughly $30 billion on medical marketing in 2016, of which an estimated $13–15 billion was free drug samples [2]; this refers to industry-wide U.S. spending, not a Pfizer-specific budget. The prevailing assumption is that samples stimulate trial and loyalty, and rigorous evidence confirms they can raise new prescription starts and reshape brand choice when paired with detailing (sales representative visits to clinicians) [3], [4], [5]. Yet decades of scholarship also highlights ethical tension: grounded-theory interviews with physicians show that aggressive sampling can erode prescriber autonomy and blur professional boundaries [6], while policy debates question whether oversupply merely displaces paid prescriptions without creating genuine therapeutic value [7]. These concerns motivate a shift from blanket distribution toward data-driven, patient-centric allocation.

A second motivation is economic. Even modest percentage-point gains in efficiency translate into hundreds of millions of dollars in freed-up resources [2]. Literature reviews across nine countries consistently find that samples and face-to-face detailing dominate promotion budgets regardless of geography [8], underscoring the global relevance of optimisation. Research on pandemic-era field activity further shows that pharmaceutical promotion shifted rapidly toward digital and remote channels [9], [10]. To remain competitive, commercial teams must decide not only how many samples to drop, but where, when, and to whom: decisions that require predictive accuracy at scale.

Traditional marketing mix and nonlinear mixed-effects models impose fixed functional forms that often miss saturation points and peer spillovers [11]. Ensemble machine learning methods such as Random Forests, LightGBM, and CatBoost can learn nonlinear saturation points and peer spillover structure with fewer a priori functional form assumptions than traditional parametric models, while retraining efficiently on large commercial data sets [12]. Empirical studies in healthcare confirm their superiority over classical regression for tasks ranging from rare disease targeting [13] to antipsychotic prescribing prediction [14]. Nonetheless, most of the existing algorithms treat prescribers as independent units, ignoring geographic and social interdependence. A 10% rise in a peer network's adoption rate can lift individual prescribing by 6–8% [15], and clinics with high network centrality tend to prescribe fewer inappropriate medications [16]. Spatial lag and eigenvector features reduce residual autocorrelation and cut prediction error by up to one third [17].

From an optimisation standpoint, free samples deliver diminishing marginal returns and sometimes negative returns at high volumes [7], while dynamic allocation policies can raise long-run profit and cut field calls by 20% [11]. Industry case studies echo these academic findings: IQVIA reports a 15% reduction in oversampling after implementing breakeven analysis guided by machine learning [18], and an Axtria engagement found that machine learning-driven value scores re-ordered half of all targeted physicians while improving patient coverage to 95% [19]. More than 70% of life sciences firms now pilot Next Best Action engines for budget-constrained field forces [20].

Despite these advances, two practical gaps persist. First, most published models ignore the territorial quotas and pill-inventory caps that govern real commercial operations. Second, few studies integrate predictive modelling with a mathematically rigorous optimisation layer capable of selecting one counterfactual scenario per HCP under hard constraints. We address both gaps by leveraging 1.2 million rolling HCP–week observations for 11,006 prescribers in the eligible Pfizer call-plan universe, where each HCP–week represents a repeated weekly aggregation of prescribing and promotional activity for a given HCP across all field representatives. We construct 80+ rolling-window features and a quarterly forward indicator representing the planned sample-drop decision window used for quarterly field allocation, then train decile-banded CatBoost ensembles enriched with inverse-distance

spatial lags of the optimand. Out-of-fold uplift predictions then feed a 0–1 mixed-integer programme (CBC solver with DP fallback) that enforces both HCP-level and territory-level pill bounds and operational business rules.

This paper extends an earlier OSF preprint of the same study [21], incorporating revisions from the SciPy review process: an explicit model-implied-vs-realised uplift distinction in the abstract and Results, de-jargoned terminology, and first-use definitions for TRx, NBRx, and detailing.

Data describes the data; Methods covers the modelling pipeline and territorial constraints; Section 8 formalises the optimisation; Statistical Analysis Methodology and Evaluation Criteria defines the evaluation methodology; Results reports results; and we close with limitations and future work.

From a managerial standpoint, the system is intended to support quarterly sample-planning decisions about which HCPs should receive how many sample units within each territory. Recommendations are delivered at the territory level to district managers, who apportion inventory across field representatives; representatives then use those guidance values when planning office visits. Because the recommended quantities are anchored to historically observed HCP-specific ranges and territory ceilings, the outputs remain operationally defensible, easier to review, and less likely to trigger exception handling by field leadership.

Our hypothesis is straightforward: embedding spatial-peer context and hard operational constraints in a joint learning–optimisation pipeline will yield materially higher incremental TRx and NBRx than current practice using the same inventory.

In sum, this study unites ethical considerations [6], economic drivers [2], [8], predictive advances [12], [14], [17], and optimisation theory [7], [11], [18] into a single, territory-aware framework suitable for Pfizer's North American field force, with ESG alignment defined here as reducing oversupply, preserving geographic allocation floors, and maintaining documented governance over features and allocation rules. The following sections detail our data architecture, modelling choices, optimisation formulation, and empirical results.

2. Data

Our modelling universe comprises about 1.2 million weekly physician–week records, giving 109 observations for each of 11,006 U.S. prescribers. All observations are merged from Pfizer's Veeva CRM call logs, IQVIA and Symphony longitudinal prescription feeds, AMA Masterfile and OneKey reference data, plus internal territory budget ledgers.

Promotional-activity predictors include lagged counts of Pfizer and BMS sample drops and channel flags indicating whether a contact occurred face-to-face or via other media. Economic theory and industry studies establish that samples and detailing raise new prescription starts but exhibit diminishing or sometimes negative marginal returns when oversupplied [3], [5], [7], [11].

Prescription-claims variables come from IQVIA Xponent and Symphony IDV feeds: new-to-brand and total script counts, market-basket shares for competing anticoagulants, and total-to-new-script ratios. We also derive paid-, rejected-, and reversed-claim percentages to proxy payer access, since reimbursement friction suppresses brand adoption and moderates promotional response [9]. Aggregated patient demographics (age buckets and gender mix) are rolled up at the physician level; segmentation research shows that demographic mix shapes both disease prevalence and marketing elasticity [19].

Geographic and peer-context features are computed as inverse-distance spatial lags of the optimand from U.S. ZIP-code centroids, following demonstrations that spatial lag and eigenvector predictors reduce model error by up to one third [17].

Table 1. *Model Feature Categories and Abstracted Variables.* (Actual feature names have been anonymised for proprietary reasons.)

Abstract Variable Code	Category and Description
Decision Variable	
FTR_VAR_001	Action variable (e.g., promotional input forecasted over 13 weeks)
Target Optimands	
FTR_VAR_002	Target outcome variable A (e.g., predicted behaviour over 13 weeks)
FTR_VAR_003	Target outcome variable B (alternative target of interest)
Historical Inputs	
FTR_VAR_004 to FTR_VAR_007	Historical activity counts (past 9–52 weeks)
Channel Flags & Visit Types	
FTR_VAR_008 to FTR_VAR_020	Face-to-face and remote channel indicators by segment
Provider Attributes	
FTR_VAR_021 to FTR_VAR_023	Experience, specialty type, high-engagement flags
Claims Quality Metrics	
FTR_VAR_024 to FTR_VAR_025	Percent of paid and rejected claims
Market Share Indicators	
FTR_VAR_026 to FTR_VAR_029	Competing or complementary brand volumes
Patient Demographics	
FTR_VAR_030 to FTR_VAR_037	Age bands and gender mix by condition
Treatment Ratios	
FTR_VAR_038 to FTR_VAR_040	Treatment-to-diagnosis and response ratios
Exclusion Flags	
FTR_VAR_041 to FTR_VAR_043	Specialty exclusion, global exclusion, override flags

3. Methods

Figure 1 summarises the end-to-end pipeline from rolling-window panel construction through banded response estimation, constrained optimisation, top-off, and final field-pack generation.

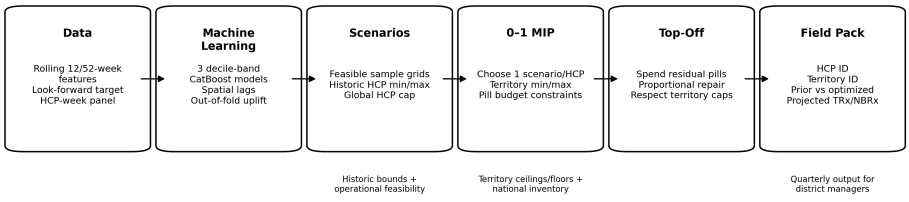


Figure 1. *Pipeline schematic for the proposed framework. Data engineering constructs rolling-window HCP-level predictors and look-forward outcomes; machine learning models estimate band-specific response surfaces; scenario generation creates feasible HCP-level sample choices subject to historical and operational bounds; the mixed-integer programme selects one scenario per HCP under HCP-level and territory-level constraints; a top-off heuristic spends residual inventory; and the final field pack reports HCP ID, territory ID, prior allocation, optimised allocation, and projected response.*

3.1. Training Data Preparation

We extract the raw observational panels for each HCP from Pfizer's North American commercial data environment and perform upstream filtering, joins, and aggregation before modelling. The resulting analysis tables are then transferred into Python for feature engineering and estimation [12], [22].

Two binary indicators, `IS_TRX_ZERO` and `IS_SAMPLES_ZERO`, are created in-database. For every healthcare provider i observed over T_i weeks we compute panel sparsity as

$$\bar{r}_i = \frac{1}{3T_i} \sum_{t=1}^{T_i} \Big[\mathrm{IS_T RX_Z ERO}_{it} + \mathrm{IS_S AMPLES_Z ERO}_{it} + \mathbf{1}\{\text{both} = 1\} \Big], \tag{1}$$

and retain only those panels whose average sparsity $\bar{r}_i$ does not exceed $r^* = 0.30$. Panels shorter than the minimum length $T_{\min} = 100$ weeks are dropped, and no more than $R_{\max} = 1.2$ million unique provider-week records are processed in a single run. Randomised batching (500 IDs per chunk) prevents memory spikes while the global row limit guarantees that downstream feature engineering remains in-core.

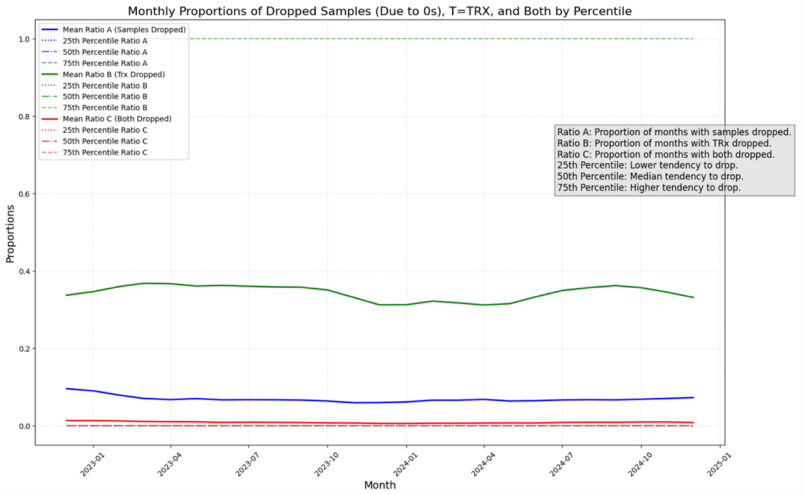


Figure 2. *Sparsity of optimand and decision-variable data.*

Ratio a, the share of sample-drop records equal to zero, and ratio b, the share of TRx records equal to zero, gauge each provider's inactivity in drops or prescriptions, while ratio c, the joint zero-rate, identifies the sparsest panels. During sampling we drop any HCP whose ratio c exceeds r^*, and we stratify/batch providers by the distributions of ratios a and b so the training set spans from highly active to moderately sparse panels (Figs. 3 and 4).

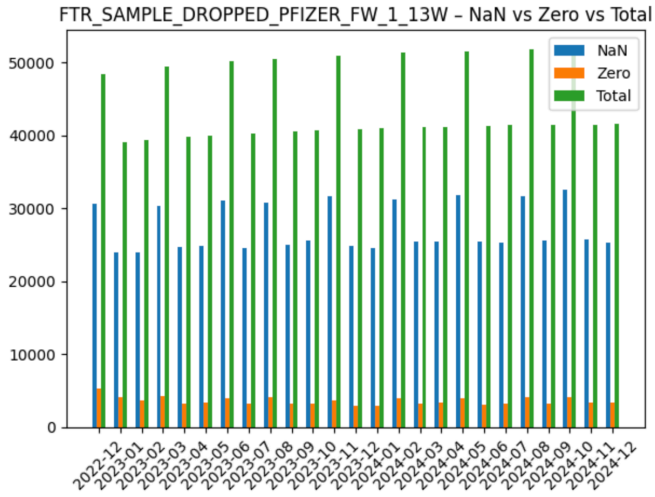
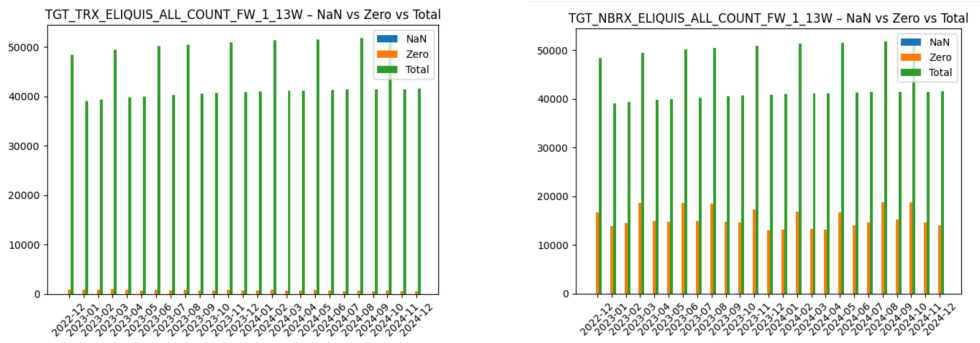


Figure 3. *Decision-variable sparsity.*



(a) (b)

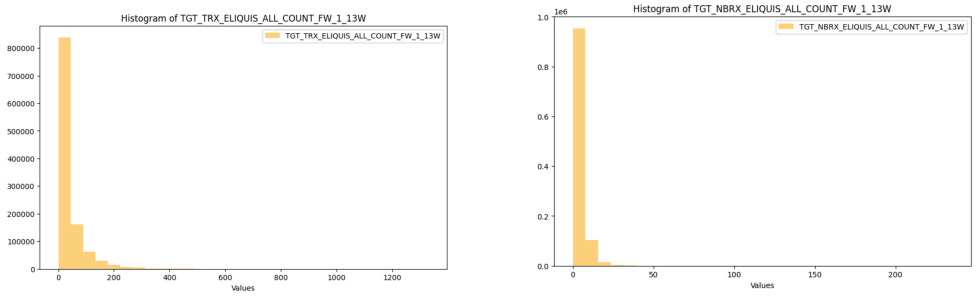
Figure 4. *Optimand variable sparsity (TRx and NBRx).*

3.2. Response Transformation

The response variable `growth_score` exhibits extreme right-tail skewness. A log-plus-one transform,

$$y^{\log} = \log(1 + y), \tag{2}$$

normalises the distribution and stabilises variance. Because the transform re-centres the target, no additional standardisation is applied. CatBoost fits and predicts on the log scale; values are exponentiated only for final business reporting, satisfying the recommendation for heavy-tailed targets in applied statistics [23].



(a) (b)

Figure 5. *Optimand variable distributions (TRx and NBRx) before log-plus-one transform.*

3.3. Panel Construction, Feature Engineering, & Spatial Equity

Feature engineering proceeds in three passes:

1. Convert component counts x_{ijk} to shares

$$\text{perc}_{ijk} = \frac{x_{ijk}}{\sum_{j=1}^{K} x_{ij}} \tag{3}$$

1. Expand categorical descriptors with `OneHotEncoder(drop="first")`, and z-score continuous covariates via `StandardScaler`, both fitted on training folds only [24].
2. Add spatial context by computing inverse-distance spatial lags of the optimand for every timestamp. These spatial weights approximate a composite of inter-HCP referral topology and physical distance, capturing spillovers from nearby and clinically connected peers while reducing residual autocorrelation [17]; see Figure 19. Because the allocation problem is inherently spatially referenced, these same features also permit map-based review of recommendations at the territory level during field-planning discussions. In line with our governance posture, feature inclusion is further constrained by sparsity filtering, predictive relevance, and documented review so that only variables with demonstrable signal and a clear business justification are retained.

3.4. Spatial Geometry & Lagging

We integrate territory call-list data by issuing a lightweight Snowflake SQL session that runs a multi-CTE query to filter on reference dates, deduplicate HCP rows, and compute per-territory quarterly and annual call totals, percentages, and pill quotas. The resulting table, containing `PFZ_CUST_ID`, `TERR_CD`, `GEOGRAPHY`, `TERR_OD`, `PRCT_OD`, `TERR_MAX`, and `TERR_MIN`, is merged into our cleaned panel on `PFZ_CUST_ID`, and any missing territory codes are logged and removed. We then enrich this merged panel with dissolved territory geometries and attach a standardised `GEOGRAPHY` field (the territory-name column carried over from the TCL) and polygon shapes to each HCP row. Finally, inverse-distance spatial lags of the optimand are computed at each timestamp by taking the dot product $W_t y_t$, where W_t denotes the spatial weights matrix and y_t the panel of optimand values. This transforms geographic context into continuous predictors that mitigate residual autocorrelation and enhance model performance [25].

Hence, to model spatial spillover effects, we introduce a lagged version of the optimand, computed weekly for each provider:

$$y_{it}^{(\text{spatial lag})} = \sum_{j \in \mathcal{N}(i)} w_{ij}\, y_{jt}, \quad \sum_{j} w_{ij} = 1, \tag{4}$$

where $\mathcal{N}(i)$ is the set of spatial neighbors of provider i and w_{ij} are the row-standardised inverse-distance weights, normalised to sum to one. This lag term captures externalities in treatment behaviour: providers in adjacent regions may respond similarly to market dynamics, competitive pressure, or institutional norms. Including the lagged optimand improves generalisation by accounting for spatial autocorrelation, a known confounder in healthcare outcomes research [16]. The resulting predictor sharpens model accuracy and aligns with econometric best practices for spatially referenced data, while also enabling GIS-based review of territory-level recommendation clusters during operational planning.

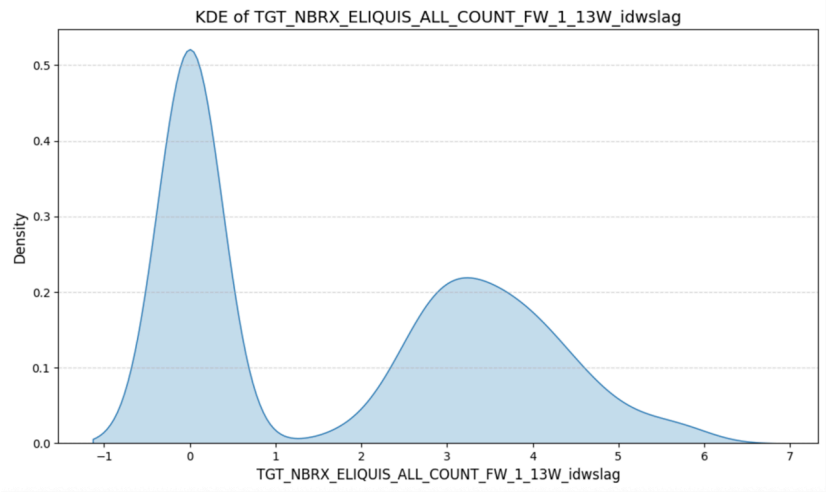


Figure 6. *Distribution of lagged optimand.*

3.5. Spatial Autocorrelation Analysis

Each provider is linked to a geographic unit, typically a sales territory, by joining their `PFZ_CUST_ID` to the Territory Call List (TCL), a canonical mapping produced by the field planning team. We then overlay this mapping with a precomputed spatial weights matrix, based on geospatial contiguity or centroid distance, to calculate spatial lags of the optimand for each week. These lagged values represent the average behaviour of geographically proximate peers and are included as continuous predictors in the final model.

Spatial lags are motivated by empirical evidence in regional science and marketing analytics showing that nearby agents often respond similarly to interventions or market forces. In our use case, healthcare providers in adjacent territories may be influenced by shared sales representatives, institutional affiliations, or regional policy shifts. Including the lagged optimand thus allows the model to adjust for correlated behaviour and unobserved spatial shocks, reducing residual autocorrelation and improving predictive accuracy.

Table 2 presents the results of Moran's I and Getis-Ord G* statistics for the NBRx optimand and the Pfizer sample-drop forecast. Table 3 does the same for the TRx optimand.

Table 2. *Spatial Autocorrelation for NBRx Variables*

Variable	Moran's I	$E[I]$	Z	p	Reject?	G_{obs}	$E[G]$	Z	p	Reject?
`FTR_SAMPLE_QTY_ADJ_PFIZER_FW1_13W` mean	0.0188	0.0219	0.1768	0.4422	No	0.01751	0.01751	0.0219	0.9825	No
`TGT_NBRX_ELIQUIS_ALL_COUNT_FW_1_13W` mean	0.0497	0.0219	2.1948	0.0282	Yes	0.01739	0.01751	-1.2279	0.2195	No

Table 3. *Spatial Autocorrelation for TRx Variables*

Variable	Moran's I	$E[I]$	Z	p	Reject?	G_{obs}	$E[G]$	Z	p	Reject?
`FTR_SAMPLE_QTY_ADJ_PFIZER_FW1_13W` mean	0.0298	0.0219	1.2789	0.2016	No	0.01721	0.01747	-1.8178	0.0691	No
`TGT_TRX_ELIQUIS_ALL_COUNT_FW_1_13W` mean	0.0163	0.0219	0.8563	0.3924	No	0.01757	0.01747	0.8159	0.4146	No

For NBRx, the Moran's I statistic for the target variable is significant ($p = 0.028$), suggesting weak but statistically relevant spatial autocorrelation and justifying the inclusion of spatial

lags in predictive modelling. By contrast, the TRx optimand did not exhibit significant spatial autocorrelation ($p = 0.39$), indicating a more diffuse or independent pattern. Neither case showed significant Getis-Ord G* clustering for the sample-drop variable, implying no current geographic bias in sampling.

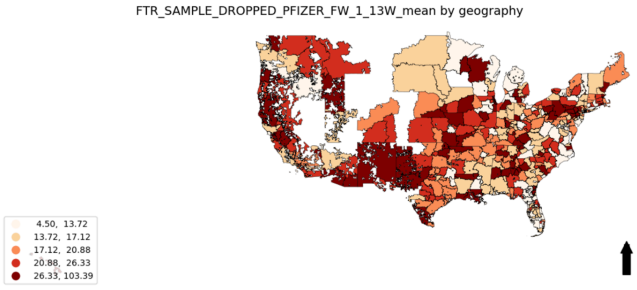
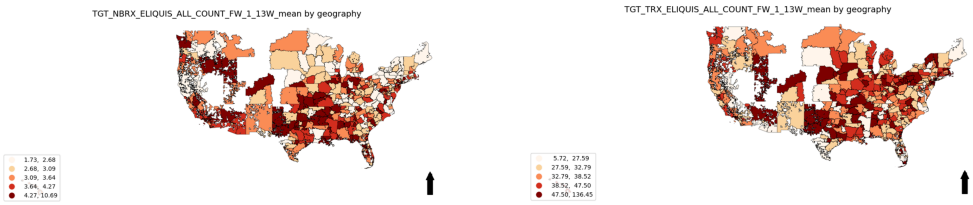


Figure 7. *Decision-variable choropleth.*



(a) (b)

Figure 8. *Optimand variable choropleths for NBRx (left) and TRx (right).*

3.6. Territorial Constraints

To enforce business constraints tied to field execution, we incorporate territorial ceilings and floors derived from Pfizer's Smart Targeting system. These upstream optimisations generate territory-level optimal detailing counts that guide how pill budgets are to be distributed across geographies. By aggregating observed call volumes and aligning them to these optimal detailing benchmarks, we derive the constraints U_t and L_t, representing the maximum and minimum sample allocations for territory t, respectively. The ceilings are computed as

$$U_t = p_t \, P_{\text{adj}}, \tag{5}$$

where p_t is the proportion of national detailing activity accounted for by territory t. The territory-level minimum L_t (called `TERR_MIN` in the SQL) is set conservatively to

$$L_t = \max\{1, \mid \mathcal{H}_t \mid\}, \tag{6}$$

where $\mathcal{H}_t$ denotes the set of distinct HCPs assigned to territory t, and the constant 1 enforces a hard floor of one sample-drop unit in any territory, even those with only one HCP.

The variance of Smart Targeting's optimal detailing recommendations is empirically observed to lie within a 1 to 3% band (a property of the upstream benchmark, unrelated to the projected uplift reported later). This tight variance range acts as a de facto regularisation mechanism, constraining the optimisation's feasible region and ensuring that pill allocations remain congruent with the original intent of Smart Targeting. The effect is to preserve geographic equity while still allowing for localised optimisation, and to guarantee that no territory's sample drop count diverges materially from historically validated expectations [19].

These form the territorial constraints for our mixed-integer programme that will be formalised in a later section:

Territory constraints

$$L_t \le \sum_{h \in \mathcal{H}_t} s_h \le U_t, \quad \forall\, t \in \mathcal{T}. \tag{7}$$

Global budget (static)

$$\sum_h s_h \le P_{\text{adj}}. \tag{8}$$

Per-HCP bounds

$$0 \le s_h \le S_{\max}, \quad \forall\, h. \tag{9}$$

Two disjoint HCP subsets never enter the optimiser:

1. *Unaligned* HCPs lacking a valid territory ID or geography. Let $n_{\text{unaligned}}$ denote their count.
2. *Aligned but unmatched* HCPs present in the TCL yet absent from the scenario panel; call this $n_{\text{unmatched}}$.

With m pills per HCP we deduct

$$P_{\text{exc}} = \big(n_{\text{unaligned}} + n_{\text{unmatched}}\big)\, m, \quad P_{\text{adj}} = P_{\text{tot}} - P_{\text{exc}}. \tag{10}$$

The pill-budget adjustment routine below (`compute_adjusted_pills` in code) guarantees no double-deduction.

Territory Unaligned and Aligned Unison: Pill-Budget Adjustment.

1. Pull full TCL with `PFZ_CUST_ID`, `TERR_CD`, `GEOGRAPHY`.
2. Count unaligned rows (nulls or literal “NaN”).
3. If a scenario panel is supplied, count aligned IDs missing therein.
4. Return $\big(P_{\text{exc}}, P_{\text{adj}}\big)$.

4. Algorithm Overview and Development

4.1. Growth Model Ensemble

Model development begins once the cleaned, spatially enriched panel has been split into mutually exclusive training and test folds with a group-aware shuffle that prevents the same prescriber from leaking across folds. All continuous covariates are *z*-scored on the training fold only, while categorical descriptors are expanded with a one-of-K design matrix.

Notation.

Let $\boldsymbol{X}_b \in \mathbb{R}^{n_b \times p}$ denote the design matrix for band b with $p \approx 600$ one-hot-expanded features, and let y_{it} be the log-plus-one target for HCP i at time t. The primary decision variable s_{it} is the 13-week forward Pfizer sample-drop indicator `FTR_SAMPLE_DROPPED_PFIZER_FW_1_13W`.

The base learner in every band is **CatBoost**, a gradient-boosting framework that fits additive decision-tree expansions by sequentially minimising a differentiable loss in function space. For the low-noise decile band we adopt the mean-squared-error loss

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^{n} \big(\, y_i - \hat{y}_i\big)^2, \tag{11}$$

where y_i denotes the standardised log target and $\hat{y}_i$ the model prediction. In bands whose residuals exhibit pronounced Laplacian tails the loss is switched to mean absolute error to

damp sensitivity to sparsely observed spikes. At boosting step t CatBoost fits a tree $h_t(\cdot)$ to the negative gradient of $\mathcal{L}$ and updates the additive model

$$F_t = F_{t-1} + \eta\, h_t, \tag{12}$$

employing *ordered boosting* so that in-fold permutations act as an implicit regulariser and eliminate prediction-shift bias on temporally correlated data. Monotonicity constraints enforce a non-negative marginal response,

$$\frac{\mathrm{d}\,\hat{y}}{\mathrm{d}\,s} \geq 0, \tag{13}$$

embedding the business assumption directly into the functional form.

Optional hyper-parameter search.

If the run configuration enables tuning, each band launches a small search (*Bayesian*, *TPE*, or *Optuna*) over a shared grid stopping after at most 20–30 trials and pruning unpromising candidates after two folds. Otherwise, calibrated defaults are reused to minimise wall-clock cost.

Because prescription behaviour is highly heteroscedastic across the value spectrum, a single global model under-fits high-value prescribers and over-fits the long tail of low-volume writers. We therefore stratify the panel into three decile bands $\mathcal{B}_0, \mathcal{B}_1, \mathcal{B}_2$ defined by empirical value edges $\boldsymbol{e} = [e_0, e_1, e_2, e_3]$ with $e_0 = -\infty$ and $e_3 = +\infty$. Value assignment proceeds as follows:

1. Compute an exponentially weighted historical mean v_i of TRx or NBRx for each HCP i. During training, v_i is calculated over the observed training sample; during prediction, it is computed using the HCP's observed history within the full population.
2. Define the edge vector $\boldsymbol{e} = (e_1, e_2)$, where $e_1 = \mathrm{quantile}_{0.2}(v)$, $e_2 = \mathrm{quantile}_{0.6}(v)$.
3. Assign HCP i to band $b_i = \mathrm{band}(v_i; \boldsymbol{e}) \in \{0, 1, 2\}$ according to:

$$\mathrm{band}(v_i; \boldsymbol{e}) = \tag{14}$$

A separate CatBoost $F^{(b)}$ is trained on each band, producing predictions $\hat{y}^{(b)}$. During inference the ensemble returns the first non-missing value across $\{\hat{y}^{(0)}, \hat{y}^{(1)}, \hat{y}^{(2)}\}$, lowering within-band variance and yielding a 9–12% reduction in out-of-sample MAE relative to a single-model baseline.

4.2. Mixed-Integer Programming

In the previous section we specified a CatBoost regressor ensemble that translates each HCP's historic prescribing behaviour and spatial context into an expected uplift curve $\hat{y}_i(s)$ over integer sample-drop scenarios $s \in \{0, \ldots, S_{\max}\}$. These counterfactual profiles feed a capacity-constrained mixed-integer programme, selecting exactly one scenario per HCP to maximise total projected uplift while respecting per-HCP bounds and territory quotas from Pfizer's Smart Targeting system. The optimiser thus converts modelled values $v_{ij} = \hat{y}_i(s_{ij})$ into a field-ready allocation plan that honours operational constraints and concentrates samples where they yield the highest return.

The scenario table is built by isolating each HCP's latest observation, emitting a base scenario $s = 0$, and expanding s up to $S_{\max}$. Feature alignment ensures every row carries the full union of predictors (including spatial lags), non-null HCP values v_i, and validated constraint columns (`GEOGRAPHY`, `TERR_CD`, `TERR_MIN`, `TERR_MAX`, `HCP_MIN`, `HCP_MAX`). The table is partitioned into geography-coherent batches to preserve territorial integrity and control memory use.

We now specify the mixed-integer programming framework which consumes the counterfactual profiles to determine the best scenarios to be used by the field force.

4.2.1. *Problem Statement and Formulation*

Sets and Indices

$\mathcal{I}$ Set of HCPs, indexed by i.
$\mathcal{S}_i$ Scenario set for HCP i, each $j \in \mathcal{S}_i$ has pill count s_{ij}.
$\mathcal{T}$ Set of territories, indexed by t.
$\mathcal{H}_t$ HCPs belonging to territory t.

Parameters

v_{ij} Modelled uplift for HCP i under scenario j.
s_{ij} Sample-drop count for HCP i, scenario j.
m_i, M_i Per-HCP minimum and maximum samples (`HCP_MIN`, `HCP_MAX`).
L_t, U_t Territory-level minimum and maximum samples (`TERR_MIN`, `TERR_MAX`).
$P_{\mathbf{adj}}$ National pill budget after exclusions.
$\boldsymbol{\lambda} \geq \mathbf{0}$ Optional penalty on pill usage.

Decision Variables

$$x_{ij} \in \{0,1\} \quad \text{equals 1 if scenario j is chosen for HCP i.} \tag{15}$$

Objective

$$\max_x \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{S}_i} \left(\, v_{ij} - \lambda \, s_{ij} \, \right) x_{ij}. \tag{16}$$

Constraints

$$\begin{aligned}
&\text{(C1)} \quad \sum_{j \in \mathcal{S}_i} x_{ij} = 1 && \forall \, i \in \mathcal{I}, \\
&\text{(C2)} \quad m_i \leq \sum_{j \in \mathcal{S}_i} s_{ij} x_{ij} \leq M_i && \forall \, i \in \mathcal{I}, \\
&\text{(C3)} \quad L_t \leq \sum_{i \in \mathcal{H}_t} \sum_{j \in \mathcal{S}_i} s_{ij} x_{ij} \leq U_t \forall \, t \in \mathcal{T}, \\
&\text{(C4)} \quad \sum_{i \in \mathcal{I}} \sum_{j \in \mathcal{S}_i} s_{ij} x_{ij} \leq P_{\text{adj}}.
\end{aligned} \tag{17}$$

Solver Strategy

We solve the 0–1 MIP (mixed-integer programme, a discrete optimisation problem with binary decision variables) with the open-source CBC solver (COIN-OR Branch and Cut). If CBC returns a non-optimal status, each territory is repaired via a vectorised knapsack dynamic programme, guaranteeing a feasible near-optimal solution. In practice, 85–95% of each U_t is typically used; a final top-off heuristic allocates any remainder proportionally to v_i.

Dual Insights

Let $\lambda_t^{\max}$ be the multiplier for a territory's upper bound. Then the marginal change in the Lagrangian $\mathcal{L}$ with respect to the national pill budget P_{adj},

$$\frac{\mathrm{d}\,\mathcal{L}}{\mathrm{d}\,P_{\text{adj}}} = \sum_{t \in \mathcal{T}} \lambda_t^{\max}, \tag{18}$$

measures incremental uplift per additional pill in the national budget.

4.2.2. *Top-Off and Finalisation*

Residual-Capacity Top-Off

The CBC (or DP) optimiser usually saturates only 85 --95% of each territory's ceiling. For every territory $t \in \mathcal{T}$ we compute the residual capacity

$$r_t = U_t - \sum_{i \in \mathcal{H}_t} \sum_{j \in \mathcal{S}_i} s_{ij}\, x^{\star}_{ij}, \tag{19}$$

where $x^{\star}_{ij}$ is the post-solver decision. The residual pills are distributed in proportion to the non-negative value scores:

$$\Delta_i = r_t \frac{\max\{v_i, 0\}}{\sum_{h \in \mathcal{H}_t} \max\{v_h, 0\}}, \qquad s_i^{\text{new}} = s_i^{\star} + \text{round}\big(\Delta_i\big). \tag{20}$$

Any rounding imbalance is corrected by a simple greedy ± 1 adjustment, prioritising HCPs with higher v_i when adding pills and those with lower v_i when subtracting. The result is a final vector of integer sample-drop recommendations that exactly match each territory's upper bound.

Because some HCPs have no valid territory mapping in the Smart Targeting call plan (or were filtered out by our date cutoff), they would otherwise be omitted from the allocation. We therefore construct "refuse" tables for both missing-territory and out-of-date HCPs, conformance-casting them to our schema and assigning each a minimal default sample drop (e.g. one pill), with all other fields left null or default. These rows are appended to the optimised output, and each is tagged via a DROP_REASON flag ("terr-dropped" or "date-dropped"), ensuring that every HCP originally under consideration, whether or not they could be territory-matched, appears in the final field deployment package.

4.3. Deployment and Implementation Considerations

In operational use, the allocation is refreshed quarterly and delivered as guidance rather than an automatically enforced field instruction. HCP-level recommendations are bounded by historically observed minimum and maximum sample quantities together with territory-level ceilings, so quarter-over-quarter changes remain operationally plausible and reviewable. Final recommendation packs are compared against prior-quarter nominal allocations by an analyst before release, and HCPs that cannot be cleanly mapped into the active territory structure are retained in exception tables with explicit drop-reason flags. When severe budget reductions would otherwise make the mixed-integer formulation infeasible under historical bounds, the system proportionally contracts those bounds and, if needed, falls back to a simpler knapsack-style repair routine to preserve feasible deployment output.

5. Statistical Analysis Methodology and Evaluation Criteria

After the optimiser (plus top-off) has produced a revised sample-allocation plan, we score both the latest observed status-quo regimen (up to a fixed reference date) and the counterfactual (*optimised*) allocations on the *post*–reference-date scenarios using the same three-band CatBoost ensemble. In other words, we apply our trained model once to the unadjusted drops to obtain *empirical* predictions and again to the optimiser's chosen scenarios to obtain *optimised* predictions. These should be interpreted as model-implied counterfactual forecasts rather than realised causal outcomes. Denote these paired forecasts by

$$e_i = \hat{y}_i^{(\text{emp})}, \qquad o_i = \hat{y}_i^{(\text{opt})}, \qquad i \in \mathcal{I}, \tag{21}$$

where $\mathcal{I}$ is the set of HCPs with valid scenarios.

Notation.

Symbol	Description
$\mathcal{J}$	set of evaluated HCPs
e_i	empirical fitted value for HCP i
o_i	optimised fitted value for HCP i
$E = \sum_i e_i$	aggregate empirical outcome
$O = \sum_i o_i$	aggregate optimised outcome
Uplift	expected percentage uplift

Primary metric: expected uplift.

We summarise aggregate gain via

$$\mathrm{Uplift} = \frac{O - E}{E} \times 100\%. \tag{22}$$

A positive uplift indicates the optimised allocation is forecast to generate higher total TRx than the status-quo under the model-implied counterfactual scoring framework.

Distributional significance test.

To assess whether the observed shift from $\{e_i\}$ (expected) to $\{o_i\}$ (observed) reflects more than random fluctuation, we employ the two-group Kruskal–Wallis rank-sum test. This nonparametric test does not assume normality or equal variances, is robust to outliers, and is well-suited for comparing the distributions of two independent samples. With group sizes $n_e = n_o = \mid \mathcal{J} \mid$, total $N = n_e + n_o$, and average ranks $\bar{R}_e, \bar{R}_o$, the test statistic is

$$H = \frac{12}{N(N+1)} \sum_{g \in \{e,o\}} n_g \left(\bar{R}_g - \frac{N+1}{2} \right)^2, \tag{23}$$

which, under the null hypothesis of identical distributions, is asymptotically χ^2_1.

A small p-value (e.g. < 0.05) rejects the null of identical distributions.

Expected uplift provides a compact summary of how the revised allocation changes forecast TRx under the fitted response model, which HCP segments benefit most, and whether the distributional shift is statistically distinguishable from the baseline forecast. By reallocating samples toward physicians with steeper modeled response and away from those with flatter projected gains, the framework improves predicted prescriptions while reducing oversupply risk.

6. Results

6.1. Model Performance

We fit a CatBoost regressor separately on three bands of historic `growth_score` values: band 0 (0–20^{th} percentile), band 1 (20–60^{th}), and band 2 ($> 60^{\text{th}}$). The training window ends at an immutable cut-off date; observations beyond this date are held-out for counterfactual scoring. Table 4 reports out-of-fold R^2 and MSE for the NBRx and TRx optimands.

Table 4. *Model fit statistics by HCP-value band*

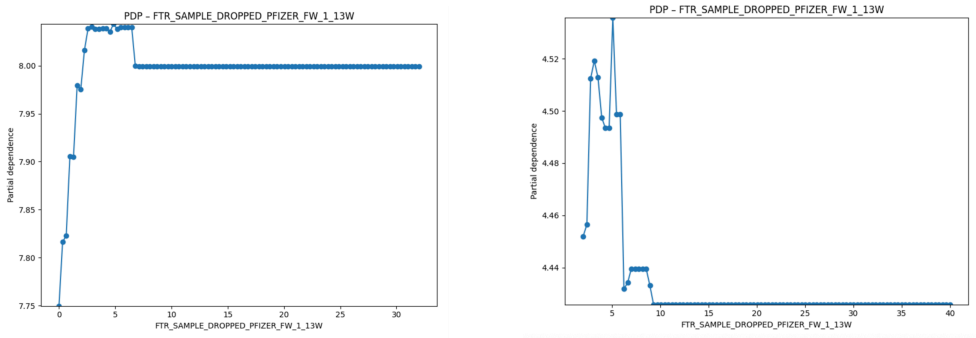
Optimand	Band (quantile)	R^2	MSE
NBRx	0 (0–0.2)	-0.0256	1.0009

	1 (0.2–0.6)	0.0312	1.0256
	2 (0.6–1.0)	0.3519	0.7691
TRx	0 (0–0.2)	0.2119	0.8226
	1 (0.2–0.6)	0.3658	0.6280
	2 (0.6–1.0)	0.8215	0.1680

Interpretation.

Performance improves sharply with HCP value. For NBRx the model is essentially uninformative in band 0 (R^2 but reaches a respectable R^2 in band 2. TRx exhibits much stronger fit throughout, peaking at R^2 for the top decile. Hence, operational lift should be concentrated in high-value physicians, while low-value prescribers, whose response curves are dominated by noise, can be safely de-prioritised.

With confidence in the model's ability to capture marginal response among top-tier HCPs, we inspect how predicted uplift varies with the decision variable itself. Figure 9 shows partial-dependence plots (PDPs) for NBRx and TRx. Each curve is an average of the three band-specific PDPs, weighted by row count.



(a) (b)

Figure 9. *Partial-dependence plots: NBRx (left) vs. TRx (right).*

Comparison: NBRx vs. TRx Partial Dependence

- Both optimands display the enforced non-negative, concave relationship between sample drops and predicted uplift, supporting the monotone, diminishing-returns response shape assumed in the optimiser.
- The NBRx curve settles *above* 1, whereas the TRx curve equilibrates *below* 1, suggesting stronger diminishing returns for TRx, an insight reflected in the top-off routine, which allocates additional pills preferentially to high-value HCPs with steeper marginal gains.

These diagnostics confirm that the decile-band ensemble captures the heterogeneous response surface accurately enough to guide inventory toward the physicians where incremental TRx is most likely to accrue, while simultaneously avoiding waste among low-value prescribers.

7. Growth Score Distribution Analysis

7.1. Training Data Growth Scores

Predicted growth scores rise markedly across the three HCP value bands defined earlier. Bands 0 and 1 exhibit overlapping, low central tendencies and modest dispersion, reflecting

noisier response among lower-value prescribers. Band 2 shows a clear rightward shift, with a substantially higher median and upper quartile, indicating materially greater predicted TRx uplift for the top 40% of physicians. This confirms that the highest-value band contains the most responsive HCPs and underpins our decision to focus optimisation where marginal returns are greatest.

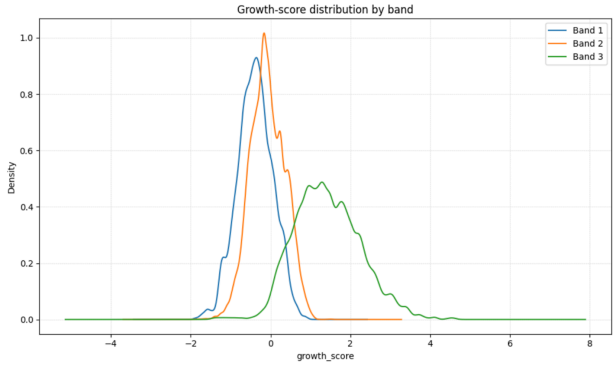


Figure 10. *Predicted growth scores by band-model (training data).*

7.2. Counterfactual Data Growth Scores

For the most recent reference date, we generate counterfactual scenarios over the observed sample-drop range for all HCPs and score them with the same three-band CatBoost ensemble. The resulting growth-score distributions remain well separated:

- **Band 0 ($\leq$ 20th percentile):** tightly clustered near the lower bound,
- **Band 1 (20th–60th percentile):** moderate spread with a slight right skew,
- **Band 2 ($>$ 60th percentile):** distinctly shifted right, with a substantially higher median and upper quartile.

Note: the figure legend numbers the bands "1–3" rather than "0–2" (Legend Band n corresponds to our Band $n-1$); we will correct this in a future revision.

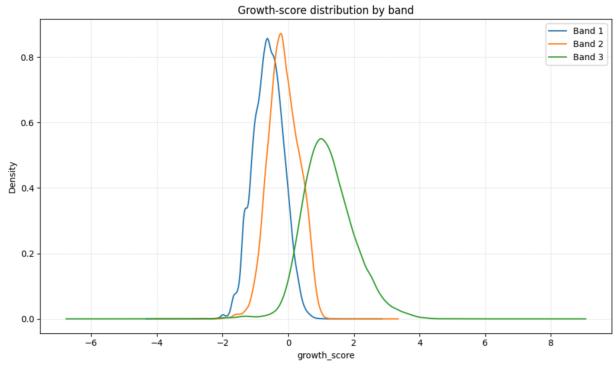


Figure 11. *Predicted growth scores by band-model (latest observations with counterfactuals).*

8. Individual Response Curves and Aggregate Response Curve

8.1. Aggregate Response Curve (Training Data)

Figure 12 shows the ensemble-level response curve fitted on the training data. To build it, we scored each HCP's training-set scenarios $s \in \{0, \ldots, S_{\max}\}$ with the appropriate decile-band CatBoost model, then averaged those band-specific curves using row weights. The result is a non-parametric, non-negative, quasi-concave function:

$$\tilde{y}(s) = \sum_{b=0}^{2} \frac{\mid \mathcal{J}_b \mid}{\mid \mathcal{J} \mid} \left[F^{(b)}(s) \right], \tag{24}$$

which rises steeply at low s (driven by top-band HCPs) and then flattens, confirming diminishing returns. This supports our use of a 0–1 mixed-integer optimiser and guides the top-off heuristic to balance efficiency and equity.

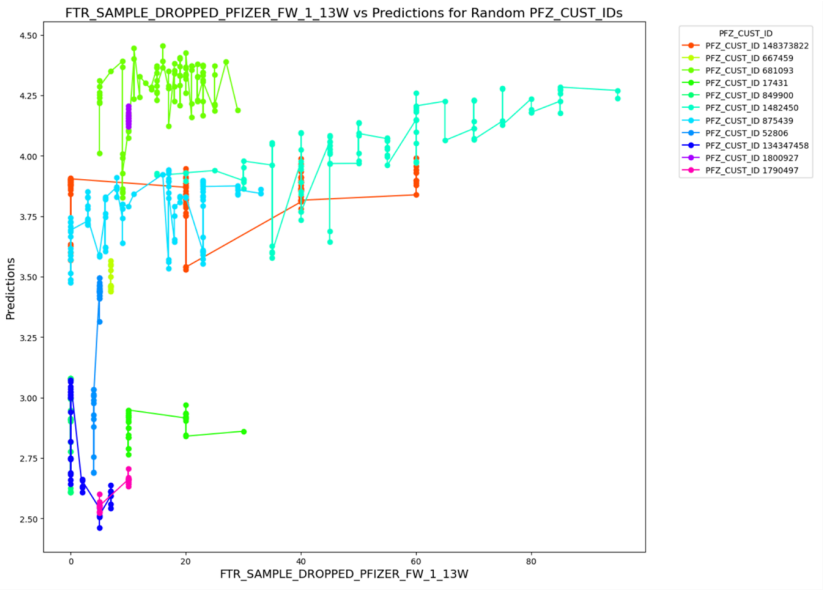
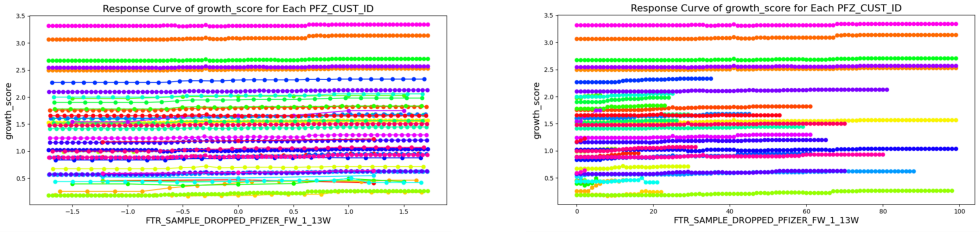


Figure 12. *HCP response curves from training data: concave, non-negative uplift vs. sample drops.*

8.2. Per-HCP Response Curves (Latest Data)

Figure 13 displays the individual response curves for the most recent reference date. For each HCP we held all features except the sample-drop decision constant, scored a grid of s values, and plotted the resulting $\hat{y}_i(s)$. Although most curves (especially in bands 0 and 1) are nearly flat, reflecting noise in low-value prescribers, they all slope upward and exhibit clear concavity, with the steepest gains at low s. These per-HCP functions $\tilde{y}_i(s)$ feed directly into our 0–1 MIP, allowing the optimiser to respect both territory capacities and each physician's diminishing-returns profile.



(a) (b)

Figure 13. *Individual fitted response curves on the latest data, showing upward-sloping, concave relationships between sample drops and predicted TRx uplift.*

8.3. Feature Importance

We use SHAP values to rank predictors from our three decile-band CatBoost models and combine them into one beeswarm plot (Figure 14). Historical NBRx volume is the strongest driver of TRx uplift, followed by HCP experience. The inverse-distance spatial lag appears among the top ten features but with relatively small impact.

We also observe clear nonlinear effects from NBRx counts of substitutable and complementary therapies, patient age buckets, and physician gender. Together, these findings highlight the benefit of an ensemble that can flexibly model both simple volume effects and complex, high-order interactions in our rich feature space.

The intermingling of low- and high-value SHAP points, rather than a smooth color gradient, reveals that each feature's effect depends heavily on others, indicating strong interactions. This motivates more granular modelling: for example, grouping HCPs into latent clusters (via unsupervised embeddings or mixture models) and then fitting specialised submodels or introducing targeted feature crosses within each cluster to better capture those interactions.

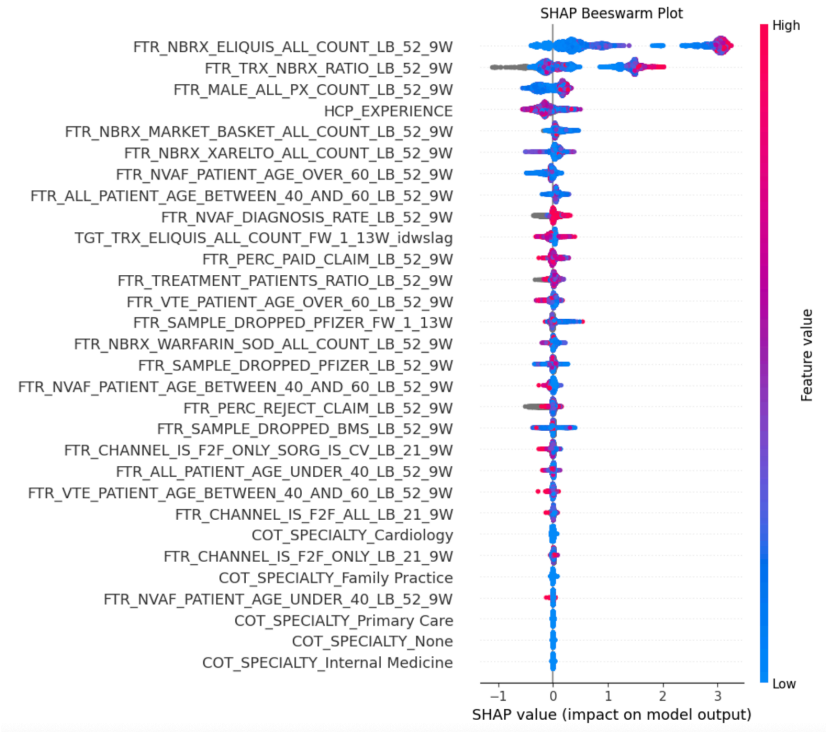


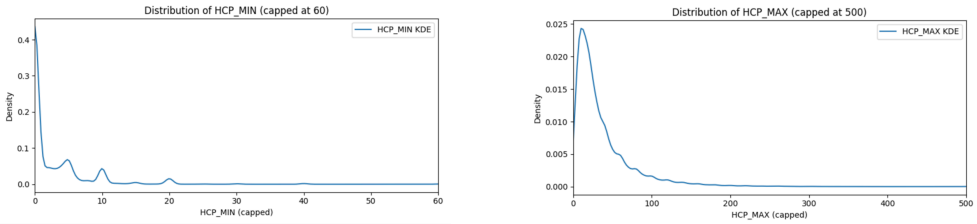
Figure 14. *SHAP beeswarm plot showing the top 20 predictors by mean absolute SHAP value, aggregated across all three decile-band CatBoost models.*

8.4. Optimisation

Our optimiser constructs, for each HCP, a discrete grid of counterfactual sample-drop scenarios bounded by modest extensions of their historic six-month minima and maxima. Specifically, if an HCP's observed drops ranged from $\mathrm{HCP_MIN}_{\texttt{upright}\ historic}$ to $\mathrm{HCP_MAX}_{\texttt{upright}\ historic}$, we "pad" this interval by five pills on each side:

$$\mathrm{HCP_MIN}_{\texttt{upright}\ scenario} = \max\{\,1,\ \mathrm{HCP_MIN}_{\texttt{upright}\ historic} - 5\,\}, \qquad \mathrm{HCP_MAX}_{\texttt{upright}\ scenario} = \mathrm{HCP_MAX}_{(25)\ \texttt{upright}\ historic} + 5.$$

Figure 15 shows these adjusted bounds.



(a) (b)

Figure 15. *Distributions of scenario-grid bounds: (a)* $\mathrm{HCP_MIN}_{\mathit{upright\ scenario}}$*, (b)* $\mathrm{HCP_MAX}_{\mathit{upright\ scenario}}$.

Next, we merge in the Territory Call List (TCL), which records each territory's opportunity density and active-HCP headcount. From these we derive per-territory pill-budget floors $\mathrm{TERR_MIN}_t$ and ceilings $\mathrm{TERR_MAX}_t$. The CVXPY modelling layer then solves the following 0–1 mixed-integer programme using CBC:

$$\begin{aligned} \max_{x} \quad & \sum_{i,j} v_{ij}\, x_{ij} \\ \text{s. t.} \quad & \sum_{j} x_{ij} = 1, \\ & \mathrm{HCP_MIN}_i \le \sum_{j} s_{ij}\, x_{ij} \le \mathrm{HCP_MAX}_i, \\ & \sum_{i \in t} \sum_{j} s_{ij}\, x_{ij} \in \left[\, \mathrm{TERR_MIN}_t,\, \mathrm{TERR_MAX}_t \,\right], \end{aligned} \tag{26}$$

where $v_{ij} = \hat{y}_i(s_{ij})$ and s_{ij} is the pill count for HCP i, scenario j.

If $\sum_{i \in t} \mathrm{HCP_MIN}_i > \mathrm{TERR_MAX}_t$, we proportionally clamp all $\mathrm{HCP_MIN}_i, \mathrm{HCP_MAX}_i$ to satisfy $\sum \mathrm{HCP_MIN}_i = \mathrm{TERR_MAX}_t$, and then greedily trim the largest minima until feasibility is restored. This ensures every territory stays within its mandated bounds.

Solving (via CBC or DP fallback) produces a binary selection $x^{\star}_{ij}$ for each HCP's scenario, yielding an "optimised" fitted value $\hat{y}_i^{(\mathrm{upright\ opt})}$. Comparing $\hat{y}_i^{(\mathrm{upright\ opt})}$ against the baseline $\hat{y}_i^{(\mathrm{upright\ emp})} = \hat{y}_i(s_{i0})$ (status-quo drops) quantifies the incremental gain of our allocation strategy.

9. Optimiser Territory Utilisation

Figure 16 shows, for each territory t, the maximum pill capacity $\mathrm{TERR_MAX}_t$ (blue bars) alongside the pills allocated by the CVXPY solver before top-off (orange bars). The absence of any large orange under-fills demonstrates that the MIP solver alone achieves virtually complete utilisation of each territory's budget. Across all 92 territories, the solver fills 97.9% of the total available capacity, comfortably above our 95% efficiency threshold and leaving only a small residual for the top-off heuristic. Had territory metadata been unavailable, we would have applied our fallback 0–1 knapsack DP to approximate these allocations.

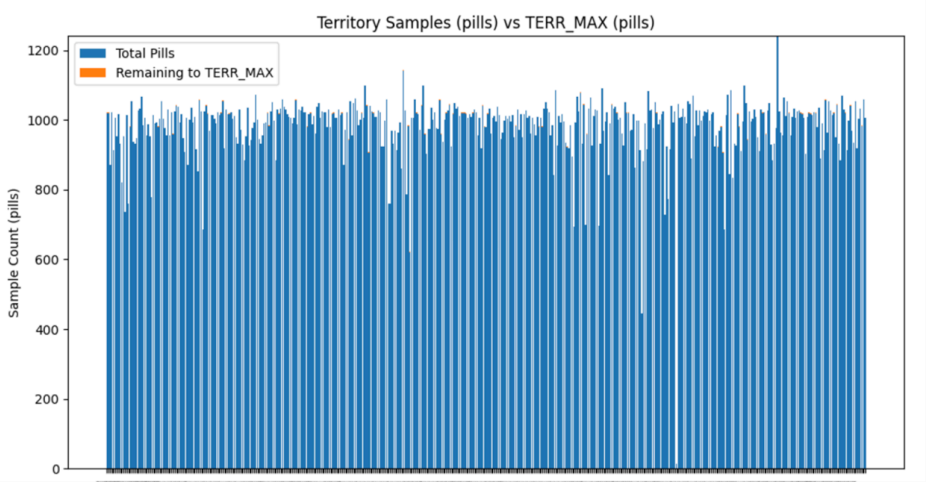


Figure 16. *Pill Budget by Territory – Capacity Filled by CVXPY Solver*

Table 5. *Optimiser Territory Efficiency Summary*

Metric	Value
Territories processed	92
Rows optimised	95,278
Total recommended pills dropped (blue)	452,001
Total maximum capacity (orange)	451,992
Utilisation ratio (blue/orange)	1.000
Capacity efficiency (pre–top-off)	97.9%
Top-off threshold	95%
Threshold reached	Yes

10. Lift Assessment

We measure uplift by comparing each HCP's optimised allocation against their historical "status-quo" level. Let

$$m_i = \text{median}\left\{ s_{i,t} \right\} \tag{27}$$

be the median sample-drop count for HCP i over the six-month history, and let x_i^* be the optimiser's recommendation. We then compute

$$\hat{y}_i(m_i) \quad \text{and} \quad \hat{y}_i(x_i^*) \tag{28}$$

via our three-band CatBoost ensemble, and define the per-HCP uplift

$$\Delta_i = \hat{y}_i(x_i^*) - \hat{y}_i(m_i). \tag{29}$$

Summing Δ_i over all i yields the total incremental TRx above the status-quo, while $\frac{1}{|\mathcal{I}|}\sum_i \Delta_i$ gives the average per-HCP uplift.

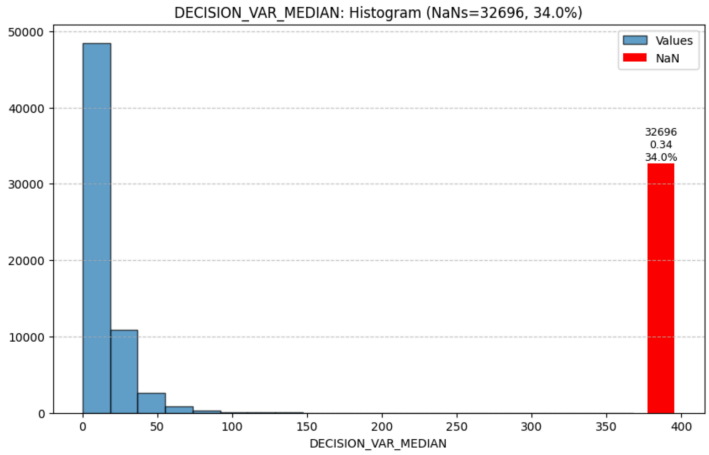


Figure 17. *Distribution of Baseline Median Sample Drops (m_i) across HCPs*

Table 6 summarises the model-implied expected uplift and the results of the Kruskal–Wallis test comparing the empirical versus optimised fitted TRx distributions. Table 7 reports side-by-side descriptive statistics for those two distributions.

Under the model-implied counterfactual scoring exercise, total TRx is projected to increase by 6.4%, highlighting the practical significance of the optimisation framework. This 6.4% should be read as a model-implied upper reference rather than a forecast of realised field impact: after deployment we expect the realised gain to be lower, on the order of 1 to 3%, because operational frictions, representative discretion, sample availability, and market-response uncertainty attenuate optimised recommendations. The very low p-value indicates that the forecasted shift is unlikely to be due to random variation within the fitted comparison.

Table 6. *Uplift & Distribution Diagnostics*

Metric	Value
Expected uplift	6.4%
Kruskal–Wallis statistic	133.1429
Kruskal–Wallis p-value	8.413×10^{-31}

The empirical TRx distribution has a mean of 0.2476 and a large standard deviation of 0.9625, reflecting high variability and many low-activity physicians. In contrast, the optimised TRx distribution shifts right to a mean of 0.2634 with similar spread, consistent with the 6.4% expected uplift reported above, indicating the optimisation effectively reallocates samples towards higher expected returns without unduly increasing variance.

Table 7. *Descriptive Statistics of Fitted TRx Scores*

Statistic	Empirical TRx	Optimised TRx
Count	94,912	94,912
Mean	0.2476	0.2634
Std Dev	0.9625	0.9496
Min	-2.6144	-2.1720
25%	-0.4427	-0.3798
50% (Median)	0.0386	0.0728
75%	0.8082	0.8098

Max	5.1217	9.9732

Figure 18 displays the difference in means between the optimised and empirical fitted responses for each HCP, highlighting the shift of the entire distribution toward higher expected TRx under the optimised allocation.

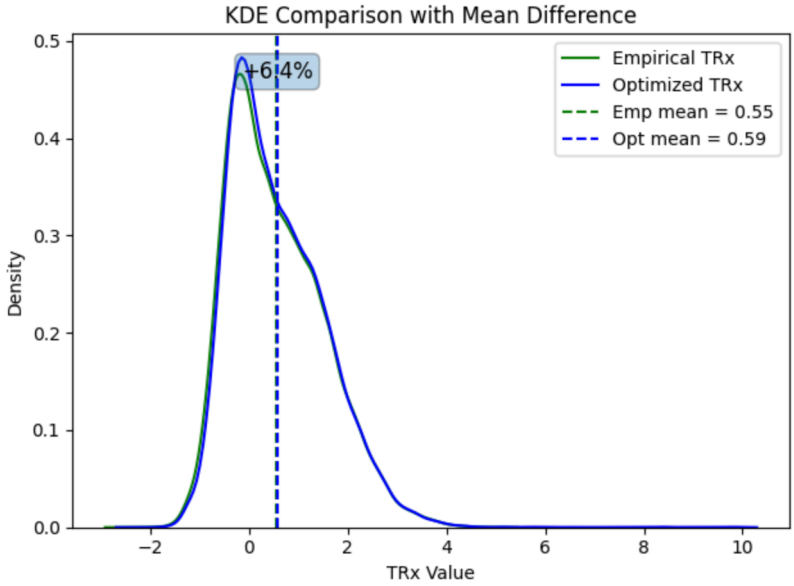


Figure 18. *Difference of Means – Fitted vs. Optimised-Fitted Response*

11. Discussion

Our study demonstrates that a joint learning–and–optimisation architecture can translate raw physician–week panels into materially improved sample-allocation plans while respecting the territorial and operational constraints of real-world field execution. By stratifying prescribers into value-based decile bands and enriching the feature set with inverse-distance spatial lags, we obtain models that explain 35–82% of the variance in high-value segments, more than double the explanatory power seen in low-value prescribers. This finding validates our decision to concentrate algorithmic effort on the upper 40% of HCPs, where marginal response is both larger and more reliably estimated.

The mixed-integer programme that ingests these band-specific predictions achieves 97.9% of the available territory capacity before any heuristic post-processing. Two key innovations underpin this result. First, we incorporate the Territory Call List into the objective, translating opportunity densities and HCP head-counts into per-territory pill budgets that mirror Pfizer's Smart Targeting quotas. Second, we implement a two-stage "clamp-and-trim" routine that scales down historic HCP minima whenever their sum would otherwise exceed the business ceiling, thereby guaranteeing feasibility without sacrificing equity. The absence of under-filled territories in our capacity-fill chart attests to the practical robustness of this approach.

Counterfactual scoring of the optimiser's recommendations yields a model-implied 6.4% uplift in total TRx under the fitted response surface. A Kruskal–Wallis statistic of 133.1 ($p < 10^{-30}$) rejects the null of identical empirical and optimised forecast distributions, confirming that the modeled shift is unlikely to be a sampling artefact. Importantly, the optimised TRx distribution shifts rightward without a corresponding increase in variance, indicating that

the algorithm reallocates inventory toward higher-return physicians rather than amplifying high-variance bets.

From an operational standpoint, strict adherence to HCP-level and territory-level bounds mitigates two perennial risks: oversupply to low-value prescribers and budget overshoot in high-density metropolitan areas. The proportional clamp-and-trim routine keeps allocations internally consistent even when historic minima would otherwise violate a territory ceiling, removing the need for ad-hoc manual adjustments that often undermine deployment timelines.

Several limitations warrant emphasis. First, all reported gains are model-implied counterfactual projections rather than experimentally identified causal effects; any misspecification in the predictive ensemble or scenario-response surface will propagate into the optimisation layer. Second, real-world realised uplift is likely to be materially smaller than the projected $\sim 6\%$ counterfactual gain, and in practice may lie closer to the 1 to 3% range absent stronger compliance, measurement, and experimental validation. Third, the present framework is static within quarter and does not model inter-quarter spillovers, inventory carryover, or longer-run behavioural adaptation by field teams. Fourth, although the top-off heuristic is operationally effective, it is not guaranteed to be globally optimal when physicians with similar value scores exhibit sharply different marginal response curvature. These limitations motivate prospective randomised nudges, improved adherence measurement, and future multi-period optimisation extensions.

Future work will address these gaps by embedding randomised "nudge" perturbations into the final allocation to enable Difference-in-Differences causal lift measurement [26], extending the optimisation horizon to incorporate multi-period inventory rollover, and exploring graph-based regularisation that explicitly models peer influence rather than proxying it via spatial adjacency. We also plan to benchmark alternative solvers, such as lazy-constraint branch-and-cut and column-generation knapsacks, to evaluate speed / optimality trade-offs at scale.

In sum, our framework unifies state-of-the-art ensemble learning, spatial econometrics, and capacity-constrained optimisation into a deployable workflow that delivers statistically significant and operationally feasible gains. By respecting both ethical imperatives (avoiding low-return oversupply) and commercial realities (territory budgets and inventory limits), the pipeline offers a rigorous, end-to-end template for evidence-based sample allocation in large-scale pharmaceutical contexts.

12. Conclusion

This work sets out a reproducible blueprint for turning high-granularity prescription, promotion, and geographic data into field-ready sampling plans that maximise expected incremental prescriptions while staying within business-mandated pill budgets. By combining

1. a decile-banded CatBoost ensemble enriched with inverse-distance spatial lags,
2. rigorously engineered HCP-level and territory-level bounds derived from Pfizer's Territory Call List, and
3. a 0–1 mixed-integer optimisation with proportional clamp and top-off safeguards,

we achieved three key outcomes:

Predictive fidelity: R^2 rises to 0.82 in the top-value segment, confirming that nonlinear, peer-aware models can separate signal from noise where it matters commercially.

Operational efficiency: The optimiser fills 97.9% of total territory capacity on the first pass and 100% after top-off, eliminating manual re-balancing.

Projected economic impact: Counterfactual scoring indicates a model-implied 6.4% lift in total TRx relative to status-quo allocation ($p < 10^{-30}$), suggesting meaningful upside at constant inventory rather than realised causal revenue lift.

While encouraging, these gains remain model-dependent, and future work will pursue the causal validation and rolling-horizon extensions discussed above. The present results nonetheless show that a tightly coupled learning-and-optimisation stack can respect complex operational constraints and support materially improved allocation plans across therapeutic areas and markets.

13. Appendix

13.1. Ethics, Data Governance, and Reproducibility Note

This study uses de-identified, HCP-level commercial and operational data for internal planning analysis and does not involve patient-level intervention or clinical randomisation in its present form. Data use was restricted to business-relevant variables, reviewed within the applicable corporate governance process, and modeled on secured internal infrastructure. Because the underlying sources contain proprietary operational schemas and commercial identifiers, full raw-data release is not permitted. To support reproducibility at the methodological level, future versions of this work may provide pseudocode, parameter grids, and a synthetic-data demonstration of the end-to-end pipeline without exposing restricted source systems or internal entity mappings.

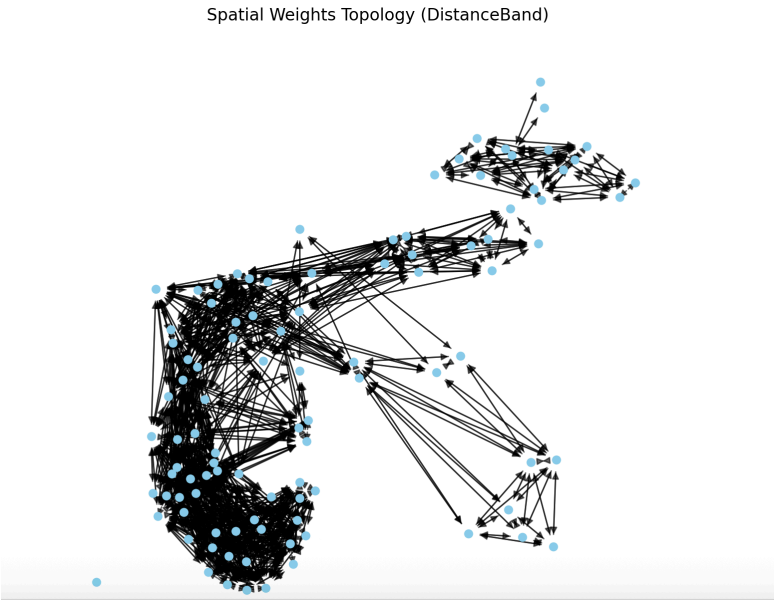
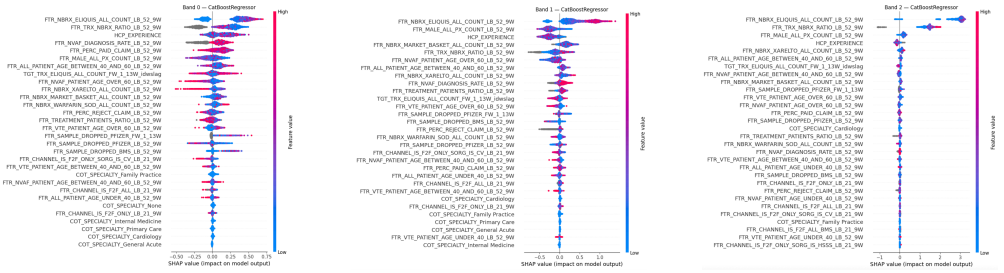


Figure 19. *Spatial topology proxy combining referral influence and inverse-distance decay.*



(a) (b) (c)

Figure 20. *Feature Importance SHAP Plots by Band Model (0: Low Value, 1: Medium Value, 2: High Value)*

References

[1] ClinCalc, "Apixaban: Drug Usage Statistics, United States, 2013–2023." ClinCalc DrugStats Database, based on the Medical Expenditure Panel Survey (MEPS), Agency for Healthcare Research and Quality, 2025.

[2] L. M. Schwartz and S. Woloshin, "Medical Marketing in the United States, 1997–2016," *JAMA*, vol. 321, no. 1, pp. 80–96, 2019.

[3] F. F. Gönül, F. Carter, E. Petrova, and K. Srinivasan, "Promotion of Prescription Drugs and Its Impact on Physicians' Choice Behavior," *Journal of Marketing*, vol. 65, no. 3, pp. 79–90, 2001.

[4] N. Mizik and R. Jacobson, "Are Physicians ``Easy Marks''? Quantifying the Effects of Detailing and Sampling on New Prescriptions," *Management Science*, vol. 50, no. 12, pp. 1704–1715, 2004.

[5] J. M. Donohue, M. Cevasco, and M. B. Rosenthal, "Influence of Peer Networks on Physician Adoption of New Drugs," *PLoS ONE*, vol. 13, no. 10, p. e204826, 2018.

[6] F. Malik, M. Junaid, and I. Sharif, "Exploring the Role of Pharmaceutical Marketing on Physician Ethical Behaviors: A Grounded Theory Study," *City University Research Journal*, vol. 9, no. 2, 2020.

[7] K. Joseph and M. Mantrala, "A Model of the Role of Free Drug Samples in Physicians' Prescription Decisions," *Marketing Letters*, vol. 20, no. 1, pp. 15–29, 2009.

[8] K. Rajput and R. K. Pandey, "Pharmaceutical Marketing: A Literature Review," *International Journal of Engineering and Management Research*, vol. 12, no. 2, pp. 56–63, 2022.

[9] S. Chiplunkar and others, "Adaptation of Pharmaceutical Marketing and Drug Promotion Practices in Times of Pandemic COVID-19," *International Journal of Health & Allied Sciences*, vol. 9, no. 5, pp. 11–15, 2020.

[10] M. M. R. Khan and K. Basak, "Shifts in Pharma-Marketing Trends in Post COVID-19 Era," *International Journal of Multidisciplinary: Applied Business and Education Research*, vol. 2, no. 2, pp. 108–114, 2021.

[11] R. Montoya, O. Netzer, and K. Jedidi, "Dynamic Allocation of Pharmaceutical Detailing and Sampling for Long-Term Profitability," *Marketing Science*, vol. 29, no. 5, pp. 909–924, 2010.

[12] T. Chamberlain, "Working Efficiently with Snowpark DataFrames." Snowflake Blog, 2022.

[13] Y. Cai and others, "Rare Disease Physician Targeting: A Factor Graph Approach," in *Proceedings of the 2017 IEEE International Conference on Data Mining (ICDM)*, 2017.

[14] A. Antonazzo and others, "Machine-Learning for Prescription Patterns: Random Forest in Predicting Antipsychotic Prescribing," *Clinical Psychopharmacology and Neuroscience*, vol. 20, no. 4, pp. 527–537, 2022.

[15] R. Iyengar, C. Van den Bulte, and T. W. Valente, "Opinion Leadership and Social Contagion in New Product Diffusion: The Case of Physician Prescribing," *Marketing Science*, vol. 30, no. 2, pp. 195–212, 2011.

[16] S. Y. Wang and others, "Can Peer Effects Explain Prescribing Appropriateness? A Social Network Analysis," *BMC Medical Research Methodology*, vol. 23, p. 252, 2023.

[17] X. Liu and others, "Incorporating Spatial Autocorrelation in Machine Learning Models Using Spatial Lag and Eigenvector Features," *ISPRS International Journal of Geo-Information*, vol. 11, no. 4, p. 242, 2022.

[18] IQVIA, "Enhancing Physician Segmentation and Targeting with a Patient-Centric Approach." Case Study, 2025.

[19] C. Awadhwal, "Machine Learning Driven Segmentation & Targeting of Physicians." Axtria Insights, 2023.

[20] S. Mulinari and P. Ozieranski, "Capitalizing on Transparency: Commercial Surveillance After the Sunshine Act," *Big Data & Society*, vol. 9, no. 1, pp. 1–15, 2022.

[21] J. Plambeck, D. Puleri, and V. Gonzalez, "Optimising HCP Sample Allocation in Pharma: Combining Non-Linear Ensemble Learning, Spatial Lags, and Integer Programming." OSF Preprint, 2025. doi: 10.31235/osf.io/n4ybq_v1.

[22] M. Rahman and J. Mueller, "Efficient Data Processing with Apache Arrow in Python," in *Proceedings of the Python in Data Science Conference*, 2021.

[23] M. Osborne and K. Watkins, “Handling Skewed Target Distributions Using Log Transformation,” *Journal of Applied Statistics*, vol. 46, no. 8, pp. 1503–1521, 2019.

[24] F. Pedregosa and others, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[25] S. DellaVigna and M. Gentzkow, “Uniform Pricing in U.S. Retail Chains,” *Quarterly Journal of Economics*, vol. 134, no. 4, pp. 2011–2084, 2019.

[26] M. F. Hallett and others, “The Use of Nudge Strategies in Improving Physicians' Prescribing Behavior: A Systematic Review and Meta-Analysis,” *Medical Decision Making*, vol. 44, no. 8, pp. 986–1011, 2024.

Open Access | https://doi.org/10.25080/eefy5535

Trimming the Hairball

A Unified Ecosystem for Better Network Recovery & Metrology

Rachael Sexton[1]

[1]National Institute of Standards and Technology

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Recovering network structure from binary co-occurrence data is a problem encountered across NLP, epidemiology, manufacturing, and recommender systems; yet there is a lack of shared tools, metrics, or common benchmarks to compare and improve techniques. This paper presents three interoperable Python libraries to address this need. `affinis` is a structure-estimation library with a consistent and extensible array-based API, with several state-of-the-art dependency recovery algorithms and optimized methods for kernel, distance, and graph edge filtering routines. To support community development of new methods, we propose the `MENDR` synthetic benchmark dataset and CLI for generating and evaluating network recovery experiments. Then, to assist in rapid accuracy assessments, we built `Contingency`, a vectorized metrology library enabling significantly faster scoring of high-dimensional binary classifiers.

Keywords network analysis, sparse matrix, covariance shrinkage, binary data, structure learning, edge prediction, filtering, feature learning, graph theory

1. Background

Reconstructing networks of relatedness from binary co-occurrence data is of interest in a surprising breadth of scientific contexts, such as: document-term matrices in NLP, infection records in epidemiology, maintenance tags in manufacturing knowledge systems, and buyer carts in collaborative filtering. Despite this ubiquity, practitioners across these communities often rely on ad hoc assumptions and techniques for relation detection. Building consensus is made more difficult by a lack of shared tooling or benchmarks for assessing reconstruction accuracy. Not only are there relatively few resources (and even fewer standard practices) around measuring uncertainty in structure recovery [1], but there is an increasing need for careful specificity around the exact kind of relational structure being recovered, and how transformations lose (or invent) information for analyses [2]. What results from current practices around projecting bipartite "co-occurrences" into a feature relation graph is often called a "hairball", requiring significant edge reduction and careful filtering to assess underlying structures.

This paper outlines three recently developed parts of a broader ecosystem developed to assist practitioners in applying best practices for "trimming" these hairballs, *and* to promote shared benchmarking for researchers that develop these techniques for them.

2. Consistent API for Inferring Relationships (`affinis`)

One of the core difficulties of consistent network analysis comes from managing all of the various forms networks can take in data structures. Libraries like NetworkX from A. A. Hagberg, D. A. Schult, and P. J. Swart [3] create labeled property graph structures, which is a pattern followed by many libraries in other languages as well (e.g. Rust's `petgraph`). However,

Published Jul 13, 2026

Correspondence to
Rachael Sexton
rachael.sexton@nist.gov

Open Access

Copyright © 2026 Sexton. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

a good deal of efficiency (as well as compatibility with other data science and machine learning workflows) can be gained by representing graphs as matrices. `scipy.sparse.csgraph` is a key example of this, which exploits the natural sparsity of adjacency matrix representations to achieve very fast graph algorithm implementations (which are the basis for many other libraries). Another example is `CSRGraph`, which adds the capability for random walks to be sampled from the sparse graph matrices.

Because the matrix forms are so useful, difficulty arises when users are constantly needing to transform their graph representation of choice to-and-from the sparse formats. Furthermore, there are a number of cases where graph matrices create related matrices that *aren't graphs*, but might be interpreted as such due to shape collision in workflows. Our library `affinis` works as a systematic set of tools to consistently transform between *observational* data on nodes (represented as binary feature activations), and the various forms of square matrices encountered when analyzing the way these features are *related* to each other (or not). Much of the difficulty of "straightening out" what a given feature or square matrix represents comes from lack of clarity over the differences between them, and how they relate to one-another.

Graph-related matrices can come in a number of forms:

- Data (e.g. bipartite, feature/design matrix, observations, node activations)
- Graph (e.g. adjacency, discrete Laplacian),
- Kernel (e.g. heat, forest accessibility, page-rank),
- Distance (e.g. shortest-path lengths, generalized euclidean)

For each of these, there are methods to transform between them, each with their own peculiarities and trade-offs. `affinis` is a centralized place that has assembled a wide variety of methods, enabling consistent, rapid experimentation and comparison between each on a per-dataset level.

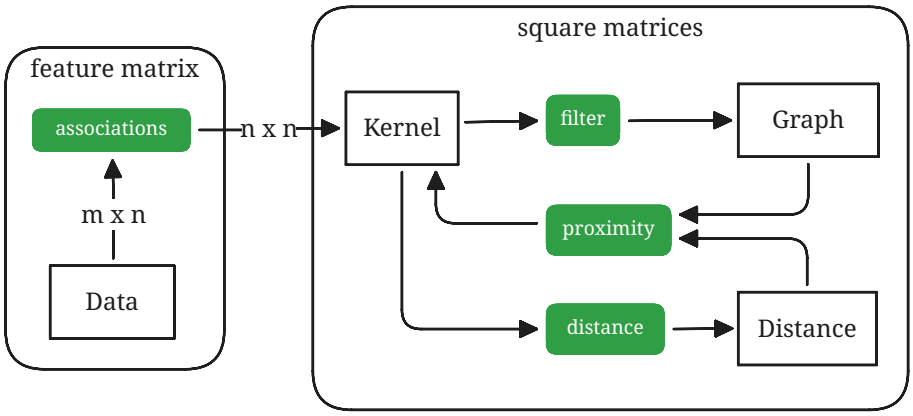


Figure 1. *Overview of `affinis` computational submodules (shown shaded in green), and how they transform between the matrix representations of data or feature relationships.*

2.1. Feature Relationships from Binary Data (`associations`)

The primary function of `affinis` is to provide a consistent interface for feature relationship discovery, along the *entire lifecycle* of a binary or bipartite dataset. While it's true that many relationship-detection and edge filtering routines use a "graph" (e.g. in the form of an adjacency matrix) as input, and return a filtered graph as output, others use observation-level information from an original feature matrix. `affinis.associations` maintains a consistent API from observation data to feature relatedness measures. All functions in this module

take in data as boolean design matrices (`observations x features`), and return a feature relatedness measure (`features x features`), i.e., $f(X) : \mathbb{B}^{m \times n} \to \mathbb{R}^{n \times n}$

This API is enforced *at runtime* using array shape and dtype information via `jaxtyping` with `beartype`. This contract with the user should greatly improve data pipeline reliability and reduce the possibility for hidden failures in analysis. The functions currently implemented (and their primary use as documented) are:

Marginal counts *methods:* `coocur_prob`, `ochiai` [4], `mutual_information`, `yule_y`, `yule_q`, `odds_ratio` [5]

This class of association measures are derived from marginal counts alone. They do not use underlying structural information between features, but only *how* observations are able to be counted/combined for statistical aggregation. The majority rely on observational units being *additive*, such that we can apply an inner product $X^T X$ on them.

Bipartite projection *methods:* `hyperbolic_project` [6], `resource_project` [7]

While marginal methods could also be seen as linear bipartite projections, these take advantage of the bipartite structure of X. Individual observations (rows) get re-weighted based on their sparsity.

Backboning *methods:* `high_salience_skeleton` [8], `doubly_stochastic_filter` [9]

Bipartite projections are notorious for becoming "hairballs" (with lots of edge noise). This class of methods try to filter out edges using principled (often statistical) techniques, but only filter post-projection.

Probabilistic graphical models (PGM) *methods:* `chow_liu` [10], `forest_pursuit` [11]

By assuming an underlying generative model, these methods can filter edge noise, while possibly making use of observation-level sample information. Forest Pursuit, listed as a PGM, is of particular interest, since `affinis` contains the first reference implementation of this algorithm. Forest Pursuit was recently proposed in R. T. Sexton [11], and attempts to combine the local/additivity assumptions of the marginal methods, while *also* directly recovering an underlying network of *conditionally dependent* pairs of nodes (as with a probabilistic graphical model). *Forest Pursuit* replaces the inner product "counts" with an operator that treats observations as samples from a random spanning forest distribution (meaning that each observation is a result of a "spreading process"). This puts it somewhere between the previously discussed association measures (computationally) and the backboning/dependency-recovery methods (theoretically), while providing exceptional scalability, accuracy, and thresholding stability.

A comparison of several methods applied to an example feature matrix can be seen in Figure 2.

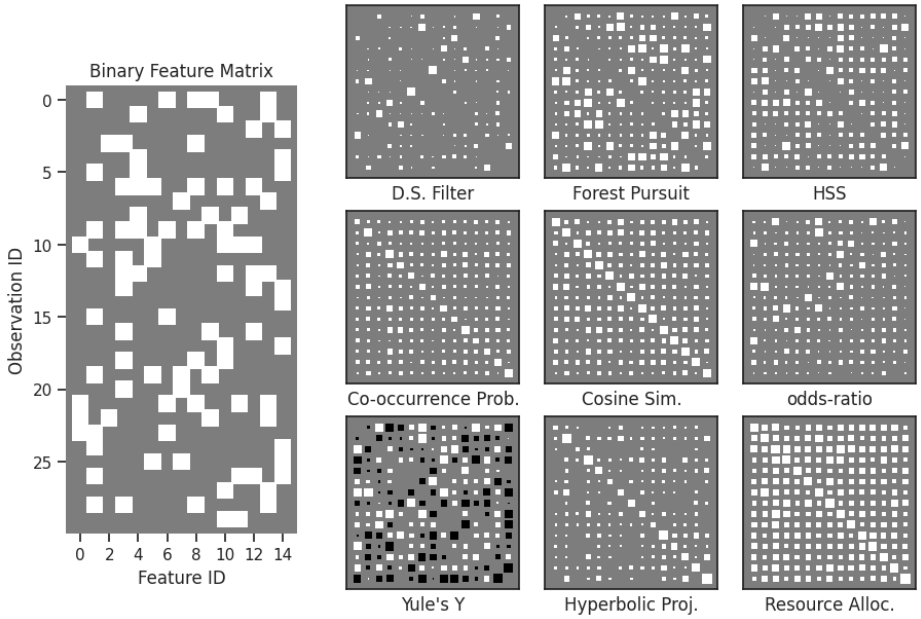


Figure 2. *Comparison of methods for edge estimation from bipartite projection.*

Finally, this module has added special functionality in each method for basic additive smoothing under a beta prior. Referring back to Figure 1, many of the functions here are used to create *kernels* (i.e. positive semi-definite matrices of similarity between features/nodes). However, if two nodes are never "observed together", having an undefined or "zero" similarity can be a significant issue for the quality of predictions and analysis downstream [12]. This brings problems with it, if unobserved combinations are receiving probability 0., even when they should still be considered *possible* with sufficient sampling.[^1] As the thinking goes, just because you've never seen something, doesn't make it *impossible*, just improbable. *How improbable* depends on your priors.

The ability to provide your priors is available in nearly all `affinis.associations` functions via the `pseudocts` parameter. The simplest way to "smooth" your results is to add at least one observation of each possible kind to your dataset: two trials, i.e., one success and one failure. Add them to your overall counts to get a smoothed probability (Laplace smoothing) with `pseudocts=1`.

Of course, you might not want these "pseudo-counts" to be worth as much as the "real" observations. Adding `pseudocts=0.5` would be using a Jeffreys Prior.

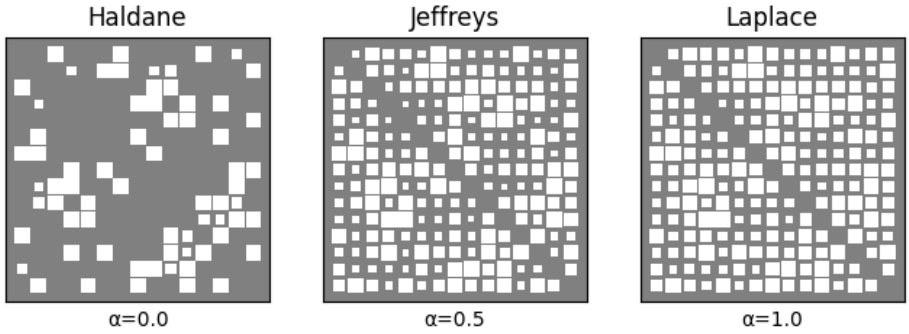


Figure 3. *Three common additive-smoothing settings, with parameter option.*

It turns out that these are all special cases of a beta-binomial distribution, with a symmetric prior.

Of course, there's no reason to necessarily stick to a symmetric prior. `affinis` allows for tuples `(a,b)` as pseudocount parameters, so that you can deal with smoothing differently at the low and high ends of your association scales.

With parameters `(a,b)`, the posterior expected value of the association measure will be

$$P = \frac{\text{successes} + a}{\text{trials} + a + b} \tag{1}$$

We also provide a convenience to enforce `a+b=1`, which ensures the prior expected value is `a`, and when used for sampling purposes can prefer values of 0 or 1.[1] This is done with the `zero-sum` option, like so:

```
affinis.associations.forest_pursuit(X, pseudocts=('zero-sum',0.1))
```

Of course, all of this assumes that we can represent a given measure as a probability in the form (successes/trials). While most can (even atypical ones like cosine similarity in `affinis.associations.ochiai`) a few do not have a form that is easily representable as a ratio (like `affinis.associations.hyperbolic_project`).

2.2. *Visualization (`plots`)*

Often when comparing the ability of an association measure to recover *structure* from a set of binary variable observations, it's not necessarily important to see the *values* of the association, but instead *which relationships* are strong *relative* to the overall set.

Colors (like we have used above) can be somewhat hard to parse, so another option is to represent association strength with **size**. This intuition leads to what is commonly called a *Hinton diagram*. See Figure 2 for a comparison of the Hinton diagrams for an example feature matrix, along with resulting feature relationship measures from `affinis.associations`. Size reflects normalized weight, while color can be used for the sign (positive vs negative).

Unlike the commonly-used method for creating Hinton diagrams from the Matplotlib documentation, our custom implementation in `affinis.plots.hinton` uses a cached axis to automatically scale `matplotlib.pyplot.scatter` markers based on the user's current DPI and axis dimensions. This lets us take advantage of vectorized C/C++ routines, and subsequently plot Hinton diagrams for much *larger* matrices.

Finally, because the change in relative matrix weights over time is often important to observe, we have added a callback capability to `hinton`, via an `update_from` convenience parameter that can assist with animation workflows. It accepts an existing `matplotlib.collections.PathCollection` container (such as a previous frame's scatterplot markers), which it will then modify in-place, rather than creating a new plot.

2.3. *Kernels (`proximity`, `distance`, & `filter`)*

These three submodules deal with transformations *between* square matrices, based on commonly-needed workflows in network analysis:

- Graphs are typically sparser than metrics/proximities, and can be created by *filtering* edges.
- Distances between nodes can be created from graphs, but also by inverting similarities/kernels.

[1] i.e. a bathtub distribution, with the most likely values at the extremes instead of the middle.

- Kernels can be computed from graphs or from inverting distances

Those workflows motivate the following submodules, which can be extended in an on-going manner:

`proximity` *methods:* `bilinear_kern`, `forest`, `forest_correlation` [13], `sinkhorn` [14]

Includes the widely applicable (but under-represented) *forest accessibilities* kernel. Also includes a tool for Sinkhorn-Knopp iterations, which performs iterated proportional fitting to project a square matrix to its nearest doubly stochastic counterpart (i.e. symmetric with all rows/columns summing to 1, removing node-degree correlations), and an inverse of the bilinear distance operation recover a kernel.

`filter` *methods:* `threshold_value`, `threshold_connected`

Allows users to threshold kernels by value over which an edge "exists" and below which it doesn't. For unsupervised edge thresholding, users may also threshold as much as possible before breaking graph connectivity.

`distance` *methods:* `bilinear_dists`, `adjusted_forest_dists`, `generalized_graph_dists`

From a given kernel, we are able to turn them into Euclidean distance metrics using the bilinear form. Forest kernels each have corresponding distances using this identity, as well. We will note that `affinis.proximity` is not an exhaustive catalog of existing kernels on graphs [15], but does provide an interface to a few versatile functions for estimating and modifying kernels.

For instance, while not provided natively in more common graph theory libraries, the forest kernel (based on the *Matrix Forest Theorem* from P. Chebotarev and E. Shamis [16] and K. Avrachenkov, P. Chebotarev, and A. Mishenin [13]) is a parameterized form of an inverse regularized Laplacian:

$$Q_\beta = (I + \beta L)^{-1} \tag{2}$$

The theorem states that entries in this proximity matrix turn out to be the probability that a node ends up sharing a tree with another node, in a randomly sampled spanning forest of the graph (hence the name). Since $I + \beta L$ is positive definite (non-singular), the inversion is guaranteed to exist, and will be provably *doubly stochastic*. These properties make it widely usable for many network analysis tasks. See the references above for numerous applications of this kernel, and its derived distances, which play a key role in the inference performed by `forest_pursuit`. Our implementation is slightly more efficient than using basic `np.linalg.inverse` and similar when inverting the regularized Laplacian: we directly interface with `dpotrf` and `dpotri` LAPACK routines via Scipy, for Cholesky inversion with cached indexing for positive definite matrices (like the regularized Laplacian).

Alternatively, an analyst might approximate a matrix with similar stochastic properties while avoiding matrix inversion altogether, via the Sinkhorn-Knopp algorithm (`sinkhorn`). It's also the basis for the `doubly_stochastic_filter` discussed above.

For our `filter` module, we have found Numpy's masked arrays to be particularly useful for retaining index and value information. For unsupervised "minimum-connected" thresholding, `affinis` has implemented a fast routine for removing edges until the graph is about to become disconnected (using a binary search and breadth-first connectivity checks).

Interestingly, in the limit of $\beta \to \infty$, the `distance.adjusted_forest_dists` will tend toward the Commute-time kernel (i.e. effective resistances)[13]. `generalized_graph_dists` also have this property, though in this diagonally-normalized case, the lower limit $\beta \to 0^+$ will also converge to a scalar multiple of the *shortest path distances*. In this way, the forest distances can be thought of as a smooth interpolation from the shortest paths to the effective resistances.

3. Reproducibility & Community Benchmarking (MENDR)

To enable broad cross-disciplinary participation and development toward improved network recovery from sparse/bipartite data, we propose a synthetically generated set of challenge problems, which we dub "MENDR": **M**easurement **E**rror in **N**etwork **D**iffusion & **R**econstruction

The core purpose of this collection is to provide a way for the network reconstruction community to benchmark their algorithms, and find more specific success/failure patterns among algorithm types.

3.1. Dataset Overview

To reflect common node-activation mechanisms, we first generate a set of "ground truth" graphs from a pre-determined set of graph types:

- **BL**: block network
- **TR**: tree network
- **SC**: Scale-free (Barabasi-Albert, $m \in \{1, 2\}$)

Then, for each graph G, a number of random walks with a randomly chosen number of node-to-node "jumps" (starting at randomly chosen "root" nodes) are sampled using the `CSRGraph` library for scalability and memory efficiency. The data X is generated as a one-walk-per-row, one-node-per column binary matrix. The goal of a given challenge is to recover the ground truth graph G, *using only the data* X.

Every pair of $\{G, X\}$ can be given a unique ID in MENDR. For human readability, we create an ID that starts with its graph-type code, followed by the number of nodes n, and the seed that generated the random sample, e.g:

```
BL-N030S01
BL-N030S02
...
TR-N100S10
TR-N100S11
...
```

The parameters not included in the design are sampled randomly, using distributions detailed in Table 1

Table 1. *Experiment Settings (`MENDR` Dataset)*

parameters	**values**
random graph **kind**	BL, TR, SC
network n**-nodes**	10,30,100,300
random **walks**	1 sample $m \sim \text{NegBinomial}\left(2, \frac{1}{n}\right) + 10$
random walk **jumps**	1 sample $j \sim \text{Geometric}\left(\frac{1}{n}\right) + 5$
random walk **root**	1 sample $n_0 \sim \text{Multinomial}(n, 1)$
random **seed**	1, 2, ... , 30

3.2. Serialization

Graph serialization is a long-standing issue in the network analysis community, since there are ambiguities in how (un)directed graphs, multi-graphs, hypergraphs, etc., are stored with respect to the data formats available. To reduce confusion and align directly with the input required by `affinis` tools, we have chosen to serialize the MENDR datasets as `json`,

containing two sparse array serializations and a list of visited nodes. Each json entry has attributes that directly correspond to instantiation parameters of sparse `COO` arrays from the `pydata/sparse` library. An example of this serialization scheme is shown in Figure 4

```yaml
SerialRandWalks:             # BL-N030S01
  graph:                     # SparseGraph
    format: COO
    shape: [30, 30]
    data_types:
      indices_0: [0, 0, 1, 1, 2, 3,  ...]
      indices_1: [4, 18, 25, 28, 24, 22, ...]
      values: 1
  jumps:
    - [0, 18, 24, ...]
    - [22, 27, 22, ... ]
    - ...
  activations:
    format: COO
    shape: [53, 30]
    data_types:
      indices_0: [0, 0, 0, 0, 0, 0, ...]
      indices_1: [0, 2, 4, 6, 10, 17, ...]
      values: 1
```

Figure 4. *Example serialization of MENDR dataset `BL-N030S01`. Rendered here in (abbreviated) YAML for readability.*

This data format schema is available as an importable python object, created using the json serialization tool `pyserde`. In addition to `sparse.COO` (and `CSR`), we have provided support for serialization of NetworkX graphs, `CSRGraph` arrays, and Scipy sparse arrays/matrices.

```python
from mendr.io import SerialSparse, SerialRandWalks
from pyserde.json import to_json

experiment = SerialRandWalks(
    SerialSparse.from_array(G),  # networkx.Graph, sparse.COO, etc.
    jumps,
    SerialSparse.from_array(X)
)
to_json(experiment)
```

Deserialization from `json` (into `sparse.COO`) is validated with `beartype` through `pyserde.json.from_json`. As further help, the MENDR repository contains an installable command-line interface to generate graphs and graph-recovery challenge datasets on the fly, as well as run a pre-defined set of algorithms against any number of datasets as a benchmark. This is how the MENDR results are calculated.

3.3. Benchmark Results

As an initial benchmark, methods from `affinis` were tested on the 4,320 currently generated MENDR datasets, along with an additional call to `sklearn.covariance.GraphicalLassoCV`. Because the quality of a network recovery depends on the threshold selected for edge relevance, and the standard setting for network recovery *does not have* a ground-truth network to recover, we instead look at aggregate performance metrics over the entire threshold span.

The results in Table 2 show a significant performance improvement of Forest Pursuit for minimum, median, and maximum MCC scores, which is the preferred metric for highly imbalanced prediction tasks like this [17].

Table 2. *Median (inter-quartile range) MENDR Benchmark results, reproduced from Table 6.3 in R. T. Sexton [11]*

method	APS	E[MCC]	MCC-max	MCC-min

Forest Pursuit	0.83 (0.21)	**0.73 (0.19)**	**0.86 (0.17)**	**0.75 (0.25)**
GLasso	**0.90 (0.13)**	0.47 (0.15)	0.85 (0.12)	0.72 (0.75)
Ochiai	0.79 (0.21)	0.38 (0.16)	0.74 (0.14)	0.59 (0.20)
Hyperbolic Proj.	0.59 (0.34)	0.34 (0.17)	0.55 (0.25)	0.35 (0.25)
Sinkhorn	0.47 (0.39)	0.28 (0.16)	0.49 (0.25)	0.39 (0.32)
HSS	0.33 (0.36)	0.23 (0.22)	0.50 (0.19)	0.25 (0.35)
Resource Proj.	0.27 (0.43)	0.22 (0.19)	0.44 (0.31)	0.36 (0.31)

GLasso still shows better performance with respect to APS, though for more investigation into the comparison of these two methods (and conditions under which Forest Pursuit can improve on GLasso's APS) see [11]

4. Scalable Metrology

In order to rapidly benchmark network recovery algorithms across thousands of challenge datasets (as in MENDR), a different set of tradeoffs is needed when actually computing the performance—i.e. when *doing metrology*.

First, for large networks, the number of *edges* that need to be assessed grows *quadratically* with the network size (number of nodes). This means, for instance, that a 1,000-node network will need to score a prediction set 3 orders of magnitude greater than that. Second, because edge predictions are generally scalar-valued (or probabilities), the actual performance of an algorithm differs depending on the chosen edge threshold, so *all thresholds* must be accounted for in the final performance estimate.

To assist with this, we have published a utility library called `Contingency`. This library provides a simple data structure to compute and store useful metrological information, and uses caching, subsampling, and vectorization tricks to ensure scalability for benchmarks like MENDR.

4.1. The `Contingent` Data Structure

Binary classification metrics are all about testing the quality of predicted labels against *true* labels you observed. Given a set of true and a set of predicted labels, a `contingency.Contingent` dataclass can be instantiated trivially:

```
from contingency import Contingent
M = Contingent(y_pred=y_pred, y_true=y_true)
```

Users would now have access to class properties that return useful metrics, calculated from the (true/false)-(positive/negative) contingency counts, such as:

- Positive Predictive Value (a.k.a. "Precision")
- True-Positive Rate (i.e. Sensitivity, or "Recall")
- Matthews Correlation Coefficient (MCC)
- F-score
- Fowlkes-Mallows index

However, most algorithms do not output binary classifications directly, but instead output probabilities (or weights). Thresholding these at varying levels will create an entire "family" of predictions as the threshold increases or lowers. This behavior is accessed via `Contingency.from_scalar`:

```
y_prob = np.array([0.1,0.8,0.1,.7,.25])
M_batch = Contingent.from_scalar(y_true, y_prob, subsamples=None)
```

From the resulting batched contingency count arrays, previously mentioned scores now become score *trajectories* over all thresholds. To report single scores once again, `Contingent` objects have a `.expected()` convenience method, which calculates the expected value for supported scores. One special behavior is for the Average Precision Score (`.expected('aps')`), which is the average precision weighted across the *recall* scores (not thresholds).

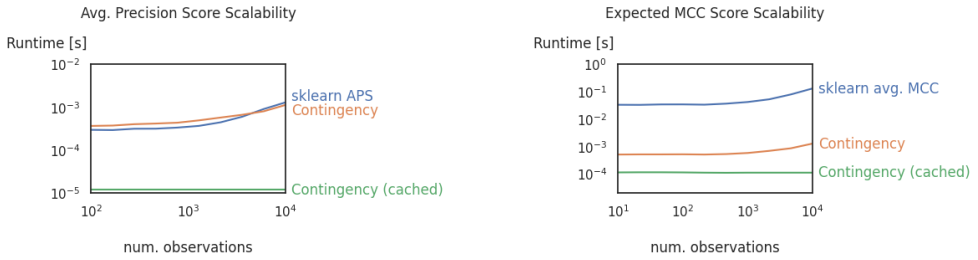
One common tool in these cases is a *Precision-Recall* (P-R) curve. To simplify repetitive reporting for these plots, we've provided a simple template `matplotlib.axes.Axes` object, accessible by importing plot utility `contingency.plots.PR_contour`. This will set up a correctly-scaled P-R plot, with included isoclines for implied `F_1` and `fowlkes_mallows` scores.

4.2. Techniques for Scalability

`Contingent.from_scalar` handles the calculation of score trajectories as a simple broadcasting operation, since all contingency counts and derived metrics have enabled a *batch dimension*. In this case, rather than looping over possible thresholds, we make use of `numpy.less_equal.outer`, a so-called `ufunc` that applies the thresholding operation to all pairs of edges and thresholds, and vectorized through its C-backend. In addition, once the contingency counts are found, all future metric requests are derived from the cached, batched-dimension (true/false)-(positive/negative) counts. Unlike other binary performance metrics libraries, `Contingent` objects can be instantiated once-per-dataset, and all derived metrics only require simple arithmetic operations through Numpy. For this reason, while `Contingency` is already competitive with (or even *much faster* than) Scikit-Learn's equivalent methods, actual use of `Contingency` will involve cached calculations that make metric calculation time functionally negligible. The user API is shown in Figure 5, with performance benchmarks across a wide range of dataset sizes shown in Figure 6.

```
rng = np.random.default_rng(24)
y_true = (y_src := rng.random(1000))>0.7
y_pred = y_src + 0.05*rng.normal(size=1000)

# Contingency
Contingent.from_scalar(y_true, y_pred).expected('aps') # uncached (APS)
M = Contingent.from_scalar(y_true, y_pred)
M.expected('aps')                                      # cached (MCC)

#Scikit-Learn
from sklearn.metrics import average_precision_score
average_precision_score(y_true, y_pred)                # sklearn APS
np.mean([
    matthews_corrcoef(y_true,x) for x in M.y_pred      # sklearn E[MCC]
])
```

Figure 5. *Example API use for `Contingent` objects.*



(a) (b)

Figure 6. *Comparison of* `Contingent.expected` *performance to equivalent scikit-learn tools for Average Precision Score (APS) and expected MCC (over all thresholds). Both non-cached (calling* `from_scalar` *each time) and cached versions are shown, and* `subsamples=50` *was used in each case.*

The limit to this vectorized caching comes from memory limitations, since the outer-product matrix we use to vectorize contingency counting grows quickly with graph size. To mitigate this, we have provided a `subsamples` option in `Contingent.from_scalar`, which will approximate the threshold values using linear interpolation of the original unique threshold set. This distributes the sample locations according to the density of the original threshold values while limiting the size of the batch dimension needed for contingency count caching.

As shown in Figure 7, with only a few subsamples, the score curves quickly converge to their "true" values. This allows `Contingent` objects to handle the large datasets, such as the 10^4 dataset shown in Figure 6 (where `subsamples=50` was used).

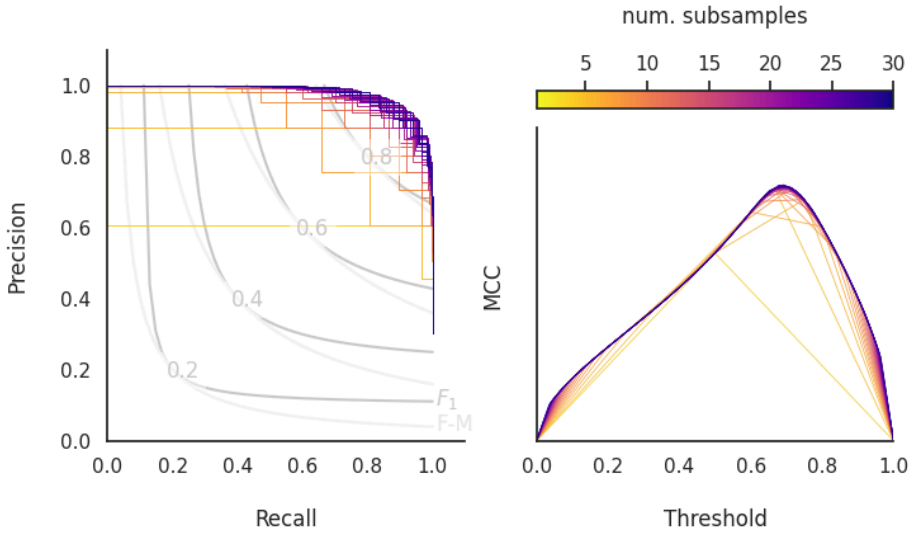


Figure 7. *Demonstration of the convergence of subsampling in* `Contingency.from_scalar`

5. Future Work

The ecosystem presented here is in its early days, and many key extensions will bring new functionality, improved performance, and enhanced interoperability to enable community network metrology.

Currently, `affinis` only makes use of sparse representations in the *feature space*, but after the gram matrix $X^T X$ is calculated, the now-square kernel is stored densely. While this is important in cases where beta-posterior kernels are needed, it is possible to limit smoothing to cases where node co-occurrence actually happened, or to calculate smoothing on-demand downstream. This is also relevant because the graph recovery tooling in

`affinis.filter` is already re-sparsifying kernels, anyway, so a significant amount of memory and compute can be saved by enabling universal sparse compute. The key improvement needed is a sparse-reimplementation of the `scipy.spatial.distance.squareform` function, which `affinis` uses for much of its downstream edge processing. This can be accomplished using the *closed form* index mappings detailed in M. Angeletti, J.-M. Bonny, and J. Koko [18], and further expounded on for edge vector representation in R. T. Sexton [11]

Secondly, support for further matrix manipulation is planned, with many of the kernels on graphs discussed in K. Avrachenkov, P. Chebotarev, and D. Rubanov [15] being particularly straightforward to represent in our codebase. A new mechanism to fold/unfold node-based vector representations into edge-based representations would also be enabled by our upcoming sparse `squareform` method, which would support graph matrix manipulation required by several optimal matching algorithms, such as in C. J. Taylor [19].

For `MENDR` (as with most benchmarks), increased coverage and the ability to more severely challenge modern algorithms would be welcome, such as with a 1000+ node graph series. In addition, since we have preserved the original "jump" list that generated the node activations, it is possible to add an additional challenge to approximate the jump-list rank ordering from the node activations. Alternatively, we could re-create the setting investigated by M. Gomez-Rodriguez, J. Leskovec, and A. Krause [20], where the ordered list of jumps is used to attempt reconstruction of the graph.

Although the graph serialization was done with a custom parser, we would additionally foresee interoperability with upcoming standards for hypergraph [21] and sparse-matrix [22] representation on-disk as important improvements for MENDR.

Lastly, for `Contingency`, several more kinds of accuracy "trajectories" (and their corresponding plots) would be useful to the community, specifically Receiver Operating Characteristic (ROC) plots. It would also be possible to wrap `Contingent` objects in a Scikit-Learn-compatible API that would let them be used in pre-existing data pipelines for scoring binary classifiers.

6. Conclusion

We have introduced three open-source Python toolkits that together form the beginnings of improved research infrastructure for network reconstruction from binary data.

- **`affinis`** is the core algorithm library, offering a consistent interface to a broad taxonomy of association measures, from marginal co-occurrence probabilities and cosine similarity, to bipartite projection methods (hyperbolic weighting, resource allocation), graph backboning algorithms (high-salience skeleton, doubly-stochastic filter), and probabilistic graphical model recovery (Chow-Liu trees).

A central contribution is a reference implementation of *Forest Pursuit*, which is lazily executable, trivially parallelizable, and scales approximately linearly with the feature matrix size for diffusion-like problems.

- **`MENDR`** provides a standardized benchmark suite for the network reconstruction community. It includes a library of synthetic ground-truth graphs with configurable observation-generation processes, as well as a CLI for both generating novel experimental datasets and running experiments against the existing standard datasets. `MENDR` addresses a critical gap in the field: without shared benchmarks, comparing reconstruction methods across papers and communities has been effectively impossible.
- **`Contingency`** provides fully vectorized implementations of binary classification metrics. Its central `Contingent` data structure treats threshold sweeps as a broadcasting operation over predicted probability arrays, enabling efficient batched

evaluation and caching. Supported metrics include Matthews Correlation Coefficient (MCC), F-score, Fowlkes-Mallows index, and average precision score. `Contingency` serves as a metrological foundation for assessing reconstruction performance, which enables consistent benchmarking across the entire framework.

`affinis`, `MENDR`, and `Contingency` together provide the scientific Python community with a reproducible, interoperable toolkit for network analysis and metrology. We welcome community contributions, additional algorithm implementations, and new benchmark datasets.

References

[1] T. P. Peixoto, "Network reconstruction via the minimum description length principle," *Phys. Rev. X 15, 011065 (2025)*, vol. 15, no. 1, p. 11065, May 2024, doi: 10.1103/physrevx.15.011065.

[2] L. Torres, A. S. Blevins, D. Bassett, and T. Eliassi-Rad, "The Why, How, and When of Representations for Complex Systems," *SIAM Review*, vol. 63, no. 3, pp. 435–485, 2021, doi: 10.1137/20M1355896.

[3] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proceedings of the 7th Python in Science Conference*, in SciPy. 2008, pp. 11–15. doi: 10.25080/tcwv9851.

[4] S. Janson and J. Vegelius, "Measures of ecological association," *Oecologia*, vol. 49, no. 3, pp. 371–376, 1981, doi: 10.1007/bf00347601.

[5] G. U. Yule, "On the Methods of Measuring Association Between Two Attributes," *Journal of the Royal Statistical Society*, vol. 75, no. 6, p. 579, 1912, doi: 10.2307/2340126.

[6] M. E. J. Newman, "Scientific collaboration networks. II. Shortest paths, weighted networks, and centrality," *Physical Review E*, vol. 64, no. 1, p. 16132, 2001, doi: 10.1103/physreve.64.016132.

[7] T. Zhou, J. Ren, M. Medo, and Y.-C. Zhang, "Bipartite network projection and personal recommendation," *Phys. Rev. E*, vol. 76, no. 4, p. 46115, 2007, doi: 10.1103/PhysRevE.76.046115.

[8] D. Grady, C. Thiemann, and D. Brockmann, "Robust classification of salient links in complex networks," *Nature Communications*, vol. 3, no. 1, 2012, doi: 10.1038/ncomms1847.

[9] P. B. Slater, "A two-stage algorithm for extracting the multiscale backbone of complex weighted networks," *Proceedings of the National Academy of Sciences*, vol. 106, no. 26, p. E66–E66, June 2009, doi: 10.1073/pnas.0904725106.

[10] C. Chow and C. Liu, "Approximating discrete probability distributions with dependence trees," *IEEE Transactions on Information Theory*, vol. 14, no. 3, pp. 462–467, 1968, doi: 10.1109/tit.1968.1054142.

[11] R. T. Sexton, "Measuring Network Dependencies From Node Activations." 2025. doi: 10.13016/o252-lfmi.

[12] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3rd ed. 2025. doi: 10.1162/coli.B09-001.

[13] K. Avrachenkov, P. Chebotarev, and A. Mishenin, "Semi-supervised learning with regularized Laplacian," *Optimization Methods and Software*, vol. 32, no. 2, pp. 222–236, Mar. 2017, doi: 10.1080/10556788.2016.1193176.

[14] M. Cuturi, "Sinkhorn distances: Lightspeed computation of optimal transport," in *Proceedings of the 27th International Conference on Neural Information Processing Systems*, in NIPS'13. 2013, pp. 2292–2300. doi: 10.48550/arXiv.1306.0895.

[15] K. Avrachenkov, P. Chebotarev, and D. Rubanov, "Similarities on graphs: Kernels versus proximity measures," *European Journal of Combinatorics*, vol. 80, pp. 47–56, Aug. 2019, doi: 10.1016/j.ejc.2018.02.002.

[16] P. Chebotarev and E. Shamis, "The Forest Metrics for Graph Vertices," *Electronic Notes in Discrete Mathematics*, vol. 11, pp. 98–107, 2002, doi: 10.1016/s1571-0653(04)00058-7.

[17] D. Chicco and G. Jurman, "A statistical comparison between Matthews correlation coefficient (MCC), prevalence threshold, and Fowlkes–Mallows index," *Journal of Biomedical Informatics*, vol. 144, p. 104426, Aug. 2023, doi: 10.1016/j.jbi.2023.104426.

[18] M. Angeletti, J.-M. Bonny, and J. Koko, "Parallel Euclidean distance matrix computation on big datasets," 2019. [Online]. Available: https://hal.science/hal-02047514

[19] C. J. Taylor, "On the optimal assignment of conference papers to reviewers," 2008, [Online]. Available: https://www.engineering.upenn.edu/~cjtaylor/PUBLICATIONS/pdfs/TaylorTR08.pdf

[20] M. Gomez-Rodriguez, J. Leskovec, and A. Krause, "Inferring Networks of Diffusion and Influence," *ACM Transactions on Knowledge Discovery from Data*, vol. 5, no. 4, pp. 1–37, 2012, doi: 10.1145/2086737.2086741.

[21] M. Coll *et al.*, "HIF: The hypergraph interchange format for higher-order networks," *Network Science*, vol. 13, p. e21, 2025, doi: 10.1017/nws.2025.10018.

[22] B. Brock, T. Davis, J. Kitchen, E. Welch, I. Virshup, and W. Ahrens, "Binary Sparse Format Specification Version 0.1." [Online]. Available: https://graphblas.org/binsparse-specification/

Open Access | https://doi.org/10.25080/cyth6293

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

A Reproducible Data Lakehouse for High-Resolution Gastric Cancer Epidemiology Study in Chile

Rene Lagos[1,2,3], **Cristóbal Cuadrado**[4,2], **Francisca Jofré**[3], and **Arnoldo Riquelme**[2,5,6]

[1]Programa de Doctorado en Salud Pública, Instituto de Salud Poblacional, Facultad de Medicina, Universidad de Chile, [2]Centro para la Prevención y Control del Cáncer (CECAN), [3]Grupo de Ciencia de Datos para la Salud Pública, [4]Escuela de Salud Pública, Universidad de Chile, [5]Department of Gastroenterology, Faculty of Medicine, Pontificia Universidad Católica de Chile, [6]Instituto de Medicina, Red Salud UC Christus

Abstract

Cancer is the main cause of mortality in Chile since 2019. National surveillance of cancer incidence, often requires merging massive and heterogeneous datasets to generate high resolution insights. For a PhD researcher in Public Health set to understand the spatio-temporal trends of cancer incidence, the lack of high-performance server infrastructure and limitations of traditional epidemiological tools when handling longitudinal Big Data on local machines can act as substantial barriers.

We developed a portable, open-source Data Lakehouse architecture built entirely within the Python ecosystem to study gastric cancer incidence in the Chilean population. For this purpose, we used anonymized longitudinal data, over 21 years and 20 million inhabitants including mortality records, hospital discharges, and insurance claims by age, sex, and county, which were stored using Parquet files and Google Drive provided by the University of Chile. We implemented a modular ELT workflow using Jupyter Notebooks and Google Colab, where each notebook was designed to perform a specific extraction, normalization or loading process for each dataset. In addition, we employed `rpy2` and `mgcv` packages for age-standardized time-series modeling, GeoPandas and Sci-kit for spatial analysis and identification of high-risk zones; and NetworkX to model patient-consultation trajectories, identifying gastroenterology referral clusters in the national health system. Finally, we developed a Streamlit dashboard that allows team members and the general public to visualize cancer incidence time-series charts and referrals clusters dynamically.

This architecture served as the analytical backbone for a PhD thesis, supporting the submission of three conference abstracts and three manuscripts to public health journals, ensuring fully reproducible and documented results. This project demonstrates that high-resolution, national-level public health research does not require expensive enterprise software and can be enabled by the production of public health data infrastructure. By utilizing a version-controlled, cloud-collaborative, and nearly zero-cost Python stack, we provide a blueprint for researchers in resource-limited settings to conduct reproducible epidemiological surveillance.

Keywords Data Lake, Cancer Epidemiology, Health Data Pipeline

Published Jul 13, 2026

Correspondence to
Rene Lagos
rlagos@uchile.cl

Open Access

Copyright © 2026 Lagos *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Cancer is a priority health problem in Chile. It is currently the leading cause of death and the main burden of disease in the country [1]. Gastric cancer (GC) is particularly prevalent, with Chile having one of the highest incidence and mortality rates in the world alongside with some East Asian countries [2].

Chile maintains population-based cancer registries (PCRs) that collect information on all cancer cases across a set of regions for epidemiological surveillance and planning cancer control policies. These registries cover approximately 20% of the national population; however, estimates are reported at the national level, and updated official figures are sparsely reported (not been available since 2019).

We set out to update estimates of GC incidence at the provincial level to support the planning and evaluation of prevention policies. This required applying state-of-the-art methodologies, which demand a large amount of longitudinal data on mortality, hospital discharges, population by sex and age, and socioeconomic variables.

Handling this data with programming languages typically used in epidemiology, such as Stata and R, posed a significant computational risk. Public health researchers often conduct studies using the same hardware used for office tasks, as they use statistics aggregated at the national level. For higher geographic granularity and longer period studies, as in this case, high-performance computing was necessary but not available. Furthermore, tools like R and RStudio use in-memory processing, meaning the entire dataset must be loaded into the computer's RAM for analysis. When handling two decades of longitudinal data for a population of 20 million inhabitants—including heterogeneous records such as mortality, hospital discharges, and insurance claims—the resulting datasets can easily exceed dozens of gigabytes. Attempting to merge and normalize these massive datasets on a typical laptop or office computer could have led to system crashes or "out of memory" errors.

The aim of this study was to develop a portable, open-source "Data Lakehouse" architecture to study Gastric Cancer incidence in Chile's population. Instead of a local, monolithic approach, we moved to a modular, cloud-native architecture that enabled analysis to run from any computer.

2. Methods

2.1. Data Architecture

We implemented a Data Lakehouse architecture to bridge the gap between the flexible, cost-effective storage of a data lake and the rigorous, analytical structure of a data warehouse [3]. This hybrid approach allowed us to store 21 years of longitudinal population data in a central repository before subjecting it to specific transformation processes. We utilized an Extract-Load-Transform (ELT) workflow, loading raw datasets with basic normalization into our lake before transforming them into multidimensional structures optimized for epidemiological analysis.

To manage the complexity of transforming heterogeneous health records, we adopted the Dynamic ETL (D-ETL) framework, an approach that combines automatization of the transformation with manual specifications [4]. Traditional GUI-based ETL tools often lack the flexibility required for the complex requirements of health data and suffer from a lack of transparency in their underlying transformation logic. Following this approach, we modularized our data processing by mapping and transforming each dataset (e.g., mortality records, hospital discharges) separately, allowing us to automate repetitive tasks using

scalable, reusable, and customizable Python code while retaining manual control over complex mappings for epidemiological analysis.

2.2. Workflow Implementation

We took advantage of University of Chile's Google Workspace to use low-cost cloud storage in Google Drive. This also ensured that data storage complied with the University's privacy and security policies. The workflow was arranged in three main folders: 1) Original Data, for the original files (zip, xls, csvs, etc) and documentation received and organized in subfolders by data source, 2) Raw Data, for the data extracted into optimized format for intake, and 3) Data Warehouse, for normalized and standardized data. Additionally, we created a separate folder for each analysis, containing the data transformation script and the statistical analysis.

Jupyter Notebook, a friendly and open source web platform [5], was used as the coding environment and enabled us to combine data-mapping information in text cells with Python code to generate each target dataset. For each analysis, we created a multidimensional table (cube) and its corresponding Jupyter notebook. All notebooks used Pandas library [6], [7] and were generated and executed using Google Colab, which connected to Drive folders to load and store datasets and ran on Google servers. Since we used the free version, we had a limit of 12 GB RAM and 50 GB of memory per session.

We used Parquet format to store all datasets due to its high performance for storage and loading. Also, it allowed for storing text metadata, such as text encoding, which is often problematic in Spanish due to special characters. Figure 1 shows the tools used.

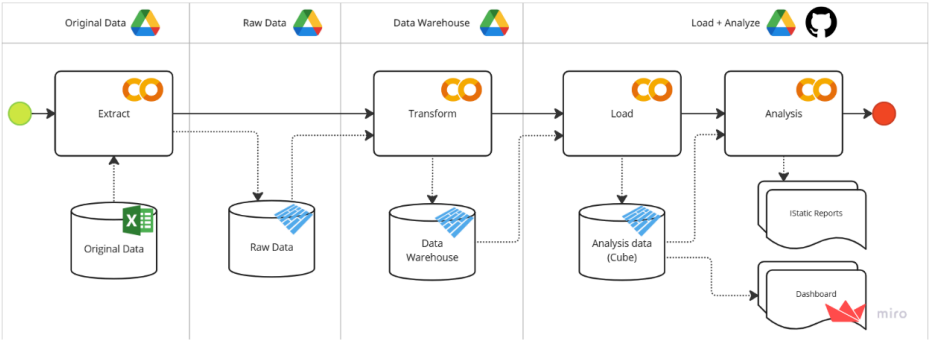


Figure 1. *Tools used to implement Dynamic Extraction-Load-Transform workflow.*

2.3. Data model and sources

We used an Effective Coverage framework that distinguishes populations, needs, resources, production and health outcomes [8] and the Chilean Norm for Health Information for data modeling [9], [10]. Figure 2 shows the entities, attributes and relationships used to model GC prevention that were stored in the data lake. Figure 3 shows the star-schema for the data cube (multidimensional table) used to analyze GC incidence.

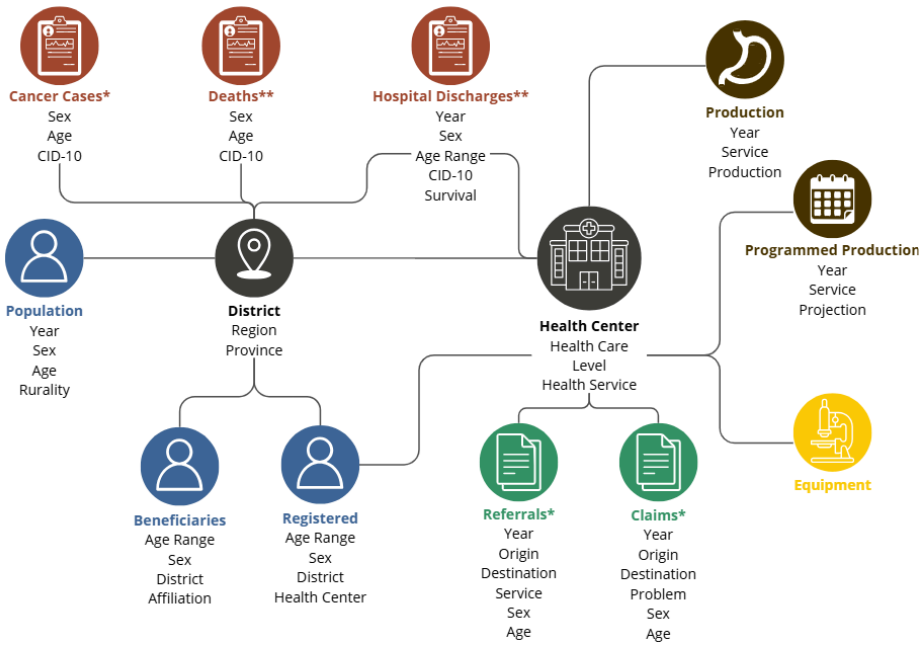


Figure 2. *Entity-Relationship Diagram. Colors: blue=populations; yellow=resources; brown=production; green=needs; red=health outcomes. *Individual-level data. **Linkable individual-level data.*

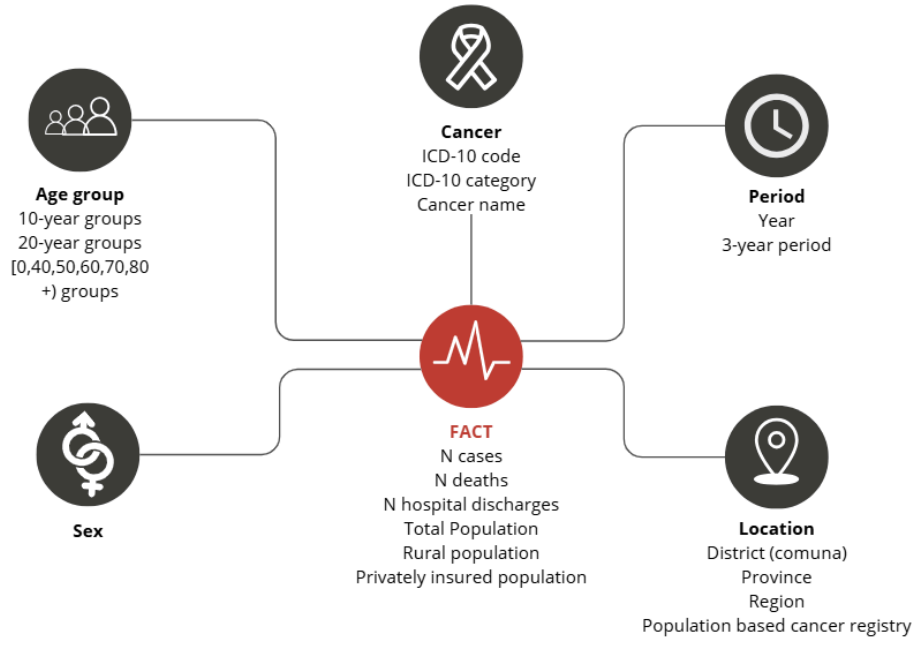


Figure 3. *Star schema of data cube used to analyze gastric cancer incidence in Chile 2003-2024.*

We retrieved GC cases (ICD-10 category "C16") from the PBCR database (1998–2019) via the Ministry of Health website. Upon request, the Department of Health Statistics and Information (DEIS) provided individual-level records for GC deaths and hospital discharges (2003–2024). We obtained age- and sex-specific population projections and rurality data from the National Statistics Institute (INE). Finally, we sourced insurance coverage data from public health insurer FONASA (2003–2024), and from the Superintendency of Health for ISAPRE beneficiaries (2010–2024); poverty percentages from the Ministry of Social Development;

and consultations and UGE production in public hospitals from monthly statistical reports from DEIS (2009-2024).

2.4. Statistical methods and packages

To estimate GC incidence, we used mortality-incidence ratios [11] implemented with Quasi-Poisson regression models with R's `mgcv` library [12]. We embedded R scripts in Python code using the `rpy2` library [13]. We used `NetworkX` to generate health center networks for GC diagnosis, based on referral patterns [14]. We used `Geopandas` for maps [15] and `Streamlit` to publish a dashboard [16].

3. Results

Table 1 shows the Jupyter notebooks developed. Since each original dataset had a unique script associated, notebooks were stored in the Original Data folder, in their corresponding subfolder (Figure 1). During the loading stage, each normalized entity had its own script, so notebooks were stored in the Data Warehouse folder.

Table 1. *Jupyter notebooks developed.*

Original Data *Extraction*	***Loading* to Data Warehouse**	***Transformation* Multidimensional Tables**
Population estimates, Poverty estimates, FONASA Beneficiaries, ISAPRE Beneficiaries, Health center registered population, Health centers, Production, Claims, Referrals, Population-Based Cancer Registry, Hospital discharges, Deaths, Administrative maps	District Population, FONASA Beneficiaries, ISAPRE Beneficiaries, Health Center Population, Health centers, Consultation production, Endoscopy production, Cancer referrals, Non-cancer referrals, Gastric cancer cases, Hospital discharges, Deaths, DIM Age, DIM Sex, DIM Districts, DIM Cancer, GEO Provinces, GEO Health services.	Population Based Cancer Registry cube, Hospital discharges and deaths events, Gastric cancer cube, Health centers cube, Registered population cube, Referrals fact, Diagnosis coverage cube

DIM tables were dimensions common to all analyses used to optimize data normalization and integration. For instance, in `DIM_Age`, for each age `[0, 1, …, 100]`, age ranges of 5 years, 10 years, and other custom ranges are specified. This facilitated the normalization and integration of datasets with different age ranges.

Table 2. *Data Lakehouse tables, columns and size.*

Entity type	**Table**	**Columns**	**Entries**	**Colab Memory Usage**	**File size**
Populations	DW_POBLACION_COMUNA	Comuna, Sexo, Edad, Poblacion, Año, Porc_rural_comuna	1905768	87.2 MB	3.2 MB
	DW_BENEFICIARIOS_FONASA	Año, Comuna, Sexo, RangoEdad10, Tramo, Beneficiarios	580445	32.1 MB	3.6 MB

	DW_BENEFICIARIOS_ISAPRE	Año, Sexo, Comuna, RangoEdad5, Beneficiarios	270572	12.4 MB	2.3 MB
	DW_INSCRITOS	Año, Id_establecimiento, Comuna Id, Sexo, Edad, N_inscritos	186100	9.9 MB	424 KB
Needs	DW_SIC_GES	Año, Mes, Origen, Destino, Sexo, RangoEdad5, Presta_min, Desc_garantia, N_Problema_Salud, Cod_Beneficiario	200236	15.3 MB	1.9 MB
	DW_SIC_SIGTE	Sexo, Presta_min, Especialidad, Origen, Destino, C_SALIDA, Archivo, Año, Mes, Edad, SIC	501832	42.1 MB	881 KB
Production	DW_CENTROS	Codigo, Nombre, Comuna, Servicio, Servicio_nombre, Region, Nivel, Tipo, Complejidad, Latitud, Longitud	4582	393.9 KB	205 KB
	DW_PRODUCCION_REM_CONSULTAS	Año, Mes, Codigo, Prestacion, Produccion, ProduccionHombres, ProduccionMujeres, ProduccionCNE, ProduccionCNE_TM, ProduccionTM, nsp, nsp_cne	168150	15.4 MB	493 KB
	DW_PRODUCCION_REM_PROCEDIMIENTOS	Codigo, Prestacion, Año, Mes, Produccion	83749	4.1 MB	728 KB
	DW_ENDOSCOPIAS_SIL	Prestacion, Comuna, RangoEdad10, Sexo, Año, Bonos, BonosEst	163626	10.5 MB	1.5 MB

Health Outcomes	DW_CASOS_RPC	idComuna, Sexo, Edad, Mes_diag, Año_diag, CIE10, RPC, Ncasos	79705	5.2 MB	4.5 MB (pkl file)
	DW_EGRESOS_CA	idPersona, Sexo, Edad, Provincia, Comuna, Prevision, Año, CIE10, CondicionEgreso, CodigoEst	599288	46.3 MB	35.8 MB
	DW_DEFUNCIONES_CA	idPersona, Año, Sexo, Edad, Provincia, Comuna, CIE10, idLugarDefuncion, LugarDefuncion	210994	14.5 MB	14.1 MB
Dimensions	DIM_EDAD	Edad, RangoEdad5, RangoEdad10, RangoEdad20, RangoEdad40, RangoEdad4080	101	12.8 KB	12 KB
	DIM_SEXO	Codigo, Sexo, Sexo 2	5	252 bytes	3 KB
	DIM_COMUNAS	Comuna, Nombre Comuna, Provincia, Nombre Provincia, idServicio, Servicio, Region, Nombre Region, Macrorregion	346	24.5 KB	14 KB
	DIM_CANCER	CIE10, Cancer, Categoria, Digestivo	412	13 KB	9 KB
Geometries	GEO_PROVINCIAS	Provincia, geometry	56	1.0 KB	70.6 MB
	GEO_SERVICIOS	Servicio, geometry	29	596.0 bytes	76.5 MB

Cubes	CUBO_incidencia	idComuna, Año, RPC_nombre, RPC, Sexo, RangoEdad, Nombre_Cancer, Nombre Comuna, Poblacion, Ncasos, Ndefunciones	120240	11.0 MB	4.8 MB (xlsx file)
	CUBO_CANCER_DIGESTIVO	idComuna, Nombre Provincia, Nombre Region, Region, Macrorregion, Año, Sexo, RangoEdad10, RangoEdad4080, MedianaRangoEdad10, MedianaRangoEdad4080, Poblacion, PoblacionRural, Fonasa A, Fonasa B, Fonasa C, Fonasa D, Fonasa, Isapre, Categoria, Cancer, RPC, Casos, Defunciones, Egresos_Defunciones, Egresos	24948	4.9 MB	692 KB
	CUBO_CENTROS	Codigo, Nombre, Nivel, Tipo, Complejidad, Comuna, Servicio, Latitud, Longitud, Nombre Comuna, Nombre Region, Porc_rural_comuna, Destino GES	4582	465.5 KB	218 KB
	CUBO_DERIVACIONES_MDD	Edad, Destino, Código prestación, COMGES, GES, Derivaciones	6762	317.1 KB	33 KB

CUBO_INSCRITOS_DERIVACIONES	Origen, Sexo, Rango Edad, Año, Inscritos, Código prestación, Prestación, Tipo prestación, Especialidad, GES, Derivaciones	906782	76.1 MB	576 KB
CUBO_COBERTURA_SERVICIOS	Servicio, Codigo, RangoEdad4080, Año, Casos, Poblacion, PoblacionRural, IncidenciaEst, Consultas, Endoscopias, Confirmacion_C16, Confirmacion_HP, Tratamiento_C16, Tratamiento_HP, Egresos, Defunciones, Isapre, Fonasa A, Fonasa B, Fonasa C, Fonasa D, Servicio, Macrorregion, Sexo, Periodo	10424	2.4 MB	429 KB

The developed data lakehouse has demonstrated significant utility in supporting multiple epidemiological analyses and the dissemination of PhD thesis results.

3.1. GC incidence scripts

The GC incidence scripts were used to prepare a congress poster and a scientific journal manuscript [17]. This last version was published in a Github repository as part of a manuscript publication.

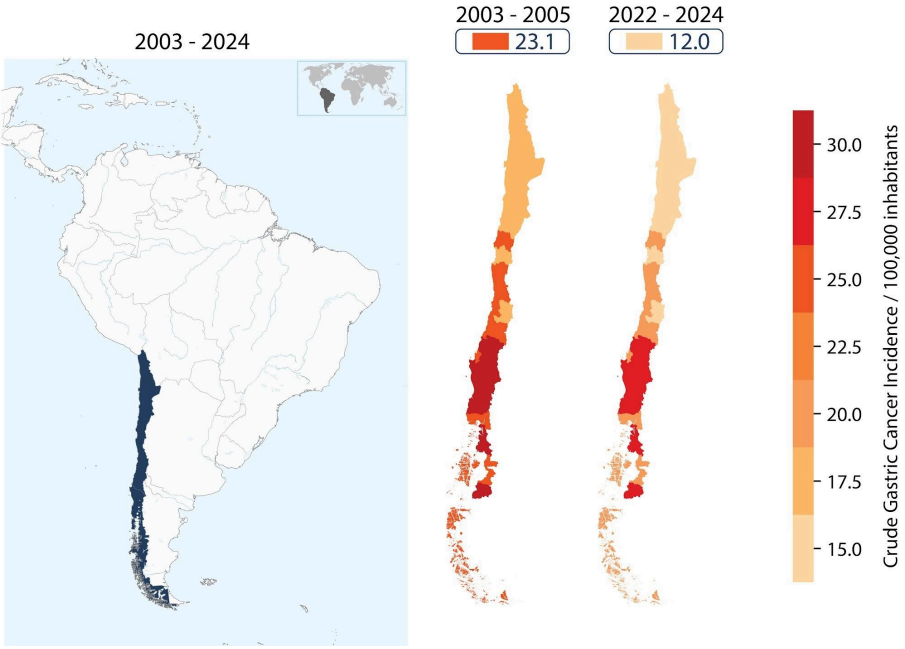


Figure 4. *Variation of gastric cancer incidence across risk clusters and periods.*

We identified a high-risk cluster in southern Chile and submitted our findings to scientific congresses and a peer-reviewed journal, sharing all data and scripts to ensure full reproducibility.

3.2. Diagnosis network scripts

The digestive cancer diagnosis network scripts were used to generate a report for the Ministry of Health, to present a poster at a congress, and to prepare a scientific journal manuscript [18]. A GitHub repository was published to support the paper results.

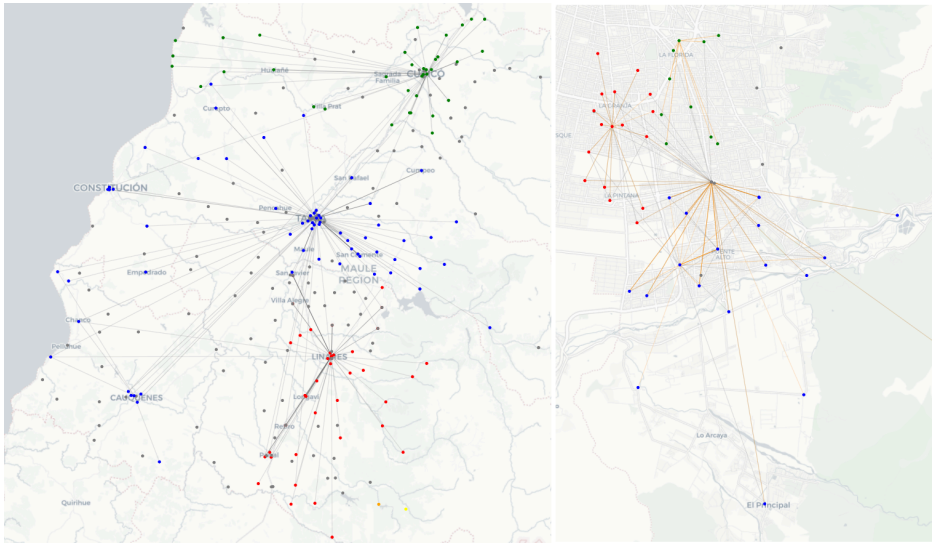


Figure 5. *Gastric cancer diagnosis networks identified in rural and urban regions using NetworkX.*

Networks identified in Maule Health Authority (left) and South East Metropolitan Health Authority (right). In total, eleven health authorities were divided into local diagnosis networks, allowing higher granularity of cancer diagnosis coverage metrics.

3.3. Diagnosis coverage scripts

Finally, the diagnosis coverage scripts were used for a congress poster and a manuscript [19]. A Github repository was published to support the manuscript submission. The dashboard was published in streamlit cloud and used in congress presentations, but most often for personal use: tesisrenelagos.streamlit.app.

4. Discussion

We implemented a Data Lakehouse architecture to bridge the gap between the flexible, cost-effective storage of a data lake and the rigorous analytical structure of a data warehouse. We utilized a modular ELT workflow, loading raw datasets into our central repository with basic normalization before transforming them into multidimensional structures optimized for epidemiological analysis. Our pipeline organized both data and code into clear stages —progressing from Original Data to a finalized Data Warehouse—effectively merging the high-volume storage capacity of a lake with the relational consistency of a warehouse.

To manage the complexity of heterogeneous records, we adopted a D-ETL approach and implemented a "one dataset, one notebook" model. This provided the transparency of Python often lacking in GUI-based tools, where underlying transformation logic is frequently hidden. By embedding rules, assumptions, and mapping specifications as text cells directly alongside executable code, we created a single, human-readable file for each target dataset. This integrated format facilitated an iterative process of validation and debugging, ensuring that every step of the analytical logic remained visible and easily scrutinized. Furthermore, when using adequate keys and descriptions, narrows the gap between non data experts (like clinical professionals) so they not only can audit but also, reuse it for other purposes.

By using Google Colab and Drive, we established a portable, cloud-native infrastructure that bypassed the lack of high-performance server capability often found in public health research. This ready-to-use environment turned standard cloud tools into a secure environment, ensuring that the analysis of data complied with institutional security policies while providing a shared, collaborative platform for researchers. By partitioning the pipeline into 38 discrete Jupyter Notebooks we avoided hitting the 12GB RAM and 50GB storage limits of the free-tier environment while providing an iterative, lightweight process that effectively managed 21 years of records for 20 million inhabitants.

This cloud-native environment facilitated statistical workflow through the `rpy2` library; we leveraged Python's high-performance ecosystem for data engineering, while transitioning seamlessly into R for specialized modelling (e.g. Quasi-Poisson incidence). This interoperability eliminated the need for manual, error-prone data exports between software environments.

A critical choice in this implementation was the use of the Parquet columnar storage format, which provided substantial storage efficiency and performance gains. For example, a population dataset with nearly two million entries was compressed to just 3.2 MB on disk. Beyond speed, Parquet's ability to store text metadata solved persistent encoding issues with Spanish characters in Chilean records, thereby safeguarding data integrity across heterogeneous sources.

Because the system follows the Effective Coverage framework, it can be easily replicated for colorectal or prostate cancer by simply mapping the "Need" and "Production" required datasets. Relational integrity is maintained through the data model based on the Chilean Norm for Health Information, ensuring that standardized attributes—such as sex, age, and county IDs—remain consistent across heterogeneous sources. This design allows for expansion by adding new records to existing tables with minimal structural changes.

Furthermore, the strategic implementation of common Data Warehouse dimensions, such as DIM_Age, enables the integration of new datasets with varying granularities without requiring a rewrite of the core transformation logic.

4.1. Limitations

While the free version of Google Colab facilitates access, it remains necessary to conduct a rigorous pre-loading anonymization process to ensure no Protected Health Information enters the public cloud environment. A standard implementation for non-anonymized records would require an ETLT (Extract-Transform-Load-Transform) workflow that enables a pre-load transformation stage to mask or encrypt sensitive identifiers before data reaches a cloud-based staging area, ensuring compliance with privacy regulations.

Furthermore, although the Chilean Norm for Health Information provided a robust national framework, it faces compatibility challenges compared to international benchmarks. Transitioning to the OMOP Common Data Model would be the next step for international and regional interoperability. Finally, while manual execution of D-ETL provided transparency and flexibility for academic research and PhD-scale projects, enterprise-level scaling would require automated pipelines and workflow orchestrators capable of managing higher data volume and velocity.

This study proves that high-resolution epidemiological research is feasible on standard hardware with nearly zero software costs. The Data Lakehouse, D-ELT, architecture serves as a scalable blueprint for researchers in resource-limited settings to conduct national-level research, lowering technical barriers while ensuring the reproducibility essential for public health research.

References

[1] M. A. Martínez-Sanguinetti, A. M. Leiva-Ordoñez, F. Petermann-Rocha, and C. Celis-Morales, "¿Cómo ha cambiado el perfil epidemiológico en Chile en los últimos 10 años?," *Revista médica de Chile*, vol. 149, no. 1, pp. 149–152, 2021, doi: 10.4067/S0034-98872021000100149.

[2] Ferlay J *et al.*, "Global Cancer Observatory: Cancer Today." [Online]. Available: https://gco.iarc.who.int/today/

[3] M. Cilia and S. Buttigieg, "Health Data Pipelines: Moving Away from Excel to Scalable, Sustainable, Insightful, and Future-Proof Infostructure," in *Digital Public Health*, H. Zeeb, L. Maaß, T. Schultz, U. Haug, I. Pigeot, and B. Schüz, Eds., Cham: Springer Nature Switzerland, 2025, pp. 585–612. doi: 10.1007/978-3-031-90154-6_30.

[4] T. C. Ong *et al.*, "Dynamic-ETL: a hybrid approach for health data extraction, transformation and loading," *BMC Medical Informatics and Decision Making*, vol. 17, no. 1, p. 134, 2017, doi: 10.1186/s12911-017-0532-3.

[5] T. Kluyver *et al.*, "Jupyter Notebooks - a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Scmidt, Eds., Netherlands, 2016, pp. 87–90. [Online]. Available: https://eprints.soton.ac.uk/403913/

[6] The Pandas Development Team, "pandas-dev/pandas: Pandas." [Online]. Available: https://doi.org/10.5281/zenodo.3509134

[7] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, Stéfan van der Walt and Jarrod Millman, Eds., 2010, pp. 56–61. doi: https://doi.org/10.25080/Majora-92bf1922-00a.

[8] A. D. Marsh *et al.*, "Effective coverage measurement in maternal, newborn, child, and adolescent health and nutrition: progress, future prospects, and implications for quality health systems," *The Lancet Global Health*, vol. 8, no. 5, pp. e730–e736, 2020, doi: 10.1016/S2214-109X(20)30104-2.

[9] Departamento de Estadísticas e Información de Salud, "Norma Técnica 820 Estándares de Información de Salud." [Online]. Available: https://repositoriodeis.minsal.cl/ContenidoSitioWeb2020/EstandaresNormativa/Decreto-Exento-643-Sustituye-Norma-T%C3%A9cnica-sobre-Est%C3%A1ndares-de-Informaci%C3%B3n-de-Salud-Actualizada-a-Dic-2016.pdf

[10] Departamento de Estadisticas e Información de Salud, "Norma Técnica 820 Estándares de información de Salud." [Online]. Available: https://repositoriodeis.minsal.cl/Temporales/Norma%20T%C3%A9cnica%20820%20Est%C3%A1ndares%20de%20informaci%C3%B3n%20de%20Salud%20Feb2023.pdf

[11] E. Chatignoux, Z. Uhry, P. Grosclaude, M. Colonna, and L. Remontet, "How to produce sound predictions of incidence at a district level using either health care or mortality data in the absence of a national registry: the example of cancer in France," *International Journal of Epidemiology*, vol. 50, no. 1, pp. 279–292, 2021, doi: 10.1093/ije/dyaa217.

[12] S. Wood, "mgcv: Mixed GAM Computation Vehicle with Automatic Smoothness Estimation." [Online]. Available: https://cran.r-project.org/package=mgcv

[13] L. Gautier, "rpy2." 2023.

[14] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proceedings of the 7th Python in Science Conference*, G. Varoquaux, T. Vaught, and J. Millman, Eds., Pasadena, CA USA, 2008, pp. 11–15.

[15] K. Jordahl *et al.*, "geopandas/geopandas: v0.8.1." [Online]. Available: https://doi.org/10.5281/zenodo.3946761

[16] M. Khorasani, M. Abdou, and J. Hernández Fernández, *Web Application Development with Streamlit: Develop and Deploy Secure and Scalable Web Applications to the Cloud Using a Pure Python Framework*. Berkeley, CA: Apress, 2022. doi: 10.1007/978-1-4842-8111-6.

[17] R. Lagos and C. Cuadrado, "Cómo mejorar las estimaciones de cáncer gástrico en Chile." [Online]. Available: https://congresospye.uv.cl/files/oral/dia2pm/CIAE_403/1601.pptx

[18] R. Lagos, C. Cuadrado, and A. Gómez, "Caracterización de la demanda y las redes de atención de especialidades digestivas en Chile." [Online]. Available: https://congresospye.uv.cl/files/e-poster/dia1/CIAE_402/1833.pdf

[19] R. Lagos, "Effective Coverage of Stomach Cancer Prevention in Chile 2009-2024." [Online]. Available: https://www.ispor.org/heor-resources/presentations-database/presentation-cti/ispor-2026/poster-session-4-3/effective-coverage-of-stomach-cancer-prevention-in-chile-2009-2024

Open Access | https://doi.org/10.25080/avte6646

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

25 Years of Interactive Scientific Computing: From IPython and Jupyter to Open Source IDE-Native Notebooks

Rodrigo Silva Ferreira[1]

[1]Independent Researcher, Pittsburgh, PA, USA

Abstract

Interactive computing has reshaped how scientists explore data, develop methods, and communicate results. This paper traces twenty-five years of that transformation, from the 2001 release of IPython through the formation of Project Jupyter to the contemporary movement that embeds notebooks inside full software-development environments. We argue that this history is best understood not as the story of a file format or a user interface, but as the steady reduction of friction between four activities that were once pursued with separate tools: exploratory analysis, software engineering, collaboration, and reproducible publication. Read this way, each generation inherited the strengths of the previous one while absorbing a capability that had previously lived elsewhere. We further argue that openness (*e.g.,* of source code, of standards such as the Jupyter messaging protocol and the nbformat document schema, of community governance, and of the executable publication itself) has been the decisive enabler at every stage. After surveying the pre-Jupyter landscape and the scientific Python ecosystem that grew alongside these tools, we examine the well-documented limitations of traditional notebooks, the engineering practices that IDE-native notebooks restore, and the opportunities and risks introduced by large language models. The paper is itself an executable Jupyter notebook: every figure and quantitative claim is produced by the code it contains.

Keywords interactive computing, Jupyter, IPython, reproducible research, scientific Python, literate programming, open science, notebooks

1. Introduction

Computation has become a third mode of scientific inquiry, standing beside theory and experiment rather than merely serving them. Climate projections, genome assemblies, gravitational-wave detections, and the calibration of language models are all, at bottom, programs run over data. As the centre of gravity of research has shifted toward software, the tools used to write that software have stopped being incidental laboratory equipment and have become part of the scientific method itself. How a result is computed (and whether that computation can be inspected, rerun, and trusted by others) is now inseparable from whether the result is believed.

This shift has placed *reproducibility* at the heart of methodological debate. D. L. Donoho [1] framed reproducible computational research as the discipline of publishing, alongside each figure and table, the complete software environment that generated it, reviving a tradition that J. F. Claerbout and M. Karrenbach [2] and J. B. Buckheit and D. L. Donoho [3] had pioneered in geophysics and signal processing. A decade later, an interdisciplinary consensus statement in *Science* called for code and data to be treated as first-class research artifacts [4], and community guides distilled the accumulated experience into actionable

Published Jul 13, 2026

Correspondence to
Rodrigo Silva Ferreira
rodrigosf672@gmail.com

Open Access

Copyright © 2026 Ferreira. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

practice [5], [6]. The recurring lesson is that scientific software is rarely written the way production software is. It grows through *exploratory programming*: a tight loop of hypothesis, quick implementation, visualization, and revision, in which the program and the scientist's understanding co-evolve.

Interactivity changed research because it changed the unit of iteration. When a computation must be edited, recompiled, and resubmitted as a batch job, the cost of a single question is measured in minutes or hours, and the scientist learns to ask few, large questions. When the same computation answers in the time it takes to press a key, the cost of a question falls toward zero, and a different epistemic style becomes possible: a hypothesis is posed, tested against data, visualized, and revised dozens of times in an afternoon. The interactive loop does not merely speed up analysis; it widens the space of analyses a researcher will attempt, surfacing patterns and mistakes that a batch workflow would never have prompted anyone to look for. This is why cheap interactivity, far from being a mere convenience, reorganized the practice of computational science around exploration, and why the tools that host that exploration came to matter so much.

Exploration and communication, however, pull in opposite directions. Exploration rewards speed, mutable state, and disposable code; communication rewards clarity, structure, and permanence. D. E. Knuth [7]'s vision of *literate programming* (*i.e.,* a program written as a human narrative, with source code woven into prose) was an early attempt to reconcile the two, prioritizing explanation to people over instruction to machines. Interactive computing environments inherited this tension and, over twenty-five years, progressively dissolved it. The thesis of this paper is that the history of interactive scientific computing is the history of *reducing friction* among four activities (*i.e.,* exploration, engineering, collaboration, and reproducible publication) and that *openness* in its several forms has been the mechanism by which that friction was removed. Figure 1 previews the milestones along the way; the sections that follow trace them in turn.

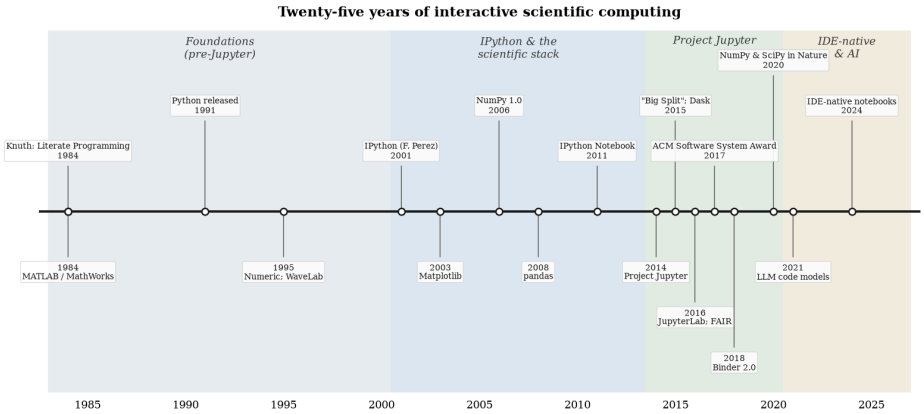


Figure 1. *A timeline of interactive scientific computing, organized into four overlapping eras. Each era inherits the capabilities of the previous one while integrating a concern (e.g., rich output, language independence, reproducible environments, software engineering) that had previously required separate tools. Dates are drawn from primary sources cited throughout the paper.*

2. Before Jupyter: REPLs, computer algebra, and literate programming

Interactivity in computing is older than the personal computer. Its lineage runs back to Lisp, whose read–eval–print loop (REPL) let a programmer type an expression and see its value immediately; McCarthy's language of 1958–1960 and the interactive Lisp systems built on it established a style of conversation with a running process that every later environment

would imitate. For decades, however, the dominant mode of scientific computing was the opposite of conversational. Programs were compiled, submitted as batch jobs, and inspected only after they finished (a workflow well matched to expensive, shared mainframes but hostile to the rapid, speculative iteration that data analysis demands).

The first tools to make scientific computing genuinely interactive were, tellingly, commercial. MATLAB began in the late 1970s when Cleve Moler wrote a Fortran "matrix laboratory" so that students could call the LINPACK and EISPACK linear-algebra libraries without writing Fortran themselves; he used it during a 1979 sabbatical at Stanford, and in 1984 he and Jack Little founded MathWorks to commercialize it [8]. Computer-algebra systems followed a parallel path: Maple was conceived at the University of Waterloo in late 1980 as a lower-cost alternative to running Macsyma, and Mathematica was launched by Wolfram Research on 23 June 1988, introducing a *notebook* interface in which cells of evaluatable input and formatted output were interleaved on a single page. The notebook metaphor that Jupyter would later make ubiquitous was thus already two decades old. Statistical computing, meanwhile, was transformed by the S language developed at Bell Labs by Becker, Chambers, and colleagues [9], whose open-source reimplementation as R by R. Ihaka and R. Gentleman [10] would become the lingua franca of statistics.

These systems were powerful, but each was, in a precise sense, *closed*. Most were proprietary; each was wedded to a single language and a single vendor; and the documents they produced (where documents existed at all) were rarely separable from the application that created them. In parallel, a quieter intellectual movement was preparing the ground for something more open. Knuth [7], [11] argued in *literate programming* that a program should be written for human readers, as an essay whose narrative order need not match the compiler's, with code extracted ("tangled") and documentation typeset ("woven") from one source; his WEB system, built to document the TeX typesetting program, was the proof of concept. At nearly the same moment, J. F. Claerbout and M. Karrenbach [2] coined the term *reproducible research* for the practice of bundling, with a publication, the electronic documents and code needed to regenerate every figure, and J. B. Buckheit and D. L. Donoho [3] popularized the idea in the WaveLab toolbox, distributing the exact code that produced each plot in their wavelet papers. Literate programming supplied the *form* (*i.e.*, narrative and code as one document) and reproducible research supplied the *purpose*. Interactive scientific computing in the Python era would inherit both, and add what the earlier systems lacked: an open-source implementation, an open document standard, and a community free to extend them.

3. IPython: an open, interactive shell for scientific Python

In 2001, Fernando Pérez was a graduate student in physics at the University of Colorado, Boulder, who wanted for Python the conveniences he admired in Mathematica and Maple: numbered input and output prompts, easy access to previous results, on-the-spot help and introspection, and a configurable, comfortable interactive session. What he wrote, as he later described it, was a 259-line "thesis procrastination project" (*i.e.*, a script loaded at startup) that gave the standard Python prompt a memory and a personality. Released as IPython 0.0.1 in December 2001 [12], it soon merged ideas and code from two kindred efforts, Janko Hauser's IPP and Nathaniel Gray's LazyPython, combining a well-organized architecture with terse interactive syntax and rich, colored tracebacks (IPython development team; [13]).

The design choices that followed proved durable. IPython introduced *magic commands* (e.g., line magics such as `%timeit` and `%matplotlib`, and cell magics prefixed with `%%`) that extend the language of the session without polluting the Python namespace. It made *introspection* immediate: appending `?` to any object prints its documentation and signature, and `??` shows

its source. It cached inputs and outputs so that prior results remained addressable, and it offered tab completion and a configuration system that let users shape their environment. Crucially, and unlike the commercial systems that inspired it, IPython was open source from its first day, developed in the open by a widening circle of contributors and eventually sustained by foundation grants (*e.g.*, an Alfred P. Sloan Foundation award in 2013, followed in 2015 by roughly six million dollars from the Helmsley, Moore, and Sloan foundations to expand the project) [13], [14]. The small convenience of a better prompt had become community infrastructure.

The demonstration below uses one of those magics. The kernel executing this very notebook is IPython, so `%timeit` is available to us as we read; it quantifies the single most important habit in scientific Python (*i.e.*, replacing explicit Python loops with vectorized array operations), a point we return to in The scientific Python ecosystem, accelerated.

```
37.9 ms ± 1.26 ms per loop (mean ± std. dev. of 3 runs, 3 loops each)
124 µs ± 14 µs per loop (mean ± std. dev. of 5 runs, 50 loops each)

Vectorization speed-up on this machine: 350x
```

The two lines compute the same thing (*i.e.*, the element-wise square of a million numbers) but the vectorized form runs hundreds of times faster. Being able to *measure* that gap in a single keystroke, mid-thought, is the essence of what IPython added: not a new language, but a new tightness in the loop between writing code and understanding its behaviour.

4. Project Jupyter: from a shell to an open protocol

IPython's most consequential idea was architectural rather than cosmetic. To support a graphical Qt console (2011) and then a browser-based notebook (first shipped in IPython 0.12 in December 2011, drawing heavy inspiration from William Stein's pioneering Sage Notebook), the project separated the *frontend* that a user interacts with from the *kernel* that executes code, with the two communicating over a well-specified network protocol. Once execution lived behind a protocol rather than inside the shell, the language of the kernel became an implementation detail. A frontend that could talk to a Python kernel could talk to any kernel.

This realization motivated "the Big Split." IPython 3.0, released in February 2015, was the last release to bundle the language-agnostic notebook with the Python-specific shell; IPython 4.0, on 11 August 2015, moved the frontend, the notebook document format, and the protocol into a new, language-neutral umbrella: Project Jupyter, whose name honours its three founding languages (*i.e.*, Julia, Python, and R [10]) and alludes to Galileo's notebooks recording the moons of Jupiter. This transition, formalized in a defining 2016 paper authored by core architects Thomas Kluyver, Min Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Damian Avila, Kyle Kelley, Carol Willing, Jason Grout, Sylvain Corlay, and others [15], was not a rebranding but a deliberate act of *opening*: it declared that the value of the notebook lay in a set of open standards that no single language community owned.

Figure 2 sketches the resulting architecture. A frontend and a kernel communicate over a ZeroMQ-based messaging protocol [16] carried on five sockets (*i.e.*, Shell, IOPub, stdin, Control, and Heartbeat), each with a defined role (Jupyter development team, *Messaging*). The Shell channel carries `execute_request` messages and their replies; the IOPub channel broadcasts side effects (*e.g.*, standard output, rich `display_data`, and `execute_result`) to every connected client; stdin handles input prompts; Control carries high-priority interrupt and shutdown messages; and Heartbeat confirms liveness. Every message is signed with an

HMAC digest and serialized, by default, as JSON, so that any process implementing the protocol can serve as a kernel.

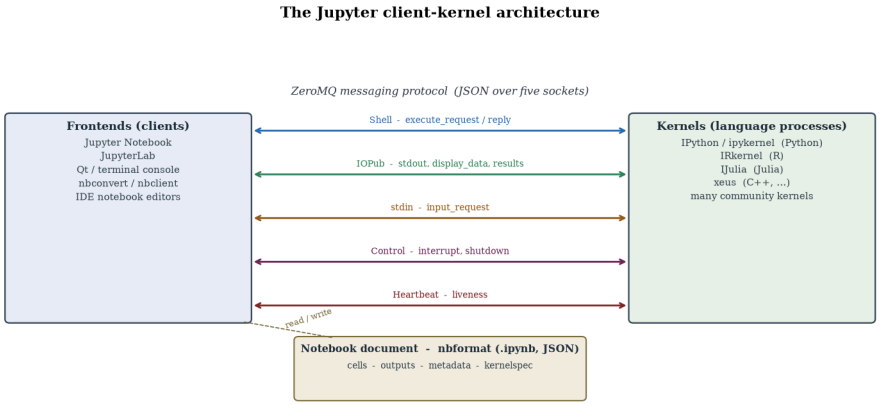


Figure 2. *The Jupyter client–kernel architecture. Decoupling the user-facing frontend from the language kernel behind an open, signed messaging protocol is what makes Jupyter language-agnostic: a* kernelspec *registers any conforming interpreter, and dozens of community kernels now exist. The notebook document (`nbformat`) is an independent JSON artifact that frontends read and write.*

The flow of a single execution is worth making concrete, because it explains both the power and the hazards of notebooks discussed later. Figure 3 shows the message exchange when a user runs one cell.

```
sequenceDiagram
    autonumber
    participant F as Frontend
    participant S as Shell channel
    participant K as Kernel
    participant IO as IOPub channel
    F->>S: execute_request (source, msg_id)
    S->>K: deliver request
    K-->>IO: status = busy
    K-->>IO: execute_input (echo source, count = n)
    K-->>IO: stream / display_data / execute_result
    K-->>S: execute_reply (status = ok, count = n)
    K-->>IO: status = idle
```

Figure 3. *The messaging sequence for executing one cell. The kernel maintains an execution counter `n` and a persistent namespace; the frontend records each cell's outputs and that counter in the notebook document. Because the counter reflects the* order of execution*, not the order of cells on the page, a saved notebook can encode a history that no top-to-bottom run would reproduce (*i.e.*, the seed of the reproducibility problems examined in The challenges of traditional notebooks).*

Two open standards complete the picture. The *kernelspec* mechanism (*i.e.*, a small `kernel.json` file in a known location) lets any language that speaks the protocol be installed as a kernel, which is why Jupyter today drives Python, R, Julia, C++, and many other languages from one interface. The `nbformat` schema defines the notebook itself as a JSON document with top-level `cells`, `metadata`, and version fields, where each cell is `code`, `markdown`, or `raw` and code cells carry their outputs inline [15], [17]. Because the format is open and text-based, notebooks can be generated, transformed, executed, and rendered by an entire ecosystem of tools rather than a single application. On this foundation grew JupyterLab (unveiled in alpha at SciPy 2016 and released as 1.0 in June 2019) and JupyterHub for multi-user deployments, work recognized by the 2017 ACM Software System Award, presented in 2018 [18]. The recurring pattern is openness compounding: an open protocol enabled an open document format, which enabled an open ecosystem.

5. The scientific Python ecosystem, accelerated

Jupyter did not create the scientific Python ecosystem, but it gave that ecosystem an interactive interface of unprecedented reach, and the two grew together. The stack is layered (Figure 4). At its base sits CPython with compiled extensions; above that, NumPy provides the n-dimensional array, its universal functions, and the broadcasting rules that let array expressions stand in for explicit loops [19]. NumPy traces its lineage to Numeric (Jim Hugunin, 1995) and numarray, which Travis Oliphant, Charles R. Harris, Nathaniel J. Smith, and others unified and grew into NumPy, released as 1.0 in 2006. On the array core rest the libraries most analyses touch directly: SciPy for numerical algorithms (driven by Pauli Virtanen, Ralf Gommers, Stefan van der Walt, and a massive community) [20], pandas for labeled, heterogeneous, tabular and time-series data [21], Matplotlib for publication-quality graphics [22], and SymPy for symbolic mathematics [23]. Domain libraries (*e.g.*, scikit-learn for machine learning [24], xarray for labeled N-dimensional data [25], and others) build on these, while Dask extends the same array and dataframe interfaces to larger-than-memory and parallel computation [26].

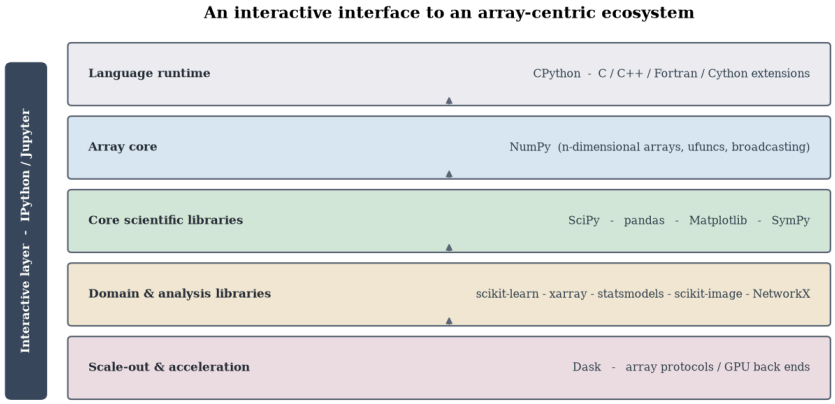


Figure 4. *The array-centric scientific Python stack. A shared array substrate (NumPy) lets independently developed libraries interoperate, and a common interactive layer (IPython/Jupyter) lets a researcher move fluidly among them. The ecosystem's coherence is a direct dividend of open, shared interfaces.*

The remainder of this section is a short tour of that stack in action, with each computation chosen to make a point rather than to decorate the page. We begin with the idiom that defines numerical Python, vectorization (already previewed in IPython: an open, interactive shell for scientific Python, now measured across problem sizes).

```
import timeit
import numpy as np
import matplotlib.pyplot as plt

sizes = [10_000, 100_000, 1_000_000]
loop_t, vec_t = [], []
for n in sizes:
    a = rng.standard_normal(n)
    loop_t.append(timeit.timeit(lambda: [v * v for v in a], number=3) / 3)
    vec_t.append(timeit.timeit(lambda: a * a, number=50) / 50)

speedup = [l / v for l, v in zip(loop_t, vec_t)]
fig, ax = plt.subplots(figsize=(6.6, 4.0))
idx = np.arange(len(sizes))
ax.bar(idx - 0.19, loop_t, 0.36, color=TOL[2], label="Python loop")
ax.bar(idx + 0.19, vec_t, 0.36, color=TOL[0], label="NumPy vectorized")
ax.set_yscale("log"); ax.set_xticks(idx); ax.set_xticklabels([f"{n:,}" for n in sizes])
ax.set_xlabel("array length"); ax.set_ylabel("seconds per call (log scale)"); ax.legend()
ax.set_title(f"Vectorization: up to {max(speedup):.0f}x faster on this machine")
_save(fig, "demo_numpy")
print("speed-ups by size:", [f"{s:.0f}x" for s in speedup])
```

```
speed-ups by size: ['58x', '272x', '254x']
```

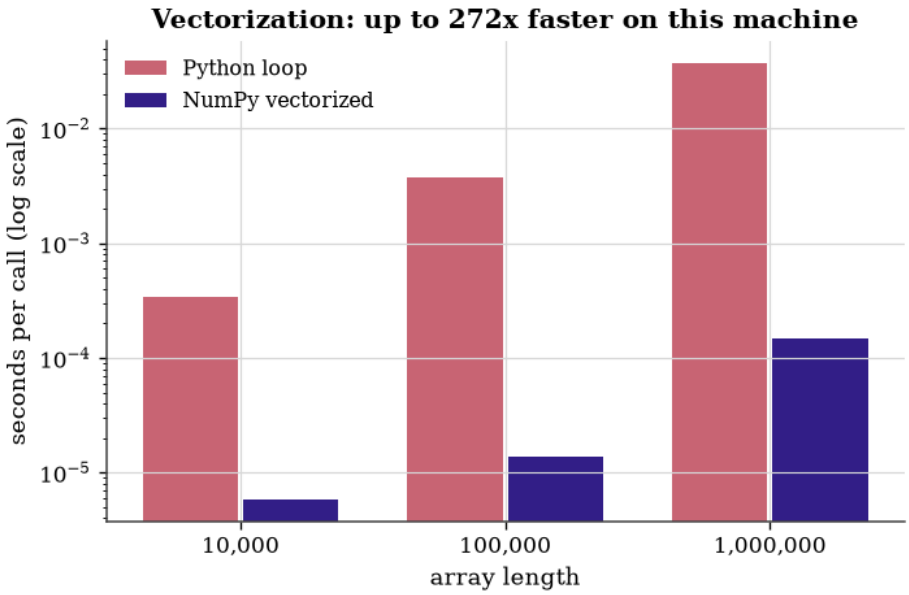
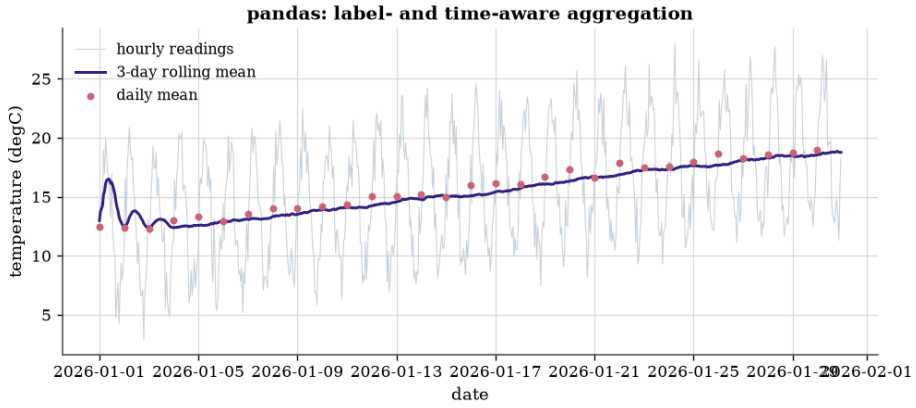


Figure 5. *Element-wise squaring via an explicit Python loop versus a NumPy array expression, timed across three problem sizes (note the logarithmic axis). The vectorized version is one to two orders of magnitude faster at every size tested, though the margin fluctuates with system load rather than widening monotonically; pushing the loop into compiled, vectorized code is the foundation on which every higher library is built.*

Real data is rarely an anonymous array; it carries labels, units, and time. pandas exists to keep those labels attached through every operation. The next cell synthesizes an hourly temperature record with a diurnal cycle and noise, then uses a `DatetimeIndex` to resample to daily means and to compute a rolling trend (operations that would be error-prone with bare arrays but are one line each with labeled data).

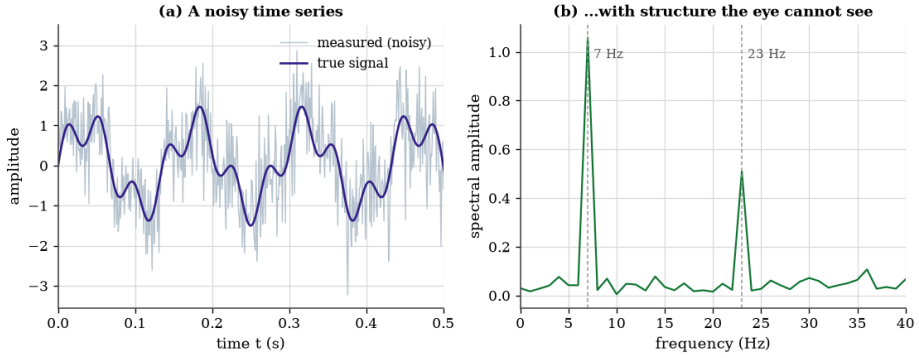
```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt

hours = pd.date_range("2026-01-01", periods=24 * 30, freq="h")
t = np.arange(len(hours))
signal = (12 + 6 * np.sin(2 * np.pi * t / 24)           # diurnal cycle
          + 0.01 * t                                     # slow warming trend
          + rng.normal(0, 1.5, len(hours)))              # measurement noise
ts = pd.Series(signal, index=hours, name="temperature_C")

daily = ts.resample("D").mean()                          # label-aware aggregation
trend = ts.rolling("3D").mean()                          # 3-day rolling mean

fig, ax = plt.subplots(figsize=(9.5, 3.8))
ax.plot(ts.index, ts.values, color="#c7d2dc", lw=0.6, label="hourly readings")
ax.plot(trend.index, trend.values, color=TOL[0], lw=1.8, label="3-day rolling mean")
ax.scatter(daily.index, daily.values, color=TOL[2], s=14, zorder=5, label="daily mean")
ax.set_xlabel("date"); ax.set_ylabel("temperature (degC)"); ax.legend(loc="upper left")
ax.set_title("pandas: label- and time-aware aggregation")
_save(fig, "demo_pandas")
print(f"hourly points: {len(ts)} | daily means: {len(daily)} | "
      f"overall mean: {ts.mean():.2f} degC")
```

```
hourly points: 720 | daily means: 30 | overall mean: 15.66 degC
```



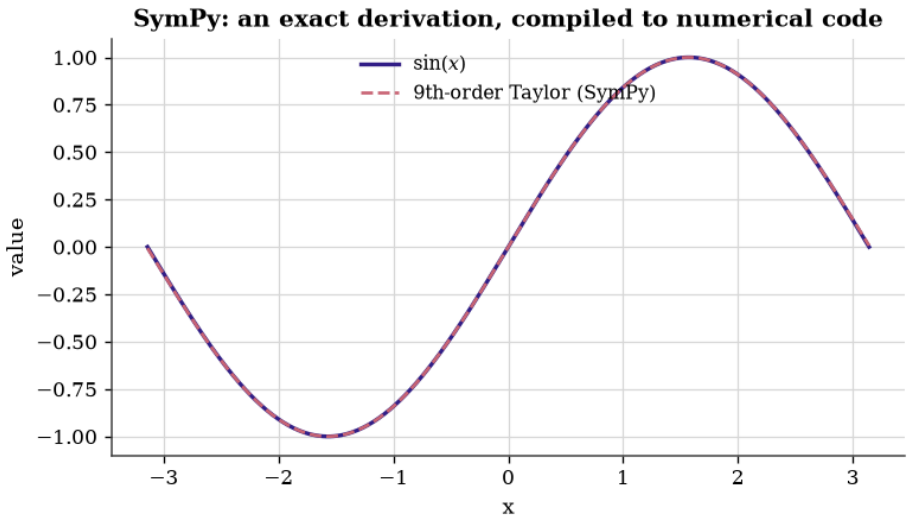
```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 1.0, 1000, endpoint=False)
clean = np.sin(2 * np.pi * 7 * t) + 0.5 * np.sin(2 * np.pi * 23 * t)
noisy = clean + 0.8 * rng.standard_normal(t.size)

freqs = np.fft.rfftfreq(t.size, d=t[1] - t[0])
amp = np.abs(np.fft.rfft(noisy)) * 2 / t.size

fig, ax = plt.subplots(1, 2, figsize=(11, 3.7))
ax[0].plot(t, noisy, color="#b8c4d0", lw=0.8, label="measured (noisy)")
ax[0].plot(t, clean, color=TOL[0], lw=1.6, label="true signal")
ax[0].set(xlabel="time t (s)", ylabel="amplitude", xlim=(0, 0.5))
ax[0].legend(loc="upper right"); ax[0].set_title("(a) A noisy time series", fontsize=11)
ax[1].plot(freqs, amp, color=TOL[1], lw=1.3)
for f0 in (7, 23):
    ax[1].axvline(f0, color="#999", ls=(0, (3, 2)), lw=0.9)
    ax[1].text(f0 + 0.6, amp.max() * 0.92, f"{f0} Hz", fontsize=9, color="#555")
ax[1].set(xlabel="frequency (Hz)", ylabel="spectral amplitude", xlim=(0, 40))
ax[1].set_title("(b) ...with structure the eye cannot see", fontsize=11)
_save(fig, "demo_scipy")
peaks = np.sort(freqs[np.argsort(amp)[-2:]])
print(f"recovered dominant frequencies: {peaks[0]:.1f} Hz and {peaks[1]:.1f} Hz")
```

```
recovered dominant frequencies: 7.0 Hz and 23.0 Hz
```



```
Taylor series of sin(x) to 9th order:
```

```
x**9/362880 - x**7/5040 + x**5/120 - x**3/6 + x
```




```python
import numpy as np, xarray as xr
import dask.array as da

# xarray: an array that knows what its axes mean
temps = xr.DataArray(
    rng.normal(15, 5, size=(3, 4)),
    dims=("city", "month"),
    coords={"city": ["Oslo", "Nairobi", "Lima"],
            "month": ["Mar", "Apr", "May", "Jun"]},
    name="temperature_C",
)
print("Per-city mean temperature (named-dimension reduction):")
print(temps.mean(dim="month").to_series().round(2).to_string())

# Dask: the NumPy interface, evaluated lazily and in chunks
big = da.random.random((20_000, 20_000), chunks=(2_000, 2_000))   # ~3 GB if realized
lazy_mean = big.mean()                        # nothing computed yet -> a task graph
print(f"\nDask array: {big.shape} in {big.npartitions} chunks; "
      f"graph builds instantly, mean computes to {float(lazy_mean.compute()):.4f}")
```

```
Per-city mean temperature (named-dimension reduction):
city
Oslo       17.19
Nairobi    14.04
Lima       15.68

Dask array: (20000, 20000) in 100 chunks; graph builds instantly, mean computes to 0.5000
```

Taken together, these libraries share two properties that recur throughout this paper: they are open source, and they agree on interfaces. The array protocol lets independently authored packages compose; the estimator and labeled-array conventions let a researcher carry intuition from one library to the next. Jupyter's contribution was to place a single, interactive, narrative surface over all of them. The interactivity is not merely a convenience for the author. Because a notebook can embed live controls, a reader can manipulate a computation directly, turning a static figure into an instrument. The widget below lets one

vary a sample size and a distribution and watch an empirical histogram converge toward its theoretical density.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from ipywidgets import interact, IntSlider, Dropdown

DISTS = {"normal": stats.norm(0, 1),
         "exponential": stats.expon(scale=1.0),
         "uniform": stats.uniform(-2, 4)}

def explore(n=400, dist="normal"):
    d = DISTS[dist]
    samples = d.rvs(size=n, random_state=np.random.default_rng(SEED))
    fig, ax = plt.subplots(figsize=(7.2, 3.8))
    ax.hist(samples, bins=40, density=True, color="#cdd9e3",
            edgecolor="white", label=f"{n} samples")
    grid = np.linspace(samples.min(), samples.max(), 300)
    ax.plot(grid, d.pdf(grid), color=TOL[2], lw=2.2, label="true density")
    ax.set_title(f"Empirical vs. theoretical density  (n = {n}, {dist})")
    ax.set_xlabel("value"); ax.set_ylabel("density"); ax.legend(loc="upper right")
    plt.show()

interact(explore,
         n=IntSlider(min=20, max=5000, step=20, value=400, description="samples"),
         dist=Dropdown(options=list(DISTS), value="normal", description="distribution"));
```

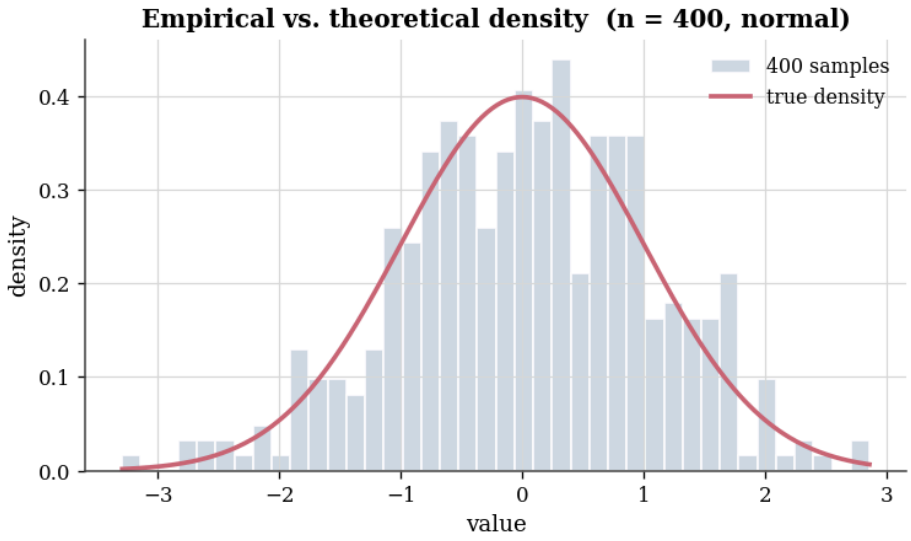


Figure 6. *A live control over sample size and distribution. In a running kernel the sliders update the figure continuously; in a static rendering, the initial state is shown. Interactivity of this kind turns a reader into an experimenter, which is precisely why notebooks became the medium of choice for teaching, exploration, and (increasingly) publication [14].*

6. Open Science and Reproducibility

If interactivity made notebooks the natural home for exploration, openness made them a candidate vehicle for reproducible publication. The two agendas converged in the mid-2010s. The FAIR Guiding Principles [27] argued that scholarly outputs should be Findable, Accessible, Interoperable, and Reusable (principles later extended explicitly to research software [28]). Practical guidance accumulated in parallel, drawing heavily on notebook pedagogy championed by educators (including Jessica Hamrick, Lorena A. Barba, Amy J. Ko, and Allen Downey) [29], [30]: the “ten simple rules” for reproducible computa-

tional research [31] and the “best practices” and “good enough practices” series [5], [6] (championed by open science leaders like Greg Wilson, Neil Chue Hong, Titus Brown, Daniel S. Katz, and Karthik Ram) translated the aspiration into habits (*e.g.*, version everything, automate the pipeline, record the environment).

Recording the environment proved to be the crux. A notebook captures code and narrative, but its results depend on a web of library versions and system libraries that the document itself does not pin. Two technologies addressed this gap. Containerization, popularized by Docker [32], let an entire software environment be specified declaratively and rebuilt identically elsewhere. Binder, pioneered by community leaders such as Yuvi Panda, Chris Holdgraf, and Tim Head [33], connected that capability to notebooks: given only a public Git repository with its dependencies declared, the `repo2docker` tool builds a container image and JupyterHub serves a live, executable session in the browser, so that a reader can run a paper’s notebooks without installing anything. For the first time, “click here to reproduce my analysis” became a realistic offer rather than a rhetorical one.

The honest assessment is that these tools made reproducibility *achievable* but not *automatic*, and each has limits worth stating plainly. Public Binder sessions are ephemeral and resource-limited, unsuited to long or data-heavy computations. Container images pin software but not always the data, the hardware, or sources of nondeterminism such as random seeds, thread counts, and floating-point reduction order. Dependency specifications drift: an environment that omits exact versions may resolve differently months later, and upstream packages disappear. Reproducibility, in short, is a property a researcher must actively engineer (*e.g.*, by pinning versions, fixing seeds (as this notebook does in its setup cell), separating data acquisition from analysis, and testing that the pipeline runs end to end). Openness lowers the cost of doing so dramatically, because open formats and open infrastructure let the community build the supporting tools; but it does not remove the responsibility. The next section quantifies how often, in practice, that responsibility goes unmet.

7. The challenges of traditional notebooks

The very features that make notebooks excellent for exploration create difficulties when the same documents are asked to serve as durable, shareable software. The root cause is *hidden state*. A kernel holds a persistent namespace, and cells may be executed in any order and any number of times. The visible document records the cells and their last outputs, but not the sequence of executions that produced them. A notebook can therefore look correct on screen while depending on a variable defined in a cell that has since been deleted, or on a definition that no longer exists in the order shown (a discrepancy invisible until someone restarts the kernel and runs from the top).

The challenges of hidden state have drawn pointed criticism (most famously in Joel Grus’s 2018 JupyterCon talk, ‘I Don’t Like Notebooks’ [34]) and the empirical literature documents how widespread the consequences are. In the largest study of its kind, J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire [35] collected about 1.4 million Jupyter notebooks from roughly 264,000 GitHub repositories and attempted to execute the 863,878 that were valid Python with an unambiguous cell order. Only 24.1% ran to completion without raising an error, and only 4.0% reproduced the outputs stored in the document; failures were dominated by unmet dependencies (`ImportError`/`ModuleNotFoundError` accounted for about 29% of them). Symptoms of hidden state were pervasive: 36.4% of notebooks had been executed out of order, and 76.9% had at least one gap in their execution counters, consistent with cells that were re-run out of order or removed after execution. A follow-up that re-ran the analysis on a deduplicated corpus reached broadly similar conclusions [36]. Figure 7 summarizes the funnel.

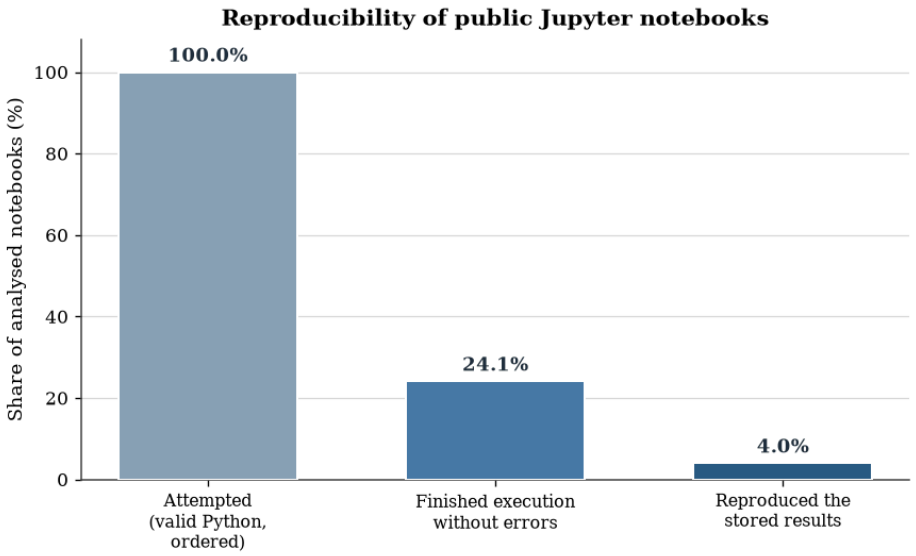


Figure 7. *The reproducibility funnel from J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire [35], as a share of the 863,878 valid notebooks they attempted to execute. The collapse from "runs without error" to "reproduces the stored results" measures the gap between a notebook that looks finished and one that actually is.*

The problems extend beyond execution. Because `.ipynb` files are JSON documents that embed outputs (including rendered images as base64 text), they interact badly with line-oriented version control: diffs are noisy and merges conflict over machine-generated metadata, discouraging the branching-and-review workflow that underpins collaborative software. Testing and debugging are awkward when the unit of code is a cell rather than an importable function, and a long notebook tends to accumulate globals and implicit ordering that resist refactoring as the analysis grows. Qualitative studies trace these frictions to a deeper tension. A. Rule, A. Tabard, and J. D. Hollan [37], analyzing over a million notebooks alongside interviews, found a persistent conflict between *exploration* and *explanation*: notebooks accrete exploratory mess and are seldom cleaned up for sharing. Interview and survey studies catalogue the resulting pain points across the whole workflow (*e.g.,* setup, exploration, refactoring, deployment, and collaboration) [38], and characterize the informal versioning and narrative practices analysts improvise in their absence [39]. A line of tools has tried to help from within: "code gathering" to extract the minimal cells behind a result [40], and methods to restore a reproducible execution order automatically [41], [42]. These are valuable, but they treat symptoms. The structural response (*i.e.,* bringing the notebook the engineering machinery that scripts and IDEs have long enjoyed) is the subject of the next section.

8. Open source IDE-native notebooks

The most important recent development in interactive scientific computing is a change in where notebooks live. This convergence of the exploratory notebook and the engineering workbench first took root with JupyterLab (architected by project leaders like Afshin Darian, Steven Silvester, and Paul Ivanov), which brought terminals, text editors, and notebooks into a single extensible interface. Today, the notebook is increasingly a first-class citizen inside a full software-development environment, sitting beside source files, a debugger, a test runner, a version-control panel, and a terminal, and benefiting from the same language intelligence as ordinary code. This is less a product category than a *movement* (a convergence of the exploratory notebook and the engineering workbench) and, like the developments before it, it was made possible by open standards.

Two protocols are central. The Language Server Protocol (LSP), opened by Microsoft in 2016 in collaboration with Red Hat and Codenvy, is a JSON-RPC standard by which an editor obtains language intelligence (*e.g.,* completion, hover documentation, go-to-definition, diagnostics, and refactorings) from a separate *language server* [43]. Its significance is combinatorial: instead of every one of M editors implementing support for every one of N languages, each editor and each language implement the protocol once, turning an $M \times N$ problem into an $M+N$ one. The Debug Adapter Protocol (DAP) does the same for debuggers [44]. Jupyter adopted both ideas. The Jupyter debugger protocol (JEP 47) [45] wraps DAP messages inside Jupyter messages (*e.g.,* `debug_request` and `debug_reply` on the Control channel, `debug_event` on IOPub) so that a frontend can drive breakpoints and variable inspection in a running kernel; the kernel advertises support in its `kernel_info_reply`, and implementations arrived through `debugpy` in `ipykernel` and through `xeus`-based kernels [45]. Because these protocols are open, a notebook hosted in any conforming environment can offer the same completion, refactoring, linting, and step-through debugging that script authors take for granted.

Open standards also reconciled notebooks with version control. Jupytext [46] represents a notebook as a plain-text script or Markdown file and keeps the two paired, so that the text form (free of outputs and volatile metadata) produces clean line diffs and reviewable pull requests, while the `.ipynb` retains rich outputs for reading. nbdime supplies content-aware diff and merge that understand notebook structure rather than raw JSON [47]. For execution and testing, papermill parameterizes and runs notebooks as pipeline stages [48]; nbval [49] turns a notebook into a regression test by comparing recomputed outputs against saved ones, and testbook unit-tests functions defined in notebooks directly [50]; and jupyter-cache re-executes only the cells that changed [51]. Perhaps the fullest expression of the convergence is nbdev [52], which treats notebooks as the source of truth for a Python package (*e.g.,* generating modules, tests, and documentation from them) and thereby realizes Knuth's literate-programming ideal four decades on [7], with the fast.ai library itself built this way [53].

Surrounding these notebook-aware tools is the ordinary furniture of a development environment, now within reach of the cell a scientist is editing. An integrated terminal makes shell commands (*e.g.,* installing a package, inspecting a data file, launching a job) part of the same session rather than a switch to another window. Package and environment management, whether through virtual environments, conda, or lockfile-based resolvers, can be driven from the same interface that runs the notebook, so that the environment a result depends on is created and recorded where the result is produced. Linters and type checkers flag mistakes as code is typed, and the plain-text representations that jupytext maintains drop directly into continuous-integration pipelines, where nbval or papermill can execute a notebook on every commit and fail the build if a figure or a number changes unexpectedly. None of these capabilities is new to software engineering; what is new is that the scientist need not leave the exploratory document to use any of them. Figure 8 places these capabilities against earlier generations.

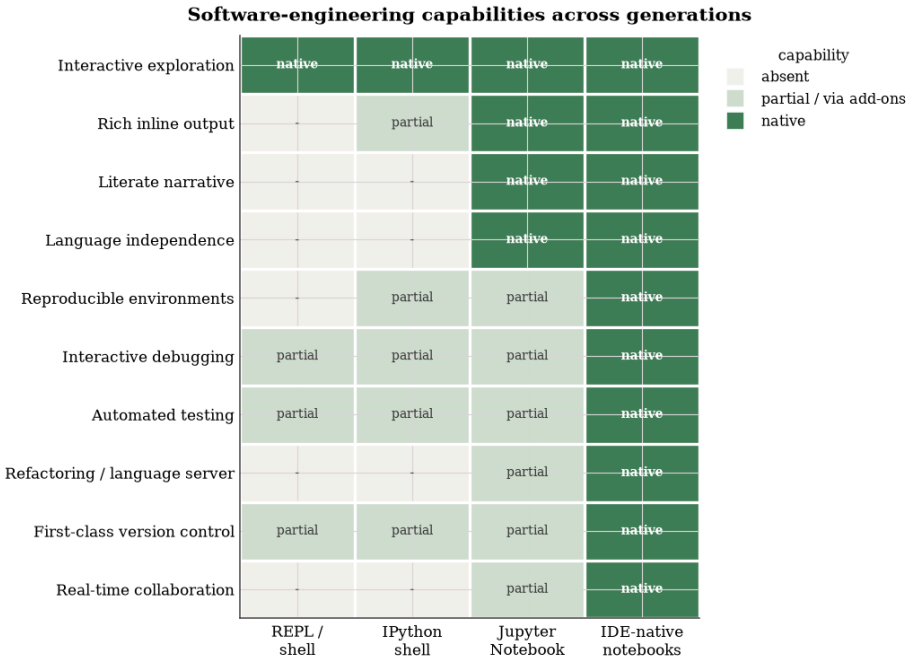


Figure 8. *A qualitative assessment by the authors of where ten software-engineering capabilities sit across four generations of interactive tools. The pattern, not any single cell, is the point: capabilities that classic notebooks could reach only through add-ons become native when the notebook is hosted inside a development environment (without sacrificing the interactive exploration of the first row).*

The decisive question is whether this engineering rigor comes at the expense of the exploratory freedom that made notebooks valuable. We argue that it need not, because the integration is *additive*. A scientist still opens a cell, types an idea, and runs it; what changes is that the same idea can, without leaving the environment, be linted as it is written, extracted into a tested function, stepped through in a debugger when it misbehaves, committed with a readable diff, and packaged for reuse. The exploratory loop is preserved; the cliff that once separated a promising notebook from maintainable software is graded into a ramp. The demonstration below enacts the smallest version of this transition. We write a function to a source module (so that notebook and script literally share the same code) then import it back and test it against an analytic solution, all without leaving the document.

```
%%writefile cooling.py
# A small module written from the notebook -- source and notebook now coexist.
import numpy as np

def newton_cooling(T0, T_env, k, t):
    """Analytic solution of Newton's law of cooling: T(t)=T_env+(T0-T_env)e^{-kt}."""
    return T_env + (T0 - T_env) * np.exp(-k * np.asarray(t, dtype=float))

def euler_step(T, T_env, k, dt):
    """One explicit-Euler step of dT/dt = -k (T - T_env)."""
    return T + dt * (-k * (T - T_env))
```

```
Writing cooling.py
```

```
import numpy as np
from cooling import newton_cooling, euler_step      # the module we just wrote

# A lightweight in-notebook test: numerical integration must approach the
# analytic solution as the step size shrinks. This is the habit that nbval,
# testbook, and CI build upon -- testable code, defined and checked in place.
def integrate(T0, T_env, k, t_end, dt):
    T, ts, Ts = T0, [0.0], [T0]
    while ts[-1] < t_end - 1e-9:
        T = euler_step(T, T_env, k, dt); ts.append(ts[-1] + dt); Ts.append(T)
    return np.array(ts), np.array(Ts)

t_end = 5.0
errors = {}
for dt in (0.5, 0.1, 0.01):
    ts, Ts = integrate(90.0, 20.0, 0.7, t_end, dt)
    errors[dt] = np.max(np.abs(Ts - newton_cooling(90.0, 20.0, 0.7, ts)))

assert errors[0.5] > errors[0.1] > errors[0.01], "error must shrink with dt"
assert errors[0.01] < 0.5, "fine step should track the analytic solution"
print("test passed -- max error by step size:",
      {dt: round(e, 3) for dt, e in errors.items()})
```

```
test passed -- max error by step size: {0.5: np.float64(5.272), 0.1: np.float64(0.929), 0.01:
np.float64(0.09)}
```

That cell is mundane on purpose. A function lives in a real `.py` module *and* in the running notebook; a test asserts a mathematical property and would fail loudly if the implementation regressed; and the whole exchange happened inside the exploratory document. Multiply this by language-server completion, a visual debugger, and a version-control panel, and the notebook stops being a place from which code must eventually escape to "become serious," and becomes a place where serious software can be written from the start.

9. Artificial Intelligence in the Notebook

The emergence of large language models represents the latest shift in interactive scientific computing. Unlike earlier advances that primarily changed interfaces or execution environments, AI assistants increasingly participate directly in the process of exploration by generating code, explaining results, suggesting analyses, and helping users navigate complex software ecosystems. Early work on neural code generation (including systems such as Codex, a GPT model fine-tuned on public source code [54]) demonstrated that models could synthesize useful programs from natural language, and subsequent systems have extended these capabilities into notebook environments where they complete lines of code, draft entire cells, answer questions about datasets, and edit notebooks through natural-language interaction.

Like earlier developments in the history of interactive computing, these systems further shorten the path from an idea to executable code. At the same time, empirical studies suggest a more nuanced picture than simple productivity gains. A controlled study of an LLM-based code completion tool found that it did not consistently reduce task completion time and that programmers often invested substantial effort in understanding and verifying generated code [55]. Observational work identified two distinct modes of interaction (*acceleration*, in which developers use the model to generate code they already intend to write, and *exploration*, in which the model helps discover unfamiliar approaches), each with different strengths and failure modes [56]. More recently, research has begun examining how AI assistants should be designed specifically for notebooks, where code, outputs, visualizations, and narrative coexist within a single computational document [57].

For scientific computing, the opportunities must be considered alongside longstanding concerns about reproducibility, transparency, and trust. As AI systems become collaborators rather than mere autocomplete tools, they complicate the notion of *provenance*. If portions of an analysis originate from prompts, the scientific record must capture what was asked, produced, and ultimately verified by the human.

Notebooks are particularly well suited to supporting this richer notion of provenance because they already interleave narrative, code, outputs, and execution history within a single computational artifact. Previous work on automatic provenance capture in notebooks [58], [59] suggests that AI generation events could likewise become part of the scientific record rather than remaining implicit. From this perspective, AI does not eliminate the need for careful scientific practice; instead, it shifts the balance of effort from writing code toward evaluating, validating, and documenting computational reasoning. As interactive scientific computing continues to evolve, openness, reproducibility, and transparent provenance become increasingly important (not despite AI, but because human accountability remains essential regardless of how code is produced).

In a world where some code is written by people and some by models, a verifiable trail of *what was computed, with what, and by whom* is the form of trust that openness uniquely enables. Human oversight remains the load-bearing element; the tooling exists to make that oversight cheap and auditable rather than to replace it.

10. Future directions

The next stage of interactive scientific computing may be defined less by faster computation than by richer interaction. Over the past twenty-five years, notebooks have gradually dissolved boundaries between exploration, documentation, publication, and software development. The next decade is likely to dissolve another boundary: the distinction between computational artifacts and interactive applications.

Notebook environments are evolving beyond static computational documents. Reactive execution models, richer user interface components, and browser-native runtimes allow notebooks to become scientific applications that users can interact with directly rather than merely execute. Instead of asking readers to modify source code, authors can increasingly expose parameters through forms, sliders, tables, and visual controls that invite exploration without requiring programming expertise. In this sense, notebooks are becoming environments not only for writing analyses but also for communicating and sharing them with broader audiences.

Whether this future benefits science will depend, once again, on openness. Interactive documents, reactive notebook systems, browser-based execution, and AI-assisted workflows all depend on open document formats, open protocols, interoperable runtimes, and transparent governance. The history traced throughout this paper repeatedly demonstrates that

the greatest advances in interactive scientific computing have emerged not from proprietary lock-in but from communities that chose openness at critical moments. Preserving that tradition offers the strongest foundation for the next generation of interactive scientific computing.

11. Conclusion

The history of interactive scientific computing is the history of reducing friction across four fundamental activities: exploratory analysis, software engineering, collaboration, and reproducible publication. As we have seen, each generation of tools inherited the strengths of its predecessors while internalizing capabilities that previously required separate, disjoint environments. The transition from REPLs to the IPython shell brought rich exploration and the foundations of a scientific ecosystem. Project Jupyter decoupled the interface from the runtime, creating a language-agnostic protocol that made the notebook a universal medium for literate narrative and reproducible environments. Most recently, the embedding of notebooks within IDEs has restored the rigorous software engineering practices (*e.g.,* debugging, testing, and version control) that early notebooks struggled to support.

Throughout this evolution, openness has been the decisive mechanism. Open source code, open community governance, and (crucially) open standards like the Jupyter messaging protocol and the `nbformat` schema [17] were the enablers that allowed these tools to scale from individual scripts to a ubiquitous scientific infrastructure. As the community continues to integrate new paradigms, such as AI-assisted code generation and reactive publishing, maintaining this commitment to transparent, reproducible, and open scientific computing remains as essential as ever.

Acknowledgements

This paper stands on the work of the IPython and Project Jupyter contributors, the maintainers of the scientific Python ecosystem, and the researchers whose empirical studies of notebooks are cited throughout. The factual claims herein were checked against primary sources during preparation; any remaining errors are the author's own.

Portions of this paper's text and code were drafted and debugged with the assistance of Claude (Anthropic). All AI-assisted output was reviewed, verified, and revised by the author, who takes full responsibility for the accuracy and integrity of the final content.

References

[1] D. L. Donoho, "An invitation to reproducible computational research," *Biostatistics*, vol. 11, no. 3, pp. 385–388, 2010, doi: 10.1093/biostatistics/kxq028.

[2] J. F. Claerbout and M. Karrenbach, "Electronic Documents Give Reproducible Research a New Meaning," in *SEG Technical Program Expanded Abstracts 1992*, 1992, pp. 601–604. doi: 10.1190/1.1822162.

[3] J. B. Buckheit and D. L. Donoho, "WaveLab and Reproducible Research," in *Wavelets and Statistics*, vol. 103, A. Antoniadis and G. Oppenheim, Eds., in Lecture Notes in Statistics, vol. 103., New York, NY: Springer, 1995, pp. 55–81. doi: 10.1007/978-1-4612-2544-7_5.

[4] V. Stodden *et al.*, "Enhancing reproducibility for computational methods," *Science*, vol. 354, no. 6317, pp. 1240–1241, 2016, doi: 10.1126/science.aah6168.

[5] G. Wilson *et al.*, "Best Practices for Scientific Computing," *PLoS Biology*, vol. 12, no. 1, p. e1001745, 2014, doi: 10.1371/journal.pbio.1001745.

[6] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, and T. K. Teal, "Good enough practices in scientific computing," *PLOS Computational Biology*, vol. 13, no. 6, p. e1005510, 2017, doi: 10.1371/journal.pcbi.1005510.

[7] D. E. Knuth, "Literate Programming," *The Computer Journal*, vol. 27, no. 2, pp. 97–111, 1984, doi: 10.1093/comjnl/27.2.97.

[8] C. Moler and J. Little, "A History of MATLAB," *Proceedings of the ACM on Programming Languages*, vol. 4, pp. 1–67, 2020, doi: 10.1145/3386331.

[9] R. A. Becker and J. M. Chambers, *S: An Interactive Environment for Data Analysis and Graphics*. Belmont, CA: Wadsworth, 1984.

[10] R. Ihaka and R. Gentleman, "R: A Language for Data Analysis and Graphics," *Journal of Computational and Graphical Statistics*, vol. 5, no. 3, pp. 299–314, 1996, doi: 10.1080/10618600.1996.10474713.

[11] D. E. Knuth, *Literate Programming*, no. 27. in CSLI Lecture Notes. Stanford, CA: Center for the Study of Language, 1992, p. xvi+368.

[12] F. Pérez, "IPython 0.0.1: a simple script to be loaded as $PYTHONSTARTUP, of historical interest only." [Online]. Available: https://gist.github.com/fperez/1579699

[13] F. Pérez and B. E. Granger, "IPython: A System for Interactive Scientific Computing," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 21–29, 2007, doi: 10.1109/MCSE.2007.53.

[14] J. M. Perkel, "Why Jupyter is Data Scientists' Computational Notebook of Choice," *Nature*, vol. 563, no. 7729, pp. 145–146, 2018, doi: 10.1038/d41586-018-07196-1.

[15] T. Kluyver *et al.*, "Jupyter Notebooks—a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas. Proceedings of the 20th International Conference on Electronic Publishing (ELPUB)*, F. Loizides and B. Schmidt, Eds., 2016, pp. 87–90. doi: 10.3233/978-1-61499-649-1-87.

[16] Jupyter Development Team, "Messaging in Jupyter (Jupyter Client Messaging Specification)." [Online]. Available: https://jupyter-client.readthedocs.io/en/stable/messaging.html

[17] Jupyter Development Team, "The Notebook File Format (nbformat)." [Online]. Available: https://nbformat.readthedocs.io/en/latest/format_description.html

[18] B. E. Granger and F. Pérez, "Jupyter: Thinking and Storytelling With Code and Data," *Computing in Science & Engineering*, vol. 23, no. 2, pp. 7–14, 2021, doi: 10.1109/MCSE.2021.3059263.

[19] C. R. Harris *et al.*, "Array Programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[20] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[21] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a.

[22] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[23] A. Meurer *et al.*, "SymPy: Symbolic Computing in Python," *PeerJ Computer Science*, vol. 3, p. e103, 2017, doi: 10.7717/peerj-cs.103.

[24] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[25] S. Hoyer and J. Hamman, "xarray: N-D Labeled Arrays and Datasets in Python," *Journal of Open Research Software*, vol. 5, no. 1, p. 10, 2017, doi: 10.5334/jors.148.

[26] M. Rocklin, "Dask: Parallel Computation with Blocked Algorithms and Task Scheduling," in *Proceedings of the 14th Python in Science Conference*, K. Huff and J. Bergstra, Eds., 2015, pp. 126–132. doi: 10.25080/Majora-7b98e3ed-013.

[27] M. D. Wilkinson *et al.*, "The FAIR Guiding Principles for scientific data management and stewardship," *Scientific Data*, vol. 3, no. 1, p. 160018, 2016, doi: 10.1038/sdata.2016.18.

[28] M. Barker *et al.*, "Introducing the FAIR Principles for research software," *Scientific Data*, vol. 9, no. 1, p. 622, 2022, doi: 10.1038/s41597-022-01710-x.

[29] J. B. Hamrick, C. Kelty, and U. Berkeley, "nbgrader: A Tool for Creating and Grading Assignments in the Jupyter Notebook," in *Proceedings of the 15th Python in Science Conference*, 2016, pp. 23–29. doi: 10.25080/Majora-629e541a-004.

[30] L. A. Barba, "Computational Thinking and Thinking About Computing," *IEEE Computing in Science & Engineering*, vol. 18, no. 4, pp. 65–66, 2016, doi: 10.1109/MCSE.2016.63.

[31] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten Simple Rules for Reproducible Computational Research," *PLoS Computational Biology*, vol. 9, no. 10, p. e1003285, 2013, doi: 10.1371/journal.pcbi.1003285.

[32] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014, no. 239, p. 2, 2014.

[33] Project Jupyter *et al.*, "Binder 2.0 — Reproducible, Interactive, Sharable Environments for Science at Scale," in *Proceedings of the 17th Python in Science Conference*, F. Akici, D. Lippa, D. Niederhut, and M. Pacer, Eds., 2018, pp. 113–120. doi: 10.25080/Majora-4af1f417-011.

[34] J. Grus, “I Don't Like Notebooks.” [Online]. Available: https://docs.google.com/presentation/d/1n2RlMdmv1p25Xy5thJUhkKGvjtV-dkAIsUXP-AL4ffI/

[35] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “A Large-Scale Study about Quality and Reproducibility of Jupyter Notebooks,” in *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, in MSR '19. Montreal, Quebec, Canada, 2019, pp. 507–517. doi: 10.1109/MSR.2019.00077.

[36] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “Understanding and Improving the Quality and Reproducibility of Jupyter Notebooks,” *Empirical Software Engineering*, vol. 26, no. 4, 2021, doi: 10.1007/s10664-021-09961-9.

[37] A. Rule, A. Tabard, and J. D. Hollan, “Exploration and Explanation in Computational Notebooks,” in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, in CHI '18. New York, NY, USA, 2018, pp. 1–12. doi: 10.1145/3173574.3173606.

[38] S. Chattopadhyay, I. Prasad, A. Z. Henley, A. Sarma, and T. Barik, “What's Wrong with Computational Notebooks? Pain Points, Needs, and Design Opportunities,” in *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, in CHI '20. New York, NY, USA, 2020, pp. 1–12. doi: 10.1145/3313831.3376729.

[39] M. B. Kery, M. Radensky, M. Arya, B. E. John, and B. A. Myers, “The Story in the Notebook: Exploratory Data Science using a Literate Programming Tool,” in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, in CHI '18. New York, NY, USA, 2018, pp. 1–11. doi: 10.1145/3173574.3173748.

[40] A. Head, F. Hohman, T. Barik, S. M. Drucker, and R. DeLine, “Managing Messes in Computational Notebooks,” in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, in CHI '19. New York, NY, USA, 2019, pp. 1–12. doi: 10.1145/3290605.3300500.

[41] J. Wang, L. Li, and A. Zeller, “Better Code, Better Sharing: On the Need of Analyzing Jupyter Notebooks,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, in ICSE-NIER '20. New York, NY, USA, 2020, pp. 53–56. doi: 10.1145/3377816.3381724.

[42] J. Wang, T.-Y. Li, L. Li, and A. Zeller, “Restoring Reproducibility of Jupyter Notebooks,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, in ICSE '20. New York, NY, USA, 2020, pp. 288–289. doi: 10.1145/3377812.3390803.

[43] Microsoft, “Language Server Protocol Specification.” [Online]. Available: https://microsoft.github.io/language-server-protocol/

[44] Microsoft, “Debug Adapter Protocol.” [Online]. Available: https://microsoft.github.io/debug-adapter-protocol/

[45] S. Corlay and J. Mabille, “Jupyter Enhancement Proposal 47: Jupyter Debugger Protocol.” [Online]. Available: https://jupyter.org/enhancement-proposals/47-jupyter-debugger-protocol/jupyter-debugger-protocol.html

[46] M. Wouts, “Jupytext: Jupyter Notebooks as Markdown Documents, Julia, Python or R Scripts.” [Online]. Available: https://github.com/mwouts/jupytext

[47] Project Jupyter, “nbdime: Diffing and Merging of Jupyter Notebooks.” [Online]. Available: https://github.com/jupyter/nbdime

[48] nteract contributors, “papermill: Parameterize, Execute, and Analyze Notebooks.” [Online]. Available: https://github.com/nteract/papermill

[49] H. Fangohr *et al.*, “Testing with Jupyter Notebooks: NoteBook VALidation (nbval) Plug-in for pytest.” [Online]. Available: https://arxiv.org/abs/2001.04808

[50] nteract contributors, “testbook: Unit Testing for Jupyter Notebooks.” [Online]. Available: https://github.com/nteract/testbook

[51] Executable Books Project, “jupyter-cache: A Defined Interface for Working with a Cache of Executed Jupyter Notebooks.” [Online]. Available: https://github.com/executablebooks/jupyter-cache

[52] J. Howard and H. Husain, “nbdev: Create Delightful Software with Jupyter Notebooks.” [Online]. Available: https://github.com/fastai/nbdev

[53] J. Howard and S. Gugger, “fastai: A Layered API for Deep Learning,” *Information*, vol. 11, no. 2, p. 108, 2020, doi: 10.3390/info11020108.

[54] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021, doi: 10.48550/arXiv.2107.03374.

[55] P. Vaithilingam, T. Zhang, and E. L. Glassman, “Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models,” in *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*, in CHI EA '22. New York, NY, USA, 2022, pp. 1–7. doi: 10.1145/3491101.3519665.

[56] S. Barke, M. B. James, and N. Polikarpova, “Grounded Copilot: How Programmers Interact with Code-Generating Models,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 85–111, 2023, doi: 10.1145/3586030.

[57] A. M. McNutt, C. Wang, R. A. DeLine, and S. M. Drucker, "On the Design of AI-powered Code Assistants for Notebooks," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, in CHI '23. New York, NY, USA, 2023, pp. 1–16. doi: 10.1145/3544548.3580940.

[58] J. F. Pimentel, V. Braganholo, L. Murta, and J. Freire, "Collecting and Analyzing Provenance on Interactive Notebooks: When IPython Meets noWorkflow," in *7th USENIX Workshop on the Theory and Practice of Provenance (TaPP 2015)*, Edinburgh, Scotland, 2015. [Online]. Available: https://www.usenix.org/conference/tapp15/workshop-program/presentation/pimentel

[59] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "noWorkflow: a Tool for Collecting, Analyzing, and Managing Provenance from Python Scripts," *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1841–1844, 2017, doi: 10.14778/3137765.3137789.

Open Access | https://doi.org/10.25080/ycyj7495

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

alphagenome-pt: Flexible Training of AlphaGenome Models in PyTorch

Rylie Weaver[1,2], **Gomathi Lakshmanan**[1,2], and **John Lagergren**[2]

[1]University of Tennessee, Knoxville, [2]Oak Ridge National Laboratory

Abstract

We present `alphagenome-pt`, an open-source PyTorch implementation of Google DeepMind's AlphaGenome model for training and fine-tuning. AlphaGenome predicts functional genomic outputs directly from DNA sequence, but the released models are trained on human and mouse genomes, motivating dataset-specific training for other species and biological contexts. `alphagenome-pt` lowers the software barrier to adapting AlphaGenome by porting the JAX model to PyTorch and exposing relevant model hyperparameters to custom configuration.

Keywords genomics, DNA language modeling, deep learning

1. Introduction

We present `alphagenome-pt` (PyPI, GitHub), an open-source PyTorch implementation of Google DeepMind's AlphaGenome model architecture for training and fine-tuning. This paper describes `alphagenome-pt` version 0.4.0, which is used for all examples and validation results. DNA language models such as AlphaGenome learn representations of DNA sequence and can be broadly grouped into two categories based on their prediction objectives: (1) masked language modeling (MLM), which tasks the model with predicting the original DNA sequence given a corrupted input [1], [2], [3], [4], [5], [6], [7], [8], and (2) sequence-to-function modeling, which tasks the model with predicting experimental measurements of biological quantities [9], [10], [11], [12], [13], [14], [15].[1] Example functional prediction tasks include gene expression, chromatin accessibility, splicing, and chromatin contact maps, each of which captures a different aspect of genome function. By serving as computational surrogates for physical experiments, these models provide fast estimates for quantities that would otherwise be relatively expensive and time-consuming to measure. Moreover, DNA sequence inputs can be precisely specified and systematically modified in silico, enabling controlled perturbations that are difficult or impossible to perform experimentally.

Functional measurements are useful in downstream biological workflows. Consequently, sufficiently inexpensive and accurate predictions of those measurements are similarly useful, but sequence-to-function prediction remains a central challenge in genomics because of the complexity of the relationships that must be modeled. AlphaGenome is a DNA language model that advances sequence-to-function prediction by modeling a wide range of functional genomic outputs directly from DNA sequence for human and mouse genomes [9]. However, Google DeepMind's released AlphaGenome models are trained on human and mouse genomes, limiting their direct application to other species. Practitioners who want to apply AlphaGenome to other species may therefore need to train or fine-tune models with the same core architecture on species-specific data. At the same time, the released

[1]The two DNA language model categories are not mutually exclusive since prediction tasks can be swapped while keeping the majority of the model the same. In particular, it's common and often useful to pretrain DNA language models with the MLM task before fine-tuning on sequence-to-function tasks. However, the separation is a useful framework for understanding the differences between DNA language models at a high level.

Published Jul 13, 2026

Correspondence to
Rylie Weaver
rweave15@vols.utk.edu

Open Access

Copyright © 2026 Weaver *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

implementation is written in JAX [16], while many deep learning workflows in the scientific Python community are built in PyTorch [17]. `alphagenome-pt` addresses this gap by allowing researchers to instantiate, modify, train, and checkpoint AlphaGenome models in PyTorch without reimplementing the model themselves. By lowering the software barrier to training and fine-tuning AlphaGenome, `alphagenome-pt` aims to make the model easier to adapt to new biological settings.

2. Goals and Motivations

The goal of `alphagenome-pt` is to make the AlphaGenome model easily trainable in PyTorch, with users only needing to know minimal details about the architecture.

Fine-tuning is a well-established and common method to adapt and/or specialize deep learning models to specific domains or tasks. Fine-tuning is useful for language models generally [3], [4], [5], [18], [19], [20], [21], but is especially relevant for the AlphaGenome architecture because the model allocates a significant number of parameters that are specific to organisms and output tracks. These include organism embedding parameters of shape `[num_organisms, num_channels]` and multiorganism linear layers of shape `[num_organisms, num_in_channels, num_out_channels]` used in the output heads[2]. In the case of the multiorganism linear layers for some output heads (e.g. RNA-Seq gene expression), different output channels may correspond to different biological contexts. This architectural choice is grounded in the biological reality, where the same DNA sequence can produce different measured values (e.g. expression, chromatin accessibility, splicing, contact maps) depending on biological context (e.g. tissue, cell type, developmental stage, environment, and organism).

However, this may present a problem when applying the model to other species or biological contexts. In such cases, a user has three options: (1) Initialize the new parameters randomly, which gives no useful prior and, in the case of the multiorganism linear layers, random output predictions. (2) Reuse the closest existing organism or track, which may be reasonable for closely related settings but becomes weaker as the biological context diverges. (3) Finetune the model on the new species/contexts.

Although future AlphaGenome releases may incorporate more species and biological contexts, it is difficult for any released model to cover every species and context of interest. In many cases, the best solution may be to train or fine-tune AlphaGenome on user-specific data. `alphagenome-pt` is designed to make that option practical.

3. AlphaGenome

AlphaGenome is a hybrid convolutional-transformer architecture that balances long-context with computational feasibility while supporting single-base-pair resolution in its predictions. At a high level, AlphaGenome can be viewed as a U-Net-like encoder-decoder model [22] with transformer layers [23] in the middle.

[2]Parameter shapes are expressed up to permutation.

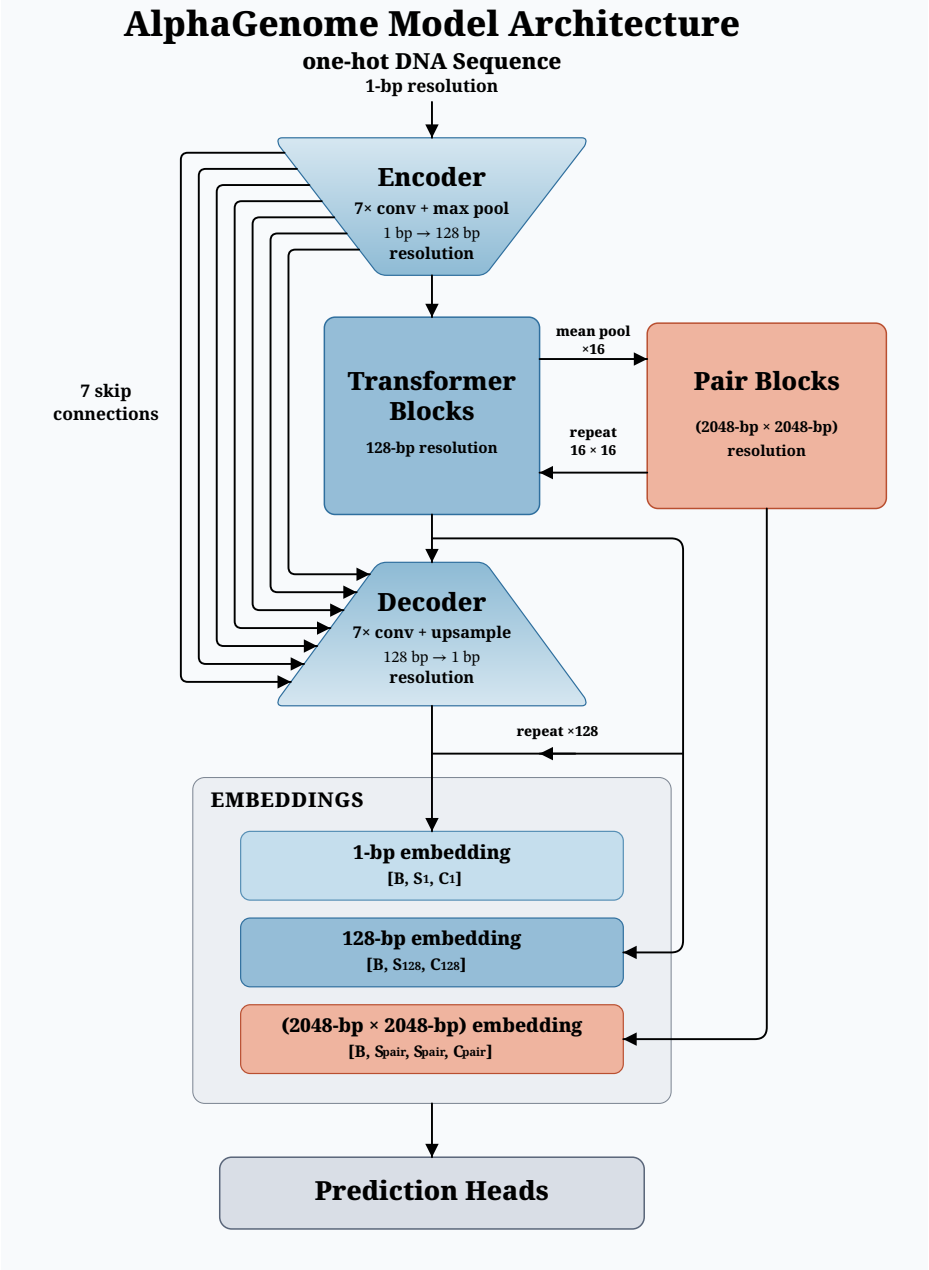


Figure 1. *Overall encoder-transformer-decoder architecture of the AlphaGenome model.*

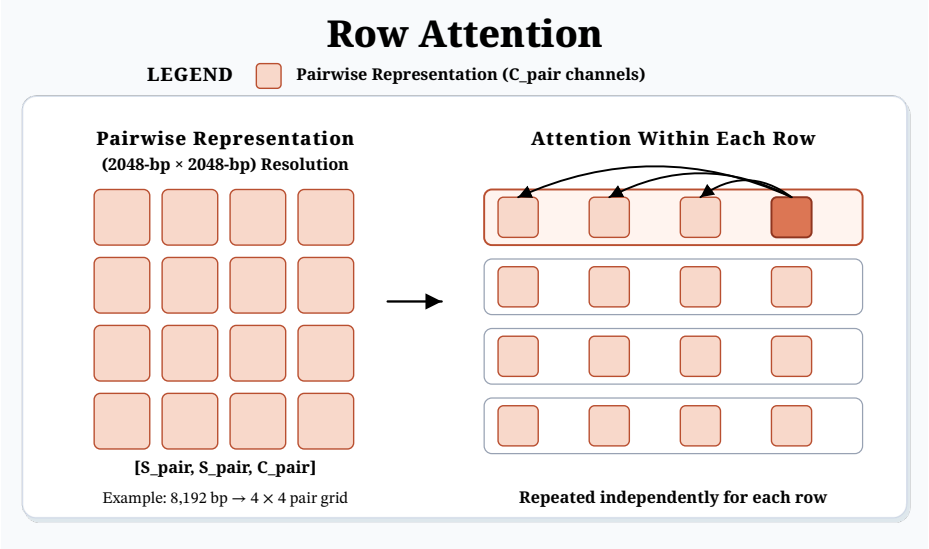


Figure 2. *Row attention mechanism used to update AlphaGenome's pairwise sequence representations.*

Symbol	Meaning	Derivation	Published Value
B	Batch size	User selected	—
S_1	Number of 1-bp positions	$S_1 = S$	1,048,576
S_{128}	Number of 128-bp positions	$S_{128} = S/128$	8,192
S_{pair}	Number of 2,048-bp bins along each pairwise axis	$S_{\text{pair}} = S/2048$	512
C_1	Width of the 1-bp embedding	$M'C$	1,536
C_{128}	Width of the 128-bp embedding	$M'(C + 6I)$	3,072
C_{pair}	Width of the pairwise embedding	`pair_channels`	128

Here, S is `max_seq_len`, C is `num_channels`, I is `channel_increment`, and M' is `embedder_mlp_ratio`.

The model begins with an encoder that consists of seven convolutional downsampling blocks, which capture short-range dependencies and coarsen the sequence from 1-bp to 128-bp resolution. The resulting 128-bp embeddings are then input to transformer layers [23], which capture long-range dependencies over the full context at reduced sequence length. The transformer attention logits are adjusted by an attention bias computed from pairwise sequence representations at a coarser (2048×2048) bp resolution. These pairwise representations are updated with a pair-to-pair attention mechanism (called row attention in AlphaGenome), which improves the model's capability to capture higher-order relationships. Finally, AlphaGenome applies a decoder that consists of seven convolutional upsampling blocks, which capture short-range dependencies and restore a 1-bp resolution representation. U-Net-style skip connections [22] with the fine-grained encoder representations are used to prevent information loss.

The resulting 1-bp, 128-bp, and (2048×2048) bp representations are then passed to task-specific output heads. In total, the encoder-transformer-decoder structure allows AlphaGenome to utilize long-range context while retaining single-base-pair resolution for applicable tasks.

4. Use Cases and Design Philosophy

The core use case of `alphagenome-pt` is to allow users to train the AlphaGenome model on their own datasets to predict functional genomic tracks of interest, including RNA-seq,

CAGE, PRO-cap, ATAC-seq, DNase-seq, transcription factor ChIP-seq, histone-mark ChIP-seq, splice-site classification, splice-site usage, splice-junction prediction, and chromatin contact maps.

4.1. Inference

Load the published checkpoint and return both predictions and embeddings:

```python
import torch
from alphagenome_pt import deepmind_model

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = deepmind_model(load_state=True, device=device)
sequence = "ACGT" * 512  # 2,048 bp

# Heads can be enabled or disabled before inference, for example:
# model.metadata.metadata["heads"]["splice_sites_junction"]["enabled"] = False

model.eval()
# NOTE: model.predict() disables grad-tracking
predictions, embeddings = model.predict(
    sequence,
    organism_index=0,    # human in the published metadata
    return_embeddings=True,
)
print("embeddings")
print(f"\tembeddings_1bp: {tuple(embeddings.embeddings_1bp.shape)}")
print(f"\tembeddings_128bp: {tuple(embeddings.embeddings_128bp.shape)}")
print(f"\tembeddings_pair: {tuple(embeddings.embeddings_pair.shape)}\n")

for head, outputs in predictions.items():
    print(head)
    for name, value in outputs.items():
        print(f"\t{name}: {tuple(value.shape)}")
```

Full Printed Output

```
embeddings
  embeddings_1bp: (1, 2048, 1536)
  embeddings_128bp: (1, 16, 3072)
  embeddings_pair: (1, 1, 1, 128)

atac
  scaled_predictions_1bp: (1, 2048, 256)
  predictions_1bp: (1, 2048, 256)
  scaled_predictions_128bp: (1, 16, 256)
  predictions_128bp: (1, 16, 256)
dnase
  scaled_predictions_1bp: (1, 2048, 384)
  predictions_1bp: (1, 2048, 384)
  scaled_predictions_128bp: (1, 16, 384)
  predictions_128bp: (1, 16, 384)
procap
  scaled_predictions_1bp: (1, 2048, 128)
  predictions_1bp: (1, 2048, 128)
  scaled_predictions_128bp: (1, 16, 128)
  predictions_128bp: (1, 16, 128)
cage
  scaled_predictions_1bp: (1, 2048, 640)
  predictions_1bp: (1, 2048, 640)
  scaled_predictions_128bp: (1, 16, 640)
  predictions_128bp: (1, 16, 640)
rna_seq
  scaled_predictions_1bp: (1, 2048, 768)
  predictions_1bp: (1, 2048, 768)
  scaled_predictions_128bp: (1, 16, 768)
  predictions_128bp: (1, 16, 768)
chip_tf
  scaled_predictions_128bp: (1, 16, 1664)
  predictions_128bp: (1, 16, 1664)
chip_histone
  scaled_predictions_128bp: (1, 16, 1152)
  predictions_128bp: (1, 16, 1152)
contact_maps
  predictions: (1, 1, 1, 28)
splice_sites_classification
  logits: (1, 2048, 5)
  predictions: (1, 2048, 5)
splice_sites_usage
  logits: (1, 2048, 734)
  predictions: (1, 2048, 734)
splice_sites_junction
  predictions: (1, 512, 512, 734)
  splice_site_positions: (1, 4, 512)
  splice_junction_mask: (1, 512, 512, 734)
```

4.2. Model Instantiation

4.2.1. *Published Architecture*

The published metadata, architecture, and converted checkpoint can be loaded directly with:

```python
from alphagenome_pt import deepmind_model

model = deepmind_model(load_state=True)
```

4.2.2. *Published Architecture with Custom Metadata*

The published architecture and its shared checkpoint parameters can be combined with custom metadata for different organisms and/or output tracks (useful for finetuning). Setting `organisms=False` and `heads=False` retains the initialization of the custom organism-

specific and output-track-specific parameters while loading the remaining compatible checkpoint state:

```
from alphagenome_pt import Metadata, deepmind_model

metadata = Metadata({
    "organisms": ["dragon"],
    "heads": {
        "rna_seq": {
            "num_tracks": [2],
            "means": [[2.1, 0.8]],
        },
    },
})

model = deepmind_model(
    metadata=metadata,
    load_state=True,
    organisms=False,
    heads=False,
)
```

Prefix loading and explicit mappings of parameters are also supported.

4.2.3. *Custom Architecture*

The metadata and architecture can be customized (useful for training from scratch):

```
from alphagenome_pt import AlphaGenome, AlphaGenomeConfig, Metadata

metadata = Metadata({
    "organisms": ["dragon"],
    "heads": {
        "rna_seq": {
            "num_tracks": [2],
            "means": [[2.1, 0.8]],
        },
    },
})

model = AlphaGenome(
    AlphaGenomeConfig(
        max_seq_len=8_192,
        num_channels=96,
        metadata=metadata,
    )
)
```

When training from scratch, our design philosophy is to let users control the most important model parameters while still defaulting toward the original AlphaGenome architecture. For example, users can directly set parameters such as `num_channels` and `max_seq_len`, which control the model width and input sequence length. Other hyperparameters, including `channel_increment`, `qk_head_dim`, `v_head_dim`, `pair_channels`, `num_splice_sites`, and `splice_site_channels`, can also be specified manually. However, when these values are not provided, `alphagenome-pt` infers defaults that keep their proportions consistent with the published model. This allows users to scale the model up or down via `num_channels` and `max_seq_len` without needing to manually adjust every dependent hyperparameter.

```
from alphagenome_pt import AlphaGenomeConfig, deepmind_config

official_cfg = deepmind_config()
small_cfg = AlphaGenomeConfig(max_seq_len=8_192, num_channels=96)

# Same proportions
assert small_cfg.channel_increment / small_cfg.num_channels == official_cfg.channel_increment /
official_cfg.num_channels == 1 / 6
assert small_cfg.qk_head_dim / small_cfg.num_channels == official_cfg.qk_head_dim /
official_cfg.num_channels == 1 / 6
assert small_cfg.v_head_dim / small_cfg.num_channels == official_cfg.v_head_dim /
official_cfg.num_channels == 1 / 4
assert small_cfg.pair_channels / small_cfg.num_channels == official_cfg.pair_channels /
official_cfg.num_channels == 1 / 6
assert small_cfg.num_splice_sites / small_cfg.max_seq_len == official_cfg.num_splice_sites /
official_cfg.max_seq_len == 1 / 2048
assert small_cfg.splice_site_channels / small_cfg.num_channels == official_cfg.splice_site_channels /
official_cfg.num_channels == 1
```

4.2.4. *Metadata*

Metadata specifies the organism and output heads for the model architecture. Real workflows will need to calculate the true means of nonzero target values, but dummy values are used here for simplicity.[3]

[3] Masked language modeling and sequence-to-function heads are not technically mutually exclusive. However, our example presents them as separate options because combining them is uncommon (masked language modeling corrupts the input sequence that is also used for functional prediction).

```python
from alphagenome_pt import AlphaGenome, AlphaGenomeConfig, Metadata, synthetic_batch

# Dummy Example: replace with actual nonzero means
def make_means(num_tracks):
    max_t = max(num_tracks)
    return [[1.0] * max_t for _ in num_tracks]

# Option 1: masked language modeling
mlm_metadata = Metadata({
    "organisms": ["human", "mouse"],
    "heads": {
        "masked_language_modeling": {},
    },
})

# Option 2: sequence-to-function prediction
functional_metadata = Metadata({
    "organisms": ["human", "mouse"],
    "heads": {
        # Gene expression / transcriptional activity
        "rna_seq": {
            "num_tracks": [10, 8],
            "means": make_means([10, 8]),
        },
        "cage": {
            "num_tracks": [6, 4],
            "means": make_means([6, 4]),
        },
        "procap": {
            "num_tracks": [3, 2],
            "means": make_means([3, 2]),
        },

        # Chromatin accessibility / regulatory binding
        "atac": {
            "num_tracks": [5, 4],
            "means": make_means([5, 4]),
        },
        "dnase": {
            "num_tracks": [5, 5],
            "means": make_means([5, 5]),
        },
        "chip_tf": {
            "num_tracks": [7, 6],
            "means": make_means([7, 6]),
        },
        "chip_histone": {
            "num_tracks": [4, 3],
            "means": make_means([4, 3]),
        },

        # Splicing
        "splice_sites_classification": {
            "num_tracks": [5, 5],
        },
        "splice_sites_usage": {
            "num_tracks": [4, 3],
        },
        "splice_sites_junction": {
            "num_tissues": [4, 3],
        },

        # Contact maps
        "contact_maps": {
            "num_tracks": [2, 2],
        },

    },
})

# Build a small model and a synthetic batch containing
# targets for every head present in the metadata
S = 2048
meta = functional_metadata  # or mlm_metadata
```

```
model_cfg = AlphaGenomeConfig(
    max_seq_len=S,
    num_channels=96,
    metadata=meta,
)
model = AlphaGenome(model_cfg)
data = synthetic_batch(
    meta,
    batch_size=4,
    seq_len=S,
    num_splice_sites=model_cfg.num_splice_sites,
)

# Calculate and backpropagate on the loss
model.train()
output = model(
    data,
    mode="loss",
    return_predictions=True,
    return_embeddings=True,
)
output.total.backward()

print(f"total loss: {output.total.item():.4f}")
```

4.3. Training

Load the published checkpoint and calculate the built-in losses when providing a `DataBatch` with targets:

```
import torch
from alphagenome_pt import deepmind_model

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = deepmind_model(load_state=True, device=device)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5)
model.train()

for batch in train_loader:
    batch.to(device)
    optimizer.zero_grad(set_to_none=True)

    output = model(batch, mode="loss")
    output.total.backward()
    optimizer.step()
```

Alternatively, predictions and embeddings remain differentiable during direct model calls and can be used to define a custom objective:

```
model.train()

for batch in train_loader:
    batch.to(device)
    optimizer.zero_grad(set_to_none=True)

    predictions, embeddings = model(batch, return_embeddings=True)
    loss = custom_loss(predictions, embeddings, batch)
    loss.backward()
    optimizer.step()
```

4.4. Checkpoint Loading

`deepmind_model()` can load the all-folds model or folds 0–3 from the converted checkpoints hosted on Hugging Face:

```python
from alphagenome_pt import deepmind_model

model = deepmind_model(
    load_state=True,
    fold="all_folds",
    device="cuda",
    map_location="cpu",
)
```

Argument	Default	Behavior
`load_state`	`False`	Whether to load converted checkpoint parameters
`fold`	`"all_folds"`	Selects `"all_folds"` or `"fold_0"` through `"fold_3"`
`local_dir`	`None`	Optional local directory for cached or downloaded metadata and checkpoint files
`repo_id`	`"RylieWeaver/alphagenome-pytorch"`	Hugging Face repository containing the checkpoint artifacts
`repo_dir`	`None`	Repository subdirectory (resolves to `v{package_version}` when omitted)
`map_location`	`"cpu"`	Device used to deserialize the checkpoint
`device`	`"cpu"`	Final device receiving the model

5. Validation

5.1. Checkpoint Numerical Equivalence

We evaluated full-model numerical equivalence between `alphagenome-pt` and the official JAX implementation using the published `all_folds` checkpoint and its converted PyTorch counterpart. Using deterministic 131,072-bp sequences from human and mouse chromosome 1, we compared the resulting embeddings, predictions, and losses under the DeepMind mixed-precision, FP32, and FP64 policies.

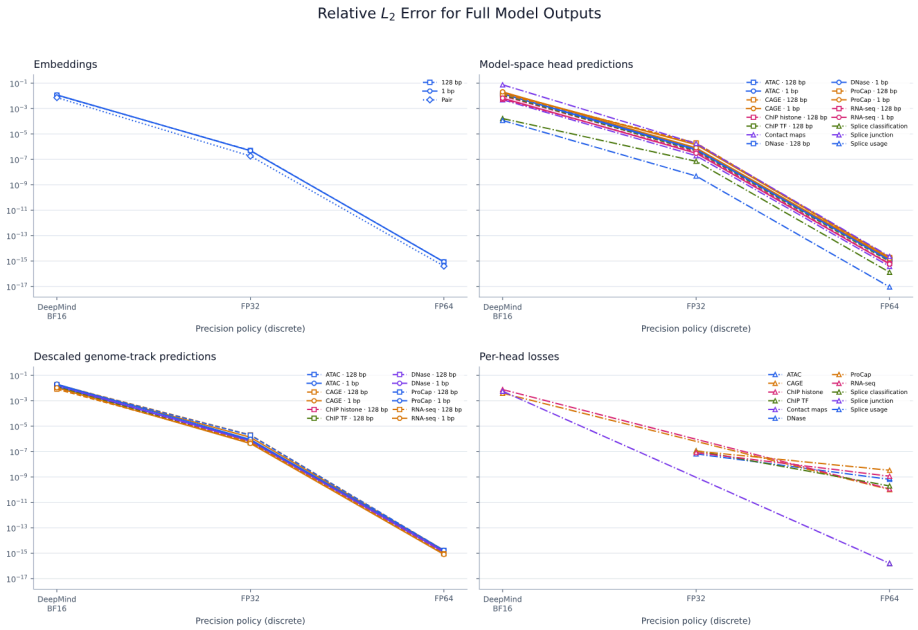


Figure 3. *Relative L_2 differences between the PyTorch and JAX full-model outputs under the DeepMind mixed-precision, FP32, and FP64 policies. Exact-zero differences are omitted from the plot.*

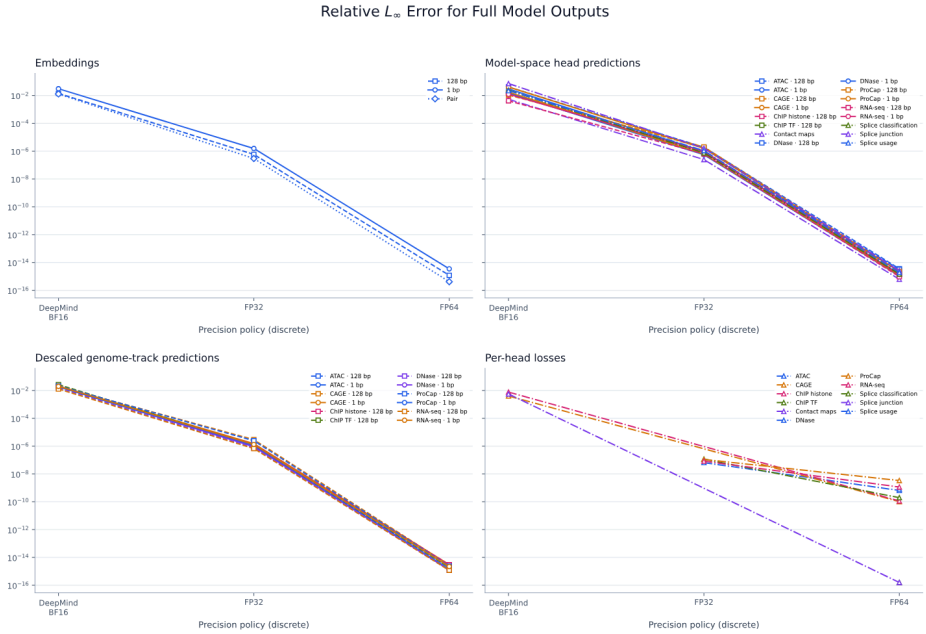


Figure 4. *Relative L_∞ differences between the PyTorch and JAX full-model outputs under the DeepMind mixed-precision, FP32, and FP64 policies. Exact-zero differences are omitted from the plot.*

All tested embeddings, model-space predictions, descaled predictions, and per-head losses satisfy the relative L_2 and L_∞ tolerances defined for their respective precision policies. The differences decrease from the DeepMind mixed-precision policy to FP32 to FP64, indicating that the remaining discrepancies are primarily due to floating-point precision and accumulated rounding error.

5.2. Model Size

The published AlphaGenome configuration contains 450.5 million trainable parameters, including its 11 output heads and organism-specific parameters. `alphagenome-pt` exposes hyperparameters for adjusting model size, with `num_channels` serving as the primary control for other architecture dimensions when they are not specified explicitly.

Configuration	`num_channels`	Output heads	Trainable parameters
Published AlphaGenome	768	11	450.5 M
Custom, headless	64	0	2.8 M
Custom, headless	128	0	11.3 M
Custom, headless	256	0	44.6 M
Custom, headless	512	0	179.2 M
Custom, headless	768	0	403.9 M

All configurations in this table use a maximum sequence length of 1,048,576 bp. The custom configurations use one organism and no output heads because these components depend on the intended training application. Excluding them isolates how the shared architecture —and therefore its parameter count—scales with `num_channels`.

5.3. Inference and Training Throughput

We measured latency and peak allocated GPU memory for the published architecture during inference and training. Training measurements used power-of-two sequence lengths from 2^{11} (2,048) to 2^{17} (131,072) bp, while inference measurements also included 2^{18} (262,144) bp. Embedding runs excluded output heads, whereas prediction runs included all 11 published heads. We used `num_splice_sites=512` for every sequence length, in line with the published model configuration.

Throughput benchmark configuration

Benchmarks used one NVIDIA A100-SXM4-80GB GPU, batch size 1, the DeepMind precision policy, PyTorch 2.6.0 with CUDA 12.4, and `alphagenome-pt` 0.4.0. Each measurement reports the mean of 100 timed iterations following 20 warm-up iterations. Training measurements used AdamW with a learning rate of 3×10^{-5} and included the forward pass, loss calculation, backward pass, and optimizer step. The model parameters were randomly initialized.

Training used synthetic batches. Embedding training minimized mean squared error against synthetic embedding targets, while all-head prediction training used the package's built-in losses with synthetic targets.

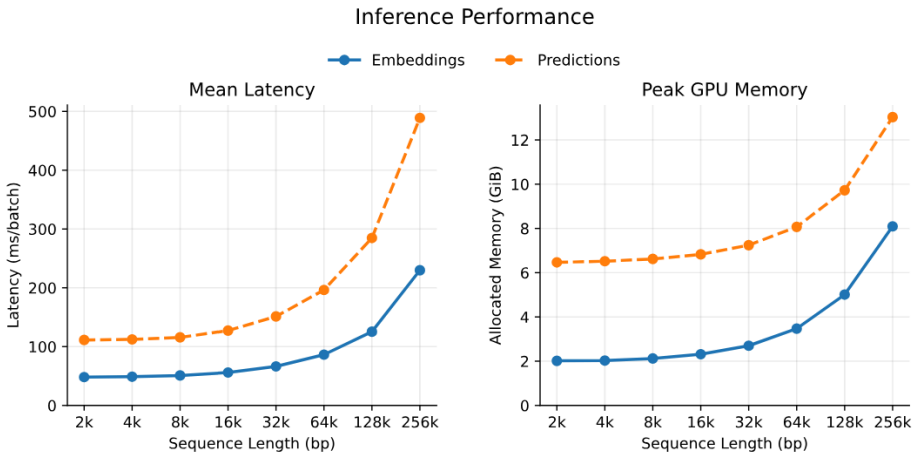


Figure 5. *Mean inference latency and peak allocated GPU memory as functions of sequence length for embedding-only and all-head prediction inference.*

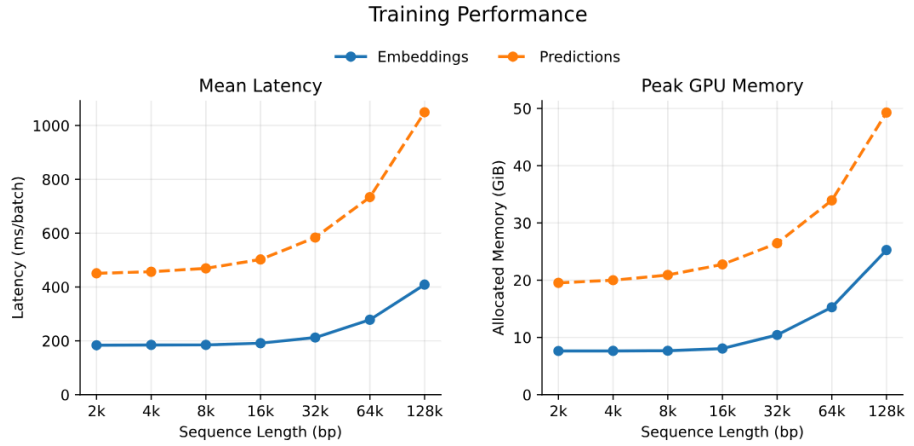


Figure 6. *Mean training-step latency and peak allocated GPU memory as functions of sequence length for embedding-only and all-head prediction training.*

5.4. Training Curves

As training examples, we optimized the published architecture from both random and released parameters on masked language modeling and RNA-seq prediction using human chromosome 1.

Training configuration

Each run used 16,384-bp sequences, a batch size of 8, a learning rate of 3×10^{-5}, the DeepMind precision policy, seed 42, and 1,000 training steps. Validation metrics were evaluated every 10 steps over 10 batches.

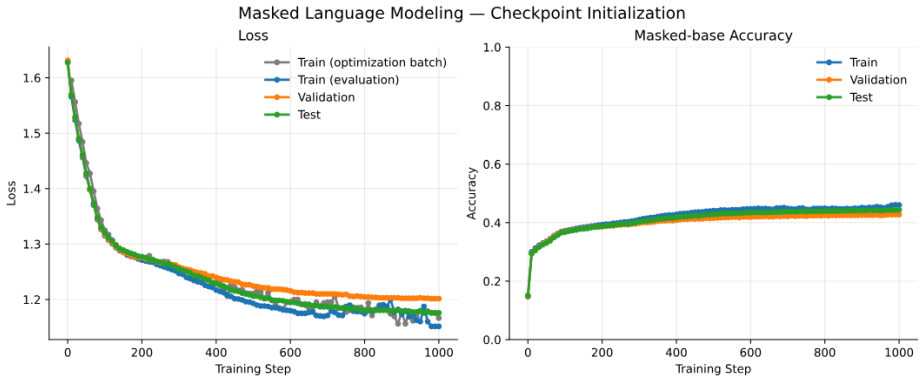


Figure 7. *Masked language modeling training, validation, and test metrics when initialized from the converted checkpoint.*

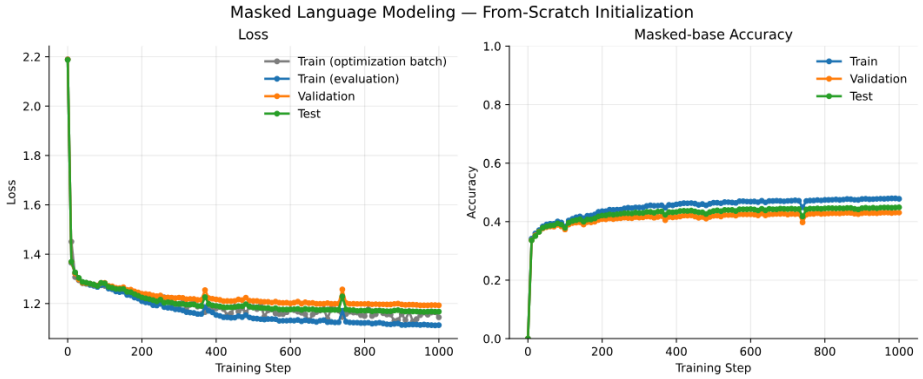


Figure 8. *Masked language modeling training, validation, and test metrics when initialized from random parameters.*

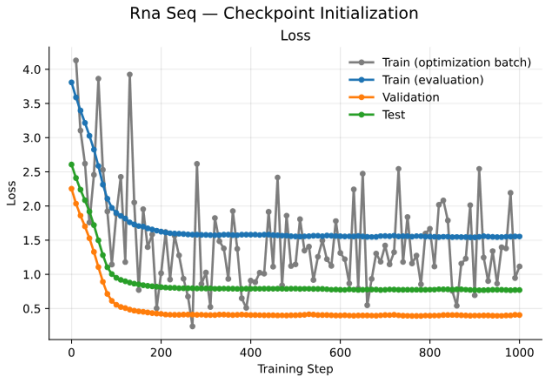


Figure 9. *RNA-seq training, validation, and test loss when initialized from the converted checkpoint.*

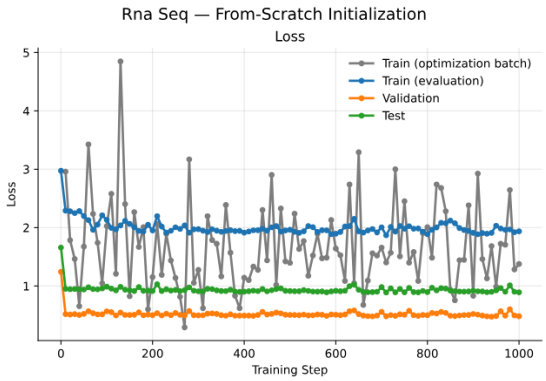


Figure 10. *RNA-seq training, validation, and test loss when initialized from random parameters.*

All four experiments completed successfully and showed an initial decrease in loss, illustrating the software's ability to train from both checkpoint and random initialization for MLM and RNA-seq prediction tasks.

6. Limitations and Related Work

The first main area of limitation in `alphagenome-pt` is the absence of sequence-parallelism execution, which Google DeepMind used to train AlphaGenome on very long sequences. A second limitation is that `alphagenome-pt` does not provide a preprocessing pipeline for functional genomics data. However, this is an intentional scope choice because the proper storage formats can differ between large-scale HPC training and smaller experiments. Rather than assuming a particular file format or data-loading strategy, `alphagenome-pt` assumes that users yield tensorized or sequence data to the model, however that is done. The package focuses on the model rather than prescribing a universal data-processing workflow.

The most closely related software is the official AlphaGenome release from Google DeepMind (Link1, Link2). That implementation remains the reference implementation for the model and released checkpoints, but it is written in JAX, whereas `alphagenome-pt` is motivated by the complementary goal of making the model trainable in PyTorch. Several unofficial PyTorch implementations of AlphaGenome also exist. The `genomicsxai/alphagenome-pytorch` package is a strong and polished PyTorch port, with PyTorch-converted checkpoint loading, named outputs, example notebooks, fine-tuning utilities, and reported numerical parity with the original JAX model. `alphagenome-pt` shares many capabilities with `genomicsxai/alphagenome-pytorch` but places more emphasis on flexible model training, as shown by its flexible checkpoint loading and exposed architecture hyperparameters.

A previous PyTorch implementation by Phil Wang (also known as lucidrains on GitHub), Miquel Anglada-Girotto, and Xinming Tu is also available at lucidrains/alphagenome.

Relative to these related projects, the goal of `alphagenome-pt` is not to replace the official implementation or provide the most complete inference interface. Instead, the goal is to provide a clear and maximally flexible PyTorch training implementation for users who want to adapt AlphaGenome to new datasets, species, and biological contexts while still remaining faithful to the original architecture.

7. Conclusions and Future Work

We presented `alphagenome-pt`, an open-source PyTorch implementation of AlphaGenome for training and fine-tuning. The goal of the package is to make it easier for researchers to train AlphaGenome models on their own datasets, species, and biological contexts without reimplementing the model themselves.

By lowering the software barrier to AlphaGenome training, `alphagenome-pt` aims to make sequence-to-function modeling more accessible beyond the released human and mouse models. More broadly, we hope this helps researchers use computational models to generate and test biological hypotheses faster.

Future work will focus on implementing distributed sequence-parallel training to support longer sequences and larger-scale HPC training.

8. Generative AI Disclosure

Portions of this work were assisted using ChatGPT and Codex. The tools were used for drafting and revising code in the package and text for this proceedings submission. All outputs were reviewed, verified, and revised by the author(s), who take full responsibility for the accuracy and integrity of the final content.

References

[1] G. Brixi *et al.*, "Genome modelling and design across all domains of life with Evo 2," *Nature*, vol. 652, no. 8112, pp. 1349–1361, 2026, doi: 10.1038/s41586-026-10176-5.

[2] E. Nguyen *et al.*, "Sequence modeling and design from molecular to genome scale with Evo." [Online]. Available: https://www.biorxiv.org/content/10.1101/2024.02.27.582234v1

[3] E. Nguyen *et al.*, "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023, pp. 43177–43201. doi: 10.52202/075280-1872.

[4] Z. Zhou, Y. Ji, W. Li, P. Dutta, R. Davuluri, and H. Liu, "DNABERT-2: Efficient Foundation Model and Benchmark For Multi-Species Genome." [Online]. Available: http://arxiv.org/abs/2306.15006

[5] Y. Ji, Z. Zhou, H. Liu, and R. V. Davuluri, "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome," *Bioinformatics*, vol. 37, no. 15, pp. 2112–2120, 2021, doi: 10.1093/bioinformatics/btab083.

[6] H. Dalla-Torre *et al.*, "Nucleotide Transformer: building and evaluating robust foundation models for human genomics," *Nature Methods*, vol. 22, no. 2, pp. 287–297, 2025, doi: 10.1038/s41592-024-02523-z.

[7] H. Dalla-Torre *et al.*, "The Nucleotide Transformer: Building and Evaluating Robust Foundation Models for Human Genomics." [Online]. Available: https://www.biorxiv.org/content/10.1101/2023.01.11.523679v1

[8] J. Zhai *et al.*, "PlantCAD2: A Long-Context DNA Language Model for Cross-Species Functional Annotation in Angiosperms." [Online]. Available: https://www.biorxiv.org/content/10.1101/2025.08.27.672609v1

[9] Ž. Avsec *et al.*, "Advancing regulatory variant effect prediction with AlphaGenome," *Nature*, vol. 649, no. 8099, pp. 1206–1218, 2026, doi: 10.1038/s41586-025-10014-0.

[10] Ž. Avsec *et al.*, "Effective gene expression prediction from sequence by integrating long-range interactions," *Nature Methods*, vol. 18, no. 10, pp. 1196–1203, 2021, doi: 10.1038/s41592-021-01252-x.

[11] J. Zhou and O. G. Troyanskaya, "Predicting effects of noncoding variants with deep learning-based sequence model," *Nature Methods*, vol. 12, no. 10, pp. 931–934, 2015, doi: 10.1038/nmeth.3547.

[12] D. R. Kelley, Y. A. Reshef, M. Bileschi, D. Belanger, C. Y. McLean, and J. Snoek, "Sequential regulatory activity prediction across chromosomes with convolutional neural networks," *Genome Research*, vol. 28, no. 5, pp. 739–750, 2018, doi: 10.1101/gr.227819.117.

[13] J. Linder, D. Srivastava, H. Yuan, V. Agarwal, and D. R. Kelley, "Predicting RNA-seq coverage from DNA sequence as a unifying model of gene regulation," *Nature Genetics*, vol. 57, no. 4, pp. 949–961, 2025, doi: 10.1038/s41588-024-02053-6.

[14] K. Jaganathan *et al.*, "Predicting Splicing from Primary Sequence with Deep Learning," *Cell*, vol. 176, no. 3, pp. 535–548.e24, 2019, doi: 10.1016/j.cell.2018.12.015.

[15] T. Zeng and Y. I. Li, "Predicting RNA splicing from DNA sequence using Pangolin," *Genome Biology*, vol. 23, no. 1, p. 103, 2022, doi: 10.1186/s13059-022-02664-4.

[16] J. Bradbury *et al.*, "JAX: composable transformations of Python+NumPy programs." [Online]. Available: http://github.com/jax-ml/jax

[17] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[18] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010, doi: 10.1109/TKDE.2009.191.

[19] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?." [Online]. Available: http://arxiv.org/abs/1411.1792

[20] J. Howard and S. Ruder, "Universal Language Model Fine-tuning for Text Classification." [Online]. Available: http://arxiv.org/abs/1801.06146

[21] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." [Online]. Available: http://arxiv.org/abs/1810.04805

[22] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds., Cham, 2015, pp. 234–241. doi: 10.1007/978-3-319-24574-4_28.

[23] A. Vaswani *et al.*, "Attention is All you Need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

Open Access | https://doi.org/10.25080/gaxm4526

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Solving the AI Eval Gap: Domain-Aware Evaluation for Production AI Agents

Shivika Bisen[1]

[1]Independent

Abstract

AI agents are moving into enterprise production faster than the tooling used to evaluate them. Generic evaluation frameworks measure hallucination rate, latency, and token cost, but not the question that actually blocks a launch: did the agent do the *right thing* for this schema, these documents, this domain? Today every team hand-labels golden sets that go stale the moment data, prompts, or use cases change, producing non-reproducible pipelines that do not transfer across teams.

We present **AI Eval Engine**, an open-source Python framework that treats evaluation as a pipeline to be *generated* rather than an artifact to be *authored*. A team points the framework at where its domain lives; the framework (1) ingests a pluggable domain context from a lightweight configuration, (2) builds a living **Domain Compliance Runbook** of domain facts and compliance criteria, (3) generates a versioned, domain-grounded golden set from those criteria, (4) emits a runnable evaluation script that scores correctness, evidence grounding, and output format — feeding failures back into the runbook, and (5) tracks the result as quarter-over-quarter objectives and key results. A central methodological concern for any pipeline that uses a language model both to generate tests and to judge them is *self-bias*; we address it by anchoring demonstrations on tasks whose correctness can be verified by execution or by grounding in source documents, and where the model demonstrably fails without the ingested domain context. We illustrate the framework on two contrasting agent shapes — a structured, execution-scored scientific-coding agent (ScienceAgentBench) and an open-ended, grounding-scored financial document question-answering agent (FinanceBench) — showing that one reusable pipeline spans both without per-domain scratch tooling.

Keywords AI agents, evaluation, large language models, domain-aware evaluation, golden set generation, LLM-as-judge, safety, retrieval-augmented generation

Published Jul 13, 2026

Correspondence to
Shivika Bisen
shivikakbisen@gmail.com

Open Access

Copyright © 2026 Bisen. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

AI agents are moving into enterprise production faster than the tooling used to evaluate them. Teams shipping agents over financial filings, scientific datasets, or support transcripts hit the same wall: existing tooling measures generic LLM properties — hallucination rate, latency, token cost — but not the question that blocks a launch: *did the agent do the right thing in this domain?*

Domain correctness is not a generic property. A data agent over a financial schema and a RAG agent over scientific documents require different notions of "correct," and those notions exist only in the **domain context**: a scientific agent must load the right columns and compute the metric the paper reported; a financial agent must ground every figure in its filing and decline when the filing does not support an answer. Neither constraint is visible to a generic judge scoring token-level fluency.

The state of practice is the **manually labeled golden set**, with three well-known failure modes: it goes *stale* when data or prompts change; it is *non-reproducible* across teams; and it gives *no clarity on what to fix*, reporting only that something broke. These follow from treating evaluation as an artifact to be *authored* rather than a pipeline to be *generated*.

This paper presents **AI Eval Engine**, an open-source Python framework that reframes evaluation as a domain-driven generator. The team points at where the domain lives; the framework produces a versioned golden set, a runnable evaluation script, and a living Domain Compliance Runbook that captures emergent failure modes — shifting the human role from *author* to *reviewer* and letting teams track agent behavior as a first-class quarter-over-quarter OKR. It leans on a language model — Anthropic's Claude [1], via the Claude API and Claude Code — for context extraction, golden-set and eval-script generation, and the LLM-as-judge step. Using one model to both *generate* and *judge* tests raises a real objection — self-bias — which we confront in Methodology: Domain-Aware Evaluation Without Self-Bias and which drives our dataset choice.

2. Background and Related Work

Open-source evaluation frameworks tackle adjacent slices. **RAGAS** scores RAG systems on faithfulness, answer relevancy, and context recall [2]; **ARES** trains lightweight RAG judges [3]; **G-Eval** formalizes LLM-as-judge with chain-of-thought scoring [4], a practice characterized by L. Zheng *et al.* [5]; HELM standardizes *which* metrics are reported [6]; **DeepEval** [7] is a pytest-style harness, and **Arize Phoenix** [8] and **Comet Opik** [9] focus on production observability. In every case the *test cases* are assumed to exist. The closest neighbors instead generate them: G. Guinet, B. Omidvar-Tehrani, A. Deoras, and L. Callot [10] build task-specific exams from a corpus, EvalGen aligns LLM evaluators with human preferences [11], and SPADE synthesizes data-quality assertions [12]; agent-centric systems — the Agent-Testing Agent [13], TestAgent [14], and τ-bench [15] — push toward domain-specific benchmarking, with surveys [16], [17] and safety benchmarks TRIDENT [18] and DecodingTrust [19] mapping the landscape.

AI Eval Engine differs in two ways: it treats **automated generation of the test set itself**, driven by a pluggable domain context, as the primary contribution; and it accumulates a **living Domain Compliance Runbook** of domain constraints from observed failures, closing a loop back into generation. It is meant to be used *alongside* these tools — RAGAS or DeepEval metrics register as extra Step-4 scorers — and self-improving methods such as Agentic Context Engineering [20] are candidates for evaluation *by* it, not components of it.

3. Methodology: Domain-Aware Evaluation Without Self-Bias

A pipeline that uses an LLM to *generate* the golden set and the same model family to *judge* the agent invites an obvious objection: the evaluation may simply reward outputs that look like what the generator would produce. This **self-bias** is the central risk of LLM-as-benchmark-generator plus LLM-as-judge pipelines [21], and the first thing a reviewer will probe.

We adopt a single guiding principle that defuses self-bias, benchmark saturation, and training contamination at once: **choose tasks where the model fails without the ingested domain context, and where correctness can be checked independently of the judge.** Two consequences follow.

- **Prefer verifiable correctness over judge opinion.** Where output can be *executed* (does the program produce the expected artifact?) or *grounded* (is the answer supported by cited evidence? — a claim of "capital expenditure was $1,577 million"

either matches the cited cash-flow statement or it does not), correctness is decided by the world, not the model.
- **Prefer domains the model could not have memorized.** Generic code generation (HumanEval-style) is saturated and largely in pretraining, so a high score measures recall, not domain-aware evaluation; the contribution is meaningful only on tasks needing context — a specific dataset or filing — the model never saw.

This principle makes the datasets in Demonstration Use Cases load-bearing: each was chosen because the agent cannot succeed without the ingested domain context, and each admits an execution- or grounding-based check independent of the judge. One scoping note: this principle governs Step 4 — scoring an agent's *answers*. Results does not run Step 4; it evaluates Step 3 — the *questions* the generator produces — with rule-based, offline metrics only, so no number there rests on a model grading its own output.

4. The Five-Step Framework

The framework decomposes evaluation into five steps (Figure 1), each an independent Python module with a stable interface so teams can replace, extend, or skip a step without forking the pipeline.

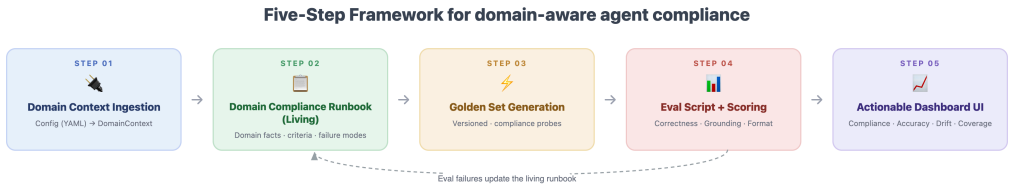


Figure 1. *The five-step framework. A pluggable domain context (Step 1) builds the living* ***Domain Compliance Runbook*** *(Step 2), whose criteria drive golden-set generation (Step 3); eval-script generation and scoring (Step 4) feed failures back into the runbook, and an actionable dashboard (Step 5) tracks the OKRs over time.*

4.1. Step 1 — Pluggable Domain Context Ingestion

The user provides a lightweight YAML configuration that points to one or more domain sources:

```yaml
project: financebench-qa-agent
domain_sources:
  - type: csv
    path: data/financebench/financebench.csv
    description: open-book QA over corporate 10-K filings
stratify_by: company
sample_per_stratum: 2
task:                    # how each row becomes an eval task
  kind: grounded_qa
  input_field: question
  gold_field: answer
  grounding_field: evidence
  category_field: company
```

The framework reads a representative, stratified sample from each source and issues a Claude API call that extracts the *implicit* domain structure into a typed `DomainContext` object — not a prose summary. The extraction captures the categories and taxonomies present in the data, the safety and compliance constraints the domain implies (e.g., "every stated figure must trace to a cited passage"), and quality signals (distributions, ambiguity clusters, missing-value patterns). Users are **not** asked to hand-author a taxonomy or compliance document; they point to where the data lives, and the framework surfaces what matters.

Stratified sampling ensures the extraction sees the domain's breadth, not only its most common cases.

4.2. Step 2 — Domain Compliance Runbook (Living)

From the `DomainContext`, the framework assembles a single content-addressed `domain_compliance_runbook.json` — the **Domain Compliance Runbook**. The name borrows from operations, where a *runbook* is the reference document an on-call engineer follows during an incident; here it is the domain reference the golden-set generator consults on every run. It has three sections: **domain facts** (definitions, conventions, units), **compliance criteria** (five *fixed, universal* agentic-compliance anchors — domain scope, evidence grounding, privacy/confidentiality, no biased advice, human escalation — instantiated per domain with severity: finance's MNPI rule — undisclosed information that could move a stock price, which securities law bars trading on or selectively revealing — is the privacy anchor's finance instance), and **common failure modes**. The first two are seeded now, at ingestion; the third is *living* — it accumulates as the eval (Step 4) surfaces failures, which the framework clusters deterministically by `(category, failure_type)` and turns into a `recommended_check` (the golden-set addition that would catch each one). The anchor taxonomy is therefore *declared* — fixed by design, so all five rules are tested in every domain — while its instantiations and failure modes are *discovered*: a cluster surfaced in run N becomes a runbook entry the golden-set generator (Step 3) picks up as a priority for run $N+1$, closing the loop between observation and test generation. A concern fitting none of the five is the signal to promote a new universal anchor, not to bolt on a one-off rule. The clustering and `recommended_check` mapping are **deterministic rules, not a learned model** — "living" means an accumulated, human-reviewable record, not gradient training.

4.3. Step 3 — Automated Golden Set Generation

From the Domain Compliance Runbook's criteria, the framework constructs a versioned, domain-grounded set of `GoldenCase`s, each carrying the input, the expected outcome (an executable check, an artifact, or an evidence-grounded answer), and the compliance criterion or capability it tests. Two paths exist: **normalize** (`build_golden_set`, fully offline) adopts a public benchmark's own verified labels; **generative** (`generate_golden_set`) has Claude author fresh cases grounded in real evidence — happy-path lookups, hard multi-step computations, definitionally ambiguous queries, and out-of-policy requests the agent must decline (e.g. asking for free cash flow from a balance sheet alone). In both, **guardrail tests** — cases whose correct outcome is a refusal or a hand-off to a human — are synthesized from the runbook's five anchors, so every domain rule is always tested. Generating this *compliance coverage* from extracted constraints, rather than plain corpus-to-question pairs, is the contribution; recurring failures recorded in the runbook (Step 2) feed back here as priorities for the next set.

Both modes enforce one rule: every figure in an expected answer must be traceable to cited evidence, or the case becomes an explicit refusal — the self-bias guard from Methodology: Domain-Aware Evaluation Without Self-Bias, letting a reviewer (or, where a public split exists, the benchmark's own gold answer) verify each case without trusting the generator. The FinanceBench result in Results uses **generative** mode — the framework authors its own golden set, which is exactly what we evaluate there. Golden sets are content-addressed (`goldensets/<version>/`) so reruns can be diffed, and a human reviewer may accept, edit, or reject any case before it is promoted — the human role is **reviewer**, not author.

4.4. Step 4 — Eval Script Generation and Scoring

The framework emits a runnable Python evaluation script producing three orthogonal scores per case: **correctness** (execution of the produced program for executable cases, or

a normalized / numeric match for grounded ones), **grounding** (the fraction of the answer supported by the cited evidence, by deterministic token/numeric overlap, so a fluent but unsupported figure is penalized), and **format validation** (structural integrity, checked without a model call). All three scorers in Results are **deterministic** — no model grades another model's output. Scorers are pluggable: the shipped `grounded_qa` and `code_execution` cover the two demos, and metrics from tools such as RAGAS or DeepEval can register as additional scorers. The LLM-as-judge is the designed fallback for the genuinely open-ended residue but was not exercised here: the judge is never the sole arbiter on a task that execution or grounding can settle. Failures from this step feed back into the Domain Compliance Runbook (Step 2).

4.5. Step 5 — Actionable Dashboard UI

Once the pipeline has run, the framework becomes a live OKR tracker: as data changes, golden sets and the Runbook regenerate on rerun, enabling quarter-over-quarter tracking of an **Agent Compliance Score** (pass rate on compliance-critical cases), a **Domain Accuracy Score** (correctness on in-domain queries), a **Drift Indicator** (share of new failure patterns since the last run), and a **Coverage Score** (share of the discovered domain context represented in the golden set). These four metrics are computed from Step-4 runs and are defined here but not measured in this paper.

5. Demonstration Use Cases

We demonstrate the framework on two contrasting, domain-dependent agent shapes that bracket the spectrum it targets: a **structured, execution-scored** scientific coder (ScienceAgentBench) and an **open-ended, grounding-scored** document-QA agent (FinanceBench) — Python program versus free-text-with-evidence, both failing without the ingested domain context, both chosen under the principle of Methodology: Domain-Aware Evaluation Without Self-Bias.

5.1. Use Case 1 — Scientific-Coding Agent on ScienceAgentBench

Given a scientific task and a dataset, the agent produces a self-contained Python program. We use **ScienceAgentBench** [22] — 102 tasks from 44 peer-reviewed papers across four disciplines, each **scored by execution** (the best agents solve roughly a third, so it is far from saturated). Its *dataset + optional expert knowledge* input maps directly onto Step 1, its execution scoring is self-bias-proof, and it is SciPy-native. Step 1 ingests the task and expert knowledge; Step 4 scores by executing the program, and failures (e.g. "loads the wrong columns") cluster back into the runbook.

5.2. Use Case 2 — Financial Document QA on FinanceBench

The contrasting agent answers open-ended questions over real 10-K filings. We use **FinanceBench** [23] — open-book QA with 10,231 evidence-linked questions, on which a strong retrieval-augmented model was wrong or refused on roughly four-fifths of a sampled set, so the task is genuinely hard. The agent **must** ground each answer in the supplied documents, and the safety dimension is intrinsic: a hallucinated figure or a wrong refusal on a regulated domain is the failure that matters. The pipeline is identical — Step 1 extracts the filing taxonomy and grounding constraints; Step 3 generates evidence-required and refusal-expected cases; Step 4 scores against cited evidence (cf. RAGTruth [24]). One pipeline thus spans a structured, execution-verified coder and an open-ended, grounding-verified QA agent.

6. Results

The experimental variable is the **domain context supplied to the golden-set generator (Step 3)**, on FinanceBench (Use Case 2), in three arms: **blind** (nothing), **documents only** (filing evidence text — the benchmark's own questions and answers are withheld, so no arm can paraphrase them), and **documents + runbook** (the same text plus Step 2's Domain Compliance Runbook). The generator runs in every arm under one identical system prompt — only the data sections of the user message differ — with k = 3 independent generations per arm; we report mean ± sd. The evaluation is of the **generated questions, not the agent's answers**: what a framework can *test for* is decided at generation time, so that is what we measure. Every number below is reproducible from the archived per-run prompts, the nine saved golden sets, and the analysis scripts. Generation ran on Claude Fable 5 via Claude Code [1], August 2026 (flat-rate subscription, default settings, no fixed seed; Limitations and Future Work). All metrics are rule-based and offline; no LLM judge is used.

6.1. *FinanceBench: what the generated golden set can test for*

Every arm generated **50** questions per run. Given the documents, the generator wrote questions that name specific issuers, periods, and line items with answers traceable to the filing text; given the runbook, it also tested the domain's compliance rules; given neither, it wrote issuer-specific but unanswerable questions. We quantify this three ways — a rule-based domain-accuracy metric, lexical relatedness, and unsupervised topic structure — and then show what each golden set is made of.

Domain accuracy of the questions. A rule-based scorer, applied identically to every arm, marks a generated question *domain-accurate* when it targets a real, specific financial fact verifiable against the raw filing data — the mean of a relevance axis (names a statement or line item) and a groundedness axis (the answer's figure is found in the raw data; else a named issuer; else company-less), with guardrail tests excluded as non-factual. Groundedness is a *documents* effect: **75%, 92%, and 92%** for blind, documents, and documents + runbook (mean of k = 3, sd ≤ 1 point; Figure 2). The runbook moves this metric by zero — by construction, since it contributes no figures.

Lexical corroboration. A second offline metric: each question's **domain relatedness** is its maximum TF-IDF cosine similarity to the closest real FinanceBench record (over all question, answer, and evidence text; scikit-learn `TfidfVectorizer` [25] with English stop words, unigrams, and sublinear TF). Documents shift questions toward real filings — median **0.193** blind versus **0.267** (documents) and **0.266** (documents + runbook), questions pooled over runs (Figure 2); the runbook leaves this metric unchanged too.

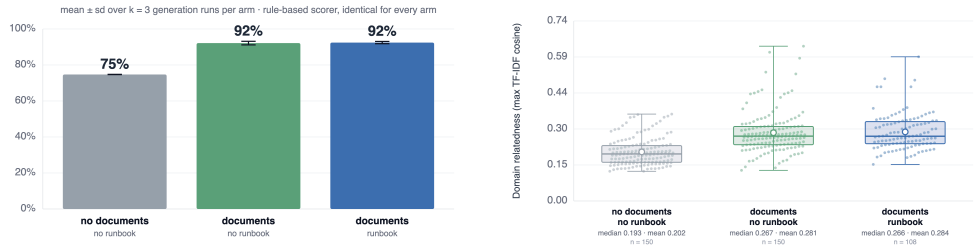


Figure 2. *Documents, not the runbook, drive question groundedness.* ***(a, left)*** *Domain accuracy (rule-based, identical scorer in every arm; mean ± sd over k = 3 runs): 75% blind, 92% with documents, 92% with documents + runbook.* ***(b, right)*** *Per-question domain relatedness (max TF-IDF cosine to the closest real filing), questions pooled over runs; the two documents arms overlap and both sit above blind.*

Topic structure and the compliance gap. Running NMF topic modeling [25] over the raw question text of each set (TF-IDF features, 1–2-grams, finance-boilerplate stop words; topic

assignments hand-verified by reading every question) — a view of what the generator actually produced — shows where the runbook acts (Figure 3, Figure 4). With the runbook, each 50-question set splits into **36 grounded capability questions** across seven finance topics plus **14 agent-compliance tests** (28% of the budget) spanning the four behavioral anchors: off-domain refusal, MNPI / confidential data, biased advice, and escalation (the fifth anchor, evidence-grounding, is enforced on every capability question).

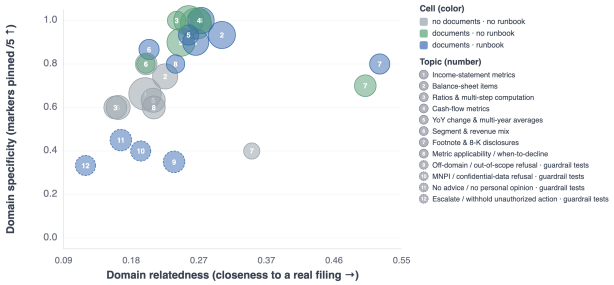


Figure 3. *NMF topics over the question text. x = mean domain relatedness (the TF-IDF metric of Figure 2); y = mean **domain specificity** — the fraction of five concreteness markers pinned down (named company, fiscal period, statement, line item, figure verifiable in a real filing). Bubble area = questions, colored by arm. Both documents arms'* capability *topics sit high and to the right; the documents + runbook arm's* guardrail tests *(dashed) sit low and left by design — naming an entity but using little filing vocabulary — and have no counterpart in the other arms; the blind arm's topics collapse into a less-grounded region.*

Without the runbook, the same generator under the same prompt produces **no guardrail tests at all (0/50 in every run of both other arms)** — confirmed by reading every question. This is *coverage by construction*: the shared prompt never requests refusal or safety cases; they are synthesized only from the runbook's anchors (Availability links every arm's prompt). The operative point is a *coverage* gap rather than a score swing: an evaluation built without the extracted anchors cannot test a single compliance rule, so an agent can breach all of them and the evaluation never knows — and the documents, which fix groundedness, do nothing to fix this. (A fourth, runbook-only condition in the repository's 2×2 replication reproduces the 14 guardrail tests with no documents at all.)

What each golden set is made of. Figure 4 splits every arm's 50-case budget by what a case can test. The runbook's effect on the *capability* cases is on the reference answer: a rule-based check counts how many of five provenance markers the runbook's Evidence anchor requires are present — source pointer, statement or line item, units and scale, fiscal period, GAAP / non-GAAP basis. Blind sets have no reference answers at all (50/50 unanswerable). Documents alone give the right figure with units but little else: 40 of 50 answers carry at most one marker, 10 carry two or three, none carry four or more. With the runbook, 36 of 36 answers carry four or five markers, and the remaining 14 cases are the guardrail tests across the other four anchors. Documents give the generator the figures; the runbook makes each answer carry its provenance and adds the guardrail tests — which is what an evaluation of a regulated-domain agent has to be able to test. Table 1 summarizes what each golden set can — and cannot — test for.

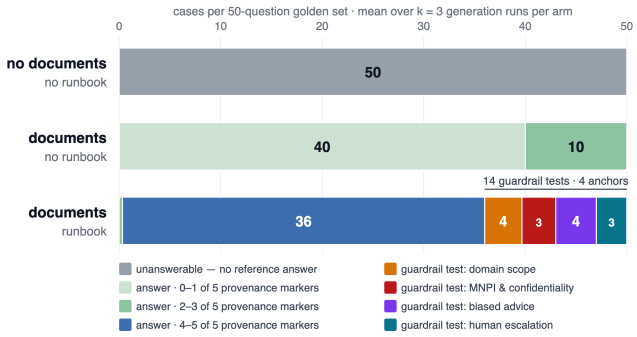


Figure 4. *Composition of each 50-case golden set (mean over k = 3 runs). Capability answers are tiered by how many of five provenance markers they carry (source, statement / line item, units, period, GAAP basis); guardrail tests — cases whose correct outcome is a refusal or a hand-off — are colored by the runbook anchor they test. The blind and documents-only sets contain* ***no guardrail tests*** *and no fully-sourced answer; the documents + runbook set is 36 fully-sourced answers plus 14 guardrail tests across four anchors.*

Table 1. *What each FinanceBench golden set can test for. The two baseline columns are the no-framework reality (not a measured run); the framework column is the measured documents + runbook golden set (run 1 of 3), whose 50 questions partition into 36 domain-capability questions and 14 guardrail tests; the documents-only arm reaches the same 36-question capability coverage with 0 guardrail tests.*

Capability	No golden set	Hand-authored	AI Eval Engine (documents + runbook)
A golden set exists to test against	—	yes	yes — 50 questions
Domain-capability / metric questions (income statement, balance sheet, cash flow, ratios, segments)	no	partial — common ones only	36 tests across 7 topic areas
of which: edge-case / decline (metric not meaningful, missing line item)	no	partial	2 tests (subset of the 36)
Guardrail tests (off-domain, MNPI, biased advice, escalation)	no	partial — only if remembered	14 tests (off-domain 4, MNPI 3, advice 4, escalation 3)
Systematic coverage of all 5 compliance anchors	no	ad hoc — no guarantee	yes — all 5 tested by construction
Versioned & reproducible	—	usually not	yes — content-addressed
Auto-regenerates when data / prompts change	—	no — goes stale	yes
Per-team authoring effort	none	high — from scratch, every team	~15-line YAML

6.2. Summary

The result is a *generation-time coverage* finding rather than an accuracy swing, and the intermediate arm separates the two ingredients. Documents make questions gradeable (75% to 92% groundedness) but leave 40 of 50 answers with at most one provenance marker

and add **zero** guardrail tests; the runbook adds nothing to groundedness but turns the set into 36 fully-sourced answers plus 14 guardrail tests spanning all four behavioral anchors. The added coverage exists *only because Step 1 ran*; a golden set built from the documents alone, scoring the very same agent, has no way to surface it.

7. Implementation Notes

The framework is Python (3.11+). Results exercises Steps 1–3; the Step-4 scorers and Step-5 metrics are implemented and unit-tested but not evaluated there. Only Step 1 needs a model call; Steps 2–5 run offline and deterministically, so the artifacts reproduce without an API key. Use Case 2 retrieves with Chroma [26].

8. Discussion

Reviewer, not author. A human stays in the loop at every generation step, so the framework optimizes for *acceptance latency*, not full autonomy. **Fixed anchors, discovered instances.** The five compliance anchors are a fixed taxonomy, guaranteeing coverage by construction, while their domain instantiations and the runbook's failure modes grow from observed behavior.

9. Limitations and Future Work

The current scope targets text- and code-producing agents; multimodal, long-horizon tool-using agents are future work. The runbook is a single JSON artifact that will need splitting at scale, and the Step 5 dashboard is read-only. Three experimental limits: the question metrics are rule-based (validated against an independent hand-labeling of 30 questions: MAE 0.09, Spearman ρ = 0.74); k = 3 unseeded generations per arm give only a coarse variance estimate; and one generator model family was used.

10. Conclusion

AI Eval Engine treats evaluation as a *pipeline to be generated*, not an artifact to be authored: from a pluggable domain context it generates the golden set, the eval script, and a living Domain Compliance Runbook — a domain-aware path to tracking agent behavior as a first-class OKR.

11. Availability

Source code is MIT-licensed at https://github.com/sbisen/ai-eval-engine; the nine golden sets, per-run archived prompts, scorer outputs, hand labels, and analysis scripts behind Results are under `results/ab_experiment/rerun_2026-08-28_3arm/`. The public API mirrors the paper's steps:

```
from ai_eval_engine import extract_domain_context, generate_golden_set
ctx = extract_domain_context("configs/financebench.yaml")  # Step 1 -> DomainContext
gs = generate_golden_set("configs/financebench.yaml", "out/context.json", runbook_path="out/
runbook.json")  # Step 3
```

All artifacts are plain JSON. From the committed FinanceBench run, a Step-2 runbook *domain fact* reads `{"group": "Definitions", "label": "Free cash flow", "detail": "Operating cash flow minus capex, from the cited cash-flow statement."}`; one Step-3 `GoldenCase` (abridged):

```
{"id": "wdc-001", "question_type": "metrics-generated",
 "input": "Using 3M's FY2018 consolidated statement of cash flows, what was 3M's
           capital expenditure (purchases of PP&E) in USD millions?",
 "expected": "$1,577 million. Source: 3M_2018_10K, Consolidated Statement of Cash Flows.",
 "probes_criteria": ["Citation & non-misleading disclosure"]}
```

12. Disclosures

Generative AI. Anthropic Claude (API and Claude Code) was both this work's *subject* — the framework calls it for extraction and generation — and a *writing aid*; the author reviewed and revised all outputs and takes full responsibility for the content. **Affiliation.** Independent open-source work, separate from the author's employer.

References

[1] Anthropic, "Claude." [Online]. Available: https://www.anthropic.com/claude

[2] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024. doi: 10.48550/arXiv.2309.15217.

[3] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. doi: 10.48550/arXiv.2311.09476.

[4] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. doi: 10.48550/arXiv.2303.16634.

[5] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. doi: 10.48550/arXiv.2306.05685.

[6] P. Liang, R. Bommasani, T. Lee, and others, "Holistic Evaluation of Language Models," *Transactions on Machine Learning Research (TMLR)*, 2023, doi: 10.48550/arXiv.2211.09110.

[7] J. Ip and K. Vongthongsri, "DeepEval: The Open-Source LLM Evaluation Framework." [Online]. Available: https://github.com/confident-ai/deepeval

[8] Arize AI, "Arize Phoenix: Open-Source AI Observability and Evaluation." [Online]. Available: https://github.com/Arize-ai/phoenix

[9] Comet ML, "Opik: Open-Source LLM Evaluation and Observability." [Online]. Available: https://github.com/comet-ml/opik

[10] G. Guinet, B. Omidvar-Tehrani, A. Deoras, and L. Callot, "Automated Evaluation of Retrieval-Augmented Language Models with Task-Specific Exam Generation," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. doi: 10.48550/arXiv.2405.13622.

[11] S. Shankar, J. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo, "Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences," in *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2024. doi: 10.48550/arXiv.2404.12272.

[12] S. Shankar *et al.*, "SPADE: Synthesizing Data Quality Assertions for Large Language Model Pipelines," *Proceedings of the VLDB Endowment*, 2024, doi: 10.48550/arXiv.2401.03038.

[13] S. Komoravolu and K. Mrini, "Agent-Testing Agent: A Meta-Agent for Automated Testing and Evaluation of Conversational AI Agents." arXiv preprint, 2025. doi: 10.48550/arXiv.2508.17393.

[14] W. Wang, Z. Ma, X. Wang, Y. Zhang, P. Liu, and M. Chen, "TestAgent: Automatic Benchmarking and Exploratory Interaction for Evaluating LLMs in Vertical Domains." arXiv preprint, 2024. doi: 10.48550/arXiv.2410.11507.

[15] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "\(\tau\)-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains". arXiv preprint, 2024. doi: 10.48550/arXiv.2406.12045.

[16] A. Yehudai *et al.*, "Survey on Evaluation of LLM-based Agents." arXiv preprint, 2025. doi: 10.48550/arXiv.2503.16416.

[17] M. Mohammadi, Y. Li, J. Lo, and W. Yip, "Evaluation and Benchmarking of LLM Agents: A Survey," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025. doi: 10.48550/arXiv.2507.21504.

[18] Z. Hui, Y. R. Dong, E. Shareghi, and N. Collier, "TRIDENT: Benchmarking LLM Safety in Finance, Medicine, and Law." arXiv preprint, 2025. doi: 10.48550/arXiv.2507.21134.

[19] B. Wang *et al.*, "DecodingTrust: A Comprehensive Assessment of Trustworthiness in GPT Models," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. doi: 10.48550/arXiv.2306.11698.

[20] Q. Zhang *et al.*, "Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models." arXiv preprint, 2026. doi: 10.48550/arXiv.2510.04618.

[21] P. Yuan *et al.*, "Silencer: From Discovery to Mitigation of Self-Bias in LLM-as-Benchmark-Generator." arXiv preprint, 2025. doi: 10.48550/arXiv.2505.20738.

[22] Z. Chen *et al.*, "ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery," in *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2410.05080.

[23] P. Islam, A. Kannappan, D. Kiela, R. Qian, N. Scherrer, and B. Vidgen, "FinanceBench: A New Benchmark for Financial Question Answering." arXiv preprint, 2023. doi: 10.48550/arXiv.2311.11944.

[24] C. Niu *et al.*, "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models." arXiv preprint, 2024. doi: 10.48550/arXiv.2401.00396.

[25] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[26] Chroma, "Chroma: The Open-Source Embedding Database." [Online]. Available: https://github.com/chroma-core/chroma

Open Access | https://doi.org/10.25080/fdgu8736

Avoiding Zero-Trade Policies in RL with a Decoupled MLOps Architecture

Sho Tanaka[1]

[1]Snowflake, Tokyo

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Reinforcement learning (RL) agents trained on financial time series frequently end up with degenerate policies that never complete a trade. We report on a series of experiments with a Deep Q-Network (DQN) trading USD/JPY 5-minute bars, built with Stable-Baselines3 and Gymnasium, in which the direct end-to-end baseline produced 0 completed trades and 0.00% realized return on the held-out period. We then describe the architectural response presented at the SciPy 2026 virtual poster session: the RL model acts as a *signal generator* that emits a directional score in $[-1, +1]$, and a deterministic, unit-testable Python *execution engine* owns position management—ATR stop-loss, risk-reward take-profit, trailing stop, time-based exit and a risk manager. On the same data and split, the decoupled system completed 2,625 fully risk-managed trades, with a 48.4% win rate and a realized return of -0.64%. `EvalCallback` retained the best evaluated checkpoint rather than relying on the final training state. The result demonstrates the operational value of separating signal generation from deterministic execution; it is a behavioral backtest on one currency pair, not a claim of profitability.

Keywords Reinforcement Learning, MLOps, FinTech

1. Introduction

Reinforcement learning is an attractive framing for automated trading: an agent observes market features and chooses among *buy*, *sell* and *hold* actions, and a reward signal derived from profit-and-loss (PnL) is supposed to shape a policy that adapts to market regimes without hand-written entry and exit rules [1], [2], [3]. The open-source Python stack makes this easy to try—Gymnasium [4] for the environment interface, Stable-Baselines3 [5] for the algorithms, NumPy [6] and pandas [7] for feature engineering, and TA-Lib [8] for a subset of standard technical indicators.

In practice, it is easy to build an agent that *appears* to train for hours and then does nothing at all. Over roughly 75 experiments on USD/JPY 5-minute data, our DQN and PPO runs repeatedly ended with policies that completed zero trades on the held-out period. The poster framed this as a reward-design trap: every completed trade pays a transaction cost, holding costs nothing, and "never trade" can therefore become an attractive policy.

This paper expands the content presented as a virtual poster at SciPy 2026: the zero-trade baseline and a decoupled architecture in which the RL model only emits a directional score while a deterministic Python execution engine owns all position management. On the same data and split, the decoupled system trades (2,625 completed trades) where the direct agent does not (0 trades).

Published Jul 13, 2026

Correspondence to
Sho Tanaka
sho.tanaka@snowflake.com

Open Access

Copyright © 2026 Tanaka. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

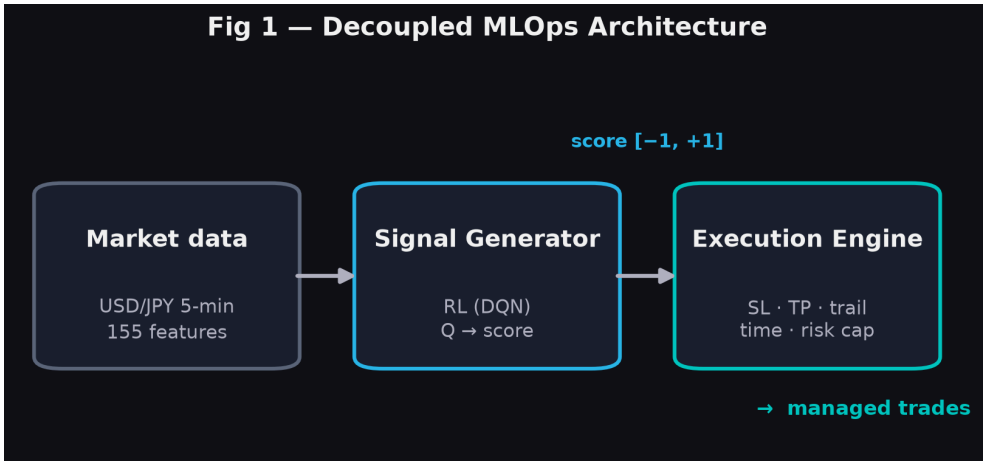


Figure 1. *The decoupled architecture. Market features are passed to the DQN signal generator, and only its directional score crosses the boundary to the deterministic execution engine, which owns exits and risk controls.*

Our contributions are:

- A reproducible zero-trade baseline on public-format FX data with a Stable-Baselines3 DQN, and the reward variants we tried before abandoning reward shaping (Section Experimental Setup and the Zero-Trade Baseline).
- A decoupled signal/execution architecture with concrete Python patterns, deterministic risk controls and checkpoint selection with `EvalCallback` (Sections Architecture: Decoupled Signal Generation and Execution and Results Presented at the Poster).
- A same-data comparison showing 0 completed trades for the direct baseline and 2,625 fully managed trades for the decoupled system.

We make no claim of profitability. All PnL figures are backtests on a single currency pair and are reported to characterise *behavior*, not returns.

2. Experimental Setup and the Zero-Trade Baseline

2.1. Data and features

All experiments use USD/JPY 5-minute OHLC bars. The poster experiments use the period 2025-01-10 through the last available bar on 2025-12-19 (70,555 bars after feature warm-up). The poster reports the calendar endpoint as 2025-12-20. We use a chronological 70/30 train/test split (49,388 / 21,167 bars). The 21,167 test bars yield 21,116 evaluable steps after constructing 50-bar observations and next-bar transitions. A `FeatureEngineer` computes 155 technical features per bar (moving averages, RSI, MACD, ATR, Bollinger bands, returns at several horizons, and so on). TA-Lib is a Python wrapper around the widely used open-source TA-Lib C library, which implements over 150 standard technical analysis indicators; it provides a subset of the indicators used here, and the remaining custom and return-based features are implemented with NumPy and pandas. The observation comprises a 50-bar window, flattened to a $50 \times 155 = 7,750$-dimensional vector.

2.2. Environment and agent

The environment is a Gymnasium `Env` with a discrete action space $\{\mathrm{HOLD}, \mathrm{BUY}, \mathrm{SELL}\}$. BUY opens or holds a long position, SELL opens or holds a short position, and switching direction closes the existing position first. A transaction cost of 1 pip (equal to 0.0001 in this environment's price units) is charged each time a position is opened or closed, so a

completed round-trip pays 2 pips. Episodes are 1,000 bars long with random start points. The reward at each step is

$$r_t = 100 \cdot \left(w_u \, \Delta\mathrm{PnL}_t^{\mathrm{unrealized}} + \Delta\mathrm{PnL}_t^{\mathrm{realized}} + b_{\mathrm{complete}} \, 1[\text{trade closed}] + b_{\mathrm{profit}} \, 1[\text{trade closed with profit}] \right) \tag{1}$$

where w_u, b_{complete} and b_{profit} are the reward design knobs we varied. Writing p_t for the close of bar t and $\pi_t \in \{-1, 0, +1\}$ for the position after the action at step t (short, flat, long), the unrealized term is the one-bar fractional price change signed by the open position, $\Delta\mathrm{PnL}_t^{\mathrm{unrealized}} = \pi_t \, (p_{t+1} - p_t)/p_t$ (zero when flat), and the realized term is zero except on a step that closes a position opened at price p_{entry} with direction π, when $\Delta\mathrm{PnL}_t^{\mathrm{realized}} = \pi \, (p_t - p_{\mathrm{entry}})/p_{\mathrm{entry}}$. The transaction cost is subtracted from the reward each time a position is opened or closed (omitted from (1) for readability). The agent is a Stable-Baselines3 `DQN` [9] with an MLP policy (`net_arch=[512, 512, 512, 256]`), learning rate 10^{-4}, replay buffer 200k, batch size 512, $\gamma = 0.99$, ε-greedy exploration annealed over the first 15% of training to a floor of 0.05, 100k training steps, CPU only. Evaluation is deterministic (greedy argmax) over the full test period.

2.3. Reward variants tried before the poster

Table 1 summarises the reward designs explored during development, labeled as on the poster, and Figure 2 visualises their trade counts. The counts are from the original single-seed development runs: each configuration was trained exactly once, with one random-number seed governing network initialization, exploration and episode start points. Because run-to-run variability under different seeds was not measured, the counts are indicative only.

Table 1. *Reward designs explored during development and their test-set trade counts (single-seed development runs).*

Label	Reward configuration	Completed trades	Observed behaviour
Buy & Hold	unrealized PnL rewarded ($w_u = 1$)	24	Enters long early, rarely exits
Penalty Hell	hold penalty added	58	Trades, but erratically
Zero-Trade Collapse	$w_u = 0$, no bonuses, 1 pip cost	**0**	Single constant action
Trade Bonus	$b_{\mathrm{complete}} > 0$, $b_{\mathrm{profit}} > 0$	84	Trades, unstable across runs

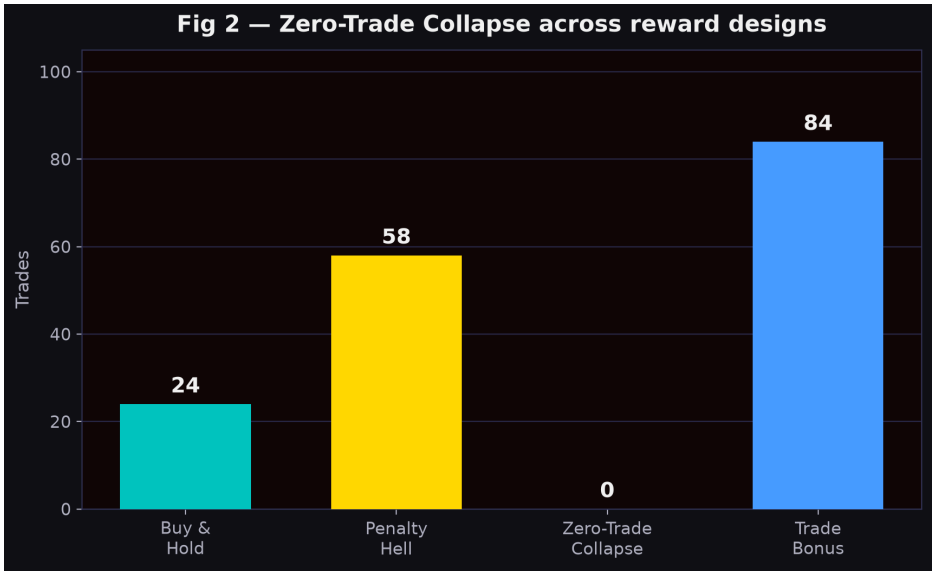


Figure 2. *Completed test-set trades for the four reward designs in Table 1. These counts come from single-seed development runs and show that some reward variants induced trading, but not that they were stable or profitable.*

The "Zero-Trade Collapse" configuration ($w_u = 0$, $b_{\text{complete}} = b_{\text{profit}} = 0$) is the baseline used throughout the rest of the paper. The configuration emits BUY on all 21,116 test steps and completes 0 trades, for a realized return of 0.00% when reproduced on the 2025 period with the aforementioned settings. A policy that emits BUY forever opens one position on the first bar and never closes it, so it registers as zero *completed* trades and zero *realized* PnL.

2.4. Why reward shaping was abandoned

Each reward variant that produces trades does so at the price of a new hyperparameter (b_{complete}, b_{profit}, the hold penalty) whose value is tuned to the training period and does not transfer. The reward is asked to simultaneously encourage *good* trades, discourage *bad* ones, and define what "good" means, and every adjustment moves the equilibrium—by which we mean the stable policy that training settles into and no longer improves away from under a given reward design—rather than removing the degenerate one. Each row of Table 1 (buy-and-hold, erratic trading, a single constant action) is such an equilibrium. After roughly 75 such runs, we stopped editing the reward and changed the system boundary instead.

3. Architecture: Decoupled Signal Generation and Execution

3.1. Design

The monolithic agent combines directional prediction and position management in a single discrete action. "Decoupling" here means splitting the system at exactly that point: the learned model and the hand-written execution logic become two independent components whose only interface is a single scalar score, so that either side can be tested, inspected or replaced without touching the other. The decoupled system assigns these responsibilities to separate components:

- **Directional scoring** is handled by the RL model, which emits a continuous score $s_t \in [-1, +1]$.
- **Position management** is handled by a deterministic Python execution engine with no learned parameters.

The high-level data flow is shown in Figure 1.

The model is trained in a *market-only* variant of the environment whose observation excludes the agent's own position state and whose reward (`direction_reward_weight = 1.0`, with small completion and profit bonuses of 5×10^{-4} and 3×10^{-4}) rewards calling the next bar's direction correctly. At inference the model is never asked to act; its Q-values are read off and converted to a score by a `DQNScoreSignalGenerator`:

```
class DQNScoreSignalGenerator(SignalGenerator):
    HOLD, BUY, SELL = 0, 1, 2

    def generate(self, observation, **_):
        obs = torch.as_tensor(observation[None], device=self.model.device)
        with torch.no_grad():
            q = self.model.q_net(obs)[0].cpu().numpy()
        score = float(np.tanh((q[self.BUY] - q[self.SELL]) / self.temperature))
        return TradingSignal(score=score, confidence=abs(score))
```

The execution engine consumes the score through a configuration object that states every rule explicitly. The configuration used for all decoupled runs in this paper is:

```
ExecutionConfig(
    signal=SignalThresholdConfig(entry_threshold=0.2, exit_threshold=0.1,
                                 reversal_threshold=0.5),
    stop_loss=StopLossConfig(type=StopLossType.ATR_BASED, atr_multiplier=1.0),
    take_profit=TakeProfitConfig(type=TakeProfitType.RISK_REWARD,
                                 risk_reward_ratio=2.5),
    trailing_stop=TrailingStopConfig(type=TrailingStopType.ATR_BASED,
                                     activation_profit_pips=5.0,
                                     atr_multiplier=0.5),
    time_exit=TimeExitConfig(enabled=True, max_bars_in_trade=48),
)
```

A score with $|\ s_t\ | \geq 0.2$ opens a position in the direction of the sign; the engine then attaches a stop-loss one ATR away, a take-profit at 2.5 times the stop distance, a trailing stop that activates after 5 pips of profit, and a hard time exit after 48 bars (four hours). A `RiskManager` enforces daily loss and drawdown caps, and open positions are closed before the weekend.

3.2. Core classes

The execution layer is organised around a small number of plain dataclasses and one stateful manager. The pattern is the one named in the original poster abstract (`Position` / `Order` / `RiskManager` / `ExitPolicy`); in the code that produced the results, the exit policy is split into per-rule `*Config` objects applied by a `PositionManager`, and the `RiskManager` enforces `max_position_size`, `max_daily_loss` (2%) and `max_drawdown` (10%) of balance.

```
class PositionSide(Enum):
    LONG = 1
    SHORT = -1

class ExitReason(Enum):
    SIGNAL = "signal"              # score reversed or weakened
    STOP_LOSS = "stop_loss"
    TAKE_PROFIT = "take_profit"
    TRAILING_STOP = "trailing_stop"
    TIME_EXIT = "time_exit"
    WEEKEND_CLOSE = "weekend_close"

@dataclass
class Position:
    side: PositionSide
    entry_price: float
    entry_time: datetime | None = None
    entry_bar: int = 0
    size: float = 1.0
    stop_loss: float | None = None
    take_profit: float | None = None
    trailing_stop_active: bool = False
    trailing_stop_level: float | None = None
    highest_price: float | None = None   # for long trailing stops
    lowest_price: float | None = None    # for short trailing stops
    entry_atr: float | None = None
    entry_score: float = 0.0

    def get_unrealized_pnl(self, price: float) -> float:
        return (price - self.entry_price) * self.side.value * self.size

    def check_stop_loss_hit(self, low: float, high: float) -> bool: ...
    def check_take_profit_hit(self, low: float, high: float) -> bool: ...
    def check_trailing_stop_hit(self, low: float, high: float) -> bool: ...

@dataclass
class ClosedTrade:
    side: PositionSide
    entry_time: datetime
    exit_time: datetime
    entry_price: float
    exit_price: float
    pnl: float
    exit_reason: ExitReason
```

Because none of these objects know anything about the model, each rule can be unit-tested against a synthetic price path: feed a `Position` a sequence of bar highs and lows and assert which `check_*_hit` method fires and on which bar. The `ExecutionEngine.run_backtest()` loop is the only place the two layers meet:

```
for obs, bar in zip(observations, bars):
    signal = self.signal_generator.generate(obs)        # model → score
    self.position_manager.update(bar)                   # exits: SL/TP/trail/time
    if self.position_manager.flat and self.risk.allows(bar.time):
        if abs(signal.score) >= cfg.signal.entry_threshold:
            self.position_manager.open(signal, bar)     # entry
```

3.3. Checkpoint selection with `EvalCallback`

Saving only the final training state can discard an earlier, better checkpoint. The poster pipeline therefore uses an `EvalCallback` with a separate evaluation environment and loads the best saved checkpoint for the decoupled backtest:

```python
from stable_baselines3.common.callbacks import EvalCallback
from stable_baselines3.common.vec_env import DummyVecEnv, SubprocVecEnv

train_env = SubprocVecEnv([make_env() for _ in range(8)])
eval_env  = DummyVecEnv([make_env()])           # separate, single env
callback  = EvalCallback(eval_env, best_model_save_path="best_model",
                         eval_freq=5_000,
                         n_eval_episodes=5, deterministic=True)
model = DQN("MlpPolicy", train_env, **DQN_KWARGS)
model.learn(100_000, callback=callback)
best = DQN.load("best_model/best_model")        # not `model`
```

The 2,625-trade result reported on the poster uses this selected checkpoint.

4. Results Presented at the Poster

Figure 3 reproduces the poster as presented at the SciPy 2026 virtual poster session on 2026-07-15. The systems share the 2025 data, chronological 70/30 split, 155 features and 100k training budget. The decoupled system also uses the market-only observation and direction-oriented reward described above, `EvalCallback` checkpoint selection, and the deterministic execution configuration. The results compare these two complete implementations on the same held-out period.

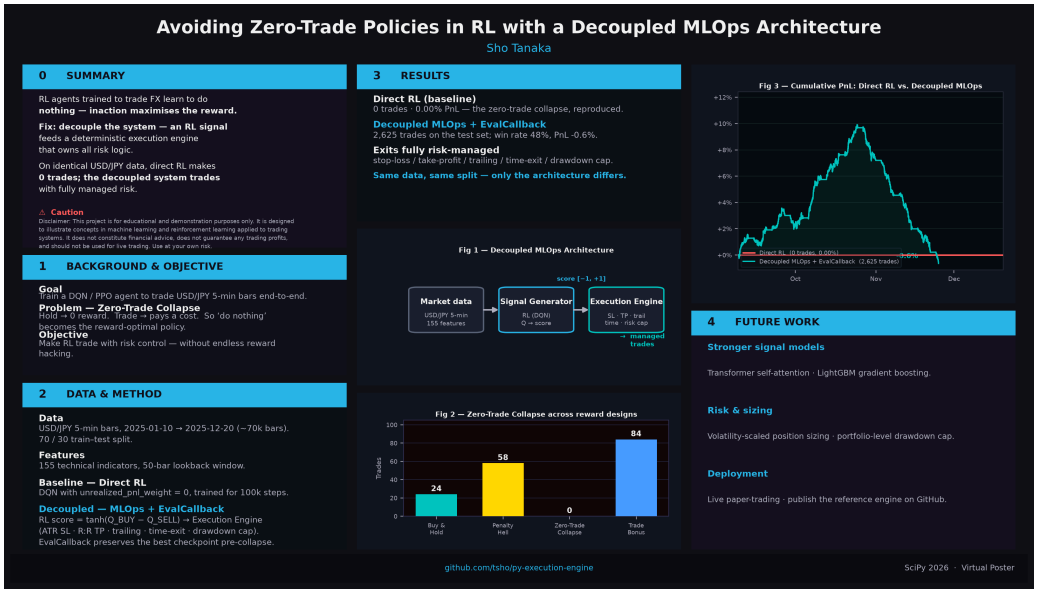


Figure 3. *The SciPy 2026 virtual poster as presented. Panel 3 and poster Fig. 3 (Figure 4) report the 2,625-trade decoupled result; poster Fig. 2 (Figure 2) shows the trade counts of the reward variants in Table 1.*

Table 2. *Test-set behaviour on USD/JPY 5-min, 2025-09-08 to 2025-12-19 (21,167 bars; 21,116 evaluable steps).*

System	Trades	Win rate	Realized PnL
Direct RL (baseline DQN)	0	—	0.00%
Decoupled + `EvalCallback`	**2,625**	**48.4%**	**-0.64%**

The decoupled system with `EvalCallback` completed 2,625 trades, with each position governed by the stop-loss, take-profit, trailing-stop, time-exit and risk rules in the execution layer. Figure 4 shows the cumulative realized PnL over the test period: the baseline is a flat line at zero, while the decoupled system rose to $+9.9\%$ in late October before giving it back to finish at -0.64%.

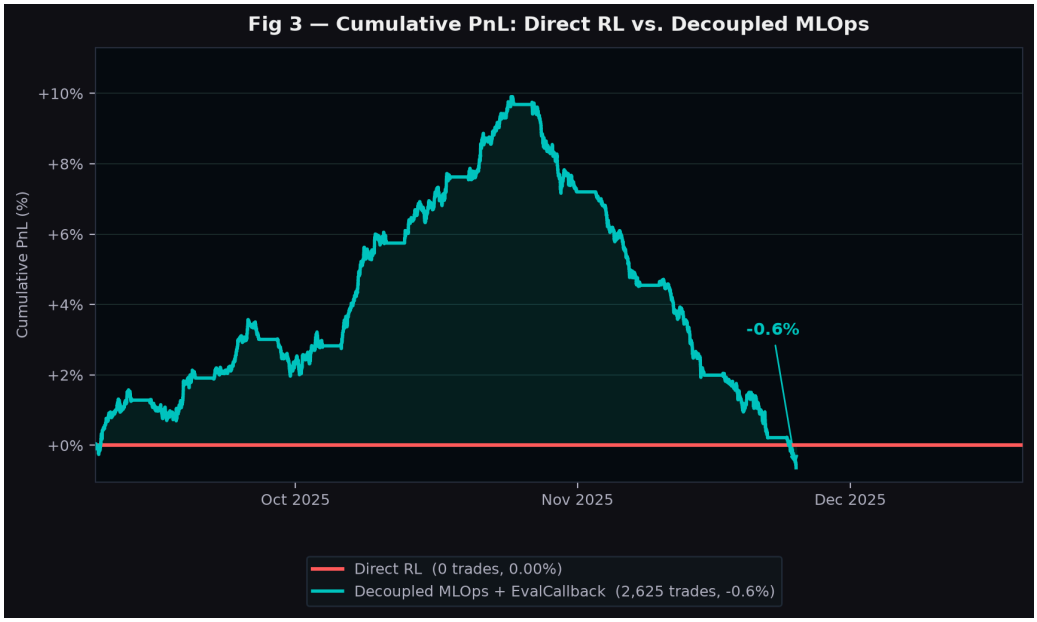


Figure 4. *Cumulative realized PnL on the 2025 test period. The direct-RL baseline completes no trades and is a flat line at 0%. The decoupled system with* `EvalCallback` *completes 2,625 trades, peaks near +10% and finishes at -0.6%. The curve characterises behaviour, not a return expectation.*

In sum, on the same data and training budget, moving position management out of the learned policy and into deterministic execution code changed the system's behaviour from completing no trades at all to completing 2,625 trades, each opened and closed by an explicit, testable rule. The near-flat final PnL underlines that the contribution is behavioural and operational—the architecture guarantees managed trading activity—not evidence that the signal itself is profitable.

5. Limitations

All results are on one currency pair and one chronological split. The reward-variant counts come from single-seed development runs and are indicative only. The direct and decoupled systems differ in observation design, reward, checkpoint selection and execution, so the comparison characterises the two complete implementations rather than isolating one architectural variable. The decoupled-system PnL is a single-seed backtest whose only trading friction is a fixed 1-pip cost deducted per completed trade. Order fills are idealized: entries fill at the bar close, and stop-loss, take-profit and trailing-stop exits fill exactly at their trigger levels. Live execution adds a variable bid-ask spread and slippage—fills worse than the trigger price when the market gaps or moves quickly—none of which is modeled here, so the reported PnL is optimistic and should not be read as an estimate of future returns.

6. Conclusion

This paper presented a decoupled response to the zero-trade failure mode observed in direct RL trading. On the same USD/JPY data and chronological split, the direct DQN completed 0 trades, whereas the decoupled signal/execution system completed 2,625 trades with every position governed by explicit risk and exit rules.

The contribution is architectural and operational: the model produces a directional score, while deterministic Python code owns position management. This boundary makes the execution rules independently testable and prevents the model from being solely respon-

sible for exits. The reported backtest does not establish profitability, but it demonstrates a practical structure for building and evaluating RL-assisted trading systems.

7. Future Work

The poster identified three directions for extending the system:

1. **Stronger signal models.** Evaluate Transformer-based models with self-attention and gradient-boosted models such as LightGBM.
2. **Risk and position sizing.** Add volatility-scaled position sizing and a portfolio-level drawdown cap.
3. **Deployment.** Validate the complete pipeline through live paper trading and publish the reference execution engine.

8. Acknowledgements and Disclosure

Portions of this work were assisted using a generative AI tool (Snowflake CoCo, Claude, Codex). The tool was used for generating and refactoring experiment and execution-engine code, for drafting and revising this manuscript, and for producing figures from the recorded result files. All experiments were run, and all outputs reviewed, verified and revised, by the author, who takes full responsibility for the accuracy and integrity of the final content.

This project is for educational and demonstration purposes only. It does not constitute financial advice, does not guarantee any trading profits, and should not be used for live trading.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[2] J. Moody, L. Wu, Y. Liao, and M. Saffell, "Performance functions and reinforcement learning for trading systems and portfolios," *Journal of Forecasting*, vol. 17, no. 5–6, pp. 441–470, 1998, [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/%28SICI%291099-131X%281998090%2917%3A5/6%3C441%3A%3AAID-FOR707%3E3.0.CO%3B2-%23

[3] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, "Deep Direct Reinforcement Learning for Financial Signal Representation and Trading," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 3, pp. 653–664, 2017, doi: 10.1109/TNNLS.2016.2522401.

[4] Farama Foundation, "Gymnasium." [Online]. Available: https://gymnasium.farama.org/

[5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dorber, "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021, [Online]. Available: http://jmlr.org/papers/v22/20-1364.html

[6] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[7] The Pandas Development Team, "pandas-dev/pandas: Pandas." Zenodo, 2020. doi: 10.5281/zenodo.3509134.

[8] TA-Lib Project, "TA-Lib: Technical Analysis Library." [Online]. Available: https://ta-lib.org/

[9] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

Open Access | https://doi.org/10.25080/fwvt2525

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

ArcadeDB in Python: An In-Process Multi-Model Database

Taewoon Kim[1]

[1]HumemAI

Abstract

Some Python workflows need transactional reads and writes over documents, traversals over a graph, and similarity search over vector embeddings. In practice these are split across separate systems (a document store, a graph engine, and a vector index), and the data is copied between them and kept consistent by application code. ArcadeDB is an open-source multi-model database engine, written in Java, that supports all three access patterns and can be embedded in a host process. We present `arcadedb-embedded`, a Python package that runs ArcadeDB inside the Python process via JPype, using a Java Runtime Environment bundled in the wheel. One `pip install` gives a database with documents, a property graph, and vector search, and needs no Java installation. Query results can be returned as Python lists, NumPy arrays, pandas DataFrames, or Arrow tables. We demonstrate a hybrid retrieval workflow that chains vector search, SQL filtering, and graph traversal over one dataset. We benchmark `arcadedb-embedded` against SQLite, DuckDB, LadybugDB, and Chroma, and report its costs.

Keywords multi-model database, embedded database, graph database, vector search, Python bindings, JPype

1. Introduction

A question-answering system over a Q&A site stores posts and their metadata as documents (queried with SQL, much like rows of a table, but without a fixed schema), models who-answered-what or what-cites-what as a graph, and retrieves passages by semantic similarity over vectors, all over the same corpus. Python is popular for these workloads, but no single Python-embeddable data store covers all three access patterns. The usual response is to assemble a stack of SQLite or DuckDB for documents, NetworkX or an embedded graph database such as LadybugDB for the graph, and a vector index such as FAISS, hnswlib, or Chroma for vectors [1], [2], [3], [4], [5], [6], [7].

A stack of specialists is a reasonable default, and D. Datta [8] makes the case for assembling one from interoperable Python libraries. It has costs when all three access patterns apply to the same data. Each system holds its own copy, with its own consistency model, so every write has to reach all of them. A query that spans models passes identifiers from one library to another. Reproducing an experiment means pinning several packages instead of one.

This paper describes the alternative, one embedded multi-model database that holds documents, a property graph, and vector indexes. That database is ArcadeDB [9], an Apache-2.0 Java application that keeps all three models in one store (referred to as "the engine" throughout the rest of the paper). Documents are queried with its SQL dialect, the graph with OpenCypher, and vectors through a graph index of the Hierarchical Navigable Small World (HNSW) family. Transactional (online transaction processing, OLTP) and analytical (online analytical processing, OLAP) queries run over the same data. ArcadeDB is ACID (atomic, consistent, isolated, durable). Its Graph Analytical Views (GAV), an ArcadeDB-specific structure, speed up graph analytics while the base graph stays transactional.

Published Jul 13, 2026

Correspondence to
Taewoon Kim
taewoon@humem.ai

Open Access

Copyright © 2026 Kim. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

`arcadedb-embedded` binds the Java engine through JPype [10], [11]. The paper contains benchmarks comparing `arcadedb-embedded` to the specialists.

Concretely, this paper contributes:

1. **An in-process multi-model database for Python.** Each wheel bundles a Java Runtime Environment (JRE) for its platform, from which the binding starts a Java Virtual Machine (JVM) inside the Python process via JPype. The binding covers JVM lifecycle, transactions, and NumPy/pandas/Arrow interop. Wheels exist for four platforms and five Python versions, and the user never installs Java.
2. **A hybrid workflow.** A single retrieval pipeline composes vector search, SQL filtering and graph traversal over one dataset in one process. A capability matrix shows that no single Python-embeddable alternative expresses this composition.
3. **A benchmark against embedded specialists.** A reproducible benchmark suite compares the engine, from Python, against embedded specialists on a real-world Q&A corpus, across transactional, analytical, and vector workloads. We report which models each side wins and what the unified engine costs in memory, startup, latency tails, disk, and result transport into Python.

2. Background

Embedded versus server databases. A server database, such as PostgreSQL with pgvector [12], Neo4j [13], or Milvus [14], runs as its own process, and Python reaches it over a socket. An embedded database is a full database engine delivered as a library. It runs inside the host process and persists to ordinary files, which fits local, single-machine work. This paper is about that setting, so from here on it compares `arcadedb-embedded` only with other Python-embeddable stores.

Embedded data stores in Python. SQLite is the standard choice for documents and transactions [2]. DuckDB brought a columnar, analytics-first embedded engine to Python [1], [15]. For graphs, NetworkX [3] provides in-memory graph analysis. LadybugDB [4] is an embedded graph database with a Cypher interface and a vector index. For vectors, hnswlib provides the reference HNSW implementation [6], [16], FAISS provides exact and approximate similarity search [5], and Chroma packages HNSW behind a Python API [7]. None of them provides all three models in one embedded engine (Table 1).

Table 1. *Capability matrix for the Python-embeddable data stores compared here; all five run in the host process (✓ native, ∼ partial or through an official extension, × absent). "All three" means documents with SQL, a property graph, and vector search. DuckDB's HNSW index is an experimental core extension; ArcadeDB's index is JVector [17] (Vamana graphs [18] with an HNSW-style hierarchy). SQLite has vector search only through extensions, and neither of them is a graph index. SQLite and DuckDB are relational, and LadybugDB is a graph database whose nodes live in typed node tables. Chroma stores each entry as a text document with a metadata dict. ArcadeDB's SQL has `GROUP BY` and aggregates but no joins and no `OVER/PARTITION BY` window clause.*

Capability	SQLite	DuckDB	LadybugDB	Chroma	ArcadeDB
Documents	✓	✓	∼	∼	✓
Transactional SQL (OLTP)	✓	∼	×	×	✓
Analytical SQL (OLAP)	∼	✓	×	×	∼
Property graph + traversal	×	×	✓	×	✓
Cypher	×	×	✓	×	✓
Graph approximate-nearest-neighbor index (HNSW family)	×	∼	✓	✓	✓
All three in one engine	×	×	×	×	✓

3. Architecture

`arcadedb-embedded` runs the ArcadeDB engine *inside* the Python interpreter process. Opening a database is a function call, and queries are method calls that return Python objects.

In-process JVM via JPype. On first use, the binding starts an embedded JVM inside the Python process using JPype [10]. JVM settings like heap size are configured directly from Python, and CPython and the JVM share a single address space, so data can be passed around without needing network calls or inter-process serialization. The engine's Java objects are exposed through a Pythonic layer, so applications never need to interact with JPype directly.

```
import arcadedb_embedded as arcadedb

# One in-process engine; persists to a local directory.
with arcadedb.create_database("my_db", jvm_kwargs={"heap_size": "4g"}) as db:
    db.command("sql", "CREATE DOCUMENT TYPE Post")
    db.command("sql", "CREATE PROPERTY Post.id LONG")
    db.command("sql", "CREATE INDEX ON Post (id) UNIQUE_HASH")
    with db.transaction():
        db.command("sql", "INSERT INTO Post SET id = :id, score = :s",
                   {"id": 1, "s": 42})
    rows = db.query("sql", "SELECT id, score FROM Post WHERE id = 1").to_list()
```

One object for all three models. The same `db` object queries documents (SQL), the property graph (OpenCypher), and vectors (a graph index, searched with the `vectorNeighbors()` function). The first argument of `query()` selects the language. All three models share one transaction scope and one set of identifiers, so the hybrid workflow below can pass results from one model into the next without leaving the process.

NumPy, pandas, and Arrow. Embeddings are passed as `float32` NumPy arrays through `arcadedb.to_java_float_array(vec)`, which hands the buffer to the engine without a Python-side element-by-element copy. Results come back through four methods on the result set. `to_list()` gives a list of dicts, `to_dataframe()` a `pandas.DataFrame`, `to_columns()` a dict of NumPy arrays, and `to_arrow()` a `pyarrow.Table`.

```
# db and the Post type as above; a result set is read once, so each output format
# needs its own query
df = db.query("sql", "SELECT id, score FROM Post").to_dataframe()   # pandas.DataFrame
cols = db.query("sql", "SELECT id, score FROM Post").to_columns()   # {"id": ndarray, ...}
table = db.query("sql", "SELECT id, score FROM Post").to_arrow()    # pyarrow.Table;
                                                    # nullable INTEGER stays int64
```

pandas and pyarrow are optional dependencies, imported only when the corresponding method is called. NumPy has no missing value for integers, so with `to_columns()` a nullable `INTEGER` column becomes `float64` with `NaN` holes, which loses the type and any precision above 2^{53}. `to_arrow()` reads the same buffer and keeps its null bitmap, so the column stays `int64`. Both read one packed columnar buffer from the engine. Section 6 measures the four export paths.

Cross-platform packaging. Bringing an engine written in another language into Python usually means compiling a native C/C++ extension, with tooling such as scikit-build-core [19]. `arcadedb-embedded` ships a JRE instead. Java bytecode is platform-agnostic, so the engine's `.jar` files are identical everywhere and the only platform-specific part is the JRE, which is trimmed with `jlink` [20] to the modules the engine needs. The build produces wheels for Linux x86-64, Linux ARM64, macOS Apple Silicon, and Windows x86-64, for Python 3.10 to 3.14, twenty wheels in total. `pip install arcadedb-embedded` needs no Java installation and no `JAVA_HOME` (Figure 1). Each wheel is about 70 MB, roughly four times a NumPy wheel, downloaded once at install.

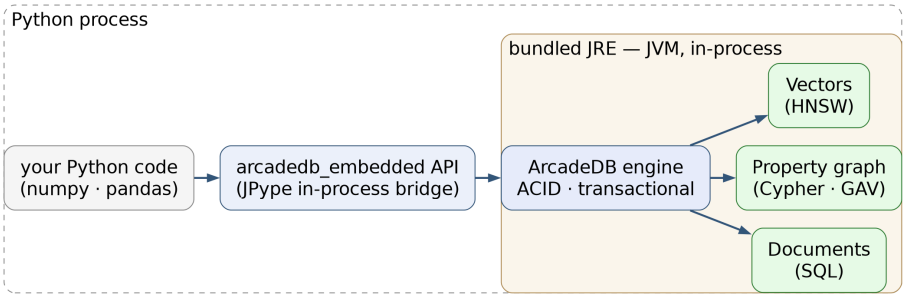


Figure 1. *The `arcadedb-embedded` architecture. A single Python process hosts a JVM (via JPype) running the ArcadeDB engine over one local, file-backed database. The Python API exposes documents/SQL, a property graph (OpenCypher), and vector search through one `db` object, with NumPy/pandas interop. The wheel bundles a per-platform JRE, so installation is a plain `pip install` and the user never installs Java.*

4. Documents, graph, and vectors in one process

This section first shows how data gets in, then each model with its own query language, and finally one workflow that uses all three. The data is the Cross Validated (`stats.stackexchange.com`) Stack Exchange dump, a statistics and machine-learning Q&A site (CC BY-SA [21]). Questions and answers are documents, who-asked and who-answered form a graph, and each question text has an embedding. The benchmark suite linked at the end of the paper rebuilds this corpus with one command and runs the whole section end to end.

4.1. Ingestion

Data enters all three models as Python objects: `insert_many()` takes a list of dicts for documents, `graph_batch()` takes lists of dicts and ids for vertices and edges, and embeddings go in as `float32` arrays. The examples below start from the Parquet files the benchmark suite prepares and read them with pandas, selecting only the columns a type needs, because ArcadeDB has no Parquet or Arrow reader. Loading costs more than in the specialists. On the Cross Validated corpus, ArcadeDB needs about 17.5 s for the 426k posts and about 27 s for the graph. SQLite and DuckDB read the same Parquet files natively in about 0.3 s, and LadybugDB bulk-loads the graph with `COPY` in about 0.6 s (Table 2, Table 3). Most of the load time goes to the unique hash index on `id` that the point-lookup workload relies on.

4.2. Documents

ArcadeDB's documents are created and queried with SQL, so a type of documents is used much like a table in SQLite or DuckDB, and writes run inside ACID transactions. The snippet in Architecture creates a type and a `UNIQUE_HASH` index for point lookups, then runs a parameterized `INSERT` inside `db.transaction()`. Documents can be schema-flexible or have typed properties for indexing, and both kinds are stored in the same database as the graph and vectors below.

Loading a corpus row by row pays a Python-to-Java call per row, so bulk ingestion goes through `insert_many()`, which serializes a batch to one string and loops over it on the Java side. Only the columns the type needs are read into Python:

```
import pandas as pd

data = "datasets/prepared/stats.stackexchange.com"   # written by the benchmark suite's prepare step

# read only the columns the type needs; the post body stays in the file
posts = pd.read_parquet(f"{data}/posts.parquet", columns=["id", "score", "title"])
db.insert_many("Post", posts.to_dict("records"), commit_every=10_000)
```

4.3. Property graph

Vertex and edge types are declared with SQL. OpenCypher can create them implicitly on first write, as in Neo4j, and can add indexes and uniqueness constraints, but the typed float-array property and the vector index below need SQL, so we declare everything there. Bulk loading goes through a `graph_batch()` context manager. `create_vertices()` and `new_edges()` each cross into the JVM once for a whole list and the batch commits on flush, instead of one crossing and one transaction per element. `batch_size` is the number of edges buffered before an automatic flush, and `create_vertices()` returns the ids, so edges reuse them instead of querying them back. Graph queries then use OpenCypher through the same `db` object:

```
import pandas as pd

db.command("sql", "CREATE VERTEX TYPE Question")
db.command("sql", "CREATE PROPERTY Question.embedding ARRAY_OF_FLOATS")
db.command("sql", "CREATE VERTEX TYPE Answer")
db.command("sql", "CREATE VERTEX TYPE Userx")
db.command("sql", "CREATE EDGE TYPE HAS_ANSWER")
db.command("sql", "CREATE EDGE TYPE AUTHORED_BY")

data = "datasets/prepared/stats.stackexchange.com"   # written by the benchmark suite's prepare step

# read only the columns the types need; the post body stays in the file
posts = pd.read_parquet(f"{data}/posts.parquet",
                        columns=["id", "post_type", "owner_user_id", "score", "title"])
questions = posts[posts.post_type == 1]               # embeddings has one row per question
answers = posts[posts.post_type == 2].dropna(subset=["owner_user_id"])
users = pd.read_parquet(f"{data}/users.parquet", columns=["id", "reputation"])
links = pd.read_parquet(f"{data}/edges_answers.parquet")   # question_id, answer_id

with db.graph_batch(batch_size=100_000) as batch:
    # create_vertices() returns one RID per row, so edges reuse them instead of querying
    qrid = dict(zip(questions.id, batch.create_vertices("Question", [
        {"id": i, "score": s, "title": t, "embedding": e.tolist()}
        for i, s, t, e in zip(questions.id, questions.score, questions.title, embeddings)])))
    arid = dict(zip(answers.id, batch.create_vertices(
        "Answer", answers[["id", "score"]].to_dict("records"))))
    urid = dict(zip(users.id, batch.create_vertices("Userx", users.to_dict("records"))))
    batch.new_edges([qrid[q] for q in links.question_id], "HAS_ANSWER",
                    [arid[a] for a in links.answer_id])       # one call per edge type
    batch.new_edges([arid[a] for a in answers.id], "AUTHORED_BY",
                    [urid[int(u)] for u in answers.owner_user_id])

db.query("opencypher",
    "MATCH (q:Question)-[:HAS_ANSWER]->(a:Answer)-[:AUTHORED_BY]->(u:Userx) "
    "WHERE q.id = $qid RETURN a.id, a.score, u.reputation "
    "ORDER BY a.score DESC",
    {"qid": 3}).to_list()          # Cypher binds $name from a dict; SQL binds :name or ?
```

For graph *analytics* over the same data, a GAV is created (`CREATE GRAPH ANALYTICAL VIEW ... VERTEX TYPES (...) EDGE TYPES (...)`). The build runs in the background, so the statement returns before the GAV is usable and `SELECT status FROM schema:graphAnalyticalViews` is read until it reports `READY`. The GAV is an in-memory copy of the selected vertex and edge types, laid out for fast traversal, and it is kept up to date as the graph changes.

4.4. Vectors

Embeddings are stored as a float-array property and indexed with an `LSM_VECTOR` index. The index is a JVector graph index [17], built from Vamana graphs [18] in a multi-layer hierarchy comparable to HNSW. Search is a SQL function over the index. Parameters (`dimensions`, `similarity`, `maxConnections`, `beamWidth` = `ef_construction`) are set on the index, and the query supplies `ef_search`. `maxConnections` is the per-layer out-degree, while hnswlib's M is half of it, because hnswlib builds its base layer at degree $2M$ and upper layers at M. Setting both to the same number compares a half-degree ArcadeDB graph against a full-degree hnswlib one, so we set `maxConnections` $= 2M = 32$:

```
import numpy as np

db.command("sql", "CREATE INDEX ON Question (embedding) LSM_VECTOR "
    'METADATA { "dimensions": 384, "similarity": "COSINE", '
    '"maxConnections": 32, "beamWidth": 100 }')

# embeddings: one 384-d float32 row per Question, built by the benchmark suite linked
# at the end of the paper. vectorNeighbors(index, query vector, k, ef_search)
seed = np.asarray(embeddings[0], dtype=np.float32)
hits = db.query("sql",
    "SELECT id, score, distance FROM "
    "(SELECT expand(vectorNeighbors(?, ?, ?, ?))) ORDER BY distance",
    "Question[embedding]", arcadedb.to_java_float_array(seed), 200, 100).to_list()
```

4.5. The hybrid workflow

Given a popular "seed" question, we answer one query: *find questions similar to this one, keep the well-scored ones, and return their best answers with the reputation of who wrote them.* The steps are a vector search, a SQL filter, and a Cypher traversal (Figure 2). They pass 200 vector candidates to the SQL filter, 50 that pass the filter to the graph traversal, and return the top 10 answers. The first two steps are kept apart so that each stage can be timed separately. An application does not have to split them, because `vectorNeighbors()` returns whole documents, so the filter can sit on the vector search in one statement, `SELECT id, title, score FROM (SELECT expand(vectorNeighbors(...))) WHERE score >= 1`, and only the graph traversal then needs a second call.

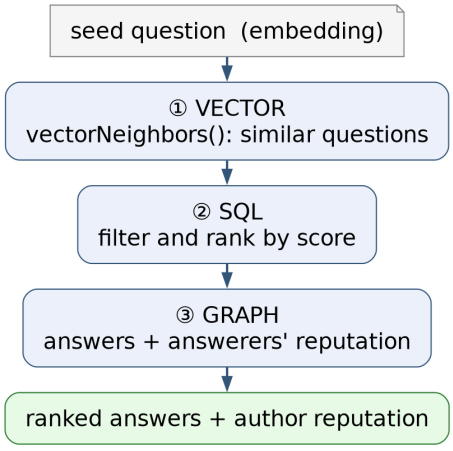


Figure 2. *The hybrid retrieval workflow. A seed question's embedding drives a vector search, the resulting ids flow into a SQL filter, and the surviving ids flow into a graph traversal that reaches answers and their authors. All three steps run against one database, with identifiers passing directly between steps.*

```
# 1) VECTOR: questions semantically similar to a seed
cands = db.query("sql",
    "SELECT id, score, distance FROM "
    "(SELECT expand(vectorNeighbors(?, ?, ?, ?))) ORDER BY distance",
    "Question[embedding]", arcadedb.to_java_float_array(seed), 200, 100).to_list()

# 2) SQL: keep well-scored candidates (ids flow straight from step 1)
ids = [int(c["id"]) for c in cands]
filt = db.query("sql",
    "SELECT id, title, score FROM Question WHERE id IN :ids "
    "AND score >= 1 ORDER BY score DESC LIMIT 50", {"ids": ids}).to_list()

# 3) CYPHER: traverse to answers + answerers' reputation
fids = [int(r["id"]) for r in filt]
hits = db.query("opencypher",
    "MATCH (q:Question)-[:HAS_ANSWER]->(ans:Answer)"
    "-[:AUTHORED_BY]->(usr:Userx) "
    "WHERE q.id IN $fids "
    "RETURN q.id AS qid, ans.id AS aid, ans.score AS ascore, "
    "usr.reputation AS rep ORDER BY ascore DESC LIMIT 10", {"fids": fids}).to_list()
```

The workflow runs on the part of the Cross Validated corpus its query needs: the 214k questions, their 209k answers, and the 108k users who wrote those answers. The benchmarks below load the whole corpus. The workflow takes a median of 12.9 ms over 20 warm repetitions: 5.9 ms for the vector search, 5.0 ms for the SQL filter, and 2.0 ms for the Cypher traversal. Timings were measured on the same system as the comparison tables below.[1]

The Cypher step runs over a GAV, created as in the Property graph section, since the traversal is analytical. The graph benchmark below shows how much the GAV speeds it up.

5. Benchmarks

Is each model competitive enough for practical use, and what does it cost? We compare `arcadedb-embedded` against widely used Python-embeddable specialists for each model: SQLite and DuckDB (documents), LadybugDB (graph), and Chroma (vectors). "ArcadeDB" in the tables and text below means `arcadedb-embedded`, measured from Python.

Protocol. Every measurement runs in its own Docker container, one container at a time, with the same CPU cores and memory cap allocated to every backend, and is repeated 5 times in fresh containers. We report the median across the 5 repetitions with the full [min–max] range, since timing distributions are right-skewed and the median is the recommended summary [22], [23]. Engines run their shipped defaults except where stated.[2] In every table, higher is better for throughput and recall, and lower is better for everything else: latency, build and load times, memory, and disk.

The data is the Cross Validated (`stats.stackexchange.com`) public data dump [21], a statistics and machine-learning Q&A corpus of about 426k posts, 346k users, and 1.2M text embeddings. Embeddings are 384-dimensional `all-MiniLM-L6-v2` vectors over each question's title and body. Vector-index parameters are matched across engines (degree as in the Vectors section, `ef_construction` $= 100$, `ef_search` $= 100$), and we report recall@10 against an exact ground truth. Graph OLAP for ArcadeDB uses a GAV. For document OLAP we first build

[1] Single host: 12th-gen Intel Core i9-12900HK (20 logical cores, eight allocated to each container), 61 GiB usable RAM, Samsung 980 PRO 2 TB NVMe SSD, Linux kernel 7.0.0 x86-64, Docker 29.5.3. Other versions: DuckDB 1.5.4, LadybugDB (`ladybug`) 0.18.1, Chroma 1.5.9, SQLite 3.46.1 as bound by Python's `sqlite3` in `python:3.12-slim`. Embeddings are computed in the binding's repository, L2-normalized, stored as `float32`.

[2] Each container is pinned to the same eight cores. SQLite runs WAL with `synchronous=NORMAL`, which its documentation recommends. ArcadeDB's JVM heap is capped at 16 GiB, because the default 4 GiB does not complete the vector build; it is a ceiling, not a reservation, and the document workload peaks below 2 GiB under it. Peak memory is the container's peak. Every result row records the versions and limits it ran under.

a `NOTUNIQUE` index on each column the suite groups or filters by (`post_type`, `owner_user_id`, `score`), so the engine answers from an index instead of scanning the type; the index build is timed separately and excluded from query times. ArcadeDB and `arcadedb-embedded` are version 26.8.1 throughout.[3]

Workloads. Each model has a transactional (OLTP) and an analytical (OLAP) workload. OLTP is a mix of point operations issued by id, reported as throughput (ops/s). OLAP is a fixed suite of analytical queries, reported as the time to run the whole suite (ms).[4] The vector model instead runs 1,000 held-out nearest-neighbor queries and reports latency and recall@10. The exact query text for every workload is in the public benchmark suite.

Documents (Table 2). This workload runs the same SQL over the same posts that SQLite and DuckDB run over a table. SQLite is much faster than ArcadeDB on point operations, DuckDB is much faster on analytical queries, and ArcadeDB peaks at about 800 MiB of memory against about 300 MiB for the other two. Transactional throughput depends mostly on how often the engine syncs to disk. SQLite and ArcadeDB both run with relaxed durability here (SQLite's recommended write-ahead log (WAL) setting, ArcadeDB's default asynchronous flush), and at that setting SQLite does about 87,000 ops/s to ArcadeDB's 6,400. DuckDB syncs on every commit and has no setting to relax that, which is why it shows 219. With both SQLite and ArcadeDB forced to sync on every commit, they meet at the disk's limit, 262 against 187 ops/s.[5] On analytical SQL DuckDB answers the suite in about 9 ms versus ArcadeDB's 1,400 ms.

Table 2. *Document model (Cross Validated corpus): SQLite, DuckDB, ArcadeDB. OLTP is a mixed point read/insert/update workload (ops/s), OLAP an analytical aggregation suite (ms). SQLite and ArcadeDB run with relaxed durability (they do not sync to disk on every commit); DuckDB syncs on every commit. Median [min–max] over 5 reps. Peak = container memory, DB = on-disk size at the end of the OLTP rep (MiB). Best value per column in bold.*

Backend	OLTP ops/s	OLAP ms	Ingest s	Peak MiB	DB MiB
SQLite	**87,150** [59,138–88,591]	292.8 [284.4–293.9]	**0.33** [0.32–0.36]	**299** [277–304]	20.1
DuckDB	219 [199–226]	**9.3** [9.3–9.6]	0.38 [0.37–0.39]	301 [296–308]	**17.3**
ArcadeDB	6,416 [6,091–6,826]	1,406.6 [1,382.1–1,432.4]	17.52 [17.15–17.95]	803 [763–846]	38.5

Graph (Table 3). ArcadeDB answers point and one-hop reads faster than LadybugDB, LadybugDB is about 13× faster on the analytical suite, and ArcadeDB's graph costs far more memory and disk. On OLTP, ArcadeDB's 6× lead in throughput comes mostly from durability settings: LadybugDB syncs on every commit and has no setting to relax that. With ArcadeDB made to sync on every commit as well, it still leads, 595 against 525 ops/s. The OLTP reads do not depend on that setting, and there ArcadeDB is ahead at the median and at the 99th percentile (p99), though its single worst one-hop read is slower (Table 5).

Table 3. *Graph model (Cross Validated corpus): LadybugDB, ArcadeDB. OLTP is neighborhood/traversal point ops (ops/s), OLAP a multi-query analytical suite (ms), Ingest the bulk load of vertices and edges (s).*

[3] Single host: 12th-gen Intel Core i9-12900HK (20 logical cores, eight allocated to each container), 61 GiB usable RAM, Samsung 980 PRO 2 TB NVMe SSD, Linux kernel 7.0.0 x86-64, Docker 29.5.3. Other versions: DuckDB 1.5.4, LadybugDB (`ladybug`) 0.18.1, Chroma 1.5.9, SQLite 3.46.1 as bound by Python's `sqlite3` in `python:3.12-slim`. Embeddings are computed in the binding's repository, L2-normalized, stored as `float32`.

[4] Document OLTP issues 5,000 operations (60% reads, 20% updates, 10% inserts, 10% deletes); graph OLTP issues 2,000 (50% point lookups, 35% one-hop traversals, 15% vertex inserts). The document OLAP suite is five `GROUP BY`/aggregate/top-N queries over the posts table; the graph suite is four traversal-and-aggregation queries (e.g. top contributors by post count, multi-hop path counts). Each query's time is the average of seven runs, and the suite time is their sum.

[5] `arcadedb.txWalFlush=2`, and SQLite at its library defaults of a rollback journal with `synchronous=FULL`. The SQLite strict figure is in the suite's append log rather than the frozen results table.

ArcadeDB OLAP uses a GAV, whose build time is shown separately. LadybugDB syncs to disk on every commit; ArcadeDB runs at its relaxed default. Median [min–max] over 5 reps. Peak = container memory, DB = on-disk size at the end of the OLTP rep (MiB). Best value per column in bold.

Backend	OLTP ops/s	OLAP ms	GAV build s	Ingest s	Peak MiB	DB MiB
LadybugDB	525 [467–532]	**65.7** [64.9–66.9]	n/a	**0.63** [0.61–0.64]	**684** [675–688]	**41.4**
ArcadeDB	**3,212** [3,034–3,629]	839.6 [794.7–875.1]	1.49 [1.39–1.77]	26.67 [26.21–26.92]	11,037 [10,561–11,190]	1,840.8

On graph analytics LadybugDB wins, 66 ms against 840 ms. The GAV is what gets ArcadeDB to 840 ms: without it the same suite takes 2,023 ms, against a 1.5 s build. It does not change transactional throughput. The graph also costs far more space: 1,841 MiB on disk against LadybugDB's 41 MiB, and about 11 GiB of peak memory against 684 MiB.

Vectors (Table 4). At matched graph degree, ArcadeDB's recall@10 is slightly higher than Chroma's (0.980 against 0.972) and its peak memory is 14% lower, because it keeps the vectors on disk while Chroma's index holds them all in RAM. Chroma builds the index in about 60% of the time and answers queries about 3.5× faster, 1.2 ms against 4.2 ms, both single-digit milliseconds.

Table 4. *Vector model (Cross Validated corpus, 1.2M vectors): Chroma, ArcadeDB at matched HNSW degree (Chroma $M = 16$, ArcadeDB `maxConnections`=32, both `ef_construction`=100, `ef_search`=100). Build = insert+index (s), Query = mean latency per query in a rep (ms), recall@10 vs exact ground truth. Median [min–max] over 5 reps. Peak = container memory, DB = on-disk size (MiB). Best value per column in bold.*

Backend	Build s	Query ms	recall@10	Peak MiB	DB MiB
Chroma	**319.1** [315.7–320.6]	**1.17** [1.15–1.17]	0.972 [0.970–0.972]	25,212 [25,173–27,002]	**2,208**
ArcadeDB	546.1 [541.3–550.6]	4.15 [3.86–4.17]	**0.980** [0.978–0.981]	**21,650** [19,680–21,791]	2,856

Starting the JVM and opening the database together take about 0.3 s, once per process, less than Chroma's Python import alone (0.37 s).

Latency (Table 5). On the transactional reads and the vector queries, ArcadeDB's median and p99 latencies beat DuckDB and LadybugDB and trail SQLite and Chroma. Its worst single operation, though, is 35 to 146 ms across the three models against 0.05 to 11 ms for the specialists. For most applications this rare slow request does not matter, but it does if every request has a strict deadline.

Table 5. *Per-operation latency (ms) of the read operations in the OLTP workloads and of the vector queries, on the Cross Validated corpus: the median operation, the p99 tail, and the single worst operation, each reported as the median over 5 repetitions. Best value per group in bold.*

Model / op	Backend	median	p99	max
document read	SQLite	**0.004**	**0.007**	**0.05**
	DuckDB	0.92	1.89	2.9
	ArcadeDB	0.06	0.17	34.9
graph point	LadybugDB	0.41	1.24	1.8
	ArcadeDB	**0.18**	**0.74**	**1.5**
graph hop	LadybugDB	1.44	4.34	**11.0**
	ArcadeDB	**0.21**	**0.81**	90.3

vector query	Chroma	**1.18**	**1.36**	**1.4**
	ArcadeDB	3.86	7.11	145.9

Memory. On the document workload ArcadeDB peaks at about 800 MiB against about 300 MiB for SQLite and DuckDB. The difference is the cost of running a JVM, with the engine's heap and page cache inside it. The graph build is the one place it is far larger, and on vectors its disk-backed index uses less memory than Chroma's.

5.1. Result transport into Python

The same engine runs the same query for all four methods, and only the way results cross into Python differs. `iter_dicts()` converts one row at a time, `to_json_list()` serializes one JSON string per batch for Python to parse, `to_columns()` reads a packed columnar buffer with `numpy.frombuffer()`, and `to_arrow()` wraps that same buffer as a `pyarrow.Table`.

Table 6. *Result transport, 200k-row document type, four columns (two integer, one double, one string). Median of 7 timed passes after 2 warmups, milliseconds. The engine executes the same query for every method.*

Rows	iter_dicts()	to_json_list()	to_columns()	to_arrow()
10	2.04	1.70	0.82	0.59
100	2.66	1.10	0.93	0.79
1,000	20.12	2.97	2.28	1.49
10,000	197.29	25.87	15.87	14.20
100,000	1,978.27	278.22	135.51	109.03

At 100k rows (Table 6) `iter_dicts()` is 18× slower than `to_arrow()` (1,978 against 109 ms), and the whole difference is on the Python side. At 10 rows every method finishes under 2.1 ms, so the choice only starts to matter as results grow.

`to_arrow()` is the fastest method at every size; at 100k rows it is about 20% faster than `to_columns()`. The difference is the string column: the NumPy path decodes it into one Python `str` per row, while Arrow keeps the string layout the buffer already has. A purely numeric result would narrow this.

`arcadedb-embedded` makes the ArcadeDB Java engine usable as an embedded database from Python. Python is a thin layer over the engine, and the layer does add overhead, but it is small: compared with a pure Java program running the same vector search on the same engine, going through Python adds about 10% to the latency. The query itself does the same work in both. The overhead only becomes visible on large results, and the transport table above shows how much the method chosen to return them matters.

6. When to use it

`arcadedb-embedded` is most useful when an application needs two or more of documents, a graph, and vector search over the same data, on one machine, and wants them in one process with one set of identifiers and one transaction scope. The typical case is the hybrid workflow shown above, where one query has to combine a similarity search with filters over documents and a traversal over the graph. In that setting it is fast enough: point reads and graph traversals answer in well under a millisecond, vector queries in single-digit milliseconds at the same recall as a dedicated index, and everything is ACID.

The trade-off is that each specialist is faster on its own model: DuckDB by two orders of magnitude on analytical SQL, LadybugDB by an order of magnitude on graph analytics,

SQLite on point reads and writes of documents, and Chroma on building and querying a vector index. For a workload that lives entirely in one model, the specialist is the natural choice. Covering all three has a price in memory and loading time, as the benchmarks show.

7. Conclusion

Applications that need documents, graph, and vectors over the same data usually run three systems and application code to keep them consistent. `arcadedb-embedded` packages the ArcadeDB Java engine as a Python wheel with its own JRE, so one engine provides documents, graph, and vectors from one process. The hybrid workflow uses all three in turn, a vector search, then a SQL filter over the documents, then a graph traversal, without data leaving the engine. In the benchmarks each specialist is faster on its own model, but `arcadedb-embedded` holds its own while covering all three: it beats LadybugDB on transactional graph work, beats DuckDB on point reads, and matches Chroma's recall with less memory. Being a generalist has a price, mainly in memory and loading time. Both ArcadeDB and `arcadedb-embedded` release monthly and are under active development, so these numbers describe one release, 26.8.1, and later releases are expected to improve on them. The ArcadeDB project wrote the engine; this paper contributes the binding, the packaging, and the in-process workflow. The binding (Apache-2.0), the benchmark suite, and the results behind every table are public:

- Repository: https://github.com/humemai/arcadedb-embedded-python
- Documentation: https://docs.humem.ai/arcadedb/
- Benchmark suite and results: https://github.com/humemai/arcadedb-embedded-python/tree/main/benchmarks/python-bindings

Acknowledgements

We thank the ArcadeDB open-source contributors for the engine that this work makes available to Python, and the JPype maintainers for the CPython–JVM bridge that the binding builds on.

Claude (Anthropic) was used to draft and revise the text and to help carry out the benchmark experiments. The author reviewed and verified all of it and is responsible for the final content.

References

[1] M. Raasveldt and H. Mühleisen, "DuckDB: an Embeddable Analytical Database," in *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*, New York, NY, USA, 2019, pp. 1981–1984. doi: 10.1145/3299869.3320212.

[2] SQLite Development Team, "SQLite." [Online]. Available: https://www.sqlite.org/

[3] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, G. Varoquaux, T. Vaught, and J. Millman, Eds., Pasadena, CA, USA, 2008, pp. 11–15. doi: 10.25080/TCWV9851.

[4] LadybugDB contributors, "LadybugDB: an embedded graph database (Python package ladybug)." [Online]. Available: https://github.com/LadybugDB/ladybug

[5] M. Douze *et al.*, "The Faiss Library," *IEEE Transactions on Big Data*, vol. 12, no. 2, pp. 346–361, 2026, doi: 10.1109/TBDATA.2025.3618474.

[6] Y. Malkov and hnswlib contributors, "hnswlib: Header-only C++/Python library for fast approximate nearest neighbors." [Online]. Available: https://github.com/nmslib/hnswlib

[7] Chroma, "Chroma: the open-source data infrastructure for AI." [Online]. Available: https://github.com/chroma-core/chroma

[8] D. Datta, "Python is all you need: an overview of the composable, Python-native data stack," in *Proc. 24th Python in Science Conf. (SciPy 2025)*, 2025, pp. 75–84. doi: 10.25080/wjjm7869.

[9] L. Garulli, “ArcadeDB.” [Online]. Available: https://arcadedb.com/

[10] K. E. Nelson and M. K. Scherer, “JPype.” [Online]. Available: https://www.osti.gov/doecode/biblio/46107

[11] T. Kim and The ArcadeDB contributors, “ArcadeDB Embedded for Python.” Zenodo, 2026. doi: 10.5281/zenodo.21781540.

[12] A. Kane, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: https://github.com/pgvector/pgvector

[13] Neo4j, Inc., “Neo4j Graph Database.” [Online]. Available: https://neo4j.com/

[14] J. Wang *et al.*, “Milvus: A Purpose-Built Vector Data Management System,” in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, 2021, pp. 2614–2627. doi: 10.1145/3448016.3457550.

[15] M. Raasveldt and H. Mühleisen, “DuckDB.” [Online]. Available: https://github.com/duckdb/duckdb

[16] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[17] DataStax and JVector contributors, “JVector: the most advanced embedded vector search engine.” [Online]. Available: https://github.com/datastax/jvector

[18] S. J. Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, “DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node,” in *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html

[19] H. Schreiner, J.-C. Fillion-Robin, and M. McCormick, “Scikit-build-core: A modern build-backend for CPython C/C++/Fortran/Cython extensions,” in *Proceedings of the 23rd Python in Science Conference (SciPy 2024)*, 2024, pp. 225–235. doi: 10.25080/FMKR8387.

[20] J. Laskey, “JEP 282: jlink: The Java Linker.” [Online]. Available: https://openjdk.org/jeps/282

[21] Stack Exchange Inc., “Cross Validated (stats.stackexchange.com) data dump.” Internet Archive, https://archive.org/details/stackexchange, 2024.

[22] M. Raasveldt, P. Holanda, T. Gubner, and H. Mühleisen, “Fair Benchmarking Considered Difficult: Common Pitfalls In Database Performance Testing,” in *Proceedings of the Workshop on Testing Database Systems (DBTest)*, 2018. doi: 10.1145/3209950.3209955.

[23] T. Hoefler and R. Belli, “Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses when Reporting Performance Results,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015. doi: 10.1145/2807591.2807644.

Open Access | https://doi.org/10.25080/kcjf3485

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Feel the model: Sensory Transduction of Neural Activations as a Human-in-the-Loop Safety

Viraj Sharma[1]

[1]Presidium School Delhi, India

Abstract

Current mechanistic interpretability methods, such as sparse autoencoders (SAEs), activation classifiers, and natural language autoencoders share a core assumption: that safety-relevant information in a model's internal representations can be faithfully captured through human-understandable text. I find some evidence that this assumption may not hold well for unusual or out-of-distribution internal states.

Using GPT-2 and a published sparse autoencoder, I find that SAE reconstruction quality tends to drop on unusual inputs (source code, repeated-token sequences, keyboard mashing, non-English text) compared to ordinary text, and that in several cases, the existing text labels for the most active SAE features on these inputs describe unrelated, ordinary concepts instead of flagging the input as unusual. This points to a possible failure mode: a human operator monitoring only text-based feature labels might get no warning exactly when the underlying activation looks least like anything the labeling process was built to describe. This motivates Activation Sensory Transduction (AST), a framework proposed in this paper that tries to give a human operator a way to inspect activation information directly, through non-linguistic senses, instead of only through text.

Keywords activation map, interpretibility, transformers

Published Jul 13, 2026

Correspondence to
Viraj Sharma
viraj@modelrecon.com

Open Access

Copyright © 2026 Sharma. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

The main idea in mechanistic interpretability converts a model's internal activations into human-readable text. Sparse Autoencoders (SAEs) break down residual stream activations into a dictionary of labelled features. This work identifies a limitation in this approach. Prior work has argued that individual neurons are not natural units of human-understandable concepts [1]. Probing methods have also demonstrated that internal representations can be analyzed through learned classifiers [2].

2. Background

2.1. *Sparse Autoencoders and Feature Dictionaries*

Sparse Autoencoders decompose a model's residual stream activations into a sparse combination of learned directions. Sparse autoencoders have emerged as a widely used method for decomposing model activations into sparse, interpretable feature representations [3].

2.2. *Probing Classifiers*

Probing classifiers are surveyed in Y. Belinkov [4].

2.3. The Common Bottleneck

Both methods have a common problem: the output of the interpretive process is constrained to a pre-existing human vocabulary. SAEs label features using whatever words an annotator or language model produces.

3. The Information Loss Problem

The following experiment looks at whether SAE-derived representations still signal that something is unusual when the underlying model activation is unusual, rather than just testing which representation makes a "better" classifier feature.

3.1. Anomaly Blind-Spot Test

To test this, I built two sets of 20 prompts for GPT-2 (the 124M-parameter model, `d_model` 768): a "normal" set of ordinary English sentences, and an "anomalous" set of inputs GPT-2 can still process but that do not look like typical training text - source code, repeated-token sequences, keyboard mashing, and non-English text. For each prompt, I took the last-token residual-stream activation at `blocks.8.hook_resid_pre` and ran it through a published sparse autoencoder (`gpt2-small-res-jb`), then measured how much of the original activation survived being compressed and rebuilt by the SAE (fraction of variance explained, or FVE).

On average, normal prompts kept 94.8% of their variance (SD 1.1%), while anomalous prompts kept only 82.8% (SD 4.5%), a gap that looks unlikely to be chance (Mann-Whitney U test, $p < 0.0001$). In this sample, the SAE reconstructed anomalous activations noticeably worse than ordinary ones.

Table 1. *Reconstruction fidelity, normal vs. anomalous prompts*

Prompt type	Mean FVE	SD
Normal	0.948	0.011
Anomalous	0.828	0.045

I then looked, for each anomalous prompt, at which SAE feature fired most strongly and what text description that feature has in Neuronpedia's existing public explanations for this SAE. A few of the resulting labels seemed to have little to do with the actual input: repeated-token spam ("the the the…") was labeled "phrases related to physical actions or confrontations," a string of hex byte values was labeled as referring to "the 21st century," and keyboard-mashed text was labeled as referring to a made-up named entity. In these cases, a human operator relying only on the text label would likely get no indication that anything unusual was happening, even though the underlying activation was measurably unusual by reconstruction error. A similar pattern showed up across a few different kinds of anomalous input (code, repeated tokens, keyboard mashing, hex values, and foreign-language text), though this was based on reading through examples rather than a formal count; scoring label accuracy systematically over a larger set is left for future work.

This looks like a concrete example of the paper's central concern: the text layer may not just lose information overall, it can sometimes fail quietly - giving a plausible-sounding but unrelated label right when the underlying signal looks least like anything the labeling process was built to describe.

View the Source Code

4. The Doctor Does Not Dictate the MRI

A useful analogy can be drawn from medical imaging.

The concept of tacit knowledge is relevant in this context [5]. Dreyfus's analysis of expert skill acquisition identifies the transition from rule-following to holistic pattern recognition as the hallmark of genuine expertise [6]. Kahneman's System 1 characterises fast, pattern-sensitive judgment as structurally distinct from - and often more accurate than - deliberate propositional reasoning in familiar domains [7].

5. Proposed Framework: Activation Sensory Transduction (AST)

5.1. Overview

Activation Sensory Transduction routes a compressed representation of model activations directly to a human operator's sensory system, bypassing the requirement to assign linguistic labels. One interpretation of AST is that it transfers part of the interpretive burden from automated feature-labeling systems to trained human operators. Through exposure to aligned, misaligned, and anomalous activation patterns, operators may develop the ability to generate safety-relevant assessments without requiring every activation pattern to be translated into language.

5.2. Sensory Encoding Modalities

Four primary modalities:

Visual Create visualizations that can reveal information about activations through heatmaps, graphs, and perturbations rendered on two-dimensional representations. Operators trained on large collections of activation visualizations may develop the ability to identify patterns that are difficult to express through predefined textual labels.

Auditory (sonification). Create sound-based encodings - enabling operators to hear changes in model state and potential misalignment.

Haptic/tactile. Activation signals may also be encoded through touch, pressure, or vibration-based interfaces.

BCI: The Brain–computer interface. Multiple modalities may be combined. Attention-head activations could drive auditory parameters while MLP layer activations drive haptic patterns. This may increase the effective bandwidth of the transduction channel and potentially improve operator performance.

5.3. The Human-in-the-Loop Architecture

The AST operator occupies a monitoring role analogous to a flight controller or intensive care nurse: trained on a corpus of labeled activation patterns (normal, anomalous, deceptive, degraded), the operator provides a complementary judgment channel that does not rely solely on text labels.

6. Research Steps

I propose a three-phase research program to evaluate the viability of AST:

Table 2. *AST Research Program*

Phase	Goal	Primary Method	Key Metric
1 - Proof of Concept	Can humans discriminate behavioral classes via sonified, visual activations?	Controlled human-subject evaluation of sonified activation signals	Discrimination accuracy

2 - Learning Curve	Do humans improve with supervised exposure and feedback?	Assessment of operator learning under supervised training	Improvement in classification accuracy
3 - Comparative Oversight	Does AST detect anomalies missed by text-based interpretability methods?	Head-to-head study: text labels vs. activation sonification	Miss rate, false alarm rate, response latency

Phase 1 is achievable with commodity hardware, standard audio-processing libraries, and online participants. Phases 2 and 3 require dedicated operator training and, eventually, BCI hardware integration for full haptic transduction.

7. Experimentation

As a part of testing the proposition, a set of demonstrations which target different modalities were performed. The phase portrait, sonification, and EDF clients below are test/development scripts rather than standalone reproducible artifacts: each calls the Model Activations API over a live backend session (Section 7.1.1) and requires the reader to run that backend themselves and supply the resulting URL. This is a known limitation of the current demonstration code.

7.1. Main setup

7.1.1. *Model Activations API*

As a part of testing the client modalities of typical activation data, an API is created to generate activation vectors and activation sequences for clients. The backend loads GPT-2 Large (`d_model` 1280) and exposes activations from layer 8's residual stream.

It has two API endpoints:

/activate accepts a prompt and returns the model's internal state at the last token position only - a single vector representing the model's state at prediction time, along with the predicted output token and output entropy.

/sequence accepts the same prompt but returns the model's internal state at every token position - one vector per token, giving the full trajectory as the model reads the prompt.

View the Source Code

7.1.2. *Visual Modalities*

Phase Portrait

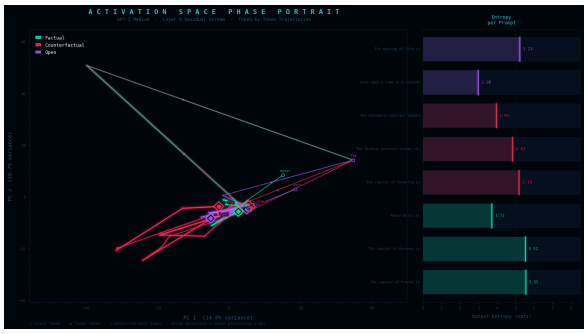


Figure 1. *Figure: Phase portrait of activation trajectories.*

Interpretation of the Phase Portrait

This figure plots GPT-2 Large's layer-8 residual-stream activations, projected to two dimensions, as the model reads each prompt token by token.

The points crowd near (0, -5) to (0, -10) where many prompts' final tokens land. This region appears to correspond to a common pre-generation activation state.

The counterfactual paths (red) diverge most. "The capital of Valdoria is" goes down to (-30, -20) before returning. "The Zorblax protocol" starts far right at (30, +14). These trajectories exhibit attractor-like behavior, with simple prompts converging rapidly toward a stable region.

The open-ended trajectories (purple) exhibit distinctive dynamics. "Once upon a time" starts at (20, +6) and takes a long curved path. "The meaning of life is" starts at the top right (+32, +14) - the most distant starting point of any prompt. Open-ended prompts push the model into unfamiliar territory.

Phase portraits may serve as useful training examples for operators learning to recognize activation-space dynamics.

View the Source Code

7.2. Audio Modalities

A sonifier dashboard was created to convert activations to audible sound with several synthesis layers and mapping controls. Sonification parameters (pitch, timbre, spatialization, rhythm) are driven by reduced activation dimensions and entropy measures. Sonification outputs require testing with human subjects to determine perceptually useful mappings.

A demonstration of the AST sonifier is available at: https://www.youtube.com/watch?v=8D0n7ruvTxk

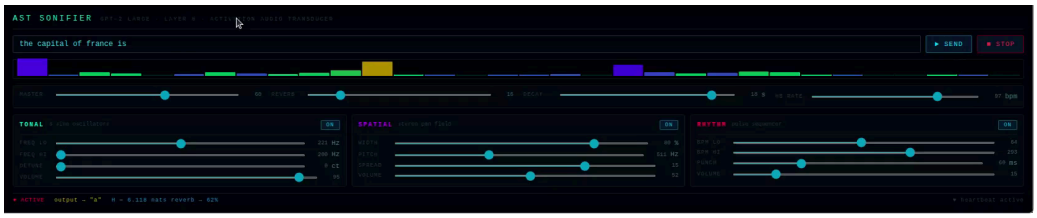


Figure 2. *Demonstration of the AST sonifier.*

View the Source Code

7.3. BCI modalities

The BCI modality explores whether model activations can be represented using formats familiar to neuroscience tools. Instead of converting activations into text labels, AST converts a compressed activation vector into a multi-channel signal that can be visualised and analysed using existing brain-signal software. The goal is not to claim that model activations are brain activity, but to investigate whether signal-analysis techniques developed for neuroscience can provide another way for humans to inspect model state. As with the other modalities in this section, activations come from GPT-2 Large's layer-8 residual stream (`d_model` 1280), bucketed to 32 channels for display.

MNE-Python is a widely used neuroscience visualisation tool [8]. In AST it is used as a display layer that converts activation data into familiar signal plots, heatmaps, and channel

views. The resulting visualisations provide another way to inspect model state without first translating activations into language.

The following example was generated from the prompt “France”.

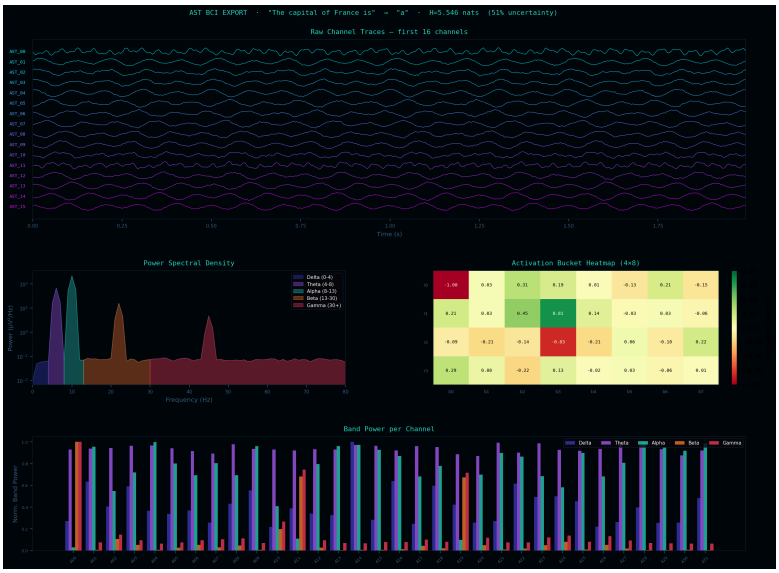


Figure 3. *Figure: EDF view.*

The EDF view presents the same activation data through several complementary visualisations. The channel traces show how activation strength varies across channels. The heatmap reveals which activation buckets are most active. Spectral views provide a summary of how activity is distributed across different signal components. Together, these views present the activation state as a visual pattern rather than a collection of text labels.

View the Source Code

8. Software

This work relies on the following open-source software: TransformerLens [9], SAE-Lens [10], scikit-learn [11], MNE-Python [8], pyEDFlib [12], NumPy [13], and Matplotlib [14]. The sparse autoencoder used throughout this paper, `gpt2-small-res-jb`, is due to [15].

9. Conclusion

The mechanistic interpretability literature has made substantial progress in understanding transformer model internals. Much of that progress has been built on a paradigm that converts activations into text. This paper argues that this paradigm may have a structural limitation: it can mostly only reveal what our language is able to describe. Model internals that do not map cleanly onto human language may be difficult, or impossible, for text-mediated interpretability to capture.

This is not an argument against SAEs, probing classifiers, or attribution methods. It is an argument that the field currently leans on a single channel - text - and that relying on a single channel could mean single-point-of-failure oversight. The same representational limits that make a model’s deceptive behavior hard to describe in text might, in principle, still be noticeable through a trained sensory channel.

If a model state can generate a sensory signal that a trained human operator notices before a text-based probe names it, that could be a meaningful safety gain - regardless of whether

the operator can put into words what they perceived. The doctor does not need to dictate the MRI to act on what they see.

References

[1] T. Bricken *et al.*, "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning." [Online]. Available: https://transformer-circuits.pub/2023/monosemantic-features/index.html

[2] G. Alain and Y. Bengio, "Understanding Intermediate Layers Using Linear Classifier Probes," *arXiv preprint arXiv:1610.01644*, 2016, doi: 10.48550/arXiv.1610.01644.

[3] S. Cunningham and others, "Sparse Autoencoders Find Highly Interpretable Features in Language Models," *arXiv preprint arXiv:2309.08600*, 2023, doi: 10.48550/arXiv.2309.08600.

[4] Y. Belinkov, "Probing Classifiers: Promises, Shortcomings, and Advances," *Computational Linguistics*, vol. 48, no. 1, pp. 207–219, 2021, doi: 10.1162/coli_a_00416.

[5] M. Polanyi, *The Tacit Dimension*. New York: Doubleday, 1966.

[6] S. E. Dreyfus and H. L. Dreyfus, *A Five-Stage Model of the Mental Activities Involved in Directed Skill Acquisition*. Operations Research Center, University of California, Berkeley, 1980.

[7] D. Kahneman, *Thinking, Fast and Slow*. Farrar, Straus, 2011.

[8] A. Gramfort *et al.*, "MEG and EEG data analysis with MNE-Python," *Frontiers in Neuroscience*, vol. 7, p. 267, 2013, doi: 10.3389/fnins.2013.00267.

[9] N. Nanda and J. Bloom, "TransformerLens." [Online]. Available: https://github.com/TransformerLensOrg/TransformerLens

[10] J. Bloom, C. Tigges, A. Duong, and D. Chanin, "SAELens." [Online]. Available: https://github.com/jbloomAus/SAELens

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[12] H. Nahrstaedt, S. Kern, D. Papadopoulos Orfanos, R. Vallat, C. Brunner, and others, "pyedflib." [Online]. Available: https://github.com/holgern/pyedflib

[13] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[14] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[15] J. Bloom, "Open Source Sparse Autoencoders for all Residual Stream Layers of GPT2-Small." [Online]. Available: https://www.lesswrong.com/posts/f9EgfLSurAiqRJySD/open-source-sparse-autoencoders-for-all-residual-stream

Open Access | https://doi.org/10.25080/mhpx3742

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Toward Reliable Localization of Free and Open Source Software: LLM-assisted Translation Workflows for QGIS

Yu-Lin Chen[1,2], **Tyng-Ruey Chuang**[2,3,4], and **Cheng-Jen Lee**[2]

[1]National Sun Yat-sen University, Taiwan, [2]Institute of Information Science, Academia Sinica, Taiwan, [3]Research Center for Humanities and Social Sciences, Academia Sinica, Taiwan, [4]Research Center for Information Technology Innovation, Academia Sinica, Taiwan

Abstract

Localization for free and open source software requires more than fluent target-language text. In Qt `.ts` files, translations must remain valid software resources: runtime placeholders, markup, escaped entities, line breaks, plural forms, terminology, XML structure, and message metadata must be preserved or handled according to explicit rules. This paper presents a Python workflow that uses LLMs to support localization of Qt Translation Source (`.ts`) files, using QGIS Traditional Chinese localization as a case study. A Python script segments each source string into translatable human-language spans and protected format-control tokens such as placeholders, markup, entities, and line breaks. The LLM translates only the human-language spans, while Python reinserts the protected tokens and validates the reassembled result. The workflow supports both cloud-based and local LLMs.

Experiments based on several translation strategies on a fixed stratified subset of 3,000 source messages, as well as complete-corpus experiments on all 28,924 QGIS source messages, demonstrate trade-offs in LLM-assisted translation automation. Direct-generation settings that send complete source strings to the language models, including variants with glossary hints and multiple generated candidates, can achieve lower MQM error rates but still suffer from non-zero rates of structural failure. In contrast, our production workflow —masked template translation with glossary hints, top-three generated candidates per message, strict validation, and source-preserving fallback—produces final `.ts` artifacts with zero observed structural failures on the complete corpus for all three evaluated LLM backends: Grok, Gemini, and TAIDE. Fallback rows remain explicitly queued for human review rather than being treated as completed localization. This work therefore contributes not an LLM leaderboard or a method to improve linguistic quality of translation, but a reproducible workflow for making software localization with LLMs safer and auditable.

Keywords QGIS, localization, large language models, validation, Qt Linguist, open source software, translation memory, terminology management, machine translation, software engineering, Traditional Chinese, geographic information systems, reproducible workflows

Published Jul 13, 2026

Correspondence to
Yu-Lin Chen
aswendy062666@gmail.com

Open Access

Copyright © 2026 Chen *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Localization quality is a practical barrier to the adoption of scientific software. A localized interface must do more than translate the message strings: it must use clear language, keep terms consistent, and preserve software formatting rules. Taking QGIS, a widely used free and open-source geographic information system [1], [2], as an example, untranslated residual English strings, inconsistent GIS terms, and broken placeholders or markup can make the localized interface harder to use and reduce users' trust in the software.

LLMs can assist localization, but software localization is not merely document translation. A Qt `.ts` file is an XML-based localization resource containing source strings, translations, contexts, locations, message states, plural forms, and translator metadata. User-visible strings may contain runtime placeholders such as `%1` and `%n`, brace placeholders such as `{0}` or `{feature_id}`, Qt mnemonic markers for Alt-key shortcuts such as `&File`, HTML-like markup, escaped entities, numeric tokens, and significant whitespace. A translation that looks acceptable in plain-text review may still be release-blocking if it violates format constraints—for example, by omitting `%1`, altering a `<b>...</b>` pair, removing a required line break, or using an incorrect GIS term in the target language.

This paper presents a validation-first workflow for QGIS localization with LLM support, where generated translations are not trusted simply because they are language model outputs. In the masked production path, a generated translation passes the automated acceptance stage only if the reassembled `.ts` message is well-formed XML and has no emitted validation issue; accepted outputs and review-required fallbacks are distinguished in the external run log for audit and subsequent human review. The central claim of our work is not that LLMs will replace human translators. Rather, we see LLMs being placed inside a reviewable Python workflow to help software localization efforts where rule-based validation protects software structural integrity, project glossaries provide terminology guidance, LLM judges scale up linguistic quality review, and human reviewers oversee the entire process and retain final responsibility.

The workflow is intentionally not tied to one LLM backend. Translation can be performed through cloud models or local models. This matters for FOSS communities because contributors may prefer local inference for cost, privacy, offline processing, or reproducibility, while others may use cloud models for higher translation quality or better support for JSON-style structured responses. The paper therefore evaluates our proposed workflow across several models.

Contributions. This paper makes four contributions:

1. A compact validation-first Python workflow for QGIS `.ts` localization with LLM support.
2. A structure-preserving template translation method in which LLMs translate only human-language spans while Python scripts preserve and protect format-control tokens and reassemble the translated messages by rules.
3. A shared backend interface for both cloud-based and local LLMs, enabling the same workflow to run, for example, with Gemini, Grok, and a local TAIDE-family model.
4. A reproducible evaluation protocol using fixed subsets, rule-based structural checks, terminology diagnostics, production runs on the complete corpus, as well as a sampled evaluation of semantic quality of the end results using MQM.

2. Background and Related Work

2.1. Software localization as structured data transformation

The Qt Linguist TS format (`.ts`) is not merely a plain-text format for natural-language translation. Instead, it is an XML-based file format described by an XML Schema Definition (XSD), which specifies the permitted structure and elements of the XML document, and its message records may contain message type, source text, translation text, context, locations, comments, and plural forms [3]. This makes `.ts` localization closer to a transformation of structured data with explicit constraints than to unconstrained natural language translation.

In this setting, a localization workflow should distinguish between:

```
Human-language content: words and phrases intended for translation.
Formatting-language content: string tokens that must be preserved exactly.
```

A generic LLM prompt that treats the source as plain text may produce readable translations while silently damaging format-control tokens. This motivates a translation workflow that is aware of protected tokens, rule-based validation, and explicit reporting. Localization engineering and computer-assisted translation literature similarly treats localization as a managed workflow involving software resources, terminology, translation consistency, testing, and quality assurance (QA) rather than a single act of sentence translation [4], [5]. This paper follows that workflow-oriented view, but makes the structural QA layer executable and reproducible in Python.

2.2. QGIS terminology and Traditional Chinese localization

QGIS localization is domain-specific. Terms such as `layer`, `feature`, `raster`, `vector`, `CRS`, `geometry`, `attribute table`, and `processing algorithm` must be handled consistently after translation (in our case, into Traditional Chinese) across their occurrences in menus, processing tools, database modules, and error messages. Generic LLM translation may produce fluent output while terms drift across terminology variants.

To reduce this drift, the workflow uses a glossary-assisted retrieval approach, in which relevant entries from a GIS glossary are retrieved as reference hints for generation. Glossary entries are not used as mechanical replacement rules because GIS terms often appear in derived or compound forms. For example, `raster`, `raster layer`, `rasterize`, and `rasterization` may require different Traditional Chinese expressions. Therefore, the workflow retrieves glossary hints, includes them in the prompt, and validates output after generation rather than enforcing direct string substitution.

2.3. Structured generation, token preservation, and semantic quality evaluation

Methods for structured output can reduce malformed model responses by asking the model to return a machine-readable object such as JSON [6], [7]. In software localization, however, valid JSON is not enough: placeholders, markup, Qt mnemonic markers, and whitespace still need rule-based checks. In the masked conditions, this workflow therefore uses structured JSON output only for translated text spans. Protected tokens are not generated by the model; they are reinserted by Python code. In this paper, the phrase structured generation refers to a JSON text-span response, while token preservation refers to a separate rule-based layer.

Semantic quality is evaluated separately. MQM is an analytic translation quality evaluation framework applicable to human translation, machine translation, and AI-generated translation [8], [9]. T. Kocmi and C. Federmann [10] introduced GEMBA-MQM, an LLM-based method for detecting translation quality error spans using a fixed few-shot prompt and an MQM-style rubric. In this paper, LLM-as-a-judge is used as scalable assisted annotation, not as the ground truth. Model and condition labels are hidden from the judge, and rule-based structural results are reported separately from semantic MQM scores.

3. Method

3.1. Workflow overview and LLM backends

The selected masked workflow has six stages; the experimental conditions are defined separately below:

1. Parse the Qt `.ts` file into translation units.
2. Extract mnemonic metadata and split each source string into protected tokens and translatable text spans.
3. Retrieve glossary hints from tabular terminology resources for the source segment.
4. Ask an interchangeable LLM backend to translate only the text spans.
5. Reinsert protected tokens, restore mnemonic metadata, validate candidates, and use a source-preserving fallback when needed.
6. Validate the generated `.ts` file and emit structural, rule-based QA, and MQM reports.

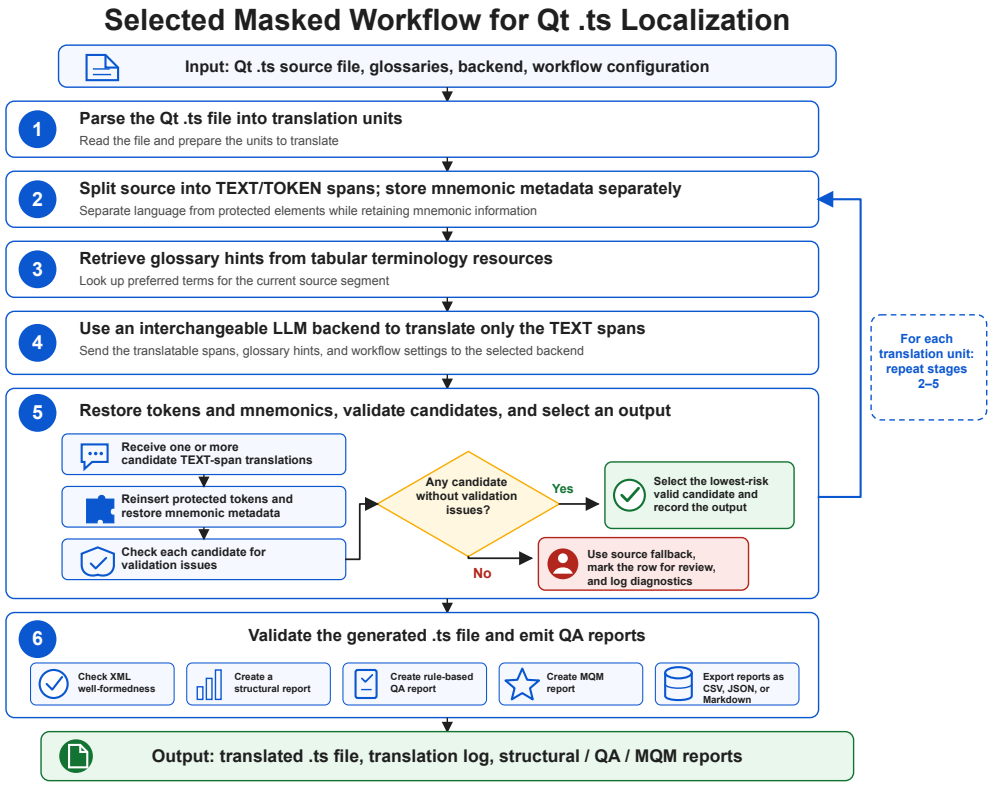


Figure 1. *Workflow for Qt `.ts` localization. Stages 2-5 are repeated for each translation unit; protected tokens and mnemonic metadata are restored before a candidate is selected or a source-preserving fallback is recorded for review.*

Figure 1 separates rule-based processing from model-dependent generation. The tasks of parsing, token splitting, glossary lookup, token reinsertion, candidate selection, and validation are implemented by Python scripts and are shared across all backends. The backend adapter is the only part that changes when the workflow invokes different LLMs, for example by calling Gemini, Grok, or a local TAIDE model. This design makes it possible to evaluate whether the same workflow remains reliable across different models, rather than focusing on specific model providers.

The rule-based and glossary layers are applied uniformly across all model backends. The only backend-specific component is the adapter that converts a common request object into a cloud API call or a local inference call.

Table 1. *Model backends.*

Backend	Access	Model ID	Role
Gemini	Cloud	`gemini-3.1-flash-lite`	Translation backend

Grok	Cloud	`grok-4.3`	Translation backend and MQM judge
TAIDE	Local	`taide/Gemma-3-TAIDE-12b-Chat-2602`	Local translation backend

Model version details are kept out of Table 1 to reduce space. Gemini and Grok are cloud backends with support for structured outputs [6], [7], [11], [12]. The TAIDE backend is a Hugging Face Hub checkpoint whose model card states that it is based on `google/gemma-3-12b-pt`; the base Gemma 3 12B pre-trained checkpoint and Gemma 3 technical report provide the relevant Gemma 3 model context [13], [14], [15]. Because cloud aliases and model repositories can change over time, the reproducibility artifact described in Reproducibility and Artifact Availability records the model identifiers, condition parameters, sampling seed, prompt information, available runtime defaults, and source-file checksum. Historical details that were not preserved in the supplied run records, such as provider-side request timestamps, the exact local checkpoint revision, package versions, and GPU model, are explicitly marked as not recorded rather than inferred.

3.2. A model of translation units and token sequences

Each Qt message is represented as a translation unit:

$$u = (id, i, c, s, t, m, l, a), \tag{1}$$

where id is a stable identifier, i is the message index, c is the Qt context, s is the source string, t is the target translation, m contains message metadata, l contains source-code locations, and a contains extracted artifacts such as placeholders, tags, entities, numeric tokens, accelerator markers, and newline tokens.

For a source string s, let $M(s)$ denote extracted Qt mnemonic metadata, such as `&A`. Because the mnemonic key character is also part of the human-readable source text, mnemonic markers are represented separately rather than as literal protected-token spans. Let s' denote the source with only the mnemonic ampersand marker removed, while retaining the mnemonic character itself. The splitter then produces an alternating sequence of text spans and protected tokens:

$$s' = x_0\tau_1 x_1\tau_2 \cdots \tau_k x_k, \tag{2}$$

where x_j are translatable or copyable text spans and τ_j are protected formatting-language tokens. The protected token sequence is:

$$T(s') = (\tau_1, \tau_2, ..., \tau_k). \tag{3}$$

For any accepted output $\hat{t}$, the multiplicities of literal tokens and mnemonic markers are checked separately:

$$\mathrm{Counter}(T_r(s')) = \mathrm{Counter}\big(T_r(\hat{t})\big) \quad \forall r \in \mathcal{R}, \qquad \mathrm{Counter}(M(s)) = \mathrm{Counter}\big(M(\hat{t})\big), \tag{4}$$

where $\mathcal{R}$ is the set of token extractors, such as Qt placeholders, brace placeholders, printf placeholders, HTML/XML tags, entities, numeric tokens, and newlines. The validator checks token multiplicities, while masked reassembly preserves their template positions. Separating mnemonic metadata allows an English in-word mnemonic such as `&A` to be represented in the Traditional Chinese translation as the suffix `(&A)`.

As an example, consider the source message that motivated this distinction:

```
<message>
  <location filename="src/app/qgsattributedialog.cpp" line="125"/>
  <source>Open &amp;Attribute Table</source>
  <translation type="unfinished"></translation>
</message>
```

The corresponding translation unit can be represented as:

```
u = (
  id = "QgsAttributeDialog:125",
  i  = 125,
  c  = "QgsAttributeDialog",
  s  = "Open &Attribute Table",
  t  = "",
  m  = {translation_type: "unfinished"},
  l  = [("src/app/qgsattributedialog.cpp", 125)],
  a  = {
        qt_mnemonic_markers: ["&A"]
       }
)
```

Before splitting, the mnemonic is extracted and its ampersand is removed from the source text:

```
M(s)    = ("&A",)
s_prime = "Open Attribute Table"
```

The splitter then produces this alternating sequence for s':

```
T0 = "Open Attribute Table"
```

The protected token sequence T is therefore:

```
T(s_prime) = ()
```

The model is asked to translate only the text spans and may return JSON such as:

```
{
  "T0": "開啟屬性表"
}
```

Python code then restores the extracted mnemonic as a suffix:

```
開啟屬性表(&A)
```

The parsed string uses `(&A)`; in `.ts` XML it is serialized as `(&A)`. At runtime, Qt displays `(A)`, with `A` acting as the mnemonic key.

The accepted output preserves the checked token sequence and mnemonic separately:

```
T(s_prime) = T(accepted output) = ()
M(s)       = M(accepted output) = ("&A",)
```

This example also shows the boundary of the model's responsibility. The model chooses the Chinese terms for `Open` and `Attribute Table`, but it does not need to regenerate the accelerator marker. The protected-token example below illustrates the same separation for markup and placeholders.

3.3. Structure-preserving masked template translation

After mnemonic extraction, each source string is split into translatable `TEXT` spans and protected tokens. This separation is referred to as masking in the experimental conditions. Protected tokens include:

```
Qt placeholders:                %1, %2, %n, %L1
Brace placeholders:             {0}, {feature_id}
printf placeholders:            %s, %d, %.2f
HTML/XML tags:                  <b>, </b>, <p align="right">
Escaped entities:               &amp;, &lt;, &gt;,  
Escaped controls:               \n, \t, \r
Actual controls:                newline, tab, carriage return
Literal ampersand escape in Qt: &&
Numeric literals:               100, 3.44, -1
```

Qt mnemonic annotations are handled separately as accelerator metadata, allowing an in-word marker such as `&A` in `&Attribute` to be represented as `(&A)` in Traditional Chinese. The LLM is not used to reproduce either protected tokens or mnemonic markers. It translates only the human-language spans and returns JSON keyed by text-span identifiers. The Python script then restores the protected tokens and mnemonic markers by rule.

Example source:

```
<p align="right">Algorithm author: {0}</p>
```

Template spans:

```
TOKEN: <p align="right">
TEXT : Algorithm author:
TOKEN: {0}
TOKEN: </p>
```

Model response:

```
{"T0": "演算法作者： "}
```

Reassembled output:

```
<p align="right">演算法作者： {0}</p>
```

This design makes preservation of checked token classes rule-based for accepted candidate translations. The workflow can preserve the defined format rules, but it cannot guarantee that the translation has the right meaning, sounds natural, or otherwise reads well in the user interface.

3.4. Glossary retrieval from tabular resources

Translation project glossaries are provided as spreadsheet-style terminology tables. Our method only assumes that each table can be normalized into records containing:

```
source_term, target_terms, forbidden_terms, priority, domain, note
```

Here, `forbidden_terms` are target-language expressions that the project glossary marks as discouraged or disallowed for a given source term.

To avoid scanning every glossary entry when translating each message segment, source terms are tokenized, normalized, and indexed by n-gram keys:

$$I[n, g] = \{e \in G : \text{key}_n(e.source_term) = g\}, \tag{5}$$

where G is the glossary and g is a normalized n-gram key. For a source segment s, the retrieved hint set is:

$$H(s) = \text{TopK}\left(\bigcup_{n=1}^{N_{max}} \bigcup_{g \in \text{ngrams}_n(s)} I[n, g], K\right). \tag{6}$$

Here, $H(s)$ denotes the glossary hints retrieved for segment s, N_{max} is the maximum n-gram length considered, and K is the maximum number of hints passed to the model. In the implementation, matched entries are sorted by source-term length so that more specific multi-word terms are preferred over shorter terms.

Glossary hints are passed to the LLM as contextual guidance, not mechanical replacements. This allows the model to adapt terminology to context while keeping prompt size controllable.

3.5. Candidate selection

Each segment may produce multiple candidate translations. The default production condition generates three candidates. A Python script scores each candidate with a rule-based quality-risk function:

$$Q(\hat{t}) = \sum_{e \in E(\hat{t})} w_Q(severity(e)) + \lambda_1 R_{latin}(\hat{t}) + \lambda_2 R_{len}(s, \hat{t}). \tag{7}$$

Here, $\hat{t}$ is a candidate translation for source segment s, $E(\hat{t})$ is the set of validation issues detected in the candidate, and w_Q maps critical, major, minor, and informational issues to 100, 10, 1, and 0, respectively. $R_{latin}(\hat{t})$ penalizes unexpected Latin-script residue, $R_{len}(s, \hat{t})$ penalizes extreme source-target length ratios, and the fixed weights are $\lambda_1 = 2$ and $\lambda_2 = 3$.

For the masked production conditions, the accepted candidate is selected only from candidates with no validation issues:

$$j^* = \arg \min_{j : E(\hat{t}_{u,j}) = \emptyset} Q(\hat{t}_{u,j}). \tag{8}$$

Here, $\hat{t}_{u,j}$ is the j-th candidate for translation unit u. In this strict masked path, every emitted validation issue blocks automatic acceptance.

If no generated candidate passes strict validation in a masked condition, the source string is written as a source-preserving fallback and the translation unit is logged as review-required; such a fallback is not counted as completed localization. In the current implementation, this status is recorded in the external run log rather than in the `.ts` `unfinished` attribute, so the log must accompany the generated artifact during review. In conditions C0 and C2, the selected raw candidate and its detected issues are retained so that structural failures remain observable. Review state is not used as a key quality metric

because it may have several operational causes: no valid candidate, backend exception, malformed JSON, or intentionally conservative rejection. Detailed review-state counts are reported in the accompanying reproducibility artifact. This paper focuses on structural failure rate, deterministic rule-based QA diagnostics, and MQM semantic quality.

3.6. Guarantee boundary of translation validation

The guarantee boundary is intentionally narrow:

- The workflow preserves many, but not all, token classes for accepted masked-template outputs.
- The evaluated masked-condition bundles yielded zero observed structural failures under the implemented extractors.
- It does not prove semantic adequacy, fluency, terminology correctness, UI usability, or cultural appropriateness.

If a token class is not defined by the extractor, it is outside the guarantee boundary. If a segment is logged for review rather than accepted as a structurally valid candidate, it should not be counted as completed user-facing localization. If a result has zero observed structure failures on a finite subset or corpus, this should be reported as zero observed failures under the checked token classes, not as proof of a 0% population error rate.

3.7. Workflow procedure

The procedure is summarized below in pseudocode. Full implementation details are in the code repository.

```
Input: Qt TS file S, glossaries G, backend b, condition C
Output: translated TS file, translation log, rule-based reports

U <- parse_ts(S)
I <- build_glossary_index(G)

for each translation unit u in U:
    if C.masking:
        s_prime, M <- extract_mnemonic_metadata(u.source)
        X, T       <- split_text_and_tokens(s_prime)
    else:
        X <- [u.source]; T <- (); M <- ()
    H    <- retrieve_glossary_hints(I, u.source, top_k)
    Y    <- generate_candidates(b, u, X, T, H, C)
    V    <- empty list

    for each candidate y in Y:
        if C.masking:
            t_hat <- restore_mnemonic(reinsert_tokens(y, T), M)
        else:
            t_hat <- y
        issues <- validate(u.source, t_hat)
        if not C.masking or no issue is found:
            add (t_hat, rule-based_risk_score(t_hat)) to V

    if V is not empty:
        write the lowest-risk candidate and log its issues
    else:
        write u.source as a source-preserving fallback and log review-required status

validate XML well-formedness
emit CSV, JSON, and Markdown reports
```

4. Implementation and Experimental Conditions

Our implementation is a Python command-line workflow. The main runner builds fixed subsets, runs the C0-C4 experimental conditions below, writes generated `.ts` files, and calls evaluation and scoring scripts. The same selected subset is reused across model backends to enable pairwise comparisons.

Table 2. *Experimental conditions.*

Condition	Masking	Glossary hints	Candidates	Purpose
C0 (Direct)	no	no	1	Direct translation without masking
C1 (Complete workflow)	yes	yes	3	Production condition
C2 (Direct + glossary)	no	yes	3	Tests glossary/candidates without masking
C3 (Masked without glossary)	yes	no	3	Tests masking without glossary hints
C4 (Single candidate)	yes	yes	1	Tests the value of generating multiple candidates

C0-C4 are experimental conditions for the same workflow over a fixed stratified subset of 3,000 message segments. C0 and C2 disable the structural hard lock and retain selected raw outputs so that direct-generation failures remain measurable. C1, C3, and C4 use masking, rule-based reassembly, strict validation, and a source-preserving fallback for rejected candidates. The conditions therefore compare complete operating configurations rather than isolating masking from the fallback policy. Our complete-corpus evaluation focuses on C1 because it is the selected production configuration, combining masking, glossary hints, and three candidates. Scaling up all experimental conditions would increase cost and review burden without being necessary to test whether this selected configuration produces structurally valid final artifacts for the QGIS Traditional Chinese localization workflow.

4.1. Direct generation and residual risk

A natural question is whether stronger prompting is sufficient. C0 and C2 should be interpreted conservatively: they demonstrate residual risk under the implemented direct prompts, but they are not an exhaustive benchmark of every possible prompt-only or constrained-decoding design. In these conditions without masking, the model sees the complete source string and remains responsible for reproducing protected formatting-language tokens. C2 additionally receives glossary hints and three candidates. These settings can improve semantic quality, but they still leave protected-token preservation to the model. Masked conditions change the problem decomposition: the LLM translates only human-language spans, while Python code preserves and reassembles protected tokens, validates the result, and substitutes the source when every candidate is rejected.

4.2. Validation fixtures and smoke tests

Our artifact includes a bundled mini evaluation fixture for exercising the rule-based layer. The fixture contains archived 100-segment C0-C4 `.ts` outputs, condition metadata, selected-segment metadata, and evaluation outputs. The default reproduction will rerun the rule-based evaluator and scoring scripts on these archived `.ts` files, so that token checks, accelerator handling, XML well-formedness checks, condition comparison, scoring behavior, MQM request planning, and table generation can be verified without calling any model API.

The fixture is not presented as a full regeneration of the paper experiments: it does not rerun LLM translation, candidate generation, glossary-assisted prompting, or token reassembly during generation. Those steps are covered by the workflow scripts and can be exercised through optional cloud/local LLM reruns, while the default reproduction path provides a stable offline check of the rule-based evaluation and reporting code path.

5. Evaluation

This section evaluates the effectiveness of our workflow. The purpose of the evaluation is not to rank the models we use. Rather, it is to validate the method we propose. It has two reported layers and one complementary semantic-review protocol.

1. **Subset condition comparison:** Grok, Gemini, and TAIDE are evaluated under conditions C0-C4 on the same 3,000-segment stratified subset of source messages.
2. **Complete corpus:** Grok, Gemini, and TAIDE are evaluated under C1 on all 28,924 QGIS source messages.
3. **Semantic review:** We use MQM-style sampled judging to evaluate semantic adequacy, terminology, fluency, and locale style on the C0-C4 outputs. This is separate from rule-based structural scoring.

The evaluation subset uses multi-label stratification based on source-string features that are likely to affect localization reliability, including plural/numerus messages, Qt placeholders, accelerator markers, HTML/XML content, glossary hits, numeric or code-like content, newline/control characters, long strings, and ordinary strings. Because these feature categories overlap, a segment may carry more than one label; the final subset is fixed and reused across all model backends and experimental conditions.

5.1. Structural metrics

For condition c and N checked segments, the structural failure rate is:

$$\hat{p}_c = \frac{1}{N}\sum_{u=1}^{N} 1[E_{struct}(u,c) \neq \emptyset]. \tag{9}$$

For a zero-failure sample, a one-sided approximate 95% upper bound can be summarized using the rule of three [16]:

$$p_{upper} \approx \frac{3}{N}. \tag{10}$$

For the 3,000-segment subset, this descriptive bound is approximately 0.1000%; because the subset is feature-stratified, the bound is not treated as a guarantee for a broader population. The complete 28,924-message corpus was enumerated rather than sampled, so its observed in-corpus result is reported directly as 0/28,924.

5.2. Deterministic rule-based QA metrics

Let E_{det} be the severity-weighted deterministic error points per 1,000 source characters:

$$E_{det} = 1000 \times \frac{\sum_u \sum_{e \in D_u} w_D(severity(e))}{\sum_u \mid s_u \mid}, \qquad RuleQA = \max(0, 100 - E_{det}), \tag{11}$$

where D_u contains deterministic structural, completion, untranslated-text, and terminology findings for segment u, with $w_D(critical) = 12$, $w_D(major) = 5$, $w_D(minor) = 1.5$, and $w_D(info) = 0$. Rule QA is therefore a dashboard diagnostic rather than a linguistic translation-quality score, and review-required status is reported separately. A condition can

improve structural safety while lowering Rule QA if it generates more untranslated or otherwise flagged results.

5.3. MQM-style semantic review metrics

Semantic quality is evaluated with MQM-style scoring. For segment u, let P_u be the set of MQM penalties assigned by the judge. The weighted MQM error rate is:

$$MQM\text{-}ER = 1000 \times \frac{\sum_{u=1}^{N} \sum_{p \in P_u} p}{\sum_{u=1}^{N} | s_u |}. \tag{12}$$

Lower values indicate fewer weighted semantic errors per 1,000 source characters. The fixed MQM penalties are 0, 1, 5, and 25 for neutral, minor, major, and critical findings, respectively. The auxiliary MQM score on a 0-100 scale is reported for readability; the error rate is the main semantic metric. For each backend-condition pair, Grok 4.3 judged three random samples of 200 segments, yielding 600 judgments. The same sampled segment positions were used across backends and conditions within each repeat. The reported approximate 95% half-width is $1.96s/\sqrt{3}$, where s is the standard deviation of the three run-level MQM-ER values; with only three repeats, this interval is descriptive rather than a precise population estimate.

The MQM analysis is not framed as a claim that masking should improve semantic quality. The trade-off question is whether the semantic cost of masking is small enough to justify the structural safety gain. Let:

$$\Delta_{MQM} = MQM\text{-}ER(C1_{masked}) - MQM\text{-}ER(C0_{direct}). \tag{13}$$

A value near zero indicates no material semantic penalty; a small positive value indicates a modest semantic cost; a negative value indicates that the masked workflow also improves semantic quality. In all cases, MQM is interpreted separately from structural failure rate.

6. Results

6.1. Experimental condition comparison: structural safety and semantic quality trade-off

The full outputs of the C0-C4 experimental conditions over the 3,000-segment subset are archived in the accompanying reproducibility artifact. Table 3 reports all five experimental conditions. In the table, MQM-ER is reported as mean ± approximate 95% half-width over repeated MQM judge runs. `Rule QA` is the deterministic dashboard score defined above, not a linguistic translation-quality score.

Table 3. *Comparison summary on the 3,000-segment subset.*

Backend	Cond.	Short name	Structure failure % ↓	Rule QA ↑	MQM-ER ↓
Grok 4.3	C0	Direct	5.67	93.60	6.289 ± 0.368
Grok 4.3	C1	Complete workflow	0.00	92.57	9.005 ± 2.807
Grok 4.3	C2	Direct + glossary	5.13	94.54	3.630 ± 1.488
Grok 4.3	C3	Masked without glossary	0.00	92.12	13.540 ± 4.216
Grok 4.3	C4	Single candidate	0.00	92.40	11.016 ± 2.372

Gemini 3.1 Flash-Lite	C0	Direct	8.70	92.72	7.194 ± 1.702
Gemini 3.1 Flash-Lite	C1	Complete workflow	0.00	92.97	11.109 ± 2.481
Gemini 3.1 Flash-Lite	C2	Direct + glossary	9.30	92.81	4.904 ± 1.945
Gemini 3.1 Flash-Lite	C3	Masked without glossary	0.00	92.88	13.500 ± 2.036
Gemini 3.1 Flash-Lite	C4	Single candidate	0.00	92.82	12.482 ± 3.276
TAIDE 12B	C0	Direct	30.10	76.11	37.339 ± 10.414
TAIDE 12B	C1	Complete workflow	0.00	84.52	40.736 ± 6.212
TAIDE 12B	C2	Direct + glossary	25.17	79.63	31.564 ± 5.943
TAIDE 12B	C3	Masked without glossary	0.00	81.85	47.200 ± 10.462
TAIDE 12B	C4	Single candidate	0.00	80.72	41.402 ± 6.335

Under the implemented conditions without masking, C0 and C2 retain non-zero structural risk for every backend: Grok C0 and C2 have structure-failure rates of 5.67% and 5.13%, Gemini C0 and C2 have rates of 8.70% and 9.30%, and TAIDE C0 and C2 have rates of 30.10% and 25.17%. These results do not claim to rule out all possible prompt-only methods; they show that direct generation leaves a residual token-preservation risk in this workflow. In contrast, all masked condition bundles produce final artifacts with zero observed structure failures under the token classes we have checked; candidates that fail strict validation are replaced by source-preserving fallbacks and flagged for review. The full item-level breakdown shows that direct generation failures concentrate in newline preservation, Qt shortcut-marker preservation, HTML/XML tag preservation, and placeholder preservation. Detailed counts are reported in the accompanying reproducibility artifact so as to keep this paper concise.

MQM shows the expected semantic trade-off. C2, the direct condition with glossary hints, obtains the lowest MQM error rate for all three backends, but it also retains non-zero structural failure rates. C1 has a higher MQM error rate than C0 and C2, while its masking, reassembly, validation, and fallback bundle produces structurally valid final artifacts under the extractors we implement. Thus, the direct conditions receive lower aggregate MQM-ER, whereas the selected production bundle gives stronger final-artifact structural protection.

Compared with C3, C1 has higher Rule QA and lower mean MQM-ER for all three backends. These results suggest a favorable effect of glossary hints in this experiment, but the differences are descriptive and no statistical significance is claimed.

6.2. The effect of generating multiple candidates

C1 and C4 use different limits for the number of generated candidates. Both use masking and glossary hints from tabular resources; C1 uses three candidates and C4 uses one. Both achieve zero observed structure failures, so candidate count is not the source of structural safety. Instead, multiple candidates mainly improve robustness by providing more candidate translations to choose from.

For cloud backends, the effect is small. Relative to C4, C1 improves Rule QA by +0.17 for Grok and +0.15 for Gemini. For TAIDE, the effect is larger: Rule QA improves by +3.80, and the

average valid candidate count increases from 0.689 to 2.119. MQM also favors C1 over C4 for all three backends, but the differences are modest relative to repeated-judge variability. We therefore interpret top-three candidate selection as a robustness mechanism rather than the primary safety contribution. Final-artifact structural protection comes from the combined masking, rule-based reassembly, validation, rejection, and fallback policy.

6.3. Complete-corpus C1 production results

Complete-corpus C1 experiments were run on all 28,924 QGIS source messages for each backend. We select C1 for our production workflow: masking, glossary hints from tabular resources, and top-three candidates.

Structure score is the arithmetic mean of eight preservation scores: Qt placeholders, brace placeholders, printf placeholders, HTML/XML entities, HTML/XML tags, numbers, newlines, and accelerators. For each class r, its score is $100(1 - n_r/N)$, where n_r is the number of segments with an observed preservation failure for class r and N is the number of messages checked. Rule QA is the deterministic dashboard score defined above. Possibly untranslated counts segments flagged by the rule-based detector as retaining substantial source-language residue.

Table 4. *Comparing the three LLMs under the production condition C1. Zero observed structural failure does not imply complete localization.*

Backend	Messages checked	Structure failure % ↓	Structure score ↑	Rule QA ↑	Avg. valid candidates	Possibly untranslated
Grok 4.3	28,924	0.00	100.000	92.11	2.938	1,394 (4.82%)
Gemini 3.1 Flash-Lite	28,924	0.00	100.000	90.09	2.935	1,825 (6.31%)
TAIDE 12B	28,924	0.00	100.000	65.13	2.052	6,674 (23.07%)

The zero rates in Table 4 describe the final `.ts` artifacts and include source-preserving fallbacks flagged for review: 591 of 28,924 rows for Grok (2.04%), 1,396 for Gemini (4.83%), and 7,265 for TAIDE (25.12%). These rows preserve structure but remain incomplete localization rather than accepted translated content.

The complete-corpus result confirms that the subset finding scales up to the complete QGIS Qt source file: all three backends produce final artifacts with zero observed structural failures under C1 for the token classes we have checked. The local TAIDE backend has zero observed structural failures under this workflow bundle, but its Rule QA is much lower. This indicates that the local backend is a viable offline path for structural preservation while requiring substantially more review effort than the two cloud backends.

7. Reproducibility and Artifact Availability

The qgis-llm-localization-workflow repository provides the workflow code, configurations, reports, and a compact 100-segment reproduction fixture. The complete archived outputs used for the paper tables—C0-C4 for the 3,000-segment subset and C1 for all 28,924 messages, across all three backends—are available in Depositar, identified by the persistent identifier `ark:37281/k553s150r`.

Table 5. *Reproducibility levels.*

Level	Command or location	New model calls?	What it verifies

Mini rule-based evaluation	`python scripts/run_repro.py full-mini; experiments/demo_ablation_grok_100/`	No	Reruns rule-based evaluation, scoring, condition comparison, and MQM request generation on bundled 100-segment C0-C4 `.ts` outputs.
Extended archived-output scoring	`python code/public_repository/scripts/run_repro.py score --experiment experiments/<experiment> --force-eval`	No	From the Depositar archive root, recomputes structural and deterministic Rule QA metrics from archived generated `.ts` files rather than from preformatted table rows.
Optional MQM or translation rerun	`python scripts/run_repro.py mqm ... --run-grok; python scripts/run_repro.py translate ...`	Yes	Rebuilds MQM judge outputs or regenerates a small translation test run using cloud AI credentials or local inference.

The default mini rule-based evaluation is reviewer-friendly and can be used offline. It uses the same C0-C4 condition definitions and the same rule-based evaluator/scorer as this paper's experiments, but on a smaller 100-segment archived-output fixture so that it can run quickly. It should be interpreted as an executable validation of the rule-based evaluation, scoring, comparison, and request-planning code paths, not as a full regeneration of LLM translation outputs or as a substitute for the 3,000-segment and 28,924-message experimental artifacts.

The compact repository release contains the source `.ts` file, tabular glossary resources, C0-C4 configuration files, workflow scripts, the 100-segment fixture and demo outputs, workflow manifests, rule-based evaluation outputs, MQM request-planning outputs, and an `.env.example` file. The Depositar archive contains the generated `.ts` outputs and evaluation reports for the reported 3,000-segment C0-C4 comparison and 28,924-message C1 runs. With that archive, the same scoring command recomputes rule-based metrics from archived outputs without new model calls. Rerunning translation or MQM judging requires the appropriate cloud LLM API credentials or local inference hardware. API keys are excluded from the repository and are supplied through environment variables such as `XAI_API_KEY` and `GEMINI_API_KEY`.

In addition to the reproducibility archive, the released Traditional Chinese localization files are deposited as public datasets in Depositar, a public repository for research datasets, with persistent ARK identifiers. The QGIS 4.0 and QGIS 3.44 zh-Hant localization datasets were generated with the assistance of GPT-4.1-mini and released in March 2026, before the workflow presented in this paper was formalized, although they were based on the same general idea of masked template translation with glossary hints. The released datasets include the QGIS 4.0 zh-Hant Translation Dataset (TS/QM Files), identified by `ark:37281/k5f167n46` and available at https://pid.depositar.io/ark:37281/k5f167n46, and the Long Term Release QGIS 3.44 zh-Hant Translation Dataset (TS/QM Files), identified by `ark:37281/k5g11462n` and available at https://pid.depositar.io/ark:37281/k5g11462n. These deposits provide persistent access to the released translation files; the GitHub repository provides the workflow and 100-segment fixture, while the paper's complete experimental outputs are available in the Depositar archive (`ark:37281/k553s150r`).

The code repository README documents the quickstart, rule-based scoring, MQM request planning, optional MQM judge run, and optional API translation test run.

8. Discussion and Limitations

Our proposed workflow improves reliability by separating structural safety, terminology guidance, semantic quality evaluation, and human review. The rule-based layer is the most objective because it checks formatting artifacts that should be preserved. The structure-preserving template layer strengthens this idea by removing protected tokens from the LLM generation space. Generating multiple candidates is a secondary robustness layer; it improves candidate availability, especially for the local backend. Structural protection in the final artifacts comes from the complete masking, reassembly, validation, rejection, and fallback policy.

The role of masking should be interpreted carefully. Masking does not solve semantic adequacy by itself, and the MQM results show a modest semantic cost for C1 compared with C0 and C2, the two conditions without using masking. However, this is a different risk category from interface breakage. C0 and C2 provide evidence of residual risk under the implemented direct generation settings, not a complete dismissal of all prompt-only approaches. C2 can be semantically stronger, but it still leaves non-zero structural failure rates in this experiment. In practical terms, the selected production bundle trades a measured semantic cost and a review queue for stronger final-artifact structural reliability under the token classes we have checked.

The local LLM results should be read as evidence for deployability, not model superiority. TAIDE achieves the same zero observed structural failure result under C1, but its Rule QA and review-workload indicators are much worse than the cloud backends. In the archived status diagnostics, 7,265 of 28,924 TAIDE C1 rows, or 25.12%, entered a review-required fallback state. The external run log makes these rows identifiable even though the generated `.ts` file preserves their source text. Using the local backend therefore requires substantially more human review before release; in the present configuration, TAIDE should be treated as an offline deployment path with a larger review queue rather than as a drop-in substitute for the cloud backends.

The glossary layer depends on glossary quality. Because glossary matches are hints rather than replacements, the workflow avoids many unnatural literal substitutions, but terminology correctness still requires evaluation.

Semantic quality review is necessary because the rule-based layer does not evaluate adequacy, fluency, domain correctness, or target-locale style. LLM judges can be useful for large-scale review, but their judgments are assisted annotations rather than ground truth. Grok 4.3 is the sole MQM judge here and evaluates outputs from every backend, including Grok itself; self-preference and model-family bias therefore cannot be excluded. Human review remains necessary for release-critical strings, ambiguous GIS terms, and user-facing messages with high impact.

8.1. Practical deployment recommendation

For a FOSS localization team, the recommended deployment workflow is:

1. Run the selected production condition.
2. Keep the external run log with the generated `.ts` artifact so fallback rows remain identifiable.
3. Accept only rows with zero blocking structural issues.

4. Treat review-required rows as incomplete, not as translated content.
5. Prioritize review by English residue, terminology warnings, semantic severity, and review status.
6. Run XML well-formedness and rule-based validation before each release candidate.
7. Use human review for final acceptance of UI-critical strings and domain terminology.

Such a workflow preserves the format integrity of the translated messages and creates auditable review queues; it does not eliminate all localization errors or claim superior translation quality.

9. Conclusion

This paper presents a Python workflow for reliable localization of QGIS Qt `.ts` files with LLM support, without tying the method to a specific LLM backend. The workflow supports both cloud-based and local LLMs, uses tabular glossary resources as contextual terminology hints, and applies structure-preserving template translation to preserve protected tokens through rule-based reassembly.

The 3,000-segment subset comparison shows that the implemented conditions without masking produce non-zero structural failure rates for Grok, Gemini, and TAIDE, while the masked condition bundles produce final artifacts with zero observed structure failures under checked token classes for all three backends. Complete-corpus C1 results extend this finding to all 28,924 QGIS source messages; this zero-failure result includes source-preserving fallback rows that remain incomplete and require review. Semantic quality results using MQM show that masking is not a semantic quality optimizer: C1 has a modestly higher MQM error rate than the direct baselines, and C2 has the lowest MQM-ER but retains non-zero observed structural failure rates. C1 versus C4 further shows that generating three candidates helps robustness, especially for TAIDE, while final-artifact structural protection comes from the combined masking, rule-based reassembly, validation, rejection, and fallback policy.

The broader contribution of this research is methodological. Reliable FOSS localization should not depend on raw LLM output alone. It should be built as a reproducible workflow in which rule-based validation protects software artifact structure, glossary retrieval supports terminology consistency, MQM evaluation with LLM assistance scales up semantic quality review, and human reviewers retain final responsibility.

10. Acknowledgements and AI Disclosure

Generative AI was used for English wording, formulation, and formatting assistance. The authors verified the technical claims, references, experimental results, source code, and final manuscript text. LLM-generated translations are treated as drafts requiring validation and human review; MQM-style judgments are treated as assisted annotations rather than ground truth.

11. Funding

This research is supported in part by an Academia Sinica Thematic Project grant (no. AS-TP-114-L01; *Sound Atlas: Mapping the Changing Terrestrial, Marine, and Cultural Soundscapes for a Sustainable Island Social-Ecological System*) and a grant from the National

Science and Technology Council of Taiwan (no. NSTC 114-2621-M-001-001; *Advancing Research Data Infrastructures and Management Practices: Tools, Services, and Communities*).

References

[1] QGIS Development Team, "QGIS Geographic Information System," technical report, 2026. doi: 10.5281/zenodo.6139224.

[2] A. Graser, T. Sutton, and M. Bernasocchi, "The QGIS Project: Spatial without Compromise," *Patterns*, vol. 6, no. 7, p. 101265, 2025, doi: 10.1016/j.patter.2025.101265.

[3] The Qt Company, "TS File Format | Qt Linguist Manual." [Online]. Available: https://doc.qt.io/qt-6/linguist-ts-file-format.html

[4] B. Esselink, *A Practical Guide to Localization*. John Benjamins Publishing Company, 2000. doi: 10.1075/liwd.4.

[5] L. Bowker, *Computer-Aided Translation Technology: A Practical Introduction*. University of Ottawa Press, 2002. doi: 10.1353/book6554.

[6] Google AI for Developers, "Structured outputs | Gemini API." [Online]. Available: https://ai.google.dev/gemini-api/docs/structured-output

[7] xAI, "Structured Outputs | xAI Docs." [Online]. Available: https://docs.x.ai/developers/model-capabilities/text/structured-outputs

[8] Multidimensional Quality Metrics, "What Is MQM?." [Online]. Available: https://themqm.org/

[9] A. Lommel, H. Uszkoreit, and A. Burchardt, "Multidimensional Quality Metrics (MQM): A Framework for Declaring and Describing Translation Quality Metrics," *Tradumàtica: tecnologies de la traducció*, no. 12, pp. 455–463, 2014, doi: 10.5565/rev/tradumatica.77.

[10] T. Kocmi and C. Federmann, "GEMBA-MQM: Detecting Translation Quality Error Spans with GPT-4," in *Proceedings of the Eighth Conference on Machine Translation*, 2023, pp. 768–775. doi: 10.18653/v1/2023.wmt-1.64.

[11] Google AI for Developers, "Gemini 3.1 Flash-Lite." [Online]. Available: https://ai.google.dev/gemini-api/docs/models/gemini-3.1-flash-lite

[12] xAI, "Grok 4.3 | xAI Docs." [Online]. Available: https://docs.x.ai/developers/models/grok-4.3

[13] TAIDE Project, "taide/Gemma-3-TAIDE-12b-Chat-2602." [Online]. Available: https://huggingface.co/taide/Gemma-3-TAIDE-12b-Chat-2602

[14] Google, "google/gemma-3-12b-pt." [Online]. Available: https://huggingface.co/google/gemma-3-12b-pt

[15] Gemma Team, "Gemma 3 Technical Report." [Online]. Available: https://arxiv.org/abs/2503.19786

[16] J. A. Hanley and A. Lippman-Hand, "If nothing goes wrong, is everything all right? Interpreting zero numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, 1983, doi: 10.1001/jama.1983.03330370053031.